

目录

前言	1
1. MySQL 专栏面试题 (关注公众号：捡田螺的小男孩)	1
1.1 MySQL 索引使用有哪些注意事项呢？	1
1.2. MySQL 遇到过死锁问题吗，你是如何解决的？	2
1.3. 日常工作中你是怎么优化 SQL 的？	3
1.4. 说说分库与分表的设计	3
1.5. InnoDB 与 MyISAM 的区别	4
1.6. 数据库索引的原理，为什么要用 B+树，为什么不用二叉树？	4
1.7. 聚集索引与非聚集索引的区别	5
1.8 limit 1000000 加载很慢的话，你是怎么解决的呢？	5
1.9 如何选择合适的分布式主键方案呢？	6
1.10 事务的隔离级别有哪些？MySQL 的默认隔离级别是什么？	6
1.11 什么是幻读，脏读，不可重复读呢？	7
1.12 在高并发情况下，如何做到安全的修改同一行数据？	7
1.13 数据库的乐观锁和悲观锁。	8
1.15. select for update 有什么含义，会锁表还是锁行还是其他。	9
1.16 MySQL 事务得四大特性以及实现原理	11
1.17 如果某个表有近千万数据，CRUD 比较慢，如何优化。	12
1.18 如何写 sql 能够有效的使用到复合索引。	12
1.19 mysql 中 in 和 exists 的区别。	13
1.20 数据库自增主键可能遇到什么问题。	14
1.21 MVCC 熟悉吗，它的底层原理？	14
1.22 数据库中间件了解过吗，sharding jdbc，mycat？	14
1.23 MYSQL 的主从延迟，你怎么解决？	15
1.24 说一下大表查询的优化方案	16
1.25 什么是数据库连接池？为什么需要数据库连接池呢？	16
1.26 一条 SQL 语句在 MySQL 中如何执行的？	17
1.27 InnoDB 引擎中的索引策略，了解过吗？	17
1.28 一条 sql 执行过长的时间，你如何优化，从哪些方面入手？	18
1.29 MYSQL 数据库服务器性能分析的方法命令有哪些？	18
1.30 Blob 和 text 有什么区别？	19
1.31 mysql 里记录货币用什么字段类型比较好？	19
1.32 Mysql 中有哪几种锁，列举一下？	19
1.33 Hash 索引和 B+树区别是什么？你在设计索引是怎么抉择的？	20
1.34 mysql 的内连接、左连接、右连接有什么区别？	20
1.35 说说 MySQL 的基础架构图	20
1.36 什么是内连接、外连接、交叉连接、笛卡尔积呢？	21
1.37 说一下数据库的三大范式	21
1.38 Mysql 的 binlog 有几种录入格式？分别有什么区别？	22
1.39 InnoDB 引擎的 4 大特性，了解过吗	22
1.40 索引有哪些优缺点？索引有哪几种类型？	22

1.41 创建索引有什么原则呢？	23
1.42 创建索引的三种方式	23
1.43 百万级别或以上的数据，你是如何删除的？	24
1.44 什么是最左前缀原则？什么是最左匹配原则？	24
1.45 B 树和 B+树的区别，数据库为什么使用 B+树而不是 B 树？	24
1.46 覆盖索引、回表等这些，了解过吗？	25
1.47 非聚簇索引一定会回表查询吗？	25
1.48 联合索引是什么？为什么需要注意联合索引中的顺序？	25
1.49 数据库隔离级别与锁的关系	25
1.50 聊聊 InnoDB 的锁，什么是死锁？怎么解决？	25
1.51 count(1)、count(*) 与 count(列名) 的区别？	33
1.52 什么是存储过程？有哪些优缺点？	33
1.53 SQL 约束有哪几种呢？	33
1.54 varchar(50)中 50 的涵义	34
1.55 mysql 中 int(20)和 char(20)以及 varchar(20)的区别	34
1.56 drop、delete 与 truncate 的区别	34
1.57 UNION 与 UNION ALL 的区别？	34
1.58 SQL 的生命周期？	35
1.59 一条 Sql 的执行顺序？	35
1.60 关心过业务系统里面的 sql 耗时吗？统计过慢查询吗？对慢查询都怎么优化过？	36
1.61 主键使用自增 ID 还是 UUID，为什么？mysql 自增主键用完了怎么办？	36
1.62 MySQL 数据库 cpu 飙升的话，要怎么处理呢？	36
1.63 读写分离常见方案？	37
1.64 MySQL 的复制原理以及流程	37
1.65 Innodb 的事务实现原理？	38
1.66 谈谈 MySQL 的 Explain	38
1.67 Innodb 的事务与日志的实现方式	39
1.68 你们数据库是否支持 emoji 表情存储，如果不支持，如何操作？	39
1.69 一个 6 亿的表 a，一个 3 亿的表 b，通过外间 tid 关联，你如何最快的查询出满足条件的第 50000 到第 50200 中的这 200 条数据记录。	40
1.70 Mysql 一条 SQL 加锁分析	40
2.Redis 专栏面试题（关注公众号：捡田螺的小男孩）	40
2.1 什么是 Redis？它主要用来什么的？	40
2.2 说说 Redis 的基本数据结构类型	41
2.3 Redis 为什么这么快？	45
2.4 什么是缓存击穿、缓存穿透、缓存雪崩？	49
2.5 什么是热 Key 问题，如何解决热 key 问题	52
2.6. Redis 过期策略和内存淘汰策略	52
2.7 说说 Redis 的常用应用场景	54
2.8 Redis 持久化有哪几种方式，怎么选？	57
2.9 怎么实现 Redis 的高可用？(Redis 主从、哨兵、集群)	59
2.10 使用过 Redis 分布式锁嘛？有哪些注意点呢？	78
2.11 使用过 Redisson 嘛？说说它的原理	81

2.12 什么是 Redlock 算法.....	81
2.13 Redis 的跳跃表.....	83
2.14 MySQL 与 Redis 如何保证双写一致性.....	84
2.15 为什么 Redis 6.0 之后改多线程呢？.....	87
2.16 聊聊 Redis 事务机制.....	88
2.17 Redis 的 Hash 冲突怎么办.....	88
2.18 在生成 RDB 期间，Redis 可以同时处理写请求么？.....	90
2.19 Redis 底层，使用的什么协议？.....	91
2.20 布隆过滤器.....	91
3.消息队列专栏面试题（关注公众号：捡田螺的小男孩）.....	93
3.1 什么是消息队列.....	93
3.2 消息队列有哪些使用场景。.....	93
3.3 消息队列如何解决消息丢失问题？.....	96
3.4 消息队列如何保证消息的顺序性。.....	98
3.5 消息队列有可能发生重复消费，如何避免，如何做到幂等？.....	99
3.6 如何处理消息队列的消息积压问题.....	100
3.7 消息队列技术选型，Kafka 还是 RocketMQ，还是 RabbitMQ.....	100
3.8 消息中间件如何做到高可用.....	102
3.9 如何保证数据一致性，事务消息如何实现.....	102
3.10 让你写一个消息队列，该如何进行架构设计？.....	104
4.计算机网络专栏面试题（关注公众号：捡田螺的小男孩）.....	105
4.1 说说 HTTP 常用的状态码及其含义？.....	105
4.2 HTTP 常用的请求方式，区别和用途？.....	106
4.3 请简单说一下你了解的端口及对应的服务？.....	106
4.4 说下计算机网络体系结构.....	107
4.5 如何理解 HTTP 协议是无状态的.....	109
4.6 从浏览器地址栏输入 url 到显示主页的过程.....	110
4.7 说下 HTTP 1.0，1.1，2.0 的区别.....	110
4.8 POST 和 GET 有哪些区别？.....	111
4.9 在交互过程中如果数据传送完了，还不想断开连接怎么办，怎么维持？.....	112
4.10 HTTP 如何实现长连接？在什么时候会超时？.....	112
4.11 HTTP 与 HTTPS 的区别。.....	113
4.12 Https 流程是怎样的？.....	113
4.13 说说 HTTP 的状态码，301 和 302 的区别？.....	114
4.14 说说什么是数字签名？什么是数字证书？.....	115
4.15 对称加密与非对称加密有什么区别.....	116
4.16 说说 DNS 的解析过程？.....	117
4.17 什么是 CSRF 攻击，如何避免.....	119
4.18 聊聊五层计算机网络体系结构中，每一层对应的网络协议有哪些？.....	120
4.19 说说 WebSocket 与 socket 的区别.....	121
4.20 什么是 DoS、DDoS、DRDoS 攻击？.....	122
4.21 什么是 XSS 攻击，如何避免？.....	122
4.23 Http 请求的过程与原理.....	123
4.24 forward 和 redirect 的区别？.....	124

4.25 聊聊 SQL 注入？.....	125
4.28 请详细介绍一下 TCP 的三次握手机制.....	127
4.29 TCP 握手为什么是三次，为什么不能是两次？不能是四次？.....	129
4.30 TCP 四次挥手过程？.....	131
4.31 TCP 挥手为什么需要四次呢？.....	132
4.32 TCP 四次挥手过程中，为什么需要等待 2MSL,才进入 CLOSED 关闭状态.....	134
4.33 说说 TCP 是如何确保可靠性的呢？.....	135
4.34 说说 TCP 报文首部有哪些字段，其作用又分别是什么？.....	135
4.35 请简述 TCP 和 UDP 的区别.....	136
4.36 说说 TCP 的重传机制.....	137
4.38 聊聊 TCP 的滑动窗口.....	142
4.38 说说 TCP 的流量控制？.....	144
4.39 TCP 的拥塞控制.....	146
4.40 说说半连接队列和 SYN Flood 攻击的关系.....	152
4.41 TCP 的粘包和拆包.....	154
4.42 Nagle 算法与延迟确认.....	155
4.43 IP 地址有哪些分类？.....	156
4.44 说下 ARP 协议的工作过程？.....	157
4.45 有了 IP 地址，为什么还要用 MAC 地址？.....	158
4.46 TCP 和 UDP 分别对应的常见应用层协议有哪些？.....	158
4.47 聊聊保活计时器的作用.....	158
4.48 聊聊 ARP 协议.....	159
4.49 forward 和 redirect 的区别？.....	160
4.50 URI 和 URL 的区别.....	161
4.51 Session 和 Cookie 的区别。.....	161
5.Zookeeper 专栏面试题（关注公众号：捡田螺的小男孩）.....	163
5.1 面试官：工作中使用过 Zookeeper 嘛？你知道它是什么，有什么用途呢？.....	163
2.2 面试官：说下什么是命名服务，什么是配置管理，又什么是集群管理吧..	164
5.3 面试官：你提到了 znode 节点，那你知道 znode 有几种类型呢？zookeeper 的数据模型是怎样的呢？.....	165
5.4 面试官：你知道 znode 节点里面存储的是什么吗？每个节点的数据最大不能超过多少呢？.....	167
5.5 面试官：你知道 znode 节点上的监听机制嘛？讲下 Zookeeper watch 机制吧。.....	168
5.6 面试官：你对 Zookeeper 的数据结构都有一定了解，那你讲下 Zookeeper 的特性吧.....	170
5.7 面试官：你刚提到顺序一致性，那 zookeeper 是如何保证事务的顺序一致性的呢？.....	171
5.8 面试官：你提到了 Leader，你知道 Zookeeper 的服务器有几种角色嘛？Zookeeper 下 Server 工作状态又有几种呢？.....	173
5.9 面试官：你说到服务器角色是基于 ZooKeeper 集群的，那你画一下 ZooKeeper 集群部署图吧？ZooKeeper 是如何保证主从节点数据一致性的呢？	

.....	174
5.10 面试官：Leader 挂了，进入崩溃恢复，是如何选举 Leader 的呢？你讲一下 ZooKeeper 选举机制吧.....	175
5.11 面试官：你前面提到在项目中使用过 Zookeeper 的分布式锁，讲一下 zk 分布式锁的实现原理吧？.....	179
5.12 面试官：好的，最后一道题，你说说 dubbo 和 Zookeeper 的关系吧，为什么选择 Zookeeper 作为注册中心？.....	184
6.Java 基础面试专栏（关注公众号：捡田螺的小男孩）.....	185
6.1 equals 与==的区别.....	185
6.2 final, finally, finalize 的区别.....	185
6.3 重载和重写的区别.....	185
6.4 两个对象的 hashCode()相同，则 equals()是否也一定为 true？.....	186
6.5 抽象类和接口有什么区别.....	186
6.6 BIO、NIO、AIO 有什么区别？.....	187
6.7 String，Stringbuffer，StringBuilder 的区别.....	187
6.8 JAVA 中的几种基本数据类型是什么，各自占用多少字节呢.....	188
6.9 Comparator 与 Comparable 有什么区别？.....	188
6.10 String 类能被继承吗，为什么。.....	189
6.11 说说 Java 中多态的实现原理.....	189
6.12 Java 泛型和类型擦除.....	190
6.13 int 和 Integer 有什么区别，还有 Integer 缓存的实现.....	190
6.14 说说反射的用途及实现原理，Java 获取反射的三种方法.....	191
6.15 面向对象的特征.....	191
6.16 &和&&的区别.....	191
6.17 Java 中 IO 流分为几种?.....	192
6.18 讲讲类的实例化顺序，比如父类静态数据，构造函数，子类静态数据，构造函数。.....	192
6.19 Java 创建对象有几种方式.....	193
6.20 如何将 GB2312 编码的字符串转换为 ISO-8859-1 编码的字符串呢？.....	193
6.21 守护线程是什么？用什么方法实现守护线程.....	193
6.22 notify()和 notifyAll()有什么区别？.....	194
6.23 Java 语言是如何处理异常的，关键字 throws、throw、try、catch、finally 怎么使用？.....	194
6.24 谈谈 Java 的异常层次结构.....	194
6.25 静态内部类与非静态内部类有什么区别.....	195
6.26 String s 与 new String 与有什么区别.....	195
6.27 反射中，Class.forName 和 ClassLoader 的区别.....	196
6.28 JDK 动态代理与 cglib 实现的区别.....	197
6.29 error 和 exception 的区别，CheckedException，RuntimeException 的区别。.....	197
6.30 深拷贝和浅拷贝区别.....	198
6.31 JDK 和 JRE 有什么区别？.....	199
6.32 String 类的常用方法都有哪些呢？.....	199
6.33 谈谈自定义注解的场景及实现.....	200

6.34 说说你熟悉的设计模式有哪些？	200
6.35 抽象工厂和工厂方法模式的区别？	200
6.36 什么是值传递和引用传递？	201
6.37 可以在 static 环境中访问非 static 变量吗？	201
6.38 Java 支持多继承么,为什么？	201
6.39 用最有效率的方法计算 2 乘以 8？	201
6.40 构造器是否可被重写？	202
6.41 char 型变量中能不能存贮一个中文汉字，为什么？	202
6.42 如何实现对象克隆？	202
6.43 Object 中定义了哪些方法？	202
6.44 hashCode 的作用是什么？	203
6.45 for-each 与常规 for 循环的效率对比	203
6.46 写出几种单例模式实现，懒汉模式和饿汉模式区别	203
6.47 请列出 5 个运行时异常。	204
6.48 2 个不相等的对象有可能具有相同的 hashCode 吗？	204
6.49 访问修饰符 public,private,protected,以及 default 的区别？	204
6.50 谈谈 final 在 Java 中的作用？	204
6.51 Java 中的 Math.round(-1.5) 等于多少呢？	205
6.52 String 属于基础的数据类型吗？	205
6.53 如何将字符串反转呢？	205
6.54 描述动态代理的几种实现方式，它们分别有什么优缺点	206
6.55 在自己的代码中，如果创建一个 java.lang.String 类，这个类是否可以被类加载器加载？为什么。	206
6.56 谈谈你对 java.lang.Object 对象中 hashCode 和 equals 方法的理解。在什么场景下需要重新实现这两个方法。	206
6.57 在 JDK1.5 中，引入了泛型，泛型的存在是用来解决什么问题。	207
6.58 什么是序列化，怎么序列化，反序列化呢？	207
6.59 Java8 的新特性。	207
6.60 匿名内部类是什么？如何访问在其外面定义的变量呢？	208
6.61 break 和 continue 有什么区别？	209
6.62 String s = "Hello";s = s + " world!";这两行代码执行后，原始的 String 对象中的内容是否会改变？	209
6.63 String s="a"+"b"+"c"+"d";创建了几个对象？	209
6.64 try-catch-finally-return 执行顺序	209
6.65 Java 7 新的 try-with-resources 语句，平时有使用吗	210
6.66 简述一下面向对象的“六原则一法则”。	211
6.67 switch 是否能作用在 byte 上，是否能作用在 long 上，是否能作用在 String 上？	211
6.68 数组有没有 length()方法？String 有没有 length()方法？	212
6.69 是否可以从一个静态 (static) 方法内部发出对非静态 (non-static) 方法的调用？	212
6.70 String s = new String("jay");创建了几个字符串对象？	212
6.71 this 和 super 关键字的作用	212
6.72 我们能否将 int 强制转换为 byte 类型的变量吗？如果该值大于 byte 类型的	

范围，将会出现什么现象？.....	213
6.73 float f=3.4;正确吗？.....	213
6.74 接口可否继承接口？抽象类是否可实现接口？抽象类是否可继承实体类？.....	214
6.75 Reader 和 InputStream 区别？.....	214
6.76 列举出 JAVA 中 6 个比较常用的包.....	214
6.77 JDK 7 有哪些新特性.....	214
6.78 同步和异步有什么区别？.....	215
6.79 synchronized 的实现原理以及锁优化？.....	215
6.80 ThreadLocal 原理，使用注意点，应用场景有哪些？.....	217
6.81 说说 CountdownLatch 与 CyclicBarrier 区别.....	220
6.82 Fork/Join 框架的理解.....	221
7. JVM 专栏面试题（关注公众号：捡田螺的小男孩）.....	223
7.1 什么情况下会发生栈内存溢出。.....	223
7.2 详解 JVM 内存模型.....	224
7.3 JVM 内存为什么要分成新生代，老年代，持久代。新生代中为什么要分为 Eden 和 Survivor。.....	224
7.4 JVM 中一次完整的 GC 流程是怎样的，对象如何晋升到老年代.....	226
7.5 你知道哪几种垃圾收集器，各自的优缺点，重点讲下 cms 和 G1，包括原理，流程，优缺点。.....	226
7.6 JVM 内存模型的相关知识了解多少，比如重排序，内存屏障，happen-before，主内存，工作内存。.....	227
7.7 简单说说你了解的类加载器，可以打破双亲委派么，怎么打破。.....	230
7.8 说说你知道的几种主要的 JVM 参数.....	232
7.9 怎么打出线程栈信息。.....	233
7.10 强引用、软引用、弱引用、虚引用的区别？.....	233
8.oppo 面试真题（关注公众号：捡田螺的小男孩。）.....	235
8.1 聊聊你印象最深刻的项目，或者做了什么优化。.....	235
8.2 你项目提到分布式锁，你们是怎么使用分布式锁的？.....	236
8.3 常见分布式事务解决方案.....	236
8.4 你们的接口幂等是如何保证的？.....	239
8.5 你们的 mySQL 架构是怎样的？.....	239
8.6 常见的索引结构有？哈希表结构属于哪种场景？.....	243
8.7 给你 ab,ac,abc 字段，你是如何加索引的？.....	244
8.8 数据库隔离级别是否了解？你们的数据库默认隔离级别是？为什么选它？.....	244
8.9 RR 隔离级别实现原理，它是如何解决不可重复读的？.....	246
8.10 你们项目使用了 RocketMQ 对吧？那你知道如何保证消息不丢失吗？..	250
8.11 事务消息是否了解？场景题：比如下单清空购物车，你是如何设计的？	252
8.12 如何快速判断一个数是奇数还是偶数，除开对 2 取余呢。.....	254
8.13 Spring 声明式事务原理？哪些场景事务会失效？.....	254
8.14 你们是微服务架构嘛？如果你来设计一个类似淘宝的系统，你怎么划分微服务？.....	256
8.15 你们是怎么分库分表的？分布式 ID 如何生成？.....	256

8.16 所有异常的共同的祖先是？运行时异常有哪几个？	257
9.小厂后端十连问（关注公众号：捡田螺的小男孩。）	258
9.1 Redis 的 key 和 value 可以存储的最大值分别是多少？	259
9.2 怎么利用 Redis 实现数据的去重？	259
9.3 Redis 什么时候需要序列化？Redis 序列化的方式有哪些？	259
9.4 MySQL 的 B+ 树的高度怎么计算？（比如有 100w 的数据，字段为 int 类型）	260
9.5 线程池的状态有哪些？获取多线程并发执行结果的方式有哪些？	261
9.6 线程池原理？各个参数的作用。	262
9.7 ThreadLocal 的使用场景有哪些？原理？内存泄漏？	265
9.8 kafka 是如何保证消息的有序性？	267
9.9 Nacos 的选举机制了解嘛？说下 Raft 算法？	268
9.10 聊一聊 TCC 补偿机制	272
10.腾讯云 15 连问（关注公众号：捡田螺的小男孩。）	274
10.1 聊聊项目，好的设计，好的代码	274
10.2 谈谈什么是零拷贝？	275
10.3 一共有几种 IO 模型？NIO 和多路复用的区别？	279
10.4 Future 实现阻塞等待获取结果的原理？	282
10.5 ReentrantLock 和 Synchronized 的区别？Synchronized 的原理？	286
10.6 聊聊 AOS？ReentrantLock 的实现原理？	287
10.7 乐观锁和悲观锁，让你来写你怎么实现？	287
10.8 Paxos 协议了解？工作流程是怎么样的？	288
10.9 B+树聊一下？B+树是不是有序？B+树和 B-树的主要区别？B+树索引，一次查找过程？	295
10.10 TCP 如何实现拥塞控制？	296
10.11. JVM 调优	302
10.12 数据库分库分表的缺点是啥？	305
10.13 分布式事务如何解决？TCC 了解？	305
10.14 RocketMQ 如何保证消息的准确性和安全性？	308
10.15. 三个数求和	308
11.社招 21 连问（关注公众号：捡田螺的小男孩。）	310
11.1 Mysql 索引在什么情况下会失效	310
11.2 MySql 的存储引擎 InnoDB 与 MyISAM 的区别	311
11.3 mysql 在项目中的优化场景，慢查询解决等	311
11.4 Mysql 有什么索引，索引模型是什么	311
11.5 B-树与 B+树的区别？为什么不用红黑树	312
11.6 Mysql 主从同步怎么做	313
11.7 乐观锁与悲观锁的区别？	313
11.8 聊聊 binlog 日志	315
11.9 Redis 持久化有哪几种方式，怎么选？	316
11.10 Redis 主从同步是怎样的过程？	318
11.11 聊聊 Redis 的 zset，它是怎么实现的？	319
11.12 Redis 过期策略和内存淘汰策略	320
11.13. Hashmap 是怎样实现的？为什么要用红黑树，而不用平衡二叉树？为	

什么在 1.8 中链表大于 8 时会转红黑树？HashMap 是线性安全的嘛？如何保证安全？.....	322
11.14 select 和 epoll 的区别.....	323
11.15 http 与 https 的区别，https 的原理，如何加密的？.....	326
11.16 Raft 算法原理.....	327
16.1 Raft 角色.....	328
11.17 消息中间件如何做到高可用.....	333
11.18 消息队列怎么保证不丢消息的.....	333
11.19 Redis 如何保证高可用？聊聊 Redis 的哨兵机制.....	335
11.20 无重复字符的最长子串.....	336
12. 蚂蚁金服一面（关注公众号：捡田螺的小男孩。）.....	338
12.1 用到分布式事务嘛？为什么用这种方案，有其他方案嘛？.....	338
12.2 JDK6、7、8 分别提供了哪些新特性.....	341
12.3 https 原理，工作流程.....	343
12.4 讲讲 java jmm volatile 的实现原理.....	344
12.5 讲一讲 7 层网络模型，tcp 的为什么要三次握手.....	347
12.6 说说线程池的工作原理.....	348
12.7 你们数据库的高可用是怎么实现的？.....	350
12.8 读写分离的场景下，怎么保证从数据库读到最新的数据？.....	355
12.9 如何保证 MySQL 数据不丢？.....	357
12.10 高并发下如何设计秒杀系统？.....	361
13. 虾皮服务端 15 连问（关注公众号：捡田螺的小男孩。）.....	363
13.1 排序链表.....	364
13.2 对称与非对称加密算法的区别.....	366
13.3 TCP 如何保证可靠性.....	367
13.4 聊聊五种 IO 模型.....	367
13.5 hystrix 工作原理.....	372
13.6 延时场景处理.....	373
13.7 https 请求过程.....	373
13.8 聊聊事务隔离级别，以及可重复读实现原理.....	375
13.9 聊聊索引在哪些场景下会失效？.....	379
13.10 什么是虚拟内存.....	380
13.11 排行榜的实现，比如高考成绩排序.....	381
13.12 分布式锁实现.....	382
13.13. 零拷贝.....	386
13.14 synchronized.....	390
13.15 分布式 id 生成方案有哪些？什么是雪花算法？.....	395
14. 字节跳动面试（关注公众号：捡田螺的小男孩。）.....	396
14.1 说说 Redis 为什么快.....	396
14.2 Redis 有几种数据结构，底层分别是怎么存储的.....	400
14.3 Redis 有几种持久化方式.....	402
14.4 多线程情况下，如何保证线程安全？.....	405
14.5 用过 volatile 吗？它是如何保证可见性的，原理是什么.....	405
14.6 MySQL 的索引结构，聚簇索引和非聚簇索引的区别.....	407

14.7 MySQL 有几种高可用方案，你们用的是哪一种.....	407
14.8 说说你做过最有挑战性的项目，你负责那个模块，哪些最有挑战性，说说你做了哪些优化.....	412
14.9 秒杀采用什么方案。.....	412
14.10 聊聊分库分表，分表为什么要停服这种操作，如果不停服可以怎么做... ..	416
14.11 redis 挂了怎么办？.....	417
14.12 你怎么防止优惠券有人重复刷？.....	420
14.13 抖音评论系统怎么设计，如果加入好友关系呢？.....	421
14.14 怎么设计一个短链地址，要考虑跨机房部署问题.....	421
14.15 有一个整型数组，数组元素不重复，数组元素先升序后降序，找出最大值。.....	422
15.字节二面（关注公众号：捡田螺的小男孩。）.....	424
15.1 http 请求头里，expire 和 cache-control 字段含义，说说 HTTP 状态码.....	424
15.2 https 原理，数字签名，数字证书。.....	425
15.3 tcp 连接 client 和 server 有哪些状态，time_wait 状态.....	427
15.4 什么是虚拟内存？什么是物理内存？.....	430
15.5 一台机器最多可以建立多少个 tcp 连接，client 端，server 端，超过了怎么办.....	431
15.6 Eureka 原理，是否是强一致性，eureka 集群。宕机了服务还能调用么？Eureka 和 ZooKeeper 对比.....	431
15.7.Hystrix 了解嘛？说说 Hystrix 的工作原理.....	433
15.8 zookeeper 一致性保证，zab 协议原理，zookeeper 属于哪种一致性，强一致性么，还是最终一致性.....	434
15.9 聊聊 zookeeper 选举机制.....	435
15.10 算法：给定一个字符串 s，请你找出其中不含有重复字符的最长连续子字符串的长度。.....	438

前言

大家好呀，我是**捡田螺的小男孩**。

为了帮助大家通过面试，拿到更好的 offer，我把以前的面试专栏的文章，整

理成 PDF 啦！有 MySQL、Redis、消息队列、计算机网络、

Zookeeper、Java 基础、以及各大厂的面试真题，大家一起学习，卷起来。

更多干货内容，大家可以关注我的公众号哈：**捡田螺的小男孩**。



1. MySQL 专栏面试题 (关注公众号：捡田螺的小男孩)

1.1 MySQL 索引使用有哪些注意事项呢？

可以从三个维度回答这个问题：索引哪些情况会失效，索引不适合哪些场景，索引一些规则特性

1.1.1 索引哪些情况会失效

- 查询条件包含 or，可能导致索引失效
- 如何字段类型是字符串，where 时一定用引号括起来，否则索引失效
- like 通配符可能导致索引失效。
- 联合索引，查询时的条件列不是联合索引中的第一个列，索引失效。
- 在索引列上使用 mysql 的内置函数，索引失效。
- 对索引列运算（如，+、-、*、/），索引失效。
- 索引字段上使用（!= 或者 < >，not in）时，可能会导致索引失效。
- 索引字段上使用 is null，is not null，可能导致索引失效。
- 左连接查询或者右连接查询查询关联的字段编码格式不一样，可能导致索引失效。
- mysql 估计使用全表扫描要比使用索引快,则不使用索引。

可以看下我这篇文章哈：[后端程序员必备：索引失效的十大杂症](#)

1.1.2 索引不适合哪些场景

- 数据量少的不适合加索引
- 更新比较频繁的也不适合加索引
- 区分度低的字段不适合加索引（如性别）

1.1.3 索引的一些潜规则

- 覆盖索引
- 回表
- 索引数据结构（B+树）
- 最左前缀原则
- 索引下推

1.2. MySQL 遇到过死锁问题吗，你是如何解决的？

我排查死锁的一般步骤是酱紫的：

- 查看死锁日志 `show engine innodb status;`
- 找出死锁 Sql
- 分析 sql 加锁情况
- 模拟死锁案发
- 分析死锁日志
- 分析死锁结果

可以看我这两篇文章哈：

- [手把手教你分析 Mysql 死锁问题](#)
- [两万字详解！InnoDB 锁专题！](#)

1.3. 日常工作中你是怎么优化 SQL 的？

可以从这几个维度回答这个问题：

- 加索引
- 避免返回不必要的数据

- 适当分批量进行
- 优化 sql 结构
- 分库分表
- 读写分离

可以看我这两篇文章哈：

- [后端程序员必备：书写高质量 SQL 的 30 条建议](#)
- [我们为什么要分库分表？](#)

1.4. 说说分库与分表的设计

分库分表方案，分库分表中间件，分库分表可能遇到的问题

分库分表方案：

- 水平分库：以字段为依据，按照一定策略（hash、range 等），将一个库中的数据拆分到多个库中。
- 水平分表：以字段为依据，按照一定策略（hash、range 等），将一个表中的数据拆分到多个表中。
- 垂直分库：以表为依据，按照业务归属不同，将不同的表拆分到不同的库中。
- 垂直分表：以字段为依据，按照字段的活跃性，将表中字段拆到不同的表（主表和扩展表）中。

常用的分库分表中间件：

- sharding-jdbc（当当）
- Mycat
- TDDL（淘宝）
- Oceanus(58 同城数据库中间件)
- vitess（谷歌开发的数据库中间件）
- Atlas(Qihoo 360)

分库分表可能遇到的问题

- 事务问题：需要用分布式事务啦
- 跨节点 Join 的问题：解决这一问题可以分两次查询实现
- 跨节点的 count,order by,group by 以及聚合函数问题：分别在各个节点上得到结果后在应用程序端进行合并。

- 数据迁移，容量规划，扩容等问题
- ID 问题：数据库被切分后，不能再依赖数据库自身的主键生成机制啦，最简单可以考虑 UUID
- 跨分片的排序分页问题（后台加大 pagesize 处理？）

这篇文章不错，小伙伴们可以去看一下哈：

- [我们为什么要分库分表？](#)

1.5. InnoDB 与 MyISAM 的区别

- InnoDB 支持事务，MyISAM 不支持事务
- InnoDB 支持外键，MyISAM 不支持外键
- InnoDB 支持 MVCC(多版本并发控制)，MyISAM 不支持
- select count(*) from table 时，MyISAM 更快，因为它有一个变量保存了整个表的总行数，可以直接读取，InnoDB 就需要全表扫描。
- InnoDB 不支持全文索引，而 MyISAM 支持全文索引（5.7 以后的 InnoDB 也支持全文索引）
- InnoDB 支持表、行级锁，而 MyISAM 支持表级锁。
- InnoDB 表必须有主键，而 MyISAM 可以没有主键
- InnoDB 表需要更多的内存和存储，而 MyISAM 可被压缩，存储空间较小，。
- InnoDB 按主键大小有序插入，MyISAM 记录插入顺序是，按记录插入顺序保存。
- InnoDB 存储引擎提供了具有提交、回滚、崩溃恢复能力的事务安全，与 MyISAM 比 InnoDB 写的效率差一些，并且会占用更多的磁盘空间以保留数据和索引

1.6. 数据库索引的原理，为什么要用 B+ 树，为什么不用二叉树？

可以从几个维度去看这个问题，查询是否够快，效率是否稳定，存储数据多少，以及查找磁盘次数，为什么不是二叉树，为什么不是平衡二叉树，为什么不是 B 树，而偏偏是 B+ 树呢？

为什么不是一般二叉树？

如果二叉树特殊化为一个链表，相当于全表扫描。平衡二叉树相比于二叉查找树来说，查找效率更稳定，总体的查找速度也更快。

为什么不是平衡二叉树呢？

我们知道，在内存比在磁盘的数据，查询效率快得多。如果树这种数据结构作为索引，那我们每查找一次数据就需要从磁盘中读取一个节点，也就是我们说的一个磁盘块，但是平衡二叉树可是每个节点只存储一个键值和数据的，如果是 B 树，可以存储更多的节点数据，树的高度也会降低，因此读取磁盘的次数就降下来啦，查询效率就快啦。

那为什么不是 B 树而是 B+树呢？

1) B+树非叶子节点上是不存储数据的，仅存储键值，而 B 树节点中不仅存储键值，也会存储数据。innodb 中页的默认大小是 16KB，如果不存储数据，那么就会存储更多的键值，相应的树的阶数（节点的子节点数）就会更大，树就会更矮更胖，如此一来我们查找数据进行磁盘的 IO 次数有会再次减少，数据查询的效率也会更快。

2) B+树索引的所有数据均存储在叶子节点，而且数据是按照顺序排列的，链表连着的。那么 B+树使得范围查找，排序查找，分组查找以及去重查找变得异常简单。

可以看这篇文章哈：[再有人问你为什么 MySQL 用 B+树做索引，就把这篇文章发给她](#)

1.7. 聚集索引与非聚集索引的区别

- 一个表中只能拥有一个聚集索引，而非聚集索引一个表可以存在多个。

- 聚集索引，索引中键值的逻辑顺序决定了表中相应行的物理顺序；非聚集索引，索引中索引的逻辑顺序与磁盘上行的物理存储顺序不同。
- 索引是通过二叉树的数据结构来描述的，我们可以这么理解聚簇索引：索引的叶节点就是数据节点。而非聚簇索引的叶节点仍然是索引节点，只不过有一个指针指向对应的数据块。
- 聚集索引：物理存储按照索引排序；非聚集索引：物理存储不按照索引排序；

1.8 limit 1000000 加载很慢的话，你是怎么解决的呢？

方案一：如果 id 是连续的，可以这样，返回上次查询的最大记录(偏移量)，再往下 limit

```
select id,name from employee where id>1000000 limit 10.
```

方案二：在业务允许的情况下限制页数：

建议跟业务讨论，有没有必要查这么后的分页啦。因为绝大多数用户都不会往后翻太多页。

方案三：order by + 索引 (id 为索引)

```
select id,name from employee order by id limit 1000000,10
```

方案四：利用延迟关联或者子查询优化超多分页场景。（先快速定位需要获取的 id 段，然后再关联）

```
SELECT a.* FROM employee a, (select id from employee where 条件 LIMIT 1000000,10 ) b
where a.id=b.id
```

可以看下这两篇文章哈：

- [实战！聊聊如何解决 MySQL 深分页问题](#)
- [mysql 查询 limit 1000,10 和 limit 10 速度一样快吗？](#)

1.9 如何选择合适的分布式主键方案呢？

- 数据库自增长序列或字段。
- UUID。
- Redis 生成 ID
- Twitter 的 snowflake 算法
- 利用 zookeeper 生成唯一 ID
- MongoDB 的 ObjectId

1.10 事务的隔离级别有哪些？MySQL 的默认隔离级别是什么？

- 读未提交 (Read Uncommitted)
- 读已提交 (Read Committed)
- 可重复读 (Repeatable Read)
- 串行化 (Serializable)

Mysql 默认的事务隔离级别是可重复读(Repeatable Read)

可以看我这两篇文章哈：

- [一文彻底读懂 MySQL 事务的四大隔离级别](#)
- [看一遍就理解，MVCC 原理详解](#)

1.11 什么是幻读，脏读，不可重复读呢？

- 事务 A、B 交替执行，事务 A 被事务 B 干扰到了，因为事务 A 读取到事务 B 未提交的数据,这就是**脏读**
- 在一个事务范围内，两个相同的查询，读取同一条记录，却返回了不同的数据，这就是**不可重复读**。
- 事务 A 查询一个范围的结果集，另一个并发事务 B 往这个范围中插入/删除了数据，并静悄悄地提交，然后事务 A 再次查询相同的范围，两次读取得到的结果集不一样了，这就是**幻读**。

可以看我这篇文章哈：[看一遍就理解，MVCC 原理详解](#)

1.12 在高并发情况下，如何做到安全的修改同一行数据？

要安全的修改同一行数据，就要保证一个线程在修改时其它线程无法更新这行记录。一般有悲观锁和乐观锁两种方案～

1.12.1 使用悲观锁

悲观锁思想就是，当前线程要进来修改数据时，别的线程都得拒之门外～ 比如，可以使用 `select...for update` ～

```
select * from User where name='jay' for update
```

以上这条 sql 语句会锁定了 User 表中所有符合检索条件 (name='jay') 的记录。本次事务提交之前，别的线程都无法修改这些记录。

1.12.2 使用乐观锁

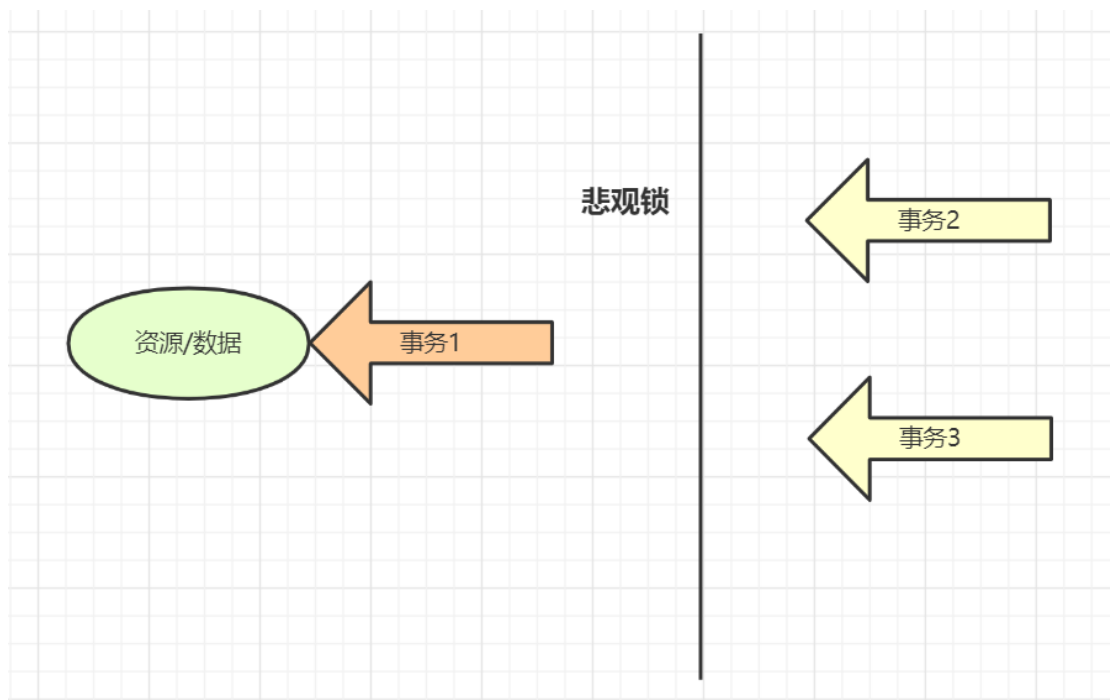
乐观锁思想就是，有线程过来，先放过去修改，如果看到别的线程没修改过，就可以修改成功，如果别的线程修改过，就修改失败或者重试。实现方式：乐观锁一般会使用版本号机制或 CAS 算法实现。

可以看一下我这篇文章，主要是思路哈～ [CAS 乐观锁解决并发问题的一次实践](#)

1.13 数据库的乐观锁和悲观锁。

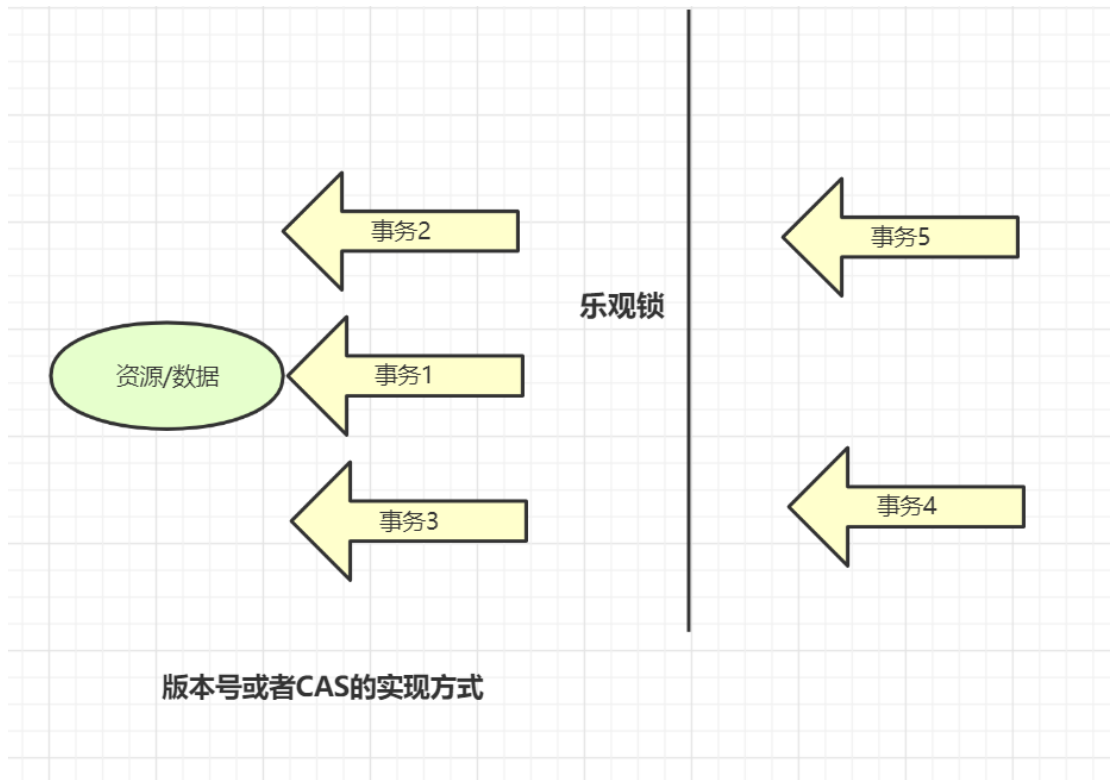
1.13.1 悲观锁：

悲观锁她专一且缺乏安全感了，她的心只属于当前事务，每时每刻都担心着它心爱的数据可能被别的事务修改，所以一个事务拥有（获得）悲观锁后，其他任何事务都不能对数据进行修改啦，只能等待锁被释放才可以执行。



1.13.2 乐观锁：

乐观锁的“乐观情绪”体现在，它认为数据的变动不会太频繁。因此，它允许多个事务同时对数据进行变动。实现方式：乐观锁一般会使用版本号机制或 CAS 算法实现。



之前转载了的这篇文章，觉得作者写得挺详细的～

- [图文并茂的带你彻底理解悲观锁与乐观锁](#)

**1.14 SQL 优化的一般步骤是什么，怎么看执行计划 (explain) ，
如何理解其中各个字段的含义。**

- show status 命令了解各种 sql 的执行频率
- 通过慢查询日志定位那些执行效率较低的 sql 语句
- explain 分析低效 sql 的执行计划 (这点非常重要，日常开发中用它分析 Sql，会大大降低 Sql 导致的线上事故)

可以看下这篇文章：

- [面试官：不会看 Explain 执行计划，简历敢写 SQL 优化？](#)

1.15. select for update 有什么含义，会锁表还是锁行还是其他。

1.15.1 select for update 含义

select 查询语句是不会加锁的，但是 select for update 除了有查询的作用外，还会加锁呢，而且它是悲观锁哦。至于加了是行锁还是表锁，这就要看是不是用了索引/主键啦。

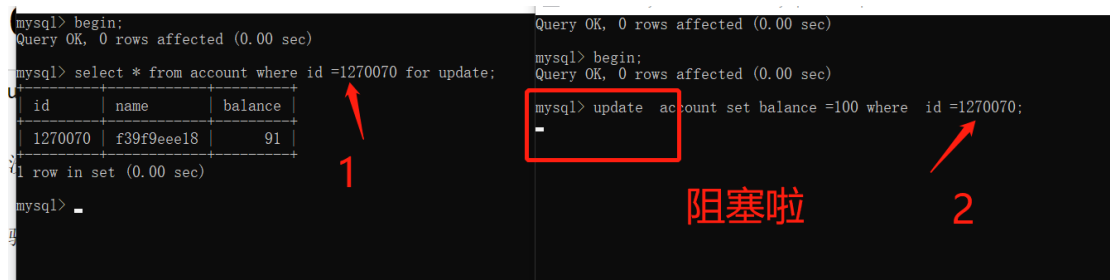
没用索引/主键的话就是表锁，否则就是是行锁。

1.15.2 select for update 加锁验证

表结构：

```
//id 为主键，name 为唯一索引
CREATE TABLE `account` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `name` varchar(255) DEFAULT NULL,
  `balance` int(11) DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `idx_name` (`name`) USING BTREE
) ENGINE=InnoDB AUTO_INCREMENT=1570068 DEFAULT CHARSET=utf8
```

id 为主键，select for update 1270070 这条记录时，再开一个事务对该记录更新，发现更新阻塞啦，其实是加锁了。如下图：



```
mysql> begin;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from account where id=1270070 for update;
+----+-----+-----+
| id | name | balance |
+----+-----+-----+
| 1270070 | f39f9eee18 | 91 |
+----+-----+-----+
1 row in set (0.00 sec)

mysql> _

mysql> begin;
Query OK, 0 rows affected (0.00 sec)

mysql> update account set balance=100 where id=1270070;
-
阻塞啦
```

我们再来开一个事务对另外一条记录 1270071 更新，发现更新成功，因此，如

果查询条件用了索引/主键，会加行锁~

```
mysql> begin;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from account where id =1270070 for update;
+----+-----+-----+
| id  | name  | balance |
+----+-----+-----+
| 1270070 | f39f9eee18 | 91 |
+----+-----+-----+
1 row in set (0.00 sec)

mysql> _
```

1

```
Query OK, 0 rows affected (0.00 sec)

mysql> begin;
Query OK, 0 rows affected (0.00 sec)

mysql> update account set balance =100 where id =1270071;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql>
```

2

没阻塞

我们继续一路向北吧，换普通字段 balance 吧，发现又阻塞了。因此，没用索引

引/主键的话，**select for update** 加的就是表锁

```
mysql> begin;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from account where balance=91 limit 1 for update;
+----+-----+-----+
| id  | name  | balance |
+----+-----+-----+
| 1270070 | f39f9eee18 | 91 |
+----+-----+-----+
1 row in set (0.00 sec)

mysql> _
```

我查询条件是91哦

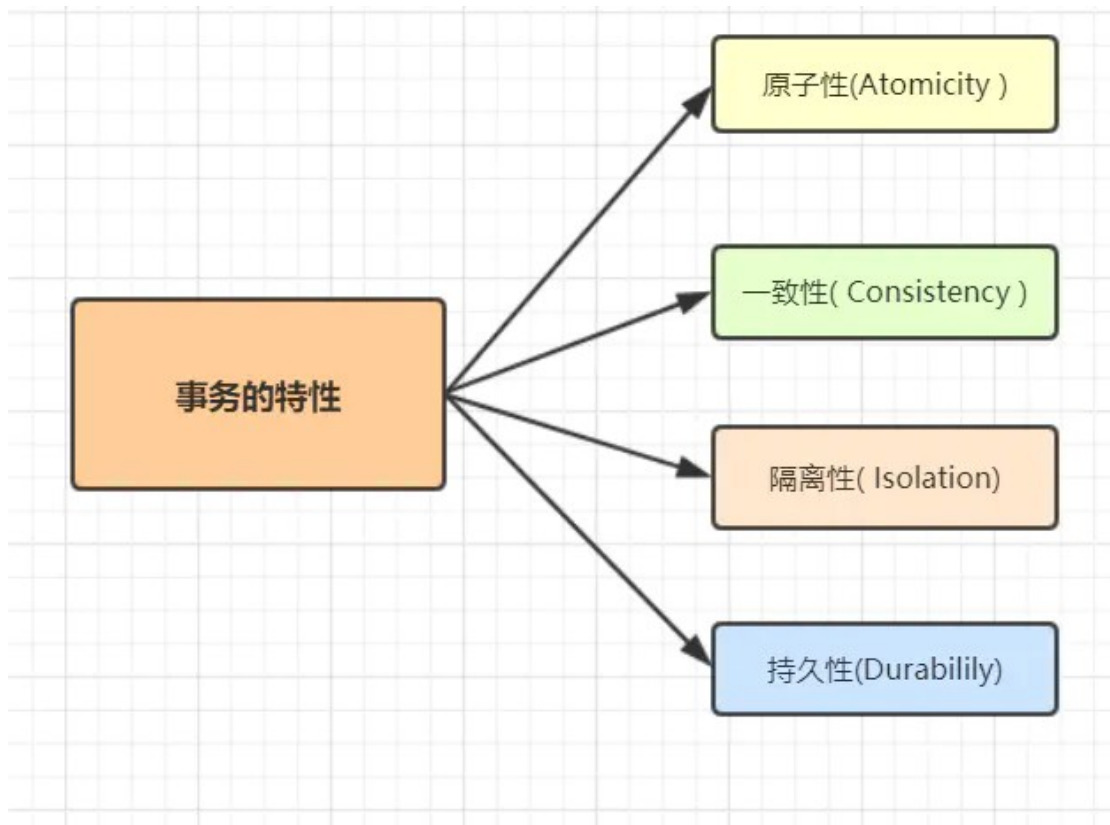
```
mysql> begin;
Query OK, 0 rows affected (0.00 sec)

mysql> update account set balance =100 where balance=1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql>
```

又阻塞了

1.16 MySQL 事务得四大特性以及实现原理



- 原子性：事务作为一个整体被执行，包含在其中的对数据库的操作要么全部被执行，要么都不执行。
- 一致性：指在事务开始之前和事务结束以后，数据不会被破坏，假如 A 账户给 B 账户转 10 块钱，不管成功与否，A 和 B 的总金额是不变的。
- 隔离性：多个事务并发访问时，事务之间是相互隔离的，即一个事务不影响其它事务运行效果。简言之，就是事务之间是进水不犯河水的。
- 持久性：表示事务完成以后，该事务对数据库所作的操作更改，将持久地保存在数据库之中。

事务 ACID 特性的实现思想

- 原子性：是使用 undo log 来实现的，如果事务执行过程中出错或者用户执行了 rollback，系统通过 undo log 日志返回事务开始的状态。
- 持久性：使用 redo log 来实现，只要 redo log 日志持久化了，当系统崩溃，即可通过 redo log 把数据恢复。
- 隔离性：通过锁以及 MVCC,使事务相互隔离开。
- 一致性：通过回滚、恢复，以及并发情况下的隔离性，从而实现一致性。

1.17 如果某个表有近千万数据，CRUD 比较慢，如何优化。

分库分表

某个表有近千万数据，可以考虑优化表结构，分表（水平分表，垂直分表），

当然，你这样回答，需要准备好面试官问你的分库分表相关问题呀，如

- 分表方案（水平分表，垂直分表，切分规则 hash 等）
- 分库分表中间件（Mycat，sharding-jdbc 等）
- 分库分表一些问题（事务问题？跨节点 Join 的问题）
- 解决方案（分布式事务等）

索引优化

除了分库分表，优化表结构，当然还有索引优化等方案～

有兴趣可以看我这两篇文章哈～

- [后端程序员必备：书写高质量 SQL 的 30 条建议](#)
- [我们为什么要分库分表？](#)

1.18 如何写 sql 能够有效的使用到复合索引。

复合索引，也叫联合索引，用户可以在多个列上建立索引,这种索引叫做复合索引。

当我们创建一个组合索引的时候，如(k1,k2,k3)，相当于创建了（k1）、

(k1,k2)和(k1,k2,k3)三个索引，这就是最左匹配原则。

```
select * from table where k1=A AND k2=B AND k3=D
```

有关于复合索引，我们需要关注查询 Sql 条件的顺序，确保最左匹配原则有效，同时可以删除不必要的冗余索引。

1.19 mysql 中 in 和 exists 的区别。

这个，跟一下 demo 来看更刺激吧，啊哈哈

假设表 A 表示某企业的员工表，表 B 表示部门表，查询所有部门的所有员工，很容易有以下 SQL:

```
select * from A where deptId in (select deptId from B);
```

这样写等价于：

先查询部门表 B

```
select deptId from B
```

再由部门 deptId，查询 A 的员工

```
select * from A where A.deptId = B.deptId
```

可以抽象成这样的循环：

```
List<> resultSet ;
for(int i=0;i<B.length;i++) {
    for(int j=0;j<A.length;j++) {
        if(A[i].id==B[j].id) {
            resultSet.add(A[i]);
            break;
        }
    }
}
```

显然，除了使用 in，我们也可以用 exists 实现一样的查询功能，如下：

```
select * from A where exists (select 1 from B where A.deptId = B.deptId);
```

因为 exists 查询的理解就是，先执行主查询，获得数据后，再放到子查询中做条件验证，根据验证结果（true 或者 false），来决定主查询的数据结果是否得意保留。

那么，这样写就等价于：

```
select * from A,先从A表做循环
```

```
select * from B where A.deptId = B.deptId,再从B表做循环。
```

同理，可以抽象成这样一个循环：

```
List<> resultSet ;
for(int i=0;i<A.length;i++) {
    for(int j=0;j<B.length;j++) {
        if(A[i].deptId==B[j].deptId) {
            resultSet.add(A[i]);
            break;
        }
    }
}
```

数据库最费劲的就是跟程序链接释放。假设链接了两次，每次做上百万次的数据集查询，查完就走，这样就只做了两次；相反建立了上百万次链接，申请链接释放反复重复，这样系统就受不了了。即 mysql 优化原则，就是小表驱动大表，小的数据集驱动大的数据集，从而让性能更优。

因此，我们要选择最外层循环小的，也就是，如果 **B 的数据量** 小于 **A**，适合使用 **in**，如果 **B 的数据量** 大于 **A**，即适合选择 **exists**，这就是 in 和 exists 的区别。

1.20 数据库自增主键可能遇到什么问题。

- 使用自增主键对数据库做分库分表，可能出现诸如主键重复等的问题。解决方案的话，简单点的话可以考虑使用 UUID 哈
- 自增主键会产生表锁，从而引发问题
- 自增主键可能用完问题。

1.21 MVCC 熟悉吗，它的底层原理？

MVCC,多版本并发控制,它是通过读取历史版本的数据，来降低并发事务冲突，从而提高并发性能的一种机制。

MVCC 需要关注这几个知识点：

- 事务版本号
- 表的隐藏列
- undo log
- read view

可以看下我的这篇文章：

- [看一遍就理解：MVCC 原理详解](#)

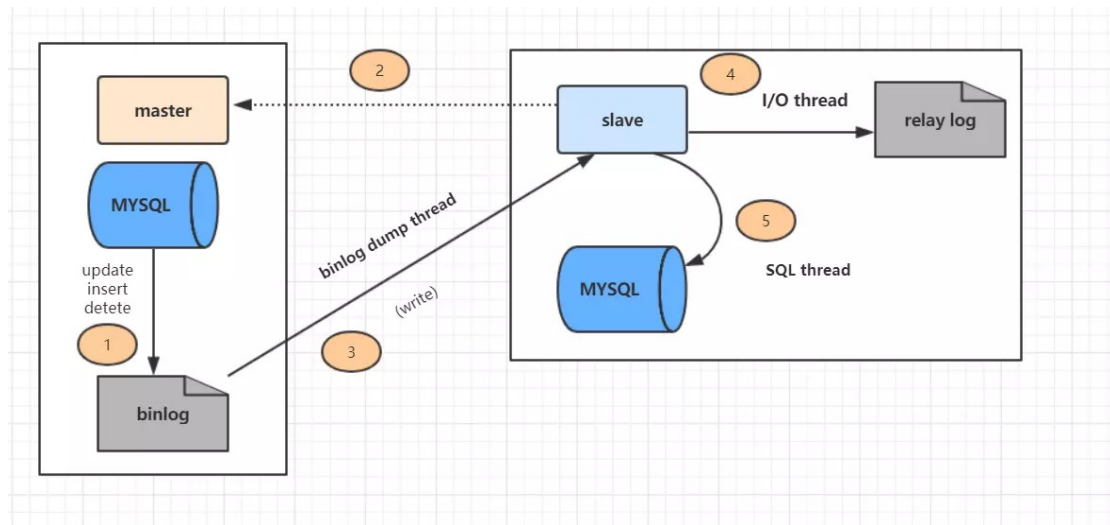
1.22 数据库中间件了解过吗，sharding jdbc，mycat？

- sharding-jdbc 目前是基于 jdbc 驱动，无需额外的 proxy，因此也无需关注 proxy 本身的高可用。
- Mycat 是基于 Proxy，它复写了 MySQL 协议，将 Mycat Server 伪装成一个 MySQL 数据库，而 Sharding-JDBC 是基于 JDBC 接口的扩展，是以 jar 包的形式提供轻量级服务的。

有网友推荐这篇文章：[深度认识 Sharding-JDBC：做最轻量级的数据库中间层](#)

1.23 MYSQL 的主从延迟，你怎么解决？

嘻嘻，先复习一下主从复制原理吧，如图：



主从复制分了五个步骤进行：

- 步骤一：主库的更新事件(update、insert、delete)被写到 binlog
- 步骤二：从库发起连接，连接到主库。
- 步骤三：此时主库创建一个 binlog dump thread，把 binlog 的内容发送到从库。
- 步骤四：从库启动之后，创建一个 I/O 线程，读取主库传过来的 binlog 内容并写入到 relay log
- 步骤五：还会创建一个 SQL 线程，从 relay log 里面读取内容，从 Exec_Master_Log_Pos 位置开始执行读取到的更新事件，将更新内容写入到 slave 的 db

可以看看我这篇文章：[面试必备：聊聊 MySQL 的主从](#)

1.23.1 主从同步延迟的原因

一个服务器开放N个链接给客户端来连接的，这样会有大并发的更新操作，但是从服务器的里面读取 binlog 的线程仅有一个，当某个 SQL 在从服务器上执行的时间稍长 或者由于某个 SQL 要进行锁表就会导致，主服务器的 SQL 大量积压，未被同步到从服务器里。这就导致了主从不一致，也就是主从延迟。

1.23.2 主从同步延迟的解决办法

- 主服务器要负责更新操作，对安全性的要求比从服务器要高，所以有些设置参数可以修改，比如 `sync_binlog=1`，`innodb_flush_log_at_trx_commit = 1` 之类的设置等。
- 选择更好的硬件设备作为 slave。
- 把一台从服务器当度作为备份使用，而不提供查询，那边他的负载下来了，执行 relay log 里面的 SQL 效率自然就高了。
- 增加从服务器喽，这个目的还是分散读的压力，从而降低服务器负载。

有兴趣的小伙伴也可以看看我这篇文章：[面试必备：聊聊 MySQL 的主从](#)

1.24 说一下大表查询的优化方案

- 优化 shema、sql 语句+索引；
- 可以考虑加缓存，memcached, redis，或者 JVM 本地缓存；
- 主从复制，读写分离；
- 分库分表；

1.25 什么是数据库连接池?为什么需要数据库连接池呢?

连接池基本原理： 数据库连接池原理：在内部对象池中，维护一定数量的数据库连接，并对外暴露数据库连接的获取和返回方法。

应用程序和数据库建立连接的过程：

- 通过 TCP 协议的三次握手和数据库服务器建立连接
- 发送数据库用户账号密码，等待数据库验证用户身份
- 完成身份验证后，系统可以提交 SQL 语句到数据库执行
- 把连接关闭，TCP 四次挥手告别。

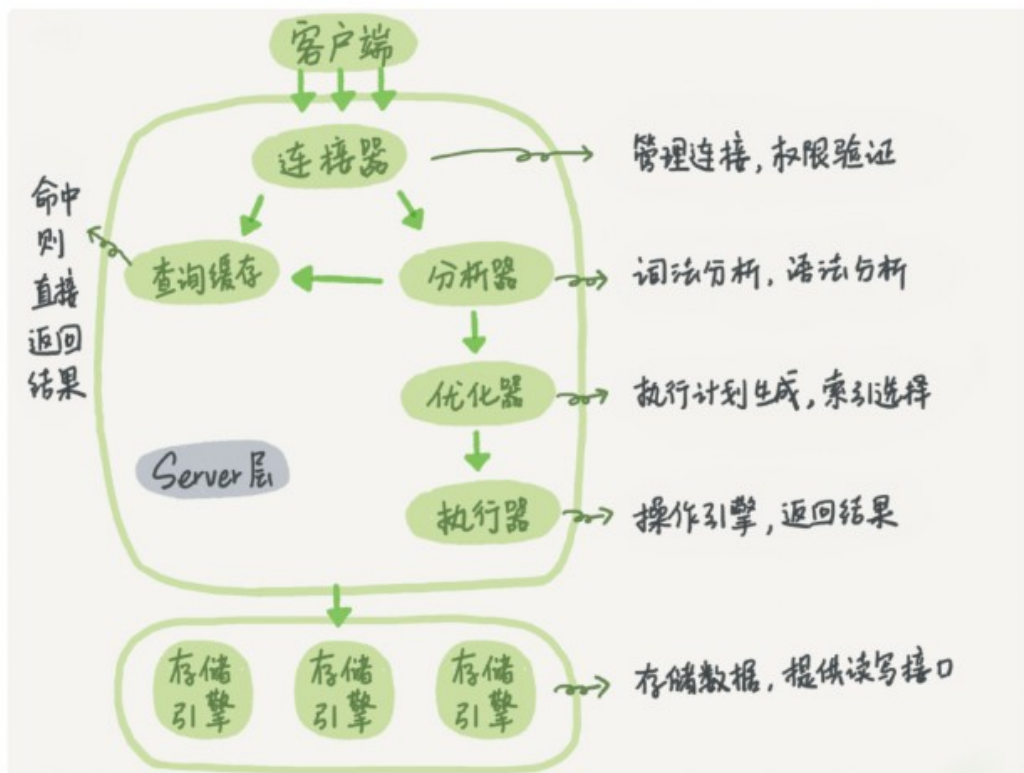
数据库连接池好处：

- 资源重用 (连接复用)
- 更快的系统响应速度
- 新的资源分配手段
- 统一的连接管理，避免数据库连接泄漏

有兴趣的伙伴可以看看我这篇文章哈~ [数据库连接池内存泄漏问题的分析和解决方案](#)

1.26 一条 SQL 语句在 MySQL 中如何执行的？

先看一下 Mysql 的逻辑架构图吧~



查询语句：

- 先检查该语句是否有权限
- 如果没有权限，直接返回错误信息
- 如果有权限，在 MySQL8.0 版本以前，会先查询缓存。
- 如果没有缓存，分析器进行词法分析，提取 sql 语句 select 等的关键元素。然后判断 sql 语句是否有语法错误，比如关键词是否正确等等。
- 优化器进行确定执行方案
- 进行权限校验，如果没有权限就直接返回错误信息，如果有权限就会调用数据库引擎接口，返回执行结果。

这篇文章非常不错，大家去看一下吧：[一条 SQL 语句在 MySQL 中如何执行的](#)

1.27 InnoDB 引擎中的索引策略，了解过吗？

- 覆盖索引
- 最左前缀原则
- 索引下推
- 索引下推优化是 MySQL 5.6 引入的，可以在索引遍历过程中，对索引中包含的字段先做判断，直接过滤掉不满足条件的记录，减少回表次数。
- 最左前缀原则：MySQL 建立联合索引时，会遵循最左前缀匹配的原则，即最左优先。如果你建立一个 (a,b,c) 的联合索引，相当于建立了 (a)、(a,b)、(a,b,c)三个索引。
- 覆盖索引：只需要在一棵索引树上就能获取 SQL 所需的所有列数据，无需回表，速度更快。

这些知识点，我在这篇文章详细介绍了哦，大家可以看看哈

- [阿里一面，给了几条 SQL，问需要执行几次树搜索操作？](#)

1.28 一条 sql 执行过长的时间，你如何优化，从哪些方面入手？

- 查看是否涉及多表和子查询，优化 Sql 结构，如去除冗余字段，是否可拆表等
- 优化索引结构，看是否可以适当添加索引
- 数量大的表，可以考虑进行分离/分表（如交易流水表）
- 数据库主从分离，读写分离
- explain 分析 sql 语句，查看执行计划，优化 sql

- 查看 mysql 执行日志，分析是否有其他方面的问题

1.29 MYSQL 数据库服务器性能分析的方法命令有哪些？

- Show status, 一些值得监控的变量值：
 - Bytes_received 和 Bytes_sent 和服务器之间来往的流量。
 - Com_*服务器正在执行的命令。
 - Created_*在查询执行期间创建的临时表和文件。
 - Handler_*存储引擎操作。
 - Select_*不同类型的联接执行计划。
 - Sort_*几种排序信息。
- Show profiles 是 MySql 用来分析当前会话 SQL 语句执行的资源消耗情况

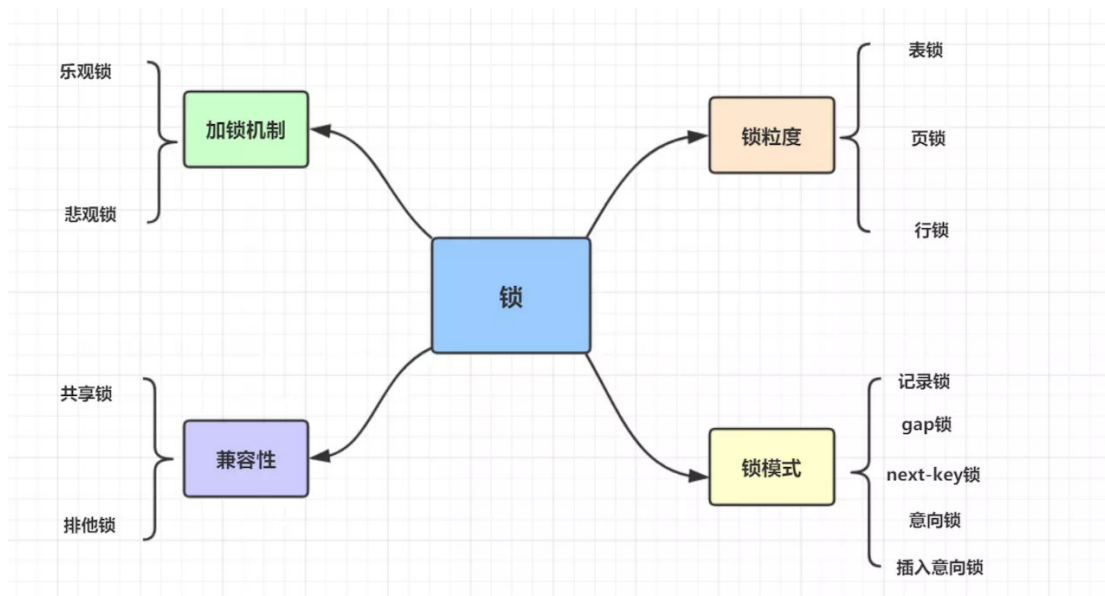
1.30 Blob 和 text 有什么区别？

- Blob 用于存储二进制数据，而 Text 用于存储大字符串。
- Blob 值被视为二进制字符串（字节字符串），它们没有字符集，并且排序和比较基于列值中的字节的数值。
- text 值被视为非二进制字符串（字符串）。它们有一个字符集，并根据字符集的排序规则对值进行排序和比较。

1.31 mysql 里记录货币用什么字段类型比较好？

- 货币在数据库中 MySQL 常用 Decimal 和 Numric 类型表示，这两种类型被 MySQL 实现为同样的类型。他们被用于保存与金钱有关的数据。
- salary DECIMAL(9,2)，9(precision)代表将被用于存储值的总的小数位数，而 2(scale)代表将被用于存储小数点后的位数。存储在 salary 列中的值的范围是从-99999999.99 到 99999999.99。
- DECIMAL 和 NUMERIC 值作为字符串存储，而不是作为二进制浮点数，以便保存那些值的小数精度。

1.32 Mysql 中有哪几种锁，列举一下？



如果按锁粒度划分，有以下 3 种：

- 表锁：开销小，加锁快；锁定力度大，发生锁冲突概率高，并发度最低;不会出现死锁。
- 行锁：开销大，加锁慢；会出现死锁；锁定粒度小，发生锁冲突的概率低，并发度高。
- 页锁：开销和加锁速度介于表锁和行锁之间；会出现死锁；锁定粒度介于表锁和行锁之间，并发度一般

可以看我这篇文章，有介绍到各种锁哈：[两万字详解！InnoDB 锁专题！](#)

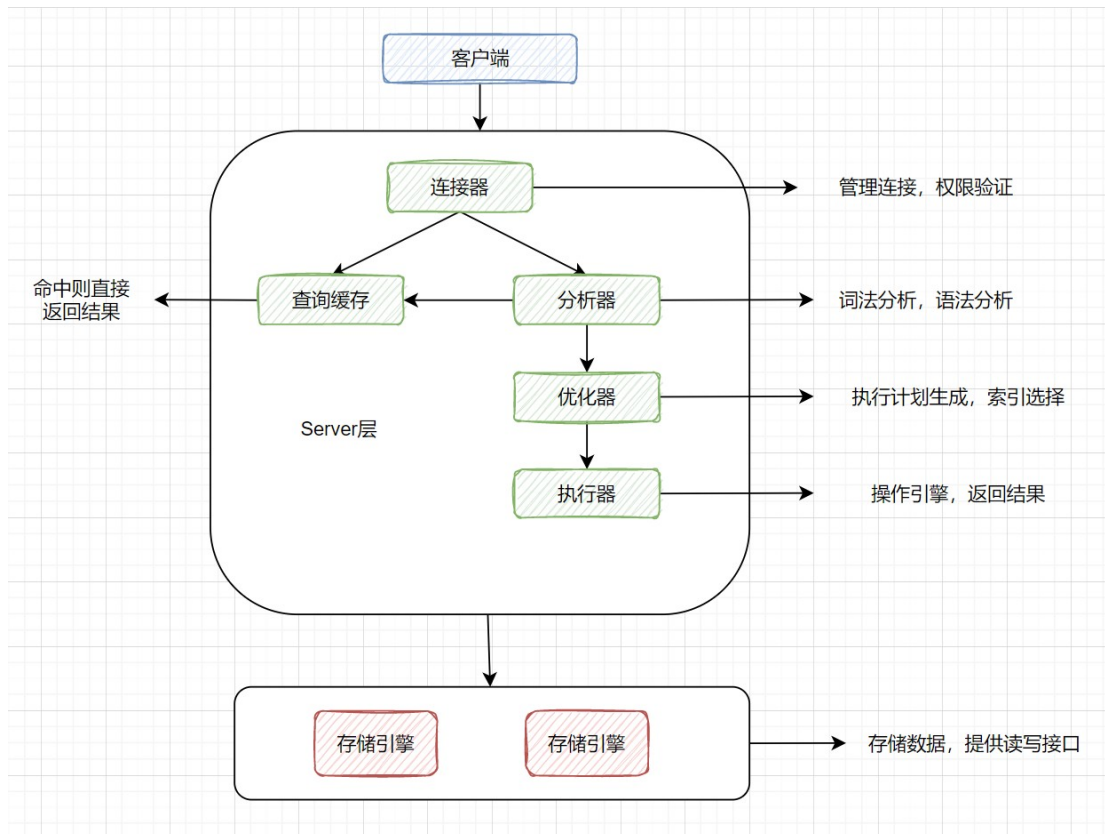
1.33 Hash 索引和 B+树区别是什么？你在设计索引是怎么抉择的？

- B+树可以进行范围查询，Hash 索引不能。
- B+树支持联合索引的最左侧原则，Hash 索引不支持。
- B+树支持 order by 排序，Hash 索引不支持。
- Hash 索引在等值查询上比 B+树效率更高。
- B+树使用 like 进行模糊查询的时候，like 后面（比如%开头）的话可以起到优化的作用，Hash 索引根本无法进行模糊查询。

1.34 mysql 的内连接、左连接、右连接有什么区别？

- Inner join 内连接，在两张表进行连接查询时，只保留两张表中完全匹配的结果集
- left join 在两张表进行连接查询时，会返回左表所有的行，即使在右表中没有匹配的记录。
- right join 在两张表进行连接查询时，会返回右表所有的行，即使在左表中没有匹配的记录。

1.35 说说 MySQL 的基础架构图



Mysql 逻辑架构图主要分三层：

- 第一层负责连接处理，授权认证，安全等等
- 第二层负责编译并优化 SQL
- 第三层是存储引擎。

1.36 什么是内连接、外连接、交叉连接、笛卡尔积呢？

- 内连接 (inner join)：取得两张表中满足存在连接匹配关系的记录。
- 外连接 (outer join)：取得两张表中满足存在连接匹配关系的记录，以及某张表 (或两张表) 中不满足匹配关系的记录。

- 交叉连接 (cross join) : 显示两张表所有记录一一对应, 没有匹配关系进行筛选, 也被称为: 笛卡尔积。

1.37 说一下数据库的三大范式

- 第一范式: 数据表中的每一列 (每个字段) 都不可以再拆分。
- 第二范式: 在第一范式的基础上, 分主键列完全依赖于主键, 而不能是依赖于主键的一部分。
- 第三范式: 在满足第二范式的基础上, 表中的非主键只依赖于主键, 而不依赖于其他非主键。

1.38 Mysql 的 binlog 有几种录入格式? 分别有什么区别?

有三种格式哈, statement, row 和 mixed。

- statement, 记录的是 SQL 的原文。不需要记录每一行的变化, 减少了 binlog 日志量, 节约了 IO, 提高性能。由于 sql 的执行是有上下文的, 因此在保存的时候需要保存相关的信息, 同时还有一些使用了函数之类的语句无法被记录复制。
- row, 不记录 sql 语句上下文相关信息, 仅保存哪条记录被修改。记录单元为每一行的改动, 基本是可以全部记下来但是由于很多操作, 会导致大量行的改动(比如 alter table), 因此这种模式的文件保存的信息太多, 日志量太大。
- mixed, 一种折中的方案, 普通操作使用 statement 记录, 当无法使用 statement 的时候使用 row。

我这篇文章, 有个章节详细说到这个知识点哈:

[面试必备: 聊聊 MySQL 的主从](#)

1.39 InnoDB 引擎的 4 大特性, 了解过吗

- 插入缓冲 (insert buffer)
- 二次写(double write)
- 自适应哈希索引(ahi)
- 预读(read ahead)

1.40 索引有哪些优缺点？索引有哪几种类型？

1.40.1 索引有哪些优缺点

优点：

- 唯一索引可以保证数据库表中每一行的数据的唯一性
- 索引可以加快数据查询速度，减少查询时间

缺点：

- 创建索引和维护索引要耗费时间
- 索引需要占物理空间，除了数据表占用数据空间之外，每一个索引还要占用一定的物理空间
- 以表中的数据进行增、删、改的时候，索引也要动态的维护。

1.40.2 索引有哪几种类型？

- 主键索引: 数据列不允许重复，不允许为 NULL，一个表只能有一个主键。
- 唯一索引: 数据列不允许重复，允许为 NULL 值，一个表允许多个列创建唯一索引。
- 普通索引: 基本的索引类型，没有唯一性的限制，允许为 NULL 值。
- 全文索引：是目前搜索引擎使用的一种关键技术，对文本的内容进行分词、搜索。
- 覆盖索引：查询列要被所建的索引覆盖，不必读取数据行
- 组合索引：多列值组成一个索引，用于组合搜索，效率大于索引合并

1.41 创建索引有什么原则呢？

- 最左前缀匹配原则
- 频繁作为查询条件的字段才去创建索引
- 频繁更新的字段不适合创建索引
- 索引列不能参与计算，不能有函数操作
- 优先考虑扩展索引，而不是新建索引，避免不必要的索引
- 在 order by 或者 group by 子句中，创建索引需要注意顺序
- 区分度低的数据列不适合做索引列(如性别)
- 定义有外键的数据列一定要建立索引。
- 对于定义为 text、image 数据类型的列不要建立索引。
- 删除不再使用或者很少使用的索引

1.42 创建索引的三种方式

- 在执行 CREATE TABLE 时创建索引

```
CREATE TABLE `employee` (  
  `id` int(11) NOT NULL,  
  `name` varchar(255) DEFAULT NULL,  
  `age` int(11) DEFAULT NULL,  
  `date` datetime DEFAULT NULL,  
  `sex` int(1) DEFAULT NULL,  
  PRIMARY KEY (`id`),  
  KEY `idx_name` (`name`) USING BTREE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

- 使用 ALTER TABLE 命令添加索引

```
ALTER TABLE table_name ADD INDEX index_name (column);
```

- 使用 CREATE INDEX 命令创建

```
CREATE INDEX index_name ON table_name (column);
```

1.43 百万级别或以上的数据，你是如何删除的？

- 我们想要删除百万数据的时候可以先删除索引
- 然后批量删除其中无用数据
- 删除完成后重新创建索引。

1.44 什么是最左前缀原则？什么是最左匹配原则？

- 最左前缀原则，就是最左优先，在创建多列索引时，要根据业务需求，where 子句中使用最频繁的一列放在最左边。
- 当我们创建一个组合索引的时候，如(k1,k2,k3)，相当于创建了 (k1)、(k1,k2)和(k1,k2,k3)三个索引，这就是最左匹配原则。。

1.45 B 树和 B+树的区别，数据库为什么使用 B+树而不是 B 树？

- 在 B 树中，键和值即存放在内部节点又存放在叶子节点；在 B+树中，内部节点只存键，叶子节点则同时存放键和值。
- B+树的叶子节点有一条链相连，而 B 树的叶子节点各自独立的。
- B+树索引的所有数据均存储在叶子节点，而且数据是按照顺序排列的，链表连着的。那么 B+树使得范围查找，排序查找，分组查找以及去重查找变得异常简单。
- B+树非叶子节点上是不存储数据的，仅存储键值，而 B 树节点中不仅存储键值，也会存储数据。innodb 中页的默认大小是 16KB，如果不存储数据，那么就会存储更多的键值，相应的树的阶数（节点的子节点数）就会更大，树就会更矮更胖，如此一来我们查找数据进行磁盘的 IO 次数会有再次减少，数据查询的效率也会更快。

1.46 覆盖索引、回表等这些，了解过吗？

- 覆盖索引：查询列要被所建的索引覆盖，不必从数据表中读取，换句话说查询列要被所使用的索引覆盖。
- 回表：二级索引无法直接查询所有列的数据，所以通过二级索引查询到聚簇索引后，再查询到想要的的数据，这种通过二级索引查询出来的过程，就叫做回表。

1.47 非聚簇索引一定会回表查询吗？

不一定，如果查询语句的字段全部命中了索引，那么就不必再进行回表查询

(哈哈，覆盖索引就是这么回事)。

举个简单的例子，假设我们在学生表的上建立了索引，那么当进行 `select age from student where age < 20` 的查询时，在索引的叶子节点上，已经包含了 age 信息，不会再次进行回表查询。

1.48 联合索引是什么？为什么需要注意联合索引中的顺序？

联合索引，用户可以在多个列上建立索引,这种索引叫做联合索引。

因为 InnoDB 引擎中的索引策略的最左原则，所以需要注意联合索引中的顺序。

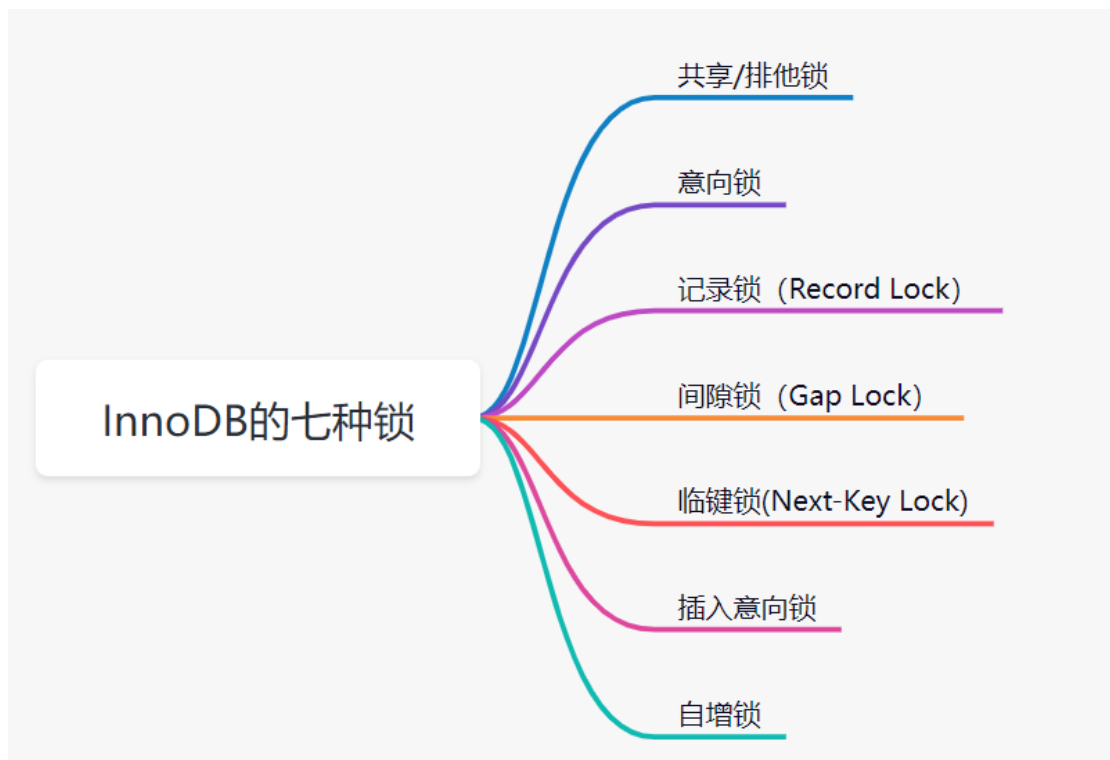
1.49 数据库隔离级别与锁的关系

回答这个问题，可以先阐述四种隔离级别，再阐述它们的实现原理。隔离级别就是依赖锁和 MVCC 实现的。

可以看我这篇文章哈：[一文彻底读懂 MySQL 事务的四大隔离级别](#)

1.50 聊聊 InnoDB 的锁，什么是死锁？怎么解决？

1.50.1 聊聊 InnoDB 的锁



1.50.1.1 共享/排他锁

InnoDB 呢实现了两种标准的行级锁：共享锁（简称 S 锁）、排他锁（简称 X 锁）。

- 共享锁：简称为 S 锁，在事务要读取一条记录时，需要先获取该记录的 S 锁。
- 排他锁：简称 X 锁，在事务需要改动一条记录时，需要先获取该记录的 X 锁。

如果事务 T1 持有行 R 的 S 锁，那么另一个事务 T2 请求访问这条记录时，会做如下处理：

- T2 请求 S 锁立即被允许，结果 T1 和 T2 都持有 R 行的 S 锁
- T2 请求 X 锁不能被立即允许，此操作阻塞

如果 T1 持有行 R 的 X 锁，那么 T2 请求 R 的 X、S 锁都不能被立即允许，T2 必须等待 T1 释放 X 锁才可以，因为 X 锁与任何的锁都不兼容。

S 锁和 X 锁的兼容关系如下：

兼容性	S	X
S	兼容	不兼容
X	不兼容	不兼容

捡田螺的小男孩

X 锁和 S 锁是对于行记录来说的话，可以称它们为行级锁或者行锁。我们认为行锁的粒度就比较细，其实一个事务也可以在表级别下加锁，我们称之为表锁。

给表加的锁，也是可以分为 X 锁和 S 锁的哈。

如果一个事务给表已经加了 S 锁，则：

- 别的事务可以继续获得该表的 S 锁，也可以获得该表中某些记录的 S 锁。
- 别的事务不可以继续获得该表的 X 锁，也不可以获得该表中某些记录的 X 锁。

如果一个事务给表加了 X 锁，那么

- 别的事务不可以获得该表的 S 锁，也不可以获得该表某些记录的 S 锁。
- 别的事务不可以获得该表的 X 锁，也不可以继续获得该表某些记录的 X 锁。

1.50.1.2 意向锁

什么是意向锁呢？意向锁是一种不与行级锁冲突的表级锁。未来的某个时刻，事务可能要加共享或者排它锁时，先提前声明一个意向。注意一下，意向锁，是一个表级别的锁哈。

为什么需要意向锁呢？或者换个通俗的说法，为什么要加共享锁或排他锁的时候，需要提前声明个意向锁呢呢？

因为 InnoDB 是支持表锁和行锁共存的，如果一个事务 A 获取到某一行的排他锁，并未提交，这时候事务 B 请求获取同一个表的表共享锁。因为共享锁和排他锁是互斥的，因此事务 B 想对这个表加共享锁时，需要保证没有其他事务持有这个表的表排他锁，同时还要保证没有其他事务持有表中任意一行的排他锁。然后问题来了，你要保证没有其他事务持有表中任意一行的排他锁的话，去遍历每一行？这样显然是一个效率很差的做法。为了解决这个问题，InnoDB 的设计大叔提出了意向锁。

意向锁是如何解决这个问题的呢？我们来看下

意向锁分为两类：

- 意向共享锁：简称 IS 锁，当事务准备在某些记录上加 S 锁时，需要现在表级别加一个 IS 锁。
- 意向排他锁：简称 IX 锁，当事务准备在某条记录上加上 X 锁时，需要现在表级别加一个 IX 锁。

比如：

- `select ... lock in share mode`，要给表设置 IS 锁；
- `select ... for update`，要给表设置 IX 锁；

意向锁又是如何解决这个效率低的问题呢：

如果一个事务 A 获取到某一行的排他锁，并未提交,这时候表上就有**意向排他锁**和这一行的**排他锁**。这时候事务 B 想要获取这个表的共享锁，此时因为检测到事务 A 持有了表的**意向排他锁**，因此事务 A 必然持有某些行的排他锁，也就是说事务 B 对表的加锁请求需要阻塞等待，不再需要去检测表的每一行数据是否存在排他锁啦。

意向锁仅仅表明意向的锁，意向锁之间不会互斥，是可以并行的，整体兼容性如下：

兼容性	IS	IX	S	X
IS	兼容	兼容	兼容	不兼容
IX	兼容	兼容	不兼容	不兼容
S	兼容	不兼容	兼容	不兼容
X	不兼容	不兼容	不兼容	不兼容

@稀土掘金技术社区

1.50.1.3 记录锁 (Record Lock)

记录锁是最简单的行锁，仅仅锁住一行。如：`SELECT c1 FROM t WHERE c1 = 10 FOR UPDATE`，如果 C1 字段是主键或者是唯一索引的话，这个 SQL 会加一个记录锁 (Record Lock)

记录锁永远都是加在索引上的，即使一个表没有索引，InnoDB 也会隐式的创建一个索引，并使用这个索引实施记录锁。它会阻塞其他事务对这行记录的插入、更新、删除。

一般我们看死锁日志时，都是找关键词，比如 `lock_mode X locks rec but not gap`)，就表示一个 X 型的记录锁。记录锁的关键词就是 **rec but not gap**。

以下就是一个记录锁的日志：

```

RECORD LOCKS space id 58 page no 3 n bits 72 index `PRIMARY` of table `test`.`t`
trx id 10078 lock_mode X locks rec but not gap
Record lock, heap no 2 PHYSICAL RECORD: n_fields 3; compact format; info bits 0
0: len 4; hex 8000000a; asc      ;;
1: len 6; hex 00000000274f; asc    'O;;
2: len 7; hex b60000019d0110; asc      ;;

```

1.50.1.4 间隙锁 (Gap Lock)

为了解决幻读问题，InnoDB 引入了间隙锁(Gap Lock)。间隙锁是一种加在两个索引之间的锁，或者加在第一个索引之前，或最后一个索引之后的间隙。它锁住的是一个区间，而不仅仅是这个区间中的每一条数据。

比如 `lock_mode X locks gap before rec` 表示 X 型 gap 锁。以下就是一个间隙锁的日志：

```

RECORD LOCKS space id 177 page no 4 n bits 80 index idx_name of table `test2`.`account`
trx id 38049 lock_mode X locks gap before rec
Record lock, heap no 6 PHYSICAL RECORD: n_fields 2; compact format; info bits 0
0: len 3; hex 576569; asc Wei;;
1: len 4; hex 80000002; asc      ;;

```

1.50.1.5 临键锁(Next-Key Lock)

Next-key 锁是记录锁和间隙锁的组合，它指的是加在某条记录以及这条记录前面间隙上的锁。说得更具体一点就是:临键锁会封锁索引记录本身，以及索引记录之前的区间，即它的锁区间是前开后闭，比如(5,10]。

如果一个会话占有了索引记录 R 的共享/排他锁，其他会话不能立刻在 R 之前的区间插入新的索引记录。

If one session has a shared or exclusive lock on record R in an index, another session cannot insert a new index record in the gap immediately before R in the index order.

1.50.1.6 插入意向锁

插入意向锁,是插入一行记录操作之前设置的一种间隙锁，这个锁释放了一种插入方式的信号。它解决的问题：多个事务，在同一个索引，同一个范围区间插入记录时，如果插入的位置不冲突，不会阻塞彼此。

假设有索引值 4、7，几个不同的事务准备插入 5、6，每个锁都在获得插入行的独占锁之前用插入意向锁各自锁住了 4、7 之间的间隙，但是不阻塞对方因为插入行不冲突。以下就是一个插入意向锁的日志：

```
RECORD LOCKS space id 31 page no 3 n bits 72 index `PRIMARY` of table `test`.`child`
\
trx id 8731 lock_mode X locks_gap before rec insert intention waiting
Record lock, heap no 3 PHYSICAL RECORD: n_fields 3; compact format; info bits 0
0: len 4; hex 80000066; asc f;;
1: len 6; hex 000000002215; asc " ;;
2: len 7; hex 9000000172011c; asc r ;;...
```

锁模式兼容矩阵（横向是已持有锁，纵向是正在请求的锁）：

兼容性	Gap	Insert Intention	Record	Next-Key
Gap	兼容	兼容	兼容	兼容
Insert Intention	冲突	兼容	兼容	冲突
Record	兼容	兼容	冲突	冲突
Next-Key	兼容	兼容	冲突	冲突

@稀土掘金技术社区

1.50.1.7 自增锁

自增锁是一种特殊的表级别锁。它是专门针对 `AUTO_INCREMENT` 类型的列，对于这种列，如果表中新增数据时就会去持有自增锁。简言之，如果一个事务正在往表中插入记录，所有其他事务的插入必须等待，以便第一个事务插入的行，是连续的主键值。

官方文档是这么描述的：

An AUTO-INC lock is a special table-level lock taken by transactions inserting into tables with AUTO_INCREMENT columns. In the simplest case, if one transaction is inserting values into the table, any other transactions must wait to do their own inserts into that table, so that rows inserted by the first transaction receive consecutive primary key values.

假设有表：

```
mysql> create table t0 (id int NOT NULL AUTO_INCREMENT,name varchar(16),primary key (id));
```

```
mysql> show variables like '%innodb_autoinc_lock_mode%';
```

```
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_autoinc_lock_mode | 1 |
+-----+-----+
1 row in set, 1 warning (0.01 sec)
```

设置事务 A 和 B 交替执行流程如下：

The diagram illustrates the interleaved execution of two transactions, A and B, on a MySQL table t0. The table has columns id (AUTO_INCREMENT) and name. Transaction A (left) and Transaction B (right) are shown with their SQL commands and output. Red arrows and numbers 1-13 indicate the sequence of operations and the state of the table at each step.

Transaction A (Left):

- 1. `mysql> begin;`
- 2. `mysql> insert into t0(name) values("a");` (Query OK, 1 row affected)
- 3. `mysql> select * from t0;` (Output: id=1, name=a)
- 4. `mysql> insert into t0(name) values("c");` (Query OK, 1 row affected)
- 5. `mysql> select * from t0;` (Output: id=1, name=a; id=3, name=c)
- 6. `mysql> select * from t0;` (Output: id=1, name=a; id=3, name=c)
- 7. `mysql> commit;` (Query OK, 0 rows affected)
- 8. `mysql> select * from t0;` (Output: id=1, name=a; id=3, name=c)

Transaction B (Right):

- 9. `mysql> begin;`
- 10. `mysql> insert into t0(name) values("b");` (Query OK, 1 row affected)
- 11. `mysql> select * from t0;` (Output: id=2, name=b)
- 12. `mysql> commit;` (Query OK, 0 rows affected)
- 13. `mysql> select * from t0;` (Output: id=1, name=a; id=2, name=b; id=3, name=c)

The sequence of operations is as follows:

- Transaction A starts (1).
- Transaction A inserts 'a' (2).
- Transaction A selects the table (3).
- Transaction A inserts 'c' (4).
- Transaction A selects the table (5).
- Transaction A selects the table (6).
- Transaction A commits (7).
- Transaction A selects the table (8).
- Transaction B starts (9).
- Transaction B inserts 'b' (10).
- Transaction B selects the table (11).
- Transaction B commits (12).
- Transaction B selects the table (13).

通过上图我们可以看到，当我们在事务 A 中进行自增列的插入操作时，另外会话事务 B 也进行插入操作，这种情况下会发生 2 个奇怪的现象：

- 事务 A 会话中的自增列好像直接增加了 2 个值。（如上图步骤 7、8）
- 事务 B 会话中的自增列直接从 2 开始增加的。（如上图步骤 5、6）

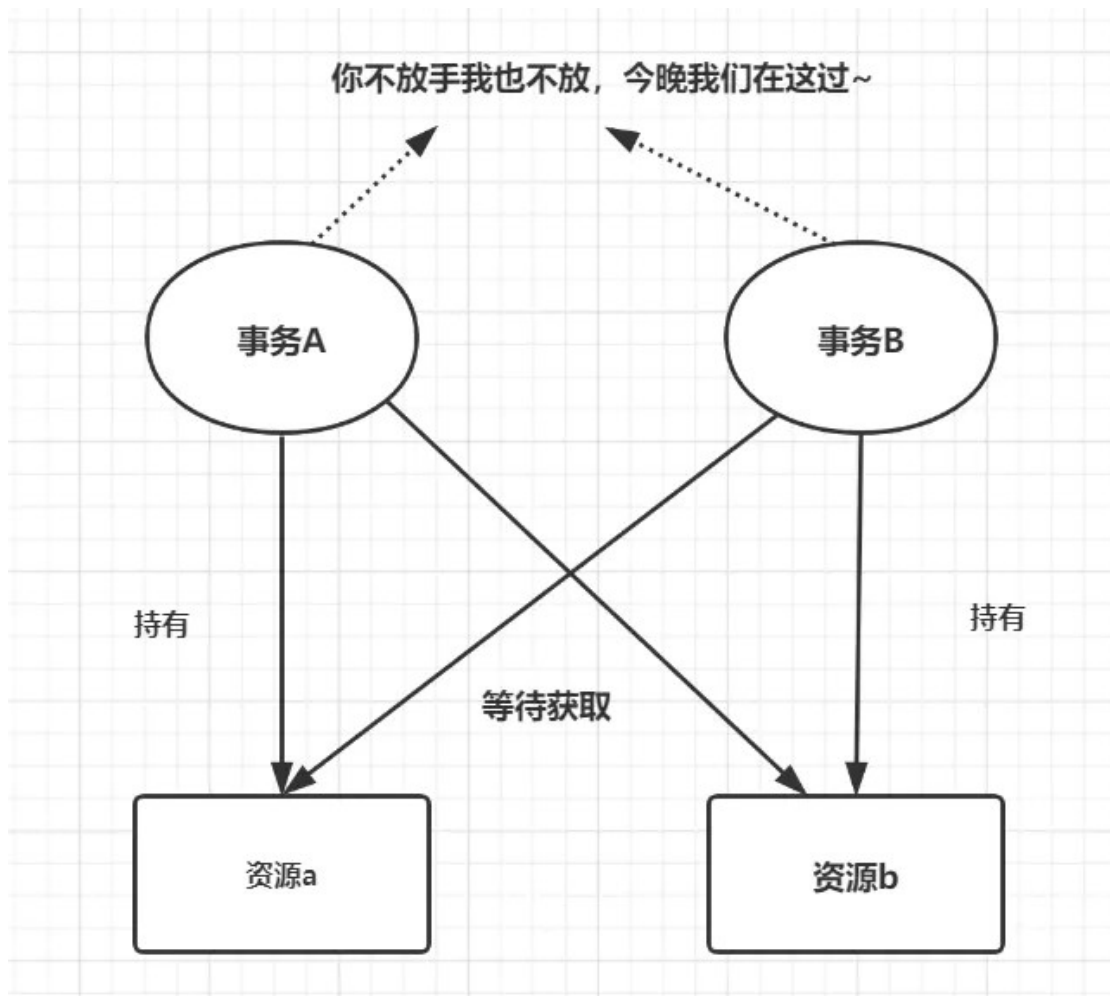
自增锁是一个表级别锁，那为什么会话 A 事务还没结束，事务 B 可以执行插入成功呢？不是应该锁表嘛？这是因为在参数 `innodb_autoinc_lock_mode` 上，这个参数设置为 1 的时候，相当于将这种 `auto_inc lock` 弱化为了一个更轻量级的互斥自增长机制去实现，官方称之为 `mutex`。

`innodb_autoinc_lock_mode` 还可以设置为 0 或者 2，

- **0**：表示传统锁模式，使用表级 `AUTO_INC` 锁。一个事务的 `INSERT-LIKE` 语句在语句执行结束后释放 `AUTO_INC` 表级锁，而不是在事务结束后释放。
 - **1**：连续锁模式，连续锁模式对于 `Simple inserts` 不会使用表级锁，而是使用一个轻量级锁来生成自增值，因为 InnoDB 可以提前直到插入多少行数据。自增值生成阶段使用轻量级互斥锁来生成所有的值，而不是一直加锁直到插入完成。对于 `bulk inserts` 类语句使用 `AUTO_INC` 表级锁直到语句完成。
 - **** 2****：交错锁模式，所有的 `INSERT-LIKE` 语句都不使用表级锁，而是使用轻量级互斥锁。
- **INSERT-LIKE**：指所有的插入语句，包括：INSERT、REPLACE、INSERT...SELECT、REPLACE...SELECT、LOAD DATA 等。
 - **Simple inserts**：指在插入前就能确定插入行数的语句，包括：INSERT、REPLACE，不包含 INSERT...ON DUPLICATE KEY UPDATE 这类语句。
 - **Bulk inserts**：指在插入前不能确定行数的语句，包括：INSERT ... SELECT/REPLACE ... SELECT/LOAD DATA。

1.50.2 什么是死锁？怎么解决？

死锁是指两个或多个事务在同一资源上相互占用，并请求锁定对方的资源，从而导致恶性循环的现象。看图形象一点，如下：



死锁有四个必要条件：互斥条件，请求和保持条件，环路等待条件，不剥夺条件。

解决死锁思路，一般就是切断环路，尽量避免并发形成环路。

- 如果不同程序会并发存取多个表，尽量约定以相同的顺序访问表，可以大大降低死锁机会。
- 在同一个事务中，尽可能做到一次锁定所需要的所有资源，减少死锁产生概率；
- 对于非常容易产生死锁的业务部分，可以尝试使用升级锁定颗粒度，通过表级锁定来减少死锁产生的概率；
- 如果业务处理不好可以用分布式事务锁或者使用乐观锁
- 死锁与索引密不可分，解决索引问题，需要合理优化你的索引，

有兴趣的朋友，可以看我的这两篇文章哈：

- [两万字详解！InnoDB 锁专题！](#)
- [手把手教你分析 Mysql 死锁问题](#)

1.51 count(1)、count(*) 与 count(列名) 的区别？

- count(*)包括了所有的列，相当于行数，在统计结果的时候，不会忽略列值为 NULL
- count(1)包括了忽略所有列，用 1 代表代码行，在统计结果的时候，不会忽略列值为 NULL
- count(列名)只包括列名那一列，在统计结果的时候，会忽略列值为空 (这里的空不是只空字符串或者 0，而是表示 null) 的计数，即某个字段值为 NULL 时，不统计。

1.52 什么是存储过程？有哪些优缺点？

存储过程，就是一些编译好了的 SQL 语句，这些 SQL 语句代码像一个方法一样实现一些功能（对单表或多表的增删改查），然后给这些代码块取一个名字，在用到这个功能的时候调用即可。

优点：

- 存储过程是一个预编译的代码块，执行效率比较高
- 存储过程在服务器端运行，减少客户端的压力
- 允许模块化程序设计，只需要创建一次过程，以后在程序中就可以调用该过程任意次，类似方法的复用
- 一个存储过程替代大量 T_SQL 语句，可以降低网络通信量，提高通信速率
- 可以一定程度上确保数据安全

缺点：

- 调试麻烦
- 可移植性不灵活
- 重新编译问题

1.53 SQL 约束有哪几种呢？

- NOT NULL: 约束字段的内容一定不能为 NULL。
- UNIQUE: 约束字段唯一性，一个表允许有多个 Unique 约束。
- PRIMARY KEY: 约束字段唯一，不可重复，一个表只允许存在一个。

- FOREIGN KEY: 用于预防破坏表之间连接的动作，也能防止非法数据插入外键。
- CHECK: 用于控制字段的值范围。

1.54 varchar(50)中 50 的涵义

- 字段最多存放 50 个字符
- 如 varchar(50) 和 varchar(200) 存储 "jay" 字符串所占空间是一样的，后者在排序时会消耗更多内存

1.55 mysql 中 int(20)和 char(20)以及 varchar(20)的区别

- int(20) 表示字段是 int 类型，显示长度是 20
- char(20)表示字段是固定长度字符串，长度为 20
- varchar(20) 表示字段是可变长度字符串，长度为 20

1.56 drop、delete 与 truncate 的区别

	delete	truncate	drop
类型	DML	DDL	DDL
回滚	可回滚	不可回滚	不可回滚
删除内容	表结构还在，删除表的全部或者一部分数据行	表结构还在，删除表中的所有数据	从数据库中删除表，所有的数据行，索引和权限也会被删除
删除速度	删除速度慢，逐行删除	删除速度快	删除速度最快

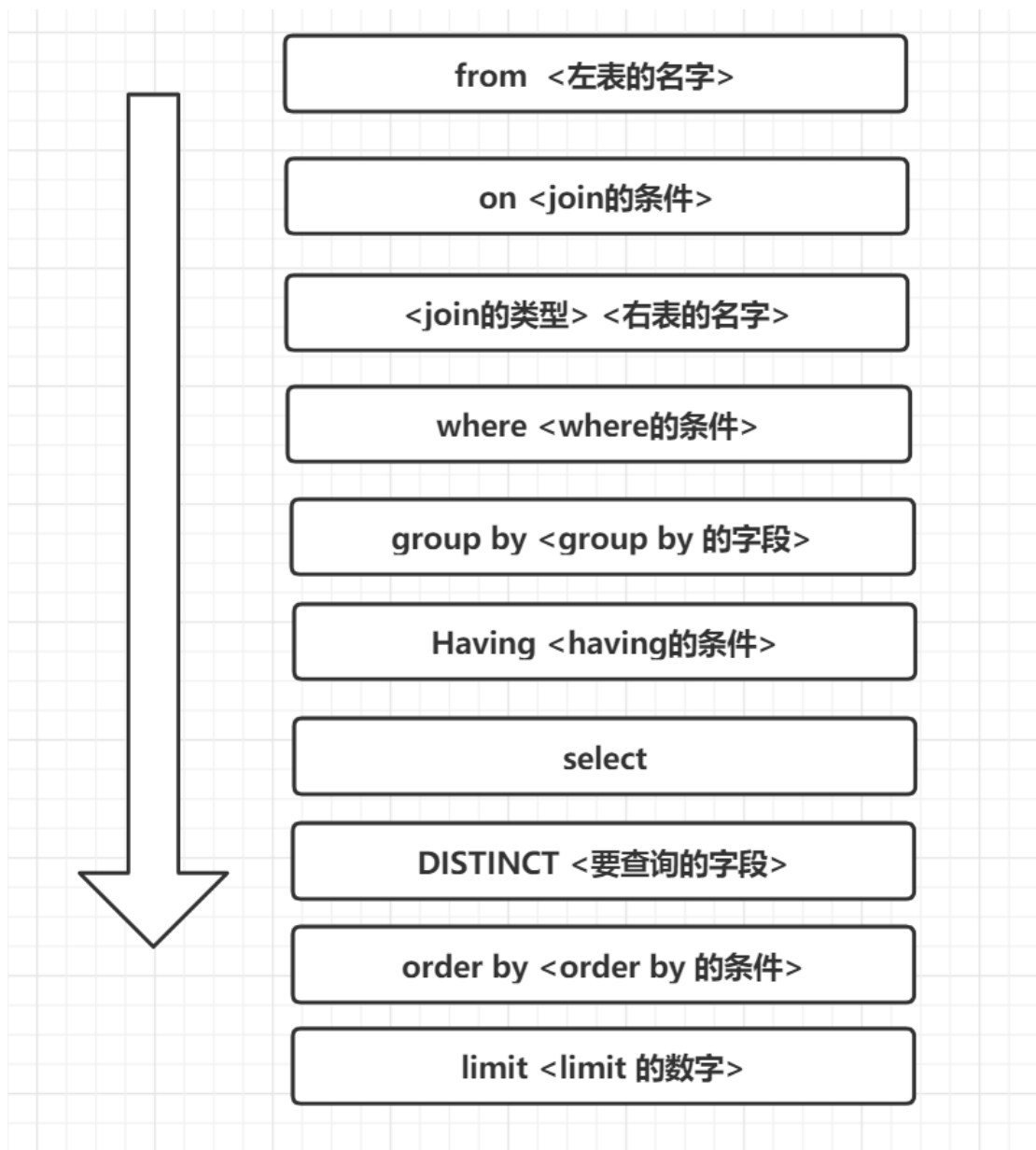
1.57 UNION 与 UNION ALL 的区别？

- Union：对两个结果集进行并集操作，不包括重复行，同时进行默认规则的排序；
- Union All：对两个结果集进行并集操作，包括重复行，不进行排序；
- UNION 的效率高于 UNION ALL

1.58 SQL 的生命周期？

- 服务器与数据库建立连接
- 数据库进程拿到请求 sql
- 解析并生成执行计划，执行
- 读取数据到内存，并进行逻辑处理
- 通过步骤一的连接，发送结果到客户端
- 关掉连接，释放资源

1.59 一条 Sql 的执行顺序？



1.60 关心过业务系统里面的 sql 耗时吗？统计过慢查询吗？

对慢查询都怎么优化过？

- 我们平时写 Sql 时，都要养成用 explain 分析的习惯。
- 慢查询的统计，运维会定期统计给我们

优化慢查询：

- 分析语句，是否加载了不必要的字段/数据。
- 分析 SQL 执行语句，是否命中索引等。
- 如果 SQL 很复杂，优化 SQL 结构
- 如果表数据量太大，考虑分表

可以看我这篇文章哈：[后端程序员必备：书写高质量 SQL 的 30 条建议](#)

1.61 主键使用自增 ID 还是 UUID，为什么？mysql 自增主键用完了怎么办？

主键使用自增 ID 还是 UUID？

如果是单机的话，选择自增 ID；如果是分布式系统，优先考虑 UUID 吧，但还是最好自己公司有一套分布式唯一 ID 生产方案吧。

- 自增 ID：数据存储空间小，查询效率高。但是如果数据量过大,会超出自增长的值范围，多库合并，也有可能有问题。
- uuid：适合大量数据的插入和更新操作，但是它无序的，插入数据效率慢，占用空间大。

mysql 自增主键用完了怎么办？

自增主键一般用 int 类型，一般达不到最大值，可以考虑提前分库分表的。

1.62 MySQL 数据库 cpu 飙升的话，要怎么处理呢？

排查过程：

- 使用 top 命令观察，确定是 mysqld 导致还是其他原因。
- 如果是 mysqld 导致的，show processlist，查看 session 情况，确定是不是有消耗资源的 sql 在运行。
- 找出消耗高的 sql，看看执行计划是否准确，索引是否缺失，数据量是否太大。

处理：

- kill 掉这些线程(同时观察 cpu 使用率是否下降)，
- 进行相应的调整(比如说加索引、改 sql、改内存参数)
- 重新跑这些 SQL。

其他情况：

也有可能是每个 sql 消耗资源并不多，但是突然之间，有大量的 session 连进来导致 cpu 飙升，这种情况就需要跟应用一起来分析为何连接数会激增，再做出相应的调整，比如说限制连接数等

1.63 读写分离常见方案？

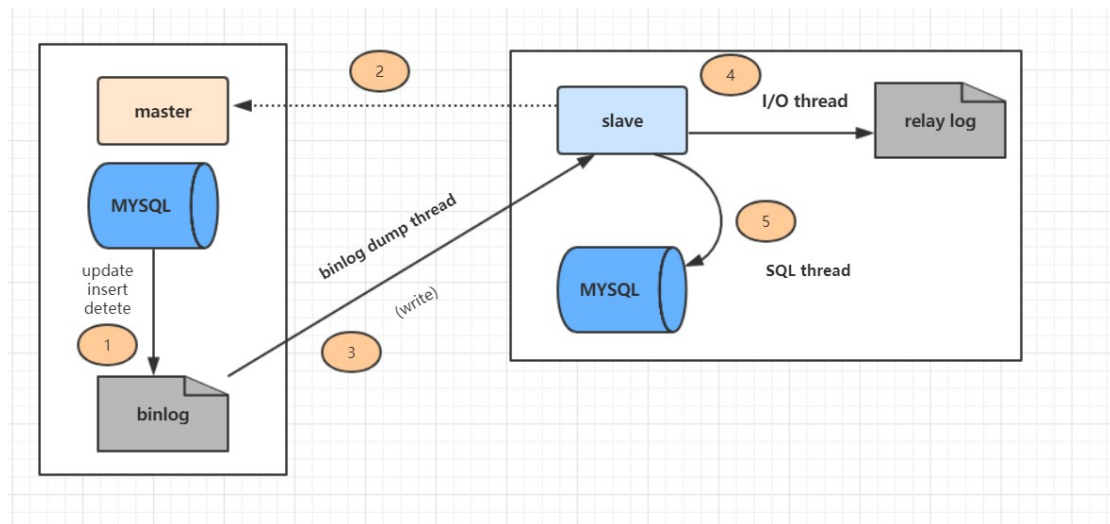
- 应用程序根据业务逻辑来判断，增删改等写操作命令发给主库，查询命令发给备库。
- 利用中间件来做代理，负责对数据库的请求识别出读还是写，并分发到不同的数据库中。（如：amoeba，mysql-proxy）

1.64 MySQL 的复制原理以及流程

主从复制原理，简言之，就三步曲，如下：

- 主数据库有个 bin-log 二进制文件，纪录了所有增删改 Sql 语句。（binlog 线程）
- 从数据库把主数据库的 bin-log 文件的 sql 语句复制过来。（io 线程）
- 从数据库的 relay-log 重做日志文件中再执行一次这些 sql 语句。（Sql 执行线程）

如下图所示：



上图主从复制分了五个步骤进行：

步骤一：主库的更新事件(update、insert、delete)被写到 binlog

步骤二：从库发起连接，连接到主库。

步骤三：此时主库创建一个 binlog dump thread，把 binlog 的内容发送到从库。

步骤四：从库启动之后，创建一个 I/O 线程，读取主库传过来的 binlog 内容并写入到 relay log

步骤五：还会创建一个 SQL 线程，从 relay log 里面读取内容，从

Exec_Master_Log_Pos 位置开始执行读取到的更新事件，将更新内容写入到 slave 的 db

我这篇文章写得比较详细，有兴趣可以看看，[面试必备：聊聊 MySQL 的主从](#)

1.65 Innodb 的事务实现原理？

事务就 ACID 四个特性，我觉得讲清楚这几个特点是怎么保证就可以了。

- 原子性：是使用 undo log 来实现的，如果事务执行过程中出错或者用户执行了 rollback，系统通过 undo log 日志返回事务开始的状态。
- 持久性：使用 redo log 来实现，只要 redo log 日志持久化了，当系统崩溃，即可通过 redo log 把数据恢复。
- 隔离性：通过锁以及 MVCC,使事务相互隔离开。
- 一致性：通过回滚、恢复，以及并发情况下的隔离性，从而实现一致性。

可以看我这两篇文章哈：

- [一文彻底读懂 MySQL 事务的四大隔离级别](#)
- [看一遍就理解，MVCC 原理详解](#)

1.66 谈谈 MySQL 的 Explain

Explain 执行计划包含字段信息如下：分别是

id、select_type、table、partitions、type、possible_keys、key、key_len、ref、rows、filtered、Extra 等 12 个字段。

我们重点关注的是 type，它的属性排序如下：

```
system > const > eq_ref > ref > ref_or_null >
index_merge > unique_subquery > index_subquery >
range > index > ALL
```

推荐大家看这篇文章哈：[面试官：不会看 Explain 执行计划，简历敢写 SQL 优化？](#)

1.67 Innodb 的事务与日志的实现方式

1.67.1 有多少种日志

innodb 两种日志 redo 和 undo。

1.67.2 日志的存放形式

- redo：在页修改的时候，先写到 redo log buffer 里面，然后写到 redo log 的文件系统缓存里面(fwrite)，然后再同步到磁盘文件 (fsync)。
- Undo：在 MySQL5.5 之前，undo 只能存放在 ibdata 文件里面，5.6 之后，可以通过设置 innodb_undo_tablespaces 参数把 undo log 存放在 ibdata 之外。

1.67.3 事务是如何通过日志来实现的

- 因为事务在修改页时，要先记 undo，在记 undo 之前要记 undo 的 redo，然后修改数据页，再记数据页修改的 redo。Redo (里面包括 undo 的修改) 一定要比数据页先持久化到磁盘。
- 当事务需要回滚时，因为有 undo，可以把数据页回滚到前镜像的状态，崩溃恢复时，如果 redo log 中事务没有对应的 commit 记录，那么需要用 undo 把该事务的修改回滚到事务开始之前。
- 如果有 commit 记录，就用 redo 前滚到该事务完成时并提交掉。

1.68 你们数据库是否支持 emoji 表情存储，如果不支持，如何操作？

更换字符集 utf8-->utf8mb4

1.69 一个 6 亿的表 a，一个 3 亿的表 b，通过外间 tid 关联，你如何最快的查询出满足条件的第 50000 到第 50200 中的这 200 条数据记录。

- 1、如果 A 表 TID 是自增长,并且是连续的,B 表的 ID 为索引 select * from a,b where a.tid = b.id and a.tid>500000 limit 200;
- 2、如果 A 表的 TID 不是连续的,那么就需要使用覆盖索引.TID 要么是主键,要么是辅助索引,B 表 ID 也需要有索引。select * from b , (select tid from a limit 50000,200) a where b.id = a .tid;

1.70 Mysql 一条 SQL 加锁分析

一条 SQL 加锁，可以分 9 种情况进行：

- 组合一：id 列是主键，RC 隔离级别
- 组合二：id 列是二级唯一索引，RC 隔离级别
- 组合三：id 列是二级非唯一索引，RC 隔离级别
- 组合四：id 列上没有索引，RC 隔离级别
- 组合五：id 列是主键，RR 隔离级别
- 组合六：id 列是二级唯一索引，RR 隔离级别
- 组合七：id 列是二级非唯一索引，RR 隔离级别
- 组合八：id 列上没有索引，RR 隔离级别
- 组合九：Serializable 隔离级别

可以看我这篇文章，有介绍到各种锁哈：[两万字详解！InnoDB 锁专题！](#)

2.Redis 专栏面试题（关注公众号：捡田螺的小男孩）

2.1 什么是 Redis？它主要用来什么的？

Redis，英文全称是 **Remote Dictionary Server**（远程字典服务），是一个开源的使用 ANSI C 语言编写、支持网络、可基于内存亦可持久化的日志型、Key-Value 数据库，并提供多种语言的 API。

与 MySQL 数据库不同的是，Redis 的数据是存在内存中的。它的读写速度非常快，每秒可以处理超过 10 万次读写操作。因此 redis 被广泛应用于缓存，另外，Redis 也经常用来做分布式锁。除此之外，Redis 支持事务、持久化、LUA 脚本、LRU 驱动事件、多种集群方案。

2.2 说说 Redis 的基本数据结构类型

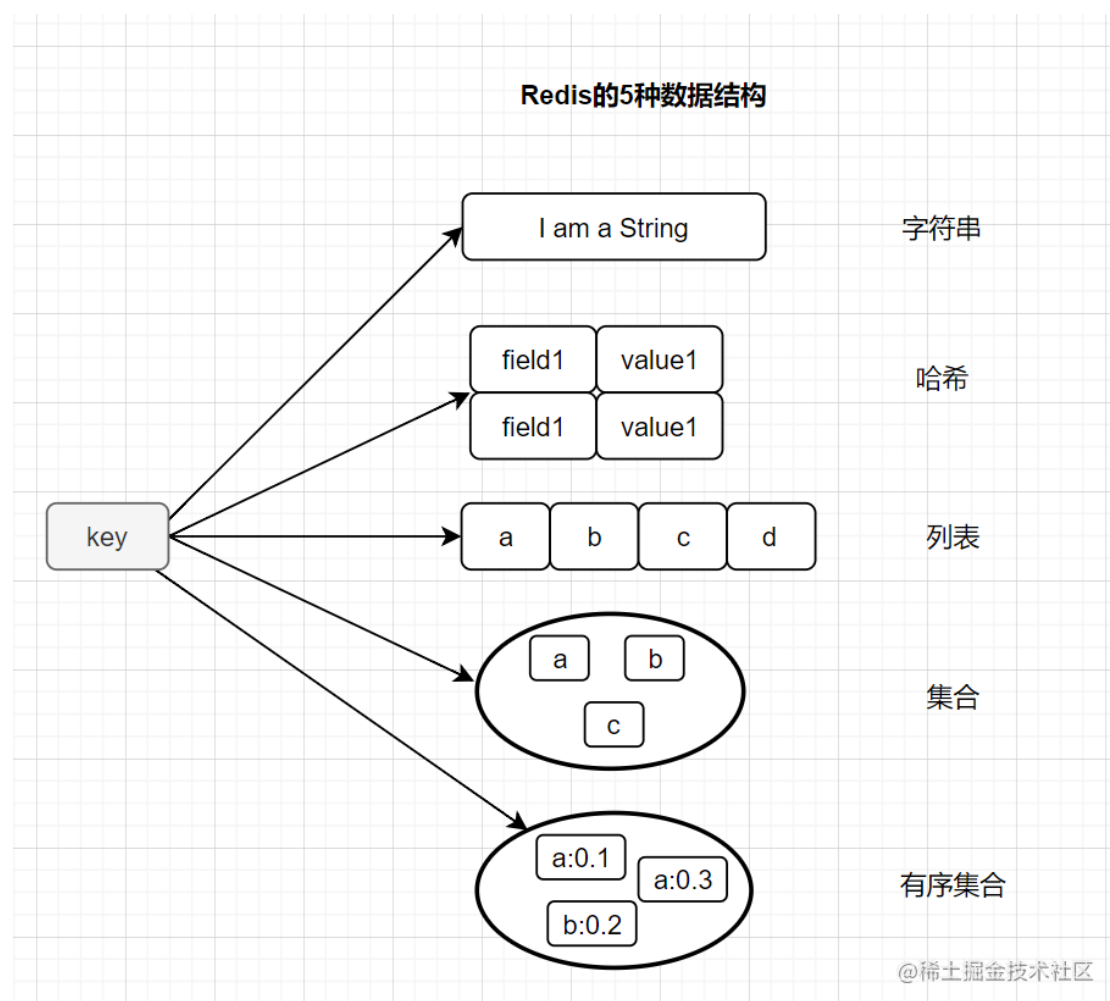
大多数小伙伴都知道，Redis 有以下这五种基本类型：

- String (字符串)
- Hash (哈希)
- List (列表)
- Set (集合)
- zset (有序集合)

它还有三种特殊的数据结构类型

- Geospatial
- Hyperloglog
- Bitmap

2.2.1 Redis 的五种基本数据类型



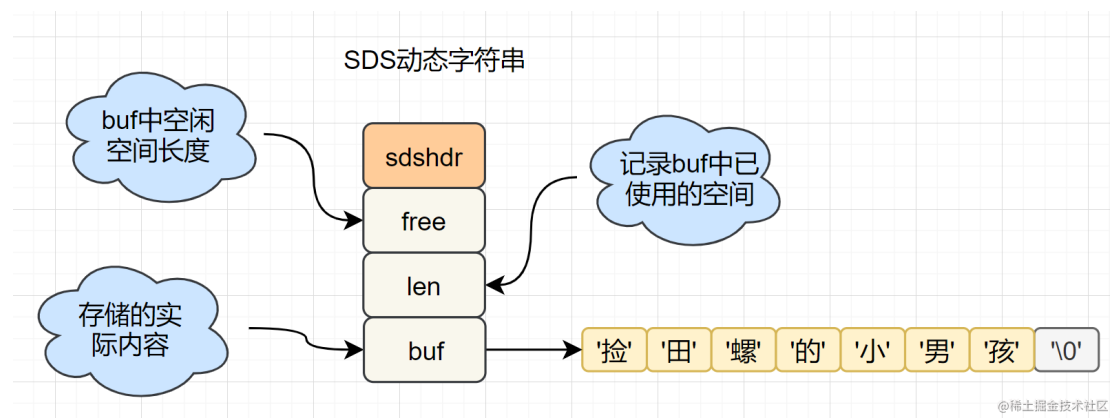
String (字符串)

- 简介:String 是 Redis 最基础的数据结构类型，它是二进制安全的，可以存储图片或者序列化的对象，值最大存储为 512M
- 简单使用举例: `set key value`、`get key` 等
- 应用场景：共享 session、分布式锁，计数器、限流。
- 内部编码有 3 种，`int` (8 字节长整型) / `embstr` (小于等于 39 字节字符串) / `raw` (大于 39 个字节字符串)

C 语言的字符串是 `char[]` 实现的，而 Redis 使用 **SDS (simple dynamic string)** 封装，sds 源码如下：

```
struct sdshdr{  
    unsigned int len; // 标记 buf 的长度  
    unsigned int free; // 标记 buf 中未使用的元素个数  
    char buf[]; // 存放元素的坑  
}
```

SDS 结构图如下：



Redis 为什么选择 **SDS** 结构，而 C 语言原生的 `char[]` 不香吗？

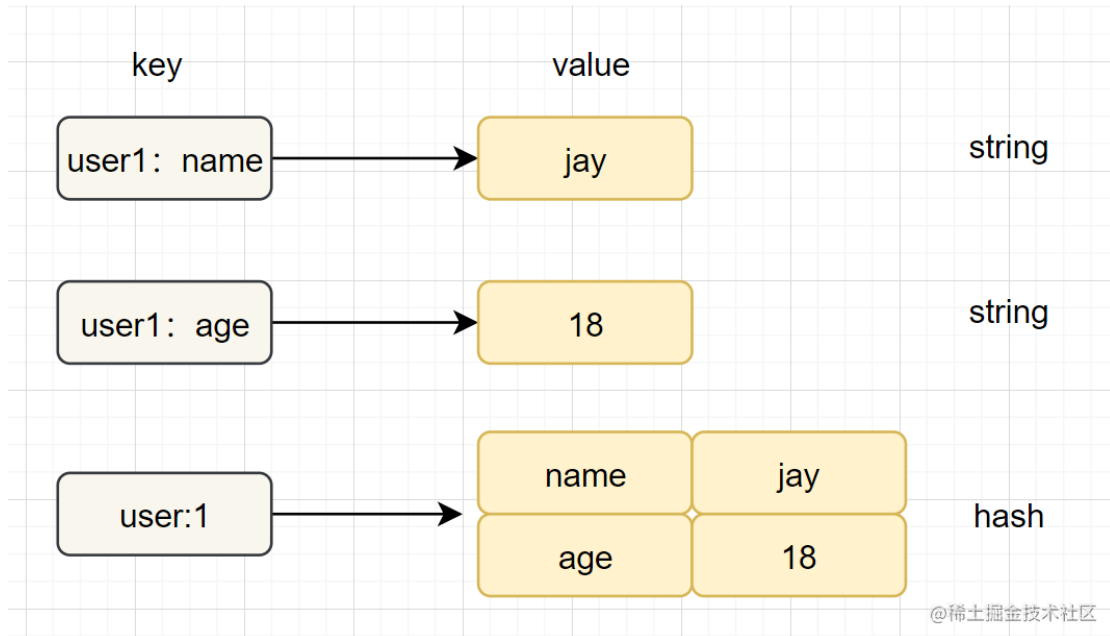
举例其中一点，SDS 中， $O(1)$ 时间复杂度，就可以获取字符串长度；而 C 字符串，需要遍历整个字符串，时间复杂度为 $O(n)$

Hash (哈希)

- 简介：在 Redis 中，哈希类型是指 `v ( 值 )` 本身又是一个键值对 (`k-v`) 结构
- 简单使用举例：`hset key field value`、`hget key field`
- 内部编码：`ziplist` (压缩列表)、`hashtable` (哈希表)

- 应用场景：缓存用户信息等。
- 注意点：如果开发使用 hgetall，哈希元素较多的话，可能导致 Redis 阻塞，可以使用 hscan。而如果只是获取部分 field，建议使用 hmget。

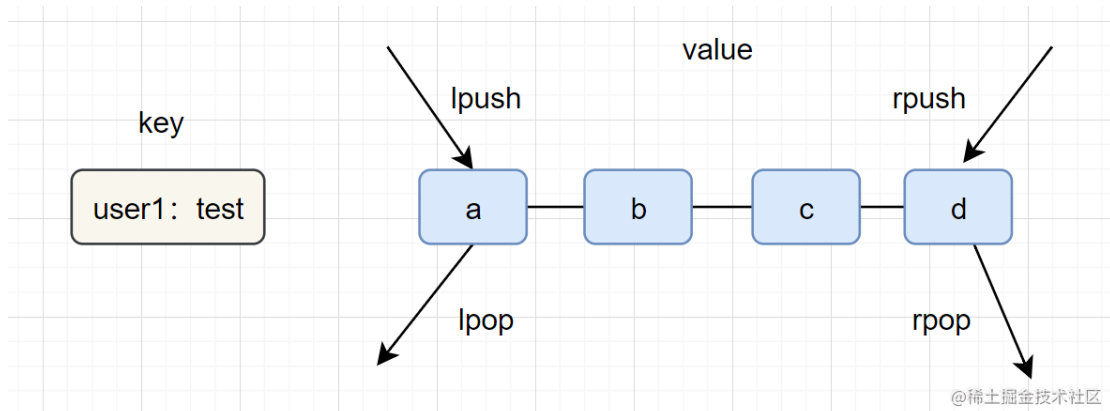
字符串和哈希类型对比如下图：



List (列表)

- 简介：列表 (list) 类型是用来存储多个有序的字符串，一个列表最多可以存储 $2^{32}-1$ 个元素。
- 简单实用举例：`lpush key value [value ...]`、`lrange key start end`
- 内部编码：ziplist (压缩列表)、linkedlist (链表)
- 应用场景：消息队列，文章列表，

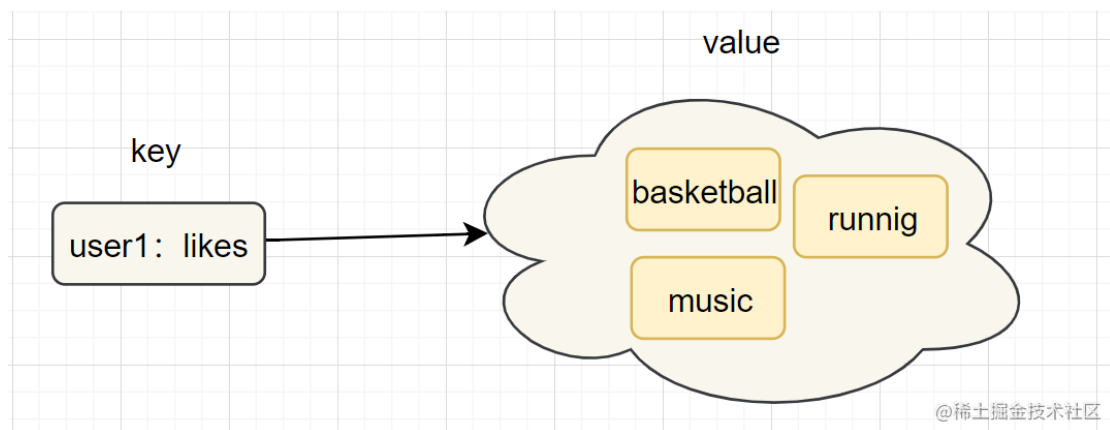
一图看懂 list 类型的插入与弹出：



list 应用场景参考以下：

- lpush+lpop=Stack (栈)
- lpush+rpop=Queue (队列)
- lpush+ltrim=Capped Collection (有限集合)
- lpush+brpop=Message Queue (消息队列)

Set (集合)



- 简介：集合 (set) 类型也是用来保存多个的字符串元素，但是不允许重复元素
- 简单使用举例：`sadd key element [element ...]`、`smembers key`
- 内部编码：`intset` (整数集合)、`hashtable` (哈希表)
- 注意点：`smembers` 和 `lrange`、`hgetall` 都属于比较重的命令，如果元素过多存在阻塞 Redis 的可能性，可以使用 `sscan` 来完成。
- 应用场景：用户标签,生成随机数抽奖、社交需求。

有序集合 (zset)

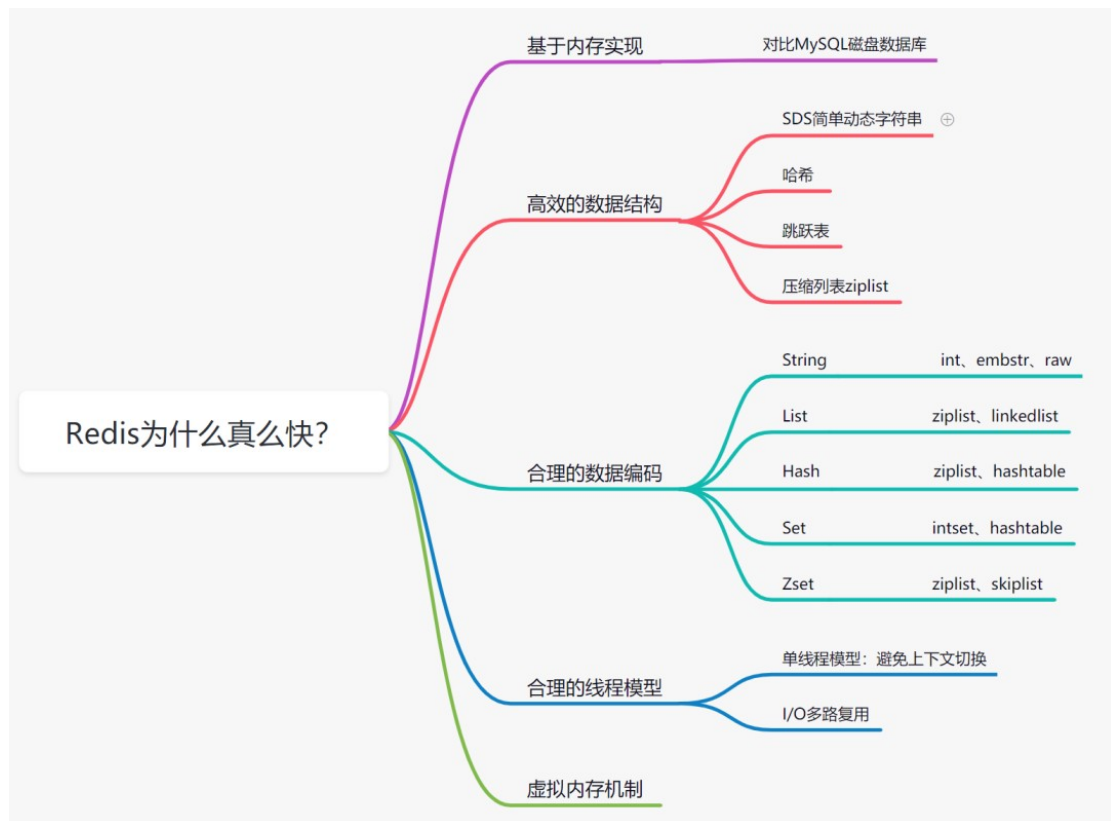
- 简介：已排序的字符串集合，同时元素不能重复
- 简单格式举例：`zadd key score member [score member ...]`，`zrank key member`
- 底层内部编码：`ziplist` (压缩列表)、`skiplist` (跳跃表)
- 应用场景：排行榜，社交需求 (如用户点赞)。

2.2.2 Redis 的三种特殊数据类型

- Geo：Redis3.2 推出的，地理位置定位，用于存储地理位置信息，并对存储的信息进行操作。
- HyperLogLog：用来做基数统计算法的数据结构，如统计网站的 UV。

- Bitmaps : 用一个比特位来映射某个元素的状态，在 Redis 中，它的底层是基于字符串类型实现的，可以把 bitmaps 成作一个以比特位为单位的数组

2.3 Redis 为什么这么快？



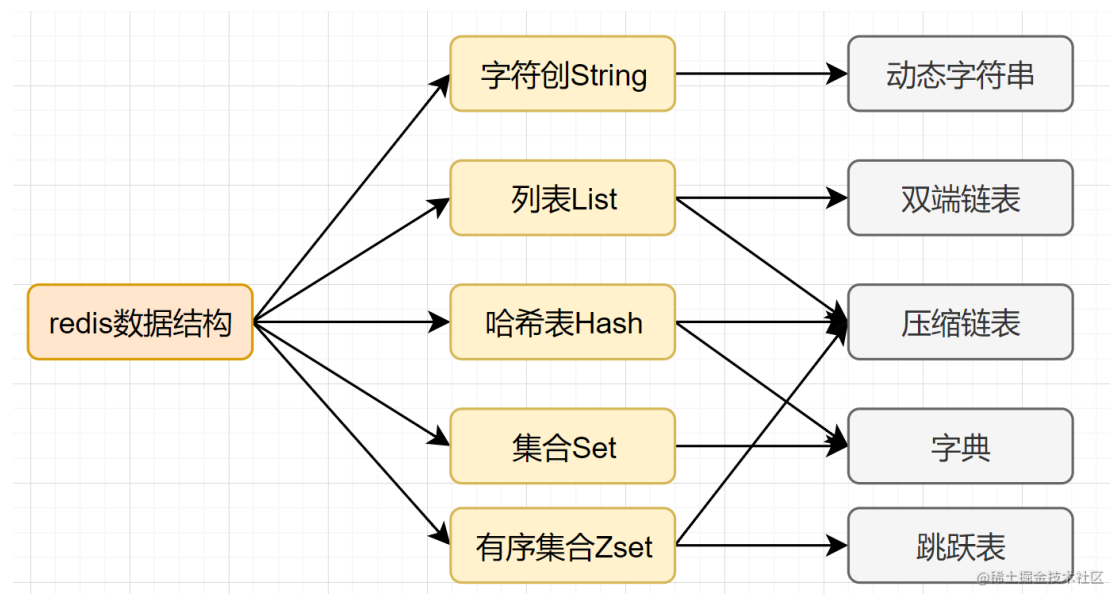
Redis 为什么这么快

2.3.1 基于内存存储实现

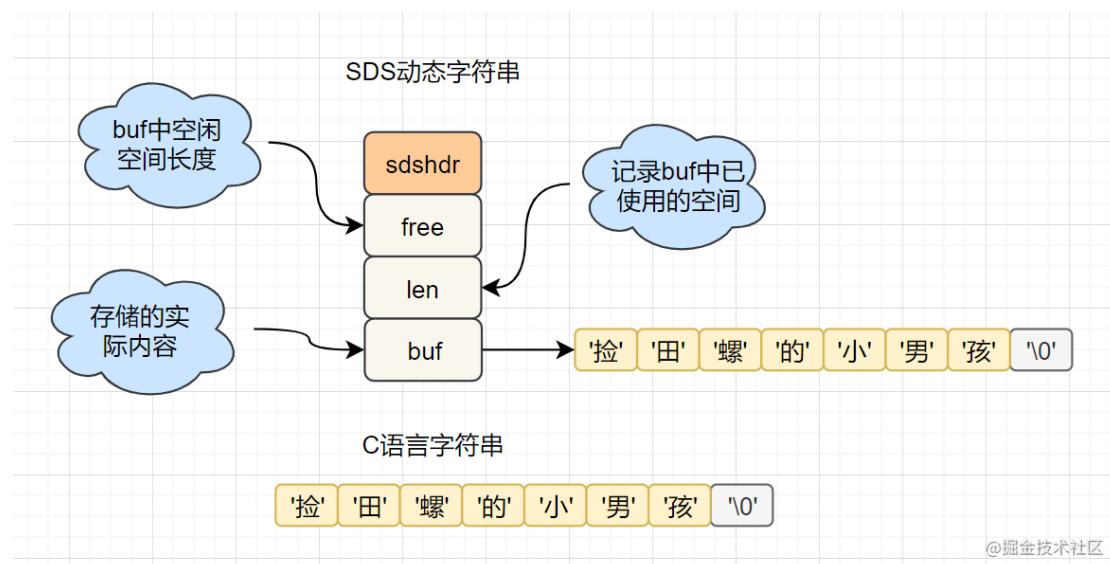
我们都知道内存读写是比在磁盘快很多的，Redis 基于内存存储实现的数据库，相对于数据存在磁盘的 MySQL 数据库，省去磁盘 I/O 的消耗。

2.3.2 高效的数据结构

我们知道，Mysql 索引为了提高效率，选择了 B+树的数据结构。其实合理的数据结构，就是可以让你的应用/程序更快。先看下 Redis 的数据结构&内部编码图：



SDS 简单动态字符串



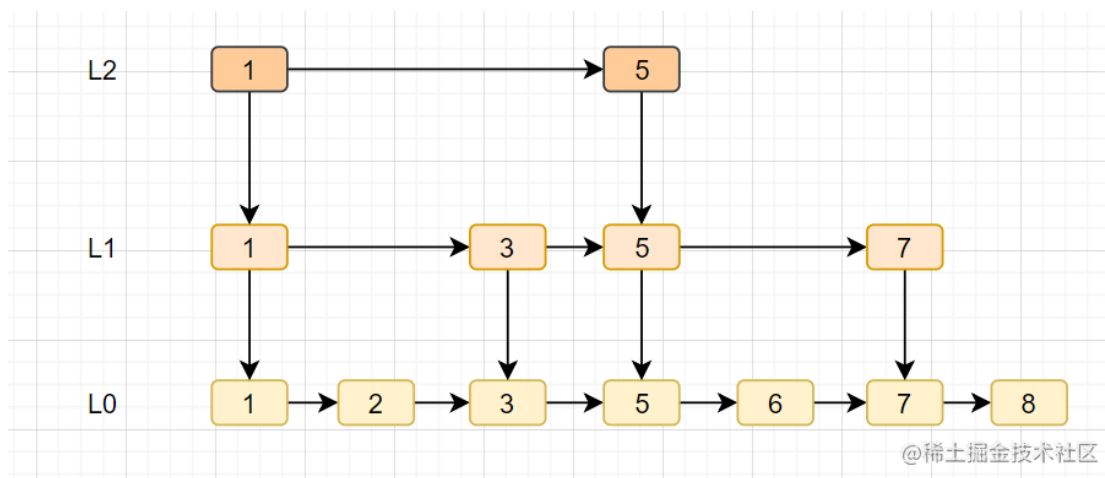
- 字符串长度处理：Redis 获取字符串长度，时间复杂度为 $O(1)$ ，而 C 语言中，需要从头开始遍历，复杂度为 $O(n)$ ；
- 空间预分配：字符串修改越频繁的话，内存分配越频繁，就会消耗性能，而 SDS 修改和空间扩充，会额外分配未使用的空间，减少性能损耗。
- 惰性空间释放：SDS 缩短时，不是回收多余的内存空间，而是 free 记录下多余的空间，后续有变更，直接使用 free 中记录的空间，减少分配。

- 二进制安全：Redis 可以存储一些二进制数据，在 C 语言中字符串遇到'\0'会结束，而 SDS 中标志字符串结束的是 len 属性。

字典

Redis 作为 K-V 型内存数据库，所有的键值就是用字典来存储。字典就是哈希表，比如 HashMap，通过 key 就可以直接获取到对应的 value。而哈希表的特性，在 $O(1)$ 时间复杂度就可以获得对应的值。

跳跃表



- 跳跃表是 Redis 特有的数据结构，就是在链表的基础上，增加多级索引提升查找效率。
- 跳跃表支持平均 $O(\log N)$ ，最坏 $O(N)$ 复杂度的节点查找，还可以通过顺序性操作批量处理节点。

2.3.3 合理的数据编码

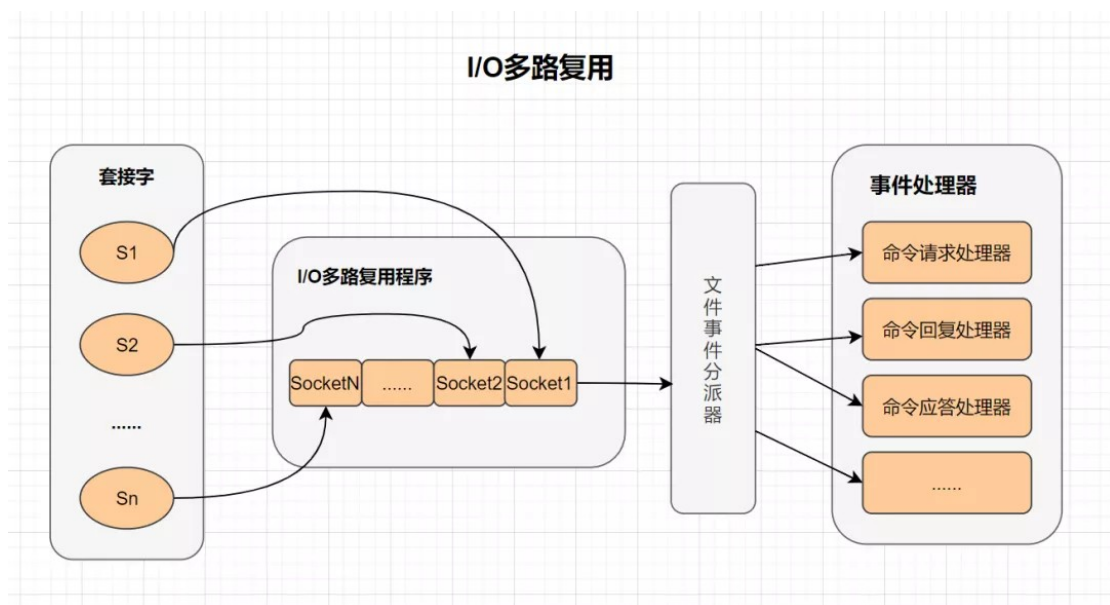
Redis 支持多种数据类型，每种基本类型，可能对多种数据结构。什么时候，使用什么样数据结构，使用什么样编码，是 redis 设计者总结优化的结果。

- String：如果存储数字的话，是用 int 类型的编码；如果存储非数字，小于等于 39 字节的字符串，是 embstr；大于 39 个字节，则是 raw 编码。
- List：如果列表的元素个数小于 512 个，列表每个元素的值都小于 64 字节（默认），使用 ziplist 编码，否则使用 linkedlist 编码

- Hash：哈希类型元素个数小于 512 个，所有值小于 64 字节的话，使用 ziplist 编码,否则使用 hashtable 编码。
- Set：如果集合中的元素都是整数且元素个数小于 512 个，使用 intset 编码，否则使用 hashtable 编码。
- Zset：当有序集合的元素个数小于 128 个，每个元素的值小于 64 字节时，使用 ziplist 编码，否则使用 skiplist (跳跃表) 编码

2.3.4 合理的线程模型

I/O 多路复用



I/O 多路复用

多路 I/O 复用技术可以让单个线程高效的处理多个连接请求，而 Redis 使用用 epoll 作为 I/O 多路复用技术的实现。并且，Redis 自身的事件处理模型将 epoll 中的连接、读写、关闭都转换为事件，不在网络 I/O 上浪费过多的时间。

什么是 I/O 多路复用？

- I/O：网络 I/O
- 多路：多个网络连接
- 复用：复用同一个线程。
- IO 多路复用其实就是一种同步 IO 模型，它实现了一个线程可以监视多个文件句柄；一旦某个文件句柄就绪，就能够通知应用程序进行相应的读写操作；而没有文件句柄就绪时,就会阻塞应用程序，交出 cpu。

单线程模型

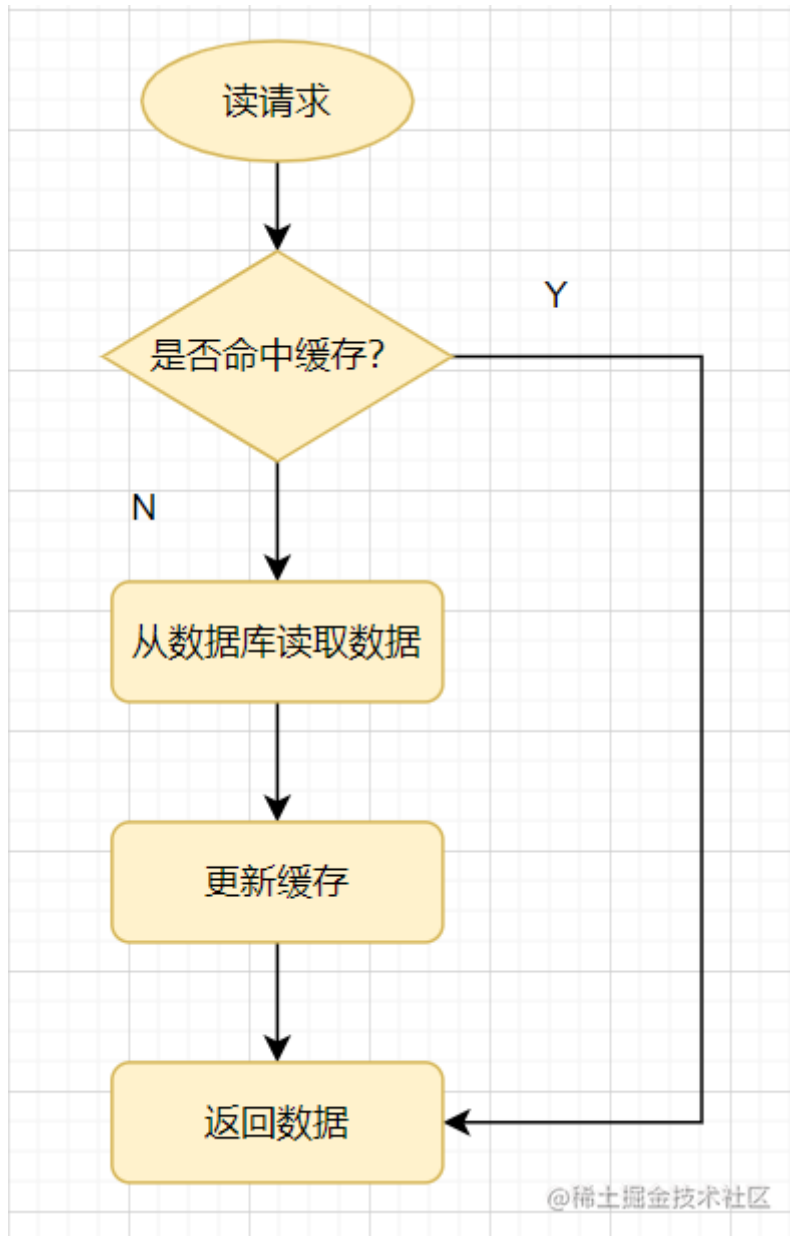
- Redis 是单线程模型的，而单线程避免了 CPU 不必要的上下文切换和竞争锁的消耗。也正因为是单线程，如果某个命令执行过长（如 hgetall 命令），会造成阻塞。Redis 是面向快速执行场景的数据库，所以要慎用如 smembers 和 lrange、hgetall 等命令。
- Redis 6.0 引入了多线程提速，它的执行命令操作内存的仍然是个单线程。

大家可以看下我的这篇文章哈，[大厂必问：Redis 为什么这么快](#)

2.4 什么是缓存击穿、缓存穿透、缓存雪崩？

2.4.1 缓存穿透问题

先来看一个常见的缓存使用方式：读请求来了，先查下缓存，缓存有值命中，就直接返回；缓存没命中，就去查数据库，然后把数据库的值更新到缓存，再返回。



读取缓存

缓存穿透：指查询一个一定不存在的数据，由于缓存是不命中时需要从数据库查询，查不到数据则不写入缓存，这将导致这个不存在的数据每次请求都要到数据库去查询，进而给数据库带来压力。

通俗点说，读请求访问时，缓存和数据库都没有某个值，这样就会导致每次对这个值的查询请求都会穿透到数据库，这就是缓存穿透。

缓存穿透一般都是这几种情况产生的：

- **业务不合理的设计**，比如大多数用户都没开守护，但是你的每个请求都去缓存，查询某个 userid 查询有没有守护。
- **业务/运维/开发失误的操作**，比如缓存和数据库的数据都被误删除了。
- **黑客非法请求攻击**，比如黑客故意捏造大量非法请求，以读取不存在的业务数据。

如何避免缓存穿透呢？ 一般有三种方法。

- 1.如果是非法请求，我们在 API 入口，对参数进行校验，过滤非法值。
- 2.如果查询数据库为空，我们可以给缓存设置个空值，或者默认值。但是如有有写请求进来的话，需要更新缓存哈，以保证缓存一致性，同时，最后给缓存设置适当的过期时间。（业务上比较常用，简单有效）
- 3.使用布隆过滤器快速判断数据是否存在。即一个查询请求过来时，先通过布隆过滤器判断值是否存在，存在才继续往下查。

布隆过滤器原理：它由初始值为 0 的位图数组和 N 个哈希函数组成。一个对一个 key 进行 N 个 hash 算法获取 N 个值，在比特数组中将这 N 个值散列后设定为 1，然后查的时候如果特定的这几个位置都为 1，那么布隆过滤器判断该 key 存在。

2.4.2 缓存雪崩问题

缓存雪崩：指缓存中数据大批量到过期时间，而查询数据量巨大，请求都直接访问数据库，引起数据库压力过大甚至 down 机。

- 缓存雪崩一般是由于大量数据同时过期造成的，对于这个原因，可通过均匀设置过期时间解决，即让过期时间相对离散一点。如采用一个较大固定值+一个较小的随机值，5 小时+0 到 1800 秒酱紫。
- Redis 故障宕机也可能引起缓存雪崩。这就需要构造 Redis 高可用集群啦。

2.4.3 缓存击穿问题

缓存击穿：指热点 key 在某个时间点过期的时候，而恰好在这个时间点对这个 Key 有大量的并发请求过来，从而大量的请求打到 db。

缓存击穿看着有点像，其实它两区别是，缓存雪崩是指数据库压力过大甚至 down 机，缓存击穿只是大量并发请求到了 DB 数据库层面。可以认为击穿是缓存雪崩的一个子集吧。有些文章认为它俩区别，是区别在于击穿针对某一热点 key 缓存，雪崩则是很多 key。

解决方案就有两种：

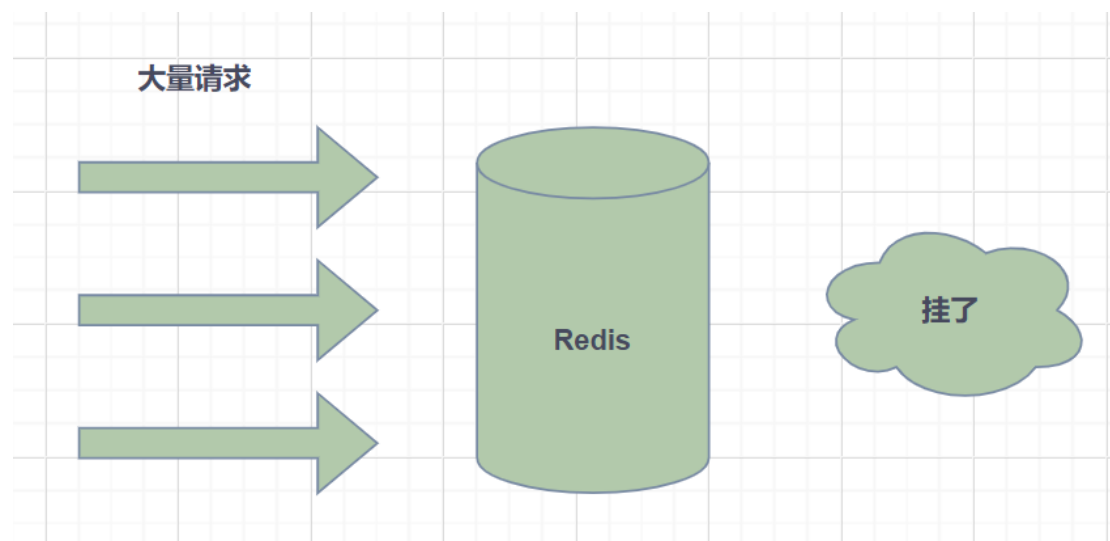
- **1.使用互斥锁方案。**缓存失效时，不是立即去加载 db 数据，而是先使用某些带成功返回的原子操作命令，如(Redis 的 setnx) 去操作，成功的时候，再去加载 db 数据库数据和设置缓存。否则就去重试获取缓存。
- **2. “永不过期”，**是指没有设置过期时间，但是热点数据快要过期时，异步线程去更新和设置过期时间。

我这篇做了详细介绍，[经典面试题：聊聊缓存击穿、缓存穿透、缓存雪崩](#)

2.5 什么是热 Key 问题，如何解决热 key 问题

什么是热 Key 呢？在 Redis 中，我们把访问频率高的 key，称为热点 key。

如果某一热点 key 的请求到服务器主机时，由于请求量特别大，可能会导致主机资源不足，甚至宕机，从而影响正常的服务。



而热点 Key 是怎么产生的呢？主要原因有两个：

- 用户消费的数据远大于生产的数据，如秒杀、热点新闻等读多写少的场景。
- 请求分片集中，超过单 Redi 服务器的性能，比如固定名称 key，Hash 落入同一台服务器，瞬间访问量极大，超过机器瓶颈，产生热点 Key 问题。

那么在日常开发中，如何识别到热点 key 呢？

- 凭经验判断哪些是热 Key；
- 客户端统计上报；
- 服务代理层上报

如何解决热 key 问题？

- Redis 集群扩容：增加分片副本，均衡读流量；
- 将热 key 分散到不同的服务器中；
- 使用二级缓存，即 JVM 本地缓存,减少 Redis 的读请求。

2.6. Redis 过期策略和内存淘汰策略



2.6.1 Redis 的过期策略

我们在 `set key` 的时候，可以给它设置一个过期时间，比如 `expire key 60`。指定这 key 60s 后过期，60s 后，redis 是如何处理的嘛？我们先来介绍几种过期策略：

定时过期

每个设置过期时间的 key 都需要创建一个定时器，到过期时间就会立即对 key 进行清除。该策略可以立即清除过期的数据，对内存很友好；但是会占用大量的 CPU 资源去处理过期的数据，从而影响缓存的响应时间和吞吐量。

惰性过期

只有当访问一个 key 时，才会判断该 key 是否已过期，过期则清除。该策略可以最大化地节省 CPU 资源，却对内存非常不友好。极端情况可能出现大量的过期 key 没有再次被访问，从而不会被清除，占用大量内存。

定期过期

每隔一定的时间，会扫描一定数量的数据库的 `expires` 字典中一定数量的 key，并清除其中已过期的 key。该策略是前两者的一个折中方案。通过调整定时扫描的时间间隔和每次扫描的限定耗时，可以在不同情况下使得 CPU 和内存资源达到最优的平衡效果。

`expires` 字典会保存所有设置了过期时间的 key 的过期时间数据，其中，key 是指向键空间中的某个键的指针，value 是该键的毫秒精度的 UNIX 时间戳表示的过期时间。键空间是指该 Redis 集群中保存的所有键。

Redis 中同时使用了 **惰性过期**和**定期过期**两种过期策略。

- 假设 Redis 当前存放 30 万个 key，并且都设置了过期时间，如果你每隔 100ms 就去检查这全部的 key，CPU 负载会特别高，最后可能会挂掉。

- 因此，redis 采取的是定期过期，每隔 100ms 就随机抽取一定数量的 key 来检查和删除的。
- 但是呢，最后可能会有很多已经过期的 key 没被删除。这时候，redis 采用惰性删除。在你获取某个 key 的时候，redis 会检查一下，这个 key 如果设置了过期时间并且已经过期了，此时就会删除。

但是呀，如果定期删除漏掉了很多过期的 key，然后也没走惰性删除。就会有
很多过期 key 积在内存内存，直接会导致内存爆的。或者有些时候，业务量大
起来了，redis 的 key 被大量使用，内存直接不够了，运维小哥哥也忘记加大
内存了。难道 redis 直接这样挂掉？不会的！Redis 用 8 种内存淘汰策略保护
自己~

2.6.2 Redis 内存淘汰策略

- volatile-lru：当内存不足以容纳新写入数据时，从设置了过期时间的 key 中使用 LRU（最近最少使用）算法进行淘汰；
- allkeys-lru：当内存不足以容纳新写入数据时，从所有 key 中使用 LRU（最近最少使用）算法进行淘汰。
- volatile-lfu：4.0 版本新增，当内存不足以容纳新写入数据时，在过期的 key 中，使用 LFU 算法进行删除 key。
- allkeys-lfu：4.0 版本新增，当内存不足以容纳新写入数据时，从所有 key 中使用 LFU 算法进行淘汰；
- volatile-random：当内存不足以容纳新写入数据时，从设置了过期时间的 key 中，随机淘汰数据；。
- allkeys-random：当内存不足以容纳新写入数据时，从所有 key 中随机淘汰数据。
- volatile-ttl：当内存不足以容纳新写入数据时，在设置了过期时间的 key 中，根据过期时间进行淘汰，越早过期的优先被淘汰；
- noeviction：默认策略，当内存不足以容纳新写入数据时，新写入操作会报错。

2.7 说说 Redis 的常用应用场景

- 缓存
- 排行榜
- 计数器应用
- 共享 Session

- 分布式锁
- 社交网络
- 消息队列
- 位操作

2.7.1 缓存

我们一提到 redis，自然而然就想到缓存，国内外中大型的网站都离不开缓存。

合理的利用缓存，比如缓存热点数据，不仅可以提升网站的访问速度，还可以降低数据库 DB 的压力。并且，Redis 相比于 memcached，还提供了丰富的数据结构，并且提供 RDB 和 AOF 等持久化机制，强的一批。

2.7.2 排行榜

当今互联网应用，有各种各样的排行榜，如电商网站的月度销量排行榜、社交 APP 的礼物排行榜、小程序的投票排行榜等等。Redis 提供的 **zset** 数据类型能够实现这些复杂的排行榜。

比如，用户每天上传视频，获得点赞的排行榜可以这样设计：

- 1. 用户 Jay 上传一个视频，获得 6 个赞，可以酱紫：

```
zadd user:ranking:2021-03-03 Jay 3
```

-

- 2. 过了一段时间，再获得一个赞，可以这样：

```
zincrby user:ranking:2021-03-03 Jay 1
```

-

- 3. 如果某个用户 John 作弊，需要删除该用户：

```
zrem user:ranking:2021-03-03 John
```

-

3. 展示获取赞数最多的 3 个用户

```
zrevrangebyrank user:ranking:2021-03-03 0 2
```

2.7.3 计数器应用

各大网站、APP 应用经常需要计数器的功能，如短视频的播放数、电商网站的浏览数。这些播放数、浏览数一般要求实时的，每一次播放和浏览都要做加 1 的操作，如果并发量很大对于传统关系型数据的性能是一种挑战。Redis 天然支持计数功能而且计数的性能也非常好，可以说是计数器系统的重要选择。

2.7.4 共享 Session

如果一个分布式 Web 服务将用户的 Session 信息保存在各自服务器，用户刷新一次可能就需要重新登录了，这样显然有问题。实际上，可以使用 Redis 将用户的 Session 进行集中管理，每次用户更新或者查询登录信息都直接从 Redis 中集中获取。

2.7.5 分布式锁

几乎每个互联网公司中都使用了分布式部署，分布式服务下，就会遇到对同一个资源的并发访问的技术难题，如秒杀、下单减库存等场景。

- 用 synchronize 或者 reentrantlock 本地锁肯定是不行的。
- 如果是并发量不大话，使用数据库的悲观锁、乐观锁来实现没啥问题。
- 但是在并发量高的场合中，利用数据库锁来控制资源的并发访问，会影响数据库的性能。
- 实际上，可以用 Redis 的 setnx 来实现分布式的锁。

2.7.6 社交网络

赞/踩、粉丝、共同好友/喜好、推送、下拉刷新等是社交网站的必备功能，由于社交网站访问量通常比较大，而且传统的关系型数据不太适合保存这种类型的数据，Redis 提供的数据结构可以相对比较容易地实现这些功能。

2.7.7 消息队列

消息队列是大型网站必用中间件，如 ActiveMQ、RabbitMQ、Kafka 等流行的消息队列中间件，主要用于业务解耦、流量削峰及异步处理实时性低的业务。Redis 提供了发布/订阅及阻塞队列功能，能实现一个简单的消息队列系统。另外，这个不能和专业的消息中间件相比。

2.7.8 位操作

用于数据量上亿的场景下，例如几亿用户系统的签到，去重登录次数统计，某用户是否在线状态等等。腾讯 10 亿用户，要几个毫秒内查询到某个用户是否在线，能怎么做？千万别说给每个用户建立一个 key，然后挨个记（你可以算一下需要的内存会很恐怖，而且这种类似的需求很多。这里要用到位操作——使用 setbit、getbit、bitcount 命令。原理是：redis 内构建一个足够长的数组，每个数组元素只能是 0 和 1 两个值，然后这个数组的下标 index 用来表示用户 id（必须是数字哈），那么很显然，这个几亿长的大数组就能通过下标和元素值（0 和 1）来构建一个记忆系统。

大家可以看下这篇文章哈，[一口气说出 Redis 16 个常见使用场景](#)

2.8 Redis 持久化有哪几种方式，怎么选？

2.8.1 AOF

Redis 是基于内存的，如果 Redis 服务器挂了，数据就会丢失。为了避免数据丢失了，Redis 提供了两种持久化方式，RDB 和 AOF。我们先来介绍 AOF。

AOF (append only file) 持久化，采用日志的形式来记录每个写操作，追加到 AOF 文件的末尾。Redis 默认情况是不开启 AOF 的。重启时再重新执行 AOF 文件中的命令来恢复数据。它主要解决数据持久化的实时性问题。

AOF 是执行完命令后才记录日志的。为什么不先记录日志再执行命令呢？这是因为 Redis 在向 AOF 记录日志时，不会先对这些命令进行语法检查，如果先记录日志再执行命令，日志中可能记录了错误的命令，Redis 使用日志回复数据时，可能会出错。

正是因为执行完命令后才记录日志，所以不会阻塞当前的写操作。但是会存在两个风险：

- 更执行完命令还没记录日志时，宕机了会导致数据丢失
- AOF 不会阻塞当前命令，但是可能会阻塞下一个操作。

这两个风险最好的解决方案是折中妙用 AOF 机制的三种写回策略

appendfsync：

- always，同步写回，每个子命令执行完，都立即将日志写回磁盘。
- everysec，每个命令执行完，只是先把日志写到 AOF 内存缓冲区，每隔一秒同步到磁盘。
- no：只是先把日志写到 AOF 内存缓冲区，有操作系统去决定何时写入磁盘。

always 同步写回，可以基本保证数据不丢失，**no** 策略则性能高但是数据可能会丢失，一般可以考虑折中选择 **everysec**。

如果接受的命令越来越多，**AOF** 文件也会越来越大，文件过大还是会带来性能问题。日志文件过大怎么办呢？**AOF 重写机制**！就是随着时间推移，**AOF** 文件会有一些冗余的命令如：无效命令、过期数据的命令等等，**AOF** 重写机制就是把它们合并为一个命令（类似批处理命令），从而达到精简压缩空间的目的。

AOF 重写会阻塞嘛？**AOF** 日志是由主线程会写的，而重写则不一样，重写过程是由后台子进程 **bgrewriteaof** 完成。

- **AOF** 的优点：数据的一致性和完整性更高，秒级数据丢失。
- 缺点：相同的数据集，**AOF** 文件体积大于 **RDB** 文件。数据恢复也比较慢。

2.8.2 RDB

因为 **AOF** 持久化方式，如果操作日志非常多的话，**Redis** 恢复就很慢。有没有在宕机快速恢复的方法呢，有的，**RDB**！

RDB，就是把内存数据以快照的形式保存到磁盘上。和 **AOF** 相比，它记录的是某一时刻的数据，并不是操作。

什么是快照？可以这样理解，给当前时刻的数据，拍一张照片，然后保存下来。

RDB 持久化，是指在指定的时间间隔内，执行指定次数的写操作，将内存中的数据集快照写入磁盘中，它是 **Redis** 默认的持久化方式。执行完操作后，在指定目录下会生成一个 **dump.rdb** 文件，**Redis** 重启的时候，通过加载 **dump.rdb** 文件来恢复数据。**RDB** 触发机制主要有以下几种：



RDB 通过 **bgsave** 命令的执行全量快照，可以避免阻塞主线程。**save** 命令会 **fork** 一个子进程，然后该子进程会负责创建 **RDB** 文件，而服务器进程会继续处理命令请求

快照时，数据能修改嘛？Redis 接住操作系统的写时复制技术 (**copy-on-write** , **COW**) ,在执行快照的同时，正常处理写操作。

虽然 **bgsave** 执行不会阻塞主线程，但是频繁执行全量快照也会带来性能开销。

比如 **bgsave** 子进程需要通过 **fork** 操作从主线程创建出来，创建后不会阻塞主线程，但是创建过程是会阻塞主线程的。可以做增量快照。

- **RDB** 的优点：与 **AOF** 相比，恢复大数据集的时候会更快，它适合大规模的数据恢复场景，如备份，全量复制等
- 缺点：没办法做到实时持久化/秒级持久化。

Redis4.0 开始支持 **RDB** 和 **AOF** 的混合持久化，就是内存快照以一定频率执行，两次快照之间，再使用 **AOF** 记录这期间的所有命令操作。

2.8.3 如何选择 **RDB** 和 **AOF**

- 如果数据不能丢失，**RDB** 和 **AOF** 混用
- 如果只作为缓存使用，可以承受几分钟的数据丢失的话，可以只使用 **RDB**。
- 如果只使用 **AOF**，优先使用 **everysec** 的写回策略。

我之前的这篇文章做了详细介绍：[经典面试题：Redis 持久化有哪几种方式，怎么选？](#)

2.9 怎么实现 Redis 的高可用？(Redis 主从、哨兵、集群)

我们在项目中使用 Redis，肯定不会是单点部署 Redis 服务的。因为，单点部署一旦宕机，就不可用了。为了实现高可用，通常的做法是，将数据库复制多个副本以部署在不同的服务器上，其中一台挂了也可以继续提供服务。Redis 实现高可用有三种部署模式：**主从模式**，**哨兵模式**，**集群模式**。

2.9.1 Redis 主从

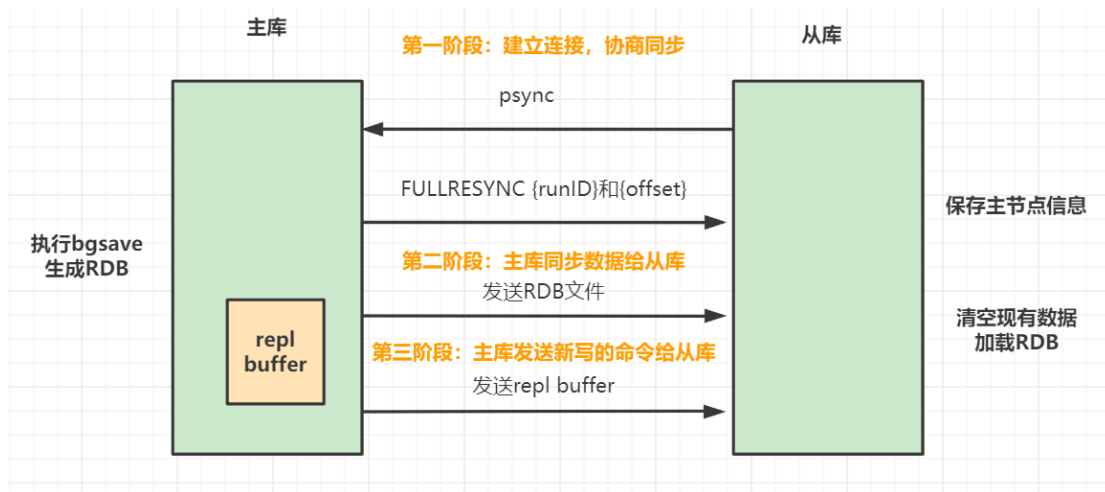
面试官经常会问到 Redis 的高可用。Redis 高可用回答包括两个层面，一个就是**数据不能丢失，或者说尽量减少丢失**；另外一个就是保证 **Redis 服务不中断**。

- 对于尽量减少数据丢失，可以通过 AOF 和 RDB 保证。
- 对于保证服务不中断的话，Redis 就不能单点部署，这时候我们先看下 Redis 主从。

2.9.1.1 Redis 主从概念

- Redis 主从模式，就是部署多台 Redis 服务器，有主库和从库，它们之间通过主从复制，以保证数据副本的一致。
- 主从库之间采用的是**读写分离**的方式，其中主库负责读操作和写操作，从库则负责读操作。
- 如果 Redis 主库挂了，切换其中的从库成为主库。

2.9.1.2 Redis 主从同步过程



Redis 主从同步包括三个阶段。

第一阶段：主从库间建立连接、协商同步。

- 从库向主库发送 `psync` 命令，告诉它要进行数据同步。
- 主库收到 `psync` 命令后，响应 `FULLRESYNC` 命令（它表示第一次复制采用的是**全量复制**），并带上主库 `runID` 和主库目前的复制进度 `offset`。

第二阶段：主库把数据同步到从库，从库收到数据后，完成本地加载。

- 主库执行 `bgsave` 命令，生成 `RDB` 文件，接着将文件发给从库。从库接收到 `RDB` 文件后，会先清空当前数据库，然后加载 `RDB` 文件。
- 主库把数据同步到从库的过程中，新来的写操作，会记录到 `replication buffer`。

第三阶段，主库把新写的命令，发送到从库。

- 主库完成 `RDB` 发送后，会把 `replication buffer` 中的修改操作发给从库，从库再重新执行这些操作。这样主从库就实现同步啦。

2.9.1.3 Redis 主从的一些注意点

- 主从数据不一致

因为主从复制是异步进行的，如果从库滞后执行，则会导致主从数据不一致。

主从数据不一致一般有两个原因：

- 主从库网路延迟。
- 从库收到了主从命令，但是它正在执行阻塞性的命令（如 `hgetall` 等）。

如何解决主从数据不一致问题呢？

1. 可以换更好的硬件配置，保证网络畅通。
2. 监控主从库间的复制进度

- **读取过期数据**

Redis 删除数据有这几种策略：

- 惰性删除：只有当访问一个 key 时，才会判断该 key 是否已过期，过期则清除。
- 定期删除：每隔一定的时间，会扫描一定数量的数据库的 expires 字典中一定数量的 key，并清除其中已过期的 key。
- 主动删除：当前已用内存超过最大限时，触发主动清理策略。

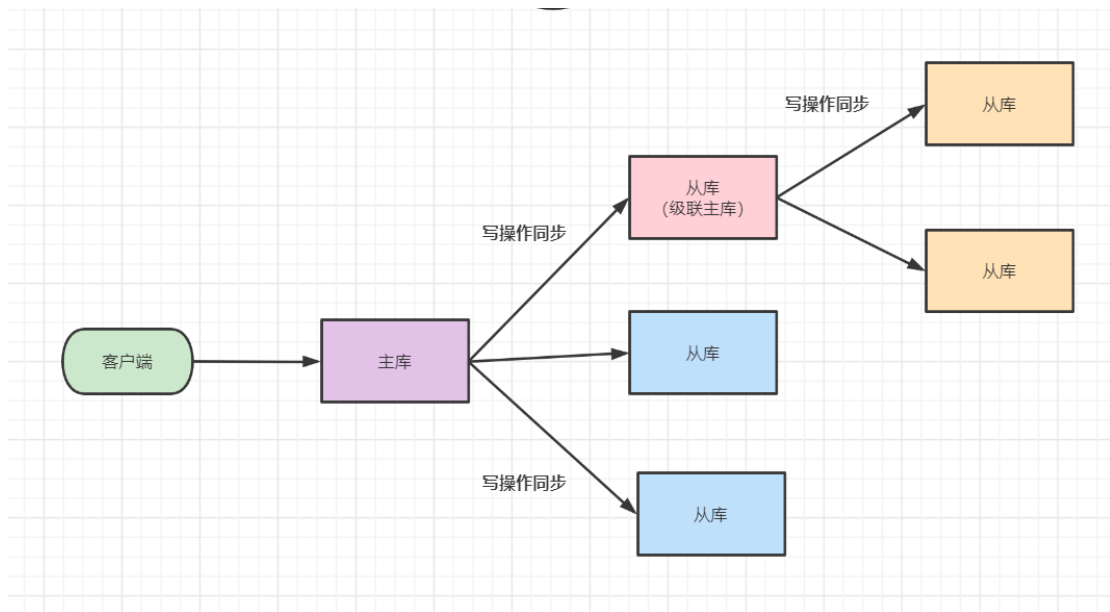
如果使用 Redis 版本低于 3.2，读从库时，并不会判断数据是否过期，而是会**返回过期数据**。而 3.2 版本后，Redis 做了改进，如果读到的数据已经过期了，从库不会删除，却会返回空值，**避免了客户端读到过期数据**。

因此，在主从 Redis 模式下，尽量使用 **Redis 3.2** 以上的版本。

- **一主多从，全量复制时主库压力问题**

如果是一主多从模式，从库很多的时候，如果每个从库都要和主库进行全量复制的话，主库的压力是很大的。因为主库 fork 进程生成 RDB，这个 fork 的过程是会阻塞主线程处理正常请求的。同时，传输大的 RDB 文件也会占用主库的网络宽带。

可以使用**主-从-从**模式解决。什么是主从从模式呢？其实就是部署主从集群时，选择硬件网络配置比较好的一个从库，让它跟**部分从库再建立主从关系**。如图：



- 主从网络断了怎么办呢？

主从库完成了全量复制后，它们之间会维护一个网络长连接，用于主库后续收到写命令传输到从库，它可以避免频繁建立连接的开销。但是，如果网络断开重连后，是否还需要进行一次全量复制呢？

如果是 Redis 2.8 之前，从库和主库重连后，确实会再进行一次全量复制，但是这样开销就很大。而 Redis 2.8 之后做了优化，重连后采用增量复制方式，即把主从库网络断连期间主库收到的写命令，同步给从库。

主从库重连后，就是利用 **repl_backlog_buffer** 实现增量复制。

当主从库断开连接后，主库会把断连期间收到的写操作命令，写入 **replication buffer**，同时也会把这些操作命令写入 **repl_backlog_buffer** 这个缓冲区。repl_backlog_buffer 是一个环形缓冲区，主库会记录自己写到的位置，从库则会记录自己已经读到的位置。

2.9.2 Redis 哨兵

主从模式中，一旦主节点由于故障不能提供服务，需要人工将从节点晋升为主节点，同时还要通知应用方更新主节点地址。显然，多数业务场景都不能接受这种故障处理方式。Redis 从 2.8 开始正式提供了 **Redis 哨兵机制**来解决这个问题。

- 哨兵作用
- 哨兵模式简介
- 哨兵如何判定主库下线
- 哨兵模式如何工作
- 哨兵是如何选主的
- 由哪个哨兵执行主从切换呢？
- 哨兵下的故障转移

2.9.2.1 哨兵作用

哨兵其实是一个运行在特殊模式下的 Redis 进程。它有三个作用，分别是：**监控、自动选主切换（简称选主）、通知。**

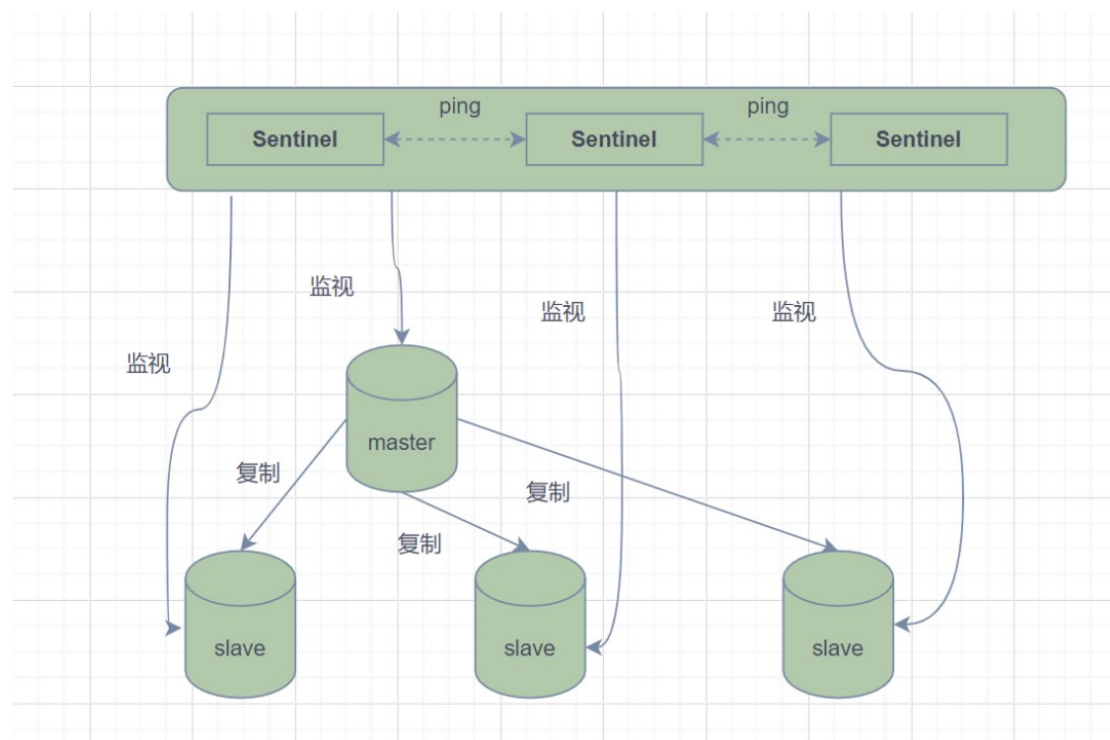
哨兵进程在运行期间，监视所有的 Redis 主节点和从节点。它通过周期性给**主从库**发送 **PING** 命令，检测主从库是否挂了。如果**从库**没有在规定时间内响应哨兵的 **PING** 命令，哨兵就会把它标记为**下线状态**；如果主库没有在规定时间内响应哨兵的 **PING** 命令，哨兵则会判定主库下线，然后开始切换到**选主**任务。

所谓**选主**，其实就是从多个从库中，按照一定规则，选出一个当做主库。至于**通知**呢，就是选出主库后，哨兵把新主库的连接信息发给其他从库，让它们和新主库建立主从关系。同时，哨兵也会把新主库的连接信息通知给客户端，让它们把请求操作发到新主库上。

2.9.2.2 哨兵模式

因为 Redis 哨兵也是一个 Redis 进程，如果它自己挂了呢，那是不是就起不了监控的作用啦。我们一起来看看 Redis 哨兵模式

哨兵模式，就是由一个或多个哨兵实例组成的哨兵系统，它可以监视所有的 Redis 主节点和从节点，并在被监视的主节点进入下线状态时，自动将下线主服务器属下的某个从节点升级为新的主节点。，一个哨兵进程对 Redis 节点进行监控，就可能会出现单点问题（单点问题）。因此，一般使用多个哨兵来进行监控 Redis 节点，并且各个哨兵之间还会进行监控。



其实哨兵之间是通过发布订阅机制组成集群的，同时，哨兵又通过 `INFO` 命令，获得了从库连接信息，也能和从库建立连接，从而进行监控。

2.9.2.3 哨兵如何判定主库下线

哨兵是如何判断主库是否下线的呢？我们先来了解两个基础概念哈：**主观下线**和**客观下线**。

- 哨兵进程向**主库、从库**发送 PING 命令，如果主库或者从库没有在规定时间内响应 PING 命令，哨兵就把它标记为**主观下线**。
- 如果是主库被标记为**主观下线**，则正在监视这个主库的**所有哨兵**要以每秒一次的频率，以确认主库是否真的进入了**主观下线**。当有多数的哨兵（**一般少数服从多数，由 Redis 管理员自行设定的一个值**）在指定的时间范围内确认主库的确进入了主观下线状态，则主库会被标记为**客观下线**。这样做的目的就是**避免对主库的误判**，以减少没有必要的主从切换，减少不必要的开销。

假设我们有 N 个哨兵实例，如果有 $N/2+1$ 个实例判断主库**主观下线**，此时就可以把节点标记为**客观下线**，就可以做主从切换了。

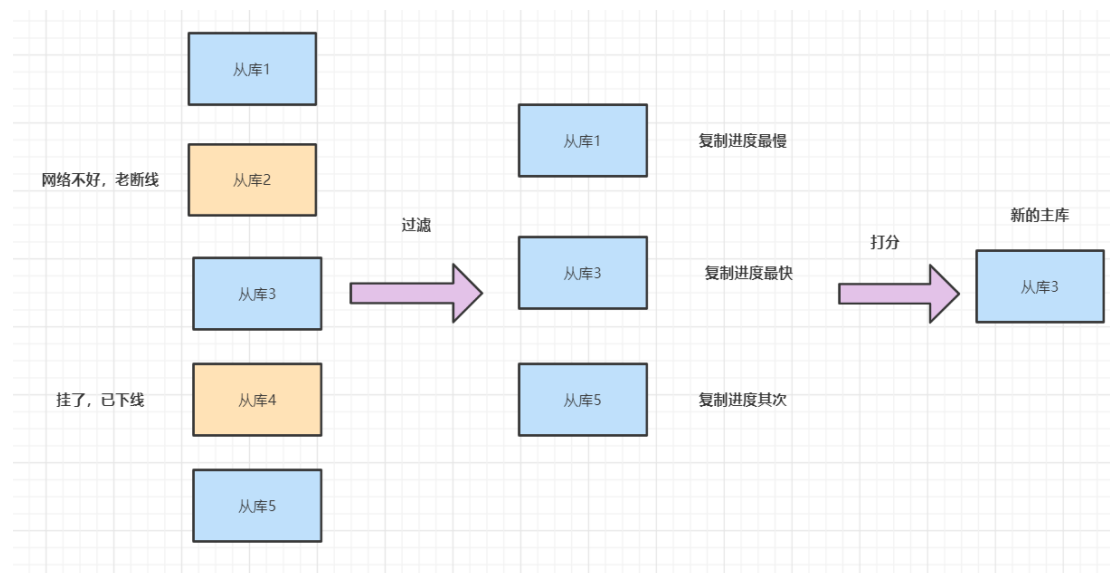
2.9.2.4 哨兵的工作模式

1. 每个哨兵以每秒钟一次的频率向它所知的主库、从库以及其他哨兵实例发送一个 **PING** 命令。
2. 如果一个实例节点距离最后一次有效回复 **PING** 命令的时间超过 **down-after-milliseconds** 选项所指定的值，则这个实例会被哨兵标记为主观下线。
3. 如果**主库**被标记为主观下线，则正在监视这个主库的所有哨兵要以每秒一次的频率确认主库的确进入了主观下线状态。
4. 当有足够数量的哨兵（**大于等于配置文件指定的值**）在指定的时间范围内确认主库的确进入了主观下线状态，则主库会被标记为**客观下线**。
5. 当主库被哨兵标记为**客观下线**时，就会进入**选主模式**。
6. 若没有足够数量的哨兵同意主库已经进入主观下线，主库的**主观下线状态就会被移除**；若主库重新向哨兵的 **PING** 命令返回有效回复，主库的主观下线状态就会被移除。

2.9.2.5 哨兵是如何选主的？

如果明确主库已经客观下线了，哨兵就开始了选主模式。

哨兵选主包括两大过程，分别是：**过滤和打分**。其实就是在多个从库中，先按照一定的筛选条件，把不符合条件的从库**过滤**掉。然后再按照一定的规则，给剩下的从库逐个打分，将得分最高的从库选为新主库



- 选主时，会判断从库的状态，如果已经下线，就**直接过滤**。
- 如果从库网络不好，老是超时，也会被过滤掉。看这个参数 `down-after-milliseconds`，它表示我们认定主从库断连的最大连接超时时间。
- 过滤掉了不适合做主库的从库后，就可以给剩下的从库打分，按这三个规则打分：**从库优先级、从库复制进度以及从库 ID 号**。
- 从库优先级最高的话，打分就越高，优先级可以通过 `slave-priority` 配置。如果优先级一样，就选与旧的主库复制进度最快的从库。如果优先级和从库进度都一样，从库 ID 号小的打分高。

2.9.2.6 由哪个哨兵执行主从切换呢？

一个哨兵标记主库为**主观下线**后，它会征求其他哨兵的意见，确认主库是否的确进入了主观下线状态。它向其他实例哨兵发送 `is-master-down-by-addr` 命令。其他哨兵会根据自己和主库的连接情况，回应 **Y** 或 **N**（Y 表示赞成，N 表示反对票）。如果这个哨兵获得足够多的赞成票数（`quorum` 配置），主库会被标记为**客观下线**。

标记主库客观下线的这个哨兵，紧接着向其他哨兵发送命令，再发起**投票**，希望它可以来执行主从切换。这个投票过程称为 **Leader 选举**。因为最终执行主从切换的哨兵称为 Leader，投票过程就是确定 Leader。一个哨兵想成为 Leader 需要满足两个条件：

- 需要拿到 $\text{num}(\text{sentinels})/2+1$ 的赞成票。
- 并且拿到的票数需要大于等于哨兵配置文件中的 **quorum** 值。

举个例子，假设有 3 个哨兵。配置的 quorum 值为 2。即一个一个哨兵想成为 Leader 至少需要拿到 2 张票。为了更好理解，大家可以看下

时间	哨兵 (A1)	哨兵 (A2)	哨兵(A3)
t1	给自己投1票，向A2， A3发送投票请求，表示想成为Leader		
t2			给自己投1票，向A1， A2发送投票请求，表示也想成为 Leader
t3	收到A3的投票请求，回复N，因为t1时已经投给自己了		
t4		收到A3的投票请求，回复Y，因为前面还没投过票	
t5		收到A1的投票请求，回复N，因为t4已经投给A3了	
t6	总票数， 1Y， 1N		总票数2Y， 可以成为Leader

- 在 t1 时刻，哨兵 A1 判断主库为**客观下线**，它想成为主从切换的 Leader，于是先给自己投一张赞成票，然后分别向哨兵 A2 和 A3 发起投票命令，表示想成为 Leader。
- 在 t2 时刻，A3 判断主库为**客观下线**，它也想成为 Leader，所以也先给自己投一张赞成票，再分别向 A1 和 A2 发起投票命令，表示也要成为 Leader。
- 在 t3 时刻，哨兵 A1 收到了 A3 的 Leader 投票请求。因为 A1 已经把票 Y 投给自己了，所以它不能再给其他哨兵投赞成票了，所以 A1 投票 **N** 给 A3。
- 在 t4 时刻，哨兵 A2 收到 A3 的 Leader 投票请求，因为哨兵 A2 之前没有投过票，它会给第一个向它发送投票请求的哨兵回复赞成票 **Y**。
- 在 t5 时刻，哨兵 A2 收到 A1 的 Leader 投票请求，因为哨兵 A2 之前已经投过赞成票给 A3 了，所以它只能给 A1 投反对票 **N**。
- 最后 t6 时刻，哨兵 A1 只收到自己的一票 **Y** 赞成票，而哨兵 A3 得到两张赞成票（A2 和 A3 投的），因此**哨兵 A3 成为了 Leader**。

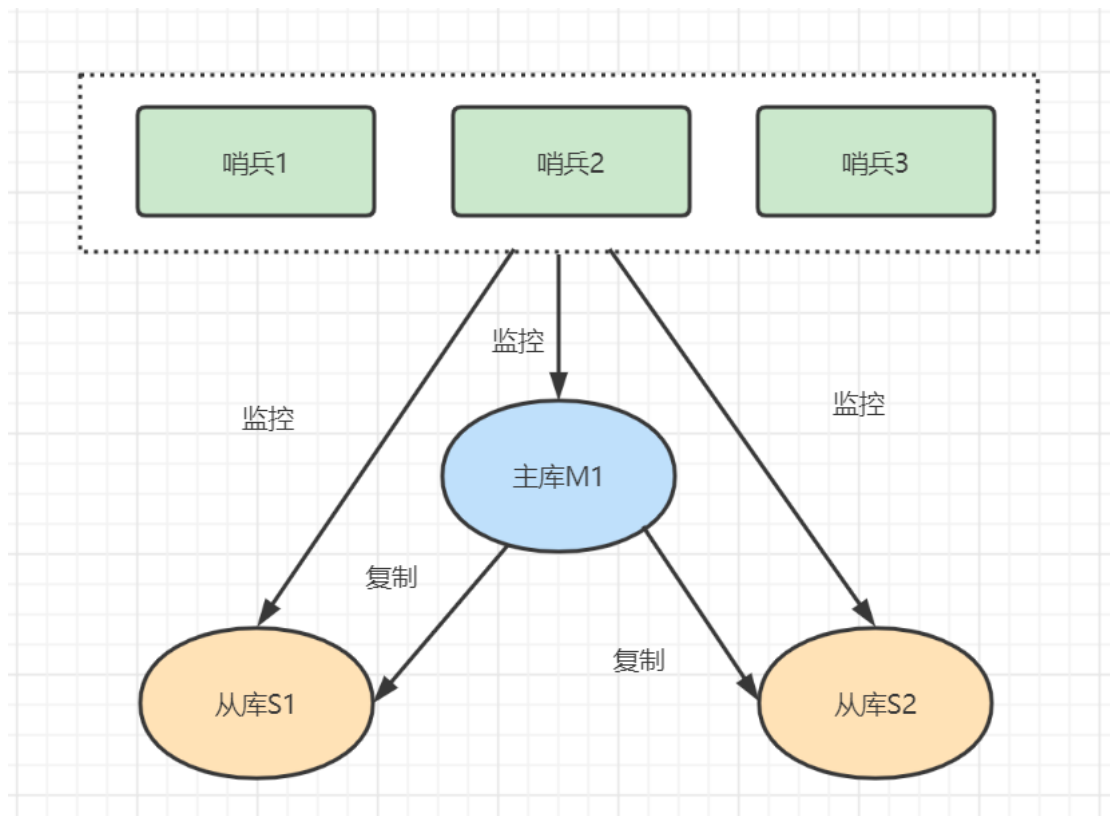
假设网络故障等原因，哨兵 A3 也没有收到两张票，那么这轮投票就不会产生

Leader。哨兵集群会等待一段时间（一般是哨兵故障转移超时时间的 2 倍），

再进行重新选举。

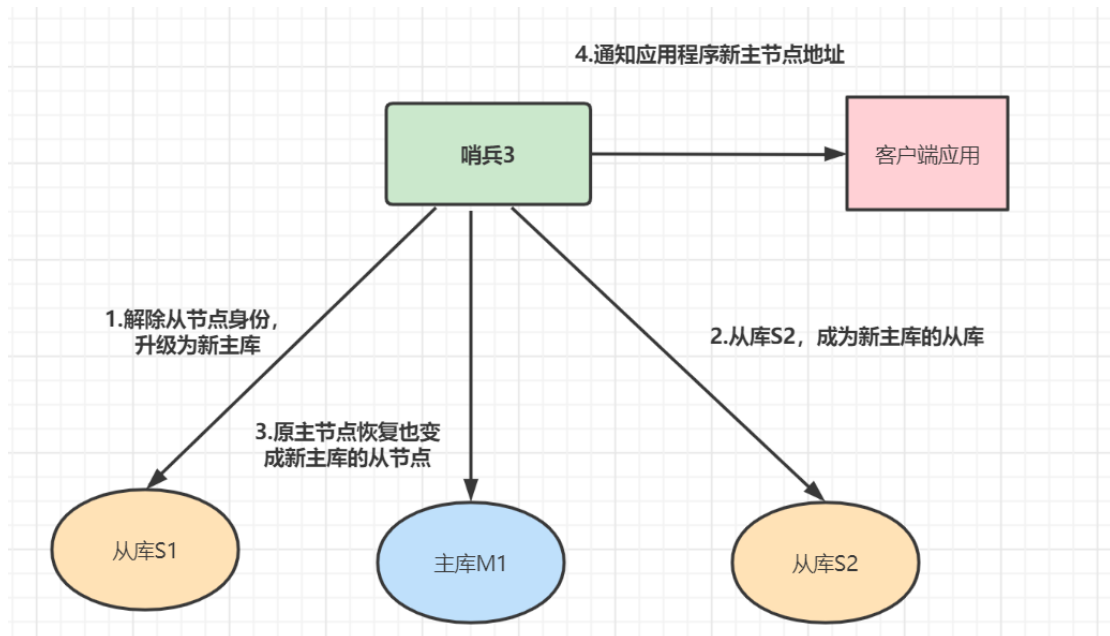
2.9.2.7 故障转移

假设哨兵模式架构如下，有三个哨兵，一个主库 M，两个从库 S1 和 S2。



当哨兵检测到 Redis 主库 M1 出现故障，那么哨兵需要对集群进行故障转移。

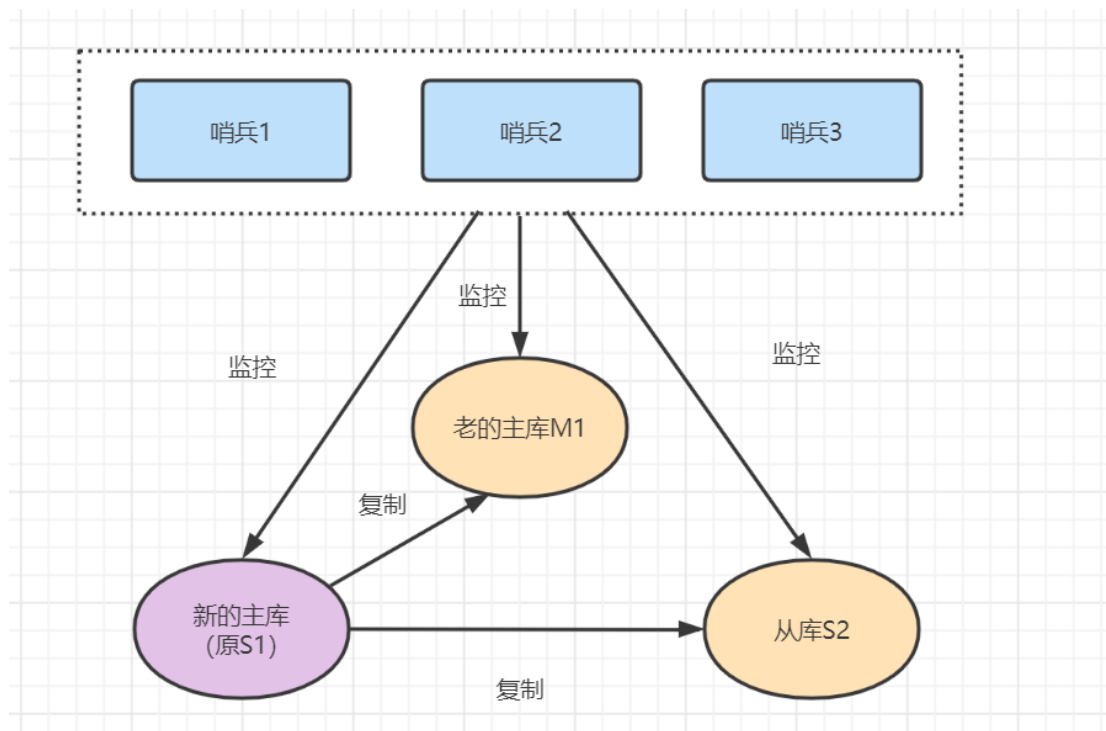
假设选出了**哨兵3**作为 Leader。故障转移流程如下：



1. 从库 S1 解除从节点身份，升级为新主库
2. 从库 S2 成为新主库的从库
3. 原主节点恢复也变成新主库的从节点

4. 通知客户端应用程序新主节点的地址。

故障转移后：



2.9.3 Redis Cluster 集群

哨兵模式基于主从模式，实现读写分离，它还可以自动切换，系统可用性更高。

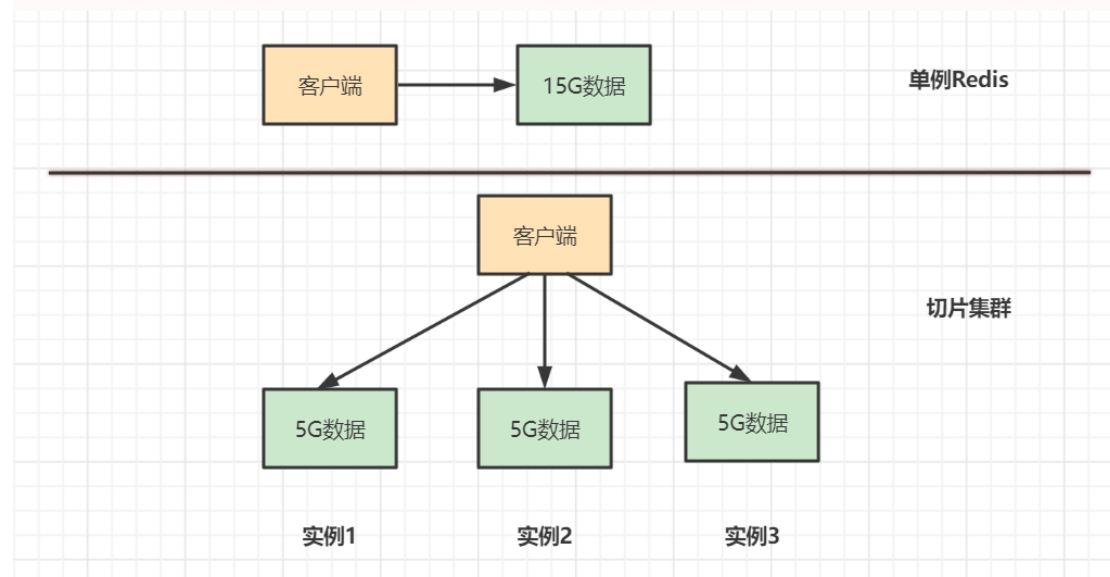
但是它每个节点存储的数据是一样的，浪费内存，并且不好在线扩容。因此，

Redis Cluster 集群 (切片集群的实现方案) 应运而生，它在 Redis 3.0 加入的，实现了 Redis 的分布式存储。对数据进行分片，也就是说每台 Redis 节点上存储不同的内容，来解决在线扩容的问题。并且，它可以保存大量数据，即分散数据到各个 Redis 实例，还提供复制和故障转移的功能。

比如你一个 Redis 实例保存 15G 甚至更大的数据，响应就会很慢，这是因为 Redis RDB 持久化机制导致的，Redis 会 fork 子进程完成 RDB 持久化操作，fork 执行的耗时与 Redis 数据量成正相关。

这时候你很容易想到，把 15G 数据分散来存储就好了嘛。这就是 **Redis 切片集群** 的初衷。切片集群是啥呢？来看个例子，如果你要用 Redis 保存 15G 的数据，可以用单实例 Redis，或者 3 台 Redis 实例组成切片集群，对比如下：

切片集群和 Redis Cluster 的区别：Redis Cluster 是从 Redis3.0 版本开始，官方提供了一种实现切片集群的方案。



既然数据是分片分布到不同 Redis 实例的，那客户端到底是怎么确定想要访问的数据在哪个实例上呢？我们一起来看看下 **Redis Cluster** 是怎么做的哈。

2.9.3.1 哈希槽 (Hash Slot)

Redis Cluster 方案采用哈希槽 (Hash Slot)，来处理数据和实例之间的映射关系。

一个切片集群被分为 16384 个 slot (槽)，每个进入 Redis 的键值对，根据 key 进行散列，分配到这 16384 插槽中的一个。使用的哈希映射也比较简单，用 CRC16 算法计算出一个 16bit 的值，再对 16384 取模。数据库中的每个键都属于这 16384 个槽的其中一个，集群中的每个节点都可以处理这 16384 个槽。集群中的每个节点负责一部分的哈希槽，假设当前集群有 A、B、C 3 个节点，每个节点上负责的哈希槽数 $= 16384 / 3$ ，那么可能存在的一种分配：

- 节点 A 负责 0~5460 号哈希槽
- 节点 B 负责 5461~10922 号哈希槽
- 节点 C 负责 10923~16383 号哈希槽

客户端给一个 Redis 实例发送数据读写操作时，如果这个实例上并没有相应的数据，会怎么样呢？MOVED 重定向和 ASK 重定向了解一下哈

2.9.3.2 MOVED 重定向和 ASK 重定向

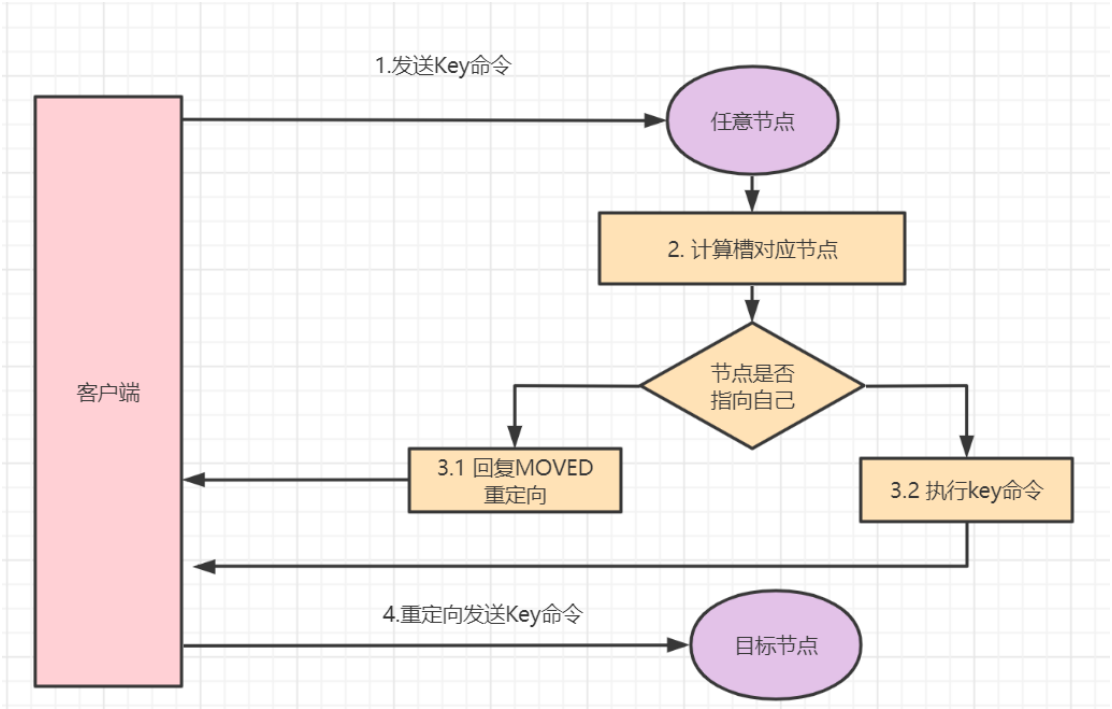
在 Redis cluster 模式下，节点对请求的处理过程如下：

1. 通过哈希槽映射，检查当前 Redis key 是否存在当前节点
2. 若哈希槽不是由自身节点负责，就返回 MOVED 重定向
3. 若哈希槽确实由自身负责，且 key 在 slot 中，则返回该 key 对应结果
4. 若 Redis key 不存在此哈希槽中，检查该哈希槽是否正在迁出 (MIGRATING) ？
5. 若 Redis key 正在迁出，返回 ASK 错误重定向客户端到迁移的目的服务器上
6. 若哈希槽未迁出，检查哈希槽是否导入中？
7. 若哈希槽导入中且有 ASKING 标记，则直接操作，否则返回 MOVED 重定向

Moved 重定向

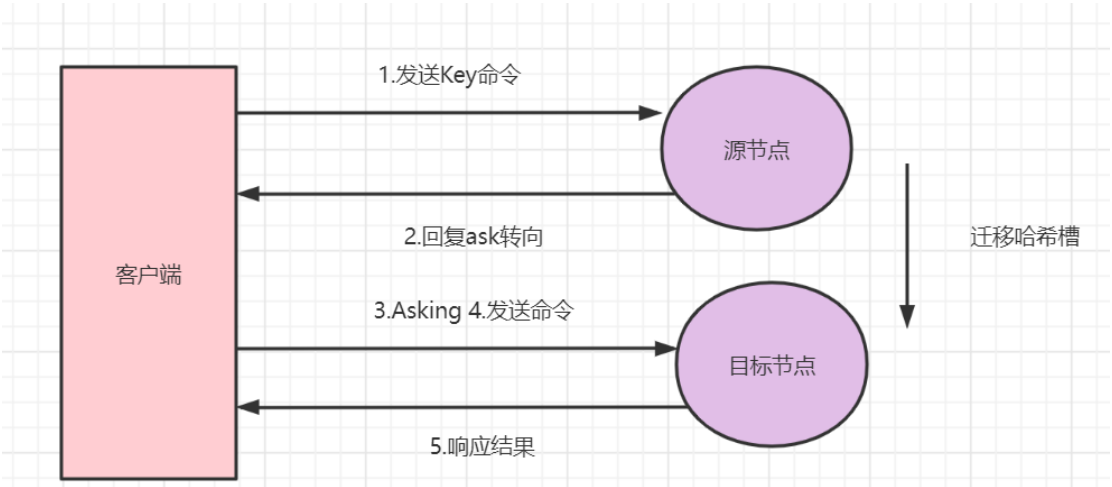
客户端给一个 Redis 实例发送数据读写操作时，如果计算出来的槽不是在该节点上，这时候它会返回 MOVED 重定向错误，MOVED 重定向错误中，会将哈

希槽所在的新实例的 IP 和 port 端口带回去。这就是 Redis Cluster 的 MOVED 重定向机制。流程图如下：



ASK 重定向

Ask 重定向一般发生于集群伸缩的时候。集群伸缩会导致槽迁移，当我们去源节点访问时，此时数据已经可能已经迁移到了目标节点，使用 **Ask 重定向** 可以解决此种情况。



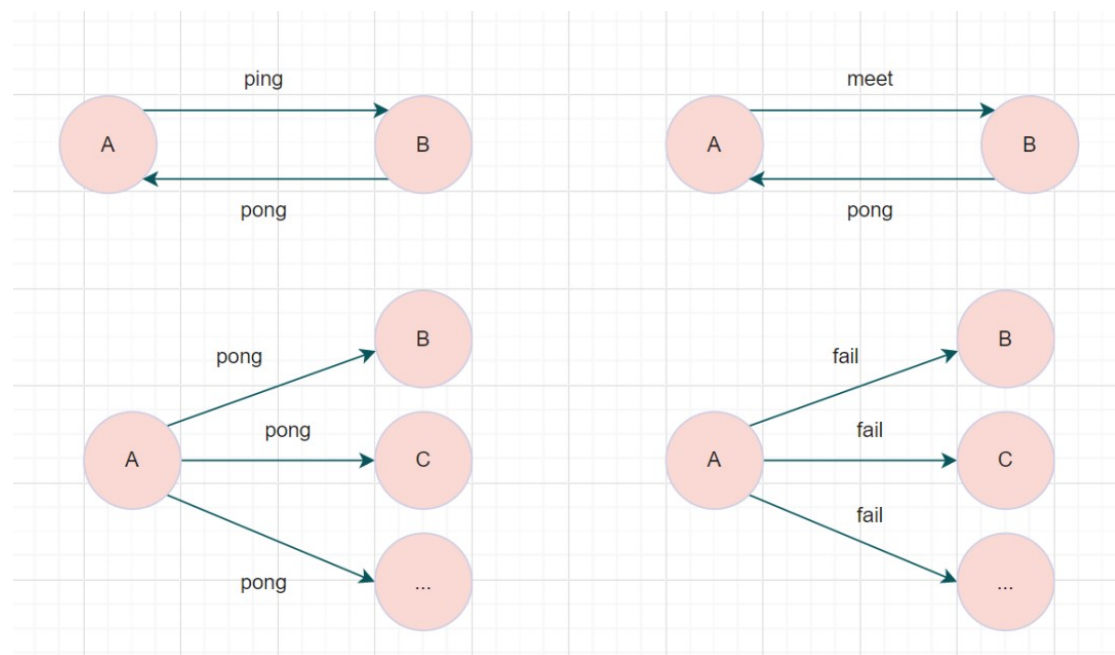
2.9.3.3 Cluster 集群节点的通讯协议：Gossip

一个 Redis 集群由多个节点组成，各个节点之间是怎么通信的呢？通过

Gossip 协议！Gossip 是一种谣言传播协议，每个节点周期性地从节点列表中选择 k 个节点，将本节点存储的信息传播出去，直到所有节点信息一致，即算法收敛了。

Gossip 协议基本思想：一个节点想要分享一些信息给网络中的其他的一些节点。于是，它周期性的随机选择一些节点，并把信息传递给这些节点。这些收到信息的节点接下来会做同样的事情，即把这些信息传递给其他一些随机选择的节点。一般而言，信息会周期性的传递给 N 个目标节点，而不只是一个。这个 N 被称为 fanout

Redis Cluster 集群通过 Gossip 协议进行通信，节点之前不断交换信息，交换的信息内容包括节点出现故障、新节点加入、主从节点变更信息、slot 信息等等。gossip 协议包含多种消息类型，包括 ping，pong，meet，fail，等等



- meet 消息：通知新节点加入。消息发送者通知接收者加入到当前集群，meet 消息通信正常完成后，接收节点会加入到集群中并进行周期性的 ping、pong 消息交换。
- ping 消息：节点每秒会向集群中其他节点发送 ping 消息，消息中带有自己已知的两个节点的地址、槽、状态信息、最后一次通信时间等
- pong 消息：当接收到 ping、meet 消息时，作为响应消息回复给发送方确认消息正常通信。消息中同样带有自己已知的两个节点信息。
- fail 消息：当节点判定集群内另一个节点下线时，会向集群内广播一个 fail 消息，其他节点接收到 fail 消息之后把对应节点更新为下线状态。

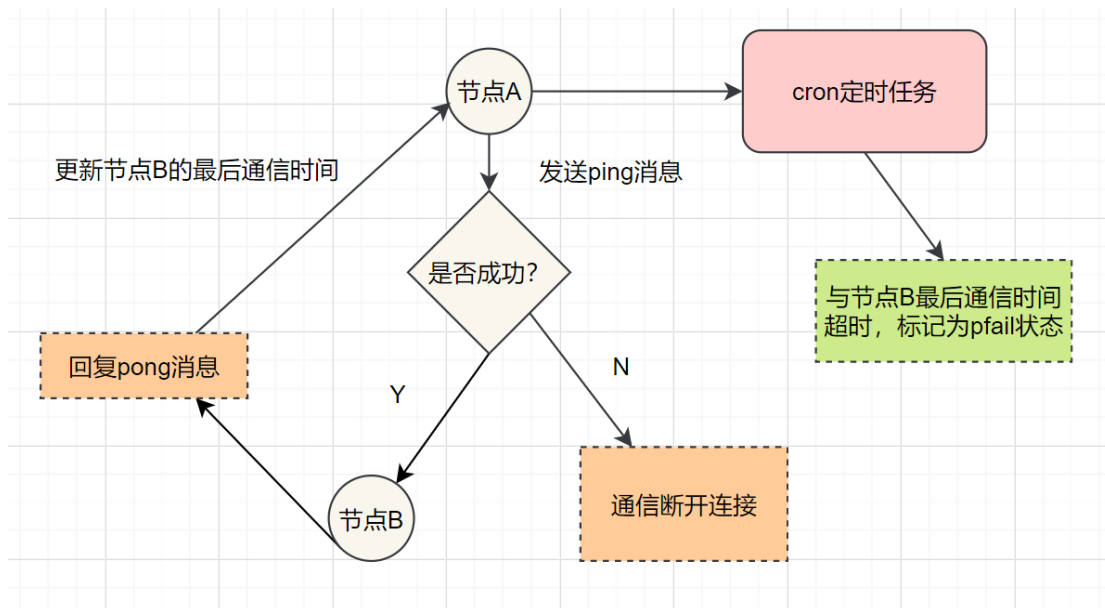
特别的，每个节点是通过集群总线(cluster bus) 与其他的节点进行通信的。通讯时，使用特殊的端口号，即对外服务端口号加 10000。例如如果某个 node 的端口号是 6379，那么它与其它 nodes 通信的端口号是 16379。nodes 之间的通信采用特殊的二进制协议。

2.9.3.4 故障转移

Redis 集群实现了高可用，当集群内节点出现故障时，通过**故障转移**，以保证集群正常对外提供服务。

redis 集群通过 ping/pong 消息，实现故障发现。这个环境包括**主观下线**和**客观下线**。

主观下线：某个节点认为另一个节点不可用，即下线状态，这个状态并不是最终的故障判定，只能代表一个节点的意见，可能存在误判情况。

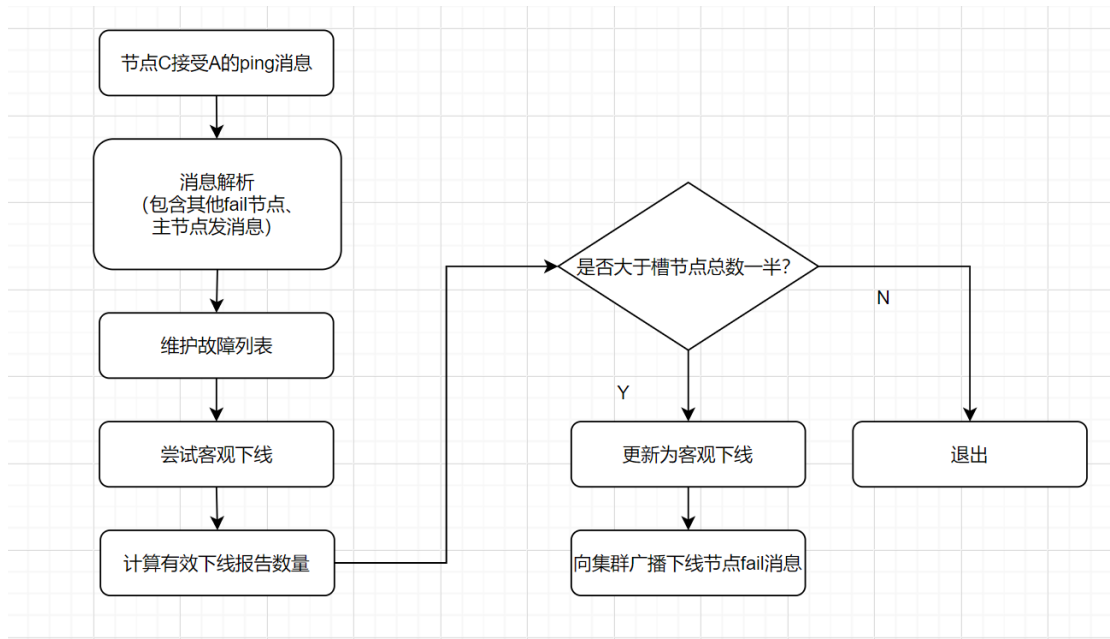


主观下线

客观下线： 指标记一个节点真正的下线，集群内多个节点都认为该节点不可用，从而达成共识的结果。如果是持有槽的主节点故障，需要为该节点进行故障转移。

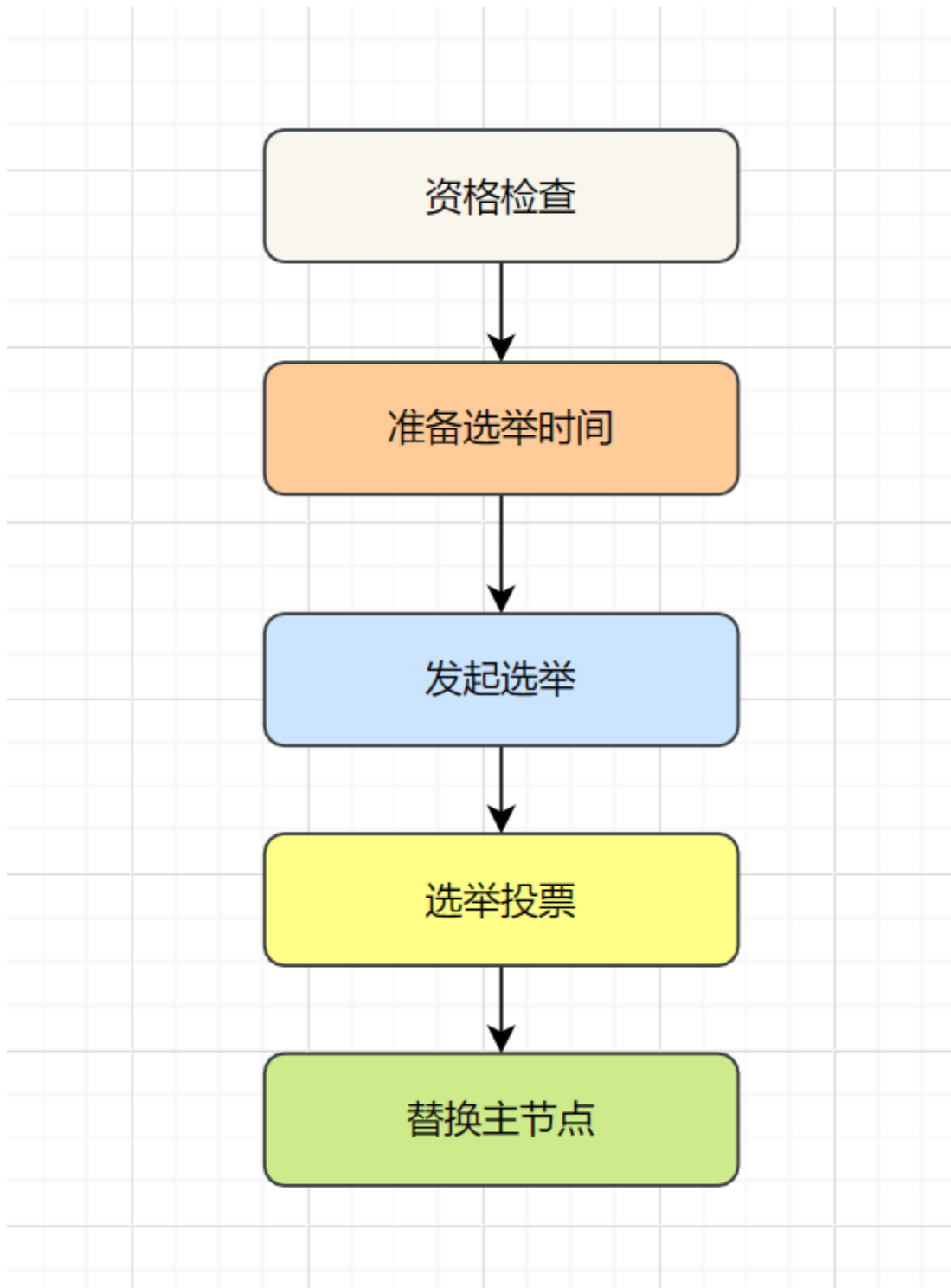
- 假如节点 A 标记节点 B 为主观下线，一段时间后，节点 A 通过消息把节点 B 的状态发到其它节点，当节点 C 接受到消息并解析出消息体时，如果发现节点 B 的 pfail 状态时，会触发客观下线流程；
- 当下线为主节点时，此时 Redis Cluster 集群为统计持有槽的主节点投票，看投票数是否达到一半，当下线报告统计数大于一半时，被标记为**客观下线**状态。

流程如下：



客观下线

故障恢复：故障发现后，如果下线节点的是主节点，则需要在它的从节点中选一个替换它，以保证集群的高可用。流程如下：



- 资格检查：检查从节点是否具备替换故障主节点的条件。
- 准备选举时间：资格检查通过后，更新触发故障选举时间。
- 发起选举：到了故障选举时间，进行选举。
- 选举投票：只有持有槽的主节点才有票，从节点收集到足够的选票（大于一半），触发替换主节点操作

2.9.3.5 加餐：为什么 Redis Cluster 的 Hash Slot 是 16384？

对于客户端请求过来的键值 key，哈希槽 = $\text{CRC16}(\text{key}) \% 16384$ ，CRC16 算法产生的哈希值是 16bit 的，按道理该算法是可以产生 $2^{16}=65536$ 个值，为什么不用 65536，用的是 $16384 (2^{14})$ 呢？

大家可以看下作者的原始回答：

why redis-cluster use 16384 slots? #2576

Closed

qunchenmy opened this issue on 12 May 2015 · 24 comments



qunchenmy commented on 12 May 2015

why redis-cluster use 16384 slots? crc16() can have $2^{16} - 1 = 65535$ different remainders.



antirez commented on 12 May 2015

Contributor

The reason is:

1. Normal heartbeat packets carry the full configuration of a node, that can be replaced in an idempotent way with the old in order to update an old config. This means they contain the slots configuration for a node, in raw form, that uses 2k of space with 16k slots, but would use a prohibitive 8k of space using 65k slots.
2. At the same time it is unlikely that Redis Cluster would scale to more than 1000 master nodes because of other design tradeoffs.

So 16k was in the right range to ensure enough slots per master with a max of 1000 masters, but a small enough number to propagate the slot configuration as a raw bitmap easily. Note that in small clusters the bitmap would be hard to compress because when N is small the bitmap would have slots/N bits set that is a large percentage of bits set.

96

Assi
No c
—
Labr
sta
—
Proj
Non
—
Mile
No r
—
Dev
No l
—
Noti
You'

Redis 每个实例节点上都保存对应有哪些 slots , 它是一个 `unsigned char`

`slots[REDIS_CLUSTER_SLOTS/8]` 类型

```
typedef struct clusterNode {
    mstime_t ctime; /* Node object creation time. */
    char name[REDIS_CLUSTER_NAMELEN]; /* Node name, hex string, sha1-size */
    int flags; /* REDIS_NODE_... */
    uint64_t configEpoch; /* Last configEpoch observed for this node */
    unsigned char slots[REDIS_CLUSTER_SLOTS/8]; /* slots handled by this node */
    int numslots; /* Number of slots handled by this node */
    int numslaves; /* Number of slave nodes, if this is a master */
    struct clusterNode **slaves; /* pointers to slave nodes */
    struct clusterNode *slaveof; /* pointer to the master node */
    mstime_t ping_sent; /* Unix time we sent latest ping */
    mstime_t pong_received; /* Unix time we received the pong */
    mstime_t fail_time; /* Unix time when FAIL flag was set */
    mstime_t voted_time; /* Last time we voted for a slave of this master */
    mstime_t repl_offset_time; /* Unix time we received offset for this node */
    long long repl_offset; /* Last known repl offset for this node. */
    char ip[REDIS_IP_STR_LEN]; /* Latest known IP address of this node */
    int port; /* Latest known port of this node */
    clusterLink *link; /* TCP/IP link with this node */
    list *fail_reports; /* List of nodes signaling this as failing */
} clusterNode;
```

- 在 redis 节点发送心跳包时需要把所有的槽放到这个心跳包里 , 如果 slots 数量是 65536 , 占空间 = $65536 / 8(\text{一个字节 } 8\text{bit}) / 1024(1024 \text{ 个字节 } 1\text{kB}) = 8\text{kB}$, 如果使用 slots 数量是 16384 , 所占空间 = $16384 / 8(\text{每个字节 } 8\text{bit}) / 1024(1024 \text{ 个字节 } 1\text{kB}) = 2\text{kB}$, 可见 16384 个 slots 比 65536 省 6kB 内存左右 , 假如一个集群有 100 个节点, 那每个实例里就省了 600kB 啦
- 一般情况下 Redis cluster 集群主节点数量基本不可能超过 1000 个 , 超过 1000 会导致网络拥堵。对于节点数在 1000 以内的 Redis cluster 集群 , 16384 个槽位其实够用了。

既然为了节省内存网络开销 , 为什么 slots 不选择用 **8192 (即 16384/2)**

呢?

$8192 / 8(\text{每个字节 } 8\text{bit}) / 1024(1024 \text{ 个字节 } 1\text{kB}) = 1\text{kB}$, 只需要 1KB ! 可以先

看下 Redis 把 Key 换算成所属 slots 的方法

```
unsigned int keyHashSlot(char *key, int keylen) {
    int s, e; /* start-end indexes of { and } */

    for (s = 0; s < keylen; s++)
        if (key[s] == '{') break;
```

```

/* No '{' ? Hash the whole key. This is the base case. */
if (s == keylen) return crc16(key,keylen) & 0x3FFF;

/* '{' found? Check if we have the corresponding '}'. */
for (e = s+1; e < keylen; e++)
    if (key[e] == '}') break;

/* No '}' or nothing between {} ? Hash the whole key. */
if (e == keylen || e == s+1) return crc16(key,keylen) & 0x3FFF;

/* If we are here there is both a { and a } on its right. Hash
 * what is in the middle between { and }. */
return crc16(key+s+1,e-s-1) & 0x3FFF;
}

```

Redis 将 key 换算成 slots 的方法：其实就是是将 `crc16(key)` 之后再和 slots 的数量进行与计算

这里为什么用 `0x3FFF(16383)` 来计算,而不是 `16384` 呢？因为在不产生溢出的情况下 $x \% (2^n)$ 等价于 $x \& (2^n - 1)$ 即 $x \% 16384 == x \& 16383$

那到底为什么不用 8192 呢？

`crc16` 出来结果,理论上出现重复的概率为 $1/65536$,但实际结果重复概率可能比这个大不少,就像 `crc32` 结果 理论上 $1/40$ 亿 分之一,但实际有人测下来 10 万碰撞的概率就比较大了。假如 slots 设置成 8192, 200 个实例的节点情况下,理论值是 每 40 个不同 key 请求,命中就会失效一次,假如节点数增加到 400,那就是 20 个请求。并且 1kb 并不会比 2k 省太多,性价比不是特别高,所以可能选 16384 会更为通用一点

之前我这篇文章，收到超级多读者好评，[Redis 主从、哨兵、Cluster 集群](#)

[一锅端！](#)

2.10 使用过 Redis 分布式锁嘛？有哪些注意点呢？

分布式锁，是控制分布式系统不同进程共同访问共享资源的一种锁的实现。秒杀下单、抢红包等业务场景，都需要用到分布式锁，我们项目中经常使用 Redis 作为分布式锁。

选了 Redis 分布式锁的几种实现方法，大家来讨论下，看有没有啥问题哈。

- 命令 setnx + expire 分开写
- setnx + value 值是过期时间
- set 的扩展命令 (set ex px nx)
- set ex px nx + 校验唯一随机值,再删除

2.10.1 命令 setnx + expire 分开写

```
if ( jedis.setnx(key,lock_value) == 1 ) { //加锁
    expire ( key , 100 ); //设置过期时间
    try {
        do something //业务请求
    }catch(){
    }
    finally {
        jedis.del(key); //释放锁
    }
}
```

如果执行完 setnx 加锁，正要执行 expire 设置过期时间时，进程 crash 掉或者要重启维护了，那这个锁就“长生不老”了，别的线程永远获取不到锁啦，所以分布式锁不能这么实现。

2.10.2 setnx + value 值是过期时间

```
long expires = System.currentTimeMillis() + expireTime; //系统时间+设置的过期时间
String expiresStr = String.valueOf(expires);

// 如果当前锁不存在，返回加锁成功
```

```

if (jedis.setnx(key, expiresStr) == 1) {
    return true;
}
// 如果锁已经存在，获取锁的过期时间
String currentValueStr = jedis.get(key);

// 如果获取到的过期时间，小于系统当前时间，表示已经过期
if (currentValueStr != null && Long.parseLong(currentValueStr) < System.currentTimeMillis()) {

    // 锁已过期，获取上一个锁的过期时间，并设置现在锁的过期时间（不了解 redis 的 getSet 命令的小伙伴，可以去官网看下哈）
    String oldValueStr = jedis.getSet(key_resource_id, expiresStr);

    if (oldValueStr != null && oldValueStr.equals(currentValueStr)) {
        // 考虑多线程并发的情况，只有一个线程的设置值和当前值相同，它才可以加锁
        return true;
    }
}

//其他情况，均返回加锁失败
return false;
}

```

笔者看过有开发小伙伴是这么实现分布式锁的，但是这种方案也有这些缺点：

- 过期时间是客户端自己生成的，分布式环境下，每个客户端的时间必须同步。
- 没有保存持有者的唯一标识，可能被别的客户端释放/解锁。
- 锁过期的时候，并发多个客户端同时请求过来，都执行了 `jedis.getSet()`，最终只能有一个客户端加锁成功，但是该客户端锁的过期时间，可能被别的客户端覆盖。

2.10.3：set 的扩展命令 (set ex px nx) (注意可能存在的问题)

```

if ( jedis.set(key, lock_value, "NX", "EX", 100s) == 1 ) { //加锁
    try {
        do something //业务处理
    } catch ( ) {
    }
    finally {
        jedis.del(key); //释放锁
    }
}

```

```
}  
}
```

这个方案可能存在这样的问题：

- 锁过期释放了，业务还没执行完。
- 锁被别的线程误删。

2.10.4 set ex px nx + 校验唯一随机值,再删除

```
if ( jedis.set(key, uni_request_id, "NX", "EX", 100s) == 1 ) { //加锁  
    try {  
        do something //业务处理  
    } catch ( ) {  
    }  
    finally {  
        //判断是不是当前线程加的锁,是才释放  
        if ( uni_request_id.equals(jedis.get(key))) {  
            jedis.del(key); //释放锁  
        }  
    }  
}
```

在这里，判断当前线程加的锁和释放锁是不是一个原子操作。如果调用 jedis.del() 释放锁的时候，可能这把锁已经不属于当前客户端，会解除他人加的锁。

```
if ( jedis.set(key_resource_id, uni_request_id, "NX", "EX", 100s) == 1 ) { //加锁  
    try {  
        do something //业务处理  
    } catch ( ) {  
    }  
    finally {  
        //判断是不是当前线程加的锁,是才释放  
        if ( uni_request_id.equals(jedis.get(key_resource_id))) {  
            jedis.del(lockKey);  
        }  
    }  
}
```

非原子

@稀土掘金技术社区
掘金技术社区

一般也是用 lua 脚本代替。lua 脚本如下：

```
if redis.call('get',KEYS[1]) == ARGV[1] then
    return redis.call('del',KEYS[1])
else
    return 0
end;
```

这种方式比较不错了，一般情况下，已经可以使用这种实现方式。但是存在锁过期释放了，业务还没执行完的问题（实际上，估算个业务处理的时间，一般没啥问题了）。

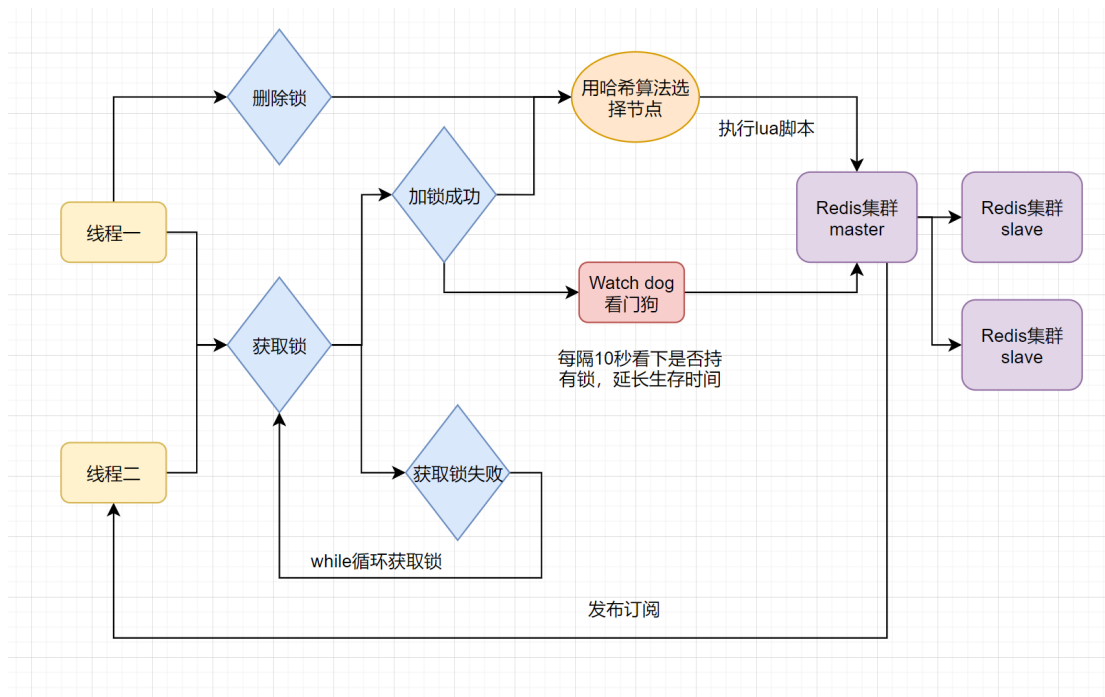
可以看下我之前这篇文章，[七种方案！探讨 Redis 分布式锁的正确使用姿势](#)

2.11 使用过 Redisson 嘛？说说它的原理

分布式锁可能存在锁过期释放，业务没执行完的问题。有些小伙伴认为，稍微把锁过期时间设置长一些就可以啦。其实我们设想一下，是否可以给获得锁的线程，开启一个定时守护线程，每隔一段时间检查锁是否还存在，存在则对锁的过期时间延长，防止锁过期提前释放。

当前开源框架 **Redisson** 就解决了这个分布式锁问题。我们一起来看下

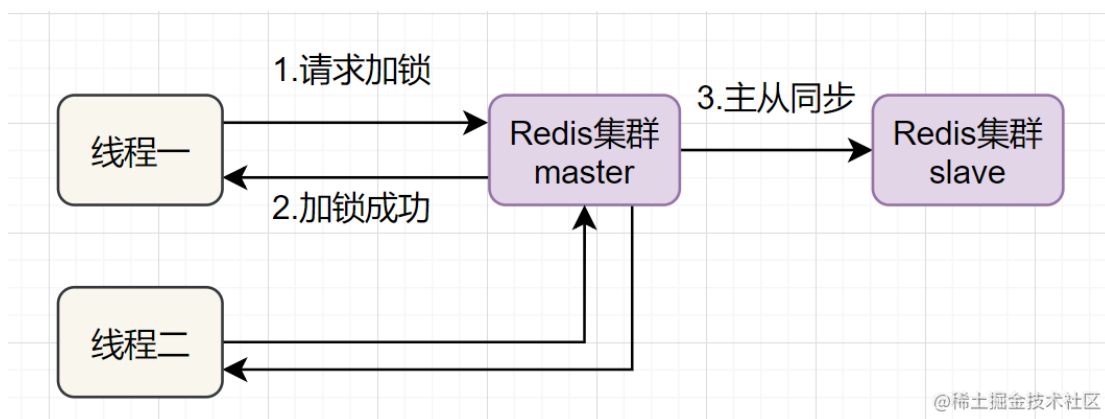
Redisson 底层原理是怎样的吧：



只要线程一加锁成功，就会启动一个 **watch dog** 看门狗，它是一个后台线程，会每隔 10 秒检查一下，如果线程 1 还持有锁，那么就会不断的延长锁 key 的生存时间。因此，Redisson 就是使用 Redisson 解决了锁过期释放，业务没执行完问题。

2.12 什么是 Redlock 算法

Redis 一般都是集群部署的，假设数据在主从同步过程，主节点挂了，Redis 分布式锁可能会有哪些问题呢？一起来看些这个流程图：



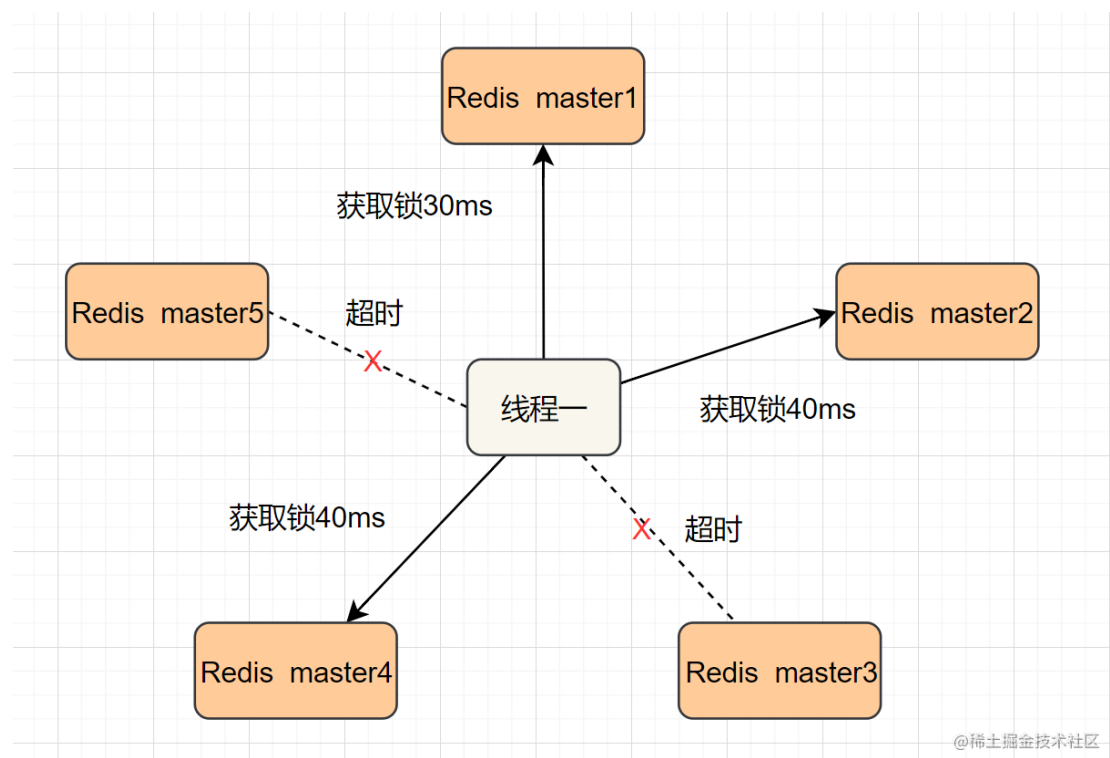
如果线程一在 Redis 的 master 节点上拿到了锁，但是加锁的 key 还没同步到 slave 节点。恰好这时，master 节点发生故障，一个 slave 节点就会升级为 master 节点。线程二就可以获取同个 key 的锁啦，但线程一也已经拿到锁了，锁的安全性就没了。

为了解决这个问题，Redis 作者 antirez 提出一种高级的分布式锁算法：

Redlock。Redlock 核心思想是这样的：

搞多个 Redis master 部署，以保证它们不会同时宕掉。并且这些 master 节点是完全相互独立的，相互之间不存在数据同步。同时，需要确保在这多个 master 实例上，是与在 Redis 单实例，使用相同方法来获取和释放锁。

我们假设当前有 5 个 Redis master 节点，在 5 台服务器上面运行这些 Redis 实例。



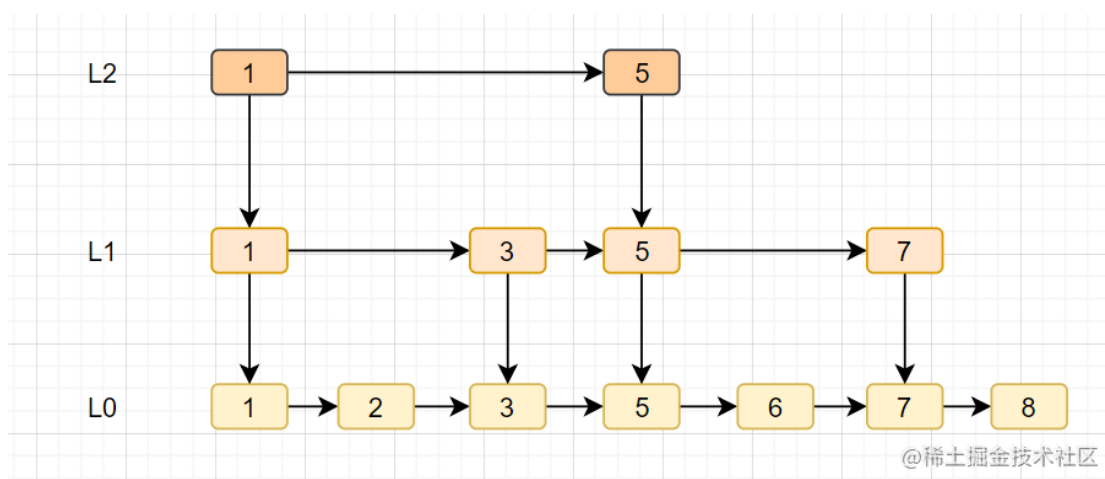
RedLock 的实现步骤:如下

- 1.获取当前时间，以毫秒为单位。
- 2.按顺序向 5 个 master 节点请求加锁。客户端设置网络连接和响应超时时间，并且超时时间要小于锁的失效时间。（假设锁自动失效时间为 10 秒，则超时时间一般在 5-50 毫秒之间,我们就假设超时时间是 50ms 吧）。如果超时，跳过该 master 节点，尽快去尝试下一个 master 节点。
- 3.客户端使用当前时间减去开始获取锁时间（即步骤 1 记录的时间），得到获取锁使用的时间。当且仅当超过一半（ $N/2+1$ ，这里是 $5/2+1=3$ 个节点）的 Redis master 节点都获得锁，并且使用的时间小于锁失效时间时，锁才算获取成功。（如上图， $10s > 30ms+40ms+50ms+40ms+50ms$ ）
- 如果取到了锁，key 的真正有效时间就变啦，需要减去获取锁所使用的时间。
- 如果获取锁失败（没有在至少 $N/2+1$ 个 master 实例取到锁，有或者获取锁时间已经超过了有效时间），客户端要在所有的 master 节点上解锁（即便有些 master 节点根本就没有加锁成功，也需要解锁，以防止有些漏网之鱼）。

简化下步骤就是：

- 按顺序向 5 个 master 节点请求加锁
- 根据设置的超时时间来判断，是不是要跳过该 master 节点。
- 如果大于等于三个节点加锁成功，并且使用的时间小于锁的有效期，即可认定加锁成功啦。
- 如果获取锁失败，解锁！

2.13 Redis 的跳跃表



跳跃表

- 跳跃表是有序集合 zset 的底层实现之一
- 跳跃表支持平均 $O(\log N)$ ，最坏 $O(N)$ 复杂度的节点查找，还可以通过顺序性操作批量处理节点。

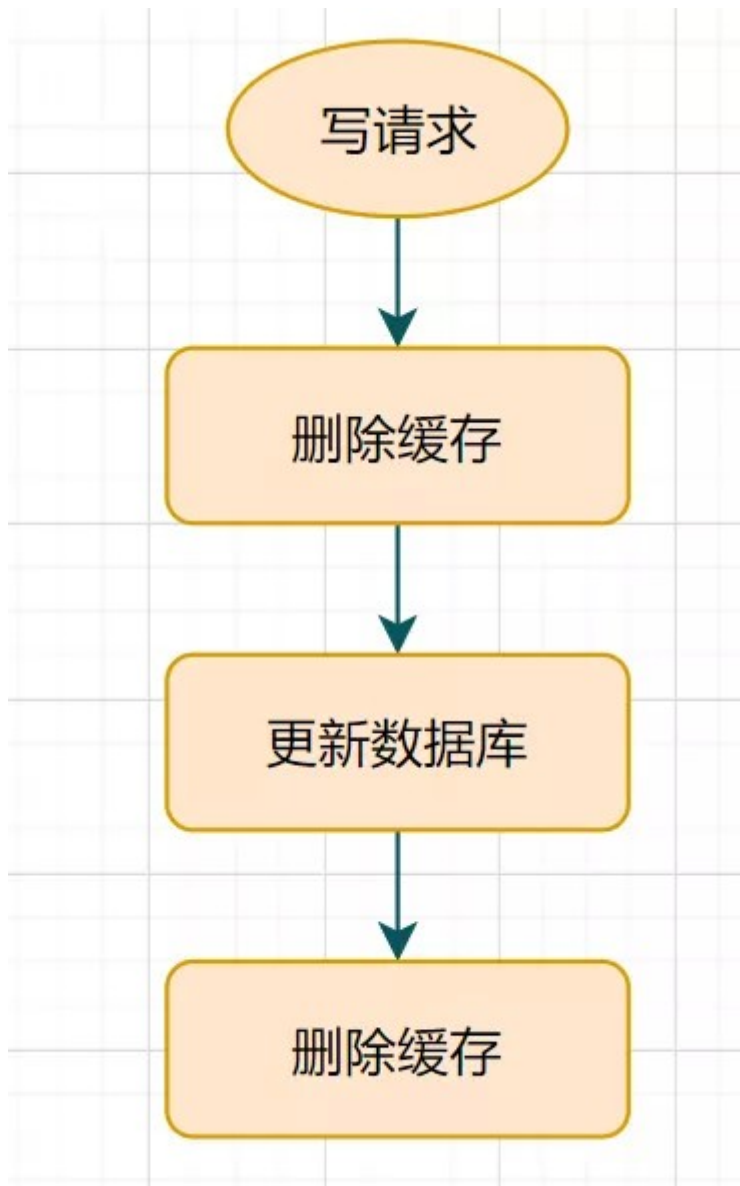
- 跳跃表实现由 **zskiplist** 和 **zskiplistNode** 两个结构组成，其中 zskiplist 用于保存跳跃表信息（如表头节点、表尾节点、长度），而 zskiplistNode 则用于表示跳跃表节点。
- 跳跃表就是在链表的基础上，增加多级索引提升查找效率。

2.14 MySQL 与 Redis 如何保证双写一致性

- 缓存延时双删
- 删除缓存重试机制
- 读取 biglog 异步删除缓存

2.14.1 延时双删？

什么是延时双删呢？流程图如下：



延时双删流程

1. 先删除缓存
2. 再更新数据库
3. 休眠一会（比如 1 秒），再次删除缓存。

这个休眠一会，一般多久呢？都是 1 秒？

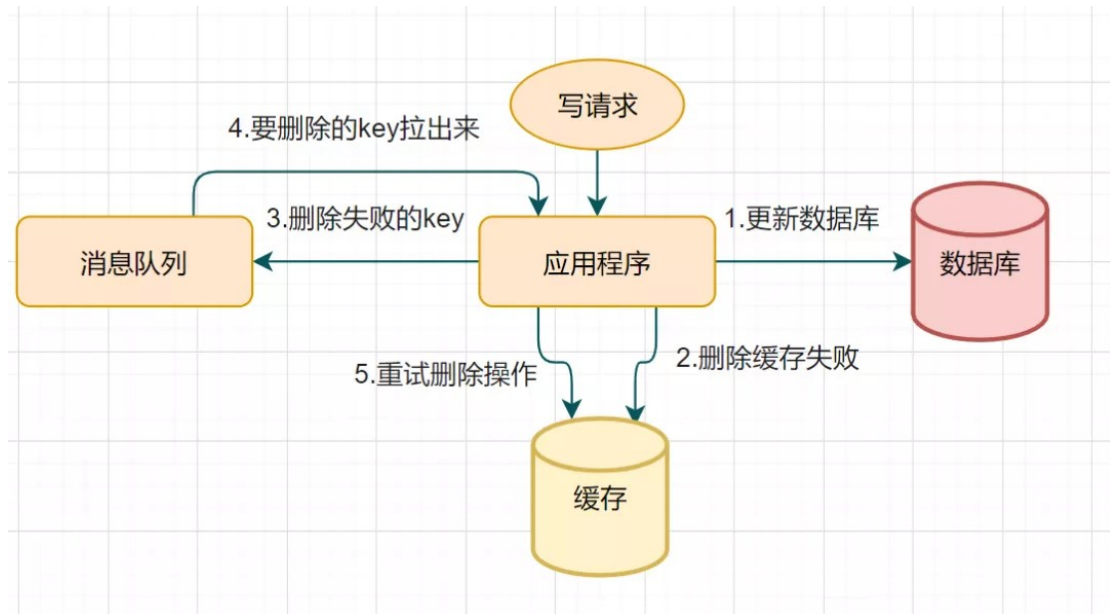
这个休眠时间 = 读业务逻辑数据的耗时 + 几百毫秒。为了确保读请求结束，写请求可以删除读请求可能带来的缓存脏数据。

这种方案还算可以，只有休眠那会（比如就那 1 秒），可能有脏数据，一般业务也会接受的。但是如果**第二次删除缓存失败**呢？缓存和数据库的数据还是

可能不一致，对吧？给 Key 设置一个自然的 expire 过期时间，让它自动过期
怎样？那业务要接受过期时间内，数据的不一致咯？还是有其他更佳方案呢？

2.14.2 删除缓存重试机制

因为延时双删可能会存在第二步的删除缓存失败，导致的数据不一致问题。可以使用这个方案优化：删除失败就多删除几次呀,保证删除缓存成功就可以了呀
~ 所以可以引入删除缓存重试机制

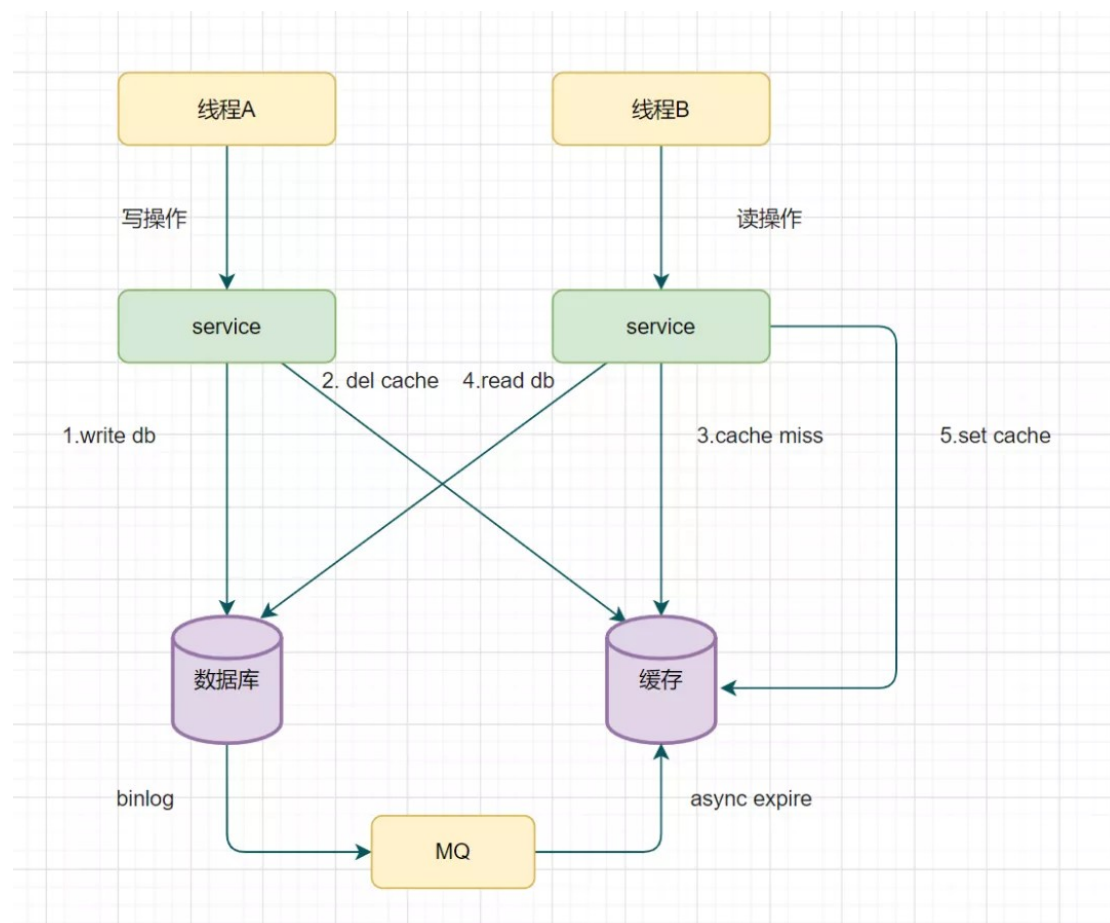


删除缓存重试流程

1. 写请求更新数据库
2. 缓存因为某些原因，删除失败
3. 把删除失败的 key 放到消息队列
4. 消费消息队列的消息，获取要删除的 key
5. 重试删除缓存操作

2.14.3 读取 biglog 异步删除缓存

重试删除缓存机制还可以吧，就是会造成好多业务代码入侵。其实，还可以这样优化：通过数据库的 binlog 来异步淘汰 key。



以 mysql 为例吧

- 可以使用阿里的 canal 将 binlog 日志采集发送到 MQ 队列里面
- 然后通过 ACK 机制确认处理这条更新消息，删除缓存，保证数据缓存一致性

我之前这篇文章，做了详细回答，[美团二面：Redis 与 MySQL 双写一致性如何保证？](#)

2.15 为什么 Redis 6.0 之后改多线程呢？

- Redis6.0 之前，Redis 在处理客户端的请求时，包括读 socket、解析、执行、写 socket 等都由一个顺序串行的主线程处理，这就是所谓的“单线程”。

- Redis6.0 之前为什么一直不使用多线程？使用 Redis 时，几乎不存在 CPU 成为瓶颈的情况，Redis 主要受限于内存和网络。例如在一个普通的 Linux 系统上，Redis 通过使用 pipelining 每秒可以处理 100 万个请求，所以如果应用程序主要使用 $O(N)$ 或 $O(\log(N))$ 的命令，它几乎不会占用太多 CPU。

redis 使用多线程并非是完全摒弃单线程，redis 还是使用单线程模型来处理客户端的请求，只是使用多线程来处理数据的读写和协议解析，执行命令还是使用单线程。

这样做的目的是因为 redis 的性能瓶颈在于网络 IO 而非 CPU，使用多线程能提升 IO 读写的效率，从而整体提高 redis 的性能。

2.16 聊聊 Redis 事务机制

Redis 通过 **MULTI**、**EXEC**、**WATCH** 等一组命令集合，来实现事务机制。

事务支持一次执行多个命令，一个事务中所有命令都会被序列化。在事务执行过程，会按照顺序串行化执行队列中的命令，其他客户端提交的命令请求不会插入到事务执行命令序列中。

简言之，Redis 事务就是**顺序性**、**一次性**、**排他性**的执行一个队列中的一系列命令。

Redis 执行事务的流程如下：

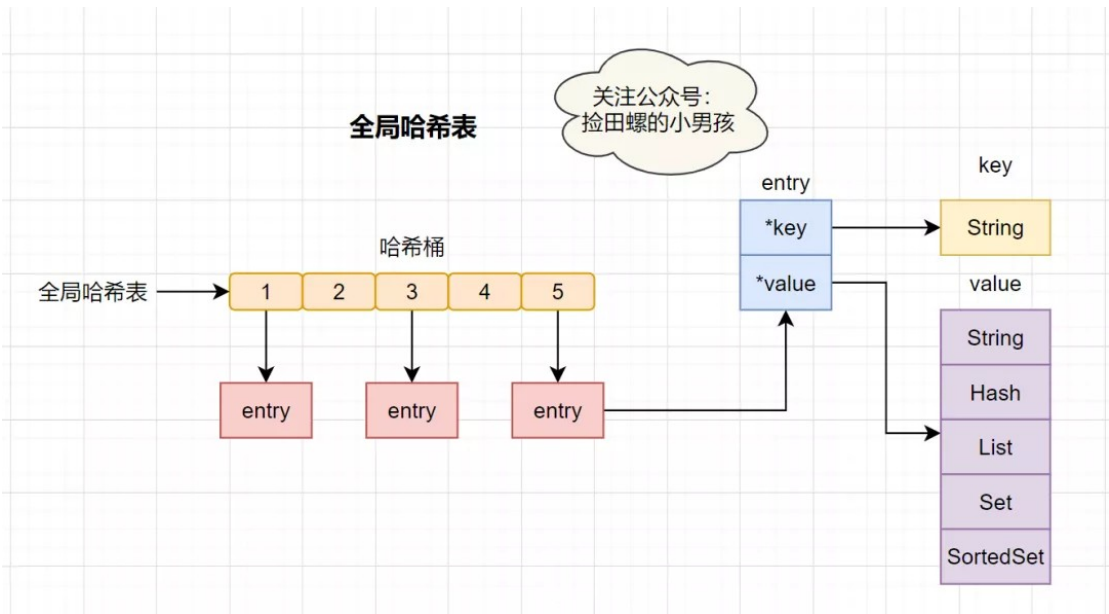
- 开始事务 (MULTI)
- 命令入队
- 执行事务 (EXEC)、撤销事务 (DISCARD)

命令	描述
EXEC	执行所有事务块内的命令
DISCARD	取消事务，放弃执行事务块内的所有命令

命令	描述
MULTI	标记一个事务块的开始
UNWATCH	取消 WATCH 命令对所有 key 的监视。
WATCH	监视 key ，如果在事务执行之前，该 key 被其他命令所改动，那么事务将被打断。

2.17 Redis 的 Hash 冲突怎么办

Redis 作为一个 K-V 的内存数据库，它使用用一张全局的哈希来保存所有的键值对。这张哈希表，有多个哈希桶组成，哈希桶中的 entry 元素保存了 *key* 和 *value* 指针，其中 *key* 指向了实际的键，*value* 指向了实际的值。

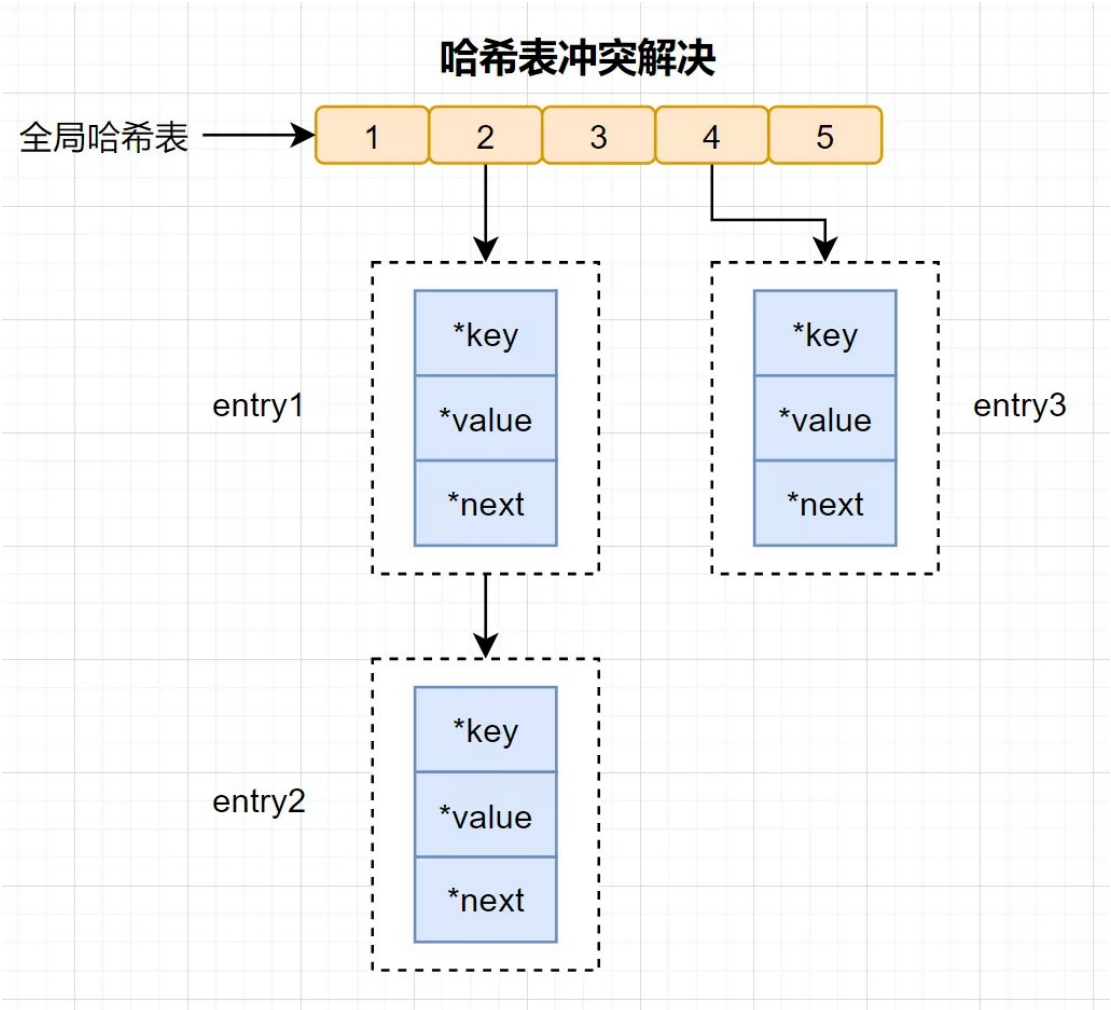


哈希表查找速率很快的，有点类似于 Java 中的 HashMap，它让我们在 $O(1)$ 的时间复杂度快速找到键值对。首先通过 key 计算哈希值，找到对应的哈希桶位置，然后定位到 entry，在 entry 找到对应的数据。

什么是哈希冲突？

哈希冲突：通过不同的 key，计算出一样的哈希值，导致落在同一个哈希桶中。

Redis 为了解决哈希冲突，采用了**链式哈希**。链式哈希是指同一个哈希桶中，多个元素用一个链表来保存，它们之间依次用指针连接。



有些读者可能还会有疑问：哈希冲突链上的元素只能通过指针逐一查找再操作。当往哈希表插入数据很多，冲突也会越多，冲突链表就会越长，那查询效率就会降低了。

为了保持高效，Redis 会对**哈希表做 rehash**操作，也就是增加哈希桶，减少冲突。为了 rehash 更高效，Redis 还默认使用了两个全局哈希表，一个用于当前使用，称为主哈希表，一个用于扩容，称为备用哈希表。

2.18 在生成 RDB 期间，Redis 可以同时处理写请求么？

可以的，Redis 提供两个指令生成 RDB，分别是 **save** 和 **bgsave**。

- 如果是 save 指令，会阻塞，因为是主线程执行的。
- 如果是 bgsave 指令，是 fork 一个子进程来写入 RDB 文件的，快照持久化完全交给子进程来处理，父进程则可以继续处理客户端的请求。

2.19 Redis 底层，使用的什么协议？

RESP，英文全称是 Redis Serialization Protocol,它是专门为 redis 设计的一套序列化协议. 这个协议其实在 redis 的 1.2 版本时就已经出现了,但是到了 redis2.0 才最终成为 redis 通讯协议的标准。

RESP 主要有**实现简单、解析速度快、可读性好**等优点。

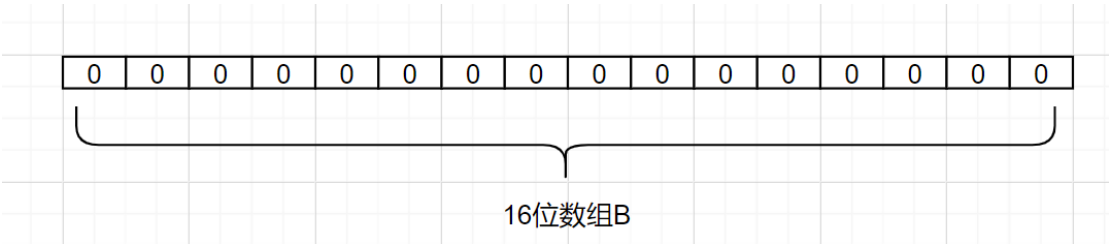
2.20 布隆过滤器

应对**缓存穿透**问题，我们可以使用**布隆过滤器**。布隆过滤器是什么呢？

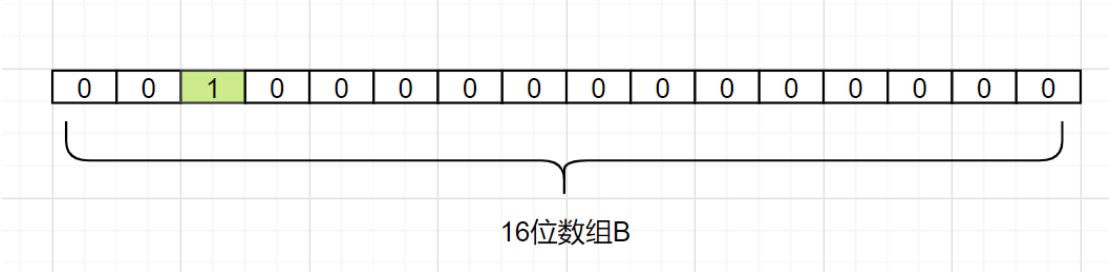
布隆过滤器是一种占用空间很小的数据结构，它由一个很长的二进制向量和一组 Hash 映射函数组成，它用于检索一个元素是否在一个集合中，空间效率和查询时间都比一般的算法要好的多，缺点是有一定的误识别率和删除困难。

布隆过滤器原理是？ 假设我们有个集合 A，A 中有 n 个元素。利用 **k 个哈希散列函数**，将 A 中的每个元素**映射**到一个长度为 a 位的数组 B 中的不同位置上，这些位置上的二进制数均设置为 1。如果待检查的元素，经过这 k 个哈希散列函数的映射后，发现其 k 个位置上的二进制数**全部为 1**，这个元素很可能属于集合 A，反之，**一定不属于集合 A**。

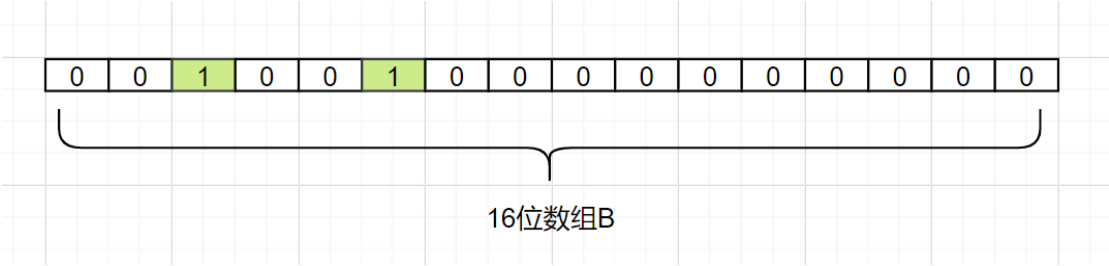
来看个简单例子吧，假设集合 A 有 3 个元素，分别为 {**d1,d2,d3**}。有 1 个哈希函数，为 **Hash1**。现在将 A 的每个元素映射到长度为 16 位数组 B。



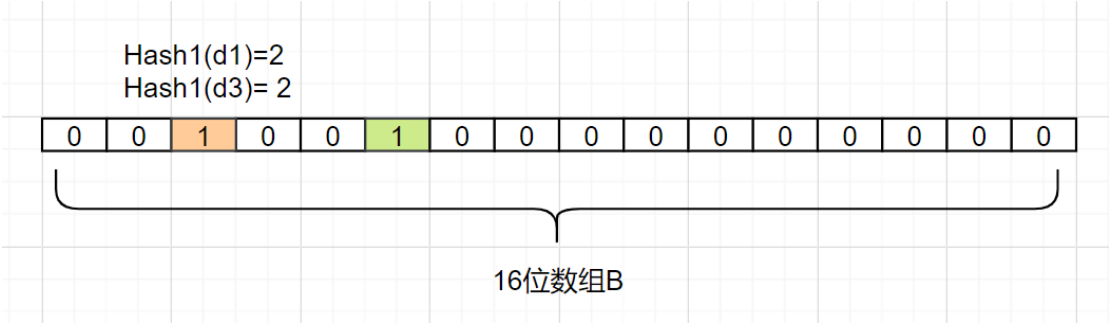
我们现在把 d1 映射过来，假设 $\text{Hash1}(d1) = 2$ ，我们就把数组 B 中，下标为 2 的格子改成 1，如下：



我们现在把 **d2** 也映射过来，假设 $\text{Hash1}(d2) = 5$ ，我们把数组 B 中，下标为 5 的格子也改成 1，如下：



接着我们把 **d3** 也映射过来，假设 $\text{Hash1}(d3)$ 也等于 2，它也是把下标为 2 的格子标 1：



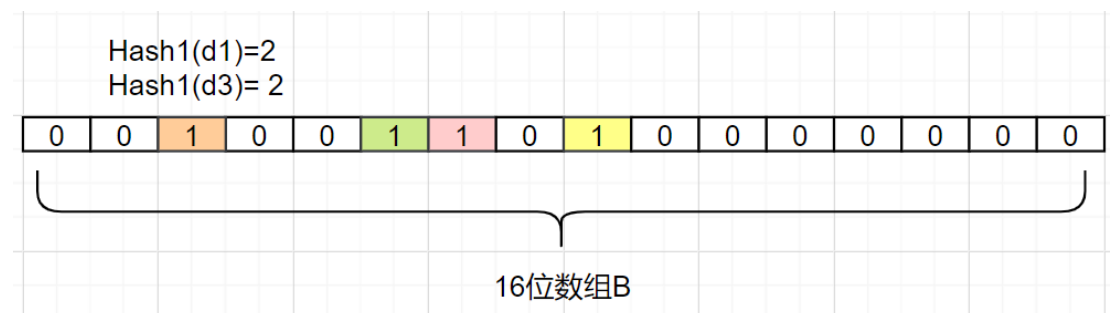
因此，我们要确认一个元素 d_n 是否在集合 A 里，我们只要算出 $\text{Hash1}(d_n)$ 得到的索引下标，只要是 0，那就表示这个元素不在集合 A，如果索引下标是 1 呢？那该元素可能是 A 中的某一个元素。因为你看， d_1 和 d_3 得到的下标值，都可能是 1，还可能是其他别的数映射的，布隆过滤器是存在这个缺点的：会存在 **hash 碰撞** 导致的假阳性，判断存在误差。

如何减少这种误差呢？

- 搞多个哈希函数映射，降低哈希碰撞的概率
- 同时增加 B 数组的 bit 长度，可以增大 hash 函数生成的数据的范围，也可以降低哈希碰撞的概率

我们又增加一个 Hash2 哈希映射函数，假设

$\text{Hash2}(d_1) = 6, \text{Hash2}(d_3) = 8$ ，它俩不就不冲突了嘛，如下：



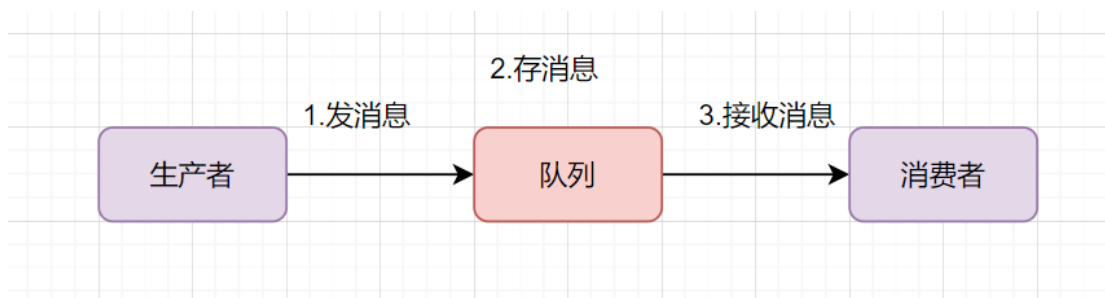
即使存在误差，我们可以发现，布隆过滤器并没有存放完整的数据，它只是运用一系列哈希映射函数计算出位置，然后填充二进制向量。如果数量很大的话，布隆过滤器通过极少的错误率，换取了存储空间的极大节省，还是挺划算的。目前布隆过滤器已经有相应实现的开源类库啦，如 **Google 的 Guava 类库**，Twitter 的 **Algebird 类库**，信手拈来即可，或者基于 Redis 自带的 Bitmaps 自行实现设计也是可以的。

可以看下我这篇文章哈，[面试必备：布隆过滤器是什么？有什么用？](#)

3.消息队列专栏面试题（关注公众号：捡田螺的小男孩）

3.1 什么是消息队列

你可以把消息队列理解为一个使用队列来通信的组件。它的本质，就是个转发器，包含发消息、存消息、消费消息的过程。最简单的消息队列模型如下：



我们通常说的消息队列，简称 **MQ (Message Queue)**，它其实就指消息中间件，当前业界比较流行的开源消息中间件包括：

RabbitMQ、RocketMQ、Kafka。

3.2 消息队列有哪些使用场景。

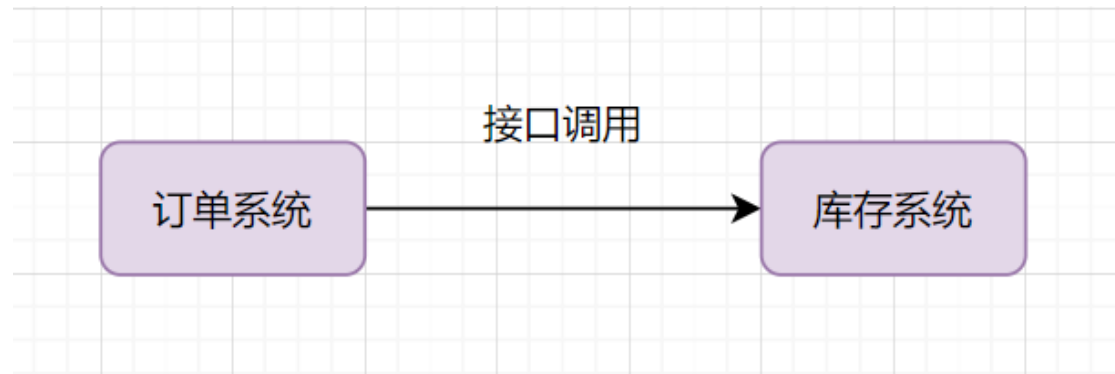
有时候面试官会换个角度问你，为什么使用消息队列。你可以回答以下这几点：

1. 应用解耦
2. 流量削峰
3. 异步处理
4. 消息通讯
5. 远程调用

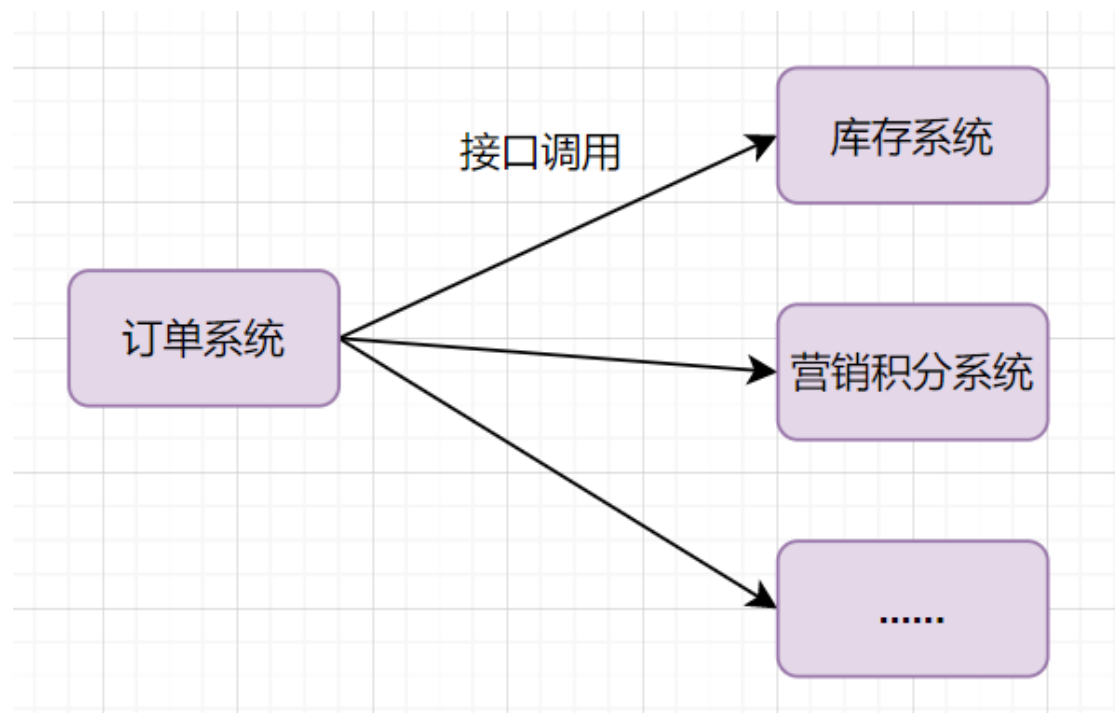
3.2.1 应用解耦

举个常见业务场景：下单扣库存，用户下单后，订单系统去通知库存系统扣减。

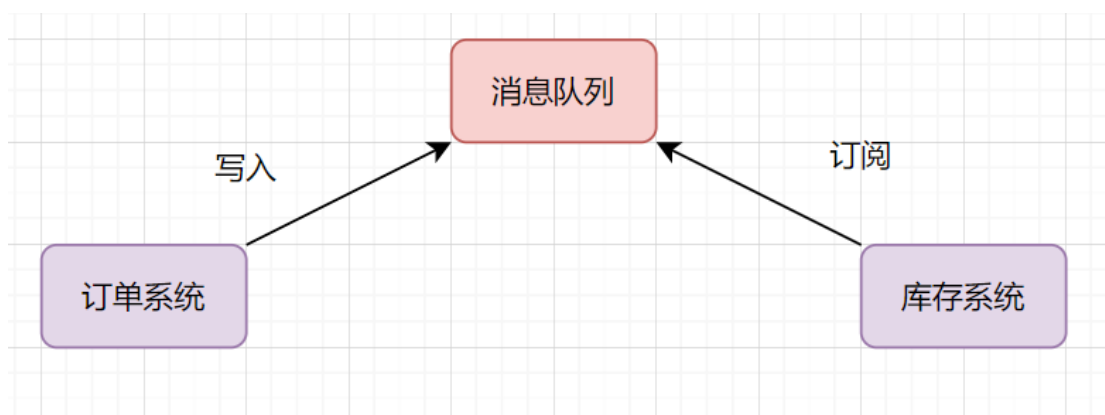
传统的做法就是订单系统直接调用库存系统：



- 如果库存系统无法访问，下单就会失败，订单和库存系统存在耦合关系
- 如果业务又接入一个营销积分服务，那订单下游系统要扩充，如果未来接入越来越多的下游系统，那订单系统代码需要经常修改



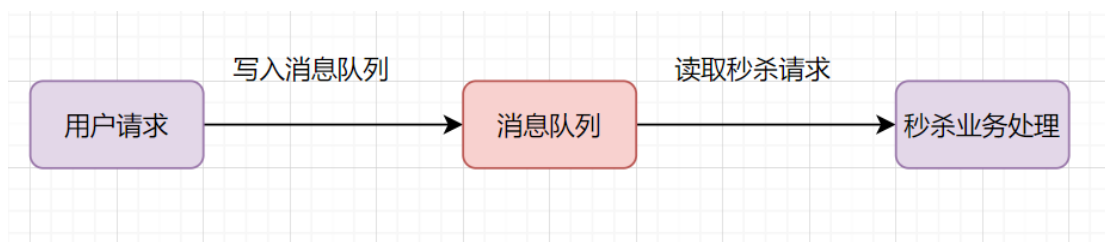
如何解决这个问题呢？可以引入消息队列



1. 订单系统：用户下单后，消息写入到消息队列，返回下单成功
2. 库存系统：订阅下单消息，获取下单信息，进行库存操作。

3.2.2 流量削峰

流量削峰也是消息队列的常用场景。我们做秒杀实现的时候，需要避免流量暴涨，打垮应用系统的风险。可以在应用前面加入消息队列。



假设秒杀系统每秒最多可以处理 2k 个请求，每秒却有 5k 的请求过来，可以引入消息队列，秒杀系统每秒从消息队列拉 2k 请求处理得了。

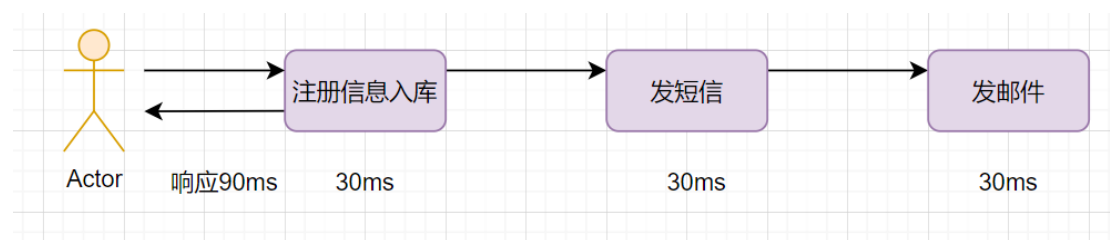
有些伙伴担心这样会出现消息积压的问题，

- 首先秒杀活动不会每时每刻都那么多请求过来，高峰期过去后，积压的请求可以慢慢处理；
- 其次，如果消息队列长度超过最大数量，可以直接抛弃用户请求或跳转到错误页面；

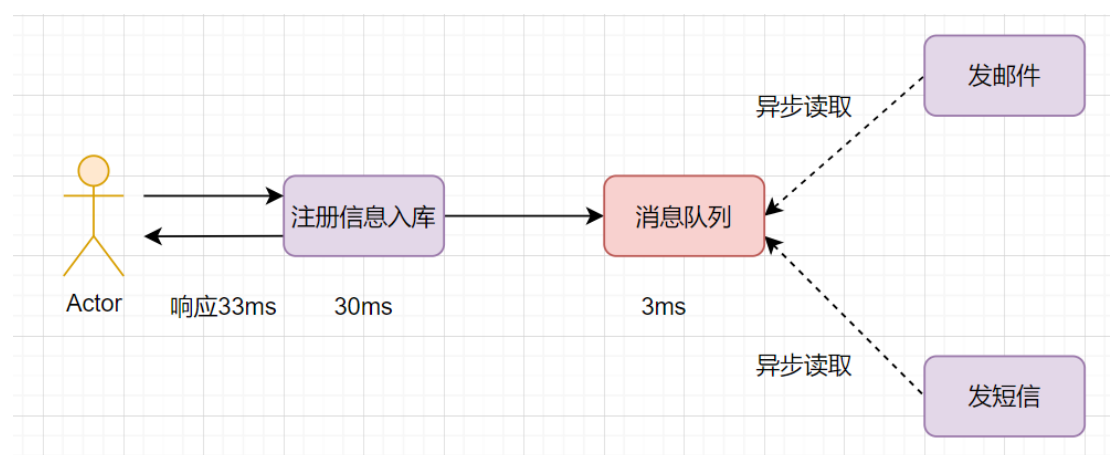
3.2.3 异步处理

我们经常会遇到这样的业务场景：用户注册成功后，给它发个短信和发个邮件。

如果注册信息入库是 30ms，发短信、邮件也是 30ms，三个动作串行执行的话，会比较耗时，响应 90ms：



如果采用并行执行的方式，可以减少响应时间。注册信息入库后，同时异步发短信和邮件。如何实现异步呢，用消息队列即可，就是说，注册信息入库成功后，写入到消息队列（这个一般比较快，如只需要 3ms），然后异步读取发邮件和短信。



3.2.4 消息通讯

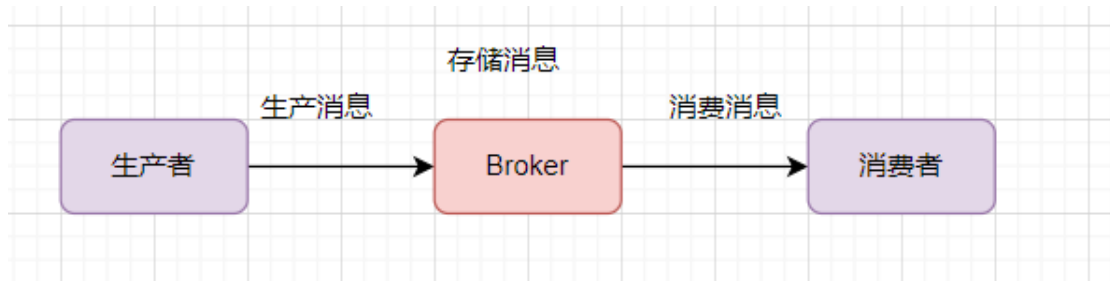
消息队列内置了高效的通信机制，可用于消息通讯。如实现点对点消息队列、聊天室等。

3.2.5 远程调用

我们公司基于 MQ，自研了远程调用框架。

3.3 消息队列如何解决消息丢失问题？

一个消息从生产者产生，到被消费者消费，主要经过这 3 个过程：



因此如何保证 MQ 不丢失消息，可以从这三个阶段阐述：

- 生产者保证不丢消息
- 存储端不丢消息
- 消费者不丢消息

3.3.1 生产者保证不丢消息

生产端如何保证不丢消息呢？确保生产的消息能到达存储端。

如果是 **RocketMQ** 消息中间件，Producer 生产者提供了三种发送消息的方式，分别是：

- 同步发送
- 异步发送
- 单向发送

生产者要想发消息时保证消息不丢失，可以：

- 采用**同步方式**发送，send 消息方法返回**成功**状态，就表示消息正常到达了存储端 Broker。
- 如果 send 消息**异常**或者返回**非成功**状态，可以**重试**。
- 可以使用事务消息，RocketMQ 的事务消息机制就是为了保证零丢失来设计的

3.3.2 存储端不丢消息

如何保证存储端的消息不丢失呢？确保消息**持久化到磁盘**。大家很容易想到就是**刷盘机制**。

刷盘机制分**同步刷盘**和**异步刷盘**：

- 生产者消息发过来时，只有持久化到磁盘，RocketMQ 的存储端 Broker 才返回一个成功的 ACK 响应，这就是**同步刷盘**。它保证消息不丢失，但是影响了性能。
- 异步刷盘的话，只要消息写入 PageCache 缓存，就返回一个成功的 ACK 响应。这样提高了 MQ 的性能，但是如果这时候机器断电了，就会丢失消息。

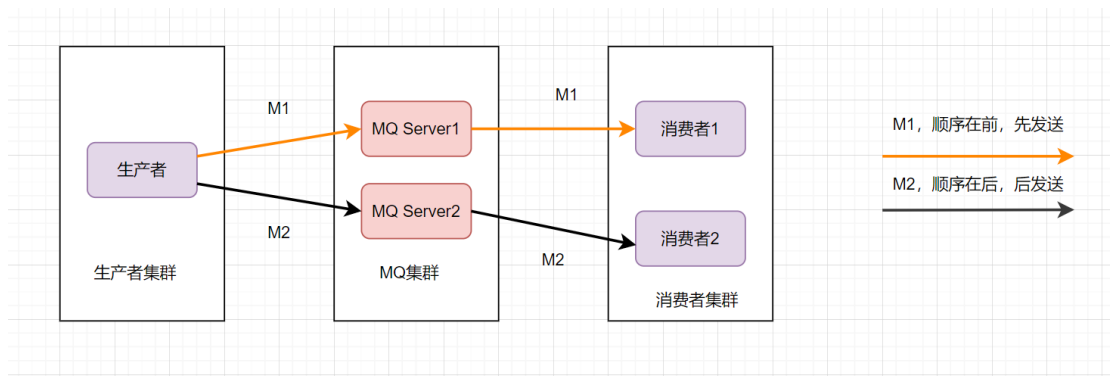
Broker 一般是**集群部署**的，有 master 主节点和 slave 从节点。消息到 Broker 存储端，只有主节点和从节点都写入成功，才反馈成功的 ack 给生产者。这就是**同步复制**，它保证了消息不丢失，但是降低了系统的吞吐量。与之对应的就是**异步复制**，只要消息写入主节点成功，就返回成功的 ack，它速度快，但是会有性能问题。

3.3.3 消费阶段不丢消息

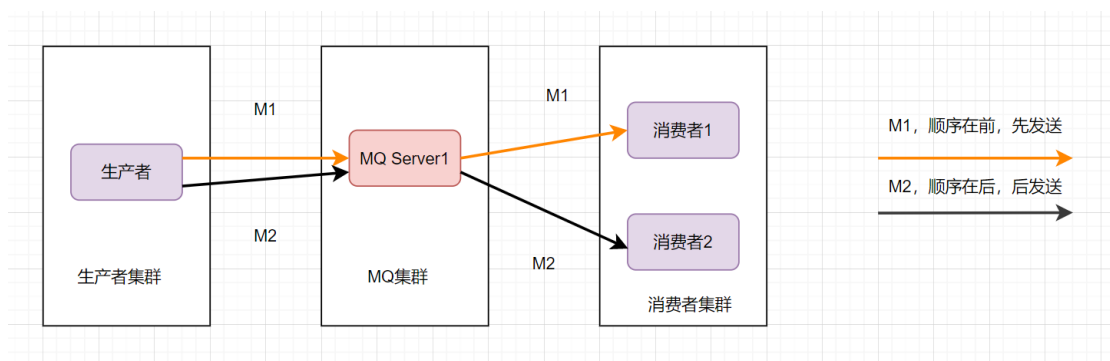
消费者执行完业务逻辑，再反馈给 Broker 说消费成功，这样才可以保证消费阶段不丢消息。

3.4 消息队列如何保证消息的顺序性。

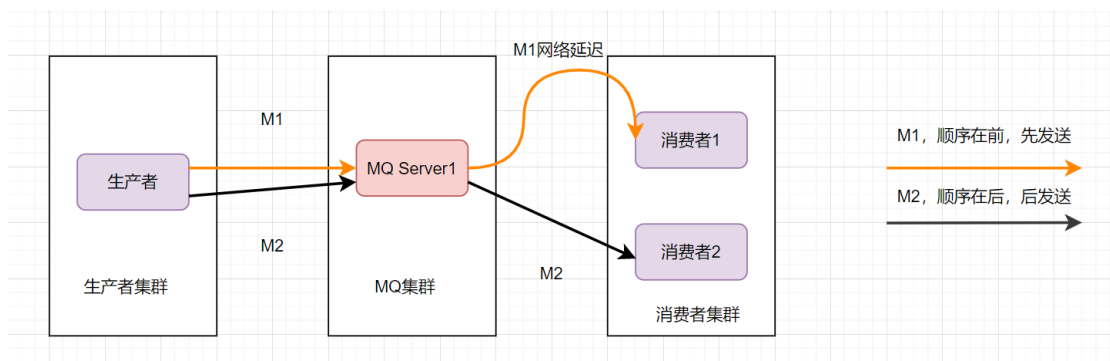
消息的有序性，就是指可以按照消息的发送顺序来消费。有些业务对消息的顺序是有要求的，比如**先下单再付款，最后再完成订单**，这样等。假设生产者先后产生了两条消息，分别是下单消息（M1），付款消息（M2），M1 比 M2 先产生，**如何保证 M1 比 M2 先被消费呢**。



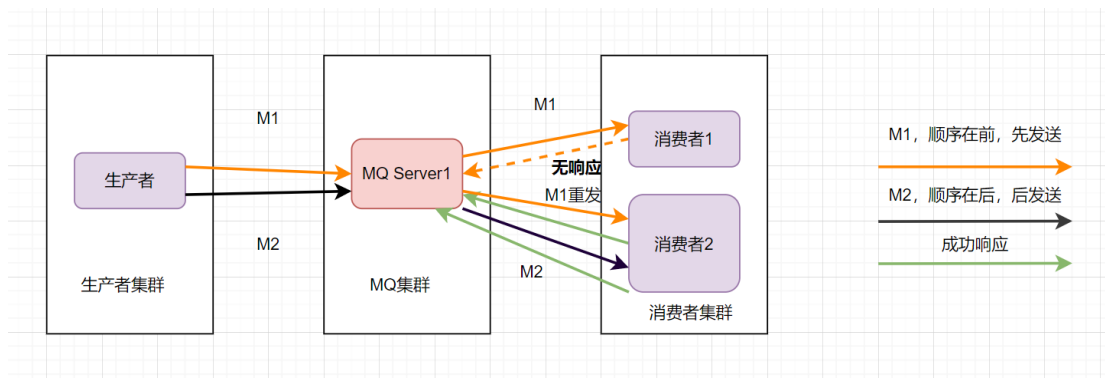
为了保证消息的顺序性，可以将 M1、M2 发送到同一个 Server 上，当 M1 发送完收到 ack 后，M2 再发送。如图：



这样还是可能会有问题，因为从 MQ 服务器到服务端，可能存在网络延迟，虽然 M1 先发送，但是它比 M2 晚到。



那还能怎么办才能保证消息的顺序性呢？将 M1 和 M2 发往同一个消费者，且发送 M1 后，等到消费端 ACK 成功后，才发送 M2 就得了。



消息队列保证顺序性**整体思路**就是这样啦。比如 Kafka 的全局有序消息，就是这种思想的体现：就是生产者发消息时，1 个 Topic 只能对应 1 个 Partition，一个 Consumer，内部单线程消费。

但是这样吞吐量太低，一般保证消息局部有序即可。在发消息的时候指定 Partition Key，Kafka 对其进行 Hash 计算，根据计算结果决定放入哪个 Partition。这样 Partition Key 相同的消息会放在同一个 Partition。然后多消费者单线程消费指定的 Partition。

3.5 消息队列有可能发生重复消费，如何避免，如何做到幂等？

消息队列是可能发生重复消费的。

- 生产端为了保证消息的可靠性，它可能往 MQ 服务器重复发送消息，直到拿到成功的 ACK。
- 再然后就是消费端，消费端消费消息一般是这个流程：拉取消息、业务逻辑处理、提交消费位移。假设业务逻辑处理完，事务提交了，但是需要更新消费位移时，消费者挂了，这时候另一个消费者就会拉到重复消息了。

如何幂等处理重复消息呢？

我之前写过一篇幂等设计的文章，大家有兴趣可以看下哈：[聊聊幂等设计](#)

幂等处理重复消息，简单来说，就是搞个本地表，带**唯一业务标记**的，利用主键或者唯一性索引，每次处理业务，先校验一下就好啦。又或者用 redis 缓存下业务标记，每次看下是否处理过了。

3.6 如何处理消息队列的消息积压问题

消息积压是因为生产者的生产速度，大于消费者的消费速度。遇到消息积压问题时，我们需要先排查，是不是有 bug 产生了。

如果不是 bug，我们可以**优化一下消费的逻辑**，比如之前是一条一条消息消费处理的话，我们可以确认是不是可以改为批量处理消息。如果还是慢，我们可以考虑水平扩容，增加 Topic 的队列数，和消费组机器的数量，提升整体消费能力。

如果是 bug 导致几百万消息持续积压几小时。有如何处理呢？需要解决 bug，**临时紧急扩容**，大概思路如下：

1. 先修复 consumer 消费者的问题，以确保其恢复消费速度，然后将现有 consumer 都停掉。
2. 新建一个 topic，partition 是原来的 10 倍，临时建立好原先 10 倍的 queue 数量。
3. 然后写一个临时的分发数据的 consumer 程序，这个程序部署上去消费积压的数据，消费之后不做耗时的处理，直接均匀轮询写入临时建立好的 10 倍数量的 queue。
4. 接着临时征用 10 倍的机器来部署 consumer，每一批 consumer 消费一个临时 queue 的数据。这种做法相当于是临时将 queue 资源和 consumer 资源扩大 10 倍，以正常的 10 倍速度来消费数据。
5. 等快速消费完积压数据之后，得恢复原先部署的架构，重新用原先的 consumer 机器来消费消息。

3.7 消息队列技术选型，Kafka 还是 RocketMQ，还是 RabbitMQ

先可以对比下它们优缺点：

	Kafka	RocketMQ	RabbitMQ
单机吞吐量	17.3w/s	11.6w/s	2.6w/s（消息做持久化）
开发语言	Scala/Java	Java	Erlang
主要维护者	Apache	Alibaba	Mozilla/Spring
订阅形式	基于 topic，按照 topic 进行正则匹配的发布订阅模式	基于 topic/messageTag，按照消息类型、属性进行正则匹配的发布订阅模式	提供了 4 种：direct, topic, Headers 和 fanout。fanout 就是广播模式
持久化	支持大量堆积	支持大量堆积	支持少量堆积
顺序消息	支持	支持	不支持
集群方式	天然的 Leader-Slave，无状态集群，每台服务器既是 Master 也是 Slave	常用多对'Master-Slave'模式，开源版本需手动切换 Slave 变成 Master	支持简单集群，'复制'模式，对高级集群模式支持不好。
性	较差	一般	好

	Kafka	RocketMQ	RabbitMQ
能 稳 定 性			

- RabbitMQ 是开源的，比较稳定的支持，活跃度也高，但是不是 Java 语言开发的。
- 很多公司用 RocketMQ，是阿里出品的。
- 如果是大数据领域的实时计算、日志采集等场景，用 Kafka 是业内标准的。

3.8 消息中间件如何做到高可用

消息中间件如何保证高可用呢？单机是没有高可用可言的，高可用都是对集群来说的，一起看下 kafka 的高可用吧。

Kafka 的基础集群架构，由多个 **broker** 组成，每个 **broker** 都是一个节点。当你创建一个 **topic** 时，它可以划分为多个 **partition**，而每个 **partition** 放一部分数据，分别存在于不同的 broker 上。也就是说，一个 topic 的数据，是分散放在多个机器上的，每个机器就放一部分数据。

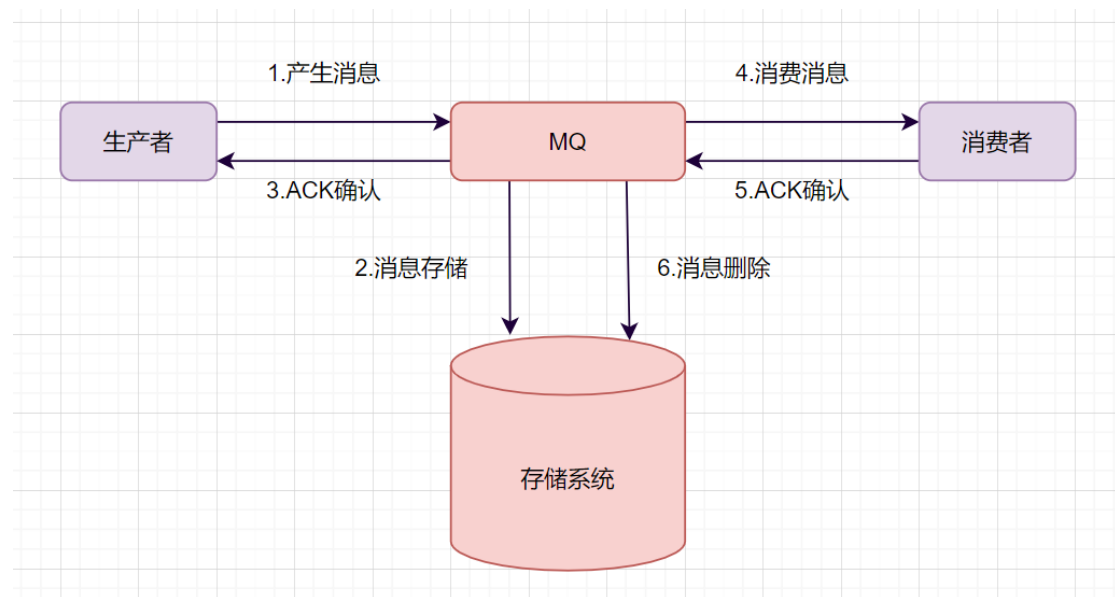
有些伙伴可能有疑问，每个 **partition** 放一部分数据，如果对应的 broker 挂了，那这部分数据是不是就丢失了？那还谈什么高可用呢？

Kafka 0.8 之后，提供了复制品副本机制来保证高可用，即每个 partition 的数据都会同步到其它机器上，形成多个副本。然后所有的副本会选举一个 leader 出来，让 leader 去跟生产和消费者打交道，其他副本都是 follower。写数据时，leader 负责把数据同步给所有的 follower，读消息时，直接读 leader 上的数据即可。如何保证高可用的？就是假设某个 broker 宕机，这个

broker 上的 partition 在其他机器上都有副本的。如果挂的是 leader 的 broker 呢？其他 follower 会重新选一个 leader 出来。

3.9 如何保证数据一致性，事务消息如何实现

一条普通的 MQ 消息，从产生到被消费，大概流程如下：

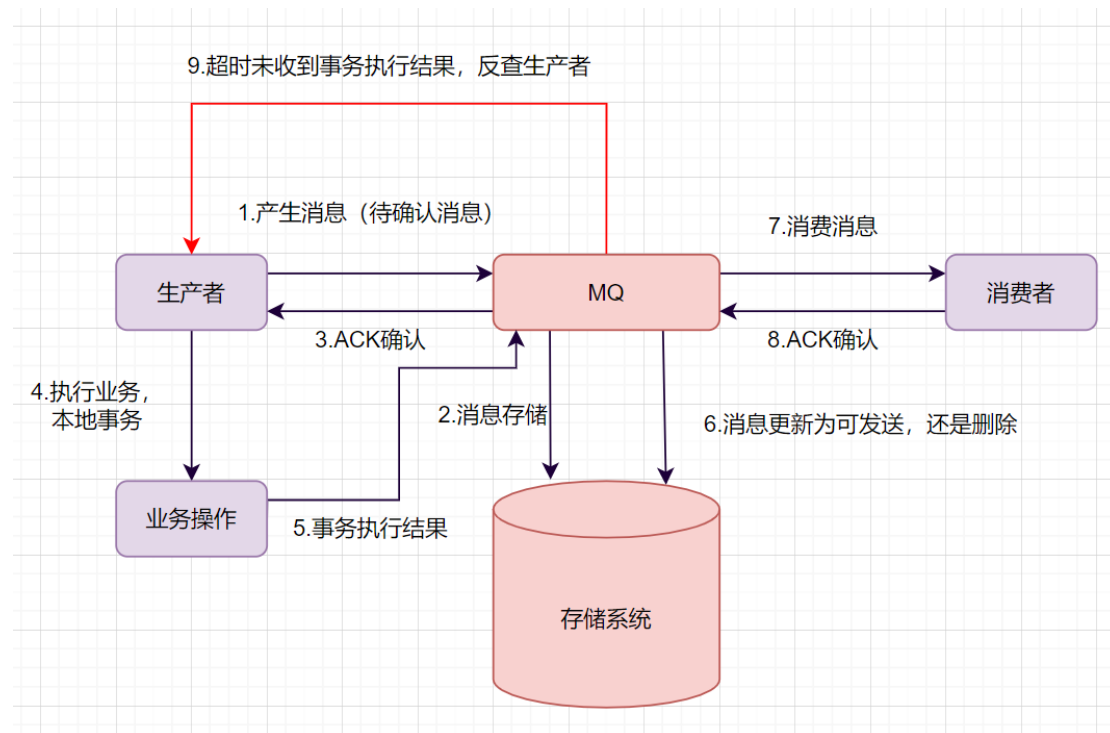


1. 生产者产生消息，发送带 MQ 服务器
2. MQ 收到消息后，将消息持久化到存储系统。
3. MQ 服务器返回 ACK 到生产者。
4. MQ 服务器把消息 push 给消费者
5. 消费者消费完消息，响应 ACK
6. MQ 服务器收到 ACK，认为消息消费成功，即在存储中删除消息。

我们举个下订单的例子吧。订单系统创建完订单后，再发送消息给下游系统。

如果订单创建成功，然后消息没有成功发送出去，下游系统就无法感知这个事情，出导致数据不一致。

如何保证数据一致性呢？可以使用**事务消息**。一起来看下事务消息是如何实现的吧。



1. 生产者产生消息，发送一条半事务消息到 MQ 服务器
2. MQ 收到消息后，将消息持久化到存储系统，这条消息的状态是待发送状态。
3. MQ 服务器返回 ACK 确认到生产者，此时 MQ 不会触发消息推送事件
4. 生产者执行本地事务
5. 如果本地事务执行成功，即 commit 执行结果到 MQ 服务器；如果执行失败，发送 rollback。
6. 如果是正常的 commit，MQ 服务器更新消息状态为可发送；如果是 rollback，即删除消息。
7. 如果消息状态更新为可发送，则 MQ 服务器会 push 消息给消费者。消费者消费完就回 ACK。
8. 如果 MQ 服务器长时间没有收到生产者的 commit 或者 rollback，它会反查生产者，然后根据查询到的结果执行最终状态。

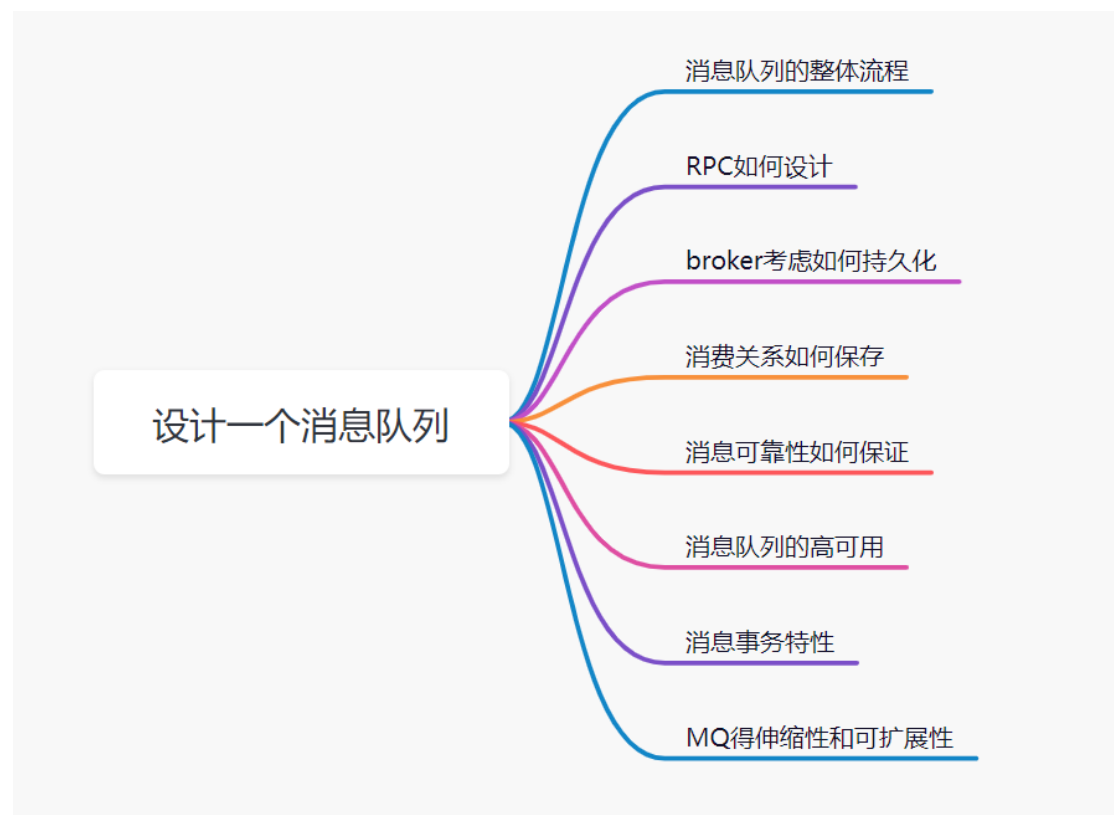
3.10 让你写一个消息队列，该如何进行架构设计？

这个问题面试官主要考察三个方面的知识点：

- 你有没有对消息队列的架构原理比较了解
- 考察你的个人设计能力
- 考察编程思想，如什么高可用、可扩展性、幂等等等。

遇到这种设计题，大部分人会很蒙圈，因为平时没有思考过类似的问题。大多数人平时埋头增删改啥，不去思考框架背后的一些原理。有很多类似的问题，比如让你来设计一个 Dubbo 框架，或者让你来设计一个 MyBatis 框架，你会怎么思考呢？

回答这类问题，并不要求你研究过那技术的源码，你知道那个技术框架的基本结构、工作原理即可。设计一个消息队列，我们可以从这几个角度去思考：



1. 首先是消息队列的整体流程，producer 发送消息给 broker，broker 存储好，broker 再发送给 consumer 消费，consumer 回复消费确认等。
2. producer 发送消息给 broker，broker 发消息给 consumer 消费，那就需要两次 RPC 了，RPC 如何设计呢？可以参考开源框架 Dubbo，你可以说说服务发现、序列化协议等等
3. broker 考虑如何持久化呢，是放文件系统还是数据库呢，会不会消息堆积呢，消息堆积如何处理呢。
4. 消费关系如何保存呢？点对点还是广播方式呢？广播关系又是如何维护呢？zk 还是 config server
5. 消息可靠性如何保证呢？如果消息重复了，如何幂等处理呢？

6. 消息队列的高可用如何设计呢？可以参考 Kafka 的高可用保障机制。多副本 -> leader & follower -> broker 挂了重新选举 leader 即可对外服务。
7. 消息事务特性，与本地业务同个事务，本地消息落库；消息投递到服务端，本地才删除；定时任务扫描本地消息库，补偿发送。
8. MQ 得伸缩性和可扩展性，如果消息积压或者资源不够时，如何支持快速扩容，提高吞吐？可以参照一下 Kafka 的设计理念，broker -> topic -> partition，每个 partition 放一个机器，就存一部分数据。如果现在资源不够了，简单啊，给 topic 增加 partition，然后做数据迁移，增加机器，不就可以存放更多数据，提供更高的吞吐量了？

之前写过消息队列经典十问，收到广大读者好评。大家可以看下哈，[消息队列](#)

[经典十连问](#)

4. 计算机网路专栏面试题（关注公众号：捡田螺的小男孩）

大家好，我是捡田螺的小男孩。

金九银十即将来临啦，我梳理了 60 道计算机网络面试题，每一道题目都特别经典，大厂也非常喜欢问。相信大家看完，会有新的收获滴~

4.1 说说 HTTP 常用的状态码及其含义？

思路：这道面试题主要考察候选人，是否掌握 HTTP 状态码这个基础知识点。

状态码	类别
1XX	信息性状态码
2XX	成功状态码
3XX	重定向状态码
4XX	客户端错误状态码
5XX	服务端错误状态码

不管是不是面试需要，我们都要知道，日常开发中的这几个状态码的含义哈：

状态码	含义
101	切换请求协议
200	请求成功
301	永久性重定向，会缓存
302	临时重定向，不会缓存
400	客户端请求的语法错误
403	服务器禁止访问，权限有关
404	服务器无法根据客户端的请求找到资源
500	服务端错误

4.2 HTTP 常用的请求方式，区别和用途？

思路：这道题主要考察候选人，是否掌握 **HTTP 请求方式** 这个基础知识点，我们用得比较多就是 **GET** 和 **POST** 啦。

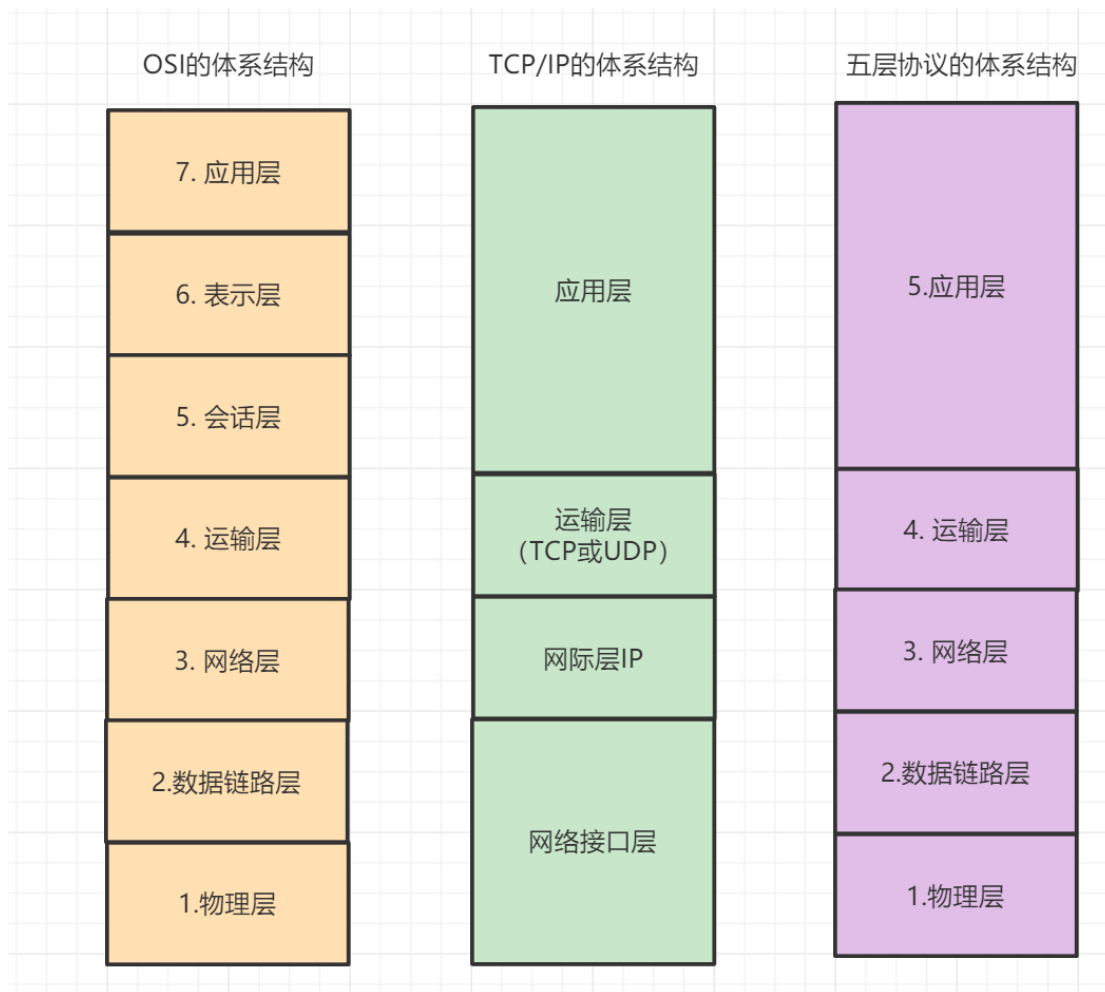
请求方式	用途
GET	对服务器资源获取的简单请求
POST	用于发送包含用户提交数据的请求
PUT	向服务器提交数据，以修改数据
HEAD	请求页面的首部，获取资源的元信息
DELETE	删除服务器上的某些资源
CONNECT	用于ssl隧道的基于代理的请求
OPTIONS	返回所有可用的方法，常用于跨域
TRACE	追踪请求-响应的传输路径

4.3 请简单说一下你了解的端口及对应的服务？

端口	服务
21	FTP(文件传输协议)
22	SSh
23	Telnet(远程登录)服务
25	SMTP（简单邮件传输协议）
53	DNS域名服务器
80	HTTP超文本传输协议
110	POP3邮件协议3
443	HTTPS
1080	Sockets
1521	Oracle数据库默认端口
3306	MySQL服务默认端口

4.4 说下计算机网络体系结构

思路：这道题主要考察候选人，**计算机网络体系结构**这个基础知识点。计算机网络体系结构呢，有三层：ISO 七层模型、TCP/IP 四层模型、五层体系结构。大家可以记住这个图，如下



计算机网络体系结构

4.4.1 OSI 七层模型

OSI (Open System Interconnect) ，即开放式系统互联。一般都叫 OSI 参考模型，是 ISO (国际标准化组织) 组织在 1985 年研究的网络互连模型，OSI 模型共分为七层

- 应用层：网络服务与最终用户的一个接口，常见的协议有：**HTTP FTP SMTP SNMP DNS**.
- 表示层：数据的表示、安全、压缩。，确保一个系统的应用层所发送的信息可以被另一个系统的应用层读取。
- 会话层：建立、管理、终止会话,对应主机进程，指本地主机与远程主机正在进行的会话.

- 传输层：定义传输数据的协议端口号，以及流控和差错校验,协议有 **TCP UDP**.
- 网络层：进行逻辑地址寻址，实现不同网络之间的路径选择,协议有 **ICMP IGMP IP** 等.
- 数据链路层：在物理层提供比特流服务的基础上，建立相邻结点之间的数据链路。
- 物理层：建立、维护、断开物理连接。

4.4.2 TCP/IP 四层模型

- 应用层：对应于 OSI 参考模型的（应用层、表示层、会话层）。
- 传输层:对应 OSI 的传输层，为应用层实体提供端到端的通信功能，保证了数据包的顺序传送及数据的完整性。
- 网际层：对应于 OSI 参考模型的网络层，主要解决主机到主机的通信问题。
- 网络接口层：与 OSI 参考模型的数据链路层、物理层对应。

4.4.3 五层体系结构

- 应用层：对应于 OSI 参考模型的（应用层、表示层、会话层）。
- 传输层：对应 OSI 参考模型的的传输层
- 网络层：对应 OSI 参考模型的的网络层
- 数据链路层：对应 OSI 参考模型的的数据链路层
- 物理层：对应 OSI 参考模型的物理层。

4.5 如何理解 HTTP 协议是无状态的

思路：这道题主要考察候选人，是否理解 Http 协议，它为什么是无状态的呢？

如何使它有状态呢？

如何理解无状态呢？

当浏览器第一次发送请求给服务器时，服务器响应了；如果同个浏览器发第二次请求给服务器时，它还是会响应，但是呢，服务器不知道你就是刚才的那个浏览器。简言之，服务器不会去记住你是谁，所以是无状态协议。

可以通过一个生活中的例子，来更好理解并记住它：

有状态场景：

- 小红：今天吃啥子？
- 小明：罗非鱼~
- 小红：味道怎么样呀？
- 小明：还不错，好香。

无状态的场景：

- 小红：今天吃啥子？
- 小明：罗非鱼~
- 小红：味道怎么样呀？
- 小明：？啊？你说啥？什么鬼？什么味道怎么样？

Http 加了 Cookie 的话：

- 小红：今天吃啥子？
- 小明：罗非鱼~
- 小红：你今天吃的罗非鱼，味道怎么样呀？
- 小明：还不错，好香。

我之前写过这么一篇文章，很形象生动，大家可以看看看：[十个形象比喻，助你理解计算机面试必备的知识点](#)

[你理解计算机面试必备的知识点](#)

4.6 从浏览器地址栏输入 url 到显示主页的过程

思路：这道题主要考察的知识点是 HTTP 的请求过程，**DNS 解析**，**TCP 三次握手**，**四次挥手**这几个要点，我们都可以讲下。

握手，四次挥手这几个要点，我们都可以讲下。

1. DNS 解析，查找域名对应的 IP 地址。
2. 与服务器通过三次握手，建立 TCP 连接
3. 向服务器发送 HTTP 请求
4. 服务器处理请求，返回网页内容
5. 浏览器解析并渲染页面
6. TCP 四次挥手，连接结束

过程	使用的协议
1.浏览器查找域名的IP地址 (DNS查找过程: 浏览器缓存、路由器缓存、DNS缓存)	DNS: 获取域名对应的IP
2.浏览器向web服务器发送一个HTTP请求 (cookies会随着请求发送给服务器)	TCP:与服务器建立TCP连接 IP: 建立TCP协议时, 需要发送数据, 发送的数据在网络层使用IP协议 OPSF:IP数据包在路由器之间, 路由选择使用OPSF协议 ARP:路由器在与服务器通信时, 需要将IP地址转换为MAC地址, 需要适应ARP协议 HTTP:在TCP连接完成后, 使用HTTP协议访问网页
3.服务器处理请求 (处理请求 & 它的参数、cookies、生成一个HTML响应)	
4.服务器发回一个HTML响应	
5.浏览器开始显示HTML	

4.7 说下 HTTP 1.0 , 1.1 , 2.0 的区别

思路: 这道题主要考察的知识点是 HTTP 几个版本的区别, 我们记住 HTTP 1.0 默认是短连接, 可以强制开启, HTTP 1.1 默认长连接, HTTP 2 采用多路复用就差不多啦。

我的答案如下:

HTTP/1.0

- 默认使用**短连接**, 每次请求都需要建立一个 TCP 连接。它可以设置 **Connection: keep-alive** 这个字段, 强制开启长连接。

HTTP/1.1

- 引入了持久连接, 即 TCP 连接默认不关闭, 可以被多个请求复用。
- 分块传输编码, 即服务端没产生一块数据, 就发送一块, 用“流模式”取代“缓存模式”。
- 管道机制, 即在同一个 TCP 连接里面, 客户端可以同时发送多个请求。

HTTP/2.0

- 二进制协议, 1.1 版本的头信息是文本 (ASCII 编码), 数据体可以是文本或者二进制; 2.0 中, 头信息和数据体都是二进制。
- 完全多路复用, 在一个连接里, 客户端和浏览器都可以同时发送多个请求或回应, 而且不用按照顺序一一对应。
- 报头压缩, HTTP 协议不带有状态, 每次请求都必须附上所有信息。Http 2.0 引入了头信息压缩机制, 使用 gzip 或 compress 压缩后再发送。
- 服务端推送, 允许服务器未经请求, 主动向客户端发送资源。

4.8 POST 和 GET 有哪些区别？

思路：这道题主要考察的知识点是 POST 和 GET 的区别，可以从数据包、编码方式、请求参数、收藏为书签、历史记录、安全性等几方面去回答哈。

请求方式	GET	POST
数据包	产生一个TCP数据包	可能产生两个TCP数据包
编码方式	只能进行url编码	支持多种编码方式
请求参数	把参数包含在URL中，用&连接起来	通过 request body 传递参数。
请求缓存	会被主动Cache	不会缓存，除非手动设置
收藏为书签	支持收藏为书签	不支持
历史记录	参数会被完整保留在浏览历史记录里	不会保留
参数数据类型	ASCII的字符	没有限制数据类型

4.9 在交互过程中如果数据传送完了，还不想断开连接怎么办，怎么维持？

这个问题记住 **keep-alive** 就好，也就是说，在 HTTP 中响应体的

Connection 字段指定为 **keep-alive** 即可

4.10 HTTP 如何实现长连接？在什么时候会超时？

思路：这道题实际上是考察 TCP 长连接的知识点，HTTP 的长连接实质是指

TCP 的长连接。至于什么时候超时，我们记住这几个参数如

tcp_keepalive_time、**tcp_keepalive_probes** 就好啦

什么是 HTTP 的长连接？

- 0. HTTP 分为长连接和短连接，其实本质上说的是 **TCP** 的长短连接。TCP 连接是一个双向的通道，它是可以保持一段时间不关闭的，因此 TCP 连接才具有真正的长连接和短连接这一说法哈。
-

0. TCP 长连接可以复用一個 TCP 连接，来发起多次的 HTTP 请求，这样就可以减少资源消耗，比如一次请求 HTML，如果是短连接的话，可能还需要请求后续的 JS/CSS。

如何设置长连接？

通过在头部（请求和响应头）设置 **Connection** 字段指定为 **keep-alive**，HTTP1.0 协议支持，但是是默认关闭的，从 HTTP 1.1 以后，连接默认都是长连接。

在什么时候会超时呢？

- HTTP 一般会有 httpd 守护进程，里面可以设置 **keep-alive timeout**，当 tcp 连接闲置超过这个时间就会关闭，也可以在 HTTP 的 header 里面设置超时时间
- TCP 的 **keep-alive** 包含三个参数，支持在系统内核的 net.ipv4 里面设置；当 TCP 连接之后，闲置了 **tcp_keepalive_time**，则会发生探测包，如果没有收到对方的 ACK，那么会每隔 **tcp_keepalive_intvl** 再发一次，直到发送了 **tcp_keepalive_probes**，就会丢弃该连接。

```
1. tcp_keepalive_intvl = 15
2. tcp_keepalive_probes = 5
3. tcp_keepalive_time = 1800
```

4.11 HTTP 与 HTTPS 的区别。

思路：这道题实际上考察的知识点是 HTTP 与 HTTPS 的区别，个人觉得这个知识点非常重要，可以从**安全性、数据是否加密、默认端口**等这几个方面去回答哈。其实，当你理解 HTTPS 的整个流程，就可以很好回答这个问题啦。

我的答案如下：

HTTP，即超文本传输协议，是一个基于 TCP/IP 通信协议来传递明文数据的协议。HTTP 会存在这几个问题：

- 请求信息是明文传输，容易被窃听截取。

- 没有验证对方身份，存在被冒充的风险
- 数据的完整性未校验，容易被中间人篡改

为了解决 Http 存在的问题，Https 它出现啦。

Https 是什么？

HTTPS = HTTP + SSL/TLS，可以理解 Https 是身披 SSL(Secure Socket Layer，安全套接层)的 HTTP。

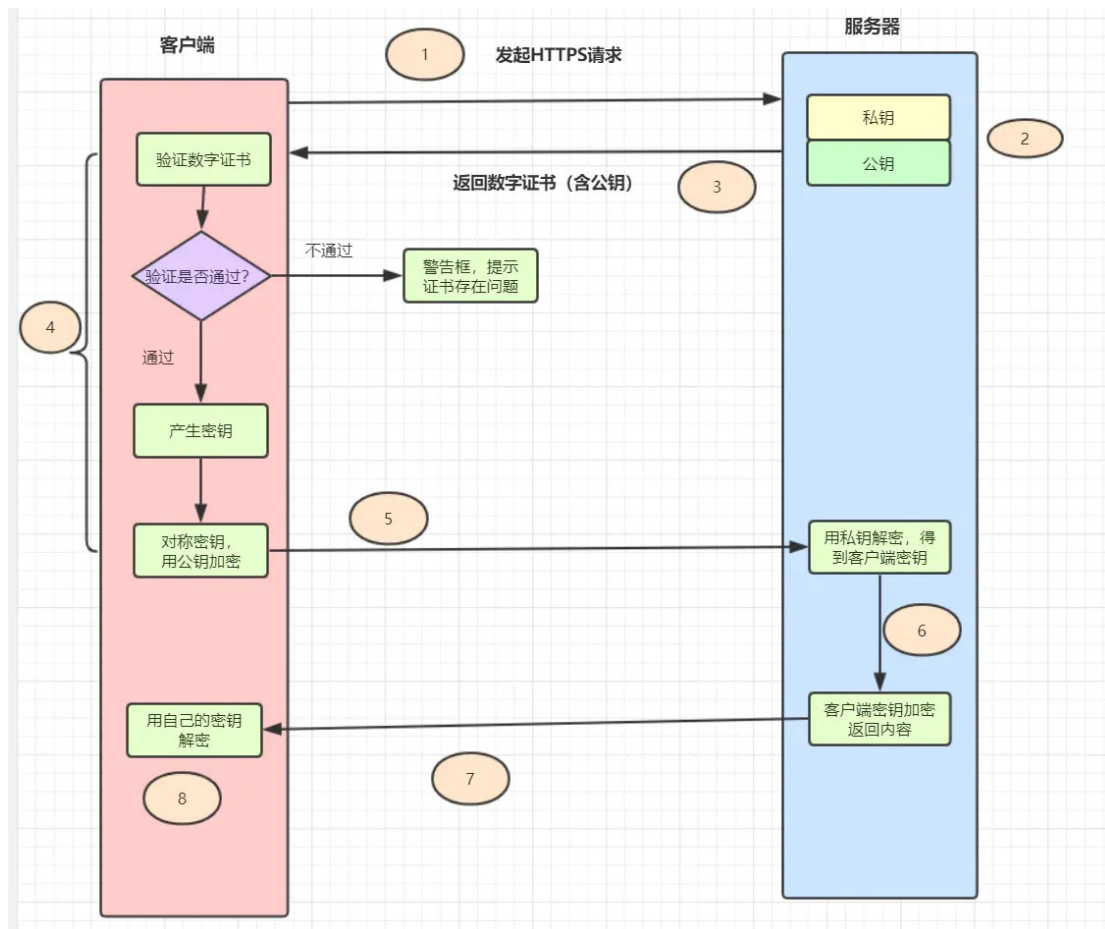
它们主要区别如下：

	Http	Https
安全性	不安全	安全
默认端口	80	443
资源消耗	较少	消耗资源较多
是否需要证书	不需要	需要
报文是否加密	明文	密文

4.12 Https 流程是怎样的？

思路：这道题实际上考察的知识点是 HTTPS 的工作流程，大家需要回答这几个要点，**公私钥、数字证书、加密、对称加密、非对称加密**。

- HTTPS = HTTP + SSL/TLS，也就是用 SSL/TLS 对数据进行加密和解密，Http 进行传输。
- SSL，即 Secure Sockets Layer (安全套接层协议)，是网络通信提供安全及数据完整性的一种安全协议。
- TLS，即 Transport Layer Security(安全传输层协议)，它是 SSL3.0 的后续版本。



Https 工作流程

1. 客户端发起 Https 请求，连接到服务器的 443 端口。
2. 服务器必须要有一套数字证书（证书内容有公钥、证书颁发机构、失效日期等）。
3. 服务器将自己的数字证书发送给客户端（公钥在证书里面，私钥由服务器持有）。
4. 客户端收到数字证书之后，会验证证书的合法性。如果证书验证通过，就会生成一个随机的对称密钥，用证书的公钥加密。
5. 客户端将公钥加密后的密钥发送到服务器。
6. 服务器接收到客户端发来的密文密钥之后，用自己之前保留的私钥对其进行非对称解密，解密之后就得到客户端的密钥，然后用客户端密钥对返回数据进行对称加密，酱紫传输的数据都是密文啦。
7. 服务器将加密后的密文返回到客户端。
8. 客户端收到后，用自己的密钥对其进行对称解密，得到服务器返回的数据。

4.13 说说 HTTP 的状态码，301 和 302 的区别？

思路：这道题考查的知识点，也是 HTTP 状态码，302 和 301 都有重定向的含义，但是它们也是有区别的。

- 301：（永久性转移）请求的网页已被永久移动到新位置。服务器返回此响应时，会自动将请求者转到新位置。
- 302：（暂时性转移）服务器目前正从不同位置的网页响应请求，但请求者应继续使用原有位置来进行以后的请求。此代码与响应 GET 和 HEAD 请求的 301 代码类似，会自动将请求者转到不同的位置。

网上有个很**形象的例子**比喻：

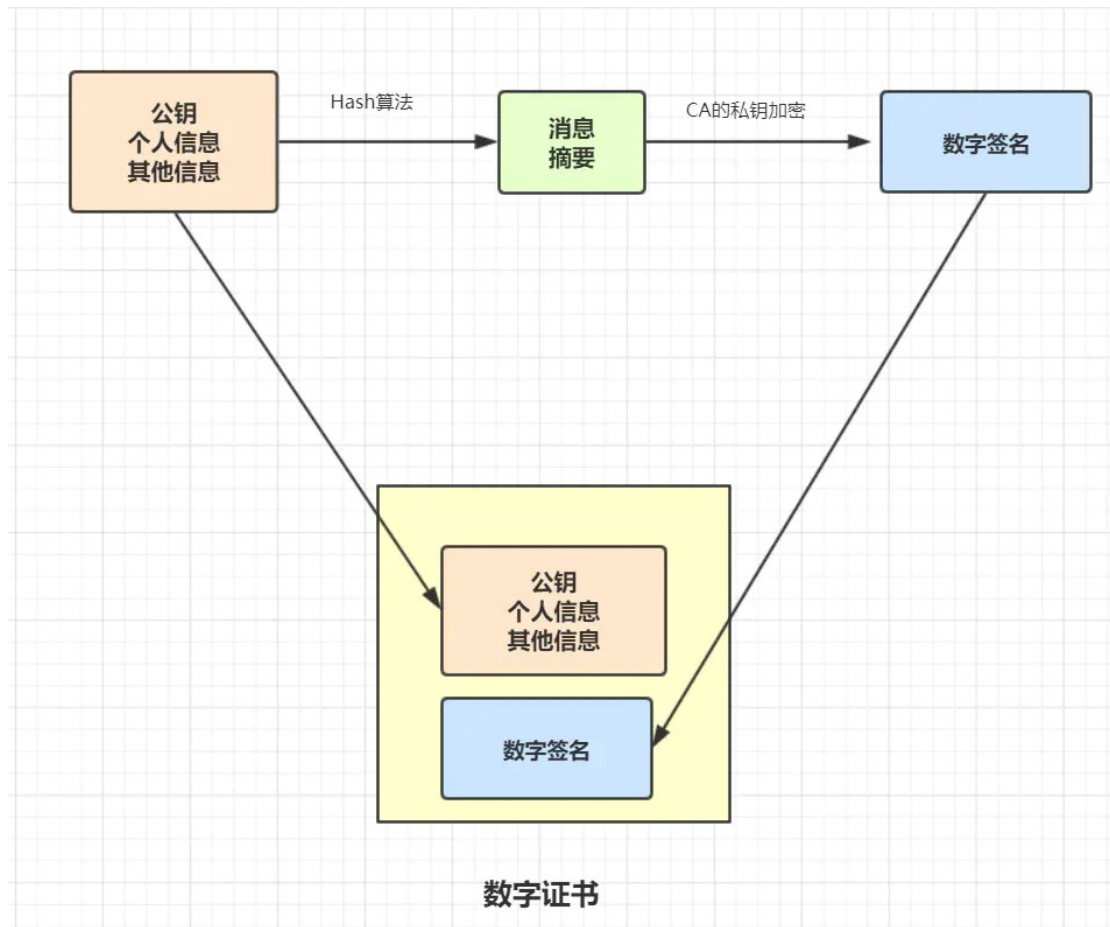
当一个网站或者网页 24—48 小时内临时移动到一个新的位置，这时候就要进行 302 跳转，打个比方说，我有一套房子，但是最近走亲戚去亲戚家住了，过两天我还回来的。而使用 301 跳转的场景就是之前的网站因为某种原因需要移除掉，然后要到新的地址访问，是永久性的，就比如你的那套房子其实是租的，现在租期到了，你又在另一个地方找到了房子，之前租的房子不住了。

4.14 说说什么是数字签名？什么是数字证书？

思路：这道题考查的知识点，不仅仅是数字签名，数字证书，很可能面试官也会问你 https 的原理的，因为 https 原理跟数字证书有关哈，大家需要掌握 https 原理哈。

数字证书是指在互联网通讯中标志通讯各方身份信息的一个数字认证，人们可以在网上用它来识别对方的身份。它的出现，是为了避免身份被篡改冒充的。

比如 Https 的数字证书，就是为了避免公钥被中间人冒充篡改：



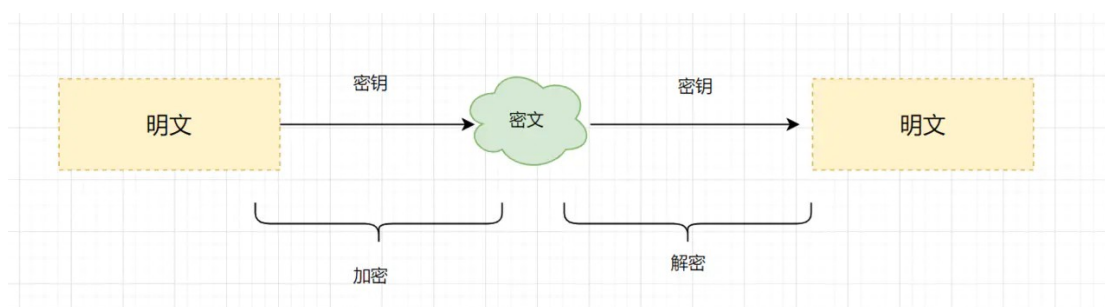
数字证书构成

- 公钥和个人等信息，经过 Hash 摘要算法加密，形成消息摘要；将消息摘要拿到拥有公信力的认证中心（CA），用它的私钥对消息摘要加密，形成**数字签名**。
- 公钥和个人信息、数字签名共同构成**数字证书**。

4.15 对称加密与非对称加密有什么区别

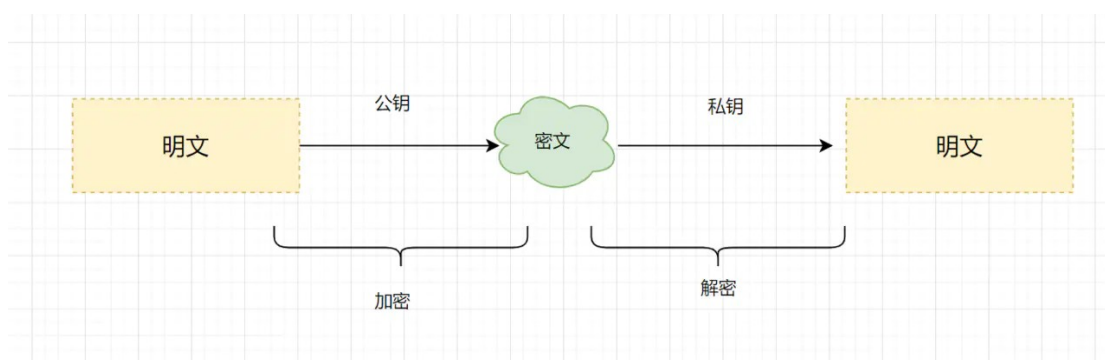
思路：这道题考察的知识点是对称加密与非对称加密算法，什么是对称加密，什么是非对称加密呢？

对称加密：指加密和解密使用同一密钥，优点是运算速度较快，缺点是如何安全将密钥传输给另一方。常见的对称加密算法有：DES、AES 等。



对称加密

非对称加密：指的是加密和解密使用不同的密钥（即公钥和私钥）。公钥与私钥是成对存在的，如果用公钥对数据进行加密，只有对应的私钥才能解密。常见的非对称加密算法有 RSA。



非对称加密

我之前这篇文章介绍过，可以详细看一下哈，[程序员必备基础：加签验签](#)

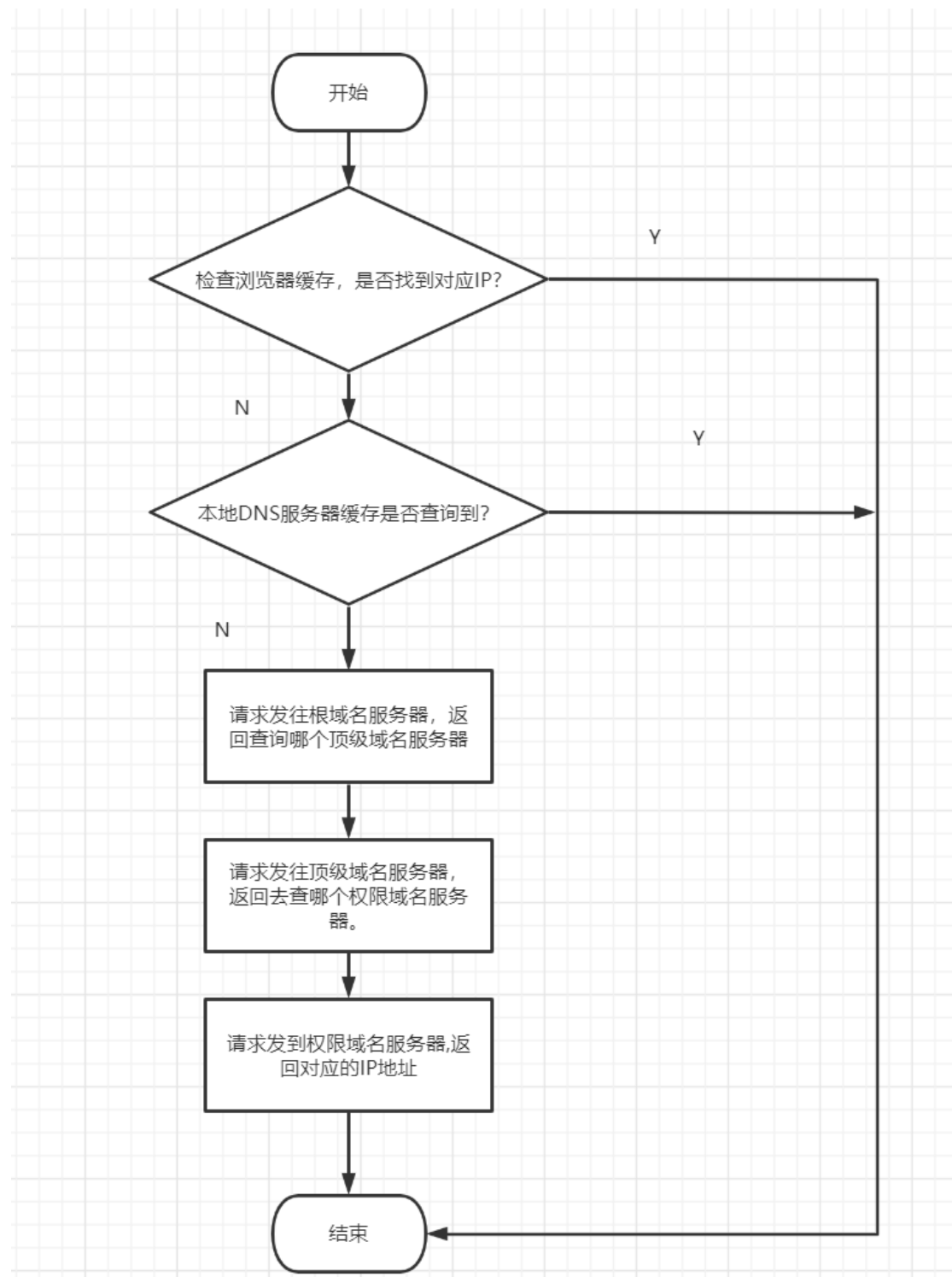
4.16 说说 DNS 的解析过程？

思路：这道题考察的知识点是 **DNS 域名解析**，http 请求的过程，是涉及到 DNS 域名解析的，这道面试题也挺经典的，大家可以看下《图解 HTTP》那本书哈。

DNS，英文全称是 **domain name system**，域名解析系统，是 Internet 上作为域名和 IP 相互映射的一个分布式数据库。它的作用很明确，就是可以根

据域名查出对应的 IP 地址。在浏览器缓存、本地 DNS 服务器、根域名服务器都是怎么查找的，大家回答的时候都可以说下哈。

DNS 的解析过程如下图：



DNS 解析查找过程

假设你要查询 **www.baidu.com** 的 IP 地址:

- 首先会查找浏览器的缓存,看看是否能找到 **www.baidu.com** 对应的 IP 地址,找到就直接返回;否则进行下一步。
- 将请求发往给本地 DNS 服务器,如果查找到也直接返回,否则继续进行下一步;
- 本地 DNS 服务器向**根域名服务器**发送请求,根域名服务器返回负责**.com**的顶级域名服务器的 IP 地址的列表。
- 本地 DNS 服务器再向其中一个负责**.com**的顶级域名服务器发送一个请求,返回负责**.baidu**的权威域名服务器的 IP 地址列表。
- 本地 DNS 服务器再向其中一个权威域名服务器发送一个请求,返回 **www.baidu.com** 所对应的 IP 地址。

4.17 什么是 CSRF 攻击, 如何避免

思路: 这道题考察的知识点是 **CSRF 攻击**, 它是属于网络安全这块的知识点, 还有 **Xss 攻击**、**SQL 注入**、**DDoS** 等这些常见的网络攻击, 我们都需要知道攻击的流程哈。

什么是 CSRF 攻击?

CSRF, 跨站请求伪造 (英文全称是 Cross-site request forgery), 是一种挟制用户在当前已登录的 Web 应用程序上执行非本意的操作的攻击方法。。

CSRF 是如何攻击的呢?

来看一个来自百度百科的例子哈:

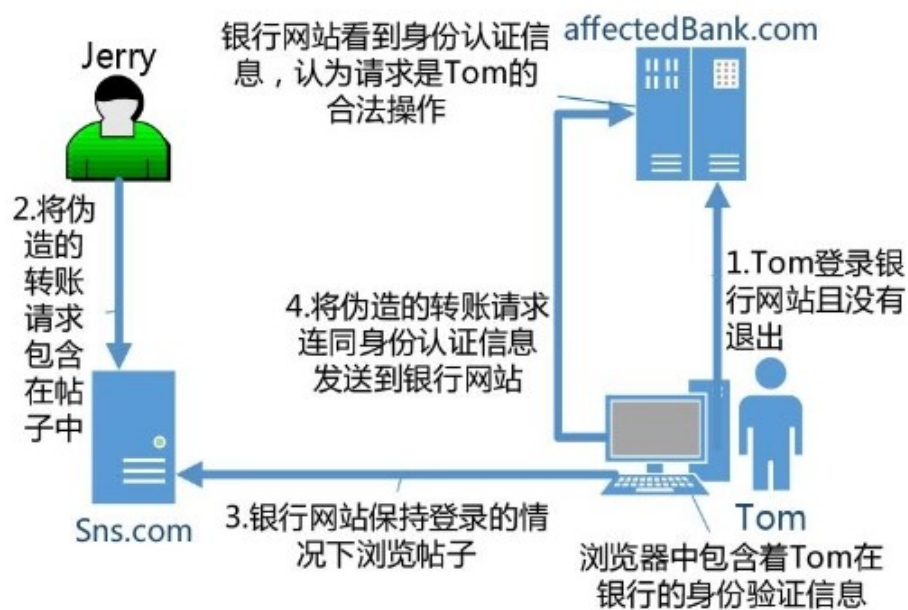


图1 典型 CSRF 的例子

- 0. Tom 登陆银行，没有退出，浏览器包含了 Tom 在银行的身份认证信息。
- 0. 黑客 Jerry 将伪造的转账请求，包含在在帖子
- 0. Tom 在银行网站保持登陆的情况下，浏览帖子
- 0. 将伪造的转账请求连同身份认证信息，发送到银行网站
- 0. 银行网站看到身份认证信息，以为就是 Tom 的合法操作，最后造成 Tom 资金损失。

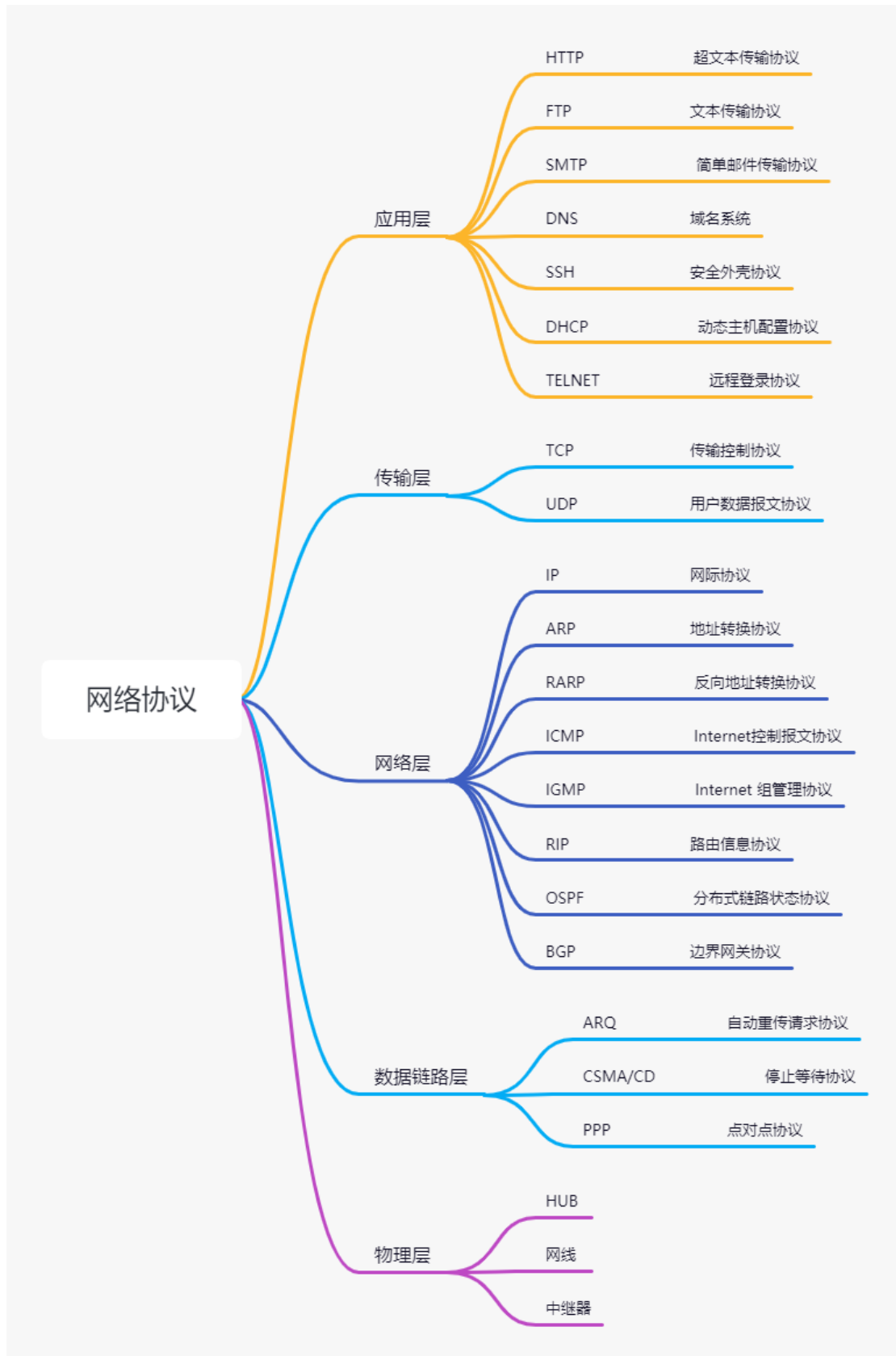
怎么解决 CSRF 攻击呢？

- 检查 Referer 字段。
- 添加校验 token。

有关于安全漏洞问题，我写过这篇：[程序员必备基础：10 种常见安全漏洞浅析](#)

4.18 聊聊五层计算机网络体系结构中，每一层对应的网络协议有哪些？

为了大家方便记忆，我还是画个思维导图吧，如下：



4.19 说说 WebSocket 与 socket 的区别

思路: 这是一个比较基础的知识点，经常有小伙伴会搞混。

- Socket 其实就是等于 **IP 地址 + 端口 + 协议**。

具体来说，Socket 是一套标准，它完成了对 TCP/IP 的高度封装，屏蔽网络细节，以方便开发者更好地进行网络编程。

- WebSocket 是一个持久化的协议，它是伴随 H5 而出的协议，用来解决 **http 不支持持久化连接** 的问题。
- Socket 一个是**网编编程的标准接口**，而 WebSocket 则是应用层通信协议。

4.20 什么是 DoS、DDoS、DRDoS 攻击？

思路: 这是涉及网络安全的一个知识点，DDos 还会挺常见的，如 SYN Flood。

- **DOS:** (Denial of Service), 翻译过来就是拒绝服务, 一切能引起 DOS 行为的攻击都被称为 DOS 攻击。最常见的 DoS 攻击就有**计算机网络宽带攻击、连通性攻击**。
- **DDoS:** (Distributed Denial of Service), 翻译过来是分布式拒绝服务。是指处于不同位置的多个攻击者同时向一个或几个目标发动攻击，或者一个攻击者控制了位于不同位置的多台机器并利用这些机器对受害者同时实施攻击。常见的 DDos 有 **SYN Flood、Ping of Death、ACK Flood、UDP Flood** 等。
- **DRDoS:** (Distributed Reflection Denial of Service), 中文是分布式反射拒绝服务，该方式靠的是发送大量带有被害者 IP 地址的数据包给攻击主机，然后攻击主机对 IP 地址源做出大量回应，从而形成拒绝服务攻击。

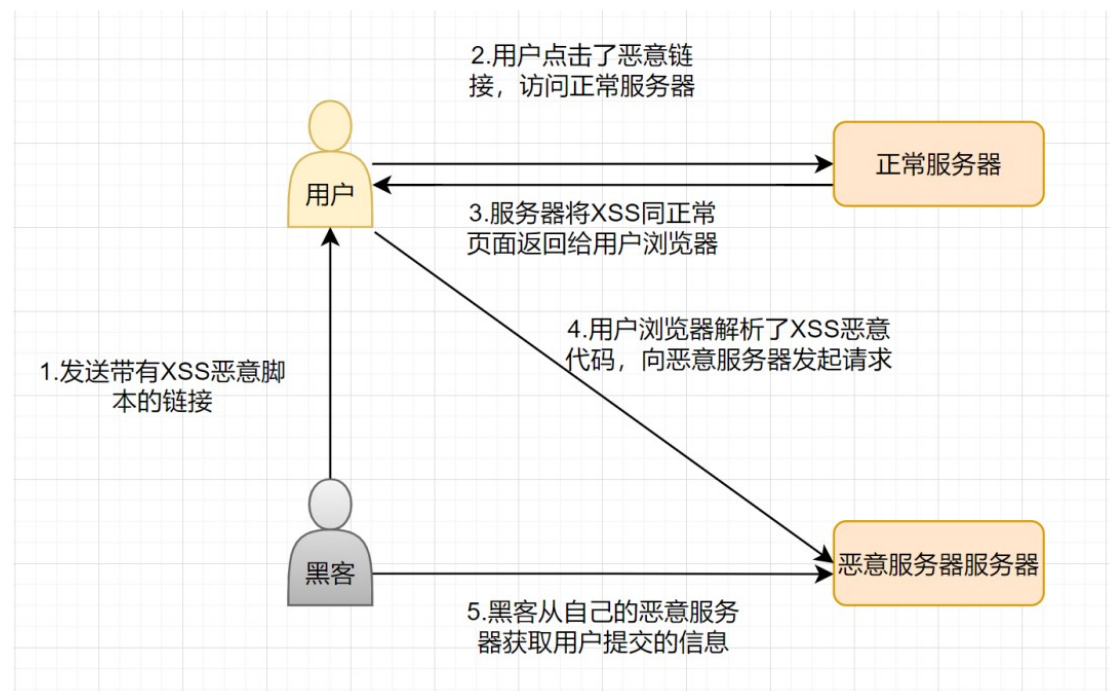
4.21 什么是 XSS 攻击，如何避免？

思路: XSS 攻击也是比较常见，XSS，叫**跨站脚本攻击 (Cross-Site Scripting)**，因为会与层叠样式表(Cascading Style Sheets, CSS)的缩写

混淆，因此有人将跨站脚本攻击缩写为 XSS。它指的是恶意攻击者往 Web 页面里插入恶意 html 代码，当用户浏览该页之时，嵌入其中 Web 里面的 html 代码会被执行，从而达到恶意攻击用户的特殊目的。XSS 攻击一般分三种类型：
存储型、反射型、DOM 型 XSS

4.22.1 XSS 是如何攻击的呢？

拿反射型举个例子吧，流程图如下：



4.22.2 如何解决 XSS 攻击问题？

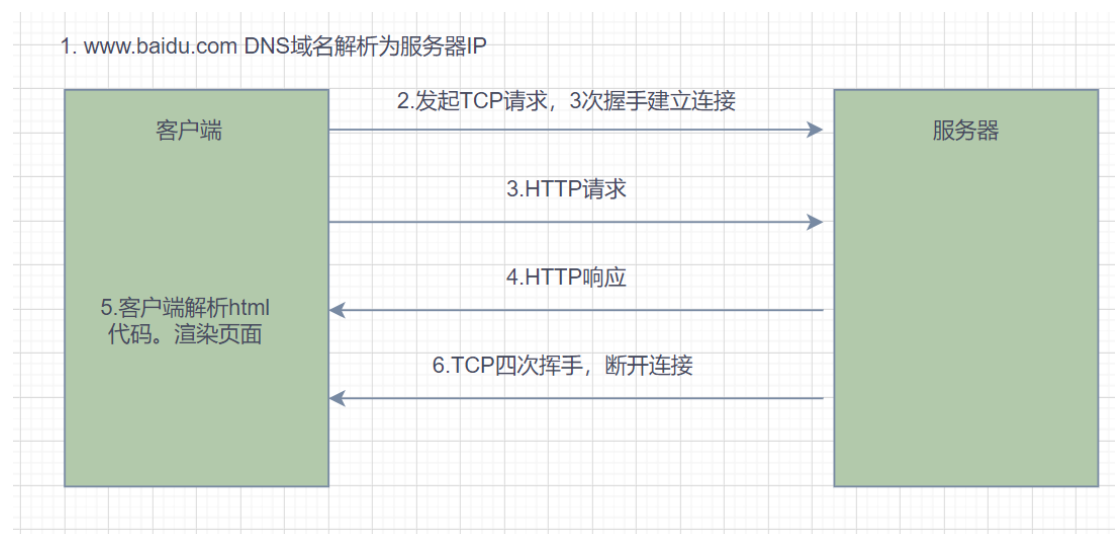
- 首先，就不能相信用户的输入，对输入进行过滤，过滤标签等，只允许合法值。
- HTML 转义
- 对于链接跳转，如 `<a href="xxx">` 等，要校验内容，禁止以 script 开头的非法链接。
- 限制输入长度等等

4.23 Http 请求的过程与原理

思路：HTTP 请求，一个非常非常基础的知识点，一定需要掌握的。其实觉得跟浏览器地址栏输入 url 到显示主页这道题有点类似。

我的答案如下：

HTTP 是一个基于 TCP/IP 协议来传递数据的超文本传输协议，传输的数据类型有 HTML,图片等。以访问百度有例子，看下一次 Http 的请求过程吧



Http 请求过程

1. 客户端进行 DNS 域名解析，得到对应的 IP 地址
2. 根据这个 IP，找到对应的服务器建立连接（三次握手）
3. 建立 TCP 连接后发起 HTTP 请求（一个完整的 http 请求报文）
4. 服务器响应 HTTP 请求，客户端得到 html 代码
5. 客户端解析 html 代码，用 html 代码中的资源(如 js,css,图片等等)渲染页面。
6. 服务器关闭 TCP 连接（四次挥手）

4.24 forward 和 redirect 的区别？

思路：这道题有点偏 Java web 方向的。以前记得刚出来实习找工作的时候，面试官可喜欢问这道题啦，当时我记的答案就是，forward 是转发，redirect 是重定向。

我的答案如下：

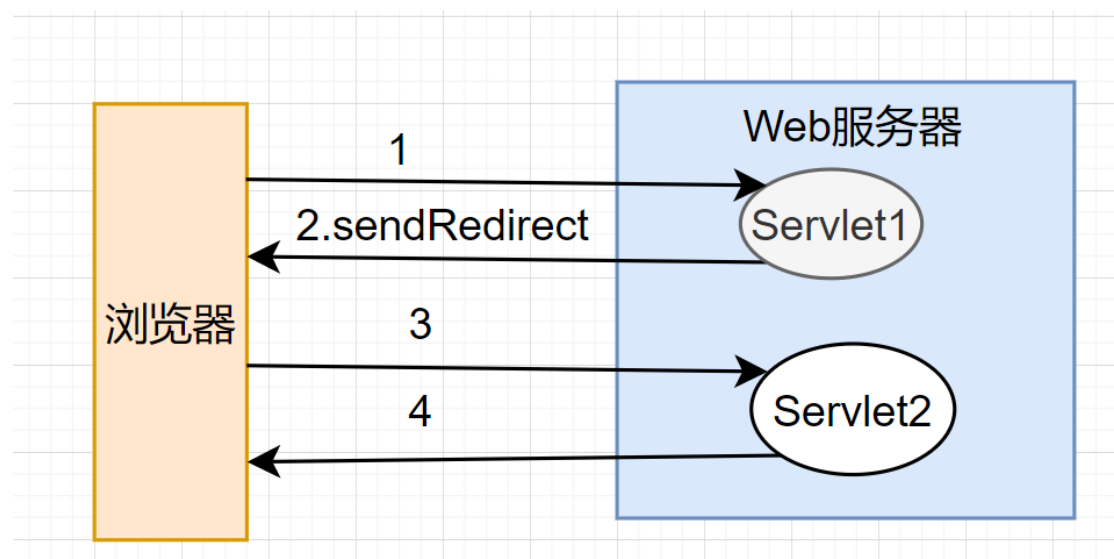
- **直接转发方式 (Forward)**，客户端和浏览器只发出一次请求，Servlet、HTML、JSP 或其它信息资源，由第二个信息资源响应该请求，在请求对象 request 中，保存的对象对于每个信息资源是共享的。
- **间接转发方式 (Redirect)** 实际是两次 HTTP 请求，服务器端在响应第一次请求的时候，让浏览器再向另外一个 URL 发出请求，从而达到转发的目的。

举个通俗的例子：

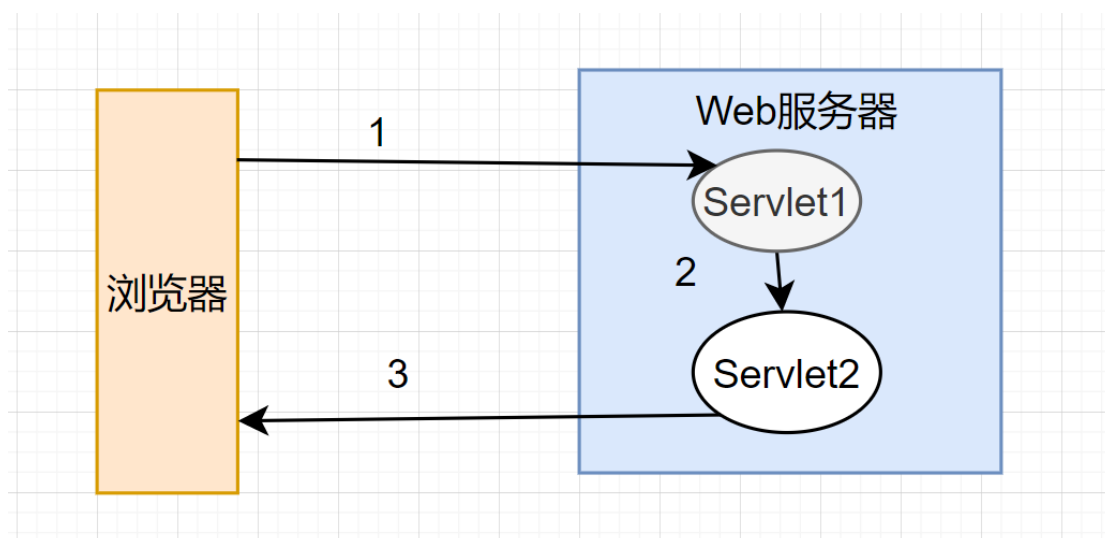
- 直接转发就相当于：“A 找 B 借钱，B 说没有，B 去找 C 借，借到借不到都会把消息传递给 A”；
- 间接转发就相当于：“A 找 B 借钱，B 说没有，让 A 去找 C 借”。**

看下这两个图，可以更容易理解一些：

- Redirect 的工作原理：



- forward 的工作原理



4.25 聊聊 SQL 注入？

思路：SQL 注入是最经典的安全问题。无论你是前端开发还是后端开发，都必须掌握的。

SQL 注入是一种代码注入技术，一般被应用于攻击 web 应用程序。它通过在 web 应用接口传入一些特殊参数字符，来欺骗应用服务器，执行恶意的 SQL 命令，以达到非法获取系统信息的目的。它目前是黑客对数据库进行攻击的最常用手段之一。

4.26.1 SQL 注入是如何攻击的？

举个常见的业务场景：在 web 表单搜索框输入员工名字，然后后台查询出对应名字的员工。

员工查询

田螺	搜索
捡田螺的小男孩	

这种场景下，一般都是前端页面,把一个名字参数 name 传到后台，然后后台通过 SQL 把结果查询出来

```
name = "田螺"; //前端传过来的
```

```
SQL= "select * from staff where name=" + name; //根据前端传过来的 name 参数，查询数据库  
员工表 staff
```

因为 SQL 是直接拼接的，如果我们完全信任前端传的参数说的话。假如前端传这么一个参数时 `' or '1'='1'`，SQL 就变成酱紫的啦。

```
select * from staff where name=' or '1'='1';
```

这个 SQL 会把所有的员工信息全都查出来了，酱紫就请求用户已经越权啦。请求者可以获取所有员工的信息，信息已经暴露了啦。

4.27.2 如何预防 SQL 注入问题

1). 使用#{ }而不是 \${ }

在 MyBatis 中,使用#{ }而不是\${ }，可以很大程度防止 sql 注入。

- 因为#{ }是一个参数占位符，对于字符串类型，会自动加上""，其他类型不加。由于 Mybatis 采用预编译，其后的参数不会再进行 SQL 编译，所以一定程度上防止 SQL 注入。
- \${ }是一个简单的字符串替换，字符串是什么，就会解析成什么，存在 SQL 注入风险

2). 不要暴露一些不必要的日志或者安全信息，比如避免直接响应一些 sql 异常信息。

如果 SQL 发生异常了，不要把这些信息暴露响应给用户，可以自定义异常进行响应

3). 不相信任何外部输入参数，过滤参数中含有的一些数据库关键词关键词

可以加个参数校验过滤的方法，过滤 union , or 等数据库关键词

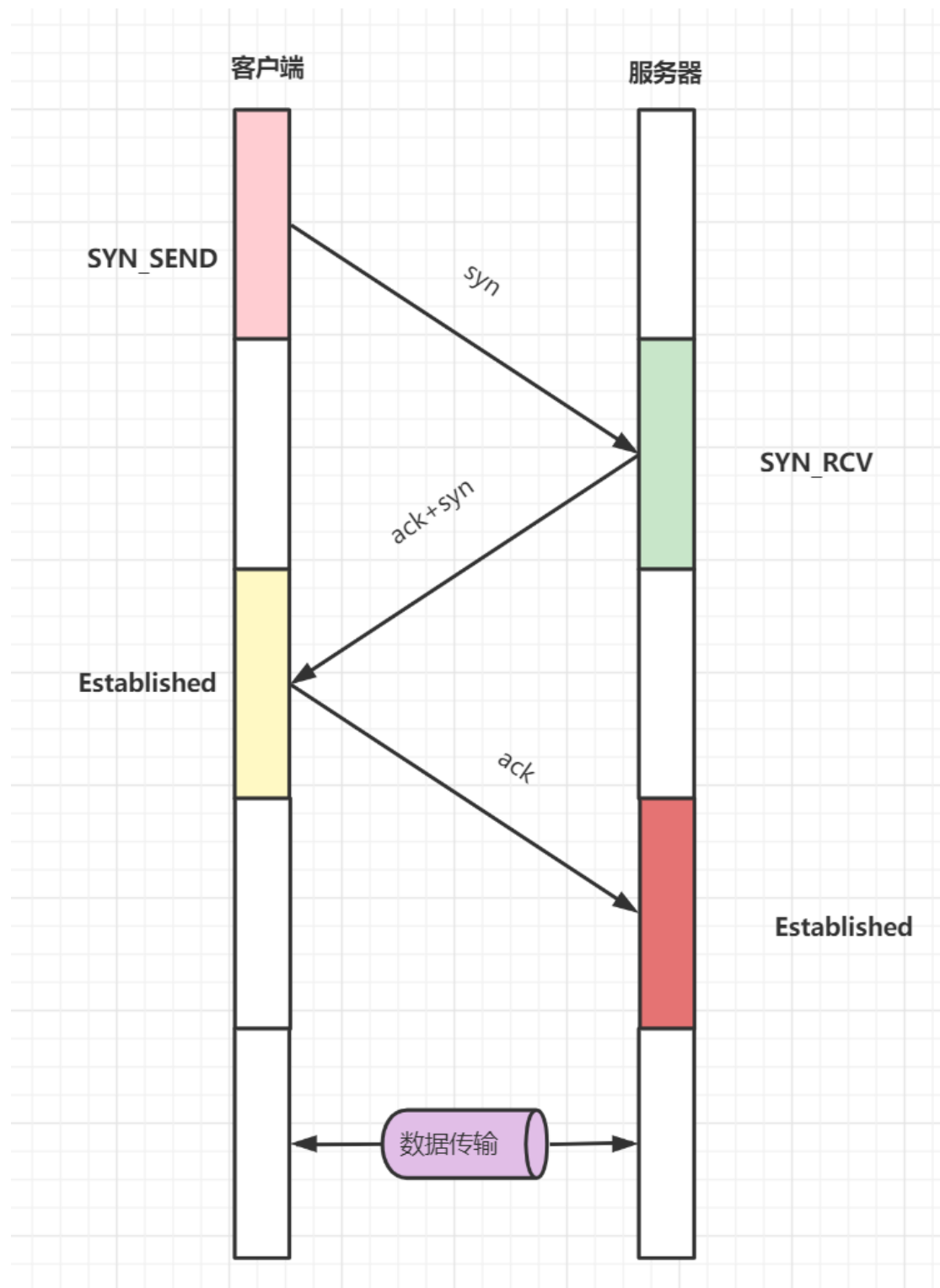
4). 适当的权限控制

在你查询信息时，先校验下当前用户是否有这个权限。比如说，实现代码的时候，可以让用户多传一个企业 Id 什么的，或者获取当前用户的 session 信息等，在查询前，先校验一下当前用户是否是这个企业下的等等，是的话才有这个查询员工的权限。

4.28 请详细介绍一下 TCP 的三次握手机制

思路： TCP 连接的三次握手机制，最重要的知识点，必须得会，通讯过程以及客户端、服务器的对应的状态都需要记住哈。

TCP 提供可靠的连接服务，连接是通过三次握手进行初始化的。三次握手的目的就是同步连接双方的序列号和确认号并交换 TCP 窗口大小信息。我们一起来看看下流程图哈：

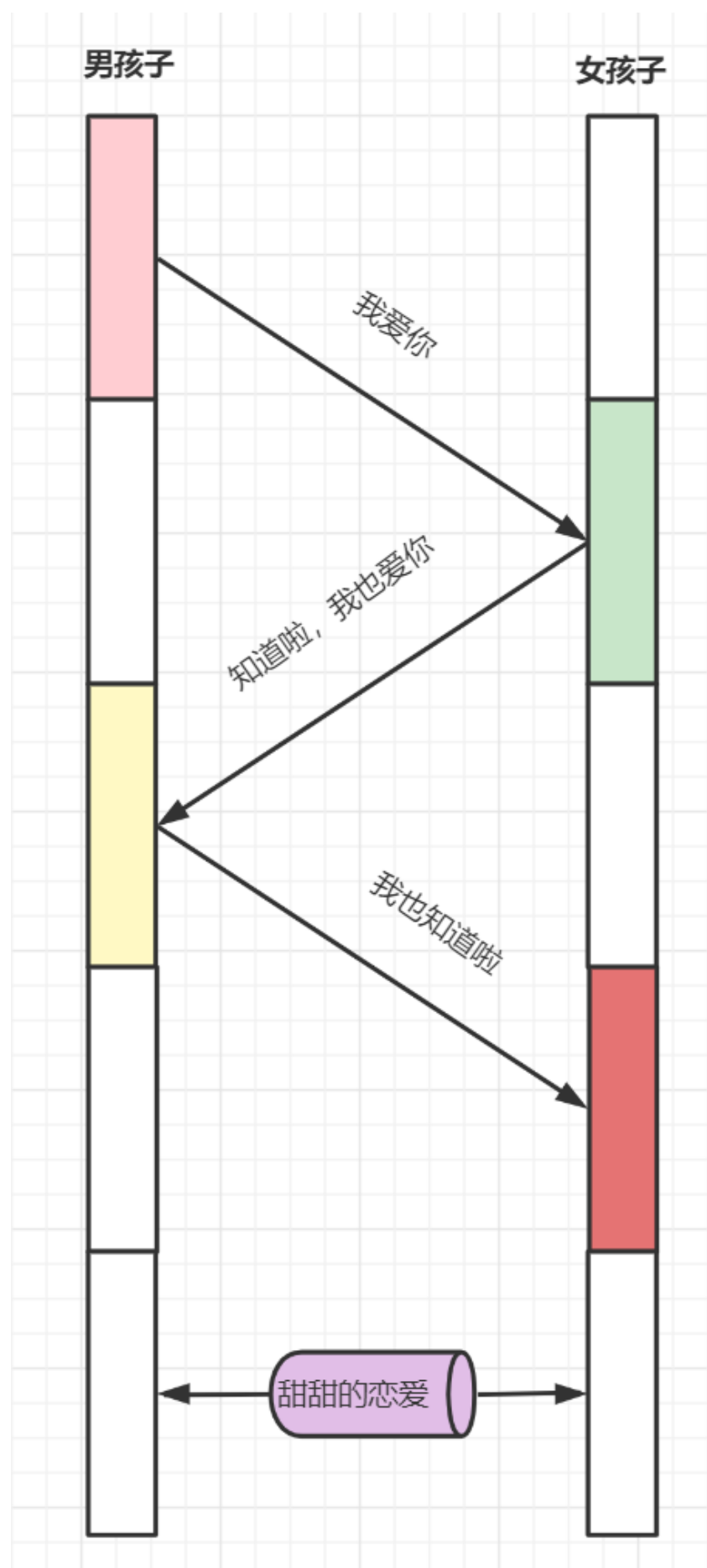


TCP 三次握手

- 第一次握手($\text{SYN}=1$, $\text{seq}=x$)，发送完毕后，客户端就进入 SYN_SEND 状态
- 第二次握手($\text{SYN}=1$, $\text{ACK}=1$, $\text{seq}=y$, $\text{ACKnum}=x+1$)，发送完毕后，服务器端就进入 SYN_RCV 状态。
- 第三次握手($\text{ACK}=1$, $\text{ACKnum}=y+1$)，发送完毕后，客户端进入 ESTABLISHED 状态，当服务器端接收到这个包时，也进入 ESTABLISHED 状态。

4.29 TCP 握手为什么是三次，为什么不能是两次？不能是四次？

思路： TCP 握手为什么不能是两次，为什么不能是四次呢？为了方便理解，我们以男孩子和女孩子谈恋爱为例子：两个人能走到一起，最重要的事情就是相爱，就是**我爱你，并且我知道，你也爱我**，接下来我们以此来模拟三次握手的过程：



为什么握手不能是两次呢？

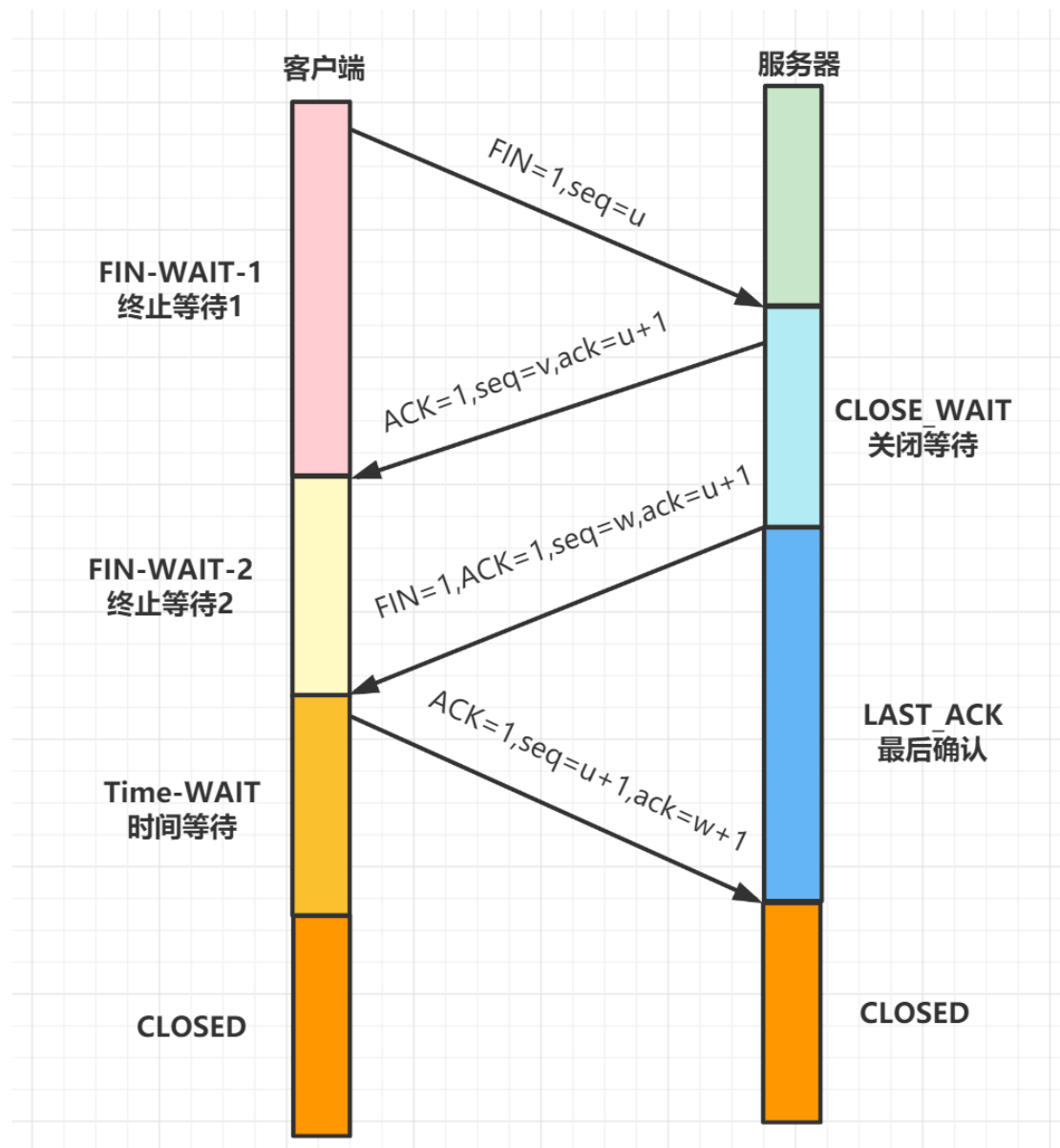
如果只有两次握手，女孩子可能就知道，她的那句**我也爱你**，男孩子是否收到，恋爱关系就不能愉快展开。

为什么握手不能是四次呢？

因为握手不能是四次呢？因为三次已经够了，三次已经能让双方都知道：你爱我，我也爱你。而四次就多余了。

4.30 TCP 四次挥手过程？

思路：TCP 的四次挥手，也是最重要的知识点，一般跟三次握手会一起考的，必须得记住。



TCP 四次挥手过程

1. 第一次挥手($\text{FIN}=1$, $\text{seq}=\text{u}$), 发送完毕后, 客户端进入 **FIN_WAIT_1** 状态。
2. 第二次挥手($\text{ACK}=1$, $\text{ack}=\text{u}+1$, $\text{seq}=\text{v}$), 发送完毕后, 服务器端进入 **CLOSE_WAIT** 状态, 客户端接收到这个确认包之后, 进入 **FIN_WAIT_2** 状态。
3. 第三次挥手($\text{FIN}=1$, $\text{ACK}=1$, $\text{seq}=\text{w}$, $\text{ack}=\text{u}+1$), 发送完毕后, 服务器端进入 **LAST_ACK** 状态, 等待来自客户端的最后一个 ACK。
4. 第四次挥手($\text{ACK}=1$, $\text{seq}=\text{u}+1$, $\text{ack}=\text{w}+1$), 客户端接收到来自服务器端的关闭请求, 发送一个确认包, 并进入 **TIME_WAIT** 状态, 等待了某个固定时间 (两个最大段生命周期, **2MSL**, **2 Maximum Segment Lifetime**) 之后, 没有收到服务器端的 ACK, 认为服务器端已经正常关闭连接, 于是自己也关闭连接, 进入 **CLOSED** 状态。服务器端接收到这个确认包之后, 关闭连接, 进入 **CLOSED** 状态。

4.31 TCP 挥手为什么需要四次呢？

思路： TCP 挥手为什么需要四次呢？为了方便大家理解，再举个生活的例子吧。

小明和小红打电话聊天，通话差不多要结束时，小红说，“我没啥要说的了”。

小明回答，“我知道了”。但是小明可能还有要说的话，小红不能要求小明跟着她自己的节奏结束通话，于是小明可能又叽叽歪歪说了一通，最后小明说，“我说完了”，小红回答，“我知道了”，这样通话才算结束。

小红

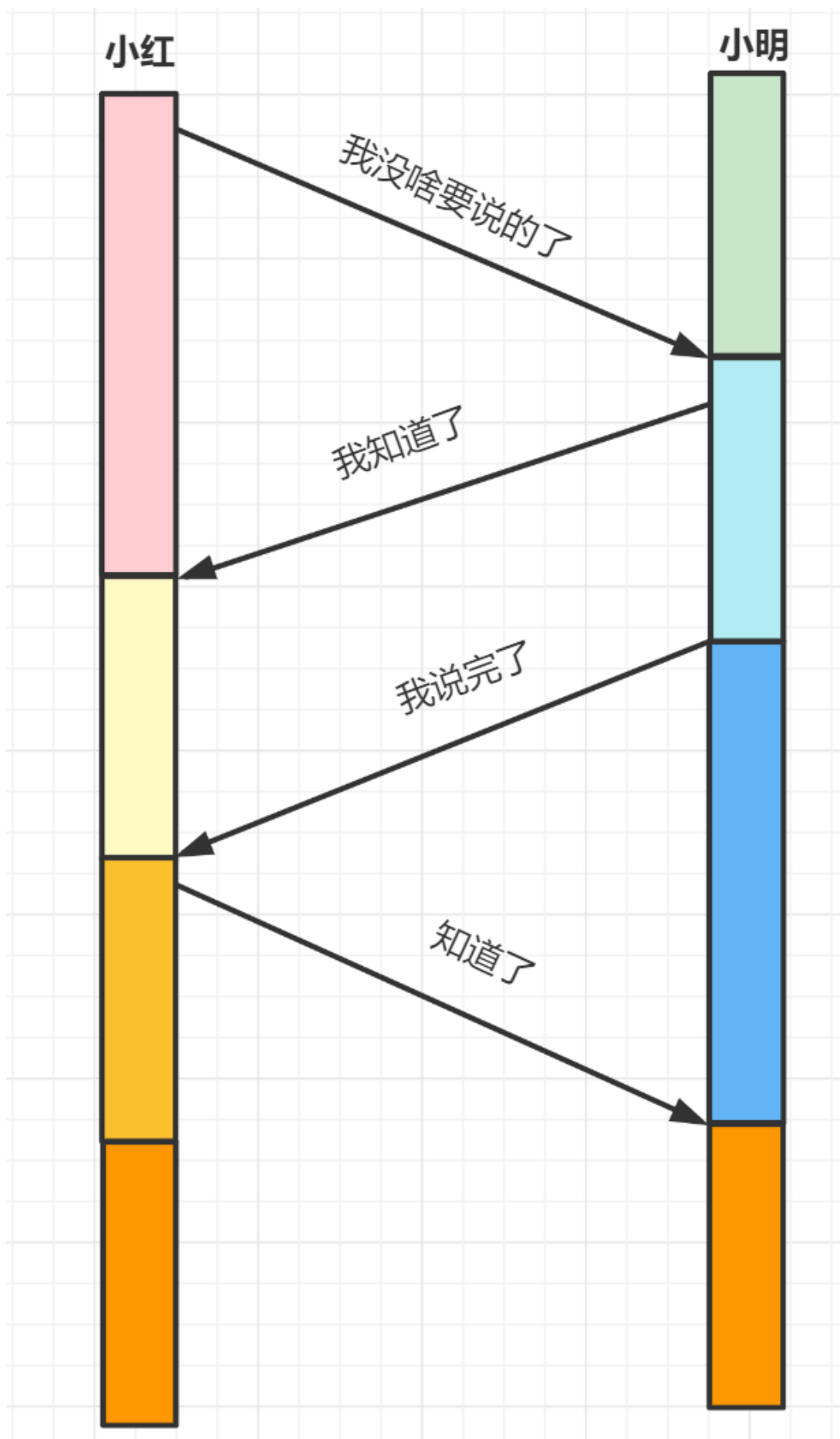
小明

我没啥要说的了

我知道了

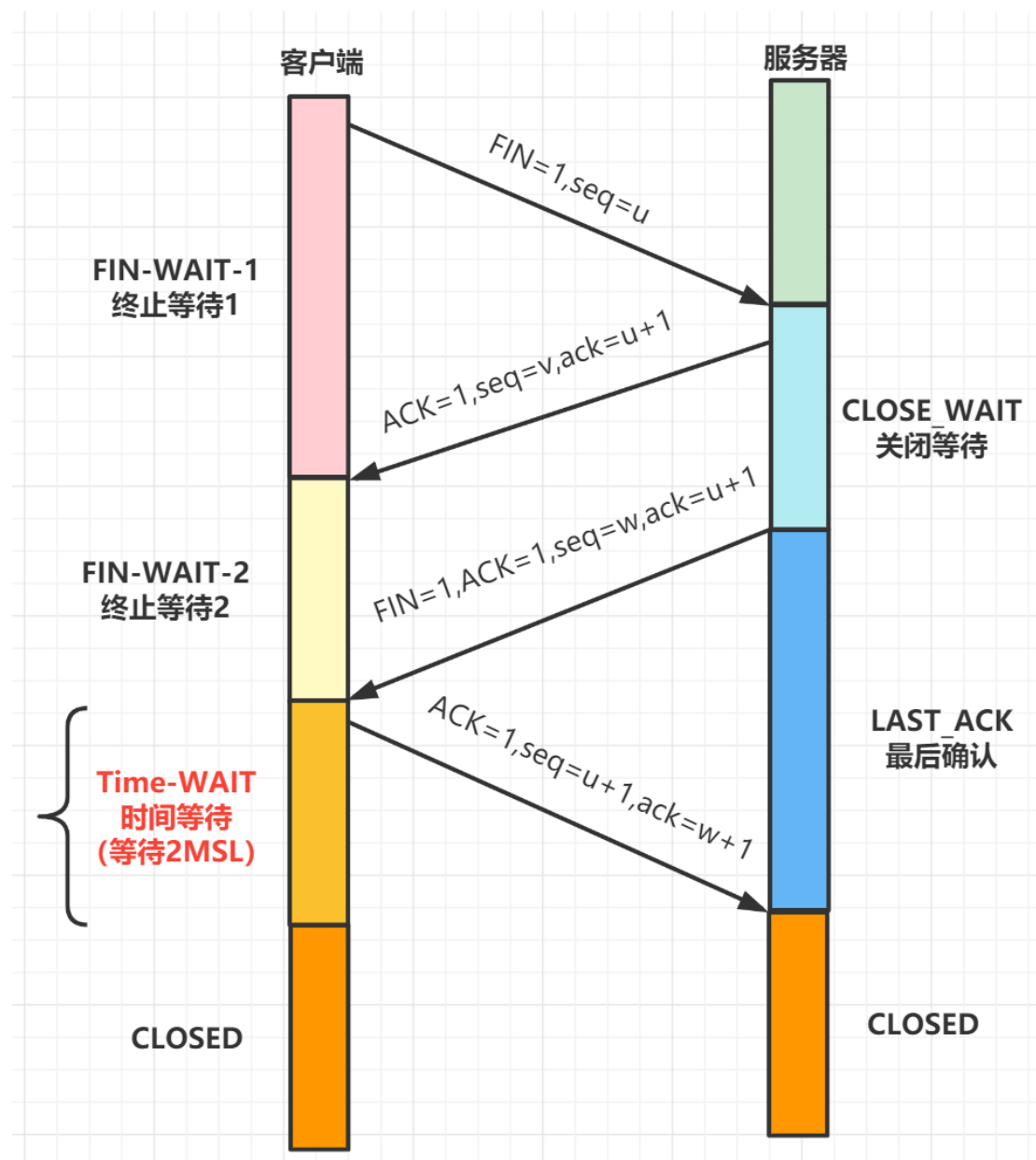
我说完了

知道了



4.32 TCP 四次挥手过程中，为什么需要等待 2MSL,才进入 CLOSED 关闭状态

思路：这个问得频率特别高。去面试前，一定要把这题拿下哈。



2MSL , **two Maximum Segment Lifetime** , 即两个最大段生命周期。

假设主动发起挥手的是客户端，那么需要 2MSL 的原因是：

- 1. 为了保证客户端发送的最后一个 **ACK** 报文段能够到达服务端。这个 ACK 报文段有可能丢失，因而使处在 **LAST-ACK** 状态的服务端就收不到对已发送的

FIN + ACK 报文段的确认。服务端会超时重传这个 FIN+ACK 报文段，而客户端就能在 2MSL 时间内（**超时 + 1MSL 传输**）收到这个重传的 FIN+ACK 报文段。接着客户端重传一次确认，重新启动 2MSL 计时器。最后，客户端和服务端都正常进入到 **CLOSED** 状态。

- **2. 防止已失效的连接请求报文段出现在本连接中。**客户端在发送完最后一个 ACK 报文段后，再经过时间 2MSL，就可以使本连接持续的时间内所产生的所有报文段都从网络中消失。这样就可以使下一个连接中不会出现这种旧的连接请求报文段。

4.33 说说 TCP 是如何确保可靠性的呢？

思路： TCP 是可靠的连接，为什么具有可靠性呢？记住这些点：连接和断开的可靠性（三次握手，四次挥手）、有状态（哪些数据发送了，哪些没发）、可控制（超时重传、流量控制、拥塞控制等）。



TCP老哥，稳！

- 首先，TCP 的连接是基于**三次握手**，而断开则是基于**四次挥手**。确保连接和断开的可靠性。
- 其次，TCP 的可靠性，还体现在**有状态**；TCP 会记录哪些数据发送了，哪些数据被接收了，哪些没有被接受，并且保证数据包按序到达，保证数据传输不出差错。
- 再次，TCP 的可靠性，还体现在**可控制**。它有数据包校验、ACK 应答、**超时重传(发送方)**、失序数据重传（接收方）、丢弃重复数据、流量控制（滑动窗口）和拥塞控制等机制。

4.34 说说 TCP 报文首部有哪些字段，其作用又分别是什么？

思路：小伙伴们，可以记下这个图。

16位源端口号							16位目的端口号						
32位序号													
32位确认号													
4位头部长度		6位保留		URG	ACK	PSH	RST	SYN	FIN	16位窗口大小			
16位校验和									16位紧急指针				
选项（长度可变）										填充			

- **16 位端口号**：源端口号，主机该报文段是来自哪里；目标端口号，要传给哪个上层协议或应用程序
- **32 位序号**：一次 TCP 通信（从 TCP 连接建立到断开）过程中某一个传输方向上的字节流的每个字节的编号。
- **32 位确认号**：用作对另一方发送的 tcp 报文段的响应。其值是收到的 TCP 报文段的序号值加 1。
- **4 位头部长度**：表示 tcp 头部有多少个 32bit 字（4 字节）。因为 4 位最大能标识 15，所以 TCP 头部最长是 60 字节。
- **6 位标志位**：URG(紧急指针是否有效)，ACK（表示确认号是否有效），PSH（缓冲区尚未填满），RST（表示要求对方重新建立连接），SYN（建立连接消息标志接），FIN（表示告知对方本端要关闭连接了）
- **16 位窗口大小**：是 TCP 流量控制的一个手段。这里说的窗口，指的是接收通告窗口。它告诉对方本端的 TCP 接收缓冲区还能容纳多少字节的数据，这样对方就可以控制发送数据的速度。
- **16 位校验和**：由发送端填充，接收端对 TCP 报文段执行 CRC 算法以检验 TCP 报文段在传输过程中是否损坏。注意，这个校验不仅包括 TCP 头部，也包括数据部分。这也是 TCP 可靠传输的一个重要保障。
- **16 位紧急指针**：一个正的偏移量。它和序号字段的值相加表示最后一个紧急数据的下一字节的序号。因此，确切地说，这个字段是紧急指针相对当前序号的偏移，不妨称之为紧急偏移。TCP 的紧急指针是发送端向接收端发送紧急数据的方法。

4.35 请简述 TCP 和 UDP 的区别

思路：这道题，校招的时候，可能问的概率高点，概念性的东西，**TCP 是面向连接，而 UDP 是无连接。**

1. TCP 面向连接（如打电话需要先拨号），UDP 面向无连接（即发送数据前不需要建立连接）。
2. TCP 提供可靠的服务，UDP 无法保证。
3. TCP 面向字节流，而 UDP 面向报文。
4. TCP 数据传输慢，UDP 数据传输快
5. TCP 是点对点连接的，UDP 可以一对一，一对多，多对多都可以。
6. TCP 适用于邮件、网页等，UDP 适用于语音广播等。

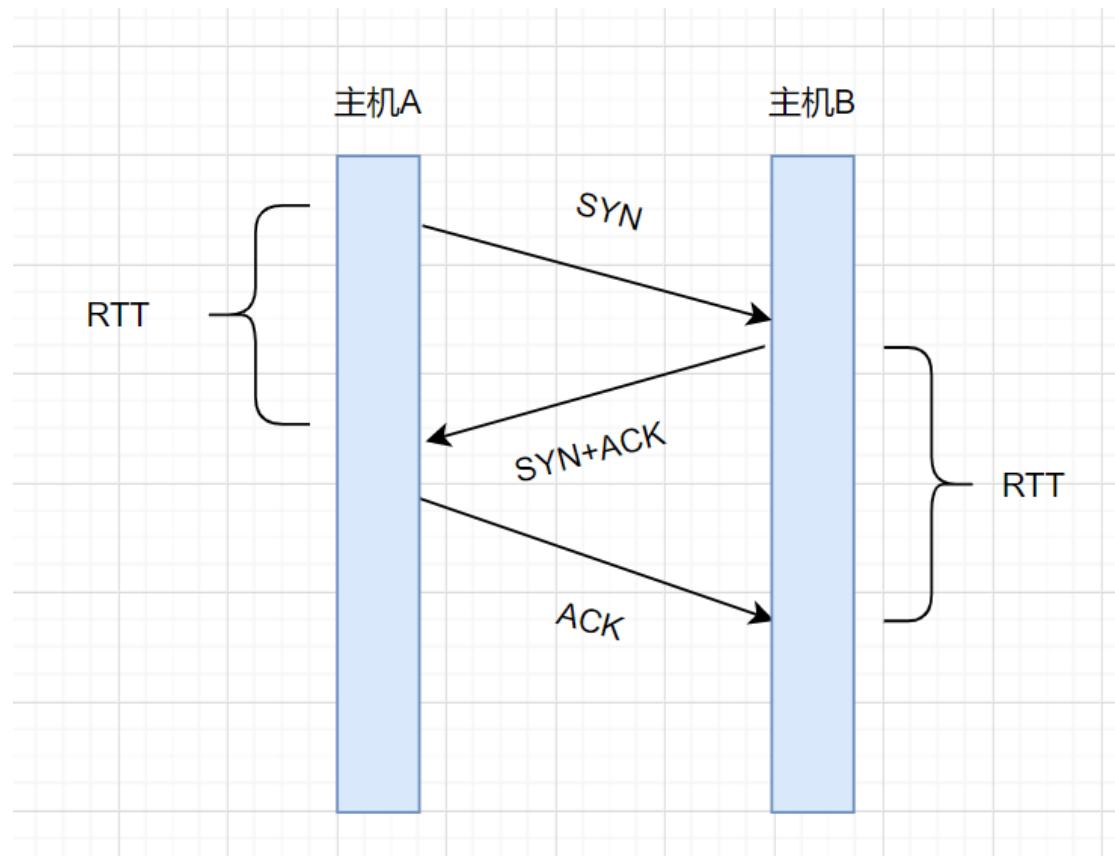
4.36 说说 TCP 的重传机制

思路讲解： TCP 的重传机制，也是高频面试题。重传包括超时重传、快速重传、带选择确认的重传（SACK）、重复 SACK。

4.37.1 超时重传

TCP 为了可靠传输，实现了重传机制。最基本的重传机制，就是**超时重传**，即在发送数据报文时，设定一个定时器，每间隔一段时间间隔，如果没收到对方的 ACK 应答报文，就会重发该报文。

这个间隔时间，一般设置为多少呢？我们先来看下什么叫 **RTT (Round-Trip Time，往返时间)**。



RTT 就是，一个数据包从发出去到回来的时间，即数据包的一次往返时间。超时重传时间，就是 Retransmission Timeout，简称 **RTO**。

RTO 设置多久呢？

- 如果 RTO 比较小，那很可能数据都没有丢失，就重发了，这会导致网络阻塞，会导致更多的超时出现。
- 如果 RTO 比较大，等到花儿都谢了还是没有重发，那效果就不好了。

一般情况下，RTO 略大于 RTT，效果是最好的。一些小伙伴会问，超时时间有没有计算公式呢？有的！有个标准方法算 RTO 的公式，也叫 **Jacobson /**

Karels 算法。我们一起来看下计算 RTO 的公式

1. 先计算 SRTT (计算平滑的 RTT)

$$SRTT = (1 - \alpha) * SRTT + \alpha * RTT$$
 //求 SRTT 的加权平均

2. 再计算 RTTVAR (round-trip time variation)

$$RTTVAR = (1 - \beta) * RTTVAR + \beta * (|RTT - SRTT|) // \text{计算 SRTT 与真实值的差距}$$

3. 最终的 RTO

$$RTO = \mu * SRTT + \vartheta * RTTVAR = SRTT + 4 * RTTVAR$$

其中， $\alpha = 0.125$ ， $\beta = 0.25$ ， $\mu = 1$ ， $\vartheta = 4$ ，这些参数都是大量结果得出的最优参数。

但是，超时重传会有这些缺点：

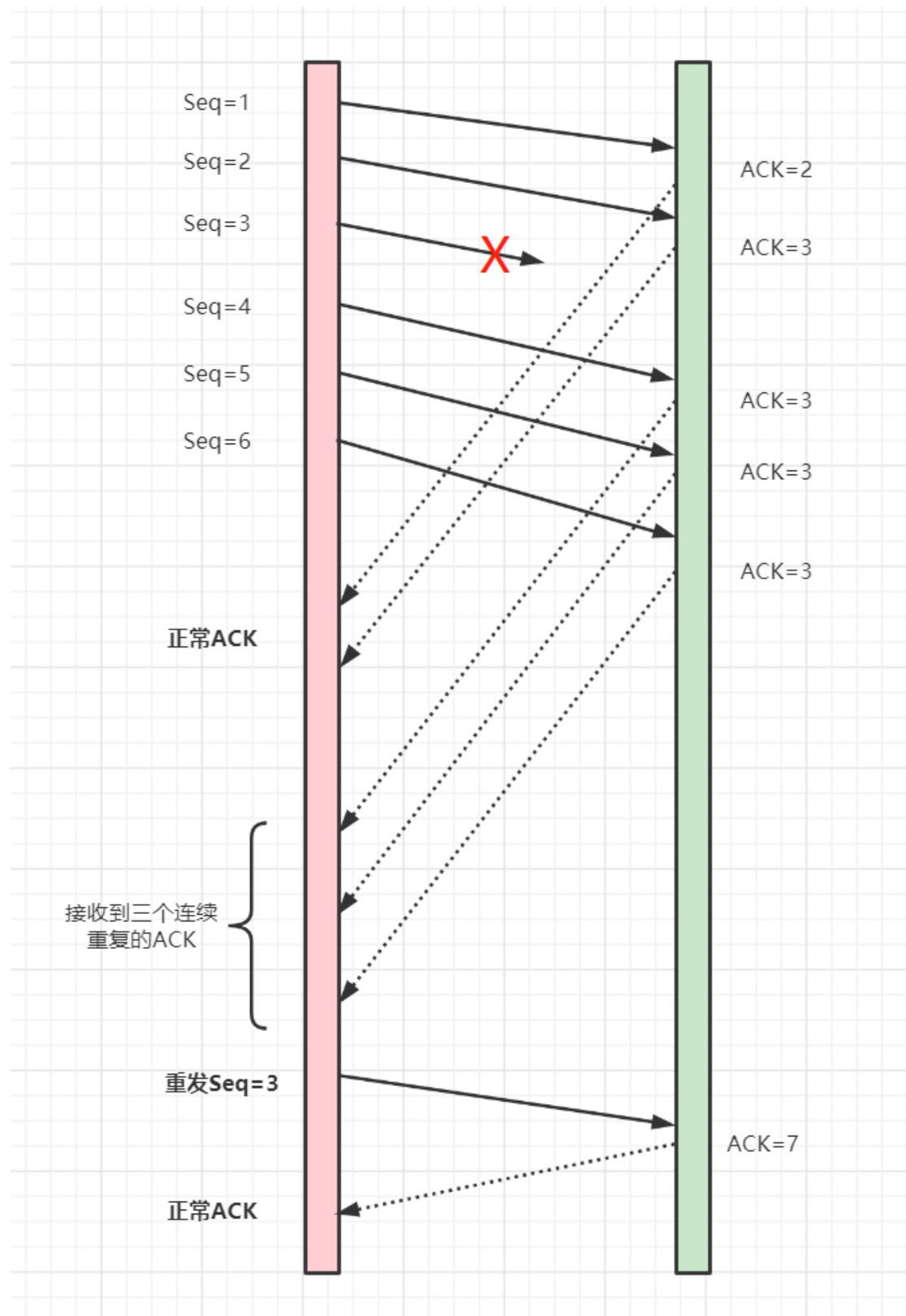
- 当一个报文段丢失时，会等待一定的超时周期然后才重传分组，增加了端到端的时延。
- 当一个报文段丢失时，在其等待超时的过程中，可能会出现这种情况：其后的报文段已经被接收端接收但却迟迟得不到确认，发送端会认为也丢失了，从而引起不必要的重传，既浪费资源也浪费时间。

并且，TCP 有个策略，就是超时时间，间隔就会加倍。超时重传需要等待很长时间。因此，还可以使用快速重传机制。

4.37.2 快速重传

快速重传机制，它不以时间驱动，而是以数据驱动。它基于接收端的反馈信息来引发重传。

一起来看下快速重传流程：



快速重传流程

发送端发送了 1, 2, 3, 4, 5, 6 份数据:

- 第一份 Seq=1 先送到了, 于是就 Ack 回 2;

- 第二份 Seq=2 也送到了，假设也正常，于是 ACK 回 3；
- 第三份 Seq=3 由于网络等其他原因，没送到；
- 第四份 Seq=4 也送到了，但是因为 Seq3 没收到。所以 ACK 回 3；
- 后面的 Seq=4,5 的也送到了，但是 ACK 还是回复 3，因为 Seq=3 没收到。
- 发送端连着收到三个重复冗余 ACK=3 的确认（实际上是 4 个，但是前面一个是正常的 ACK，后面三个才是重复冗余的），便知道哪个报文段在传输过程中丢失了，于是在定时器过期之前，重传该报文段。
- 最后，接收到收到了 Seq3，此时因为 Seq=4，5，6 都收到了，于是 ACK 回 7。

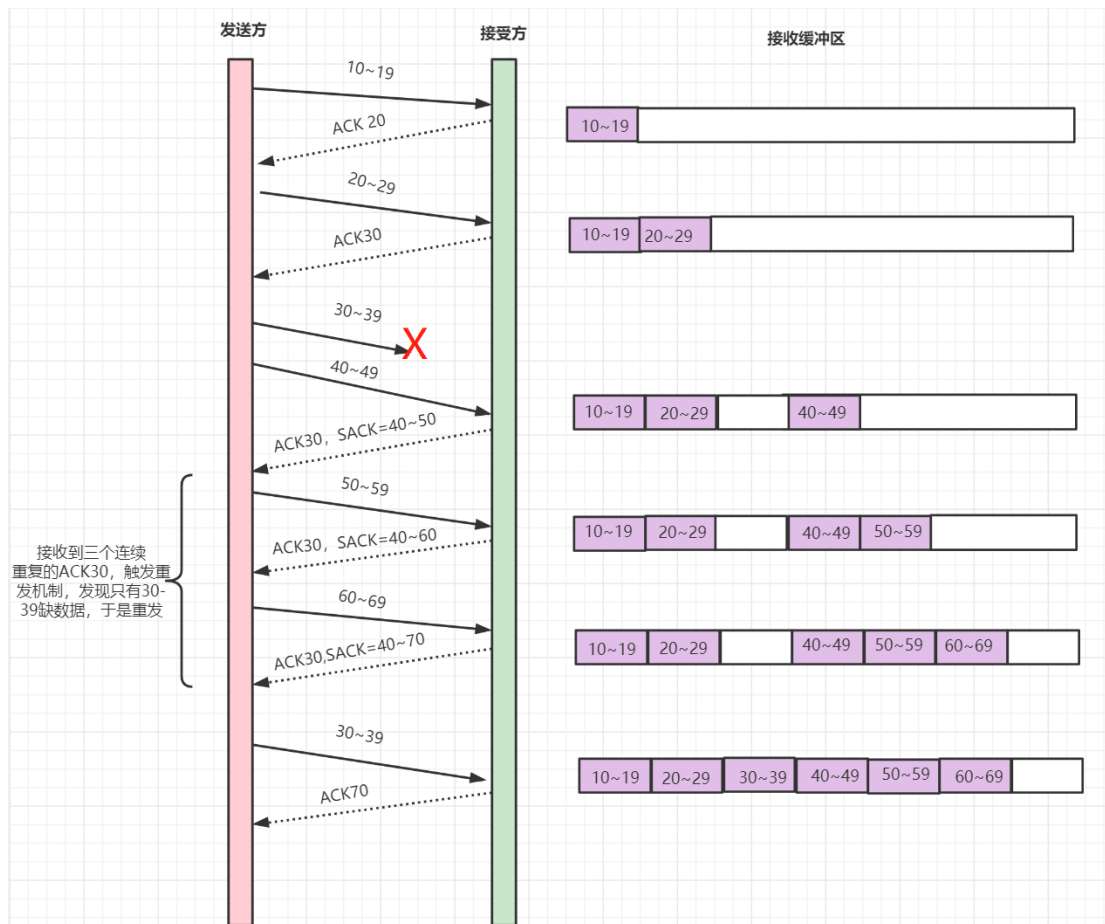
但**快速重传**还可能会有个问题：ACK 只向发送端告知最大的有序报文段，到底是哪个报文丢失了呢？**并不确定**！那到底该重传多少个包呢？

是重传 Seq3 呢？还是重传 Seq3、Seq4、Seq5、Seq6 呢？因为发送端并不清楚这三个连续的 ACK3 是谁传回来的。

4.37.3 带选择确认的重传 (SACK)

为了解决快速重传的问题：**应该重传多少个包**？TCP 提供了 **SACK 方法**（带选择确认的重传，Selective Acknowledgment）。

SACK 机制就是，在快速重传的基础上，接收端返回最近收到的报文段的序列号范围，这样发送端就知道接收端哪些数据包没收到，**酱紫**就很清楚该重传哪些数据包啦。SACK 标记是加在 TCP 头部**选项**字段里面的。

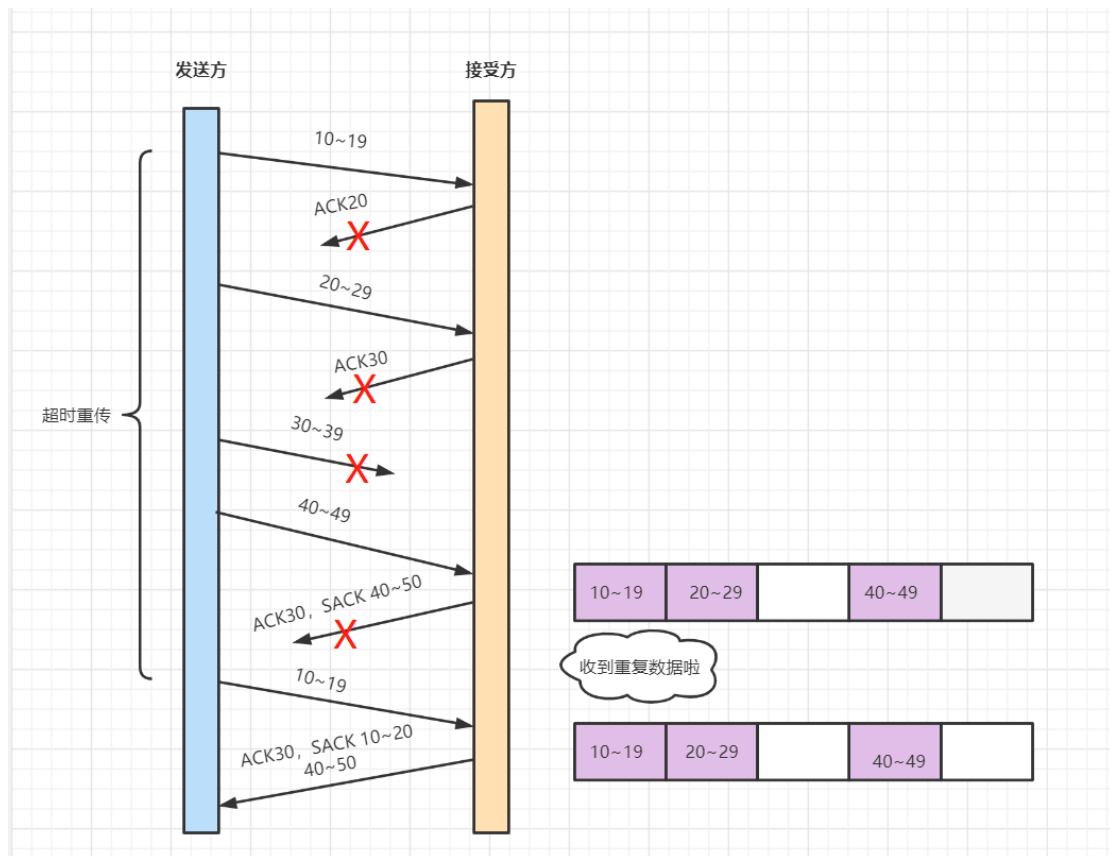


SACK 机制

如上图，发送端收到了三次同样的 $ACK=30$ 的确认报文，于是就会触发快速重发机制，通过 SACK 信息发现只有 $30\sim39$ 这段数据丢失，于是重发时就只选择了这个 $30\sim39$ 的 TCP 报文段进行重发。

4.37.4 D-SACK

D-SACK，即 Duplicate SACK (重复 SACK)，在 SACK 的基础上做了一些扩展，，主要用来告诉发送方，有哪些数据包自己重复接受了。DSACK 的目的是帮助发送方判断，是否发生了包失序、ACK 丢失、包重复或伪重传。让 TCP 可以更好的做网络流控。来看个图吧：



D-SACK 简要流程

4.38 聊聊 TCP 的滑动窗口

思路讲解： TCP 滑动窗口是个高频考点，我们需要知道 TCP 报文首部有个字段 **win** 控制窗口大小的，同时也需要掌握，滑动窗口是怎么滑的。

TCP 发送一个数据，如果需要收到确认应答，才会发送下一个数据。这样的话就会有个缺点：效率会比较低。

这就好像我们面对面在聊天，你说完一句，我应答之后，你才能说下一句。那么，如果我在忙其他事情，没有能够及时回复你呢？你说完一句后，要等到我忙完回复你，你才说下句，这显然不现实，效率太低。

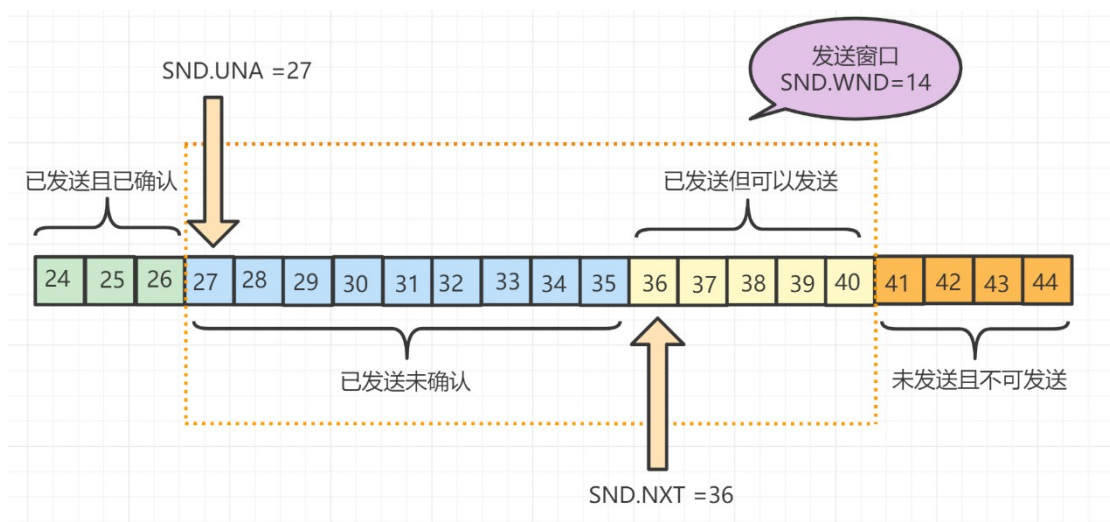
为了解决这个问题，TCP 引入了**窗口**，它是操作系统开辟的一个缓存空间。窗口大小值表示无需等待确认应答，而可以继续发送数据的最大值。

TCP 头部有个字段叫 win，也即那个 **16 位的窗口大小**，它告诉对方本端的 TCP 接收缓冲区还能容纳多少字节的数据，这样对方就可以控制发送数据的速度，从而达到**流量控制**的目的。

通俗点讲，就是接受方每次收到数据包，在发送确认报文的时候，同时告诉发送方，自己的缓存区还有多少空余空间，缓冲区的空余空间，我们就称之为**接受窗口大小**。这就是 win。

TCP 滑动窗口分为两种：发送窗口和接收窗口。**发送端的滑动窗口**包含四大部分，如下：

- 已发送且已收到 ACK 确认
- 已发送但未收到 ACK 确认
- 未发送但可以发送
- 未发送也不可以发送

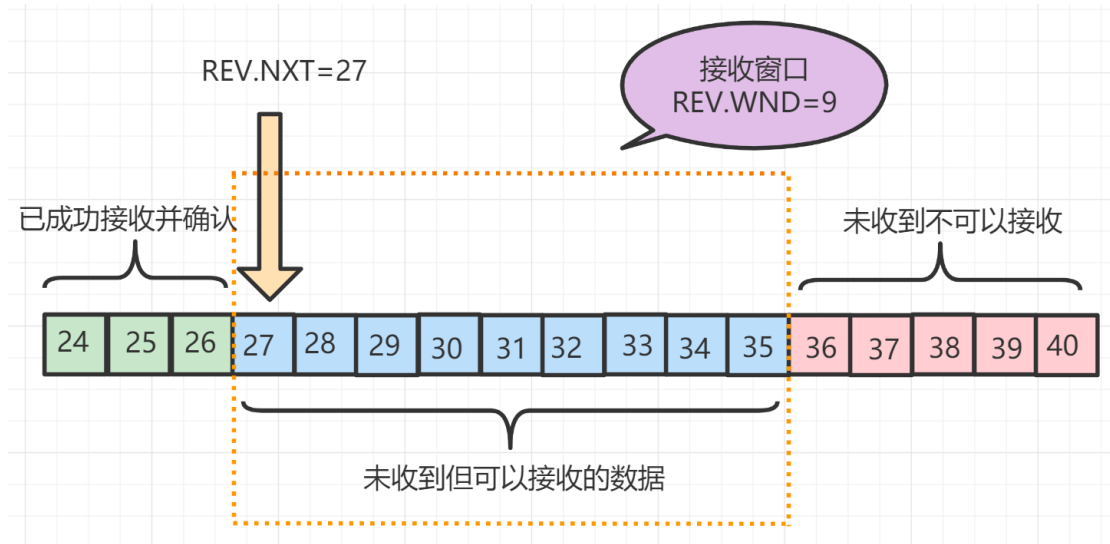


- 虚线矩形框，就是发送窗口。
- SND.WND: 表示发送窗口的大小,上图虚线框的格子数是 14 个，即发送窗口大小是 14。
- SND.NXT: 下一个发送的位置，它指向未发送但可以发送的第一个字节的序列号。

- **SND.UNA**: 一个绝对指针，它指向的是已发送但未确认的第一个字节的序列号。

接收方的滑动窗口包含三大部分，如下：

- 已成功接收并确认
- 未收到数据但可以接收
- 未收到数据并不可以接收的数据



- 虚线矩形框，就是接收窗口。
- **REV.WND**: 表示接收窗口的大小,上图虚线框的格子就是9个。
- **REV.NXT**: 下一个接收的位置，它指向未收到但可以接收的第一个字节的序列号。

4.38 说说 TCP 的流量控制？

思路讲解： TCP 流量控制是个高频考点，一般跟滑动窗口一起考的，大家需要掌握一次流量控制过程哈。

TCP 三次握手完成，发送方和接收方都进入到 **ESTABLISHED** 状态，它们就可以开始传输数据啦。

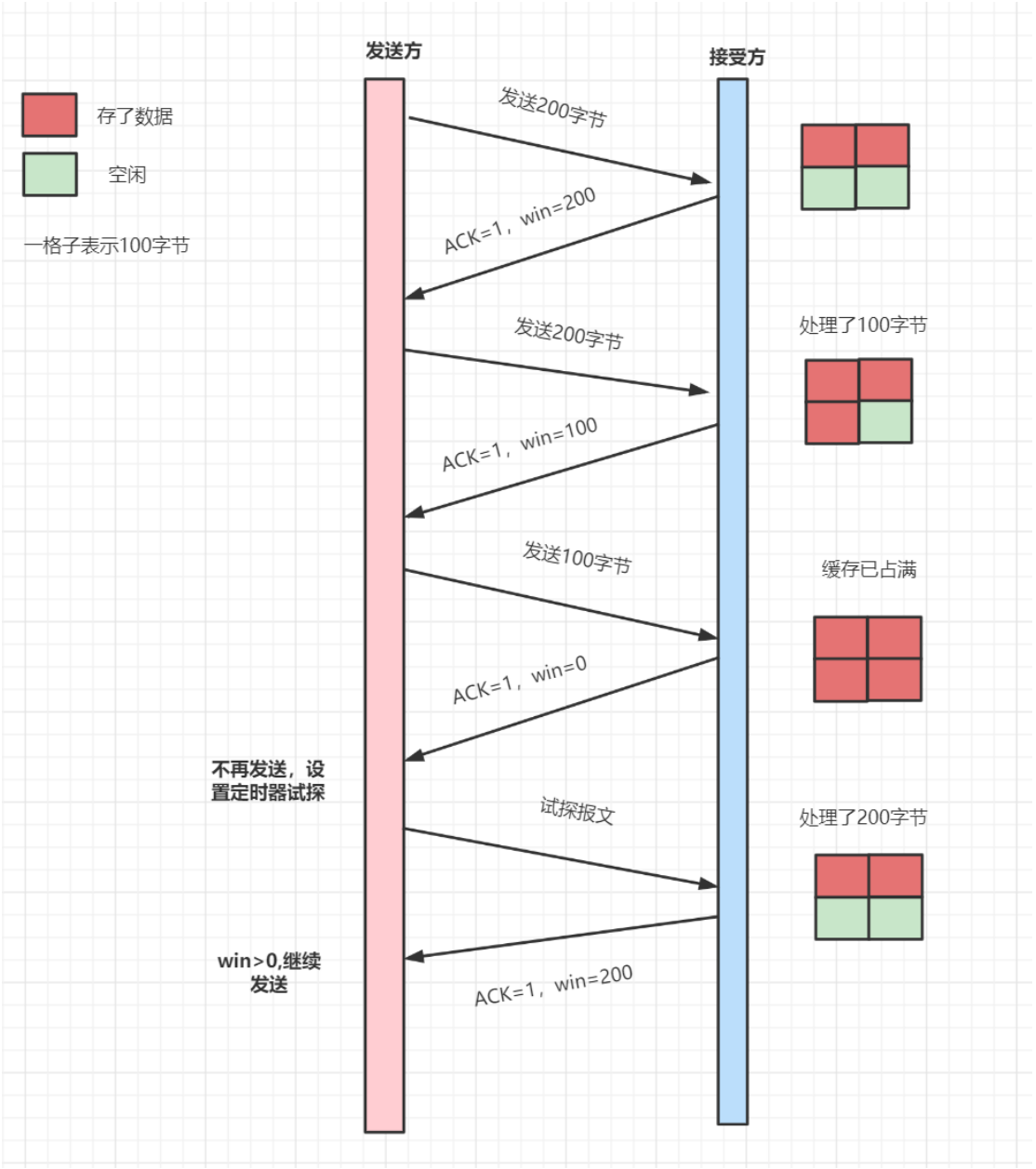
但是发送方不能疯狂地向接收方发送数据，因为接收方接收不过来的话，接收方就只能把处理不过来的数据存在缓存区里了。如果缓存区都满了，发送方还

在疯狂发送数据的话，接收方只能把收到的数据包丢掉，这就浪费了网络资源啦。

TCP 提供一种机制可以让发送方根据接收端的实际接收能力控制发送的数据量，这就是**流量控制**。

TCP 通过滑动窗口来控制流量，我们看下流量控制的**简要流程**吧：

首先双方三次握手，初始化各自的窗口大小，假设均为 400 个字节。



TCP 的流量控制

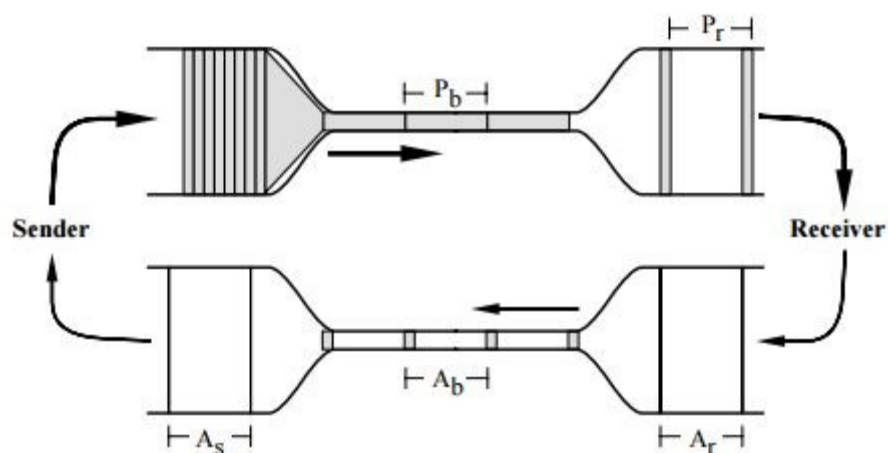
1. 假如当前发送方给接收方发送了 200 个字节，那么，发送方的 **SND.NXT** 会右移 200 个字节，也就是说当前的可用窗口减少了 200 个字节。
2. 接受方收到后，放到缓冲队列里面， $REV.WND = 400 - 200 = 200$ 字节，所以 $win = 200$ 字节返回给发送方。接收方会在 ACK 的报文首部带上缩小后的滑动窗口 200 字节
3. 发送方又发送 200 字节过来，200 字节到达，继续放到缓冲队列。不过这时候，由于大量负载的原因，接受方处理不了这么多字节，只能处理 100 字节，剩余的 100 字节继续放到缓冲队列。这时候， $REV.WND = 400 - 200 - 100 = 100$ 字节，即 $win = 100$ 返回发送方。
4. 发送方继续干活，发送 100 字节过来，这时候，接受窗口 win 变为 0。
5. 发送方停止发送，开启一个定时任务，每隔一段时间，就去询问接受方，直到 win 大于 0，才继续开始发送。

4.39 TCP 的拥塞控制

思路讲解： TCP 拥塞机制也是个高频考点，需要掌握它跟**流量控制**的区别，也需要掌握流量控制的这几种算法：**慢启动算法、拥塞避免、拥塞发生、快速恢复算法。**

拥塞控制是作用于网络的，防止过多的数据包注入到网络中，避免出现网络负载过大的情况。它的目标主要是最大化利用网络上瓶颈链路的带宽。它跟**流量控制**又有什么区别呢？流量控制是作用于接收者的，根据**接收端的实际接收能力控制发送速度**，防止分组丢失的。

我们可以把网络链路比喻成一根水管，如果我们想最大化利用网络来传输数据，那就是尽快让水管达到最佳充满状态。



发送方维护一个**拥塞窗口 cwnd (congestion window)** 的变量，用来估算在一段时间内这条链路（水管）可以承载和运输的数据（水）的数量。它大小代表着网络的拥塞程度，并且是动态变化的，但是为了达到最大的传输效率，我们该如何知道这条水管的运送效率是多少呢？

一个比较简单的方法就是不断增加传输的水量，直到水管快要爆裂为止（对应到网络上就是发生丢包），用 TCP 的描述就是：

只要网络中没有出现拥塞，拥塞窗口的值就可以再增大一些，以便把更多的数据包发送出去，但只要网络出现拥塞，拥塞窗口的值就应该减小一些，以减少注入到网络中的数据包数。

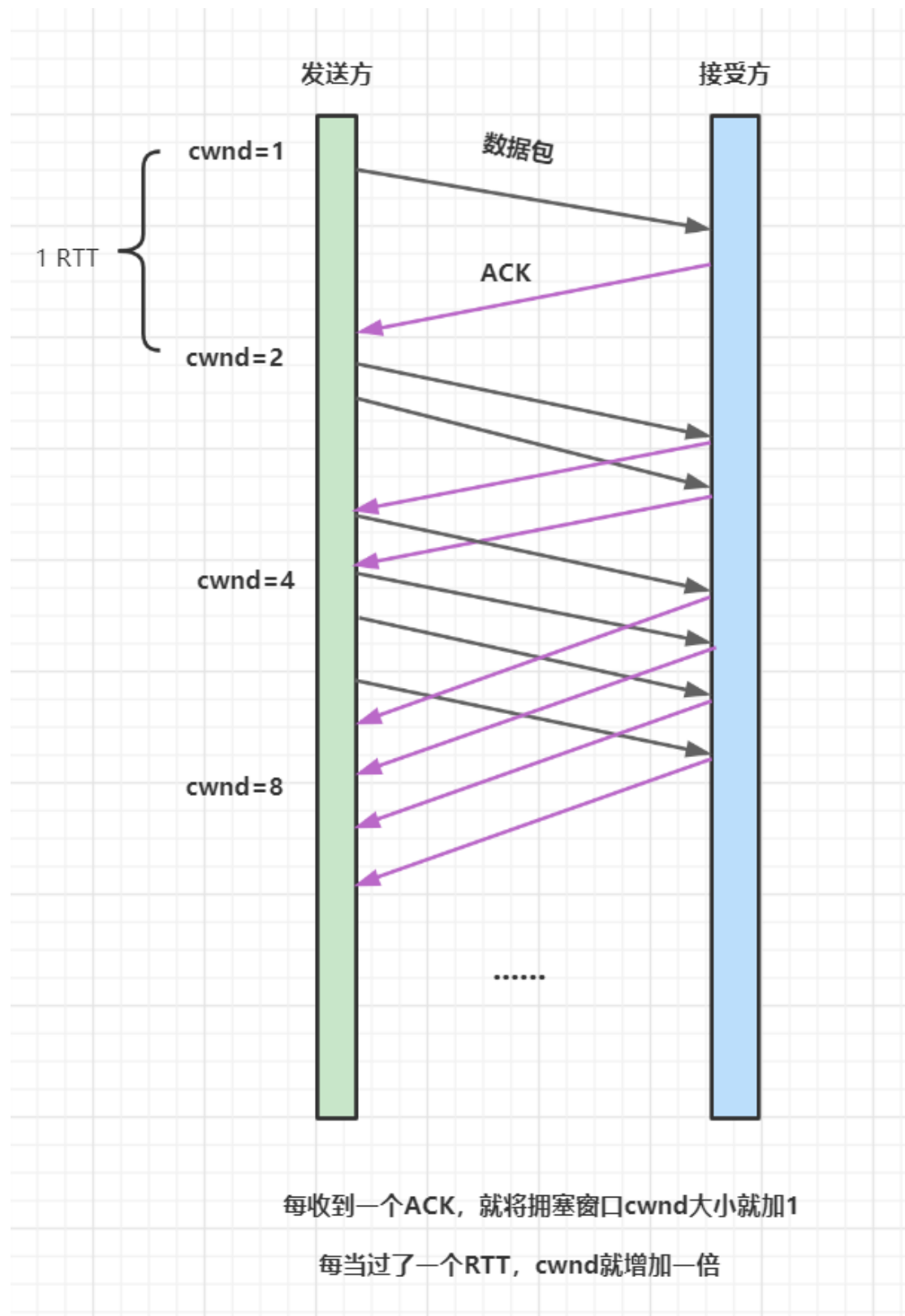
实际上，拥塞控制主要有这几种常用算法

- 慢启动
- 拥塞避免
- 拥塞发生
- 快速恢复

4.39.1 慢启动算法

慢启动算法，表面意思就是，别急慢慢来。它表示 TCP 建立连接完成后，一开始不要发送大量的数据，而是先探测一下网络的拥塞程度。由小到大逐渐增加拥塞窗口的大小，如果没有出现丢包，**每收到一个 ACK，就将拥塞窗口 cwnd 大小就加 1 (单位是 MSS)**。每轮次发送窗口增加一倍，呈指数增长，如果出现丢包，拥塞窗口就减半，进入拥塞避免阶段。

- TCP 连接完成，初始化 $cwnd = 1$ ，表明可以传一个 MSS 单位大小的数据。
- 每当收到一个 ACK，cwnd 就加一；
- 每当过了一个 RTT，cwnd 就增加一倍；呈指数让升



为了防止 cwnd 增长过大引起网络拥塞，还需设置一个**慢启动阈值**

ssthresh (slow start threshold) 状态变量。当 cwnd 到达该阈值后，就好

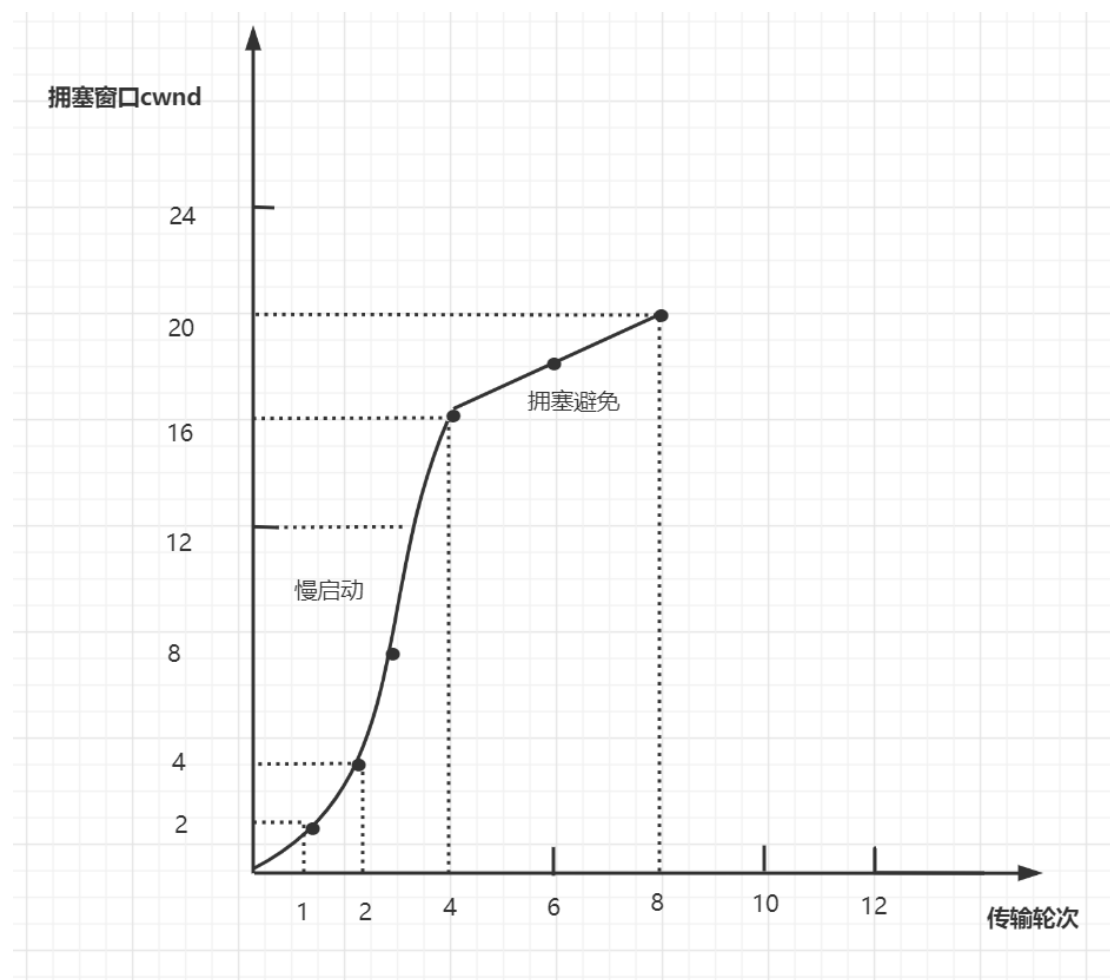
像水管被关小了水龙头一样，减少拥塞状态。即当 $\text{cwnd} > \text{ssthresh}$ 时，进入了拥塞避免算法。

4.39.2 拥塞避免算法

一般来说，慢启动阈值 ssthresh 是 65535 字节， cwnd 到达慢启动阈值后

- 每收到一个 ACK 时， $\text{cwnd} = \text{cwnd} + 1/\text{cwnd}$
- 当每过一个 RTT 时， $\text{cwnd} = \text{cwnd} + 1$

显然这是一个线性上升的算法，避免过快导致网络拥塞问题。



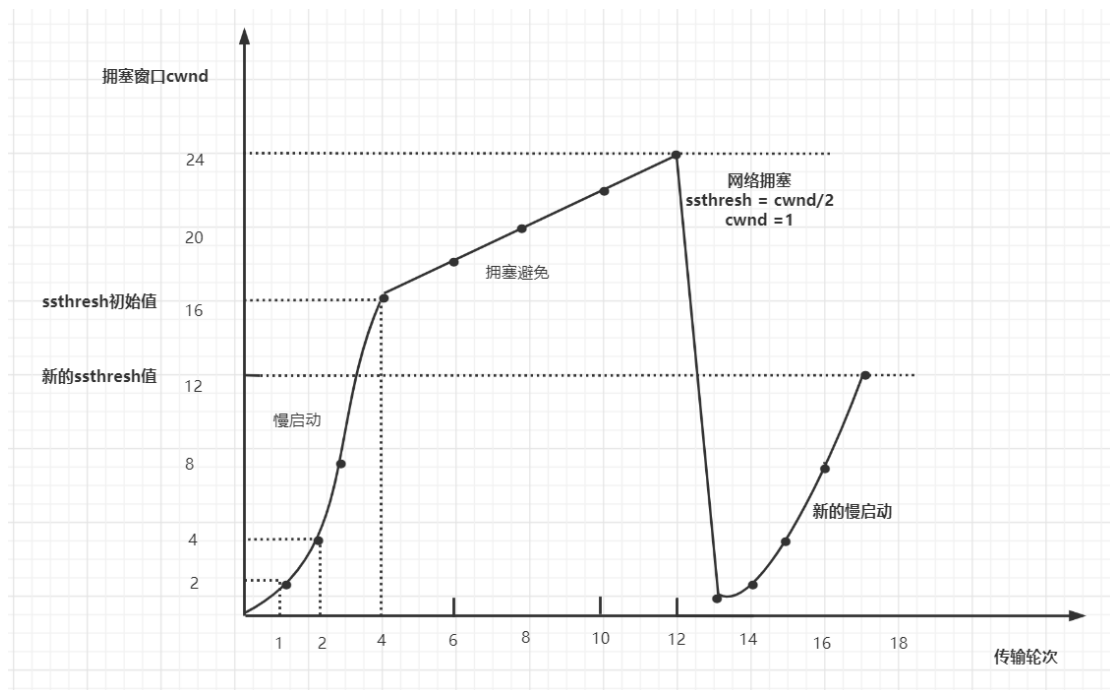
4.39.3 拥塞发生

当网络拥塞发生丢包时，会有两种情况：

- RTO 超时重传
- 快速重传

如果是发生了 **RTO 超时重传**，就会使用拥塞发生算法

- 慢启动阈值 $ssthresh = cwnd / 2$
- $cwnd$ 重置为 1
- 进入新的慢启动过程



这真的是辛辛苦苦几十年，一朝回到解放前。其实还有更好的处理方式，就是快速重传。发送方收到 3 个连续重复的 ACK 时，就会快速地重传，不必等待 **RTO 超时**再重传。

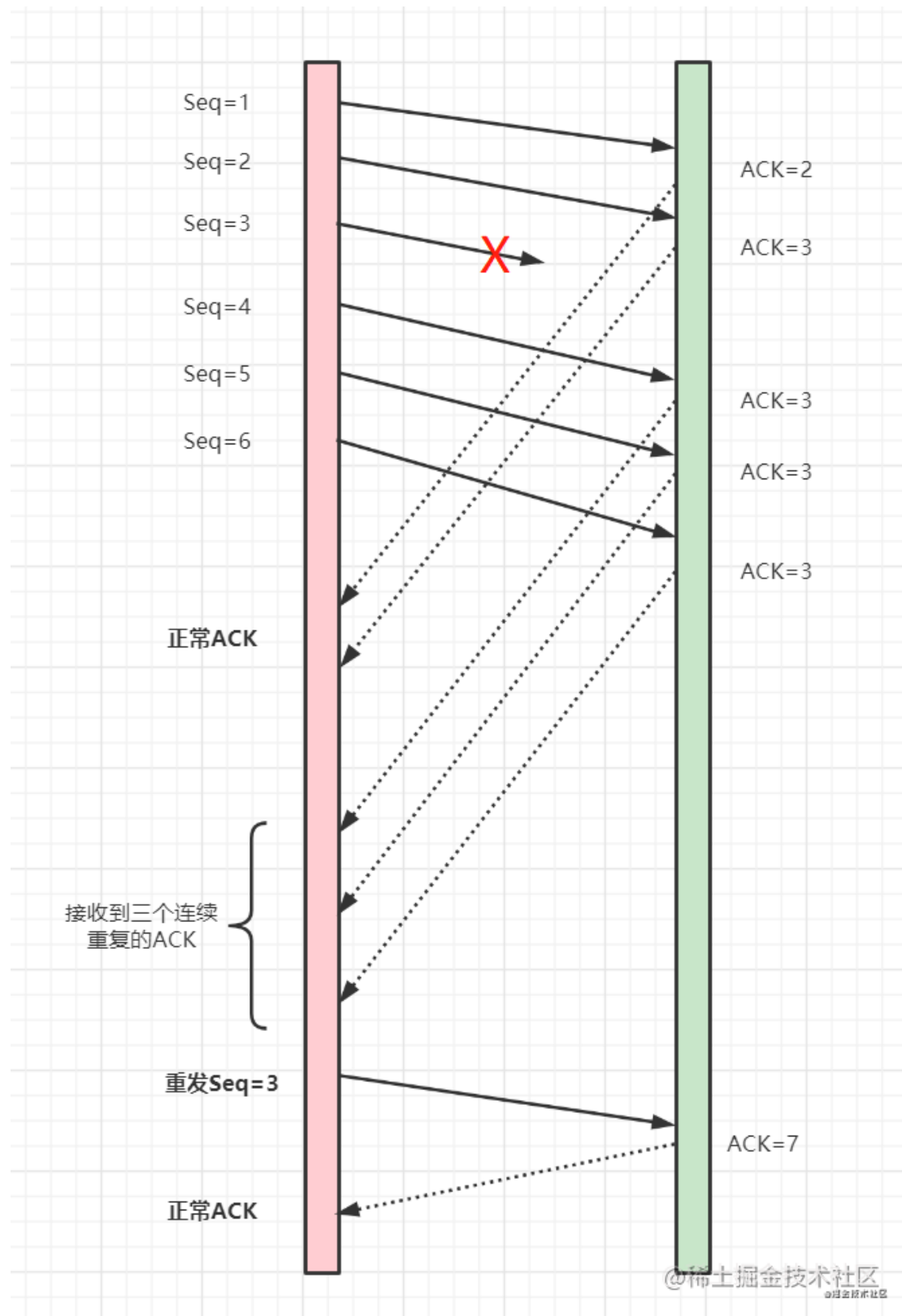


image.png

慢启动阈值 ssthresh 和 cwnd 变化如下：

- 拥塞窗口大小 $cwnd = cwnd/2$

- 慢启动阈值 $ssthresh = cwnd$
- 进入快速恢复算法

4.39.4 快速恢复

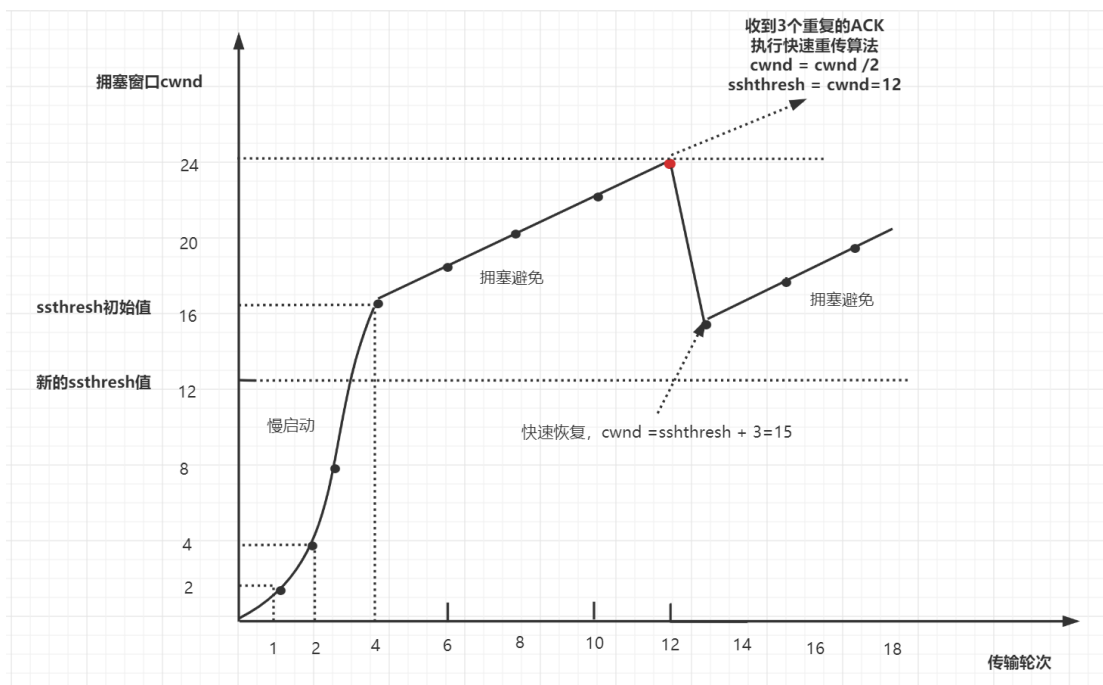
快速重传和快速恢复算法一般同时使用。快速恢复算法认为，还有 3 个重复 ACK 收到，说明网络也没那么糟糕，所以没有必要像 RTO 超时那么强烈。

正如前面所说，进入快速恢复之前， $cwnd$ 和 $ssthresh$ 已被更新：

```
- cwnd = cwnd / 2
- ssthresh = cwnd
```

然后，真正的快速算法如下：

- $cwnd = ssthresh + 3$
- 重传重复的那几个 ACK (即丢失的那几个数据包)
- 如果再收到重复的 ACK，那么 $cwnd = cwnd + 1$
- 如果收到新数据的 ACK 后， $cwnd = ssthresh$ 。因为收到新数据的 ACK，表明恢复过程已经结束，可以再次进入了拥塞避免的算法了。



4.40 说说半连接队列和 SYN Flood 攻击的关系

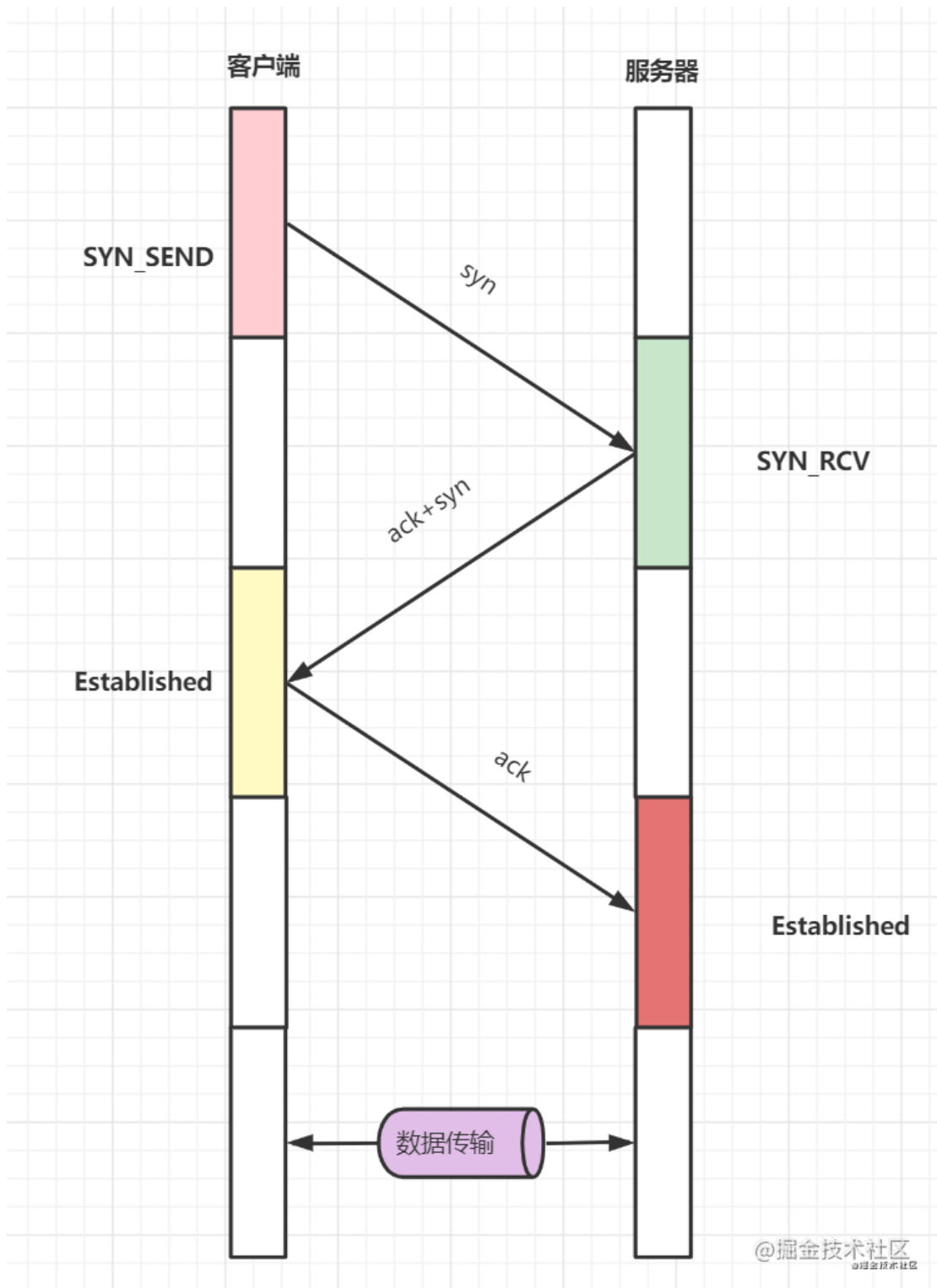
思路讲解：我以前面试的时候，面试官就问我什么是半连接队列、什么是全连接队列，哈哈。我们记需要掌握半连接队列、全连接对列是啥，还需要清楚半连接队列和 SYN Flood 攻击有什么关系。

我的答案如下：

TCP 进入三次握手前，服务端会从 **CLOSED** 状态变为 **LISTEN** 状态,同时在内部创建了两个队列：半连接队列（SYN 队列）和全连接队列（ACCEPT 队列）。

什么是半连接队列（**SYN 队列**）呢？什么是全连接队列（**ACCEPT 队列**）呢？

回忆下 TCP 三次握手的图：



三次握手

- TCP 三次握手时，客户端发送 SYN 到服务端，服务端收到之后，便回复 **ACK** 和 **SYN**，状态由 **LISTEN** 变为 **SYN_RCVD**，此时这个连接就被推入了 **SYN** 队列，即半连接队列。
- 当客户端回复 ACK, 服务端接收后，三次握手就完成了。这时连接会等待被具体的应用取走，在被取走之前，它被推入 **ACCEPT** 队列，即全连接队列。

SYN Flood 是一种典型的 DDos 攻击，它在短时间内，伪造不存在的 IP 地址，向服务器大量发起 SYN 报文。当服务器回复 SYN+ACK 报文后，不会收到 ACK 回应报文，导致服务器上建立大量的半连接半连接队列满了，这就无法处理正常的 TCP 请求啦。

那么有哪些方案应对呢？主要有 **syn cookie** 和 **SYN Proxy 防火墙**等。

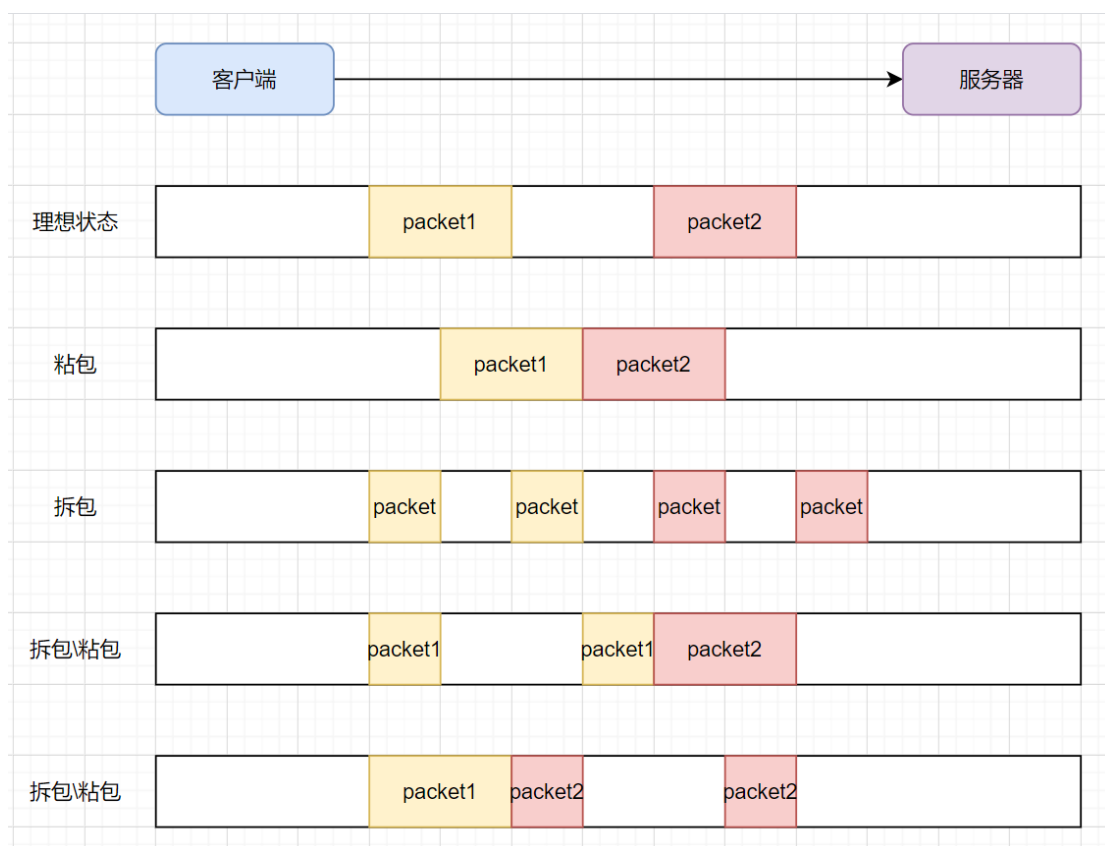
- **syn cookie**：在收到 SYN 包后，服务器根据一定的方法，以数据包的源地址、端口等信息为参数计算出一个 cookie 值作为自己的 SYNACK 包的序列号，回复 SYN+ACK 后，服务器并不立即分配资源进行处理，等收到发送方的 ACK 包后，重新根据数据包的源地址、端口计算该包中的确认序列号是否正确，如果正确则建立连接，否则丢弃该包。
- **SYN Proxy 防火墙**：服务器防火墙会对收到的每一个 SYN 报文进行代理和回应，并保持半连接。等发送方将 ACK 包返回后，再重新构造 SYN 包发到服务器，建立真正的 TCP 连接。

4.41 TCP 的粘包和拆包

思路讲解：这道题真的是屡见不鲜了，什么是 TCP 的粘包？什么是 TCP 的拆包？如何解决粘包和拆包问题？都是需要掌握的知识点。

我的答案如下：

TCP 是面向流，没有界限的一串数据。TCP 底层并不了解上层业务数据的具体含义，它会根据 TCP 缓冲区的实际情况进行包的划分，所以在业务上认为，一个完整的包可能会被 **TCP 拆分成多个包进行发送**，也有可能把多个小的包封装成一个大的数据包发送，这就是所谓的 TCP 粘包和拆包问题。



TCP 的粘包和拆包

为什么会产生粘包和拆包呢？

- 要发送的数据小于 TCP 发送缓冲区的大小，TCP 将多次写入缓冲区的数据一次发送出去，将会发生粘包；
- 接收数据端的应用层没有及时读取接收缓冲区中的数据，将发生粘包；
- 要发送的数据大于 TCP 发送缓冲区剩余空间大小，将会发生拆包；
- 待发送数据大于 MSS（最大报文长度），TCP 在传输前将进行拆包。即 TCP 报文长度 - TCP 头部长度 > MSS。

解决方案：

- 发送端将每个数据包封装为固定长度
- 在数据尾部增加特殊字符进行分割
- 将数据分为两部分，一部分是头部，一部分是内容体；其中头部结构大小固定，且有一个字段声明内容体的大小。

4.42 Nagle 算法与延迟确认

4.42.1 Nagle 算法

如果发送端疯狂地向接收端发送很小的包，比如就 1 个字节，那么亲爱的小伙伴们，你们觉得会有什么问题呢？

TCP/IP 协议中，无论发送多少数据，总是要在数据前面加上协议头，同时，对方接收到数据，也需要发送 ACK 表示确认。为了尽可能的利用网络带宽，TCP 总是希望尽可能的发送足够大的数据。**Nagle 算法**就是为了尽可能发送大块数据，避免网络中充斥着许多小数据块。

Nagle 算法的基本定义是：**任意时刻，最多只能有一个未被确认的小段**。所谓“小段”，指的是小于 MSS 尺寸的数据块，所谓“未被确认”，是指一个数据块发送出去后，没有收到对方发送的 ACK 确认该数据已收到。

Nagle 算法的实现规则：

- 如果包长度达到 MSS，则允许发送；
- 如果该包含有 FIN，则允许发送；
- 设置了 TCP_NODELAY 选项，则允许发送；
- 未设置 TCP_CORK 选项时，若所有发出去的小数据包（包长度小于 MSS）均被确认，则允许发送；
- 上述条件都未满足，但发生了超时（一般为 200ms），则立即发送。

4.42.2 延迟确认

如果接受方刚接收到发送方的数据包，在很短很短的时间内，又接收到第二个包。那么请问接收方是一个一个地回复好点，还是合并一起回复好呢？

接收方收到数据包后，如果暂时没有数据要发给对端，它可以等一段时再确认（Linux 上默认是 40ms）。如果这段时间刚好有数据要传给对端，ACK 就随

着数据传输，而不需要单独发送一次 ACK。如果超过时间还没有数据要发送，也发送 ACK，避免对端以为丢包。

但是有些场景不能延迟确认，比如发现了乱序包、接收到了大于一个 frame 的报文，且需要调整窗口大小等。

一般情况下，Nagle 算法和延迟确认不能一起使用，Nagle 算法意味着延迟发，延迟确认意味着延迟接收，酱紫就会造成更大的延迟，会产生性能问题。

4.43 IP 地址有哪些分类？

一般可以这么认为，IP 地址=网络号+主机号。

- 1. 网络号：它标志主机所连接的网络地址表示属于互联网的哪一个网络。
- 2. 主机号：它标志主机地址表示其属于该网络中的哪一台主机。

IP 地址分为 A，B，C，D，E 五大类：

- A 类地址(1~126)：以 0 开头，网络号占前 8 位，主机号占后面 24 位。
- B 类地址(128~191)：以 10 开头，网络号占前 16 位，主机号占后面 16 位。
- C 类地址(192~223)：以 110 开头，网络号占前 24 位，主机号占后面 8 位。
- D 类地址(224~239)：以 1110 开头，保留位多播地址。
- E 类地址(240~255)：以 11110 开头，保留位为将来使用

A类地址：	0	网络号（7位）	主机号（24位）	1.0.0.1-127.255.255.254
B类地址：	10	网络号（14位）	主机号（16位）	128.0.0.1-191.255.255.254
C类地址：	110	网络号（21位）	主机号（8位）	192.0.0.1-223.255.255.254
D类地址：	1110	组播地址（28位）		224.0.0.1~239.255.255.254
E类地址：	11110	保留为将来使用		240.0.0.1~255.255.255.254

IP 地址分类

4.44 说下 ARP 协议的工作过程？

ARP 协议，**Address Resolution Protocol**，地址解析协议，它是用于实现 IP 地址到 MAC 地址的映射。

- 0. 首先，每台主机都会在自己的 ARP 缓冲区中建立一个 ARP 列表，以表示 IP 地址和 MAC 地址的对应关系。
- 0. 当源主机需要将一个数据包发送到目的主机时，会首先检查自己的 ARP 列表，是否存在该 IP 地址对应的 MAC 地址；如果有就直接将数据包发送到这个 MAC 地址；如果没有，就向本地网段发起一个 ARP 请求的广播包，查询此目的主机对应的 MAC 地址。此 ARP 请求的数据包里，包括源主机的 IP 地址、硬件地址、以及目的主机的 IP 地址。
- 0. 网络中的所有的主机收到这个 ARP 请求后，会检查数据包中的目的 IP 是否和自己的 IP 地址一致。如果不相同，就会忽略此数据包；如果相同，该主机首先将发送端的 MAC 地址和 IP 地址添加到自己的 ARP 列表中，如果 ARP 表中已经存在该 IP 的信息，则将其覆盖，然后给源主机发送一个 ARP 响应数据包，告诉对方自己是它需要查找的 MAC 地址。
- 0. 源主机收到这个 ARP 响应数据包后，将得到的目的主机的 IP 地址和 MAC 地址添加到自己的 ARP 列表中，并利用此信息开始数据的传输。如果源主机一直没有收到 ARP 响应数据包，表示 ARP 查询失败。

4.45 有了 IP 地址，为什么还要用 MAC 地址？

- 简而言之，标识网络中的一台计算机，比较常用的就是 **IP 地址**和 **MAC 地址**，但计算机的 IP 地址可由用户自行更改，管理起来就相对困难，而 MAC 地址不可更改，所以一般会把 IP 地址和 MAC 地址组合起来使用。
- 那只使用 MAC 地址不用 IP 地址行不行呢？不行的！因为最早就是 MAC 地址先出现的，并且当时并不用 IP 地址，只用 MAC 地址，后来随着网络中的设备越来越多，整个路由过程越来越复杂，便出现了子网的概念。对于目的地址在其他子网的数据包，路由只需要将数据包送到那个子网即可。
- 那为什么要用 IP 地址呢？是因为 IP 地址是和地域相关的，对于同一个子网上的设备，IP 地址的前缀都是一样的，这样路由器通过 IP 地址的前缀就知道设备

在在哪个子网上了，而只用 MAC 地址的话，路由器则需要记住每个 MAC 地址在哪个子网，这需要路由器有极大的存储空间，是无法实现的。

- IP 地址可以比作为地址，MAC 地址为收件人，在一次通信过程中，两者是缺一不可的。

4.46 TCP 和 UDP 分别对应的常见应用层协议有哪些？

基于 TCP 的应用层协议有：**HTTP、FTP、SMTP、TELNET、SSH**

- **HTTP**：HyperText Transfer Protocol (超文本传输协议)，默认端口 80
- **FTP**：File Transfer Protocol (文件传输协议)，默认端口(20 用于传输数据，21 用于传输控制信息)
- **SMTP**：Simple Mail Transfer Protocol (简单邮件传输协议)，默认端口 25
- **TELNET**：Teletype over the Network (网络电传)，默认端口 23
- **SSH**：Secure Shell (安全外壳协议)，默认端口 22

基于 UDP 的应用层协议：**DNS、TFTP、SNMP**

- **DNS**：Domain Name Service (域名服务)，默认端口 53
- **TFTP**：Trivial File Transfer Protocol (简单文件传输协议)，默认端口 69
- **SNMP**：Simple Network Management Protocol (简单网络管理协议)，通过 UDP 端口 161 接收，只有 Trap 信息采用 UDP 端口 162。

4.47 聊聊保活计时器的作用

除时间等待计时器外，TCP 还有一个保活计时器 (keepalive timer)。设想这样的场景：客户已主动与服务器建立了 TCP 连接。但后来客户端的主机突然发生故障。显然，服务器以后就不能再收到客户端发来的数据。因此，应当有措施使服务器不要再白白等待下去。这就需要使用保活计时器了。

服务器每收到一次客户的数据，就重新设置保活计时器，时间的设置通常是两个小时。若两个小时都没有收到客户端的数据，服务端就发送一个探测报文段，

以后则每隔 75 秒钟发送一次。若连续发送 10 个探测报文段后仍然无客户端的响应，服务端就认为客户端出了故障，接着就关闭这个连接。

4.48 聊聊 ARP 协议

ARP 协议，地址解析协议，是一个由 IP 地址获取 MAC 物理地址的 TCP/IP 协议。

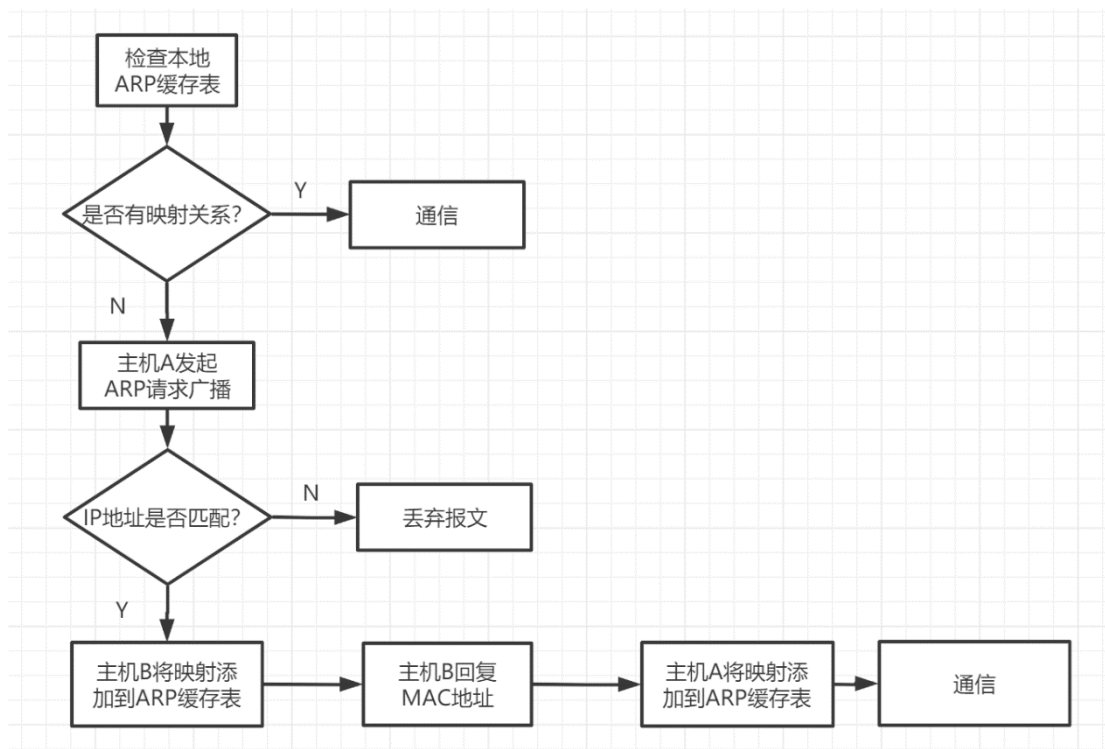
什么是 IP 地址，什么是 MAC 地址？

- IP 地址：是互联网协议地址，它是 IP 协议提供了一种统一的地址格式，它为互联网上的每一个网络和每一台主机分配一个逻辑地址，以此来屏蔽物理地址的差异。
- MAC 地址：以太网地址或物理地址，它是一个用来确认网络设备位置的位址。

为什么需要 ARP 协议呢？

- 在网络访问层中，同一局域网中的一台主机要和另一台主机进行通信，需要通过 MAC 地址进行定位，然后才能进行数据包的发送。
- 而在网络层和传输层中，计算机之间是通过 IP 地址定位目标主机，对应的数据报文只包含目标主机的 IP 地址，而没有 MAC 地址。
- 因此，在发送之前需要根据 IP 地址获取 MAC 地址，然后才能将数据包发送到正确的目标主机，而这个获取过程是通过 ARP 协议完成的。

ARP 的工作流程



当主机 A 与主机 B 要通信时，工作流程如下：

1. 查询本地 ARP 缓存表,看是否有 IP 地址及其对应的 MAC 地址。
2. 如果没匹配到主机 B 的 MAC 地址，主机 A 会在局域网内广播发送一个 ARP 请求分组，局域网内所有主机都会收到该请求分组。
3. 主机 B 收到请求分组报文，发现报文中的 IP 与自己匹配，就 A 的 IP 和 MAC 地址添加到本地 ARP 缓存表中。
4. 主机 B 向主机 A 响应一个含自身 MAC 地址的报文。
5. 主机 A 收到报文后，将 B 的 IP 和 MAC 地址添加至 ARP 缓存表中。

4.49 forward 和 redirect 的区别？

- **直接转发方式 (Forward)**，客户端和浏览器只发出一次请求，Servlet、HTML、JSP 或其它信息资源，由第二个信息资源响应该请求，在请求对象 request 中，保存的对象对于每个信息资源是共享的。
- **间接转发方式 (Redirect)** 实际是两次 HTTP 请求，服务器端在响应第一次请求的时候，让浏览器再向另外一个 URL 发出请求，从而达到转发的目的。

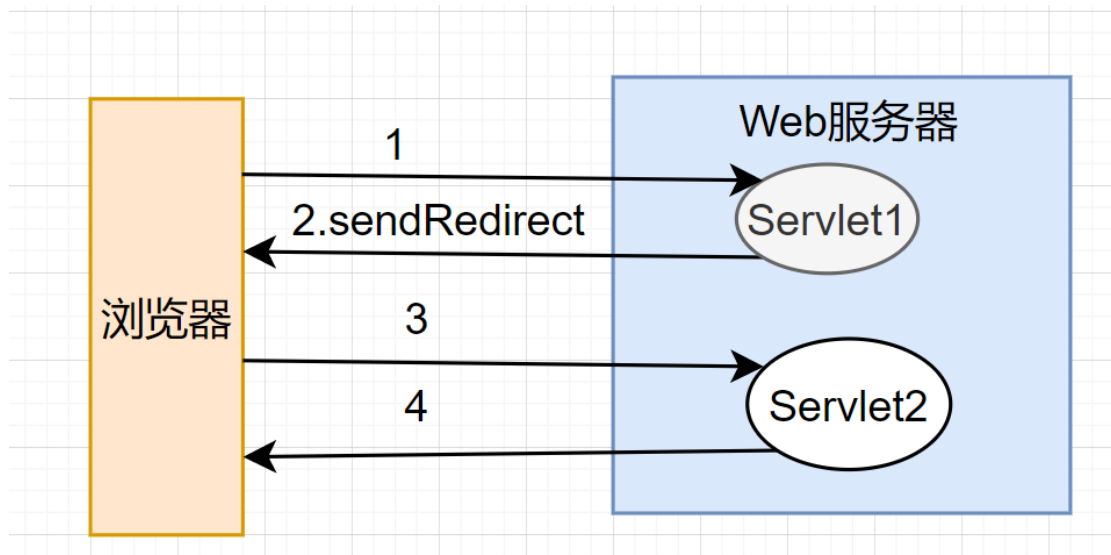
举个通俗的例子：

- 直接转发就相当于：“A 找 B 借钱，B 说没有，B 去找 C 借，借到借不到都会把消息传递给 A”；

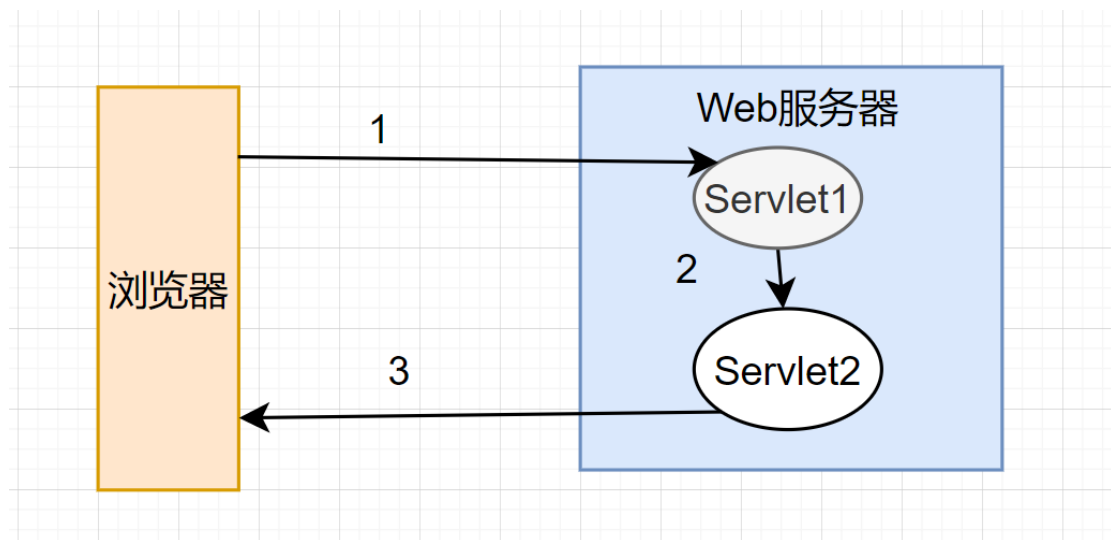
- 间接转发就相当于：“A 找 B 借钱，B 说没有，让 A 去找 C 借”。**

看这两个图，可以更容易理解一些：

- Redirect 的工作原理：



- forward 的工作原理



4.50 URI 和 URL 的区别

- URI，全称是 Uniform Resource Identifier)，中文翻译是统一资源标志符，主要作用是唯一标识一个资源。

- URL，全称是 Uniform Resource Location)，中文翻译是统一资源定位符，主要作用是提供资源的路径。打个经典比喻吧，URI 像是身份证，可以唯一标识一个人，而 URL 更像一个住址，可以通过 URL 找到这个人。

4.51 Session 和 Cookie 的区别。

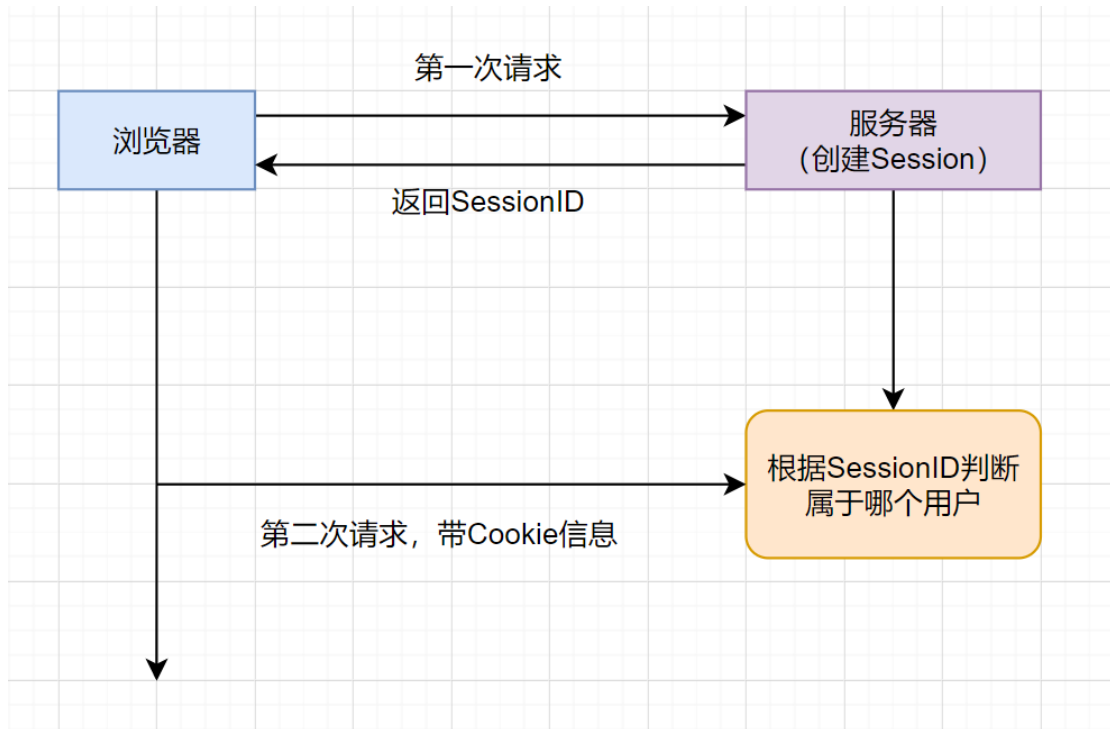
我们先来看 Session 和 Cookie 的概念吧：

- Cookie 是保存在客户端的一小块文本串的数据。客户端向服务器发起请求时，服务端会向客户端发送一个 Cookie，客户端就把 Cookie 保存起来。在客户端下次向同一服务器再发起请求时，Cookie 被携带发送到服务器。服务器就是根据这个 Cookie 来确认身份的。
- session 指的就是服务器和客户端一次会话的过程。Session 利用 Cookie 进行信息处理的，当用户首先进行了请求后，服务端就在用户浏览器上创建了一个 Cookie，当这个 Session 结束时，其实就是意味着这个 Cookie 就过期了。Session 对象存储着特定用户会话所需的属性及配置信息。

Session 和 Cookie 的区别主要有这些：

	Session	Cookie
存放位置	服务器	客户端
存储数据类型	任意数据类型	ASCII
安全性	较高	较低
存储空间大小	大	小
有效期	较短，超时或者客户端关闭都会失效	可设置长时间保存

来看个图吧：



- 用户第一次请求服务器时，服务器根据用户提交的信息，创建对应的 Session，请求返回时将此 Session 的唯一标识信息 SessionID 返回给浏览器，浏览器接收到服务器返回的 SessionID 信息后，会将此信息存入 Cookie 中，同时 Cookie 记录此 SessionID 是属于哪个域名。
- 当用户第二次访问服务器时，请求会自动判断此域名下是否存在 Cookie 信息，如果存在，则自动将 Cookie 信息也发送给服务端，服务端会从 Cookie 中获取 SessionID，再根据 SessionID 查找对应的 Session 信息，如果没有找到，说明用户没有登录或者登录失效，如果找到 Session 证明用户已经登录可执行后面操作。

5.Zookeeper 专栏面试题 (关注公众号：捡田螺的小男孩)

5.1 面试官：工作中使用过 Zookeeper 嘛？你知道它是什么，有什么用途呢？

小菜鸡的我：

- 有使用过的，使用 ZooKeeper 作为 **dubbo** 的注册中心，使用 ZooKeeper 实现分布式锁。
- ZooKeeper，它是一个开源码的**分布式协调服务**，它是一个集群的管理者，它将简单易用的接口提供给用户。
- 可以基于 Zookeeper 实现诸如数据发布/订阅、负载均衡、命名服务、分布式协调/通知、集群管理、Master 选举、分布式锁和分布式队列**等功能**。
- Zookeeper 的**用途**：命名服务、配置管理、集群管理、分布式锁、队列管理

用途跟功能不是一个意思咩？给我一个眼神，让我自己体会



2.2 面试官：说下什么是命名服务，什么是配置管理，又什么是集群管理吧

小菜鸡的我(幸好我刷过面试题)，无所畏惧



- **命名服务就是：**

命名服务是指通过**指定的名字**来获取资源或者服务地址。Zookeeper 可以创建一个**全局唯一的路径**，这个路径就可以作为一个名字。被命名的实体可以是**集群中的机器，服务的地址，或者是远程的对象等**。一些分布式服务框架 (RPC、RMI) 中的服务地址列表，通过使用命名服务，客户端应用能够根据特定的名字来获取资源的实体、服务地址和提供者信息等。

- **配置管理：**：

实际项目开发中，我们经常使用.properties 或者 xml 需要配置很多信息，如数据库连接信息、fps 地址端口等等。因为你的程序一般是分布式部署在不同的机器上（如果你是单机应用当我没说），如果把程序的这些配置信息**保存在 zk 的 znode 节点下**，当你要修改配置，即 znode 会发生变化时，可以通过

改变 zk 中某个目录节点的内容，利用 **watcher** 通知给各个客户端，从而更改配置。

- **集群管理**

集群管理包括集群监控和集群控制，其实就是监控集群机器状态，剔除机器和加入机器。zookeeper 可以方便集群机器的管理，它可以实时监控 znode 节点的变化，一旦发现有机挂挂了，该机器就会与 zk 断开连接，对用的临时目录节点会被删除，其他所有机器都收到通知。新机器加入也是类似酱紫，所有机器收到通知：有新兄弟目录加入啦。

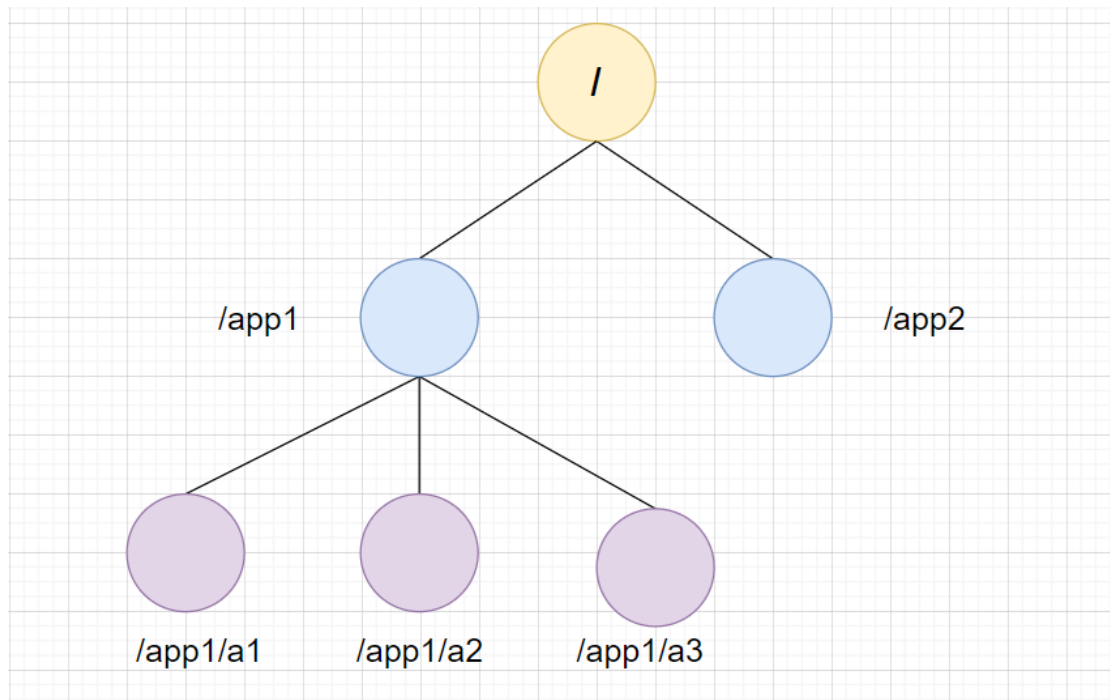
5.3 面试官：你提到了 znode 节点，那你知道 znode 有几种类型呢？zookeeper 的数据模型是怎样的呢？

小菜鸡的我（我先想想）：



zookeeper 的数据模型

ZooKeeper 的视图数据结构，很像 Unix 文件系统，也是树状的，这样可以确定每个路径都是唯一的。zookeeper 的节点统一叫做 **znode**，它是可以通过路径来标识，结构图如下：



znode 的 4 种类型

根据节点的生命周期，znode 可以分为 4 种类型，分别是持久节点 (PERSISTENT)、持久顺序节点 (PERSISTENT_SEQUENTIAL)、临时节点 (EPHEMERAL)、临时顺序节点 (EPHEMERAL_SEQUENTIAL)

- 持久节点 (PERSISTENT)

这类节点被创建后，就会一直存在于 Zk 服务器上。直到手动删除。

- 持久顺序节点 (PERSISTENT_SEQUENTIAL)

它的基本特性同持久节点，不同在于增加了顺序性。父节点会维护一个自增整数数字，用于子节点的创建的先后顺序。

- 临时节点 (EPHEMERAL)

临时节点的生命周期与客户端的会话绑定，一旦客户端会话失效（非 TCP 连接断开），那么这个节点就会被自动清理掉。zk 规定临时节点只能作为叶子节点。

- 临时顺序节点 (EPHEMERAL_SEQUENTIAL)

基本特性同临时节点，添加了顺序的特性。

5.4 面试官：你知道 **znode** 节点里面存储的是什么呢？每个节点的数据最大不能超过多少呢？



我想跟你说

小菜鸡的我：

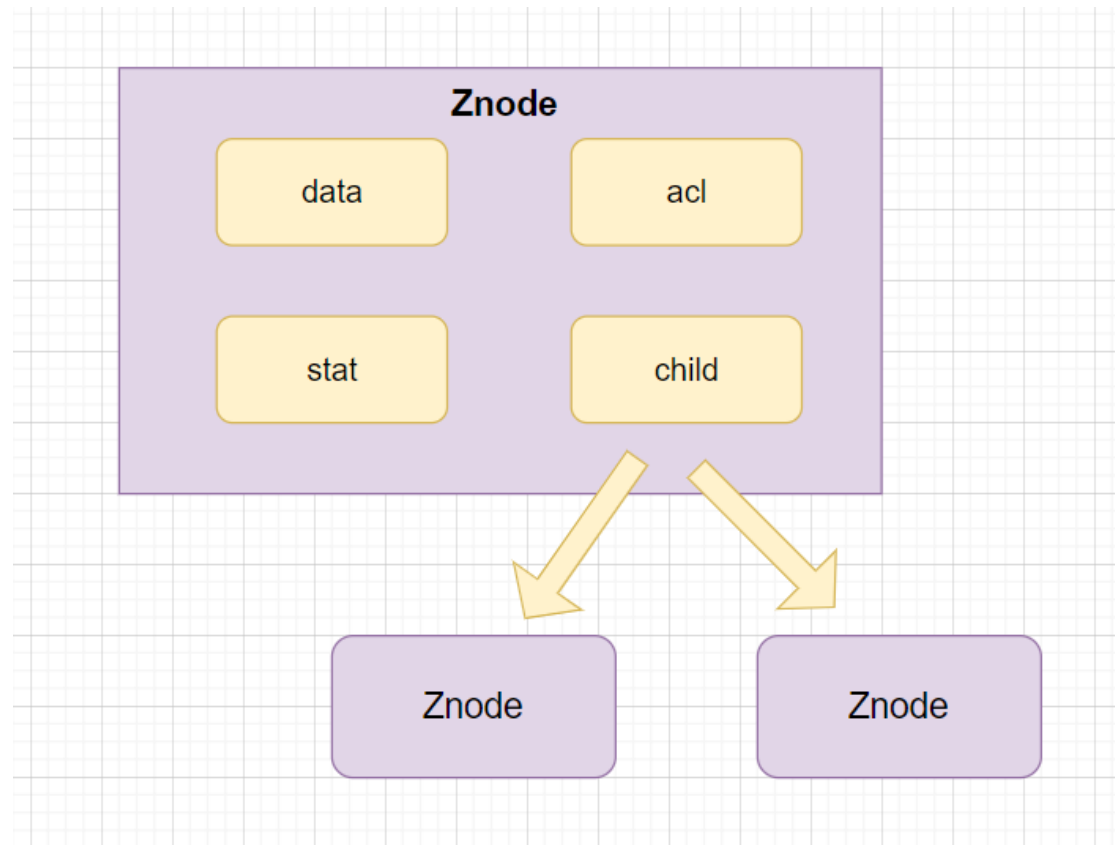
znode 节点里面存储的是什么呢？

Znode 数据节点的代码如下

```
public class DataNode implements Record {  
    byte data[];
```

```
Long acl;  
public StatPersisted stat;  
private Set<String> children = null;  
}
```

哈哈，Znode 包含了存储数据、访问权限、子节点引用、节点状态信息，如图：



- **data:** znode 存储的业务数据信息
- **ACL:** 记录客户端对 znode 节点的访问权限，如 IP 等。
- **child:** 当前节点的子节点引用
- **stat:** 包含 Znode 节点的状态信息，比如事务 id、版本号、时间戳等等。

每个节点的数据最大不能超过多少呢

为了保证高吞吐和低延迟，以及数据的一致性，znode 只适合存储非常小的数据，不能超过 1M，最好都小于 1K。

5.5 面试官：你知道 znode 节点上的监听机制嘛？讲下 Zookeeper watch 机制吧。

小菜鸡的我：



那我就知道了

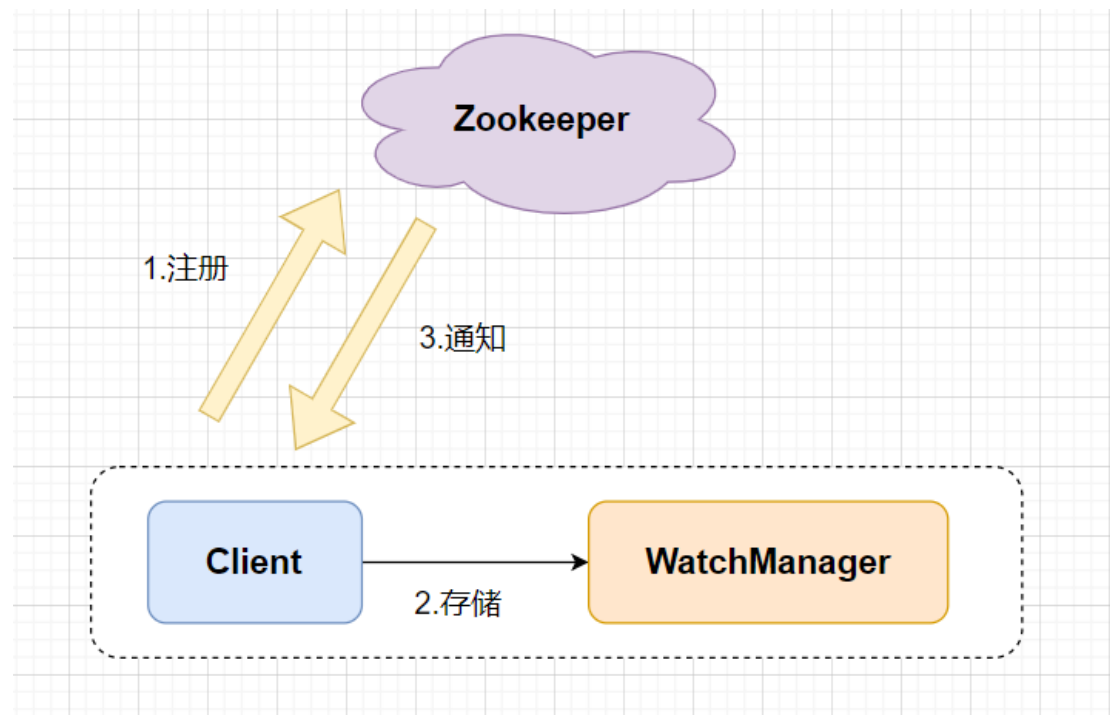
- Watcher 机制
- 监听机制的工作原理
- Watcher 特性总结

Watcher 监听机制

Zookeeper 允许客户端向服务端的某个 Znode 注册一个 Watcher 监听，当服务端的一些指定事件触发了这个 Watcher，服务端会向指定客户端发送一个事件通知来实现分布式的通知功能，然后客户端根据 Watcher 通知状态和事件类型做出业务上的改变。

可以把 Watcher 理解成客户端注册在某个 Znode 上的触发器，当这个 Znode 节点发生变化时（增删改查），就会触发 Znode 对应的注册事件，注册的客户端就会收到异步通知，然后做出业务的改变。

Watcher 监听机制的工作原理



- ZooKeeper 的 Watcher 机制主要包括客户端线程、客户端 WatchManager、Zookeeper 服务器三部分。
- 客户端向 ZooKeeper 服务器注册 Watcher 的同时，会将 Watcher 对象存储在客户端的 WatchManager 中。
- 当 zookeeper 服务器触发 watcher 事件后，会向客户端发送通知，客户端线程从 WatchManager 中取出对应的 Watcher 对象来执行回调逻辑。

Watcher 特性总结

- **一次性**：一个 Watch 事件是一个一次性的触发器。一次性触发，客户端只会收到一次这样的信息。
- ****异步的****：Zookeeper 服务器发送 watcher 的通知事件到客户端是异步的，不能期望能够监控到节点每次的变化，Zookeeper 只能保证最终的一致性，而无法保证强一致性。
- **轻量级**：Watcher 通知非常简单，它只是通知发生了事件，而不会传递事件对象内容。
- **客户端串行**：执行客户端 Watcher 回调的过程是一个串行同步的过程。
- 注册 watcher 用 `getData`、`exists`、`getChildren` 方法
- 触发 watcher 用 `create`、`delete`、`setData` 方法

5.6 面试官：你对 Zookeeper 的数据结构都有一定了解，那你讲下 Zookeeper 的特性吧

小菜鸡的我：（我背过书，啊哈哈）



Zookeeper 保证了

如下分布式一致性特性：

- **顺序一致性**：从同一客户端发起的事务请求，最终将会严格地按照顺序被应用到 ZooKeeper 中去。
- **原子性**：所有事务请求的处理结果在整个集群中所有机器上的应用情况是一致的，也就是说，要么整个集群中所有的机器都成功应用了某一个事务，要么都没有应用。
- **单一视图**：无论客户端连到哪一个 ZooKeeper 服务器上，其看到的服务端数据模型都是一致的。
- **可靠性**：一旦服务端成功地应用了一个事务，并完成对客户端的响应，那么该事务所引起的服务端状态变更将会被一直保留下来。
- **实时性（最终一致性）**：Zookeeper 仅仅能保证在一定的时间段内，客户端最终一定能够从服务端上读取到最新的数据状态。

5.7 面试官：你刚提到顺序一致性，那 zookeeper 是如何保证事务的顺序一致性的呢？

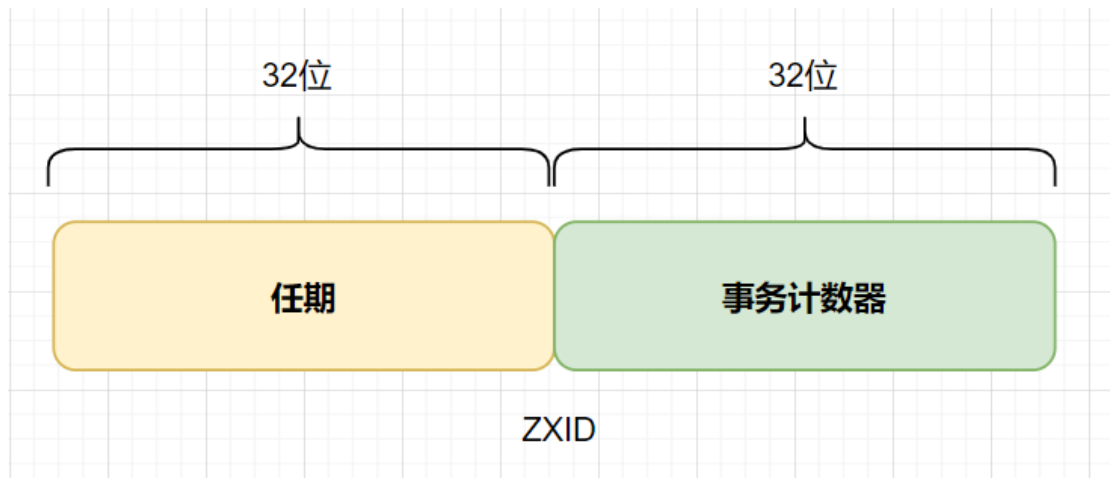


小菜鸡的我：（完蛋了这题不会）

这道题可以看下这篇文章（本题答案来自该文章）：[聊一聊 ZooKeeper 的顺序一致性](#)

需要了解事务 ID，即 zxid。ZooKeeper 的在选举时通过比较各结点的 zxid 和机器 ID 选出新的主结点的。zxid 由 Leader 节点生成，有新写入事件时，Leader 生成新 zxid 并随提案一起广播，每个结点本地都保存了当前最近一次事务的 zxid，zxid 是递增的，所以谁的 zxid 越大，就表示谁的数据是最新的。

ZXID 的生成规则如下：



ZXID 有两部分组成：

- 任期：完成本次选举后，直到下次选举前，由同一 Leader 负责协调写入；
- 事务计数器：单调递增，每生效一次写入，计数器加一。

ZXID 的低 32 位是计数器，所以同一任期内，ZXID 是连续的，每个结点又都保存着自身最新生效的 ZXID，通过对比新提案的 ZXID 与自身最新 ZXID 是否相差“1”，来保证事务严格按照顺序生效的。

5.8 面试官：你提到了 Leader，你知道 Zookeeper 的服务
器有几种角色嘛？Zookeeper 下 Server 工作状态又有几种
呢？



小菜鸡的我：

Zookeeper 服务器角色

Zookeeper 集群中，有 Leader、Follower 和 Observer 三种角色

Leader

Leader 服务器是整个 ZooKeeper 集群工作机制中的核心，其主要工作：

- 事务请求的唯一调度和处理者，保证集群事务处理的顺序性
- 集群内部各服务的调度者

Follower

Follower 服务器是 ZooKeeper 集群状态的跟随者，其主要工作：

- 处理客户端非事务请求，转发事务请求给 Leader 服务器
- 参与事务请求 Proposal 的投票
- 参与 Leader 选举投票

Observer

Observer 是 3.3.0 版本开始引入的一个服务器角色，它充当一个观察者角色——观察 ZooKeeper 集群的最新状态变化并将这些状态变更同步过来。其工作：

- 处理客户端的非事务请求，转发事务请求给 Leader 服务器
- 不参与任何形式的投票

Zookeeper 下 Server 工作状态

服务器具有四种状态，分别是

LOOKING、FOLLOWING、LEADING、OBSERVING。

- 1.LOOKING：寻找 Leader 状态。当服务器处于该状态时，它会认为当前集群中没有 Leader，因此需要进入 Leader 选举状态。
- 2.FOLLOWING：跟随者状态。表明当前服务器角色是 Follower。
- 3.LEADING：领导者状态。表明当前服务器角色是 Leader。
- 4.OBSERVING：观察者状态。表明当前服务器角色是 Observer。

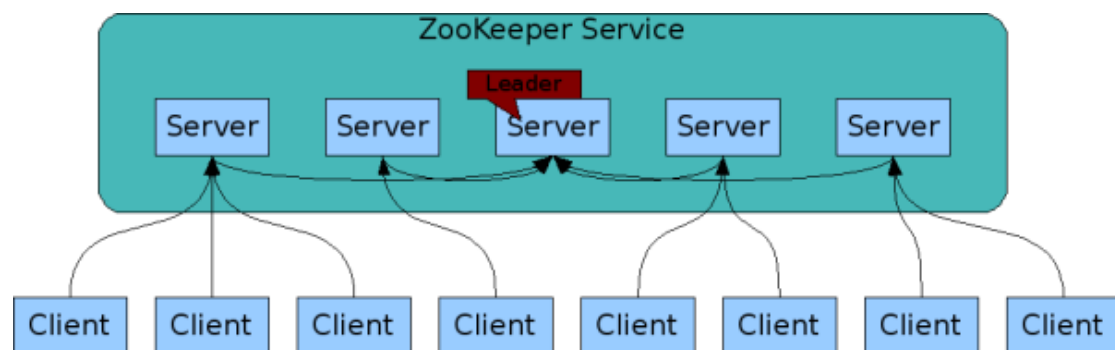
5.9 面试官：你说到服务器角色是基于 ZooKeeper 集群的，那你画一下 ZooKeeper 集群部署图吧？ZooKeeper 是如何保证主从节点数据一致性的呢？



心好累

小菜鸡的我：

ZooKeeper 集群部署图



ZooKeeper 集群是一主多从的结构：

- 如果是写入数据，先写入主服务器（主节点），再通知从服务器。
- 如果是读取数据，既读主服务器的，也可以读从服务器的。

ZooKeeper 如何保证主从节点数据一致性

我们知道集群是主从部署结构，要保证主从节点一致性问题，无非就是两个主要问题：

- 主服务器挂了，或者重启了
- 主从服务器之间同步数据~

Zookeeper 是采用 ZAB 协议 (Zookeeper Atomic

Broadcast , Zookeeper 原子广播协议) 来保证主从节点数据一致性的，

ZAB 协议支持崩溃恢复和消息广播两种模式，很好解决了这两个问题：

- 崩溃恢复：Leader 挂了，进入该模式，选一个新的 leader 出来
- 消息广播：把更新的数据，从 Leader 同步到所有 Follower

Leader 服务器挂了，所有集群中的服务器进入 LOOKING 状态，首先，它们会选举产生新的 Leader 服务器；接着，新的 Leader 服务器与集群中 Follower 服务进行数据同步，当集群中超过半数机器与该 Leader 服务器完成数据同步之后，退出恢复模式进入消息广播模式。Leader 服务器开始接收客户端的事务请求生成事务 Proposal 进行事务请求处理。

5.10 面试官：Leader 挂了，进入崩溃恢复，是如何选举 Leader 的呢？你讲一下 ZooKeeper 选举机制吧

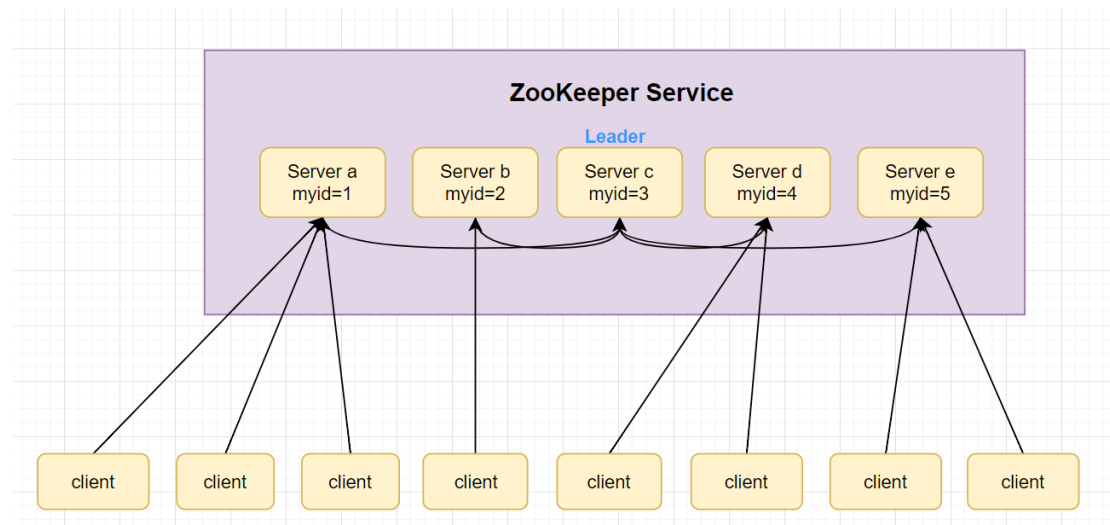
小菜鸡的我：



我真的太难了

服务器启动或者服务器运行期间（Leader 挂了），都会进入 Leader 选举，我们来看一下~假设现在 ZooKeeper 集群有五台服务器，它们 myid 分别是服务

器 1、2、3、4、5，如图：



服务器启动的 Leader 选举

zookeeper 集群初始化阶段，服务器 (myid=1-5) 依次启动，开始

zookeeper 选举 Leader~



- 服务器 1 (myid=1) 启动，当前只有一台服务器，无法完成 Leader 选举
- 服务器 2 (myid=2) 启动，此时两台服务器能够相互通讯，开始进入 Leader 选举阶段

1. 每个服务器发出一个投票

服务器 1 和 服务器 2 都将自己作为 Leader 服务器进行投票，投票的基本元素包括：服务器的 myid 和 ZXID，我们以 (myid , ZXID) 形式表示。初始阶段，服务器 1 和服务器 2 都会投给自己，即服务器 1 的投票为 (1,0)，服务器 2 的投票为 (2,0)，然后各自将这个投票发给集群中的其他所有机器。

1. 接受来自各个服务器的投票

每个服务器都会接受来自其他服务器的投票。同时，服务器会校验投票的有效性，是否本轮投票、是否来自 LOOKING 状态的服务器。

1. 处理投票

收到其他服务器的投票，会将别人的投票跟自己的投票 PK，PK 规则如下：

- 优先检查 ZXID。ZXID 比较大的服务器优先作为 leader。
- 如果 ZXID 相同的话，就比较 myid，myid 比较大的服务器作为 leader。服务器 1 的投票是 (1,0)，它收到投票是 (2,0)，两者 zxid 都是 0，因为收到的 myid=2，大于自己的 myid=1，所以它更新自己的投票为 (2,0)，然后重新将投票发出去。对于服务器 2 呢，即不再需要更新自己的投票，把上一次的投票信息发出即可。

1. 统计投票

每次投票后，服务器会统计所有投票，判断是否有过半的机器接受到相同的投票信息。服务器 2 收到两票，少于 3 ($n/2+1$, n 为总服务器)，所以继续保持 LOOKING 状态

- 服务器 3 (myid=3) 启动，继续进入 Leader 选举阶段

跟前面流程一致，服务器 1 和 2 先投自己一票，因为服务器 3 的 myid 最大，所以大家把票改投给它。此时，服务器为 3 票 (大于等于 $n/2+1$)，所以服务器 3 当选为 Leader。服务器 1，2 更改状态为 FOLLOWING，服务器 3 更改状态为 LEADING；

- 服务器 4 启动，发起一次选举。

此时服务器 1, 2, 3 已经不是 LOOKING 状态，不会更改选票信息。选票信息结果：服务器 3 为 3 票，服务器 4 为 1 票。服务器 4 并更改状态为 FOLLOWING；

- 服务器 5 启动，发起一次选举。

同理，服务器也是把票投给服务器 3，服务器 5 并更改状态为 FOLLOWING；

- 投票结束，服务器 3 当选为 Leader

服务器运行期间的 Leader 选举

zookeeper 集群的五台服务器 (myid=1-5) 正在运行中，突然某个瞬间，Leader 服务器 3 挂了，这时候便开始 Leader 选举～



- 1.变更状态

Leader 服务器挂了之后，余下的非 Observer 服务器都会把自己的服务器状态更改为 LOOKING，然后开始进入 Leader 选举流程。

- 2.每个服务器发起投票

每个服务器都把票投给自己，因为是运行期间，所以每台服务器的 ZXID 可能不相同。假设服务 1,2,4,5 的 zxid 分别为 333,666,999,888，则分别产生投票 (1,333)，(2, 666)，(4,999) 和 (5,888)，然后各自将这个投票发给集群中的其他所有机器。

- 3.接受来自各个服务器的投票
- 4.处理投票

投票规则是跟 Zookeeper 集群启动期间一致的，优先检查 ZXID，大的优先作为 Leader，所以显然服务器 zxid=999 具有优先权。

- 5.统计投票
- 6.改变服务器状态

5.11 面试官：你前面提到在项目中使用过 Zookeeper 的分布式锁，讲一下 zk 分布式锁的实现原理吧？



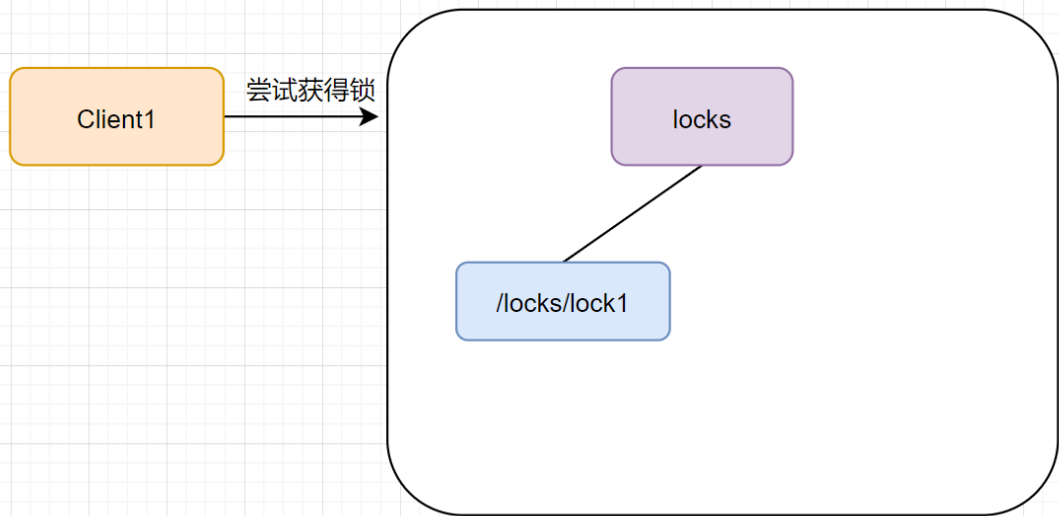
小菜鸡的我：

Zookeeper 就是使用临时顺序节点特性实现分布式锁的。

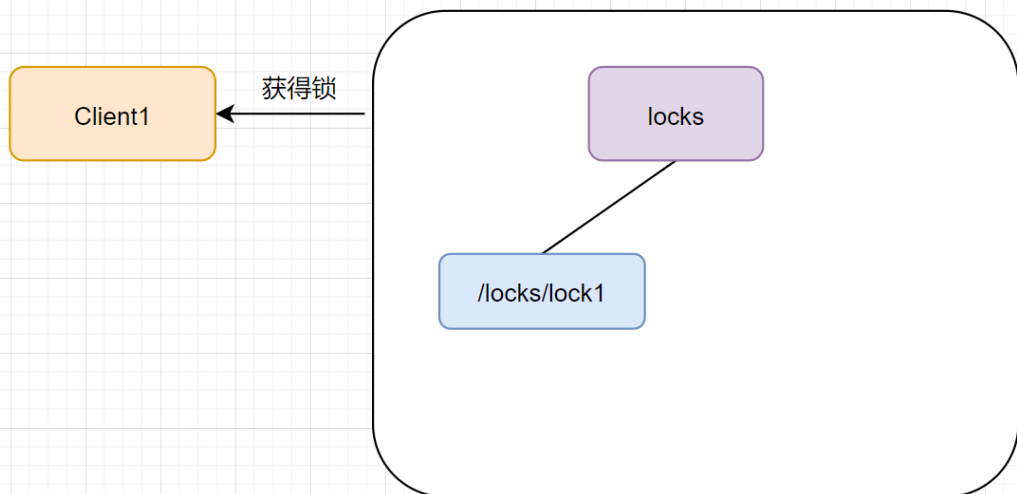
- 获取锁过程（创建临时节点，检查序号最小）
- 释放锁（删除临时节点，监听通知）

获取锁过程

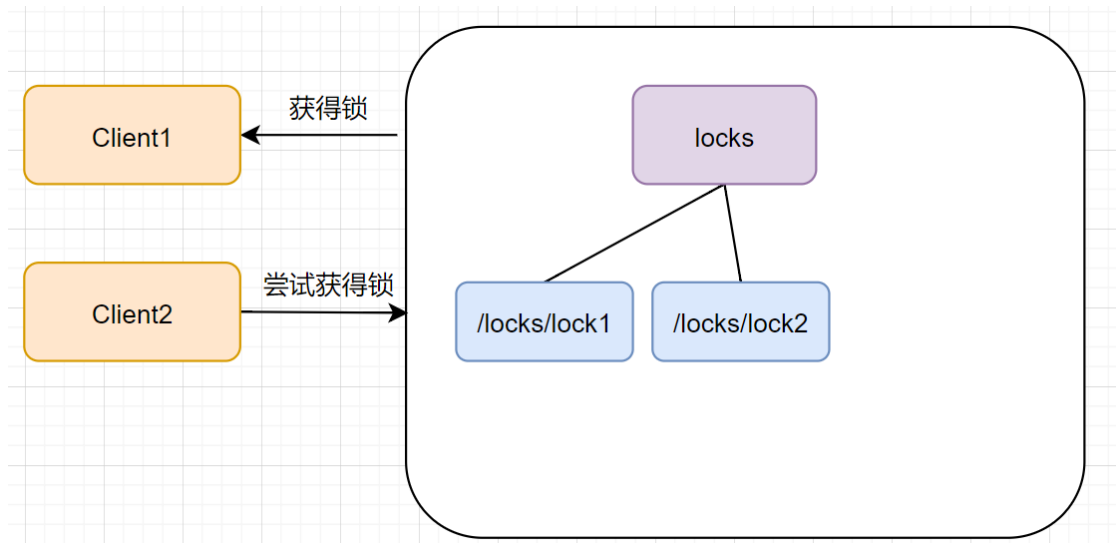
- 当第一个客户端请求过来时，Zookeeper 客户端会创建一个持久节点/locks。如果它 (Client1) 想获得锁，需要在 locks 节点下创建一个顺序节点 lock1.如图



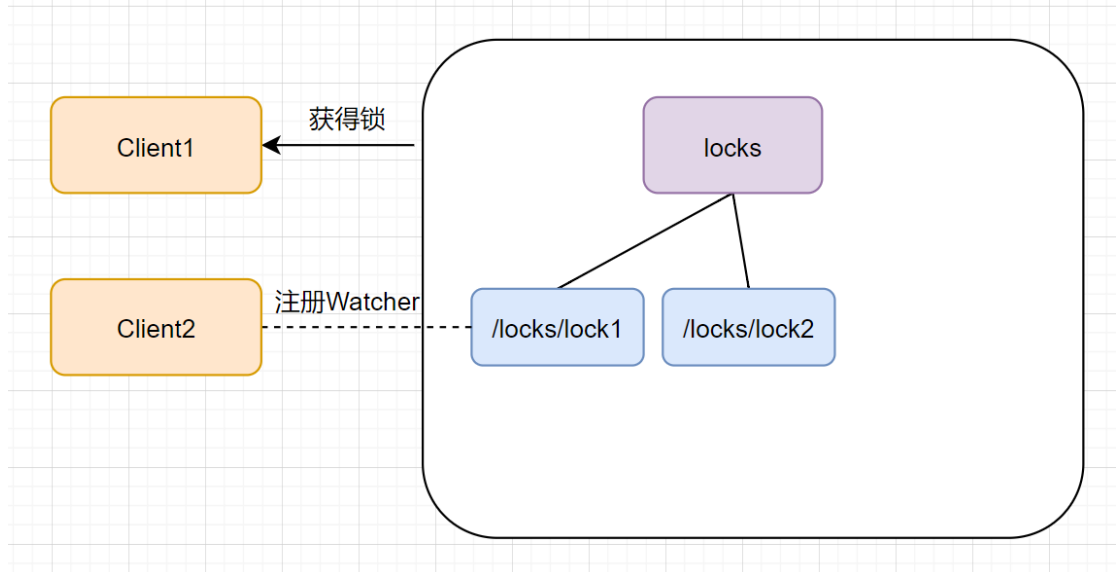
- 接着，客户端 Client1 会查找 locks 下面的所有临时顺序子节点，判断自己的节点 lock1 是不是排序最小的那一个，如果是，则成功获得锁。



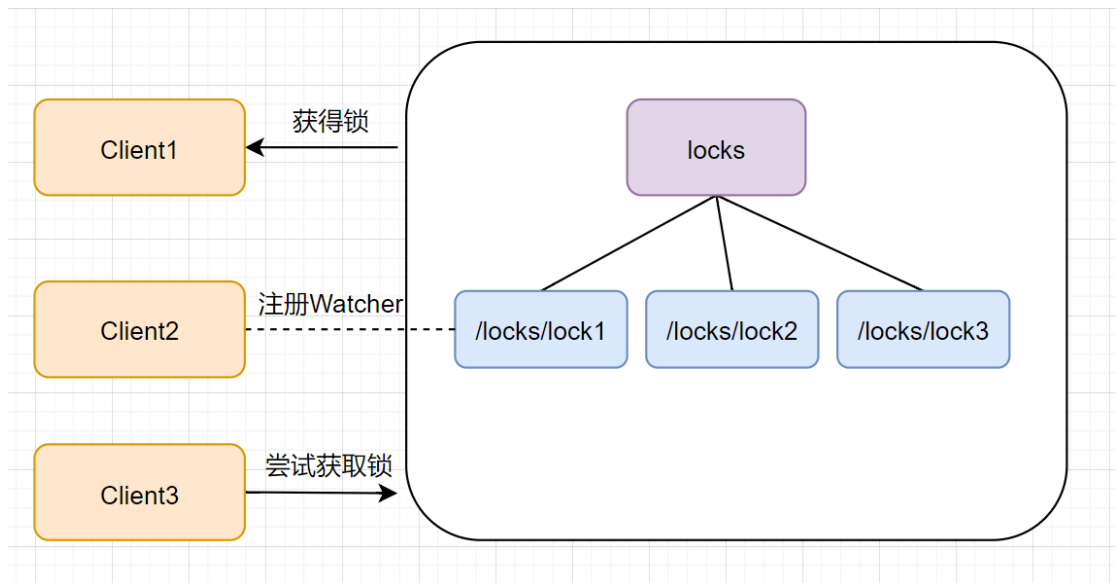
- 这时候如果又来一个客户端 client2 前来尝试获得锁，它会在 locks 下再创建一个临时节点 lock2



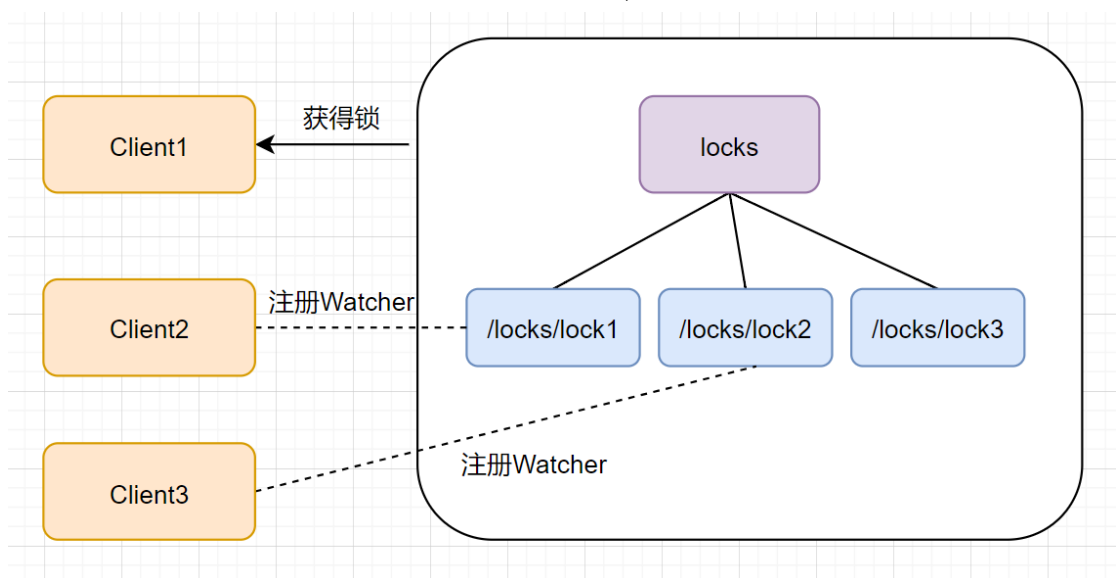
- 客户端 client2 一样也会查找 locks 下面的所有临时顺序子节点，判断自己的节点 lock2 是不是最小的，此时，发现 lock1 才是最小的，于是获取锁失败。获取锁失败，它是不会甘心的，client2 向它排序靠前的节点 lock1 注册 Watcher 事件，用来监听 lock1 是否存在，也就是说 client2 抢锁失败进入等待状态。



- 此时，如果再来一个客户端 Client3 来尝试获取锁，它会在 locks 下再创建一个临时节点 lock3

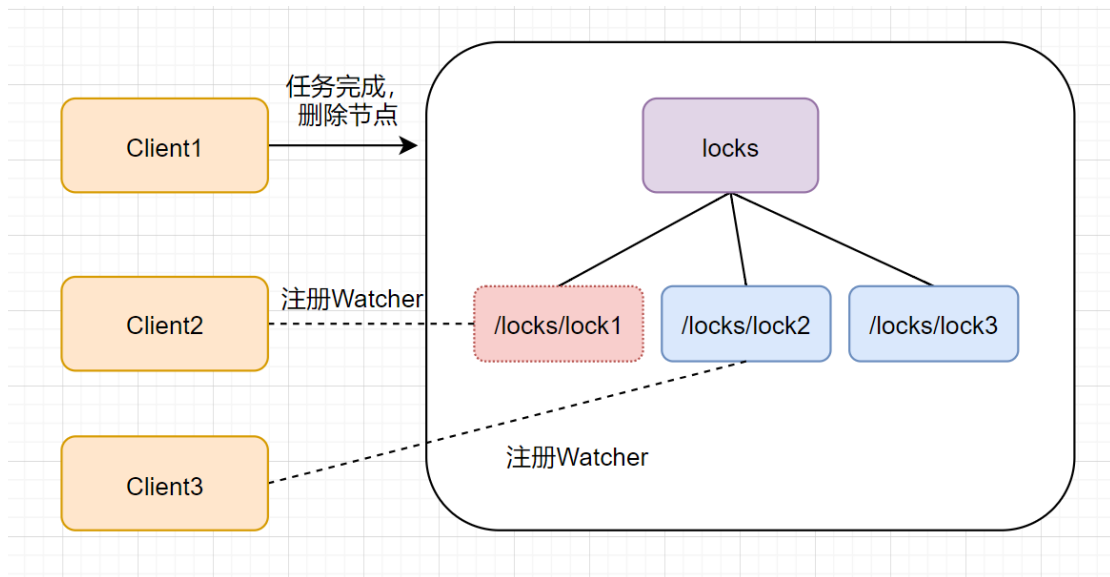


- 同样的，client3 一样也会查找 locks 下面的所有临时顺序子节点，判断自己的节点 lock3 是不是最小的，发现自己不是最小的，就获取锁失败。它也是不会甘心的，它会向在它前面的节点 lock2 注册 Watcher 事件，以监听 lock2 节点是否存在。

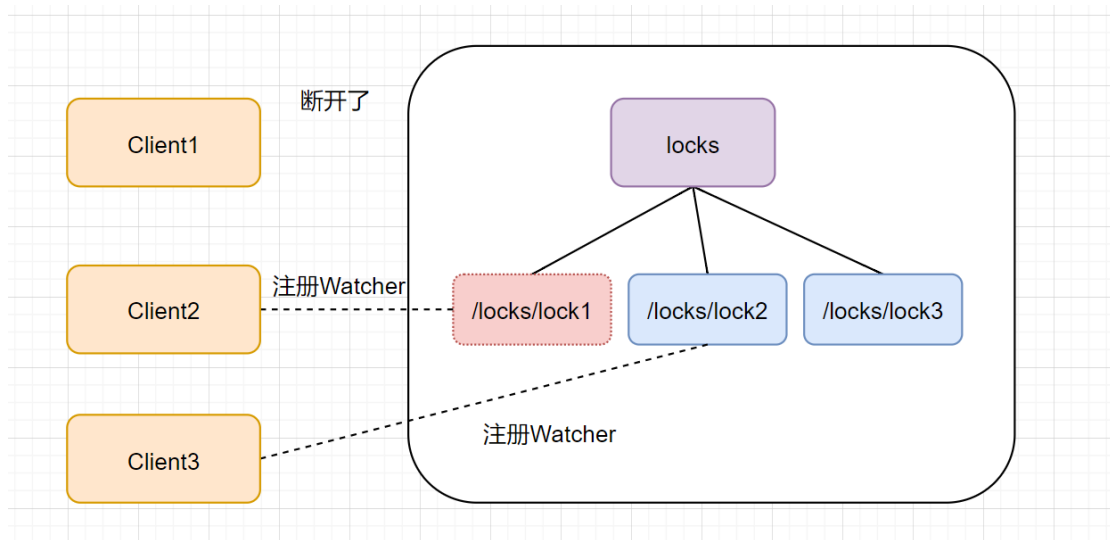


释放锁

我们再来看看释放锁的流程，zookeeper 的客户端业务完成或者故障，都会删除临时节点，释放锁。如果是任务完成，Client1 会显式调用删除 lock1 的指令

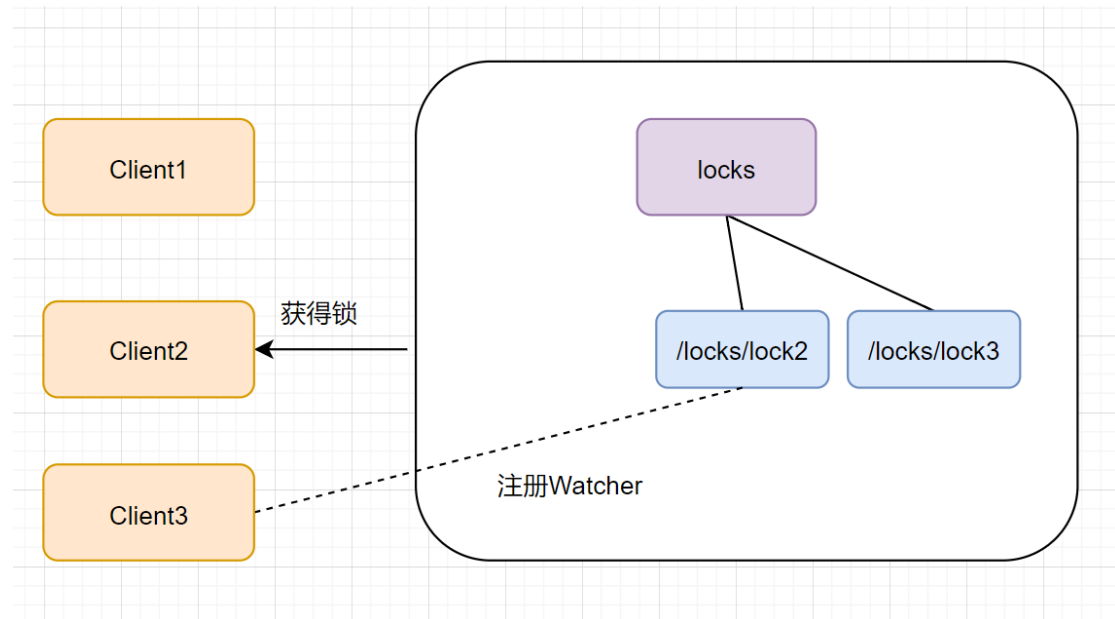


如果是客户端故障了，根据临时节点的特性，lock1 是会自动删除的



lock1 节点被删除后，Client2 可开心了，因为它一直监听着 lock1。lock1 节点删除，Client2 立刻收到通知，也会查找 locks 下面的所有临时顺序子节点，

发下 lock2 是最小，就获得锁。



同理，Client2 获得锁之后，Client3 也对它虎视眈眈，啊哈哈~

5.12 面试官：好的，最后一道题，你说说 dubbo 和 Zookeeper 的关系吧，为什么选择 Zookeeper 作为注册中心？

小菜鸡的我（答了这么多道题，不会还不给我过吧？）：

面试通过的时候



dubbo 的注册中心可以选 Zookeeper，memcached，redis 等。为什么选择 Zookeeper，因为它的功能特性咯～

- 命名服务，服务提供者向 Zookeeper 指定节点写入 url，完成服务发布。
- 负载均衡，注册中心的承载能力有限，而 Zookeeper 集群配合 web 应用很容易达到负载均衡。
- zk 支持监听事件，特别适合发布/订阅的场景，dubbo 的生产者和消费者就类似这场景。
- 数据模型简单，数据存在内存，可谓高性能
- Zookeeper 其他特点都可以搬出来讲一下～

6.Java 基础面试专栏（关注公众号：捡田螺的小男孩）

更多干货，关注公众号：捡田螺的小男孩

6.1 equals 与==的区别

==

- 如果是基本类型，==表示判断它们值是否相等；
- 如果是引用对象，==表示判断两个对象指向的内存地址是否相同。

equals

- 如果是字符串，表示判断字符串内容是否相同；
- 如果是 object 对象的方法，比较的也是引用的内存地址值；
- 如果自己的类重写 equals 方法，可以自定义两个对象是否相等。

6.2 final, finally, finalize 的区别

- final 用于修饰属性,方法和类, 分别表示属性不能被重新赋值, 方法不可被覆盖, 类不可被继承.
- finally 是异常处理语句结构的一部分，一般以 try-catch-finally 出现，finally 代码块表示总是被执行.
- finalize 是 Object 类的一个方法，该方法一般由垃圾回收器来调用，当我们调用 System.gc() 方法的时候，由垃圾回收器调用 finalize()方法，回收垃圾，JVM 并不保证此方法总被调用.

6.3 重载和重写的区别

- 重写必须继承，重载不用。
- 重载表示同一个类中可以有多个名称相同的方法，但这些方法的参数列表各不相同（即参数个数或类型不同）
- 重写表示子类中的方法与父类中的某个方法的名称和参数完全相同啦，通过子类实例对象调用这个方法时，将调用子类中的定义方法，这相当于把父类中定义的那个完全相同的方法给覆盖了，这是面向对象编程的多态性的一种表现。
- 重写的方法修饰符大于等于父类的方法，即访问权限只能比父类的更大，不能更小，而重载和修饰符无关。
- 重写覆盖的方法中，只能比父类抛出更少的异常，或者是抛出父类抛出的异常的子异常，因为不能坑爹啊哈哈~

6.4 两个对象的 hashCode()相同，则 equals()是否也一定为 true？

两个对象 equals 相等，则它们的 hashCode 必须相等，如果两个对象的

hashCode()相同，则 equals()不一定为 true。

hashCode 的常规协定：

- 在 Java 应用程序执行期间，在对同一对象多次调用 hashCode 方法时，必须一致地返回相同的整数，前提是将对象进行 equals 比较时所用的信息没有被修改。从某一应用程序的一次执行到同一应用程序的另一次执行，该整数无需保持一致。
- 两个对象的 equals() 相等，那么对这两个对象中的每个对象调用 hashCode 方法都必须生成相同的整数结果。
- 两个对象的 equals() 不相等，那么对这两个对象中的任一对象上调用 hashCode 方法不要求一定生成不同的整数结果。但是，为不相等的对象生成不同整数结果可以提高哈希表的性能。

6.5 抽象类和接口有什么区别

- 抽象类要被子类继承，接口要被子类实现。
- 抽象类可以有构造方法，接口中不能有构造方法。
- 抽象类中可以有普通成员变量，接口中没有普通成员变量，它的变量只能是公共的静态的常量
- 一个类可以实现多个接口，但是只能继承一个父类，这个父类可以是抽象类。
- 接口只能做方法声明，抽象类中可以作方法声明，也可以做方法实现。
- 抽象级别（从高到低）：接口 > 抽象类 > 实现类。
- 抽象类主要是用来抽象类别，接口主要是用来抽象方法功能。
- 抽象类的关键字是 abstract，接口的关键字是 interface

6.6 BIO、NIO、AIO 有什么区别？

- BIO：线程发起 IO 请求，不管内核是否准备好 IO 操作，从发起请求起，线程一直阻塞，直到操作完成。
- NIO：线程发起 IO 请求，立即返回；内核在做好 IO 操作的准备之后，通过调用注册的回调函数通知线程做 IO 操作，线程开始阻塞，直到操作完成。
- AIO：线程发起 IO 请求，立即返回；内存做好 IO 操作的准备之后，做 IO 操作，直到操作完成或者失败，通过调用注册的回调函数通知线程做 IO 操作完成或者失败。

BIO 是一个连接一个线程。，NIO 是一个请求一个线程。，AIO 是一个有效请求一个线程。

- BIO：同步并阻塞，服务器实现模式为一个连接一个线程，即客户端有连接请求时服务器端就需要启动一个线程进行处理，如果这个连接不做任何事情会造成不必要的线程开销，当然可以通过线程池机制改善。
- NIO：同步非阻塞，服务器实现模式为一个请求一个线程，即客户端发送的连接请求都会注册到多路复用器上，多路复用器轮询到连接有 I/O 请求时才启动一个线程进行处理。
- AIO：异步非阻塞，服务器实现模式为一个有效请求一个线程，客户端的 IO 请求都是由 OS 先完成了再通知服务器应用去启动线程进行处理。

大家可以看我这篇文章哈，[看一遍就理解：IO 模型详解](#)

6.7 String，Stringbuffer，StringBuilder 的区别

String：

- String 类是一个不可变的类，一旦创建就不可以修改。
- String 是 final 类，不能被继承
- String 实现了 equals()方法和 hashCode()方法

StringBuffer：

- 继承自 AbstractStringBuilder，是可变类。
- StringBuffer 是线程安全的
- 可以通过 append 方法动态构造数据。

StringBuilder：

- 继承自 AbstractStringBuilder，是可变类。
- StringBuilder 是非线性安全的。
- 执行效率比 StringBuffer 高。

6.8 JAVA 中的几种基本数据类型是什么，各自占用多少字节呢

基本类型	位数	字节
int	32	4

基本类型	位数	字节
short	16	2
long	64	8
byte	8	1
char	16	2
float	32	4
double	64	8
boolean	?	?

对于 boolean，官方文档未明确定义，它依赖于 JVM 厂商的具体实现。逻辑

上理解是占用 1 位，但是实际中会考虑计算机高效存储因素

感兴趣的小伙伴，可以去看[官网](#)

6.9 Comparator 与 Comparable 有什么区别？

在牛客网看到这道题的答案，觉得写的最好~

链接：[https://www.nowcoder.com/questionTerminal/](https://www.nowcoder.com/questionTerminal/99f7d1f4f8374e419a6d6924d35d9530)

99f7d1f4f8374e419a6d6924d35d9530 来源：牛客网

- Comparable & Comparator 都是用来实现集合中元素的比较、排序的，只是 Comparable 是在集合内部定义的方法实现的排序，Comparator 是在集合外部实现的排序，所以，如想实现排序，就需要在集合外定义 Comparator 接口的方法或在集合内实现 Comparable 接口的方法。
- Comparator 位于包 java.util 下，而 Comparable 位于包 java.lang 下。
- Comparable 是一个对象本身就已经支持自比较所需要实现的接口（如 String、Integer 自己就可以完成比较大小的操作，已经实现了 Comparable 接口）自定义的类要在加入 list 容器中后能够排序，可以实现 Comparable 接口，在用 Collections 类的 sort 方法排序时，如果不指定 Comparator，那么就以自然顺序排序，这里的自然顺序就是实现 Comparable 接口设定的排序方式。
- 而 Comparator 是一个专用的比较器，当这个对象不支持自比较或者自比较函数不能满足你的要求时，你可以写一个比较器来完成两个对象之间大小的比较。

- 可以说一个是自己完成比较，一个是外部程序实现比较的差别而已。用 Comparator 是策略模式 (strategy design pattern)，就是不改变对象自身，而用一个策略对象 (strategy object) 来改变它的行为。比如：你想对整数采用绝对值大小来排序，Integer 是不符合要求的，你不需要去修改 Integer 类 (实际上你也不能这么做) 去改变它的排序行为，只要使用一个实现了 Comparator 接口的对象来实现控制它的排序就行了。

6.10 String 类能被继承吗，为什么。

首先，String 是一个 final 修饰的类，final 修饰的类不可以被继承。

```
public final class String
    implements java.io.Serializable, Comparable<String>, CharSequence {
```

String 类为什么不能被继承呢？

有两个原因：

- 效率性，String 类作为最常用的类之一，禁止被继承和重写，可以提高效率。
- 安全性，String 类中有很多调用底层的本地方法，调用了操作系统的 API，如果方法可以重写，可能被植入恶意代码，破坏程序。

6.11 说说Java 中多态的实现原理

- 多态机制包括静态多态 (编译时多态) 和动态多态 (运行时多态)
- 静态多态比如说重载，动态多态一般指在运行时才能确定调用哪个方法。
- 我们通常所说的多态一般指运行时多态，也就是编译时不确定究竟调用哪个具体方法，一直等到运行时才能确定。
- 多态实现方式：子类继承父类 (extends) 和类实现接口 (implements)
- 多态核心之处就在于对父类方法的改写或对接口方法的实现，以取得在运行时不同的执行效果。
- Java 里对象方法的调用是依靠类信息里的方法表实现的，对象方法引用调用和接口方法引用调用的大致思想是一样的。当调用对象的某个方法时，JVM 查找该对象类的方法表以确定该方法的直接引用地址，有了地址后才真正调用该方法。

举个例子吧，假设有个 Fruit 父类，一个 taste 味道方法，两个子类 Apple 和 Pear，如下：

```

abstract class Fruit {
    abstract String taste() ;
}

class Apple extends Fruit {
    @Override
    String taste() {
        return "酸酸的";
    }
}

class Pear extends Fruit {
    @Override
    String taste() {
        return "甜甜的";
    }
}

public class Test {
    public static void main(String[] args) {
        Fruit f = new Apple();
        System.out.println(f.taste());
    }
}

```

程序运行，当调用对象 Fruit f 的方法 taste 时，JVM 查找 Fruit 对象类的方法表以确定 taste 方法的直接引用地址，到底来自 Apple 还是 Pear，确定后才真正调用对应子类的 taste 方法，

6.12 Java 泛型和类型擦除

这个面试题，可以看我这篇文章哈～

[Java 程序员必备基础：泛型解析](#)

6.13 int 和 Integer 有什么区别，还有 Integer 缓存的实现

这里考察 3 个知识点吧：

- int 是基本数据类型，Integer 是 int 的封装类
- int 默认值为 0，而 Integer 默认值为 null，Integer 使用需要判空处理
- Integer 的缓存机制：为了节省内存和提高性能，Integer 类在内部通过使用相同的对象引用实现缓存和重用，Integer 类默认在 -128 ~ 127 之间，可以通过 -XX:AutoBoxCacheMax 进行修改，且这种机制仅在自动装箱的时候有用，在使用构造器创建 Integer 对象时无用。

看个 Integer 的缓存的例子，加深一下印象哈：

```
Integer a = 10;
```

```
Integer b = 10;
```

```
Integer c = 129;
```

```
Integer d = 129;
```

```
System.out.println(a == b);
```

```
System.out.println(c == d);
```

输出结果：

```
true
```

```
false
```

6.14 说说反射的用途及实现原理，Java 获取反射的三种方法

这道面试题，看我这篇文章哈：[谈谈 Java 反射：从入门到实践，再到原理](#)

Java 获取反射的三种方法：

- 第一种，使用 Class.forName 静态方法。
- 第二种，使用类的 .class 方法
- 第三种，使用实例对象的 getClass() 方法。

6.15 面向对象的特征

面向对象的三大特征：

- 封装
- 继承

- 多态

6.16 &和&&的区别

- 按位与, `a&b` 表示把 `a` 和 `b` 都转换成二进制数, 再进行与的运算;
- `&`和`&&`都是逻辑运算符, `&&`又叫短路运算符
- 逻辑与, `a&& b`, `a&b` 都表示当且仅当两个操作数均为 `true` 时, 其结果才为 `true`, 否则为 `false`。
- 逻辑与跟短路与的差别是非常巨大的, 虽然二者都要求运算符左右两端的布尔值都是 `true`, 整个表达式的值才是 `true`。但是, `&&`之所以称为短路运算, 是因为如果 `&&`左边的表达式的值是 `false`, 右边的表达式会被直接短路掉, 不会进行运算。

6.17 Java 中 IO 流分为几种?

- Java 中的流分为两种: 一种是字节流, 另一种是字符流。
- IO 流分别由四个抽象类来表示 (两输入两输出): `InputStream`, `OutputStream`, `Reader`, `Writer`。

6.18 讲讲类的实例化顺序, 比如父类静态数据, 构造函数, 子类静态数据, 构造函数。

直接看个例子吧:

```
public class Parent {
    {
        System.out.println("父类非静态代码块");
    }
    static {
        System.out.println("父类静态块");
    }
    public Parent() {
        System.out.println("父类构造器");
    }
}
public class Son extends Parent {
    public Son() {
        System.out.println("子类构造器");
    }
}
```



```

    static {
        System.out.println("子类静态代码块");
    }
    {
        System.out.println("子类非静态代码块");
    }
}

public class Test {
    public static void main(String[] args) {
        Son son = new Son();
    }
}

```

运行结果：

```

父类静态块
子类静态代码块
父类非静态代码块
父类构造器
子类非静态代码块
子类构造器

```

所以，类实例化顺序为：父类静态代码块/静态域->子类静态代码块/静态域 -> 父类非静态代码块 -> 父类构造器 -> 子类非静态代码块 -> 子类构造器

6.19 Java 创建对象有几种方式

Java 创建对象有 5 种方式

- 用 new 语句创建对象。
- 使用反射，使用 Class.newInstance()创建对象/调用类对象的构造方法——Constructor
- 调用对象的 clone()方法。
- 运用反序列化手段，调用 java.io.ObjectInputStream 对象的 readObject()方法。
- 使用 Unsafe

6.20 如何将 GB2312 编码的字符串转换为 ISO-8859-1 编码的字符串呢？

```
public class Test {  
    public static void main(String[] args) throws UnsupportedOperationException {  
        String str = "捡田螺的小男孩";  
        String strIso = new String(str.getBytes("GB2312"), "ISO-8859-1");  
        System.out.println(strIso);  
    }  
}
```

6.21 守护线程是什么？用什么方法实现守护线程

- 守护线程是运行在后台的一种特殊进程。
- 它独立于控制终端并且周期性地执行某种任务或等待处理某些发生的事件。
- 在 Java 中垃圾回收线程就是特殊的守护线程。

6.22 notify()和 notifyAll()有什么区别？

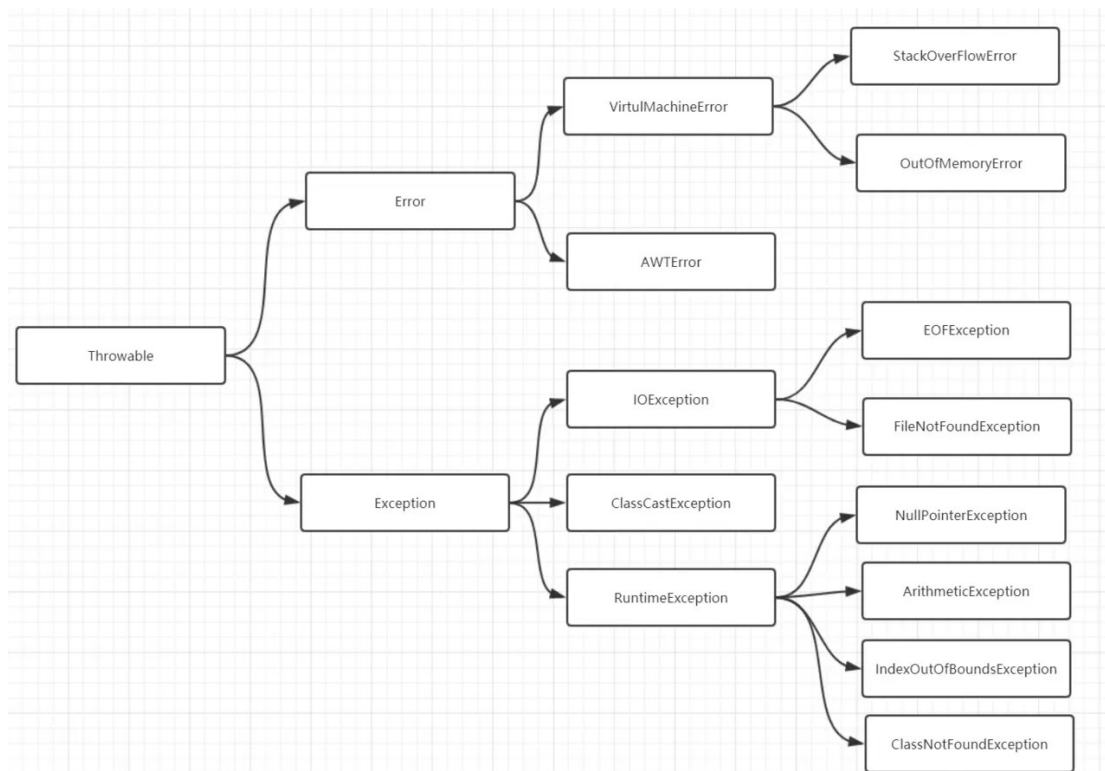
- notify 是唤醒一个处于该对象 wait 的线程，而 notifyAll 是唤醒所有处于该对象 wait 的线程。
- 但是唤醒不等于就能执行了，需要得到锁对象才能有权利继续执行，而锁只有一把，所以多个线程被唤醒时需要争取该锁。

6.23 Java 语言是如何处理异常的，关键字

throws、throw、try、catch、finally 怎么使用？

这道面试题，可以看我这篇文章哈：[Java 程序员必备：异常的十个关键知识点](#)

6.24 谈谈 Java 的异常层次结构



从前从前，有位老人，他的名字叫 **Throwable**，他生了两个儿子，大儿子叫 **Error**，二儿子叫 **Exception**。

Error

表示编译时或者系统错误，如虚拟机相关的错误，`OutOfMemoryError` 等，`error` 是无法处理的。

Exception

代码异常，Java 程序员关心的基类型通常是 `Exception`。它被程序本身可以处理，这也是它跟 `Error` 的区别。

它可以分为 `RuntimeException`（运行时异常）和 `CheckedException`（可检查的异常）。

常见的 **`RuntimeException`** 异常：

- `NullPointerException` 空指针异常
- `ArithmeticException` 出现异常的运算条件时，抛出此异常
- `IndexOutOfBoundsException` 数组索引越界异常

- `ClassNotFoundException` 找不到类异常
- `IllegalArgumentException` (非法参数异常)

常见的 **Checked Exception** 异常：

- `IOException` (操作输入流和输出流时可能出现的异常)
- `ClassCastException` (类型转换异常类)

- Checked Exception 就是编译器要求你必须处置的异常。
- 与之相反的是，Unchecked Exceptions，它指编译器不要求强制处置的异常，它包括 Error 和 RuntimeException 以及他们的子类。

6.25 静态内部类与非静态内部类有什么区别

这道面试题，可以看我这篇文章哈：[Java 程序员必备基础：内部类解析](#)

- 静态内部类可以有静态成员(方法，属性)，而非静态内部类则不能有静态成员(方法，属性)。
- 静态内部类只能够访问外部类的静态成员和静态方法,而非静态内部类则可以访问外部类的所有成员(方法，属性)。
- 实例化静态内部类与非静态内部类的方式不同
- 调用内部静态类的方法或静态变量,可以通过类名直接调用

6.26 String s 与 new String 与有什么区别

```
String str = "whx";  
String newStr = new String ("whx");
```

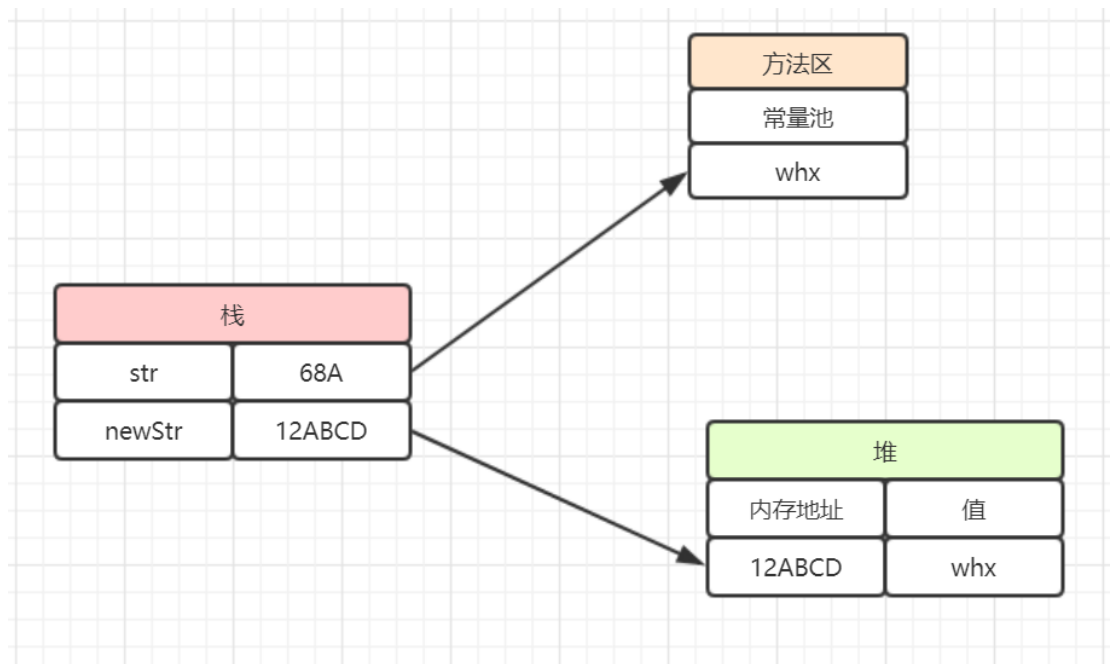
String str = "whx"

先在常量池中查找有没有"whx" 这个对象,如果有，就让 str 指向那个"whx".如果没有，在常量池中新建一个“whx”对象，并让 str 指向在常量池中新建的对象"whx"。

String newStr = new String ("whx");

是在堆中建立的对象"whx" ,在栈中创建堆中"whx" 对象的内存地址。

如图所示：



网上这篇文章讲的挺好的：[String 和 New String\(\)的区别](#)

6.27 反射中，Class.forName 和 ClassLoader 的区别

Class.forName 和 ClassLoader 都可以对类进行加载。它们区别在哪里呢？

ClassLoader 负责加载 Java 类的字节代码到 Java 虚拟机中。

Class.forName 其实是调用了 ClassLoader，如下：

```
@CallerSensitive
public static Class<?> forName(String className)
    throws ClassNotFoundException {
    Class<?> caller = Reflection.getCallerClass();
    return forName0(className, initialize: true, ClassLoader.getClassLoader(caller), caller);
}
```

这里面，forName0 的第二个参数为 true，表示对加载的类进行初始化。其

实还可以调用 `Class<?> forName(String name, boolean initialize,`

`ClassLoader loader)` 方法实现一样的功能，它的源码如下：

```

* @param name fully qualified name of the desired class
* @param initialize if {@code true} the class will be initialized.
* See Section 12.4 of <em>The Java Language Specification</em>.
* @param loader class loader from which the class must be loaded
* @return class object representing the desired class
*
* @exception LinkageError if the linkage fails
* @exception ExceptionInInitializerError if the initialization provoked
* by this method fails
* @exception ClassNotFoundException if the class cannot be located by
* the specified class loader
*
* @see java.lang.Class#forName(String)
* @see java.lang.ClassLoader
* @since 1.2
*/
@CallerSensitive
public static Class<?> forName(String name, boolean initialize,
                               ClassLoader loader)
    throws ClassNotFoundException
{

```

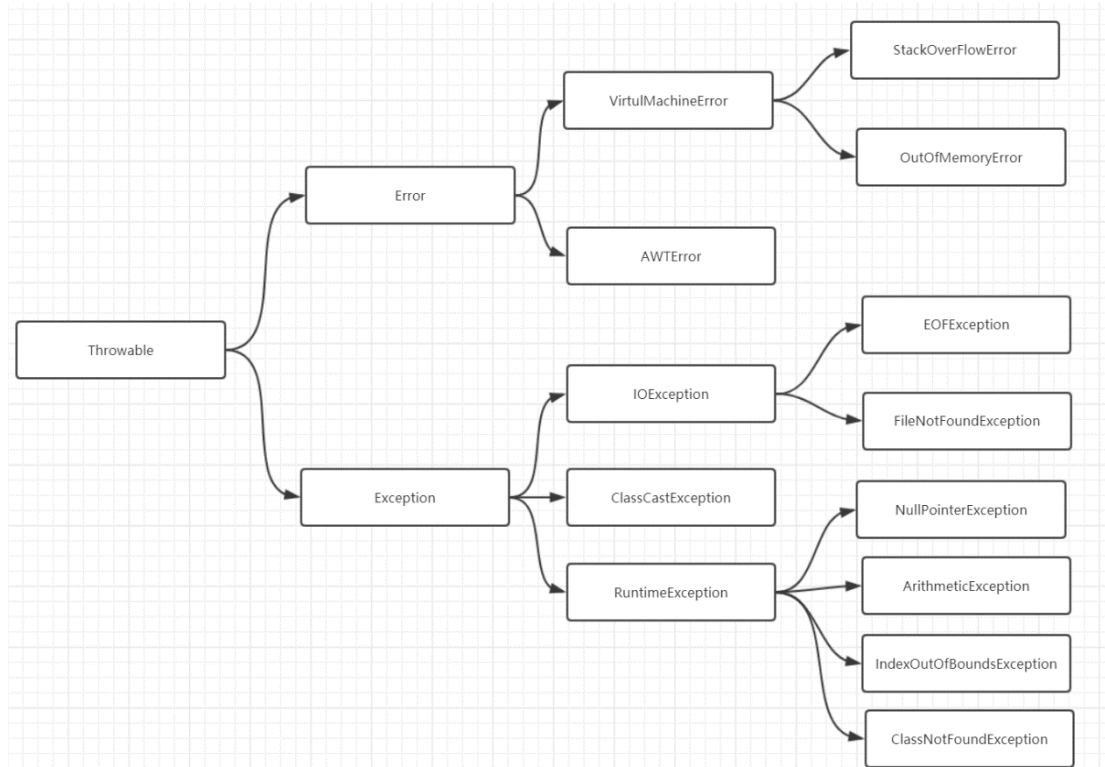
所以，Class.forName 和 ClassLoader 的区别，就是在类加载的时候，class.forName 有参数控制是否对类进行初始化。

6.28 JDK 动态代理与 cglib 实现的区别

- java 动态代理是利用反射机制生成一个实现代理接口的匿名类，在调用具体方法前调用 InvokeHandler 来处理。
- cglib 动态代理是利用 asm 开源包，对代理对象类的 class 文件加载进来，通过修改其字节码生成子类来处理。
- JDK 动态代理只能对实现了接口的类生成代理，而不能针对类
- cglib 是针对类实现代理，主要是对指定的类生成一个子类，覆盖其中的方法。因为是继承，所以该类或方法最好不要声明成 final

网上这篇文章写得不错，[描述 Java 动态代理的几种实现方式，分别说出相应的优缺点](#)

6.29 error 和 exception 的区别， CheckedException，RuntimeException 的区别。



Error: 表示编译时或者系统错误，如虚拟机相关的错误，
OutOfMemoryError 等，error 是无法处理的。

Exception: 代码异常，Java 程序员关心的基类型通常是 Exception。它
能被程序本身可以处理，这也是它跟 Error 的区别。

它可以分为 RuntimeException (运行时异常) 和 CheckedException (可
检查的异常)。常见的 RuntimeException 异常：

- NullPointerException 空指针异常
- ArithmeticException 出现异常的运算条件时，抛出此异常
- IndexOutOfBoundsException 数组索引越界异常
- ClassNotFoundException 找不到类异常
- IllegalArgumentException(非法参数异常)

常见的 Checked Exception 异常：

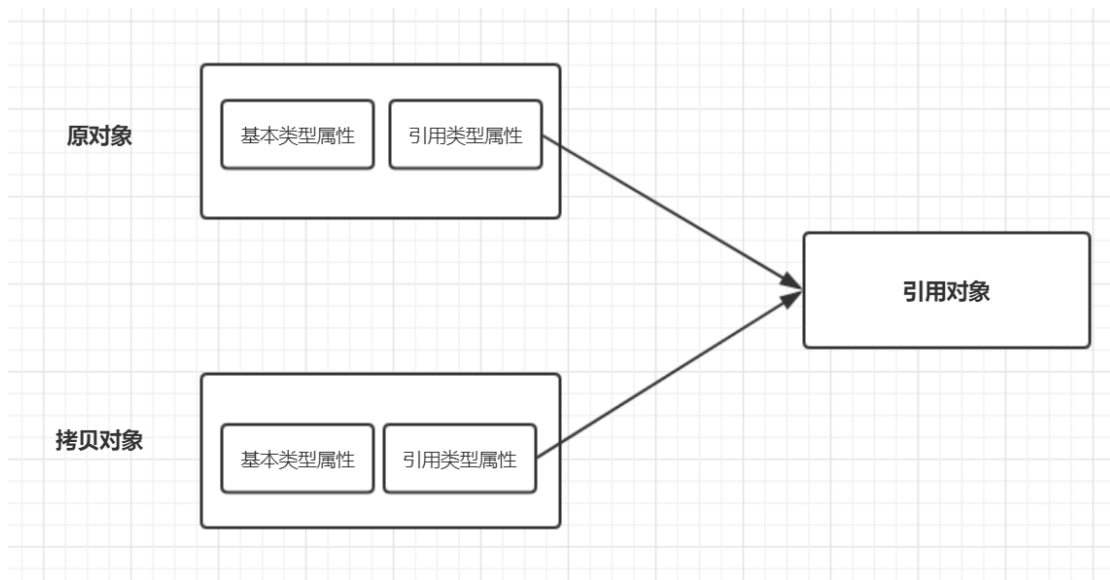
- IOException (操作输入流和输出流时可能出现的异常)
- ClassCastException(类型转换异常类)

有兴趣可以看我之前写得这篇文章：[Java 程序员必备：异常的十个关键知识点](#)

6.30 深拷贝和浅拷贝区别

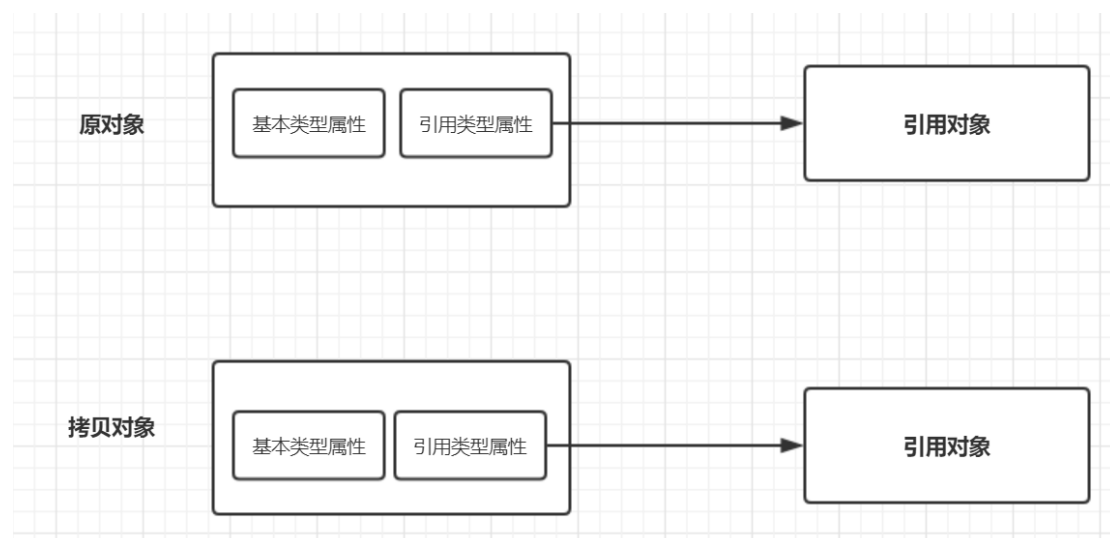
浅拷贝

复制了对象的引用地址，两个对象指向同一个内存地址，所以修改其中任意的值，另一个值都会随之变化。



深拷贝

将对象及值复制过来，两个对象修改其中任意的值另一个值不会改变



6.31 JDK 和 JRE 有什么区别？

- JDK : Java Development Kit 的简称，Java 开发工具包，提供了 Java 的开发环境和运行环境。
- JRE : Java Runtime Environment 的简称，Java 运行环境，为 Java 的运行提供了所需环境。

6.32 String 类的常用方法都有那些呢？

- `indexOf()` : 返回指定字符的索引。
- `charAt()` : 返回指定索引处的字符。
- `replace()` : 字符串替换。
- `trim()` : 去除字符串两端空白。
- `split()` : 分割字符串，返回一个分割后的字符串数组。
- `getBytes()` : 返回字符串的 `byte` 类型数组。
- `length()` : 返回字符串长度。
- `toLowerCase()` : 将字符串转成小写字母。
- `toUpperCase()` : 将字符串转成大写字符。
- `substring()` : 截取字符串。
- `equals()` : 字符串比较。

6.33 谈谈自定义注解的场景及实现

- 之前我这边有这么一个业务场景，用 Redis 控制接口调用频率，有使用过自定义注解。

- 通过 AOP (动态代理机制) 给方法添加切面，通过反射来获取方法包含的注解，如果包含自定义关键字注解，就通过 Redis 进行校验拦截请求。

有关于注解，建议大家看一下 java 编程思想的注解篇章哈～

6.34 说说你熟悉的设计模式有哪些？

设计模式分为三大类：

- 创建型模式：工厂方法模式、抽象工厂模式、单例模式、建造者模式、原型模式 (5 种)
- 结构型模式：适配器模式、装饰者模式、代理模式、外观模式、桥接模式、组合模式、享元模式。(7 种)
- 行为型模式：策略模式、模板方法模式、观察者模式、迭代子模式、责任链模式、命令模式、备忘录模式、状态模式、访问者模式、中介者模式、解释器模式。(11 种)

最好平时积累一下，单例模式 (7 种实现方式)，工厂模式，模板方法设计模式，策略模式，装饰者模式、代理模式这几种怎么写吧～

大家可以看下我的这篇文章哈，[实战！工作中常用到哪些设计模式](#)

6.35 抽象工厂和工厂方法模式的区别？

工厂方法模式	抽象工厂模式
针对的是一个产品等级结构	针对的是面向多个产品等级结构
一个抽象产品类	多个抽象产品类
可以派生出多个具体产品类	每个抽象产品类可以派生出多个具体产品类
一个抽象工厂类，可以派生出多个具体工厂类	一个抽象工厂类，可以派生出多个具体工厂类
每个具体工厂类只能创建一个具体产品类的实例	每个具体工厂类可以创建多个具体产品类的实例

6.36 什么是值传递和引用传递？

- 值传递是对基本型变量而言的,传递的是该变量的一个副本，改变副本不影响原变量。

- 引用传递一般是对于对象型变量而言的,传递的是该对象地址的一个副本,并不是原对象本身。所以对引用对象进行操作会同时改变原对象。

6.37 可以在 static 环境中访问非 static 变量吗？

static 变量在 Java 中是属于类的，它在所有的实例中的值是一样的。当类被 Java 虚拟机载入的时候，会对 static 变量进行初始化。因为静态的成员属于类，随着类的加载而加载到静态方法区内存，当类加载时，此时不一定有实例创建，没有实例，就不可以访问非静态的成员。类的加载先于实例的创建，因此静态环境中，不可以访问非静态！

6.38 Java 支持多继承么,为什么？

不支持多继承，原因：

- 安全性的考虑，如果子类继承的多个父类里面有相同的方法或者属性，子类将不知道具体要继承哪个。
- Java 提供了接口和内部类以达到实现多继承功能，弥补单继承的缺陷。

6.39 用最有效率的方法计算 2 乘以 8？

2 << 3

- 将一个数左移 n 位，就相当于这个数乘以了 2 的 n 次方。
- 那么，一个数乘以 8 只要将其左移 3 位即可。
- 而 cpu 直接支持位移运算，且效率最高。

6.40 构造器是否可被重写？

构造器是不能被继承的，因为每个类的类名都不相同，而构造器名称与类名相同，所以谈不上继承。又由于构造器不能被继承，所以相应的就不能被重写了。

6.41 char 型变量中能不能存贮一个中文汉字，为什么？

在 Java 中，char 类型占 2 个字节，而且 Java 默认采用 Unicode 编码，一个 Unicode 码是 16 位，所以一个 Unicode 码占两个字节，Java 中无论汉字还是英文字母都是用 Unicode 编码来表示的。所以，在 Java 中，char 类型变量可以存储一个中文汉字。

```
char ch = '啦';  
System.out.println("char:" + ch);
```

6.42 如何实现对象克隆？

- 实现 Cloneable 接口，重写 clone() 方法。
- Object 的 clone() 方法是浅拷贝，即如果类中属性有自定义引用类型，只拷贝引用，不拷贝引用指向的对象。
- 对象的属性的 Class 也实现 Cloneable 接口，在克隆对象时也手动克隆属性，完成深拷贝
- 结合序列化(JDK java.io.Serializable 接口、JSON 格式、XML 格式等)，完成深拷贝

6.43 object 中定义了哪些方法？

- getClass(); 获取类结构信息
- hashCode() 获取哈希码
- equals(Object) 默认比较对象的地址值是否相等，子类可以重写比较规则
- clone() 用于对象克隆
- toString() 把对象转变成字符串
- notify() 多线程中唤醒功能
- notifyAll() 多线程中唤醒所有等待线程的功能
- wait() 让持有对象锁的线程进入等待
- wait(long timeout) 让持有对象锁的线程进入等待，设置超时毫秒数时间
- wait(long timeout, int nanos) 让持有对象锁的线程进入等待，设置超时纳秒数时间
- finalize() 垃圾回收前执行的方法

6.44 hashCode 的作用是什么？

- hashCode 的存在主要是用于查找的快捷性，如 Hashtable，HashMap 等，hashCode 是用来在散列存储结构中确定对象的存储地址的；
- 如果两个对象相同，就是适用于 equals(java.lang.Object) 方法，那么这两个对象的 hashCode 一定要相同；
- 如果对象的 equals 方法被重写，那么对象的 hashCode 也尽量重写，并且产生 hashCode 使用的对象，一定要和 equals 方法中使用的一致，否则就会违反上面提到的第 2 点；
- 两个对象的 hashCode 相同，并不一定表示两个对象就相同，也就是不一定适用于 equals(java.lang.Object) 方法，只能够说明这两个对象在散列存储结构中。

6.45 for-each 与常规 for 循环的效率对比

关于这个问题，《Effective Java》给我们做的解答：

for-each 能够让代码更加清晰，并且减少了出错的机会。下面的惯用代码适用于集合与数组类型：

```
for (Element e : elements) {  
    doSomething(e);  
}
```

使用 for-each 循环与常规的 for 循环相比，并不存在性能损失，即使对数组进行迭代也是如此。实际上，在有些场合下它还能带来微小的性能提升，因为它只计算一次数组索引的上限。

6.46 写出几种单例模式实现，懒汉模式和饿汉模式区别

7 种：

- 第一种（懒汉，线程不安全）
- 第二种（懒汉，线程安全）
- 第三种（饿汉）
- 第四种（饿汉，变种）

- 第五种（静态内部类）
- 第六种（枚举）：
- 第七种（双重校验锁）

可以看这篇文章：[单例模式的七种写法](#)

6.47 请列出 5 个运行时异常。

```
- NullPointerException 空指针异常
- ArithmeticException 出现异常的运算条件时，抛出此异常
- IndexOutOfBoundsException 数组索引越界异常
- ClassNotFoundException 找不到类异常
- IllegalArgumentException(非法参数异常)
```

6.48 2 个不相等的对象有可能具有相同的 hashCode 吗？

有可能哈～

hashCode 的常规协定：

- 在 Java 应用程序执行期间，在对同一对象多次调用 hashCode 方法时，必须一致地返回相同的整数，前提是将对象进行 equals 比较时所用的信息没有被修改。从某一应用程序的一次执行到同一应用程序的另一次执行，该整数无需保持一致。
- 两个对象的 equals() 相等，那么对这两个对象中的每个对象调用 hashCode 方法都必须生成相同的整数结果。
- 两个对象的 equals() 不相等，那么对这两个对象中的任一对象上调用 hashCode 方法不要求一定生成不同的整数结果。但是，为不相等的对象生成不同整数结果可以提高哈希表的性能。

6.49 访问修饰符 public,private,protected,以及 default 的区别？

修饰符	当前类	同包	子类	其他包
public	√	√	√	√
protected	√	√	√	×
default	√	√	×	×
private	√	×	×	×

6.50 谈谈 final 在 java 中的作用？

- final 修饰的类叫最终类，该类不能被继承。
- final 修饰的方法不能被重写。
- final 修饰的变量叫常量，常量必须初始化，初始化之后值就不能被修改。

6.51 java 中的 Math.round(-1.5) 等于多少呢？

```

3  ▶ public class Test {
4  ▶     public static void main(String[] args) {
5      System.out.println("round" + Math.round(-1.5));
6      System.out.println("ceil" + Math.ceil(-1.5));
7      System.out.println("floor" + Math.floor(-1.5));
8  }
Test ▶ main()
st
"C:\Program Files\Java\jdk1.8.0_161\bin\java" ...
round-1
ceil-1.0
floor-2.0

```

JDK 中的 java.lang.Math 类:

- round()：返回四舍五入，负 .5 小数返回较大整数，如 -1.5 返回 -1。
- ceil()：返回小数所在两整数间的较大值，如 -1.5 返回 -1.0。
- floor()：返回小数所在两整数间的较小值，如 -1.5 返回 -2.0。

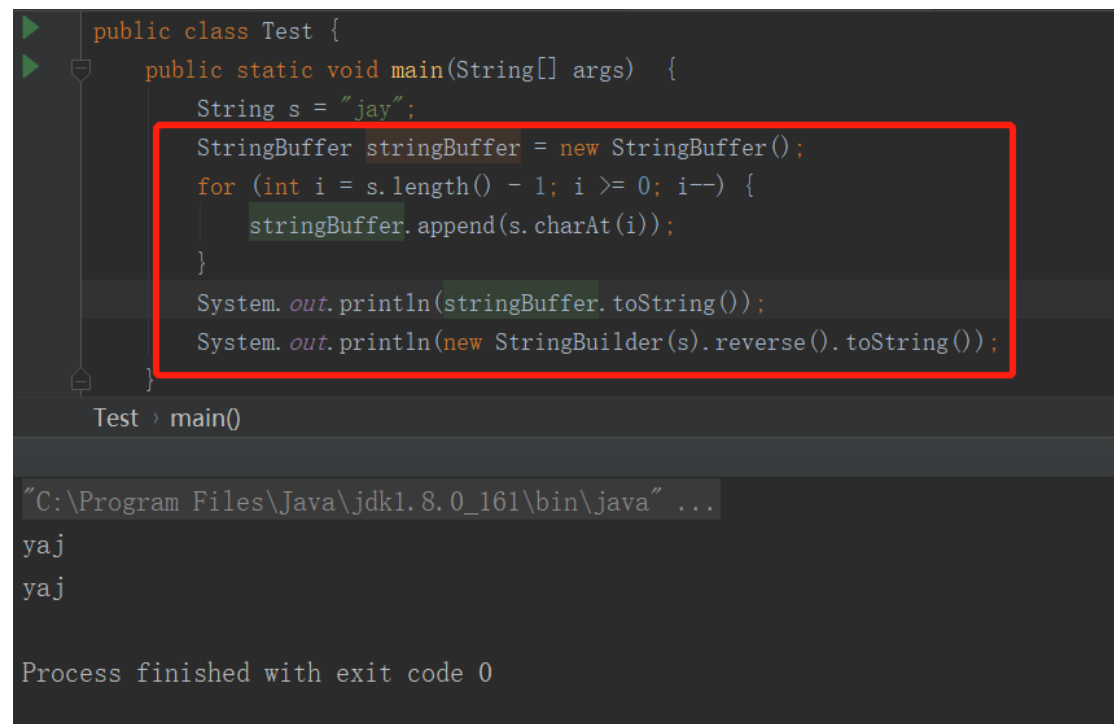
6.52 String 属于基础的数据类型吗？

String 不属于基础类型，基础类型有 8 种：

byte、boolean、char、short、int、float、long、double，而 String 属于对象。

6.53 如何将字符串反转呢？

- 使用 StringBuilder 或 StringBuffer 的 reverse 方法，本质都调用了它们的父类 AbstractStringBuilder 的 reverse 方法实现。（JDK1.8）
- 使用 charAt 函数，倒过来输出；



```
public class Test {  
    public static void main(String[] args) {  
        String s = "jay";  
        StringBuffer stringBuffer = new StringBuffer();  
        for (int i = s.length() - 1; i >= 0; i--) {  
            stringBuffer.append(s.charAt(i));  
        }  
        System.out.println(stringBuffer.toString());  
        System.out.println(new StringBuilder(s).reverse().toString());  
    }  
}
```

Test > main()

"C:\Program Files\Java\jdk1.8.0_161\bin\java" ...
yaj
yaj
Process finished with exit code 0

6.54 描述动态代理的几种实现方式，它们分别有什么优缺点

- JDK 动态代理
- CGLIB 动态代理
- JDK 原声动态代理时 java 原声支持的、不需要任何外部依赖、但是它只能基于接口进行代理
- CGLIB 通过继承的方式进行代理、无论目标对象有没有实现接口都可以代理，但是无法处理 final 的情况

6.55 在自己的代码中，如果创建一个 `java.lang.String` 类，这个类是否可以被类加载器加载？为什么。

不可以。因为 JDK 处于安全性的考虑，基于双亲委派模型，优先加载 JDK 的 `String` 类，如果 `java.lang.String` 已经加载，便不会再次被加载。

6.56 谈谈你对 `java.lang.Object` 对象中 `hashCode` 和 `equals` 方法的理解。在什么场景下需要重新实现这两个方法。

- 在我们的业务系统中判断对象时有时候需要的不是一种严格意义上的相等，而是一种业务上的对象相等。在这种情况下，原生的 `equals` 方法就不能满足我们的需求了，所以这个时候我们需要重写 `equals` 方法，来满足我们的业务系统上的需求。
- 那么为什么在重写 `equals` 方法的时候需要重写 `hashCode` 方法呢？如果只重写了 `equals` 方法而没有重写 `hashCode` 方法的话，则会违反约定的第二条：相等的对象必须具有相等的散列码。所以 `hashCode` 和 `equals` 方法都需要重写

可以看网上这篇文章哈：[java 为什么要重写 `hashCode` 和 `equals` 方法](#)

6.57 在 `jdk1.5` 中，引入了泛型，泛型的存在是用来解决什么问题。

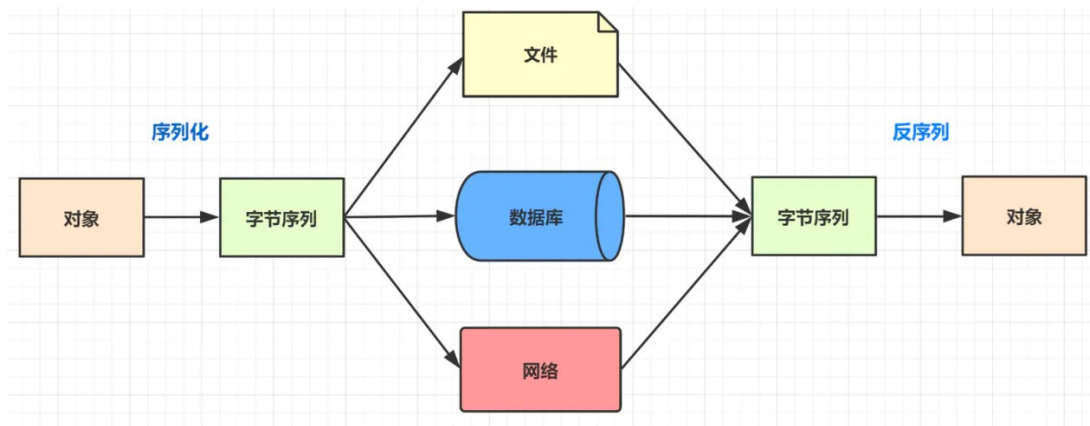
Java 泛型 (generics) 是 JDK 5 中引入的一个新特性，其本质是参数化类型，解决不确定具体对象类型的问题。

这个面试题，可以看我这篇文章哈~[Java 程序员必备基础：泛型解析](#)

6.58 什么是序列化，怎么序列化，反序列呢？

- 序列化：把 Java 对象转换为字节序列的过程

- 反序列：把字节序列恢复为 Java 对象的过程



可以看我这篇文章哈~ [Java 程序员必备：序列化全方位解析](#)

6.59 java8 的新特性。

- Lambda 表达式：Lambda 允许把函数作为一个方法的参数
- Stream API：新添加的 Stream API (java.util.stream) 把真正的函数式编程风格引入到 Java 中
- 方法引用：方法引用提供了非常有用的语法，可以直接引用已有 Java 类或对象（实例）的方法或构造器。
- 默认方法：默认方法就是一个在接口里面有了一个实现的方法。
- Optional 类：Optional 类已经成为 Java 8 类库的一部分，用来解决空指针异常。
- Date Time API：加强对日期与时间的处理。

6.60 匿名内部类是什么？如何访问在其外面定义的变量呢？

匿名内部类就是没有名字的内部类，日常开发中使用的比较多。

```

public class Outer {

    private void test(final int i) {
        new Service() {
            public void method() {
                for (int j = 0; j < i; j++) {
                    System.out.println("匿名内部类");
                }
            }
        }.method();
    }
}
  
```

```

    }
}
//匿名内部类必须继承或实现一个已有的接口
interface Service{
    void method();
}

```

匿名内部类还有以下特点：

- 没有名字
- 匿名内部类必须继承一个抽象类或者实现一个接口。
- 匿名内部类不能定义任何静态成员和静态方法。
- 当所在的方法的形参需要被匿名内部类使用时，必须声明为 final。
- 匿名内部类不能是抽象的，它必须要实现继承的类或者实现的接口的所有抽象方法。
- 匿名内部类不能访问外部类方法中的局部变量，除非该变量被声明为 final 类型

可以看我这篇文章哈~[Java 程序员必备基础：内部类解析](#)

6.61 break 和 continue 有什么区别？

- break 可以使流程跳出 switch 语句体，也可以在循环结构终止本层循环体，从而提前结束本层循环。
- continue 的作用是跳过本次循环体中余下尚未执行的语句，立即进行下一次的循环条件判定，可以理解为仅结束本次循环

6.62 String s = "Hello";s = s + " world!";这两行代码执行后，原始的 String 对象中的内容是否会改变？

没有。因为 String 被设计成不可变(immutable)类，所以它的所有对象都是不可变对象。

6.63 String s="a"+"b"+"c"+"d";创建了几个对象？

1 个而已啦。

Java 编译器对字符串常量直接相加的表达式进行优化，不等到运行期去进行加法运算，在编译时就去掉了加号，直接将其编译成一个这些常量相连的结果。所以 "a"+"b"+"c"+"d" 相当于直接定义一个 "abcd" 的字符串。

6.64 try-catch-finally-return 执行顺序

try-catch-finally-return 执行描述:

- 如果不发生异常，不会执行 catch 部分。
- 不管有没有发生异常，finally 都会执行到。
- 即使 try 和 catch 中有 return 时，finally 仍然会执行
- finally 是在 return 后面的表达式运算完后再执行的。（此时并没有返回运算后的值，而是先把要返回的值保存起来，若 finally 中无 return，则不管 finally 中的代码怎么样，返回的值都不会改变，仍然是之前保存的值），该情况下函数返回值是在 finally 执行前确定的)
- finally 部分就不要 return 了，要不然，就回不去 try 或者 catch 的 return 了。

看一个例子

```
public static void main(String[] args) throws IOException {  
    System.out.println("result : " + test());  
}  
  
private static int test() {  
    int temp = 1;  
    try {  
        System.out.println("start execute try,temp is:"+temp);  
        return ++temp;  
    } catch (Exception e) {  
        System.out.println("start execute catch temp is: "+temp);  
        return ++temp;  
    } finally {  
        System.out.println("start execute finally,temp is:" + temp);  
        ++temp;  
    }  
}
```

运行结果：

```
start execute try,temp is:1
start execute finally,temp is:2
result:2
```

分析：

- 先执行 try 部分，输出日志，执行`++temp` 表达式，temp 变为 2,这个值被保存起来。
- 因为没有发生异常，所以 catch 代码块跳过。
- 执行 finally 代码块，输出日志，执行`++temp` 表达式。
- 返回 try 部分保存的值 2。

6.65 Java 7 新的 try-with-resources 语句，平时有使用吗

try-with-resources，是Java7 提供的一个新功能，它用于自动资源管理。

- 资源是指在程序用完了之后必须要关闭的对象。
- try-with-resources 保证了每个声明了的资源在语句结束的时候会被关闭
- 什么样的对象才能当做资源使用呢？只要实现了 java.lang.AutoCloseable 接口或者 java.io.Closeable 接口的对象，都 OK。

在 `try-with-resources` 出现之前

```
try{
    //open resources like File, Database connection, Sockets etc
} catch (FileNotFoundException e) {
    // Exception handling like FileNotFoundException, IOException etc
}finally{
    // close resources
}
```

Java7，`try-with-resources` 出现之后，使用资源实现

```
try{// open resources here){
    // use resources
} catch (FileNotFoundException e) {
    // exception handling
}
// resources are closed as soon as try-catch block is executed.
```

Java7 使用资源 demo

```
public class Java7TryResourceTest {  
    public static void main(String[] args) {  
        try (BufferedReader br = new BufferedReader(new FileReader(  
            "C:/jaywei.txt"))) {  
            System.out.println(br.readLine());  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

使用了 `try-with-resources` 的好处

- 代码更加优雅，行数更少。
- 资源自动管理，不用担心内存泄漏问题。

6.66 简述一下面向对象的“六原则一法则”。

- 单一职责原则:一个类只做它该做的事情。
- 开闭原则：软件实体应当对扩展开放，对修改关闭。
- 依赖倒转原则：面向接口编程。
- 接口隔离原则：接口要小而专，绝不能大而全。
- 合成聚合复用原则：优先使用聚合或合成关系复用代码。
- 迪米特法则：迪米特法则又叫最少知识原则，一个对象应当对其他对象有尽可能少的了解。

6.67 switch 是否能作用在 byte 上，是否能作用在 long 上，是否能作用在 String 上？

- switch 可作用于 char byte short int
- switch 可作用于 char byte short int 对应的包装类
- switch 不可作用于 long double float boolean，以及他们的包装类

6.68 数组有没有 length()方法？String 有没有 length()方法？

- 数组没有 length() 方法，而是 length；
- String 有 length() 方法

6.69 是否可以从一个静态 (static) 方法内部发出对非静态 (non-static) 方法的调用？

不可以。

- 非 static 方法是要与对象实例息息相关的，必须在创建一个对象后，才可以在该对象上进行非 static 方法调用，而 static 方法跟类相关的，不需要创建对象，可以由类直接调用。
- 当一个 static 方法被调用时，可能还没有创建任何实例对象，如果从一个 static 方法中发出对非 static 方法的调用，那个非 static 方法是关联到哪个对象上的呢？这个逻辑是不成立的
- 因此，一个 static 方法内部不可以发出对非 static 方法的调用。

6.70 String s = new String("jay"); 创建了几个字符串对象？

一个或两个

- 第一次调用 new String("jay"); 时，会在堆内存中创建一个字符串对象，同时在字符串常量池中创建一个对象 "jay"
- 第二次调用 new String("jay"); 时，只会在堆内存中创建一个字符串对象，指向之前在字符串常量池中创建的 "jay"

6.71 this 和 super 关键字的作用

this :

- 对象内部指代自身的引用
- 解决成员变量和局部变量同名问题
- 可以调用成员变量，不能调用局部变量
- 可以调用成员方法
- 在普通方法中可以省略 this

- 在静态方法当中不允许出现 this 关键字

super :

- 调用父类 的成员或者方法
- 调用父类的构造函数

6.72 我们能够将 int 强制转换为 byte 类型的变量吗？如果该值大于 byte 类型的范围，将会出现什么现象？

可以，我们可以做强制转换，但是在Java中，int是32位，byte是8位，如果强制做转化，int类型的高24位将会被丢弃。

```
public class Test {  
    public static void main(String[] args) {  
        int a =129;  
        byte b = (byte) a;  
        System.out.println(b);  
        int c =10;  
        byte d = (byte) c;  
        System.out.println(d);  
    }  
}
```

输出：
-127
10

6.73 float f=3.4;正确吗？

不正确，精度不准确,应该用强制类型转换

```
public static void main(String[] args) {  
    float f=3.4;  
}
```


6.74 接口可否继承接口？抽象类是否可实现接口？抽象类是否可继承实体类？

都可以的

6.75 Reader 和 InputStream 区别？

- InputStream 是表示字节输入流的所有类的超类
- Reader 是用于读取字符流的抽象类

6.76 列举出 JAVA 中 6 个比较常用的包

- java.lang;
- java.util;
- java.io;
- java.sql;
- java.awt;
- java.net;

6.77 JDK 7 有哪些新特性

- Try-with-resource 语句
- NIO2 文件处理 Files
- switch 可以支持字符串判断条件
- 泛型推导
- 多异常统一处理

可以看下我这篇文章哈，[Java 程序员必备基础：JDK 5-15 都有哪些经典新特性](#)

6.78 同步和异步有什么区别？

- 同步，可以理解为在执行完一个函数或方法之后，一直等待系统返回值或消息，这时程序是出于阻塞的，只有接收到返回的值或消息后才往下执行其他的命令。

- 异步，执行完函数或方法后，不必阻塞性地等待返回值或消息，只需要向系统委托一个异步过程，那么当系统接收到返回值或消息时，系统会自动触发委托的异步过程，从而完成一个完整的流程。
- 同步，就是实时处理（如打电话）
- 异步，就是分时处理（如收发短信）

6.79 synchronized 的实现原理以及锁优化？

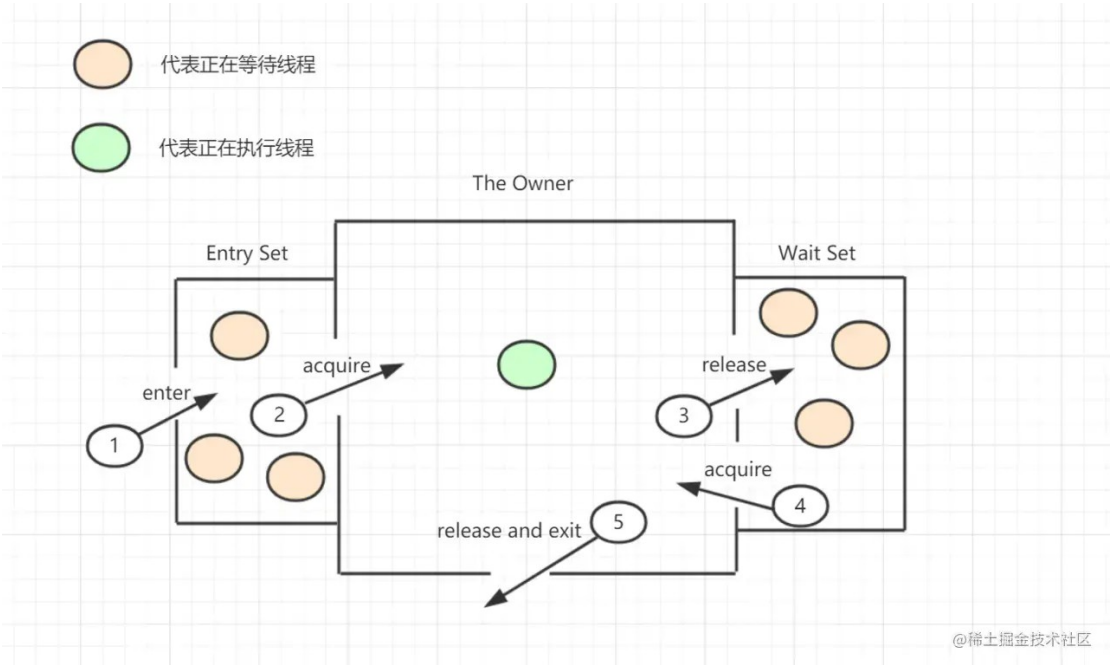
synchronized 的实现原理

- synchronized 作用于「方法」或者「代码块」，保证被修饰的代码在同一时间只能被一个线程访问。
- synchronized 修饰代码块时，JVM 采用「monitorenter、monitorexit」两个指令来实现同步
- synchronized 修饰同步方法时，JVM 采用「ACC_SYNCHRONIZED」标记符来实现同步
- monitorenter、monitorexit 或者 ACC_SYNCHRONIZED 都是「基于 Monitor 实现」的
- 实例对象里有对象头，对象头里面有 Mark Word，Mark Word 指针指向了「monitor」
- Monitor 其实是一种「同步工具」，也可以说是一种「同步机制」。
- 在 Java 虚拟机（HotSpot）中，Monitor 是由「ObjectMonitor 实现」的。ObjectMonitor 体现出 Monitor 的工作原理～

```
ObjectMonitor() {
    _header      = NULL;
    _count       = 0; // 记录线程获取锁的次数
    _waiters     = 0;
    _recursions  = 0; // 锁的重入次数
    _object      = NULL;
    _owner       = NULL; // 指向持有 ObjectMonitor 对象的线程
    _WaitSet     = NULL; // 处于 wait 状态的线程，会被加入到 _WaitSet
    _WaitSetLock = 0;
    _Responsible = NULL;
    _succ        = NULL;
    _cxq         = NULL;
    FreeNext     = NULL;
    _EntryList   = NULL; // 处于等待锁 block 状态的线程，会被加入到该列表
    _SpinFreq    = 0;
    _SpinClock   = 0;
    OwnerIsThread = 0;
}
```

ObjectMonitor 的几个关键属性

_count、_recursions、_owner、_WaitSet、_EntryList 体现了 monitor 的工作原理



锁优化

在讨论锁优化前，先看看 JAVA 对象头(32 位 JVM)中 Mark Word 的结构图吧

~

锁状态	25 bit		4 bit	1 bit	2 bit
	23 bit	2 bit		是否偏向锁	锁标志位
无锁	对象的hashCode		分代年龄	0	01
偏向锁	线程ID	Epoch	分代年龄	1	01
轻量级锁	指向栈中锁记录的指针				00
重量级锁	指向重量级锁的指针				10
GC标记	空				11

@稀土掘金技术社区

Mark Word 存储对象自身的运行数据，如「哈希码、GC 分代年龄、锁状态标志、偏向时间戳 (Epoch)」等，为什么区分「偏向锁、轻量级锁、重量级锁」等几种锁状态呢？

在JDK1.6之前，synchronized的实现直接调用ObjectMonitor的enter和exit，这种锁被称之为「重量级锁」。从JDK6开始，HotSpot虚拟机开发团队对Java中的锁进行优化，如增加了适应性自旋、锁消除、锁粗化、轻量级锁和偏向锁等优化策略。

- 偏向锁：在无竞争的情况下，把整个同步都消除掉，CAS操作都不做。
- 轻量级锁：在没有多线程竞争时，相对重量级锁，减少操作系统互斥量带来的性能消耗。但是，如果存在锁竞争，除了互斥量本身开销，还额外有CAS操作的开销。
- 自旋锁：减少不必要的CPU上下文切换。在轻量级锁升级为重量级锁时，就使用了自旋加锁的方式
- 锁粗化：将多个连续的加锁、解锁操作连接在一起，扩展成一个范围更大的锁。

举个例子，买门票进动物园。老师带一群小朋友去参观，验票员如果知道他们是个集体，就可以把他们看成一个整体（锁粗化），一次性验票过，而不需要一个个找他们验票。

- 锁消除:虚拟机即时编译器在运行时，对一些代码上要求同步，但是被检测到不可能存在共享数据竞争的锁进行消除。

有兴趣的小伙伴可以看下我这篇文章哈：[Synchronized 解析——如果你愿意一层一层剥开我的心](#)

6.80 ThreadLocal 原理，使用注意点，应用场景有哪些？

回答四个主要点：

- ThreadLocal 是什么？

- ThreadLocal 原理
- ThreadLocal 使用注意点
- ThreadLocal 的应用场景

ThreadLocal 是什么?

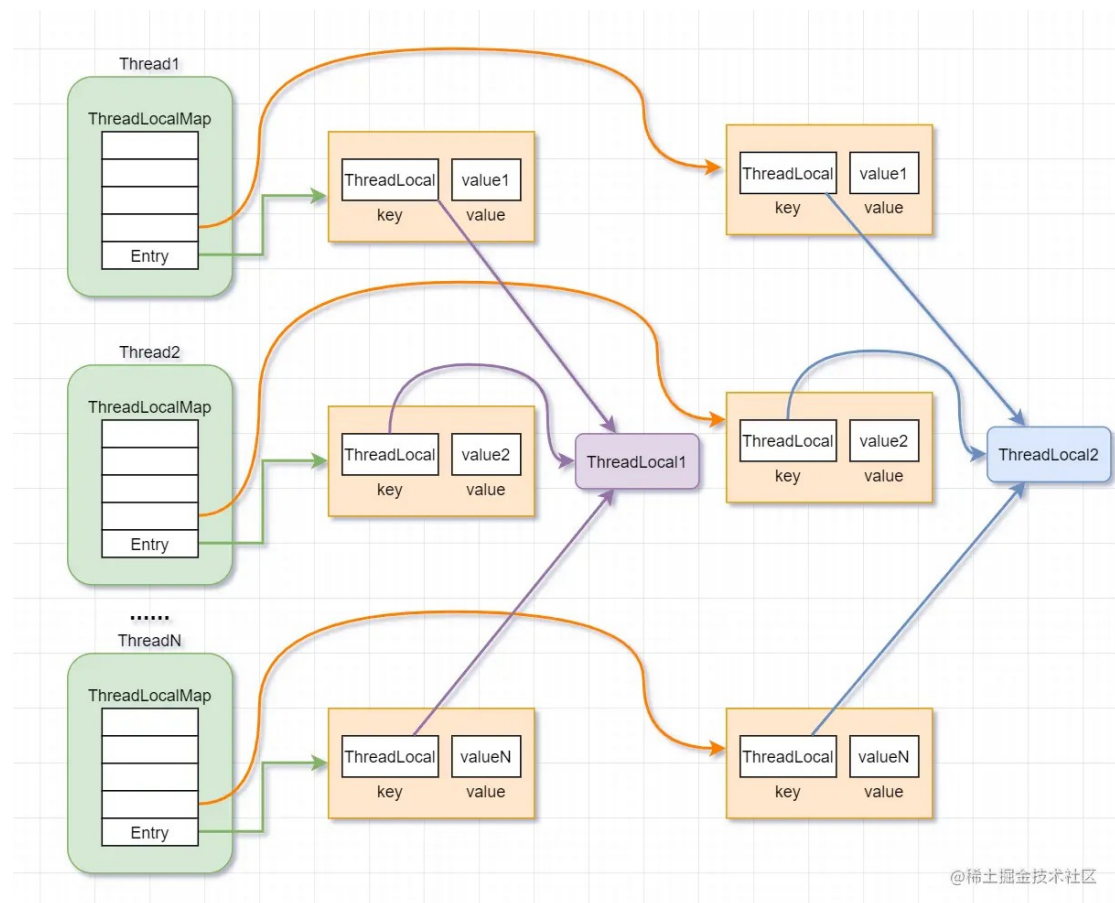
ThreadLocal，即线程本地变量。如果你创建了一个 ThreadLocal 变量，那么访问这个变量的每个线程都会有这个变量的一个本地拷贝，多个线程操作这个变量的时候，实际是操作自己本地内存里面的变量，从而起到线程隔离的作用，避免了线程安全问题。

//创建一个 ThreadLocal 变量

```
static ThreadLocal<String> localVariable = new ThreadLocal<>();
```

ThreadLocal 原理

ThreadLocal 内存结构图：



由结构图是可以看出：

- Thread 对象中持有一个 ThreadLocal.ThreadLocalMap 的成员变量。
- ThreadLocalMap 内部维护了 Entry 数组，每个 Entry 代表一个完整的对象，key 是 ThreadLocal 本身，value 是 ThreadLocal 的泛型值。

对照着几段关键源码来看，更容易理解一点哈～

```
public class Thread implements Runnable {  
    //ThreadLocal.ThreadLocalMap 是 Thread 的属性  
    ThreadLocal.ThreadLocalMap threadLocals = null;  
}
```

ThreadLocal 中的关键方法 set()和 get()

```
public void set(T value) {  
    Thread t = Thread.currentThread(); //获取当前线程 t  
    ThreadLocalMap map = getMap(t); //根据当前线程获取到 ThreadLocalMap  
    if (map != null)  
        map.set(this, value); //K, V 设置到 ThreadLocalMap 中  
    else  
        createMap(t, value); //创建一个新的 ThreadLocalMap  
}
```

```
public T get() {  
    Thread t = Thread.currentThread(); //获取当前线程 t  
    ThreadLocalMap map = getMap(t); //根据当前线程获取到 ThreadLocalMap  
    if (map != null) {  
        //由 this (即 ThreadLocal 对象) 得到对应的 Value, 即 ThreadLocal 的泛型值  
        ThreadLocalMap.Entry e = map.getEntry(this);  
        if (e != null) {  
            @SuppressWarnings("unchecked")  
            T result = (T)e.value;  
            return result;  
        }  
    }  
    return setInitialValue();  
}
```

ThreadLocalMap 的 Entry 数组

```
static class ThreadLocalMap {  
    static class Entry extends WeakReference<ThreadLocal<?>> {
```

```

    /** The value associated with this ThreadLocal. */
    Object value;

    Entry(ThreadLocal<?> k, Object v) {
        super(k);
        value = v;
    }
}
}
}

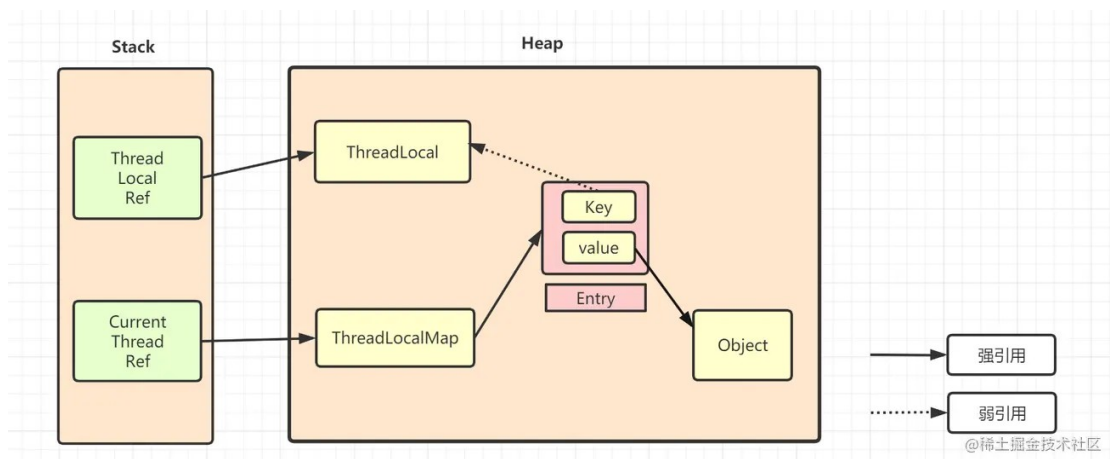
```

所以怎么回答「ThreadLocal 的实现原理」？如下，最好是能结合以上结构图一起说明哈～

- Thread 类有一个类型为 ThreadLocal.ThreadLocalMap 的实例变量 threadLocals，即每个线程都有一个属于自己的 ThreadLocalMap。
- ThreadLocalMap 内部维护着 Entry 数组，每个 Entry 代表一个完整的对象，key 是 ThreadLocal 本身，value 是 ThreadLocal 的泛型值。
- 每个线程在往 ThreadLocal 里设置值的时候，都是往自己的 ThreadLocalMap 里存，读也是以某个 ThreadLocal 作为引用，在自己的 map 里找对应的 key，从而实现了线程隔离。

ThreadLocal 内存泄露问题

先看看一下的 ThreadLocal 的引用示意图哈，



ThreadLocalMap 中使用的 key 为 ThreadLocal 的弱引用，如下

```

static class ThreadLocalMap {
    static class Entry extends WeakReference<ThreadLocal<?>> {
        /** The value associated with this ThreadLocal. */
        Object value;

        Entry(ThreadLocal<?> k, Object v) {
            super(k);
            value = v;
        }
    }
}

```

@稀土掘金技术社区

弱引用：只要垃圾回收机制一运行，不管 JVM 的内存空间是否充足，都会回收该对象占用的内存。

弱引用比较容易被回收。因此，如果 ThreadLocal (ThreadLocalMap 的 Key) 被垃圾回收器回收了，但是因为 ThreadLocalMap 生命周期和 Thread 是一样的，它这时候如果不被回收，就会出现这种情况：ThreadLocalMap 的 key 没了，value 还在，这就会「造成了内存泄漏问题」。

如何「解决内存泄漏问题」？使用完 ThreadLocal 后，及时调用 remove() 方法释放内存空间。

ThreadLocal 的应用场景

- 数据库连接池
- 会话管理中使用

可以看下这篇文章哈，[ThreadLocal 夺命 11 连问](#)

6.81 说说 CountDownLatch 与 CyclicBarrier 区别

- CountDownLatch：一个或者多个线程，等待其他多个线程完成某件事情之后才能执行；

- **CyclicBarrier**：多个线程互相等待，直到到达同一个同步点，再继续一起执行。

区别	CountDownLatch	CyclicBarrier
计数方式	递减计数	加法计数
可重复利用性	不可重复利用	可重复利用
初始值	初始值为N, $N > 0$	N为0
计数方式	调用countDown, $N-1$	调用await, $N+1$
阻塞条件	$N > 0$, 调用await一直阻塞	N小于指定值
何时释放等待线程	计数为0时	计数达到指定值N

@稀土掘金技术社区

举个例子吧：

- **CountDownLatch**：假设老师跟同学约定周末在公园门口集合，等人齐了再发门票。那么，发门票（这个主线程），需要等各位同学都到齐（多个其他线程都完成），才能执行。
- **CyclicBarrier**:多名短跑运动员要开始田径比赛，只有等所有运动员准备好，裁判才会鸣枪开始，这时候所有的运动员才会疾步如飞。

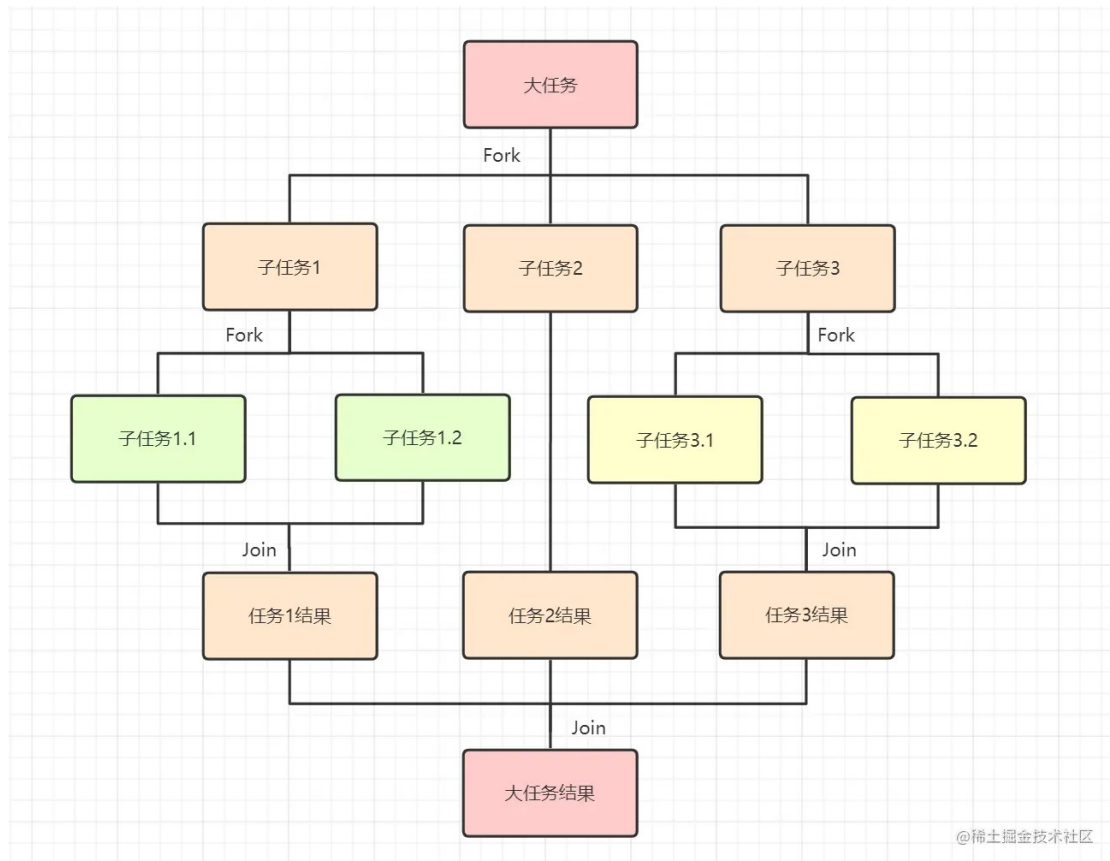
6.82 Fork/Join 框架的理解

Fork/Join 框架是 Java7 提供的一个用于并行执行任务的框架，是一个把大任务分割成若干个小任务，最终汇总每个小任务结果后得到大任务结果的框架。

Fork/Join 框架需要理解两个点，「分而治之」和「工作窃取算法」

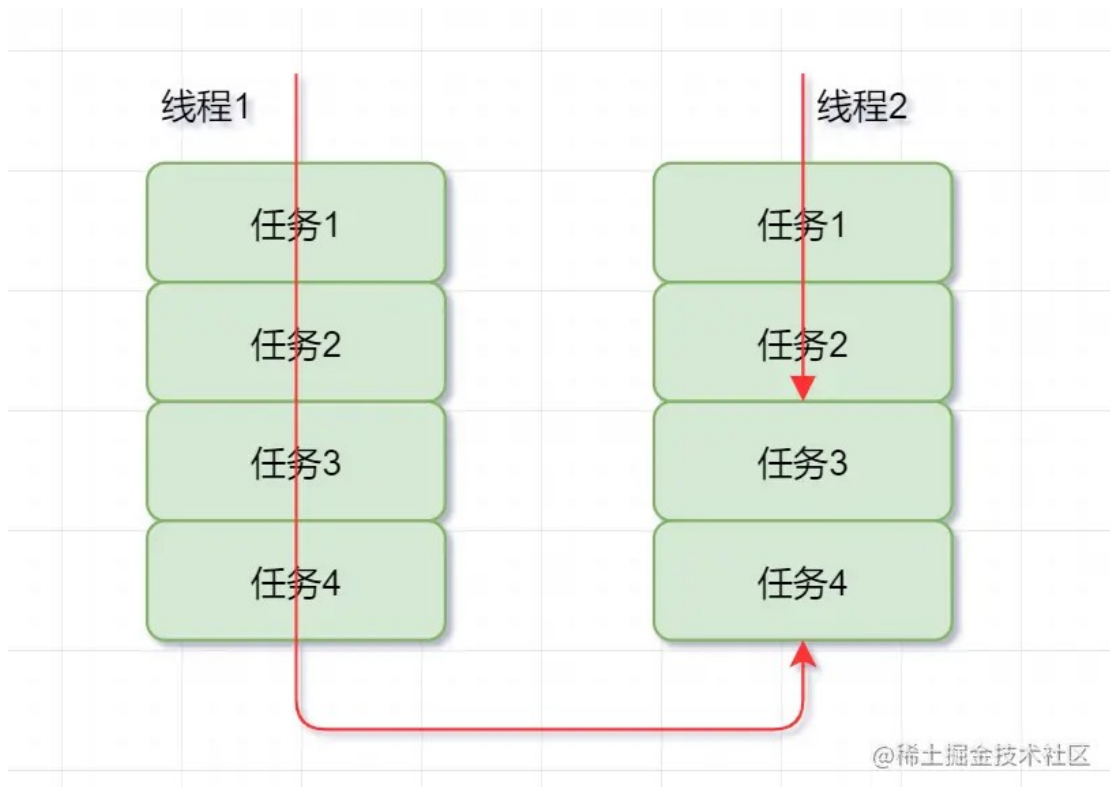
分而治之

以上 Fork/Join 框架的定义，就是分而治之思想的体现啦



工作窃取算法

把大任务拆分成小任务，放到不同队列执行，交由不同的线程分别执行时。有的线程优先把自己负责的任务执行完了，其他线程还在慢慢悠悠处理自己的任务，这时候为了充分提高效率，就需要工作盗窃算法啦~



工作盗窃算法就是，「某个线程从其他队列中窃取任务进行执行的过程」。一般就是指做得快的线程（盗窃线程）抢慢的线程的任务来做，同时为了减少锁竞争，通常使用双端队列，即快线程和慢线程各在一端。

7. JVM 专栏面试题（关注公众号：捡田螺的小男孩）

7.1 什么情况下会发生栈内存溢出。

思路：描述栈定义，再描述为什么会溢出，再说明一下相关配置参数，OK 的话可以给面试官手写是一个栈溢出的 demo。

我的答案：

- 栈是线程私有的，他的生命周期与线程相同，每个方法在执行的时候都会创建一个栈帧，用来存储局部变量表，操作数栈，动态链接，方法出口等信息。局部变量表又包含基本数据类型，对象引用类型

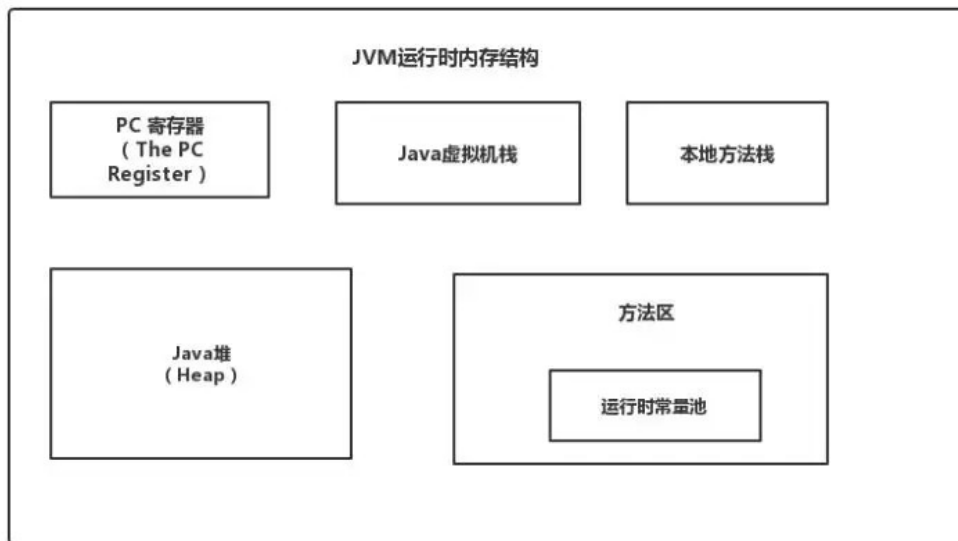
- 如果线程请求的栈深度大于虚拟机所允许的最大深度，将抛出 StackOverflowError 异常，方法递归调用产生这种结果。
- 如果Java 虚拟机栈可以动态扩展，并且扩展的动作已经尝试过，但是无法申请到足够的内存去完成扩展，或者在新建立线程的时候没有足够的内存去创建对应的虚拟机栈，那么Java 虚拟机将抛出一个 OutOfMemory 异常。(线程启动过多)
- 参数 -Xss 去调整 JVM 栈的大小

7.2 详解JVM 内存模型

思路： 给面试官画一下JVM 内存模型图，并描述每个模块的定义，作用，以及可能会存在的问题，如栈溢出等。

我的答案：

- JVM 内存结构



程序计数器：当前线程所执行的字节码的行号指示器，用于记录正在执行的虚拟机字节指令地址，线程私有。

Java 虚拟栈：存放基本数据类型、对象的引用、方法出口等，线程私有。

Native 方法栈：和虚拟栈相似，只不过它服务于 Native 方法，线程私有。

Java 堆：java 内存最大的一块，所有对象实例、数组都存放在 java 堆，GC 回收的地方，线程共享。

方法区：存放已被加载的类信息、常量、静态变量、即时编译器编译后的代码数据等。（即永久带），回收目标主要是常量池的回收和类型的卸载，各线程共享

7.3 JVM 内存为什么要分成新生代，老年代，持久代。新生代中为什么要分为 Eden 和 Survivor。

思路：先讲一下 JAVA 堆，新生代的划分，再谈谈它们之间的转化，相互之间一些参数的配置（如：-XX:NewRatio，-XX:SurvivorRatio 等），再解释为什么要这样划分，最好加一点自己的理解。

我的答案：

1) 共享内存区划分

- 共享内存区 = 持久带 + 堆
- 持久带 = 方法区 + 其他
- Java 堆 = 老年代 + 新生代
- 新生代 = Eden + S0 + S1

2) 一些参数的配置

- 默认的，新生代（Young）与老年代（Old）的比例的值为 1:2，可以通过参数 -XX:NewRatio 配置。

- 默认的，Eden : from : to = 8 : 1 : 1 (可以通过参数 -XX:SurvivorRatio 来设定)
- Survivor 区中的对象被复制次数为 15(对应虚拟机参数 -XX:+MaxTenuringThreshold)

3)为什么要分为 Eden 和 Survivor?为什么要设置两个 Survivor 区？

- 如果没有 Survivor，Eden 区每进行一次 Minor GC，存活的对象就会被送到老年代。老年代很快被填满，触发 Major GC.老年代的内存空间远大于新生代，进行一次 Full GC 消耗的时间比 Minor GC 长得多,所以需要分为 Eden 和 Survivor。
- Survivor 的存在意义，就是减少被送到老年代的对象，进而减少 Full GC 的发生，Survivor 的预筛选保证，只有经历 16 次 Minor GC 还能在新生代中存活的对象，才会被送到老年代。
- 设置两个 Survivor 区最大的好处就是解决了碎片化，刚刚新建的对象在 Eden 中，经历一次 Minor GC，Eden 中的存活对象就会被移动到第一块 survivor space S0，Eden 被清空；等 Eden 区再满了，就再触发一次 Minor GC，Eden 和 S0 中的存活对象又会被复制送入第二块 survivor space S1 (这个过程非常重要，因为这种复制算法保证了 S1 中来自 S0 和 Eden 两部分的存活对象占用连续的内存空间，避免了碎片化的发生)

7.4 JVM 中一次完整的 GC 流程是怎样的，对象如何晋升到老年代

思路：先描述一下 Java 堆内存划分，再解释 Minor GC，Major GC，full GC，描述它们之间转化流程。

我的答案：

- Java 堆 = 老年代 + 新生代
- 新生代 = Eden + S0 + S1
- 当 Eden 区的空间满了，Java 虚拟机会触发一次 Minor GC，以收集新生代的垃圾，存活下来的对象，则会转移到 Survivor 区。
- **大对象** (需要大量连续内存空间的 Java 对象，如那种很长的字符串) **直接进入老年态**；

- 如果对象在 Eden 出生，并经过第一次 Minor GC 后仍然存活，并且被 Survivor 容纳的话，年龄设为 1，每熬过一次 Minor GC，年龄+1，若年龄超过一定限制（15），则被晋升到老年态。即长期存活的对象进入老年态。
- 老年代满了而无法容纳更多的对象，Minor GC 之后通常就会进行 Full GC，Full GC 清理整个内存堆 – 包括年轻代和老年代。
- Major GC 发生在老年代的 GC，清理老年区，经常会伴随至少一次 Minor GC，比 Minor GC 慢 10 倍以上。

7.5 你知道哪几种垃圾收集器，各自的优缺点，重点讲下 cms 和 G1，包括原理，流程，优缺点。

思路：一定要记住典型的垃圾收集器，尤其 cms 和 G1，它们的原理与区别，涉及的垃圾回收算法。

我的答案：

1) 几种垃圾收集器：

- **Serial 收集器**：单线程的收集器，收集垃圾时，必须 stop the world，使用复制算法。
- **ParNew 收集器**：Serial 收集器的多线程版本，也需要 stop the world，复制算法。
- **Parallel Scavenge 收集器**：新生代收集器，复制算法的收集器，并发的多线程收集器，目标是达到一个可控的吞吐量。如果虚拟机总共运行 100 分钟，其中垃圾花掉 1 分钟，吞吐量就是 99%。
- **Serial Old 收集器**：是 Serial 收集器的老年代版本，单线程收集器，使用标记整理算法。
- **Parallel Old 收集器**：是 Parallel Scavenge 收集器的老年代版本，使用多线程，标记-整理算法。
- **CMS(Concurrent Mark Sweep) 收集器**：是一种以获得最短回收停顿时间为目标的收集器，标记清除算法，运作过程：初始标记，并发标记，重新标记，并发清除，收集结束会产生大量空间碎片。
- **G1 收集器**：标记整理算法实现，运作流程主要包括以下：初始标记，并发标记，最终标记，筛选标记。不会产生空间碎片，可以精确地控制停顿。

2) CMS 收集器和 G1 收集器的区别：

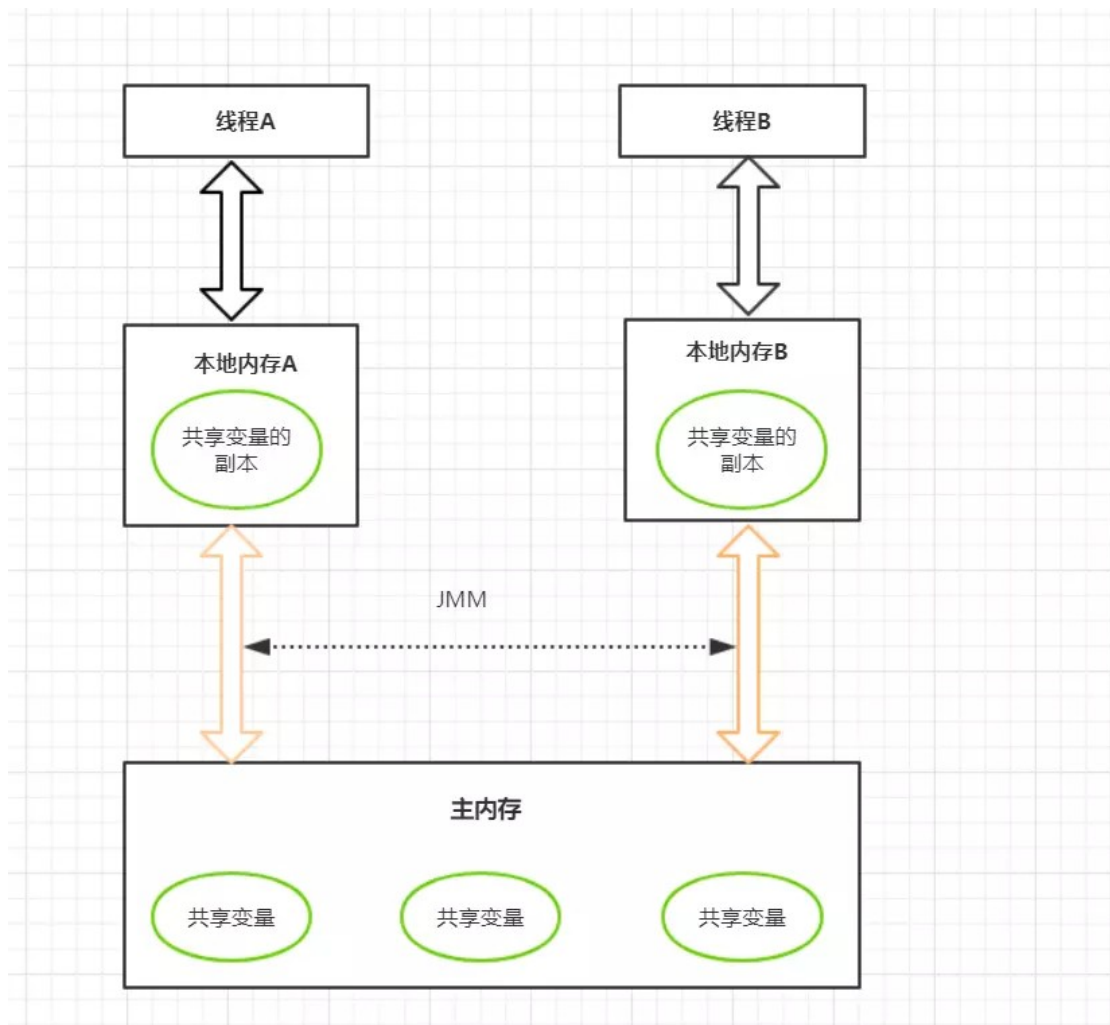
- CMS 收集器是老年代的收集器，可以配合新生代的 Serial 和 ParNew 收集器一起使用；
- G1 收集器收集范围是老年代和新生代，不需要结合其他收集器使用；
- CMS 收集器以最小的停顿时间为目标的收集器；
- G1 收集器可预测垃圾回收的停顿时间
- CMS 收集器是使用“标记-清除”算法进行的垃圾回收，容易产生内存碎片
- G1 收集器使用的是“标记-整理”算法，进行了空间整合，降低了内存空间碎片。

7.6 JVM 内存模型的相关知识了解多少，比如重排序，内存屏障，happen-before，主内存，工作内存。

思路：先画出 Java 内存模型图，结合例子 volatile，说明什么是重排序，内存屏障，最好能给面试官写以下 demo 说明。

我的答案：

1) Java 内存模型图：



Java 内存模型规定了所有的**变量都存储在主内存**中，每条**线程还有自己的工作内存**，线程的工作内存中保存了该线程中是用到的变量的主内存副本拷贝，**线程对变量的所有操作都必须在工作内存中进行，而不能直接读写主内存**。不同的线程之间也**无法直接访问对方工作内存中的变量**，线程间变量的传递均需要自己的工作内存和主存之间进行数据同步进行。

2) 指令重排序。

在这里，先看一段代码

```
public class PossibleReordering {  
    static int x = 0, y = 0;
```

```

static int a = 0, b = 0;

public static void main(String[] args) throws InterruptedException {
    Thread one = new Thread(new Runnable() {
        public void run() {
            a = 1;
            x = b;
        }
    });

    Thread other = new Thread(new Runnable() {
        public void run() {
            b = 1;
            y = a;
        }
    });

    one.start();other.start();
    one.join();other.join();
    System.out.println("(" + x + "," + y + ")");
}

```

运行结果可能为(1,0)、(0,1)或(1,1)，也可能是(0,0)。因为，在实际运行时，代码指令可能并不是严格按照代码语句顺序执行的。大多数现代微处理器都会采用将指令乱序执行 (out-of-order execution，简称 OoOE 或 OOE) 的方法，在条件允许的情况下，直接运行当前有能力立即执行的后续指令，避开获取下一条指令所需数据时造成的等待³。通过乱序执行的技术，处理器可以大大提高执行效率。而这就是**指令重排**。

3) 内存屏障

内存屏障，也叫内存栅栏，是一种 CPU 指令，用于控制特定条件下的重排序和内存可见性问题。

- **LoadLoad 屏障**：对于这样的语句 Load1; LoadLoad; Load2，在 Load2 及后续读取操作要读取的数据被访问前，保证 Load1 要读取的数据被读取完毕。

- **StoreStore 屏障**：对于这样的语句 Store1; StoreStore; Store2，在 Store2 及后续写入操作执行前，保证 Store1 的写入操作对其它处理器可见。
- **LoadStore 屏障**：对于这样的语句 Load1; LoadStore; Store2，在 Store2 及后续写入操作被刷出前，保证 Load1 要读取的数据被读取完毕。
- **StoreLoad 屏障**：对于这样的语句 Store1; StoreLoad; Load2，在 Load2 及后续所有读取操作执行前，保证 Store1 的写入对所有处理器可见。它的开销是四种屏障中最大的。在大多数处理器的实现中，这个屏障是个万能屏障，兼具其它三种内存屏障的功能。

4) happen-before 原则

- **单线程 happen-before 原则**：在同一个线程中，书写在前面的操作 happen-before 后面的操作。锁的 happen-before 原则：同一个锁的 unlock 操作 happen-before 此锁的 lock 操作。
- **volatile 的 happen-before 原则**：对一个 volatile 变量的写操作 happen-before 对此变量的任意操作(当然也包括写操作了)。
- **happen-before 的传递性原则**：如果 A 操作 happen-before B 操作，B 操作 happen-before C 操作，那么 A 操作 happen-before C 操作。
- **线程启动的 happen-before 原则**：同一个线程的 start 方法 happen-before 此线程的其它方法。
- **线程中断的 happen-before 原则**：对线程 interrupt 方法的调用 happen-before 被中断线程的检测到中断发送的代码。
- **线程终结的 happen-before 原则**：线程中的所有操作都 happen-before 线程的终止检测。
- **对象创建的 happen-before 原则**：一个对象的初始化完成先于他的 finalize 方法调用。

7.7 简单说说你了解的类加载器，可以打破双亲委派么，怎么打破。

思路：先说明一下什么是类加载器，可以给面试官画个图，再说一下类加载器存在的意义，说一下双亲委派模型，最后阐述怎么打破双亲委派模型。

我的答案：

1) 什么是类加载器？

类加载器 就是根据指定全限定名称将 class 文件加载到 JVM 内存，转为 Class 对象。

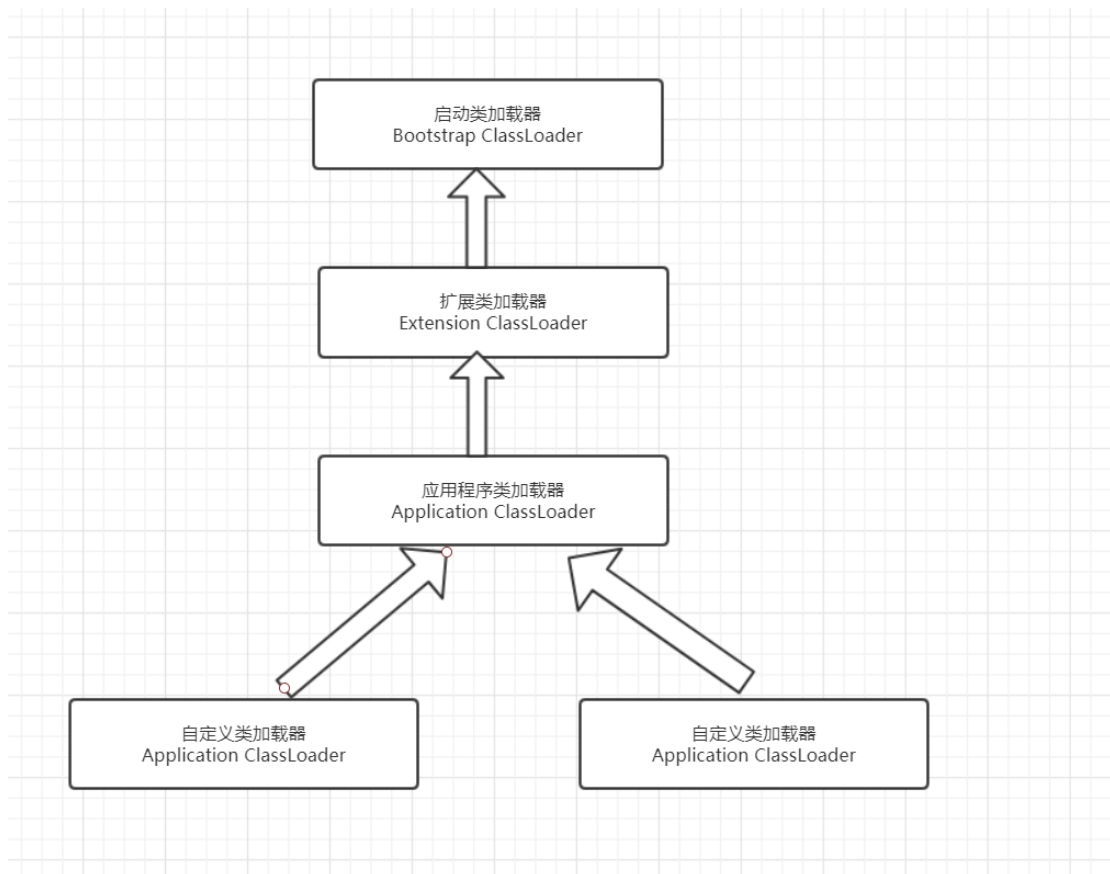
- 启动类加载器 (Bootstrap ClassLoader) : 由 C++ 语言实现 (针对 HotSpot) ,负责将存放在<JAVA_HOME>\lib 目录或-Xbootclasspath 参数指定的路径中的类库加载到内存中。
- 其他类加载器 : 由 Java 语言实现 , 继承自抽象类 ClassLoader。如 :
- 扩展类加载器 (Extension ClassLoader) : 负责加载<JAVA_HOME>\lib\ext 目录或 java.ext.dirs 系统变量指定的路径中的所有类库。
- 应用程序类加载器 (Application ClassLoader) 。负责加载用户类路径 (classpath) 上的指定类库 , 我们可以直接使用这个类加载器。一般情况 , 如果我们没有自定义类加载器默认就是用这个加载器。

2) 双亲委派模型

双亲委派模型工作过程是 :

如果一个类加载器收到类加载的请求 , 它首先不会自己去尝试加载这个类 , 而是把这个请求委派给父类加载器完成。每个类加载器都是如此 , 只有当父加载器在自己的搜索范围内找不到指定的类时 (即 ClassNotFoundException) , 子加载器才会尝试自己去加载。

双亲委派模型图 :



3) 为什么需要双亲委派模型？

在这里，先想一下，如果没有双亲委派，那么用户是不是可以自己定义一个 **java.lang.Object** 的同名类，**java.lang.String** 的同名类，并把它放到 ClassPath 中,那么类之间的比较结果及类的唯一性将无法保证，因此，为什么需要双亲委派模型？防止内存中出现多份同样的字节码

4) 怎么打破双亲委派模型？

打破双亲委派机制则不仅要继承 **ClassLoader** 类，还要重写 **loadClass** 和 **findClass** 方法。

7.8 说说你知道的几种主要的 JVM 参数

思路：可以说一下堆栈配置相关的，垃圾收集器相关的，还有一下辅助信息相关的。

我的答案：

1) 堆栈配置相关

```
java -Xmx3550m -Xms3550m -Xmn2g -Xss128k  
-XX:MaxPermSize=16m -XX:NewRatio=4 -XX:SurvivorRatio=4 -XX:MaxTenuringThreshold=0
```

-Xmx3550m：最大堆大小为 3550m。

-Xms3550m：设置初始堆大小为 3550m。

-Xmn2g：设置年轻代大小为 2g。

-Xss128k：每个线程的堆栈大小为 128k。

-XX:MaxPermSize：设置持久代大小为 16m

-XX:NewRatio=4：设置年轻代（包括 Eden 和两个 Survivor 区）与年老代的比值（除去持久代）。

-XX:SurvivorRatio=4：设置年轻代中 Eden 区与 Survivor 区的大小比值。

设置为 4，则两个 Survivor 区与一个 Eden 区的比值为 2:4，一个 Survivor 区占整个年轻代的 1/6

-XX:MaxTenuringThreshold=0：设置垃圾最大年龄。如果设置为 0 的话，则年轻代对象不经过 Survivor 区，直接进入年老代。

2) 垃圾收集器相关

```
-XX:+UseParallelGC
```

```
-XX:ParallelGCThreads=20
```

```
-XX:+UseConcMarkSweepGC
```

```
-XX:CMSFullGCsBeforeCompaction=5
```

```
-XX:+UseCMSCompactAtFullCollection :
```

-XX:+UseParallelGC：选择垃圾收集器为并行收集器。

-XX:ParallelGCThreads=20：配置并行收集器的线程数

-XX:+UseConcMarkSweepGC：设置年老代为并发收集。

-XX:CMSFullGCsBeforeCompaction：由于并发收集器不对内存空间进行压缩、整理，所以运行一段时间以后会产生“碎片”，使得运行效率降低。此值设置运行多少次 GC 以后对内存空间进行压缩、整理。

-XX:+UseCMSCompactAtFullCollection：打开对年老代的压缩。可能会影响性能，但是可以消除碎片

3) 辅助信息相关

```
-XX:+PrintGC
```

```
-XX:+PrintGCDetails
```

-XX:+PrintGC 输出形式:

```
[GC 118250K->113543K(130112K), 0.0094143 secs] [Full GC
121376K->10414K(130112K), 0.0650971 secs]
```

-XX:+PrintGCDetails 输出形式:

```
[GC [DefNew: 8614K->781K(9088K), 0.0123035 secs] 118250K-
>113543K(130112K), 0.0124633 secs] [GC [DefNew: 8614K-
>8614K(9088K), 0.0000665 secs][Tenured: 112761K-
>10414K(121024K), 0.0433488 secs] 121376K->10414K(130112K),
0.0436268 secs
```

7.9 怎么打出线程栈信息。

思路：可以说一下 jps , top , jstack 这几个命令，再配合一次排查线上问题进行解答。

我的答案：

- 输入 jps , 获得进程号。
- top -Hp pid 获取本进程中所有线程的 CPU 耗时性能
- jstack pid 命令查看当前 java 进程的堆栈状态
- 或者 jstack -l > /tmp/output.txt 把堆栈信息打到一个 txt 文件。
- 可以使用 fastthread 堆栈定位 , <http://fastthread.io/>

7.10 强引用、软引用、弱引用、虚引用的区别？

思路：先说一下四种引用的定义，可以结合代码讲一下，也可以扩展谈到 ThreadLocalMap 里弱引用用处。

我的答案：

1) 强引用

我们平时 new 了一个对象就是强引用，例如 `Object obj = new Object();`即使内存不足的情况下，JVM 宁愿抛出 `OutOfMemory` 错误也不会回收这种对象。

2) 软引用

如果一个对象只具有软引用，则内存空间足够，垃圾回收器就不会回收它；如果内存空间不足了，就会回收这些对象的内存。

```
SoftReference<String> softRef=new SoftReference<String>(str); // 软引用
```

用处：软引用在实际中有重要的应用，例如浏览器的后退按钮。按后退时，这个后退时显示的网页内容是重新进行请求还是从缓存中取出呢？这就要看具体的实现策略了。

(1) 如果一个网页在浏览结束时就进行内容的回收，则按后退查看前面浏览过的页面时，需要重新构建

(2) 如果将浏览过的网页存储到内存中会造成内存的大量浪费，甚至会造成内存溢出

如下代码：

```
Browser prev = new Browser(); // 获取页面进行浏览
SoftReference sr = new SoftReference(prev); // 浏览完毕后置为软引用
if(sr.get()!=null){
    rev = (Browser) sr.get(); // 还没有被回收器回收，直接获取
}else{
    prev = new Browser(); // 由于内存吃紧，所以对软引用的对象回收了
    sr = new SoftReference(prev); // 重新构建
}
```

3) 弱引用

具有弱引用的对象拥有更短暂的生命周期。在垃圾回收器线程扫描它所管辖的内存区域的过程中，一旦发现了只具有弱引用的对象，不管当前内存空间足够与否，都会回收它的内存。

```
String str=new String("abc");
WeakReference<String> abcWeakRef = new WeakReference<String>(str);
str=null;
```

等价于

```
str = null;  
System.gc();
```

4) 虚引用

如果一个对象仅持有虚引用，那么它就和没有任何引用一样，在任何时候都可能被垃圾回收器回收。虚引用主要用来跟踪对象被垃圾回收器回收的活动。

8.oppo 面试真题（关注公众号：捡田螺的小男孩。）

大家好，我是捡田螺的小男孩。最近有为读者去面试了 oppo，给大家整理了面试真题的答案。希望对大家有帮助哈，一起学习，一起进步。更多干货，关注公众号：捡田螺的小男孩。



8.1 聊聊你印象最深刻的项目，或者做了什么优化。

大家平时做的项目，如果很多知识点跟面试八股文相关的话，就可以相对条理清晰地写到简历去。

比如缓存数据库相关的，查询为空，你设置了一个-1到缓存，代表数据库没记录。下次判断-1，就不查库了，以解决缓存穿透问题。又比如你设置缓存过期时间比较分散，解决缓存击穿问题，都可以条理清晰写到简历去，这样面试官很可能会问你相关的问题，这时候就对答如流啦。

还有平时你做的项目，有一些比较好的设计，都可以说一下哈，比如你是如何保证数据一致性的，怎么优化接口性能的。如果是讲**优化接口**这一块的话，你可以看下我这篇文章哈，结合来一起讲。其实就是**缓存、分批、并发调用、异步**等那几个关键知识点。

记一次接口性能优化实践总结：优化接口性能的八个建议

如果是代码优化细节，可以结合我这篇：工作四年，分享 50 个让你代码更好的小建议。你可以挑个简单的来讲，比如**复杂的 if 逻辑条件**，可以调整顺序，让**程序更高效**，这样会让面试官眼前一亮哦。

如果是慢 SQL 优化这一块，可以看下我之前 MySQL 专栏系列文章，理解透之后，还是挺稳的：

- 看一遍就理解：order by 详解
- 看一遍就理解：group by 详解
- 实战！聊聊如何解决 MySQL 深分页问题
- 后端程序员必备：书写高质量 SQL 的 30 条建议
- 阿里一面，给了几条 SQL，问需要执行几次树搜索操作？
- 生产问题分析！delete in 子查询不走索引？！
- 面试官问如何优化慢 SQL？

8.2 你项目提到分布式锁，你们是怎么使用分布式锁的？

一般你讲述你做的项目时，面试官会根据你项目涉及的一些面试点，然后抽他感兴趣的一两个来问。所以大家对哪些知识点熟悉，讲述项目时，就说你用该知识点，解决了什么问题。

- 比如，你看了田螺哥的 [《七种方案！探讨 Redis 分布式锁的正确使用姿势》](#)，很熟悉，就可以说用分布式锁解决了超卖问题什么的。
- 如果你看了田螺哥的 [《美团二面：Redis 与 MySQL 双写一致性如何保证？》](#)，你就可以说你是用什么方案保证缓存好数据库一致性的。

8.3 常见分布式事务解决方案

分布式事务：就是指事务的参与者、支持事务的服务器、资源服务器以及事务管理器分别位于不同的分布式系统的不同节点之上。简单来说，分布式事务指的就是分布式系统中的事务，它的存在就是为了保证不同数据库节点的数据一致性。

聊到分布式事务，大家记得这两个理论哈：**CAP 理论** 和 **BASE 理论**

分布式事务的几种解决方案：

- 2PC(二阶段提交)方案、3PC
- TCC (Try、Confirm、Cancel)
- 本地消息表
- 最大努力通知
- seata

2PC(二阶段提交)方案

2PC，即两阶段提交，它将分布式事务的提交拆分为 2 个阶段：**prepare** 和 **commit/rollback**，即准备阶段和提交执行阶段。在 prepare 准备阶段需要等待所有参与子事务的反馈，因此可能造成数据库资源锁定时间过长，不适合并发

高以及子事务生命周长较长的业务场景。并且协调者宕机，所有的参与者都收不到提交或回滚指令。

3PC

两阶段提交分别是：`CanCommit`，`PreCommit` 和 `doCommit`，这里不再详述。3PC 利用超时机制解决了 2PC 的同步阻塞问题，避免资源被永久锁定，进一步加强了整个事务过程的可靠性。但是 3PC 同样无法应对类似的宕机问题，只不过出现多数据源中数据不一致问题的概率更小。

TCC

TCC 采用了补偿机制，其核心思想是：针对每个操作，都要注册一个与其对应的确认和补偿（撤销）操作。它分为三个阶段：`Try-Confirm-Cancel`

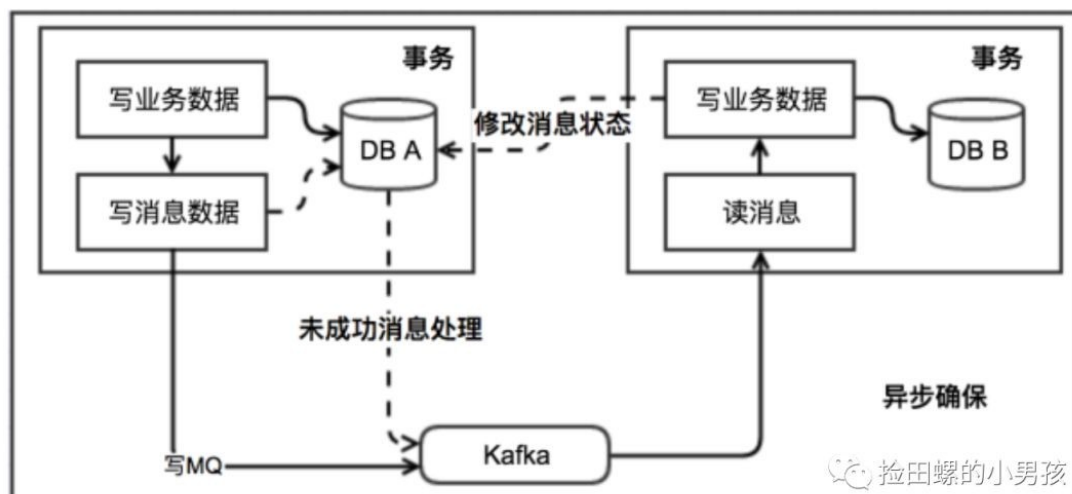
- try 阶段：尝试去执行，完成所有业务的一致性检查，预留必须的业务资源。
- Confirm 阶段：该阶段对业务进行确认提交，不做任何检查，因为 try 阶段已经检查过了，默认 Confirm 阶段是不会出错的。
- Cancel 阶段：若业务执行失败，则进入该阶段，它会释放 try 阶段占用的所有业务资源，并回滚 Confirm 阶段执行的所有操作。

TCC 方案让应用可以自定义数据库操作的粒度，降低了锁冲突，可以提升性能。

但是应用侵入性强，try、confirm、cancel 三个阶段都需要业务逻辑实现。

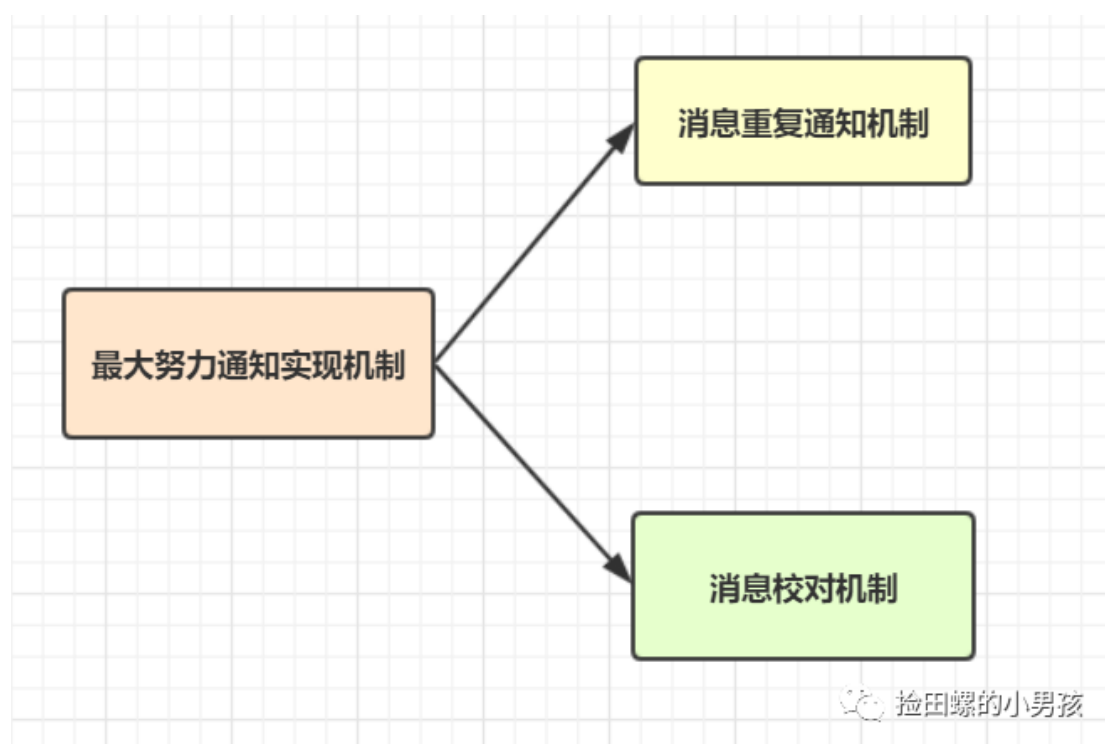
本地消息表

ebay 最初提出本地消息表这个方案，来解决分布式事务问题。业界目前使用这种方案是比较多的，它的核心思想就是将分布式事务拆分成本地事务进行处理。可以看一下基本的实现流程图：



最大努力通知

最大努力通知方案的目标，就是发起通知方通过一定的机制，最大努力将业务处理结果通知到接收方。



seata

Saga 模式是 Seata 提供的长事务解决方案。核心思想是将长事务拆分为多个本地短事务，由 Saga 事务协调器协调，如果正常结束那就正常完成，如果某个步骤失败，则根据相反顺序一次调用补偿操作。

Saga 的并发度高，但是一致性弱，对于转账，可能发生用户已扣款，最后转账又失败的情况。

整理了这几篇分布式事务文章，大家可以看看哈：

- [看一遍就理解：分布式事务详解](#)
- [1.4 w 字，25 张图让你彻底掌握分布式事务原理](#)
- [后端程序员必备：分布式事务基础篇](#)

8.4 你们的接口幂等是如何保证的？

如果你调用下游接口超时了，是不是考虑重试？如果重试，下游接口就需要支持幂等啦。

实现幂等一般有这 8 种方案：

- select+insert+主键/唯一索引冲突
- 直接 insert + 主键/唯一索引冲突
- 状态机幂等
- 抽取防重表
- token 令牌
- 悲观锁(如 select for update，很少用)
- 乐观锁
- 分布式锁

大家平时是用哪个方案解决幂等的，最后结合工作实际讲讲哈。可以看下我之

前这篇文章：[聊聊幂等设计](#)

8.5 你们的 MySQL 架构是怎样的？

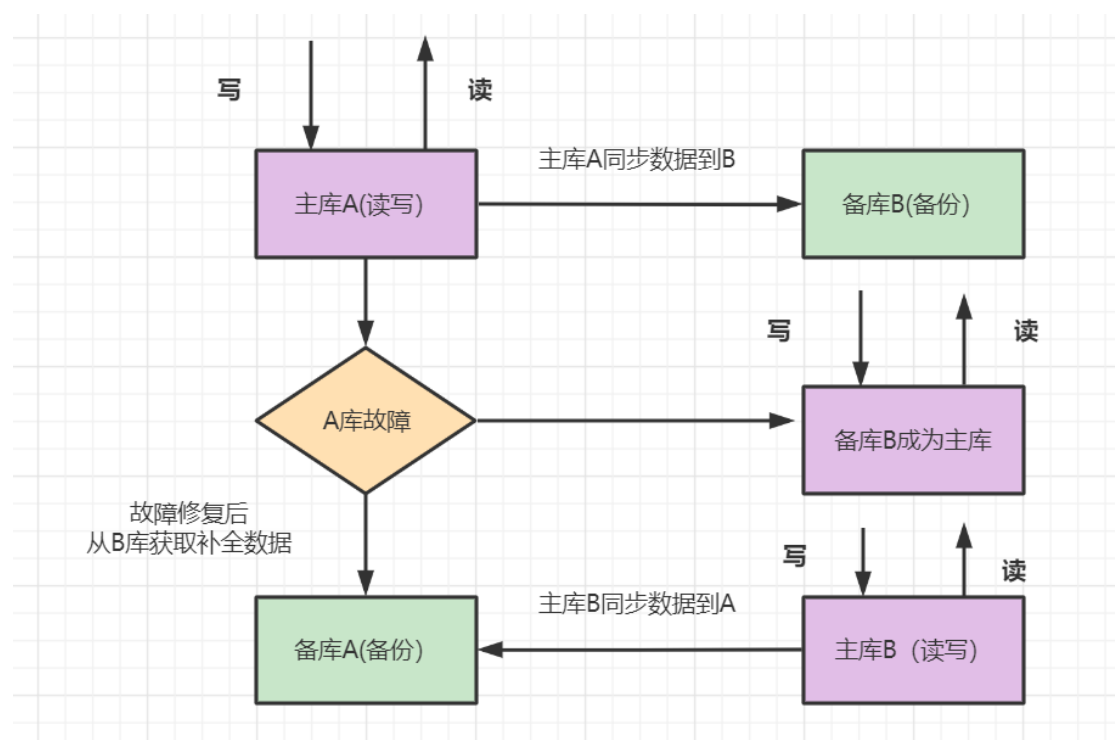
大家可以结合自己公司的 MySQL 架构聊聊。如果不是很清楚的话，可以结合

我之前写的来看看哈：[面试必备：聊聊 MySQL 的主从](#)

数据的库高可用方案

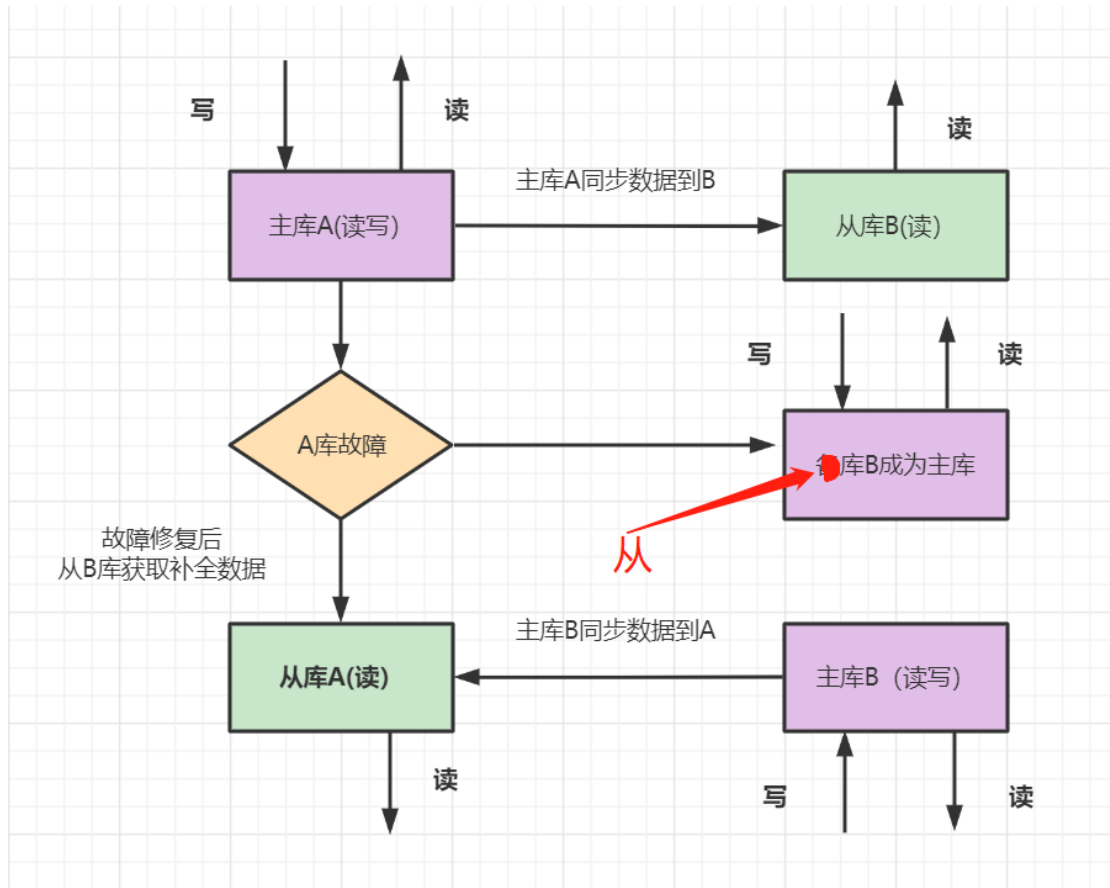
- 双机主备
- 一主一从
- 一主多从
- MariaDB 同步多主机
- 数据库中间件

8.5.1 双机主备



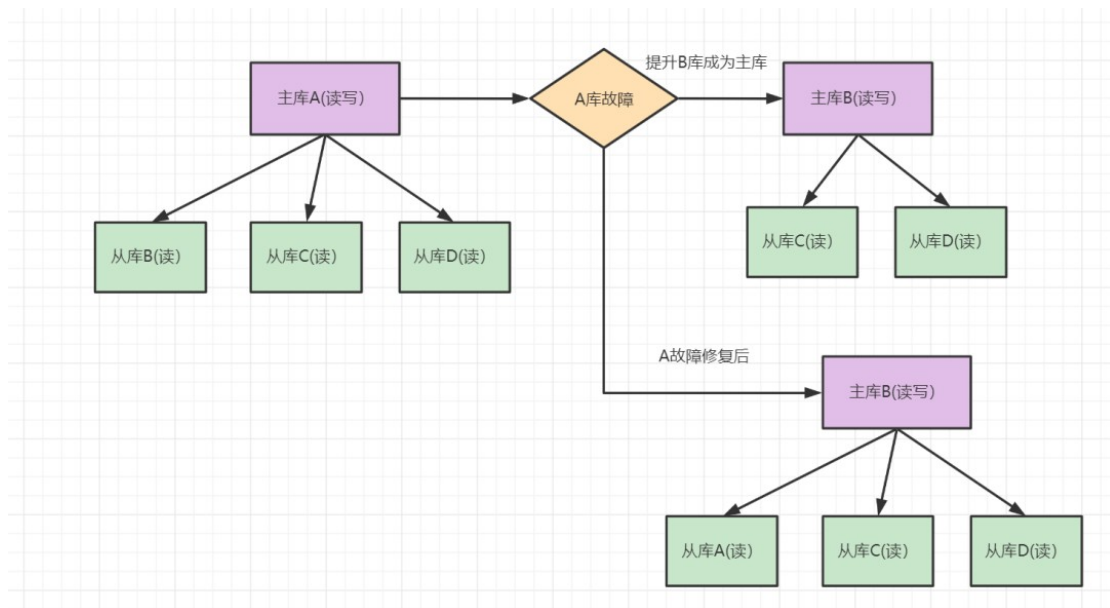
- 优点：一个机器故障了可以自动切换，操作比较简单。
- 缺点：只有一个库在工作，读写压力大，未能实现读写分离，并发也有一定限制

8.5.2 一主一从



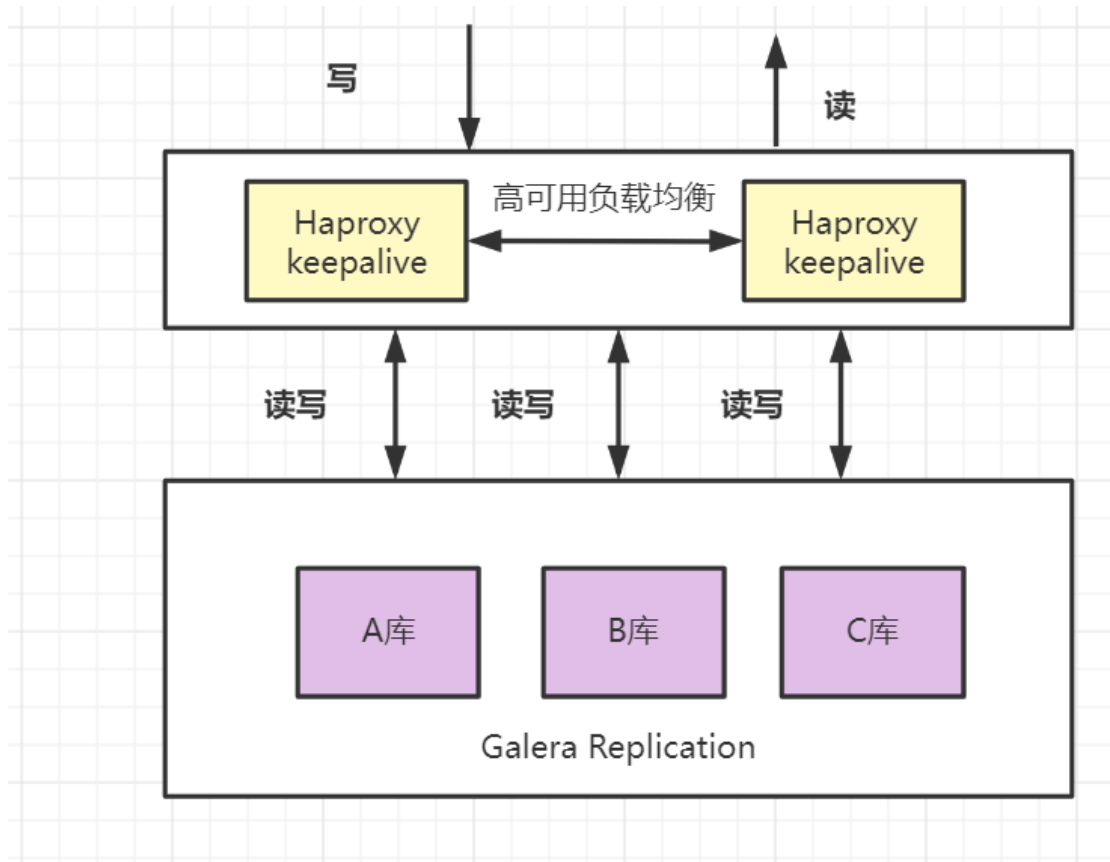
- 优点：从库支持读，分担了主库的压力，提升了并发度。一个机器故障了可以自动切换，操作比较简单。
- 缺点：一台从库，并发支持还是不够，并且一共两台机器，还是存在同时故障的几率，不够高可用。

8.5.3 一主多从



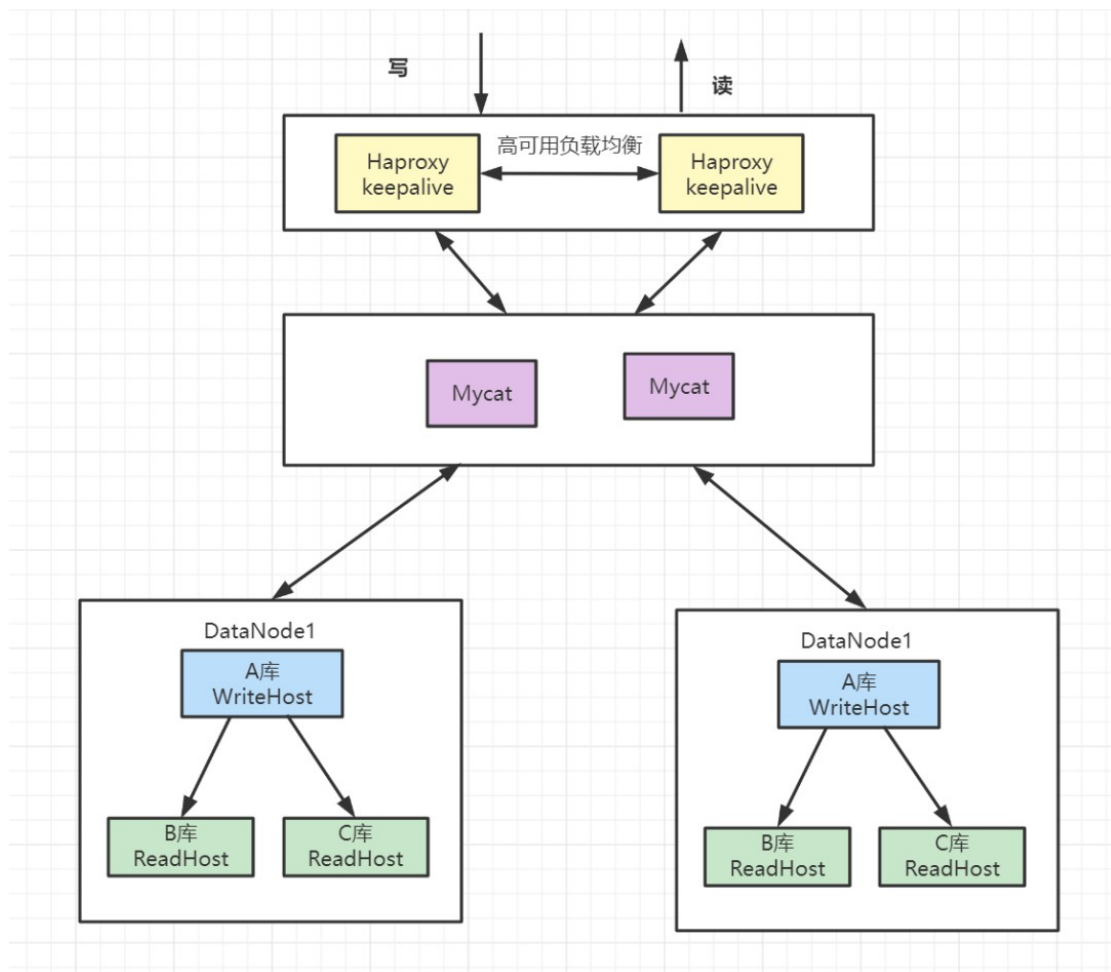
- 优点：多个从库支持读，分担了主库的压力，明显提升了读的并发度。
- 缺点：只有一台主机写，因此写的并发度不高

8.5.4 MariaDB 同步多主机集群



- 有代理层实现负载均衡，多个数据库可以同时进行读写操作；各个数据库之间可以通过 Galera Replication 方法进行数据同步，每个库理论上数据是完全一致的。
- 优点：读写的并发度都明显提升，可以任意节点读写，可以自动剔除故障节点，具有较高的可靠性。
- 缺点：数据量不支持特别大。要避免大事务卡死，如果集群节点一个变慢，其他节点也会跟着变慢。

8.5.5 数据库中间件



- mycat 分片存储，每个分片配置一主多从的集群。
- 优点：解决高并发高数据量的高可用方案
- 缺点：维护成本比较大。

大家有兴趣可以看看我这篇文章哈：[面试必备：聊聊 MySQL 的主从](#)

8.6 常见的索引结构有？哈希表结构属于哪种场景？

哈希表、有序数组和搜索树。

- 哈希表这种结构适用于只有等值查询的场景
- 有序数组适合范围查询，用二分法快速得到，时间复杂度为 $O(\log(N))$ 。查询还好，如果是插入，就得挪动后面所有的记录，成本太高。因此它一般只适用静态存储引擎，比如保存 2018 年某个城市的所有人口信息。
- B+树适合范围查询，我们一般建的索引结构都是 B+树。

8.7 给你 `ab,ac,abc` 字段，你是如何加索引的？

这主要考察联合索引的最左前缀原则知识点。

- 这个最左前缀可以是联合索引的最左 `N` 个字段。比如组合索引 `(a,b,c)` 可以相当于建了 `(a)`，`(a,b)`，`(a,b,c)` 三个索引，大大提高了索引复用能力。
- 最左前缀也可以是字符串索引的最左 `M` 个字符。

因此给你 `ab,ac,abc` 字段，你可以直接加 `abc` 联合索引和 `ac` 联合索引即可。

8.8 数据库隔离级别是否了解？你们的数据库默认隔离级别是？为什么选它？

四大数据库隔离级别，分别是读未提交，读已提交，可重复读，串行化

(`Serializable`)。

- **读未提交**：事务即使未提交，却可以被别的事务读取到的，这级别的事务隔离有脏读、重复读、幻读的问题。
- **读已提交**：当前事务只能读取到其他事务提交的数据，这种事务的隔离级别解决了脏读问题，但还是会存在不可重复读、幻读问题；
- **可重复读**：限制了读取数据的时候，不可以进行修改，所以解决了不可重复读的问题，但是读取范围数据的时候，是可以插入数据，所以还会存在幻读问题。

- **串行化**：事务最高的隔离级别，在该级别下，所有事务都是进行串行化顺序执行的。可以避免脏读、不可重复读与幻读所有并发问题。但是这种事务隔离级别下，事务执行很耗性能。

MySQL 选择 **Repeatable Read (可重复读)** 作为默认隔离级别，我们的数据库隔离级别选的是 **读已提交**。

8.8.1 为什么 MySQL 的默认隔离级别是 RR?

binlog 的格式也有三种：statement，row，mixed。设置为 **statement** 格式，binlog 记录的是 SQL 的原文。又因为 MySQL 在主从复制的过程是通过 **binlog** 进行数据同步，如果设置为读已提交（RC）隔离级别，当出现事务乱序的时候，就会导致备库在 SQL 回放之后，结果和主库内容不一致。

比如一个表 t，表中有两条记录：

```
CREATE TABLE t (
  a int(11) DEFAULT NULL,
  b int(11) DEFAULT NULL,
  PRIMARY KEY a (a),
  KEY b(b)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
insert into t1 values(10,666),(20,233);
```

两个事务并发写操作，如下：

	事务1	事务2
1	设置session为read committed	
2	set autocommit =0	
3	update t set a=30 where b=666	
4		设置session为read committed
5		set autocommit =0
6		update t set b=666 where b=233
7		commit
8	commit	

在读已提交 (RC) 隔离级别下，两个事务执行完后，数据库的两条记录就变成了 (30,666)、(20,666)。这两个事务执行完后，binlog 也就有两条记录，因为事务 binlog 用的是 statement 格式，事务 2 先提交，因此 update t set b=666 where b=233 优先记录，而 update t set a=30 where b=666 记录在后面。

当 bin log 同步到从库后，执行 update t set b=666 where b=233 和 update t set a=30 where b=666 记录，数据库的记录就变成 (30,666)、(30,666)，这时候主从数据不一致啦。

因此 MySQL 的默认隔离级别选择了 RR 而不是 RC。RR 隔离级别下，更新数据的时候不仅对更新的行加行级锁，还会加间隙锁 (gap lock)。事务 2 要执行时，因为事务 1 增加了间隙锁，就会导致事务 2 执行被卡住，只有等事务 1 提交或者回滚后才能继续执行。

并且，MySQL 还禁止在使用 statement 格式的 binlog 的情况下，使用 READ COMMITTED 作为事务隔离级别。

8.8.2 我们的数据库隔离级别最后选的是读已提交 (RC)。

那为什么 MySQL 官方默认隔离级别是 RR，而有些大厂选择了 RC 作为默认的隔离级别呢？

- 提升并发

RC 在加锁的过程中，不需要添加 Gap Lock 和 Next-Key Lock 的，只对要修改的记录添加行级锁就行了。因此 RC 的支持的并发度比 RR 高得多，

- 减少死锁

正式因为 RR 隔离级别增加了 Gap Lock 和 Next-Key Lock 锁，因此它相对于 RC，更容易产生死锁。

8.9 RR 隔离级别实现原理，它是如何解决不可重复读的？

8.9.1 什么是不可重复读

先回忆下什么是不可重复读。假设现在有两个事务 A 和 B：

- 事务 A 先查询 Jay 的余额，查到结果是 100
- 这时候事务 B 对 Jay 的账户余额进行扣减，扣去 10 后，提交事务
- 事务 A 再去查询 Jay 的账户余额发现变成了 90

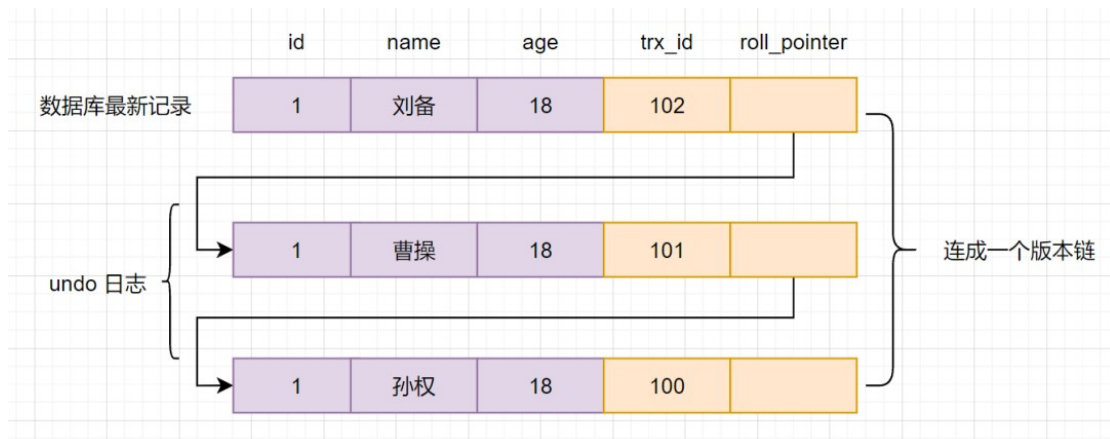
时间编号	事务 A	事务 B
①	Begin;	
②	select balance from account where name='Jay';	
③	读到 Jay 的 balance 为 100	
④		Begin;
⑤		update account set balance =balance-10 where name ='Jay';
⑥		Commit;
⑦	select balance from account where name='Jay';	
⑧	读到 Jay 的 balance 为 100-10=90	
⑨	Commit;	

事务 A 被事务 B 干扰到了！在事务 A 范围内，两个相同的查询，读取同一条记录，却返回了不同的数据，这就是不可重复读。

8.9.2 undo log 版本链 + Read View 可见性规则

RR 隔离级别实现原理，就是 MVCC 多版本并发控制，而 MVCC 是通过 Read View+ Undo Log 实现的，Undo Log 保存了历史快照，Read View 可见性规则帮助判断当前版本的数据是否可见。

Undo Log 版本链长这样：



Read view 的几个重要属性

- **m_ids**: 当前系统中那些活跃(未提交)的读写事务 ID, 它数据结构为一个 List。
- **min_limit_id**: 表示在生成 Read View 时, 当前系统中活跃的读写事务中最小的事务 id, 即 m_ids 中的最小值。
- **max_limit_id**: 表示生成 Read View 时, 系统中应该分配给下一个事务的 id 值。
- **creator_trx_id**: 创建当前 Read View 的事务 ID

Read view 可见性规则如下：

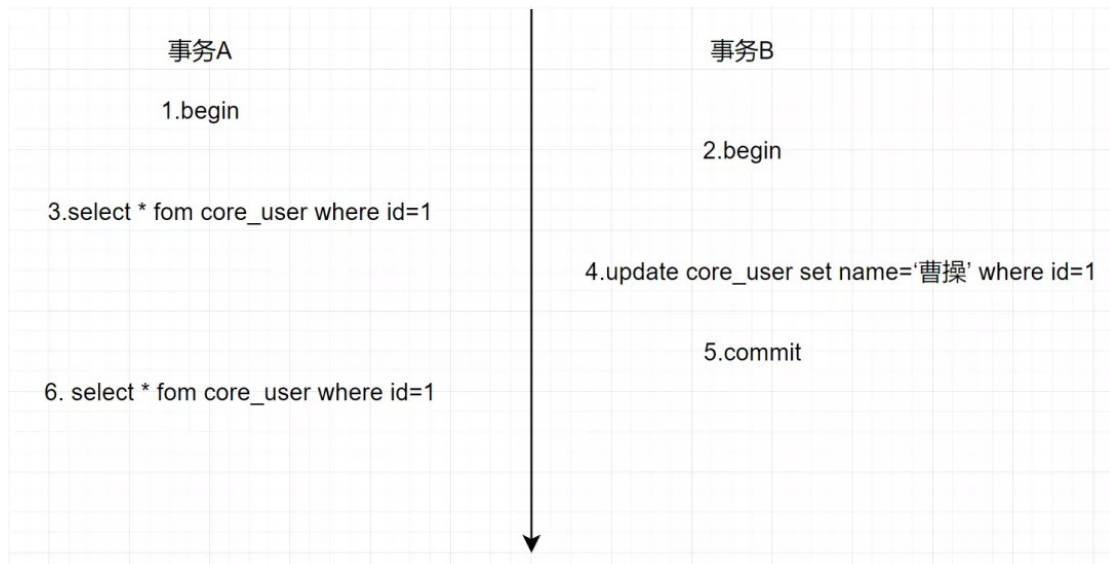
1. 如果数据事务 ID $trx_id < min_limit_id$, 表明生成该版本的事务在生成 Read View 前, 已经提交(因为事务 ID 是递增的), 所以该版本可以被当前事务访问。
2. 如果 $trx_id \geq max_limit_id$, 表明生成该版本的事务在生成 Read View 后才生成, 所以该版本不可以被当前事务访问。
3. 如果 $min_limit_id \leq trx_id < max_limit_id$, 需要分 3 种情况讨论
 - 3.1 如果 **m_ids** 包含 **trx_id**, 则代表 Read View 生成时刻, 这个事务还未提交, 但是如果数据的 **trx_id** 等于 **creator_trx_id** 的话, 表明数据是自己生成的, 因此是可见的。
 - 3.2 如果 **m_ids** 包含 **trx_id**, 并且 **trx_id** 不等于 **creator_trx_id**, 则 Read View 生成时, 事务未提交, 并且不是自己生产的, 所以当前事务也是看不见的;
 - 3.3 如果 **m_ids** 不包含 **trx_id**, 则说明你这个事务在 Read View 生成之前就已经提交了, 修改的结果, 当前事务是能看见的。

8.9.3 RR 如何解决不可重复读

查询一条记录, 基于 MVCC, 是怎样的流程

1. 获取事务自己的版本号，即事务 ID
2. 获取 Read View
3. 查询得到的数据，然后 Read View 中的事务版本号进行比较。
4. 如果不符合 Read View 的可见性规则，即就需要 Undo log 中历史快照;
5. 最后返回符合规则的数据

假设存在事务 A 和 B，SQL 执行流程如下



在可重复读 (RR) 隔离级别下，一个事务里只会获取一次 **Read View**，都是副本共用的，从而保证每次查询的数据都是一样的。

假设当前有一张 core_user 表，插入一条初始化数据,如下：

id	name	age	trx_id	roll_pointer
1	孙权	18	100	

基于 MVCC，我们来看看执行流程

1. A 开启事务，首先得到一个事务 ID 为 100
2. B 开启事务，得到事务 ID 为 101
3. 事务 A 生成一个 Read View，read view 对应的值如下

变量	值
m_ids	100 , 10

变量	值
	1
max_limit_id	102
min_limit_id	100
creator_trx_id	100

然后回到版本链：开始从版本链中挑选可见的记录：

id	name	age	trx_id	roll_pointer
1	孙权	18	100	

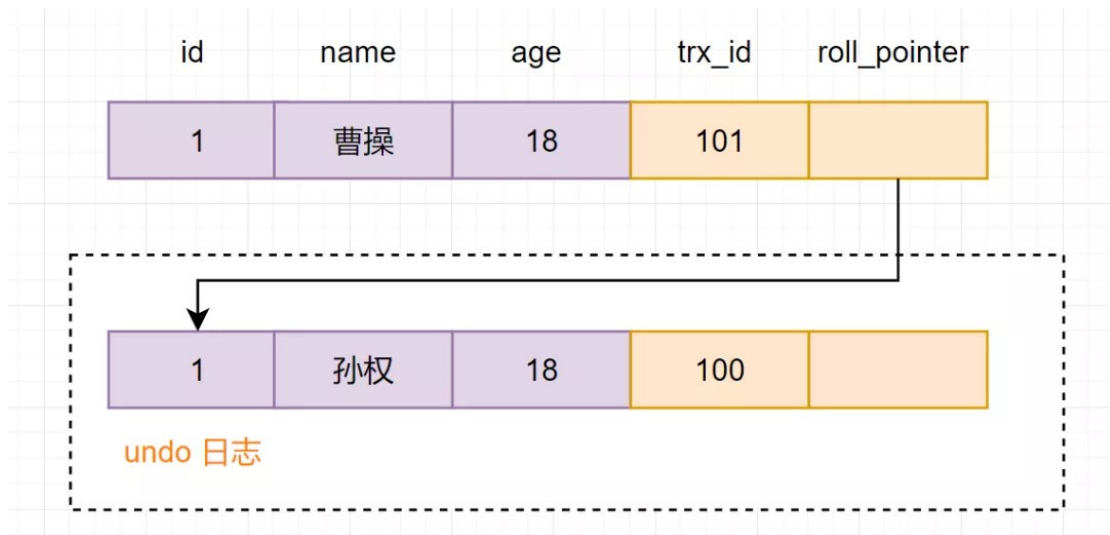
由图可以看出，最新版本的列 name 的内容是孙权，该版本的 trx_id 值为

100。开始执行 **read view 可见性规则** 校验：

```
min_limit_id(100) <= trx_id ( 100 ) < 102;
creator_trx_id = trx_id = 100;
```

由此可得，trx_id=100 的这个记录，当前事务是可见的。所以查到是 **name 为孙权的记录**。

1. 事务 B 进行修改操作，把名字改为曹操。把原数据拷贝到 undo log, 然后对数据进行修改，标记事务 ID 和上一个数据版本在 undo log 的地址。



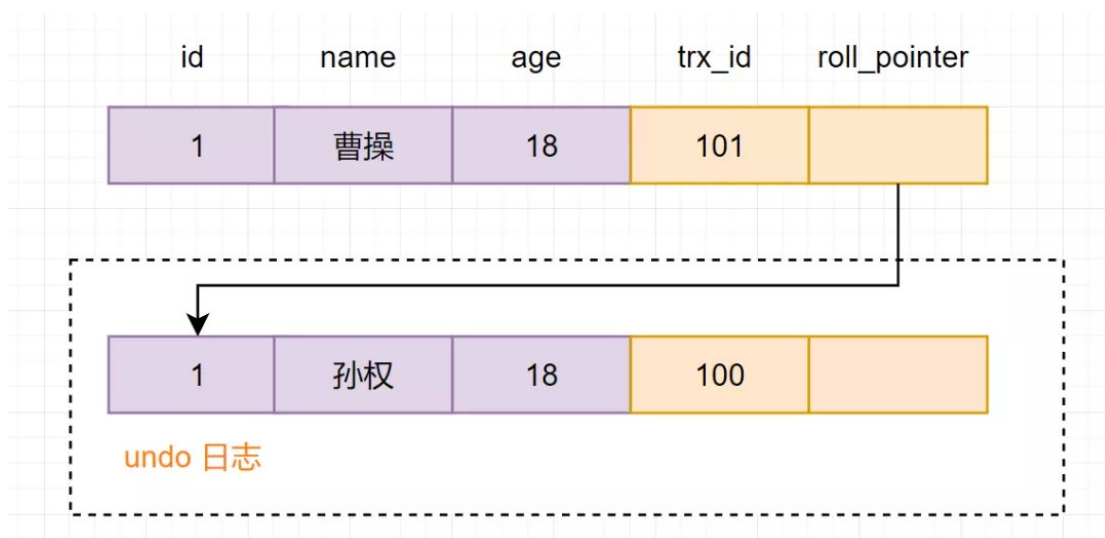
1. 事务 B 提交事务

2. 事务 A 再次执行查询操作，因为是 RR (可重复读) 隔离级别，因此会

复用老的 Read View 副本，Read View 对应的值如下

变量	值
m_ids	100 , 101
max_limit_id	102
min_limit_id	100
creator_trx_id	100

然后再次回到版本链：从版本链中挑选可见的记录：



从图可得，最新版本的列 name 的内容是曹操，该版本的 `trx_id` 值为 101。开始执行 read view 可见性规则校验：

```
min_limit_id(100)=<trx_id(101)<max_limit_id(102);  
因为 m_ids{100,101} 包含 trx_id(101),  
并且 creator_trx_id(100) 不等于 trx_id(101)
```

所以，`trx_id=101` 这个记录，对于当前事务是不可见的。这时候呢，版本链 `roll_pointer` 跳到下一个版本，`trx_id=100` 这个记录，再次校验是否可见：

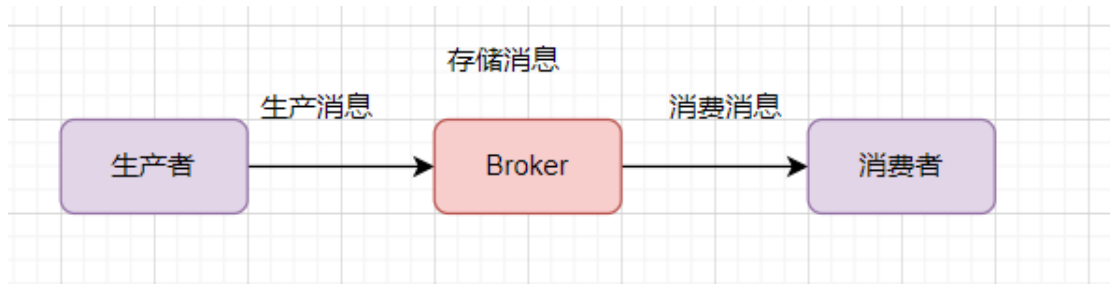
```
min_limit_id(100)=<trx_id(100)<max_limit_id(102);  
因为 m_ids{100,101} 包含 trx_id(100),  
并且 creator_trx_id(100) 等于 trx_id(100)
```

所以，`trx_id=100` 这个记录，对于当前事务是可见的，所以两次查询结果，都是 name=孙权的那个记录。即在可重复读（RR）隔离级别下，复用老的 Read View 副本，解决了不可重复读的问题。

大家可以回头多看几遍我这篇文章哈：[看一遍就理解：MVCC 原理详解](#)

8.10 你们项目使用了 RocketMQ 对吧？那你知道如何保证消息不丢失吗？

一个消息从生产者产生，到被消费者消费，主要经过这 3 个过程：



1. 生产者产生消息
2. 消息发送到存储端，保存下来
3. 消息推送到消费者，消费者消费完，ack 应答。

因此如何保证 MQ 不丢失消息，可以从这三个阶段阐述：

- 生产者保证不丢消息
- 存储端不丢消息
- 消费者不丢消息

8.10.1 生产者保证不丢消息

生产端如何保证不丢消息呢？确保生产的消息能顺利到达存储端。

如果是 **RocketMQ** 消息中间件的话，**Producer** 生产者提供了三种发送消息的方式，
分别是：

- 同步发送
- 异步发送
- 单向发送

生产者要想发消息时保证消息不丢失，可以：

- 采用同步方式发送，send 消息方法返回成功状态，即消息正常到达了存储端 **Broker**。
- 如果 **send** 消息异常或者返回非成功状态，可以发起重试。
- 可以使用事务消息，**RocketMQ** 的事务消息机制就是为了保证零丢失来设计的

8.10.2 存储端不丢消息

如何保证存储端的消息不丢失呢？确保消息持久化到磁盘，那就是刷盘机制嘛。

刷盘机制分**同步刷盘**和**异步刷盘**：

- 同步刷盘：生产者消息发过来时，只有持久化到磁盘，RocketMQ 的存储端 **Broker** 才返回一个成功的 ACK 响应。它保证消息不丢失，但是影响了性能。
- 异步刷盘：只要消息写入 **PageCache** 缓存，就返回一个成功的 ACK 响应。这样提高了 MQ 的性能，但是如果这时候机器断电了，就会丢失消息。

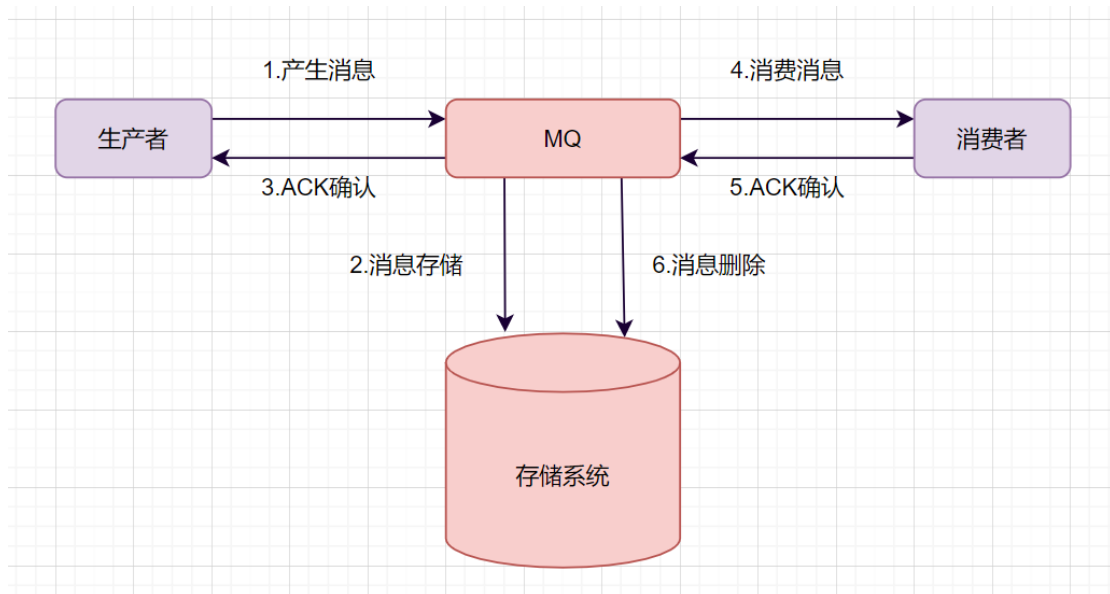
出了同步刷盘机制，还有一个维度需要考虑。**Broker** 一般是集群部署的，有主节点和从节点。消息到 **Broker** 存储端，只有主节点和从节点都写入成功，才反馈成功的 **ack** 给生产者。这就是**同步复制**，它保证了消息不丢失，但是降低了系统的吞吐量。与之对应即是**异步复制**，只要消息写入主节点成功，就返回成功的 **ack**，它速度快，但是会有性能问题。

8.10.3 消费阶段不丢消息

消费者执行完业务逻辑，再反馈会 **Broker** 说消费成功，这样才可以保证消费阶段不丢消息。

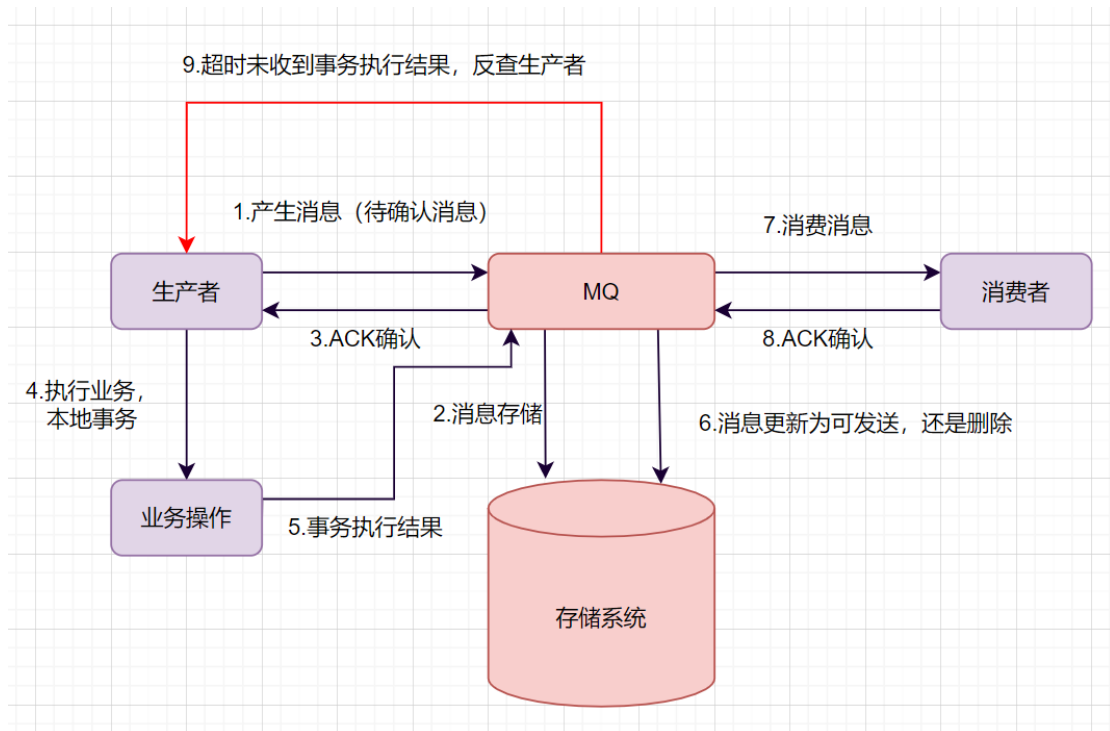
8.11 事务消息是否了解？场景题：比如下单清空购物车，你是如何设计的？

事务消息主要用来解决消息生产者和消息消费者的**数据一致性问题**。我们先来回忆一下：一条普通的消息队列消息，从产生到被消费，经历的流程：



1. 生产者产生消息，发送带 MQ 服务器
2. MQ 收到消息后，将消息持久化到存储系统。
3. MQ 服务器返回 ACK 到生产者。
4. MQ 服务器把消息 push 给消费者
5. 消费者消费完消息，响应 ACK
6. MQ 服务器收到 ACK，认为消息消费成功，即在存储中删除消息。

消息队列的事务消息流程是怎样的呢？



1. 生产者产生消息，发送一条半事务消息到 MQ 服务器
2. MQ 收到消息后，将消息持久化到存储系统，这条消息的状态是待发送状态。
3. MQ 服务器返回 ACK 确认到生产者，此时 MQ 不会触发消息推送事件
4. 生产者执行本地事务
5. 如果本地事务执行成功，即 commit 执行结果到 MQ 服务器；如果执行失败，发送 rollback。
6. 如果是正常的 commit，MQ 服务器更新消息状态为可发送；如果是 rollback，即删除消息。
7. 如果消息状态更新为可发送，则 MQ 服务器会 push 消息给消费者。消费者消费完就回 ACK。
8. 如果 MQ 服务器长时间没有收到生产者的 commit 或者 rollback，它会反查生产者，然后根据查询到的结果执行最终状态。

我们举个下**订单清空购物车**的例子吧。订单系统创建完订单后，然后发消息给下游系统购物车系统，清空购物车。

1. 生产者（订单系统）产生消息，发送一条半事务消息到 MQ 服务器
2. MQ 收到消息后，将消息持久化到存储系统，这条消息的状态是待发送状态。
3. MQ 服务器返回 ACK 确认到生产者，此时 MQ 不会触发消息推送事件
4. 生产者执行本地事务（订单创建成功，提交事务消息）
5. 如果本地事务执行成功，即 commit 执行结果到 MQ 服务器；如果执行失败，发送 rollback。
6. 如果是 commit 正常提交，MQ 服务器更新消息状态为**可发送**；如果是 rollback，即**删除消息**。
7. 如果消息状态更新为可发送，则 MQ 服务器会 push 消息给消费者（购物车系统）。消费者消费完（即拿到订单消息，清空购物车成功）就应答 ACK。
8. 如果 MQ 服务器长时间没有收到生产者的 commit 或者 rollback，它会反查生产者，然后根据查询到的结果（回滚操作或者重新发送消息）执行最终状态。

有些伙伴可能有疑惑，如果消费者消费失败怎么办呢？那数据是不是不一致啦？

所以需要消费者消费成功，执行业务逻辑成功，再反馈 ack 嘛。如果消费者消费失败，那就自动重试嘛，接口支持幂等即可。

8.12 如何快速判断一个数是奇数还是偶数，除开对 2 取余呢。

判断一个数是奇数还是偶数，我们最容易想到的就是对 2 取余。

```
if( x % 2 )  
// 奇数  
else  
// 偶数
```

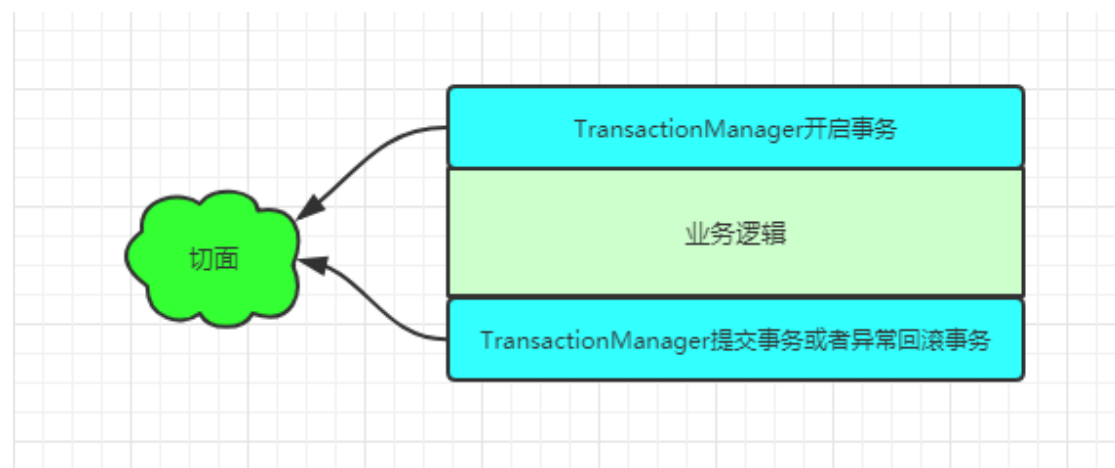
还有一种方法，就是与 1 相与 (&1)，具体实现如下：

```
if( x & 1 )  
// 奇数  
else  
// 偶数
```

8.13 Spring 声明式事务原理？哪些场景事务会失效？

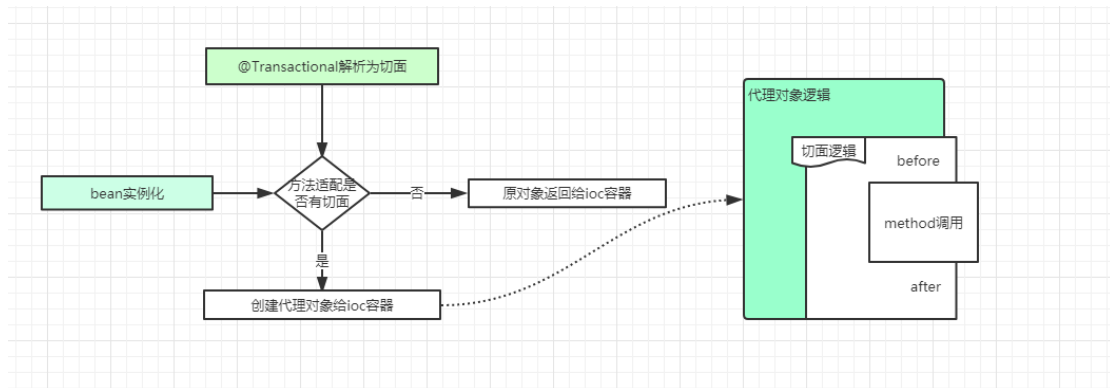
8.13.1 声明式事务原理

spring 声明式事务，即`@Transactional`，它可以帮助我们事务开启、提交或者回滚的操作，通过 Aop 的方式进行管理。



在 spring 的 bean 的初始化过程中，就需要对实例化的 bean 进行代理，并且生成代理对象。生成代理对象的代理逻辑中，进行方法调用时，需要先获取切

面逻辑，@Transactional 注解的切面逻辑类似于@Around，在 spring 中是实现一种类似代理逻辑。



详解可以看下这篇文章哈：[Spring 中的@Transactional 实现原理](#)

8.13.2 spring 声明式事务哪些场景会失效

- 方法的访问权限必须是 public，其他 private 等权限，事务失效
- 方法被定义成了 final 的，这样会导致事务失效。
- 在同一个类中的方法直接内部调用，会导致事务失效。
- 一个方法如果没交给 spring 管理，就不会生成 spring 事务。
- 多线程调用，两个方法不在同一个线程中，获取到的数据库连接不一样的。
- 表的存储引擎不支持事务
- 如果自己 try...catch 误吞了异常，事务失效。
- 错误的传播

详解大家可以看下这篇文章：[聊聊 spring 事务失效的 12 种场景，太坑了](#)

8.14 你们是微服务架构嘛？如果你来设计一个类似淘宝的系统，你怎么划分微服务？

可以按业务领域、功能、重要程度进行划分。

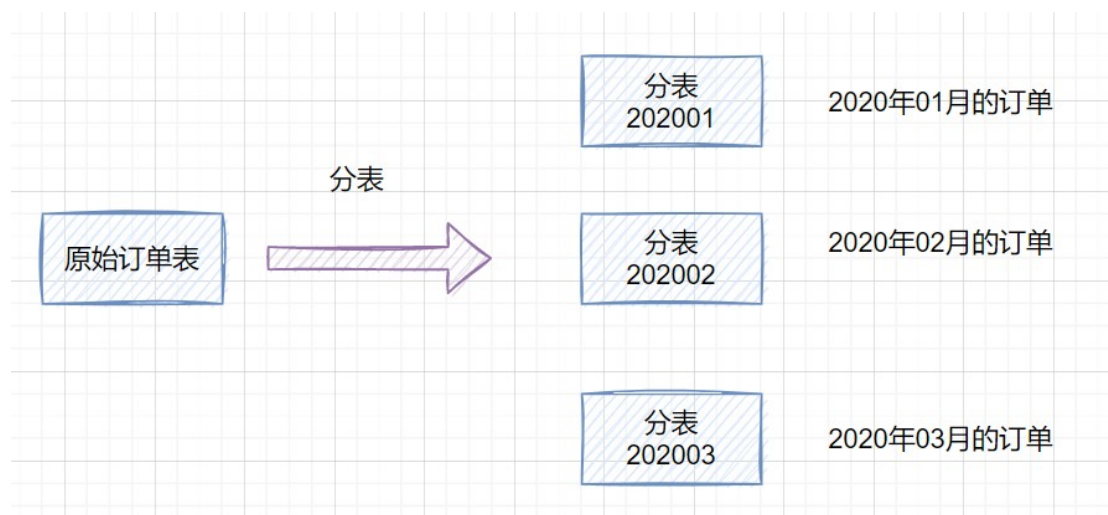
- 可以按业务领域，把用户、社区、商品信息、消息等模块等划分。
- 单一功能职责，按功能拆分，比如订单、支付、物流、权限。
- 按重要程度划分，区分核心和非核心功能，比如支付、订单就是核心功能。

8.15 你们是怎么分库分表的？分布式 ID 如何生成？

如果是我们公司的话，使用了水平分库的方式，就是一个用户注册时，就划分了属于哪个数据库，然后具体的表结构是一样的。

业界还有垂直分库，就是按照不同的系统中的不同业务进行拆分，比如拆分成用户库、订单库、积分库、商品库，把它们部署在不同的数据库服务器。

分表的话也有水平分表和垂直分表，垂直分表就是将一些不常用的、数据较大或者长度较长的列拆分到另外一张表，水平分表就是可以按照某种规则（如 hash 取模、range），把数据切分到多张表去。一张订单表，按时间 range 拆分如下：



range 划分利于数据迁移，但是存在数据热点问题。**hash** 取模划分，不会存在明显的热点问题，但是不利于扩容。可以 **range+hash** 取模结合使用。

分布式 ID 可以使用雪花算法生成

雪花算法是一种生成分布式全局唯一 ID 的算法，生成的 ID 称为 Snowflake IDs。这种算法由 Twitter 创建，并用于推文的 ID。

一个 Snowflake ID 有 64 位。

- [illegible]

```
graph LR; Throwable[Throwable] --- Error[Error]; Throwable --- Exception[Exception]; Error --- VirtualMachineError[Virtual MachineError]; Error --- AWTError[AWTError]; VirtualMachineError --- StackOverflowError[StackOverflowError]; VirtualMachineError --- OutOfMemoryError[OutOfMemoryError]; Exception --- IOException[IOException]; Exception --- RuntimeException[RuntimeException]; IOException --- EOFException[EOFException]; IOException --- FileNotFoundException[FileNotFoundException]; RuntimeException --- ArithmeticException[ArithmeticException]; RuntimeException --- ClassNotFoundException[ClassNotFoundException]; RuntimeException --- NullPointerException[NullPointerException]; RuntimeException --- IllegalArgumentException[IllegalArgumentException]; RuntimeException --- ArrayIndexOutOfBoundsException[ArrayIndexOutOfBoundsException];
```

The diagram illustrates the hierarchy of Java exceptions. It starts with 'Throwable' at the root, which branches into 'Error' and 'Exception'. 'Error' further branches into 'Virtual MachineError' and 'AWTError'. 'Virtual MachineError' branches into 'StackOverflowError' and 'OutOfMemoryError'. 'Exception' branches into 'IOException' and 'RuntimeException'. 'IOException' branches into 'EOFException' and 'FileNotFoundException'. 'RuntimeException' branches into 'ArithmeticException', 'ClassNotFoundException', 'NullPointerException', 'IllegalArgumentException', and 'ArrayIndexOutOfBoundsException'.

- **Error**：是程序无法处理的错误，一般表示系统错误，例如虚拟机相关的错误
OutOfMemoryError

- **Exception** : 程序本身可以处理的异常。它可以分为 RuntimeException (运行时异常) 和 CheckedException (可检查的异常) 。

什么是 RuntimeException (运行时异常) ?

运行时异常是不检查异常，程序中可以选择捕获处理，也可以不处理。这些异常一般是由程序逻辑错误引起的，程序应该从逻辑角度尽可能避免这类异常的发生。

常见的 **RuntimeException** 异常：

- NullPointerException : 空指针异常
- ArithmeticException : 出现异常的运算条件时，抛出此异常
- IndexOutOfBoundsException : 数组索引越界异常
- ClassNotFoundException : 找不到类异常
- IllegalArgumentException(非法参数异常)

什么是 CheckedException (可检查的异常) ?

从程序语法角度讲是必须进行处理的异常，如果不处理，程序就不能编译通过。如 IOException、SQLException 等。

常见的 **Checked Exception** 异常：

- IOException : (操作输入流和输出流时可能出现的异常)
- SQLException

9.小厂后端十连问 (关注公众号：捡田螺的小男孩。)

大家好，我是捡田螺的小男孩。最近一位朋友去面试，分享了一份面试真题，我整理了一下答案给大家。如果有不正确的，欢迎指出哈，一起进步。更多干货，关注公众号：捡田螺的小男孩。



9.1 Redis 的 key 和 value 可以存储的最大值分别是多少？

- 虽然 Key 的大小上限为 512M,但是一般建议 key 的大小不要超过 1KB，这样既可以节约存储空间，又有利于 Redis 进行检索。
- value 的最大值也是 512M。对于 String 类型的 value 值上限为 512M，而集合、链表、哈希等 key 类型，单个元素的 value 上限也为 512M。

9.2 怎么利用 Redis 实现数据的去重？

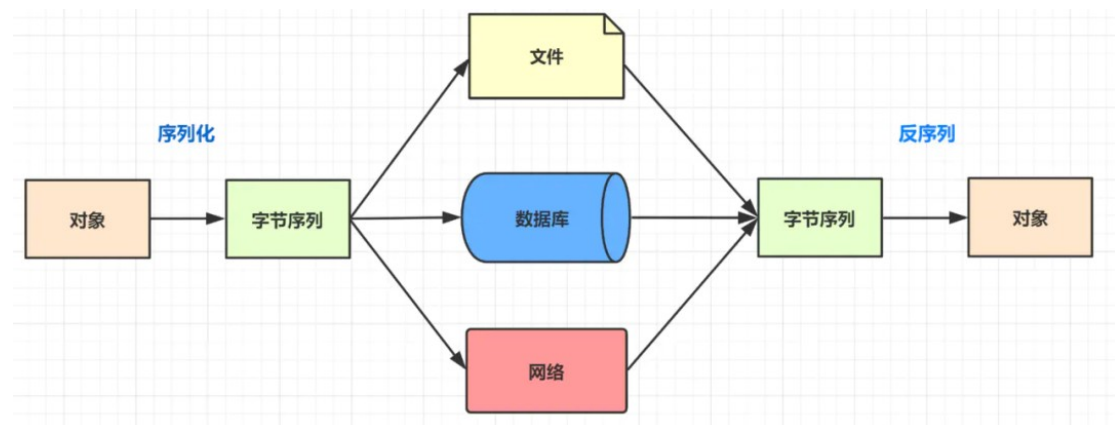
- Redis 的 set：它可以去除重复元素，也可以快速判断某一个元素是否存在于集合中，如果元素很多（比如上亿的计数），消占用内存很大
- Redis 的 bit：它可以用来实现比 set 内存高度压缩的计数，它通过一个 bit 设置为 1 或者 0，表示存储某个元素是否存在信息。例如网站唯一访客计数，可以把 user_id 作为 bit 的偏移量 offset，如设置为 1 表示有访问，使用 1 MB 的空间就可以存放 800 多万用户的一天访问计数情况。
- HyperLogLog：实现超大数据量精确的唯一计数都是比较困难的，HyperLogLog 可以仅仅使用 12 k 左右的内存，实现上亿的唯一计数，而且误差控制在百分之一左右。
- bloomfilter 布隆过滤器：布隆过滤器是一种占用空间很小的数据结构，它由一个很长的二进制向量和一组 Hash 映射函数组成，它用于检索一个元素是否在一个集合中

对于布隆过滤器，大家有兴趣可以看我这篇文章哈，[面试必备：布隆过滤器是什么？有什么用？](#)

9.3 Redis 什么时候需要序列化？Redis 序列化的方式有哪些？

大家先回忆下 Java 序列化，什么时候需要序列化？

- 序列化：将 Java 对象转换成字节流的过程。
- 反序列化：将字节流转换成 Java 对象的过程。



为什么需要序列化呢？

打个比喻：作为大城市漂泊的码农，搬家是常态。当我们搬书桌时，桌子太大了就通不过比较小的门，因此我们需要把它拆开再搬过去，这个拆桌子的过程就是序列化。而我们把书桌复原回来（安装）的过程就是反序列化啦。

- 比如想把内存中的对象状态保存到一个文件中或者数据库中的时候（最常用，如保存到 redis）；
- 再比喻想用套接字在网络上传送对象的时候，都需要序列化。

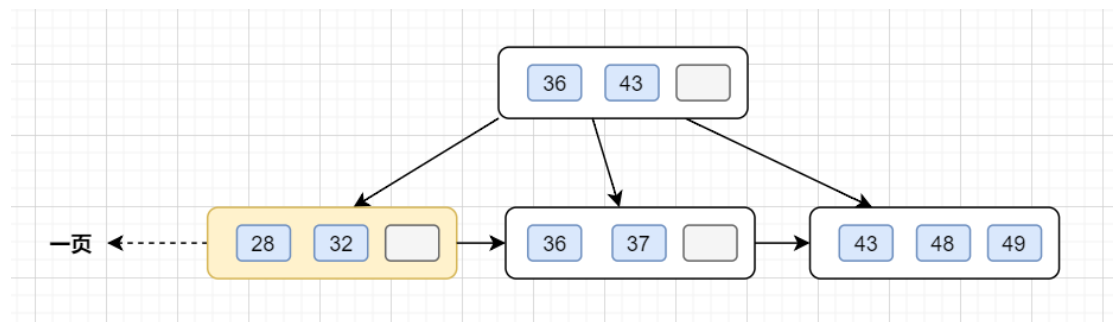
RedisSerializer 接口 是 Redis 序列化接口，用于 Redis KEY 和 VALUE 的序列化

- JDK 序列化方式（默认）
- String 序列化方式
- JSON 序列化方式
- XML 序列化方式

9.4 MySQL 的 B+树的高度怎么计算？（比如有 100w 的数据，字段为 int 类型）

InnoDB 存储引擎最小储存单元是页，一页大小就是 16k。

B+树叶子存的是数据，内部节点存的是键值+指针。索引组织表通过非叶子节点的二分查找法以及指针确定数据在哪个页中，进而再去数据页中找到需要的数据；



假设 B+树的高度为 2 的话，即有一个根结点和若干个叶子结点。这棵 B+树的存放总记录数为 = 根结点指针数 * 单个叶子节点记录行数。

- 如果一行记录的数据大小为 1k，那么单个叶子节点可以存的记录数 = $16k/1k = 16$ 。
- 非叶子节点内存放多少指针呢？我们假设主键 ID 为 **bigint** 类型，长度为 8 字节（面试官问你 **int** 类型，一个 **int** 就是 32 位，4 字节），而指针大小在 InnoDB 源码中设置为 6 字节，所以就是 $8+6=14$ 字节， $16k/14B = 16*1024B/14B = 1170$

因此，一棵高度为 2 的 B+树，能存放 $1170 * 16 = 18720$ 条这样的数据记录。

同理一棵高度为 3 的 B+树，能存放 $1170 * 1170 * 16 = 21902400$ ，也就是

说，可以存放两千万左右的记录。B+树高度一般为 1-3 层，已经满足千万级别的数据存储。

9.5 线程池的状态有哪些？获取多线程并发执行结果的方式有哪些？

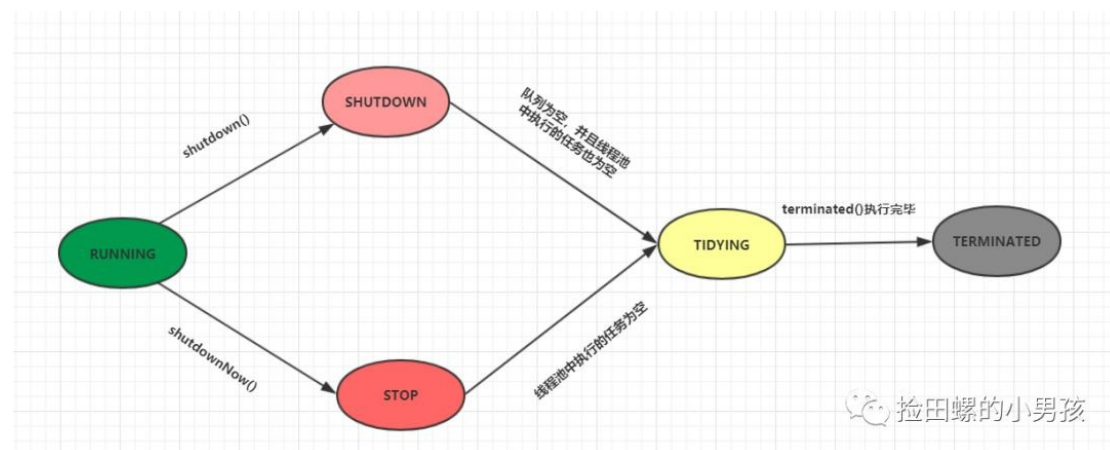
线程池和线程的状态是不一样的哈，线程池有这几个状态：

RUNNING, SHUTDOWN, STOP, TIDYING, TERMINATED。

//线程池状态

```
private static final int RUNNING    = -1 << COUNT_BITS;  
private static final int SHUTDOWN   =  0 << COUNT_BITS;  
private static final int STOP       =  1 << COUNT_BITS;  
private static final int TIDYING    =  2 << COUNT_BITS;  
private static final int TERMINATED =  3 << COUNT_BITS;
```

线程池各个状态切换状态图如下：



RUNNING

- 该状态的线程池会接收新任务，并处理阻塞队列中的任务；
- 调用线程池的 `shutdown()` 方法，可以切换到 SHUTDOWN 状态；
- 调用线程池的 `shutdownNow()` 方法，可以切换到 STOP 状态；

SHUTDOWN

- 该状态的线程池不会接收新任务，但会处理阻塞队列中的任务；
- 队列为空，并且线程池中执行的任务也为空,进入 TIDYING 状态；

STOP

- 该状态的线程不会接收新任务，也不会处理阻塞队列中的任务，而且会中断正在运行的任务；
- 线程池中执行的任务为空,进入 TIDYING 状态;

TIDYING

- 该状态表明所有的任务已经运行终止，记录的任务数量为 0。
- `terminated()` 执行完毕，进入 TERMINATED 状态

TERMINATED

- 该状态表示线程池彻底终止

9.6 线程池原理？各个参数的作用。

ThreadPoolExecutor 的构造函数：

```
public ThreadPoolExecutor(int corePoolSize, int maximumPoolSize, long keepAliveTime,
    TimeUnit unit,
    BlockingQueue<Runnable> workQueue,
    ThreadFactory threadFactory,
    RejectedExecutionHandler handler)
```

几个核心参数的作用：

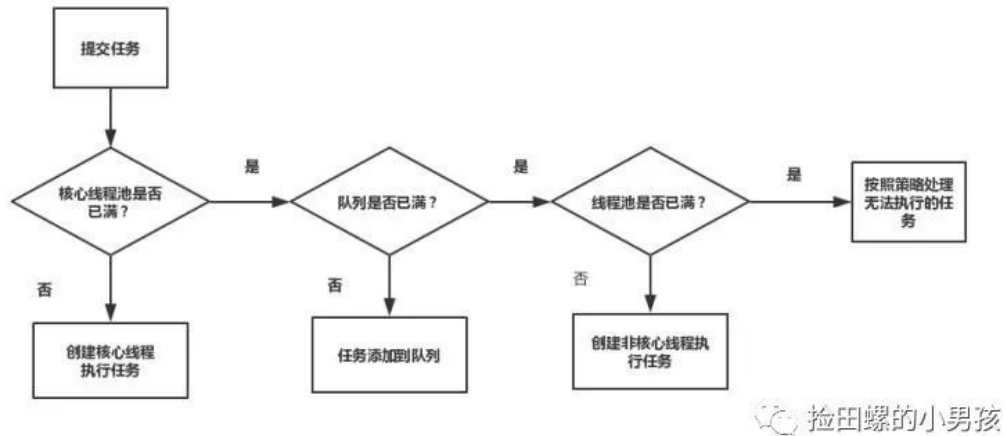
- `corePoolSize`：线程池核心线程数最大值
- `maximumPoolSize`：线程池最大线程数大小
- `keepAliveTime`：线程池中非核心线程空闲的存活时间大小
- `unit`：线程空闲存活时间单位
- `workQueue`：存放任务的阻塞队列
- `threadFactory`：用于设置创建线程的工厂，可以给创建的线程设置有意义的名字，可方便排查问题。
- `handler`：线程池的饱和策略事件，主要有四种类型。

四种饱和拒绝策略

- `AbortPolicy`(抛出一个异常，默认的)
- `DiscardPolicy`(直接丢弃任务)
- `DiscardOldestPolicy` (丢弃队列里最老的任务，将当前这个任务继续提交给线程池)

- CallerRunsPolicy (交给线程池调用所在的线程进行处理)

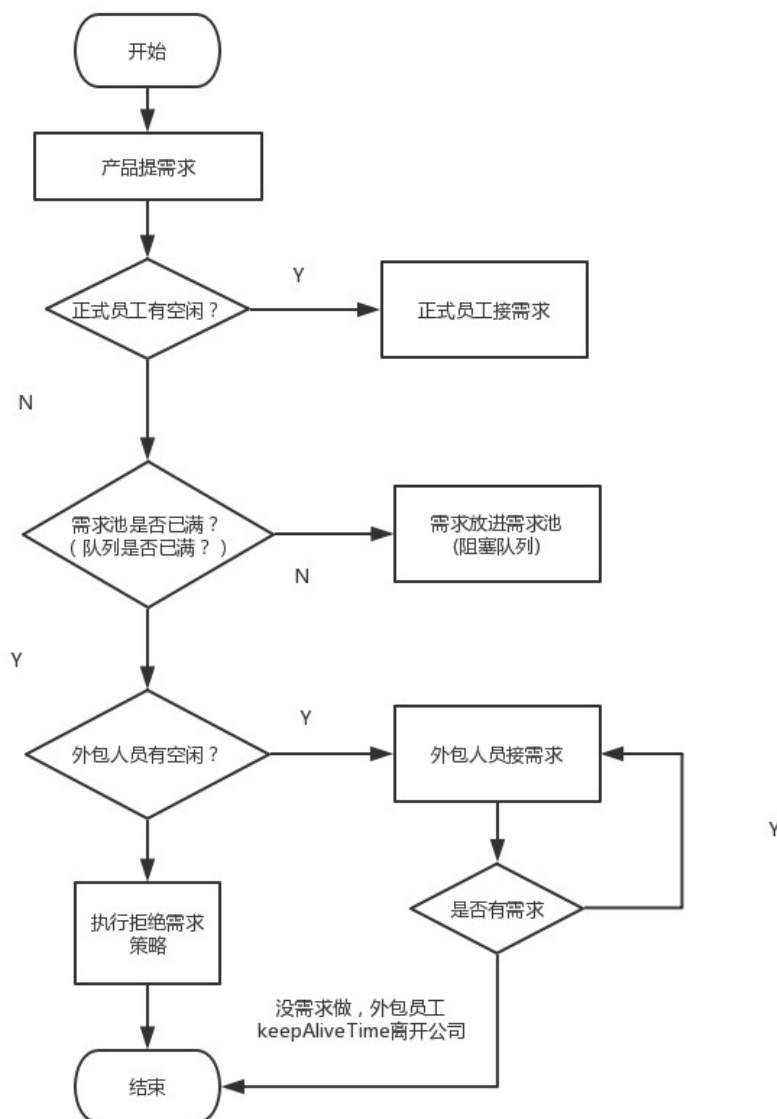
线程池原理：



- 提交一个任务，线程池里存活的核心线程数小于线程数 `corePoolSize` 时，线程池会创建一个核心线程去处理提交的任务。
- 如果线程池核心线程数已满，即线程数已经等于 `corePoolSize`，一个新提交的任务，会被放进任务队列 `workQueue` 排队等待执行。
- 当线程池里面存活的线程数已经等于 `corePoolSize` 了,并且任务队列 `workQueue` 也满，判断线程数是否达到 `maximumPoolSize`，即最大线程数是否已满，如果没到达，创建一个非核心线程执行提交的任务。
- 如果当前的线程数达到了 `maximumPoolSize`，还有新的任务过来的话，直接采用拒绝策略处理。

为了形象描述线程池执行，我打个比喻：

- 核心线程比作公司正式员工
- 非核心线程比作外包员工
- 阻塞队列比作需求池
- 提交任务比作提需求



捡田螺的小男孩

- 当产品提个需求，正式员工（核心线程）先接需求（执行任务）
- 如果正式员工都有需求在做，即核心线程数已满），产品就把需求先放需求池（阻塞队列）。
- 如果需求池(阻塞队列)也满了，但是这时候产品继续提需求,怎么办呢？那就请外包（非核心线程）来做。
- 如果所有员工（最大线程数也满了）都有需求在做了，那就执行拒绝策略。
- 如果外包员工把需求做完了，它经过一段（keepAliveTime）空闲时间，就离开公司了。

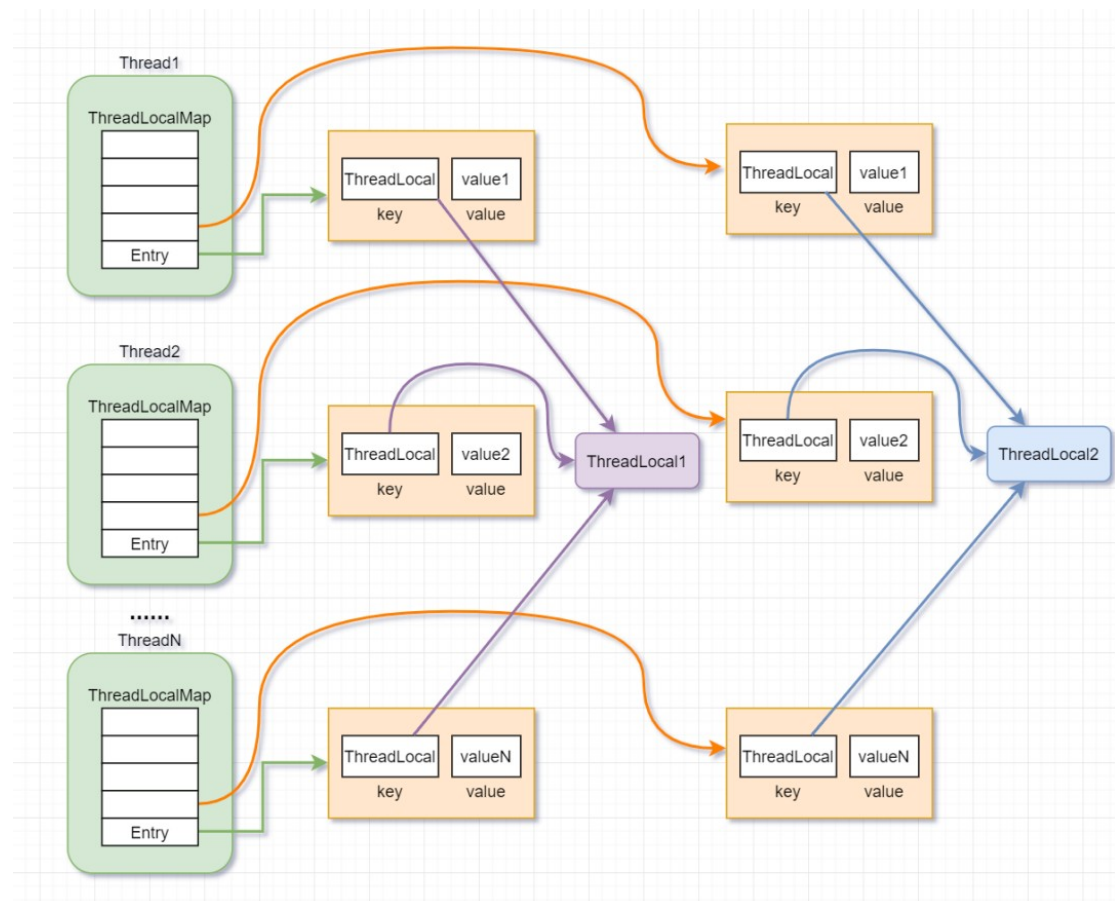
9.7 ThreadLocal 的使用场景有哪些？原理？内存泄漏？

ThreadLocal，即线程本地变量。如果你创建了一个 ThreadLocal 变量，那么访问这个变量的每个线程都会有这个变量的一个本地拷贝，多个线程操作这个变量的时候，实际是操作自己本地内存里面的变量，从而起到线程隔离的作用，避免了线程安全问题。

ThreadLocal 的应用场景

- 数据库连接池
- 会话管理中使用

ThreadLocal 内存结构图：



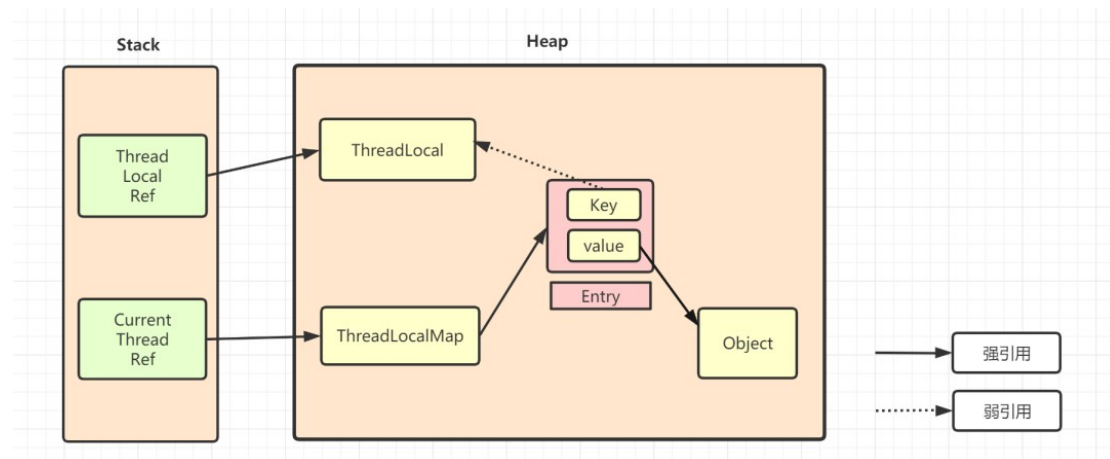
ThreadLocal 原理

- Thread 对象中持有一个 ThreadLocal.ThreadLocalMap 的成员变量。
- ThreadLocalMap 内部维护了 Entry 数组，每个 Entry 代表一个完整的对象，key 是 ThreadLocal 本身，value 是 ThreadLocal 的泛型值。

- 每个线程在往 ThreadLocal 里设置值的时候，都是往自己的 ThreadLocalMap 里存，读也是以某个 ThreadLocal 作为引用，在自己的 map 里找对应的 key，从而实现了线程隔离。

ThreadLocal 内存泄露问题

先看看一下的 ThreadLocal 的引用示意图哈，



ThreadLocalMap 中使用的 key 为 ThreadLocal 的弱引用，如下

```
static class ThreadLocalMap {
    static class Entry extends WeakReference<ThreadLocal<?>> {
        /** The value associated with this ThreadLocal. */
        Object value;

        Entry(ThreadLocal<?> k, Object v) {
            super(k);
            value = v;
        }
    }
}
```

弱引用：只要垃圾回收机制一运行，不管 JVM 的内存空间是否充足，都会回收该对象占用的内存。

弱引用比较容易被回收。因此，如果 ThreadLocal (ThreadLocalMap 的 Key) 被垃圾回收器回收了，但是因为 ThreadLocalMap 生命周期和 Thread

是一样的，它这时候如果不被回收，就会出现这种情况：ThreadLocalMap 的 key 没了，value 还在，这就会造成了内存泄漏问题。

如何解决内存泄漏问题？使用完 ThreadLocal 后，及时调用 remove() 方法释放内存空间。

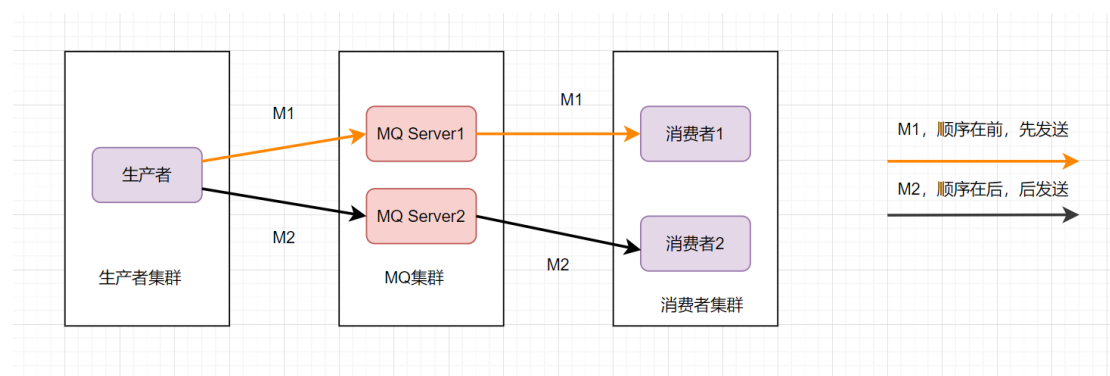
9.8 kafka 是如何保证消息的有序性？

kafka 这样保证消息有序性的：

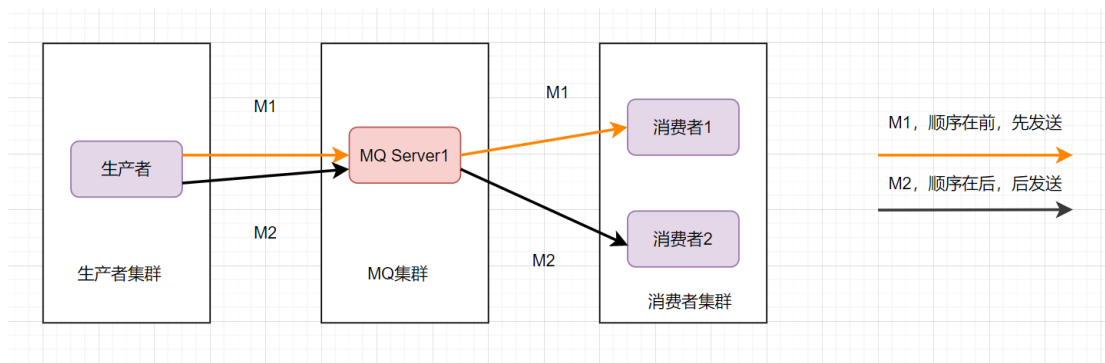
- 一个 topic，一个 partition，一个 consumer，内部单线程消费，单线程吞吐量太低，一般不用这个。（全局有序性）
- 写 N 个内存 queue，具有相同 key 的数据都到同一个内存 queue；然后对于 N 个线程，每个线程分别消费一个内存 queue 即可，这样就能保证顺序性。

大家可以看下消息队列的有序性是怎么推导的哈：

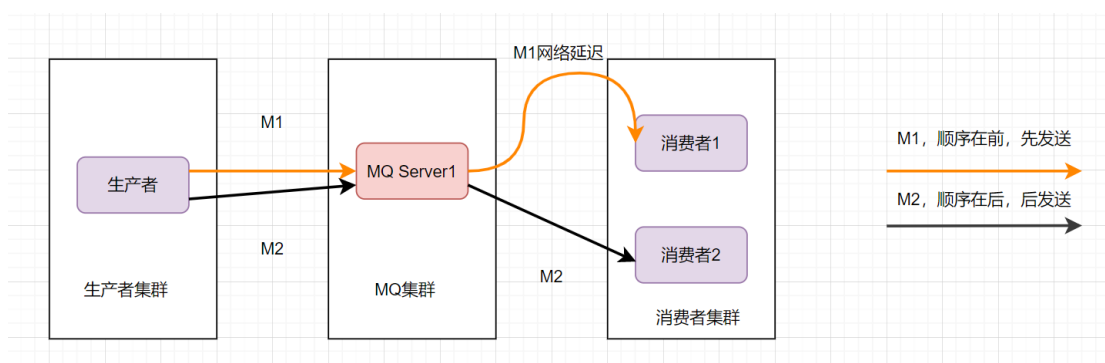
消息的有序性，就是指可以按照消息的发送顺序来消费。有些业务对消息的顺序是有要求的，比如先下单再付款，最后再完成订单，这样等。假设生产者先后产生了两条消息，分别是下单消息（M1），付款消息（M2），M1 比 M2 先产生，如何保证 M1 比 M2 先被消费呢。



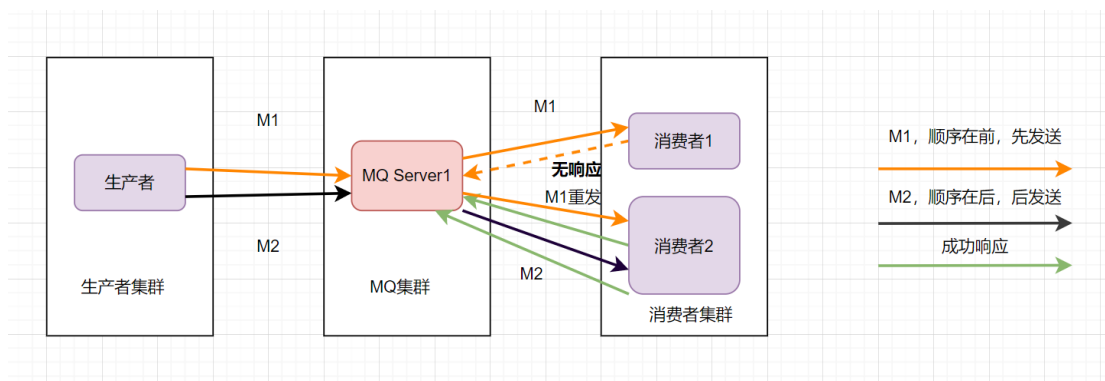
为了保证消息的顺序性，可以将 M1、M2 发送到同一个 Server 上，当 M1 发送完收到 ack 后，M2 再发送。如图：



这样还是可能会有问题，因为从 MQ 服务器到服务端，可能存在网络延迟，虽然 M1 先发送，但是它比 **M2** 晚到。



那还能怎么办才能保证消息的顺序性呢？将 M1 和 M2 发往**同一个消费者**，且发送 M1 后，等到消费端 ACK 成功后，才发送 M2 就得了。



消息队列保证顺序性**整体思路**就是这样啦。比如 Kafka 的全局有序消息，就是这种思想的体现：就是生产者发消息时，1 个 Topic 只能对应 1 个 Partition，一个 Consumer，内部单线程消费。

但是这样吞吐量太低，一般保证消息局部有序即可。在发消息的时候指定 **Partition Key**，Kafka 对其进行 Hash 计算，根据计算结果决定放入哪个 **Partition**。这样 Partition Key 相同的消息会放在同一个 Partition。然后多消费者单线程消费指定的 Partition。

9.9 Nacos 的选举机制了解嘛？说下 Raft 算法？

Nacos 作为配置中心的功能是基于 **Raft 算法**来实现的。

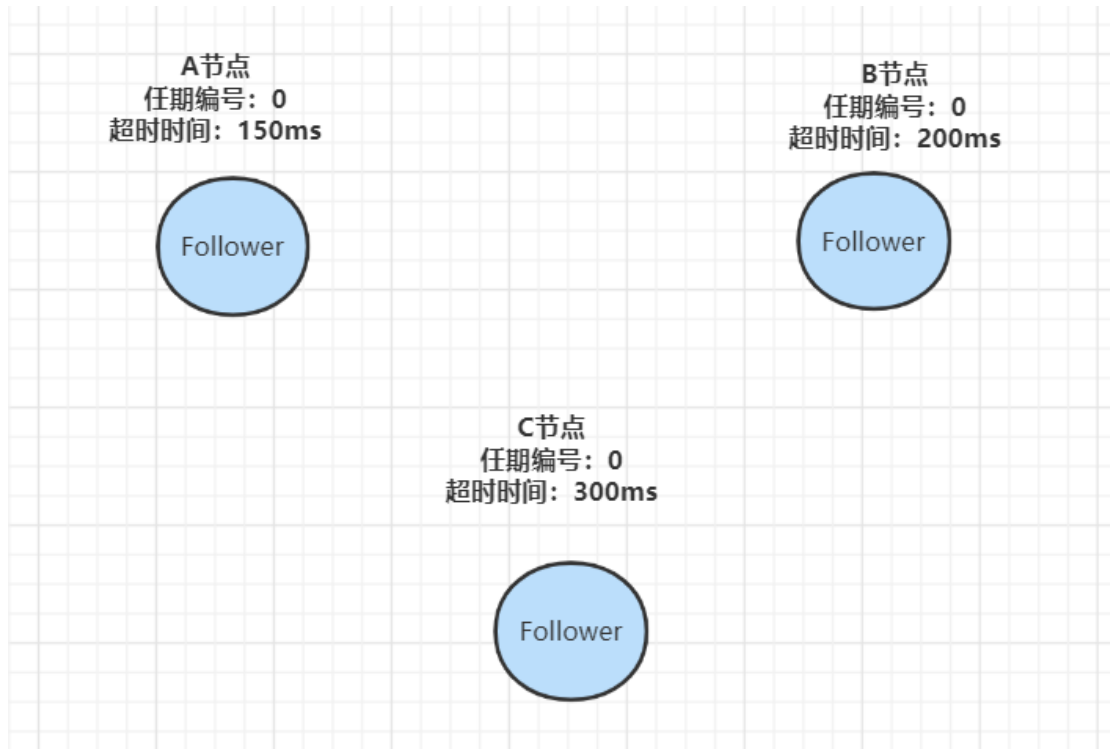
Raft 算法是分布式系统开发首选的共识算法，它通过“一切以领导者为准”的方式，实现一系列值的共识和各节点日志的一致。

Raft 选举规程 涉及三种角色和任期 (Term)，

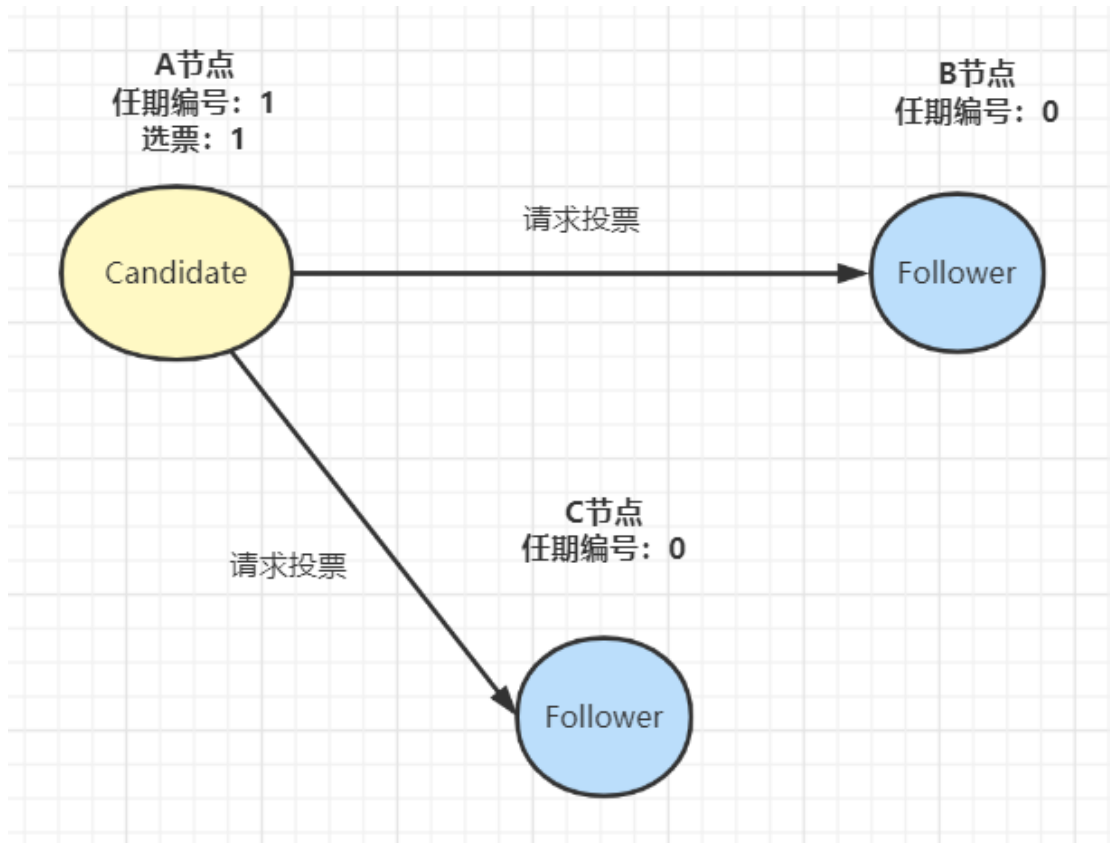
- Follower：默默地接收和处理来自 Leader 的消息，当等待 Leader 心跳信息超时的时候，就主动站出来，推荐自己当 Candidate。
- Candidate：向其他节点发送投票请求，通知其他节点来投票，如果赢得了大多数 ($N/2+1$) 选票，就晋升 Leader。
- Leader：负责处理客户端请求，进行日志复制等操作，每一轮选举的目标就是选出一个领导者；领导者会不断地发送心跳信息，通知其他节点“我是领导者，我还活着，你们不要发起新的选举，不用找个新领导者来替代我。”
- Term：这跟民主社会的选举很像，每一届新的履职期称之为 一届任期

领导选举过程

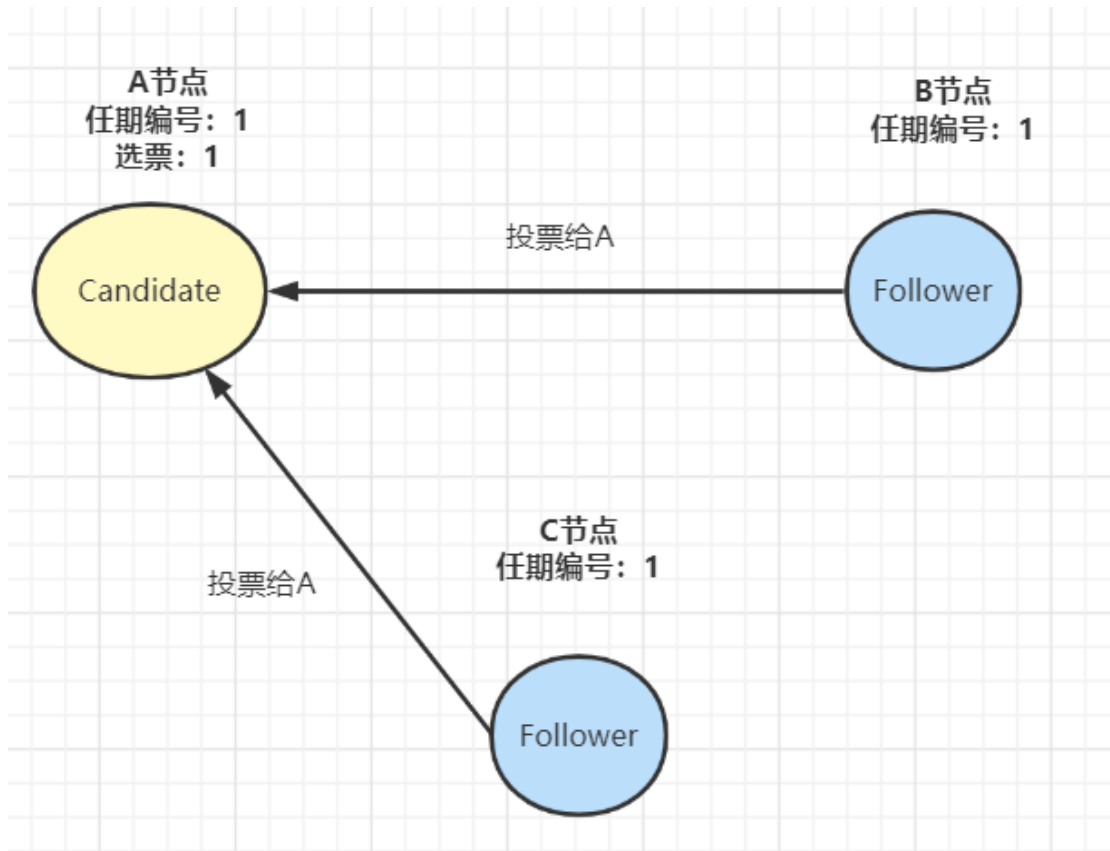
1. 在初始时，集群中所有的节点都是 Follower 状态，都被设定一个随机选举超时时间 (一般 150ms-300ms)：



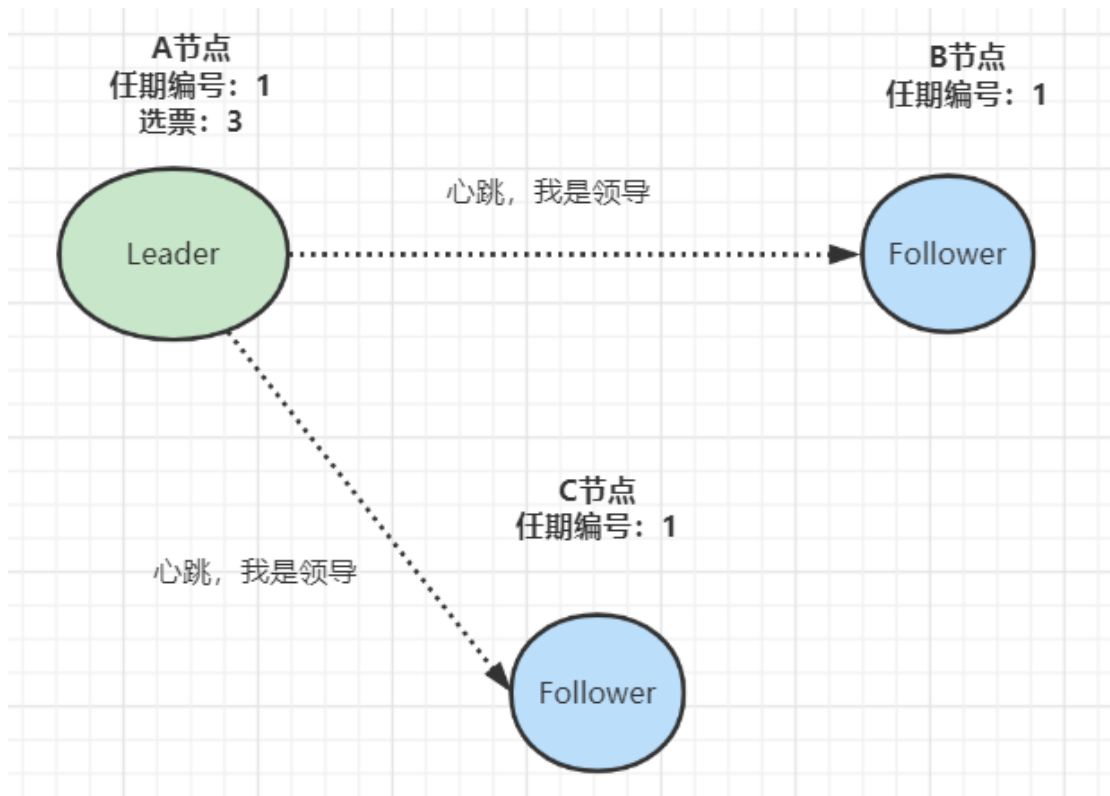
1. 如果 Follower 在规定的超时时间，都没有收到来自 Leader 的心跳，它就发起选举：将自己的状态切为 Candidate，增加自己的任期编号，然后向集群中的其它 Follower 节点发送请求，询问其是否选举自己成为 Leader：



1. 其他节点收到候选人 A 的请求投票消息后，如果在编号为 1 的这届任期内还没有进行过投票，那么它将把选票投给节点 A，并增加自己的任期编号：



1. 当收到来自集群中过半节点的接受投票后，A 节点即成为本届任期内 Leader，他将周期性地发送心跳消息，通知其他节点我是 Leader，阻止 Follower 发起新的选举：



9.10 聊一聊 TCC 补偿机制

TCC 是分布式事务的一种解决方案。它采用了补偿机制，其核心思想是：针对每个操作，都要注册一个与其对应的确认和补偿（撤销）操作。TCC (Try-Confirm-Cancel) 包括三段流程：

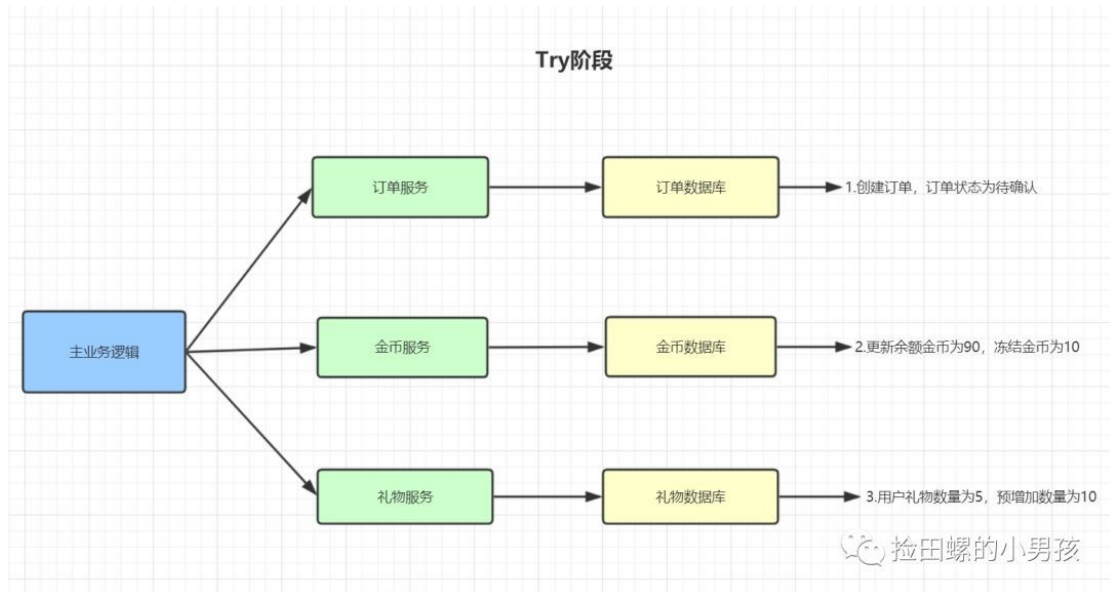
- try 阶段：尝试去执行，完成所有业务的一致性检查，预留必须的业务资源。
- Confirm 阶段：该阶段对业务进行确认提交，不做任何检查，因为 try 阶段已经检查过了，默认 Confirm 阶段是不会出错的。
- Cancel 阶段：若业务执行失败，则进入该阶段，它会释放 try 阶段占用的所有业务资源，并回滚 Confirm 阶段执行的所有操作。

下面再拿用户下单购买礼物作为例子来模拟 TCC 实现分布式事务的过程：

假设用户 A 余额为 100 金币，拥有的礼物为 5 朵。A 花了 10 个金币，下订单，购买 10 朵玫瑰。余额、订单、礼物都在不同数据库。

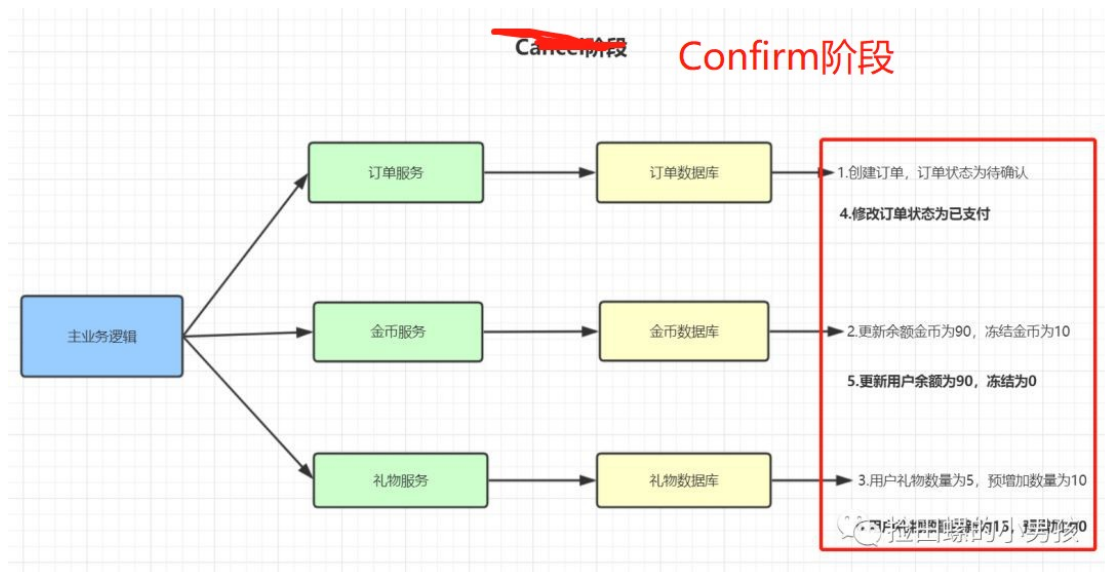
TCC 的 Try 阶段：

- 生成一条订单记录，订单状态为待确认。
- 将用户 A 的账户金币中余额更新为 90，冻结金币为 10（预留业务资源）
- 将用户的礼物数量为 5，预增加数量为 10。
- Try 成功之后，便进入 Confirm 阶段
- Try 过程发生任何异常，均进入 Cancel 阶段



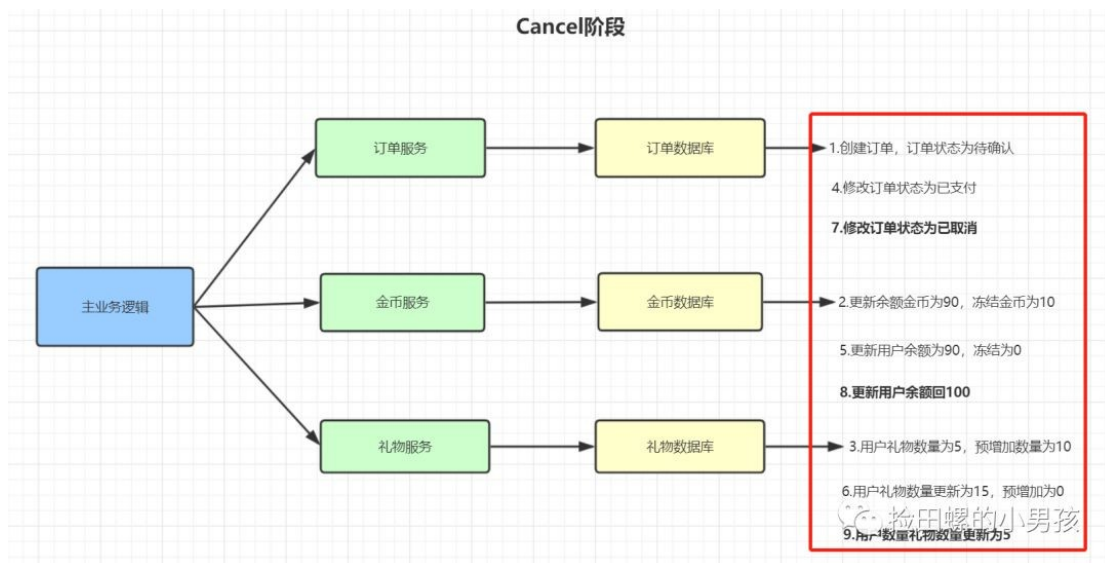
TCC 的 Confirm 阶段：

- 订单状态更新为已支付
- 更新用户余额为 90，可冻结为 0
- 用户礼物数量更新为 15，预增加为 0
- Confirm 过程发生任何异常，均进入 Cancel 阶段
- Confirm 过程执行成功，则该事务结束



TCC 的 Cancel 阶段：

- 修改订单状态为已取消
- 更新用户余额回 100
- 更新用户礼物数量为 5



- TCC 的优点是可以自定义数据库操作的粒度，降低了锁冲突，可以提升性能
- TCC 的缺点是应用侵入性强，需要根据网络、系统故障等不同失败原因实现不同的回滚策略，实现难度大，一般借助 TCC 开源框架，ByteTCC，TCC-transaction，Himly。

10.腾讯云 15 连问（关注公众号：捡田螺的小男孩。）

大家好，我是捡田螺的小男孩。最近一位朋友去面试腾讯云，分享了一份面试真题，我整理了一下答案给大家。如果有不正确的，欢迎指出哈，一起进步。

更多干货，关注公众号：捡田螺的小男孩。



10.1 聊聊项目，好的设计，好的代码

项目的话，你可以聊聊你平时做的项目，尤其有亮点的项目。如果没有什么特别亮点的项目，也可以说说一些好的设计，或者你优化了什么接口，性能提升了多少，优化了什么慢 SQL 都可以。甚至是一些好的代码写法都可以。

如果是讲优化接口的话，你可以看下我这篇文章哈：

[记一次接口性能优化实践总结：优化接口性能的八个建议](#)

如果是代码优化细节，可以看我这篇：

[工作四年，分享 50 个让你代码更好的小建议](#)

如果是慢 SQL 优化，可以看下我之前 MySQL 专栏系列文章哈：

- [看一遍就理解：order by 详解](#)
- [看一遍就理解：group by 详解](#)
- [实战！聊聊如何解决 MySQL 深分页问题](#)

- 后端程序员必备：书写高质量 SQL 的 30 条建议
- 阿里一面，给了几条 SQL，问需要执行几次树搜索操作？
- 生产问题分析！delete in 子查询不走索引？！

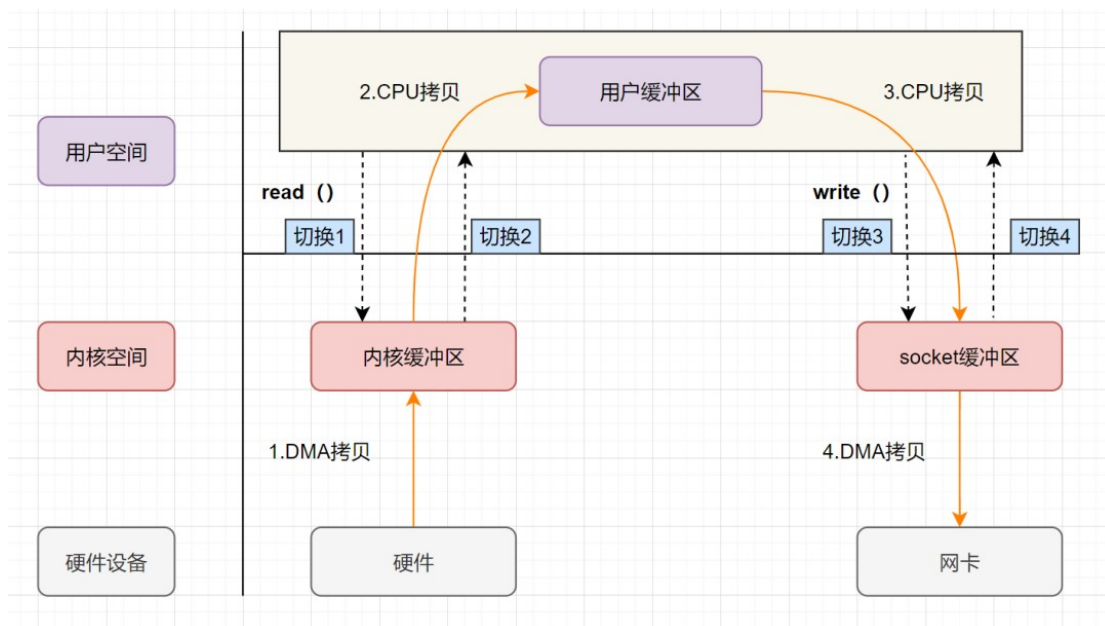
10.2 谈谈什么是零拷贝？

零拷贝是指计算机执行 IO 操作时，CPU 不需要将数据从一个存储区域复制到另一个存储区域，从而可以减少上下文切换以及 CPU 的拷贝时间。它是一种 I/O 操作优化技术。

传统 IO 的执行流程

传统的 IO 流程，包括 read 和 write 的过程。

- read：把数据从磁盘读取到内核缓冲区，再拷贝到用户缓冲区
- write：先把数据写入到 socket 缓冲区，最后写入网卡设备。



1. 用户应用进程调用 read 函数，向操作系统发起 IO 调用，上下文从用户态转为内核态（切换 1）
2. DMA 控制器把数据从磁盘中，读取到内核缓冲区。
3. CPU 把内核缓冲区数据，拷贝到用户应用缓冲区，上下文从内核态转为用户态（切换 2），read 函数返回
4. 用户应用进程通过 write 函数，发起 IO 调用，上下文从用户态转为内核态（切换 3）

5. CPU 将用户缓冲区中的数据，拷贝到 socket 缓冲区
6. DMA 控制器把数据从 socket 缓冲区，拷贝到网卡设备，上下文从内核态切换回用户态（切换 4），write 函数返回

传统 IO 的读写流程，包括了 **4 次上下文切换**（4 次用户态和内核态的切换），

4 次数据拷贝（两次 CPU 拷贝以及两次的 DMA 拷贝）。

零拷贝实现方式：

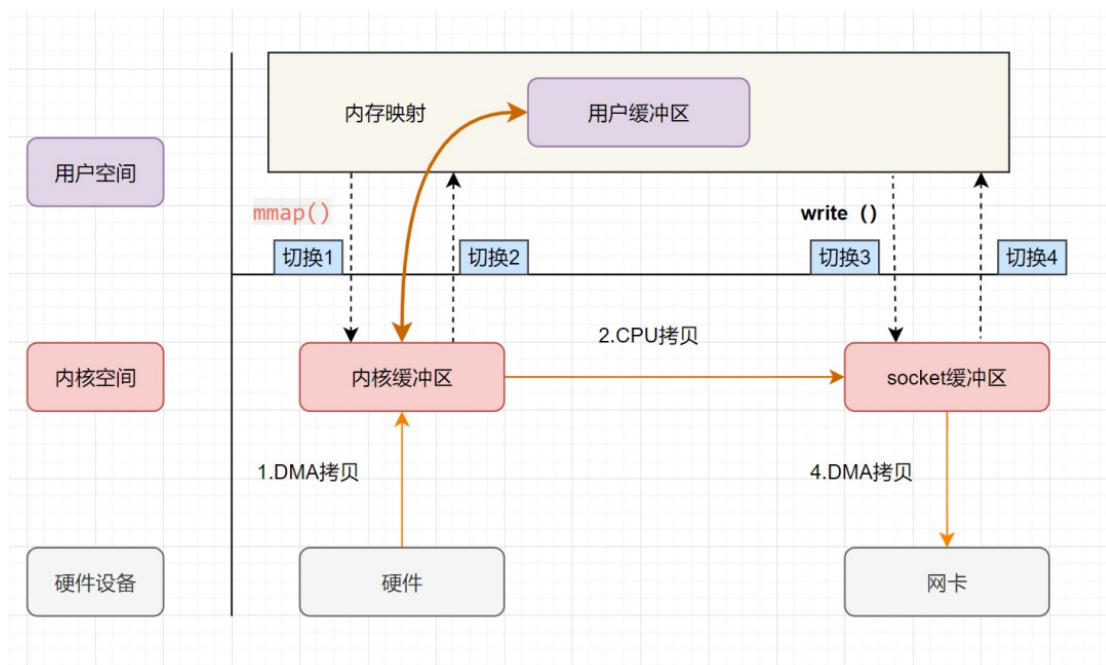
零拷贝并不是没有拷贝数据，而是减少用户态/内核态的切换次数以及 CPU 拷

贝的次数。零拷贝一般有这三种实现方式：

- mmap+write
- sendfile
- 带有 DMA 收集拷贝功能的 sendfile

mmap+write

mmap 就是用了虚拟内存这个特点，它将内核中的读缓冲区与用户空间的缓冲区进行映射，以减少数据拷贝次数！



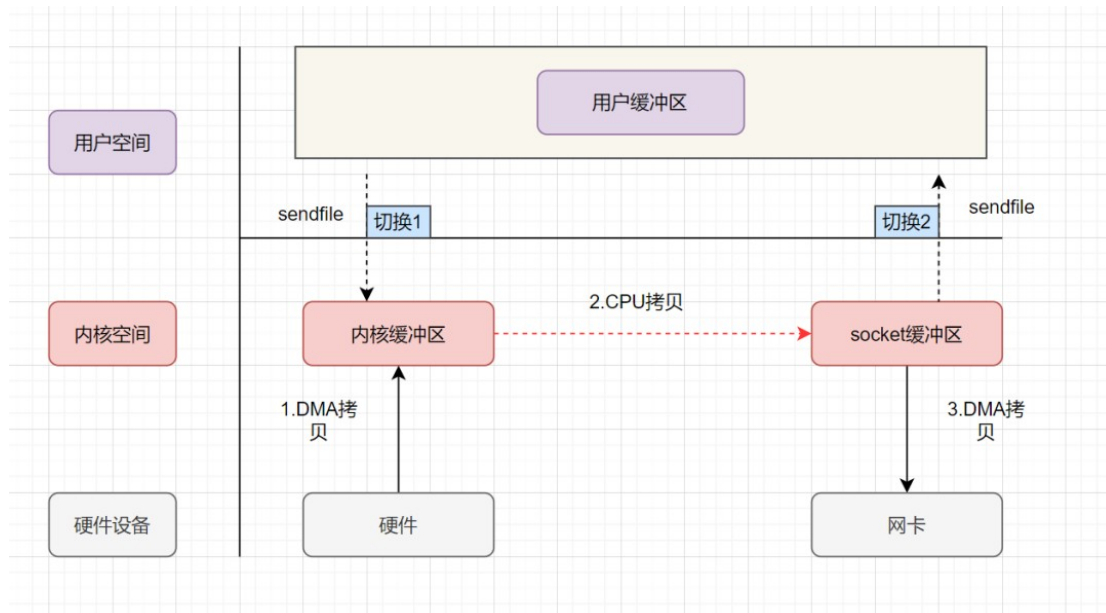
1. 用户进程通过 mmap 方法向操作系统内核发起 IO 调用，上下文从用户态切换为内核态。
2. CPU 利用 DMA 控制器，把数据从硬盘中拷贝到内核缓冲区。

3. 上下文从内核态切换回用户态，mmap 方法返回。
4. 用户进程通过 write 方法向操作系统内核发起 IO 调用，上下文从用户态切换为内核态。
5. CPU 将内核缓冲区的数据拷贝到的 socket 缓冲区。
6. CPU 利用 DMA 控制器，把数据从 socket 缓冲区拷贝到网卡，上下文从内核态切换回用户态，write 调用返回。

mmap+write 实现的零拷贝，I/O 发生了 4 次用户空间与内核空间的上下文切换，以及 3 次数据拷贝（包括了 2 次 DMA 拷贝和 1 次 CPU 拷贝）。

sendfile

sendfile 表示在两个文件描述符之间传输数据，它是在操作系统内核中操作的，避免了数据从内核缓冲区和用户缓冲区之间的拷贝操作

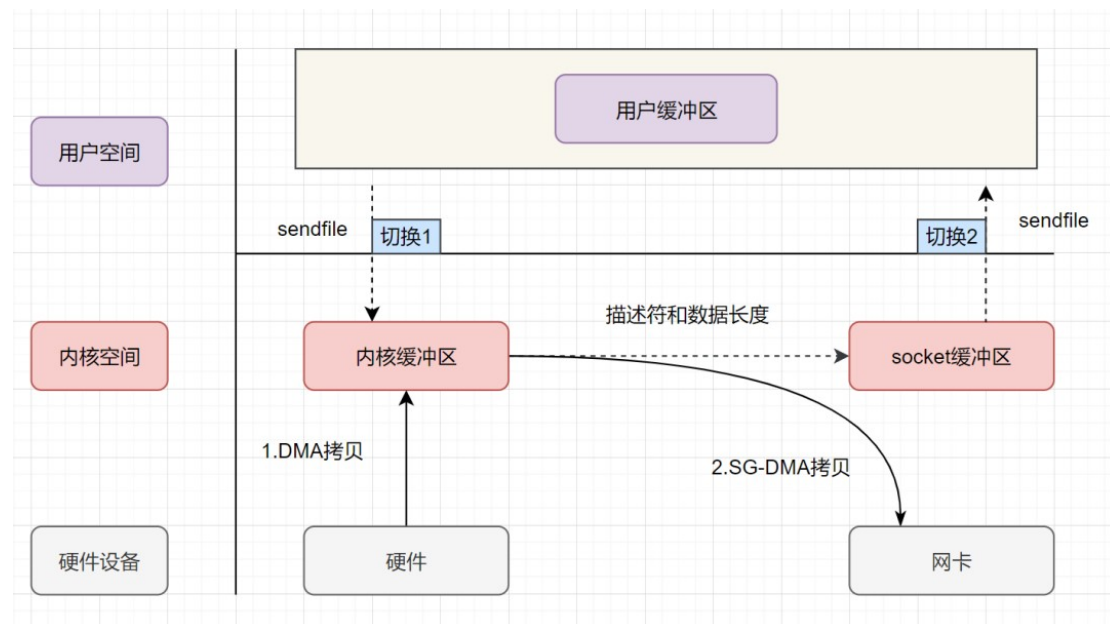


1. 用户进程发起 sendfile 系统调用，上下文（切换 1）从用户态转向内核态
2. DMA 控制器，把数据从硬盘中拷贝到内核缓冲区。
3. CPU 将读缓冲区中数据拷贝到 socket 缓冲区
4. DMA 控制器，异步把数据从 socket 缓冲区拷贝到网卡，
5. 上下文（切换 2）从内核态切换回用户态，sendfile 调用返回。

sendfile 实现的零拷贝，I/O 发生了 2 次用户空间与内核空间的上下文切换，以及 3 次数据拷贝。其中 3 次数据拷贝中，包括了 2 次 DMA 拷贝和 1 次 CPU 拷贝。

带有 DMA 收集拷贝功能的 sendfile

linux 2.4 版本之后，对 sendfile 做了优化升级，引入 SG-DMA 技术，其实就是对 DMA 拷贝加入了 scatter/gather 操作，它可以直接从内核空间缓冲区中将数据读取到网卡。使用这个特点搞零拷贝，即还可以多省去一次 CPU 拷贝。



1. 用户进程发起 sendfile 系统调用，上下文（切换 1）从用户态转向内核态
2. DMA 控制器，把数据从硬盘中拷贝到内核缓冲区。
3. CPU 把内核缓冲区中的文件描述符信息（包括内核缓冲区的内存地址和偏移量）发送到 socket 缓冲区
4. DMA 控制器根据文件描述符信息，直接把数据从内核缓冲区拷贝到网卡
5. 上下文（切换 2）从内核态切换回用户态，sendfile 调用返回。

可以发现，**sendfile+DMA scatter/gather** 实现的零拷贝，I/O 发生了 2 次用户空间与内核空间的上下文切换，以及 2 次数据拷贝。其中 2 次数据拷贝都是包 DMA 拷贝。这就是真正的 零拷贝 (Zero-copy) 技术，全程都没有通过 CPU 来搬运数据，所有的数据都是通过 DMA 来进行传输的。

[看一遍就理解：零拷贝详解](#)

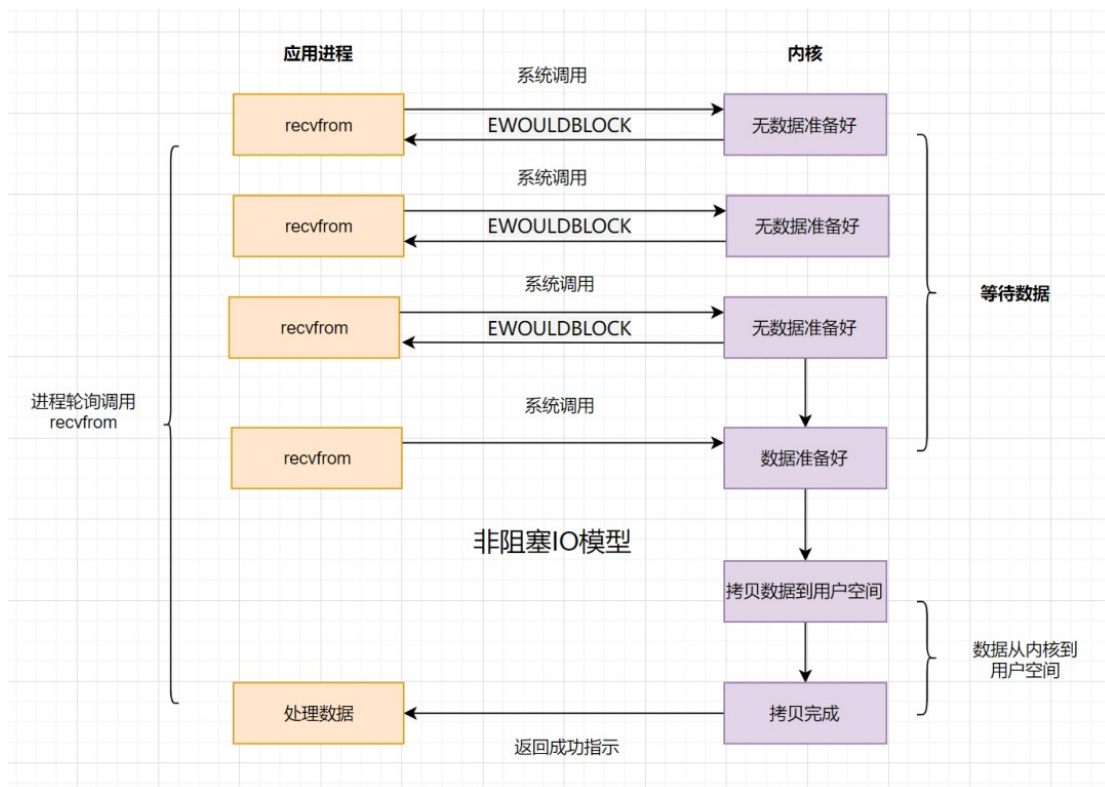
10.3 一共有几种 IO 模型？NIO 和多路复用的区别？

一共有五种 IO 模型

- 阻塞 IO 模型
- 非阻塞 IO 模型
- IO 多路复用模型
- IO 模型之信号驱动模型
- IO 模型之异步 IO(AIO)

NIO (非阻塞 IO 模型)

NIO，即 Non-Blocking IO，是非阻塞 IO 模型。非阻塞 IO 的流程如下：



1. 应用进程向操作系统内核，发起 recvfrom 读取数据。
2. 操作系统内核数据没有准备好，立即返回 EWOULDBLOCK 错误码。
3. 应用程序进程轮询调用，继续向操作系统内核发起 recvfrom 读取数据。
4. 操作系统内核数据准备好了，从内核缓冲区拷贝到用户空间。
5. 完成调用，返回成功提示。

NIO(非阻塞 IO 模型)存在性能问题，即频繁的轮询，导致频繁的系统调用，同样会消耗大量的 CPU 资源。可以考虑 IO 复用模型去解决这个问题。

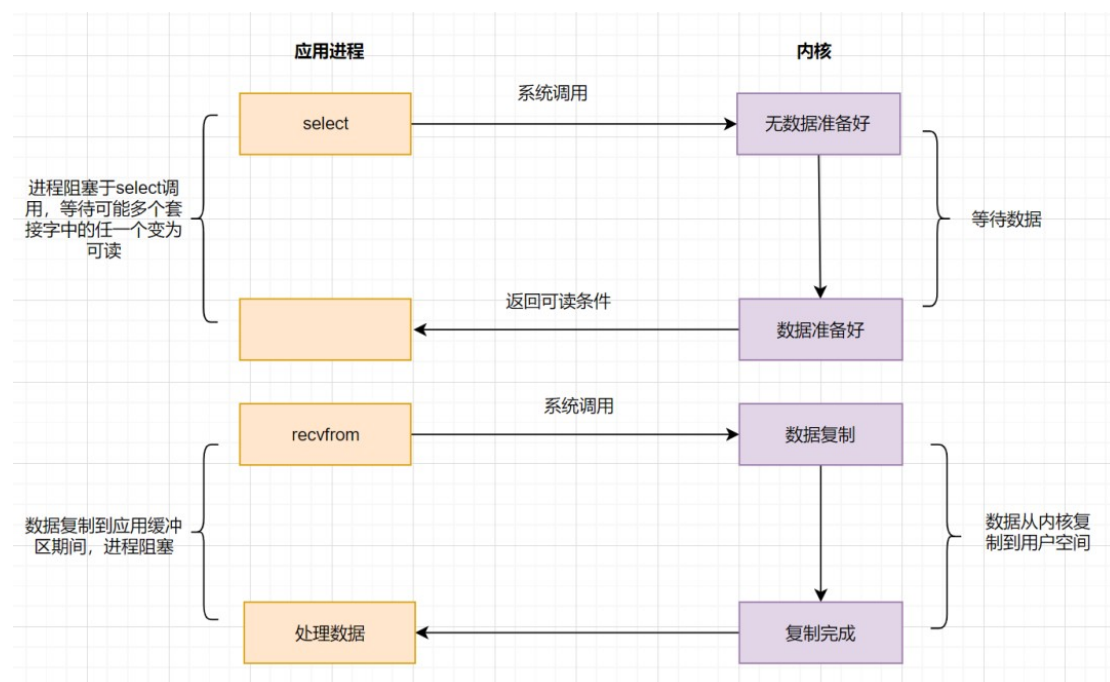
IO 多路复用模型

IO 多路复用就是，等到内核数据准备好了，主动通知应用进程再去进行系统调用。

IO 复用模型核心思路：系统给我们提供一类函数（如我们耳濡目染的 select、poll、epoll 函数），它们可以同时监控多个 fd 的操作，任何一个返回内核数据就绪，应用进程再发起 recvfrom 系统调用。

IO 多路复用之 select

应用进程通过调用 select 函数，可以同时监控多个 fd，在 select 函数监控的 fd 中，只要有任何一个数据状态准备就绪了，select 函数就会返回可读状态，这时应用进程再发起 recvfrom 请求去读取数据。



非阻塞 IO 模型 (NIO) 中，需要 N ($N \geq 1$) 次轮询系统调用，然而借助

select 的 IO 多路复用模型，只需要发起一次询问就够了,大大优化了性能。

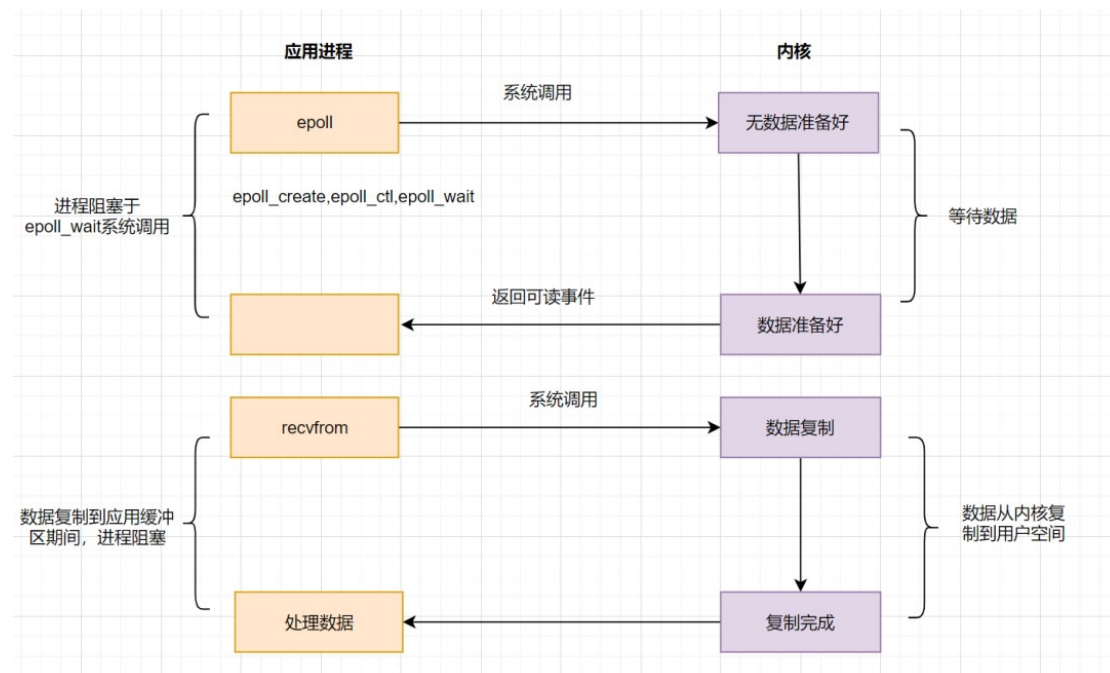
但是呢，select 有几个缺点：

- 监听的 IO 最大连接数有限，在 Linux 系统上一般为 1024。
- select 函数返回后，是通过遍历 fdset，找到就绪的描述符 fd。（仅知道有 I/O 事件发生，却不知是哪几个流，所以遍历所有流）

因为存在连接数限制，所以后来又提出了 poll。与 select 相比，poll 解决了连接数限制问题。但是呢，select 和 poll 一样，还是需要通过遍历文件描述符来获取已经就绪的 socket。如果同时连接的大量客户端，在一时刻可能只有极少处于就绪状态，伴随着监视的描述符数量的增长，效率也会线性下降。

IO 多路复用之 epoll

为了解决 select/poll 存在的问题，多路复用模型 epoll 诞生，它采用事件驱动来实现，流程图如下：

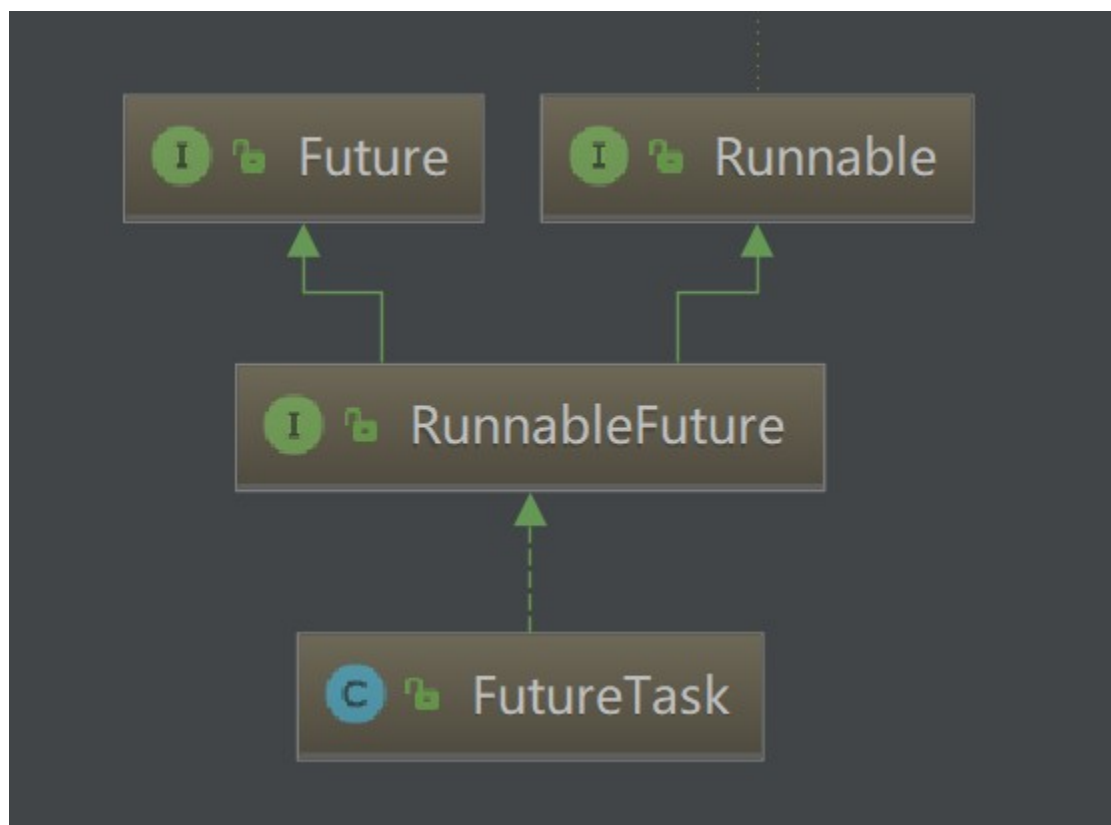


epoll 先通过 `epoll_ctl()` 来注册一个 fd (文件描述符)，一旦基于某个 fd 就绪时，内核会采用回调机制，迅速激活这个 fd，当进程调用 `epoll_wait()` 时便得到通知。这里去掉了遍历文件描述符的坑爹操作，而是采用监听事件回调的机制。这就是 epoll 的亮点。

10.4 Future 实现阻塞等待获取结果的原理？

`Future.get()` 用于异步结果的获取。它是阻塞的，背后原理是什么呢？

我们可以看下 `FutureTask` 的类结构图：



`FutureTask` 实现了 `RunnableFuture` 接口，`RunnableFuture` 继承了 `Runnable` 和 `Future` 这两个接口，对于 `Runnable`，我们太熟悉了，那么 `Future` 呢？

Future 表示一个任务的生命周期，并提供了相应的方法来判断是否已经完成或取消，以及获取任务的结果和取消任务等。

```
public interface Future<V> {  
  
    boolean cancel(boolean mayInterruptIfRunning);  
    //Future 是否被取消  
    boolean isCancelled();  
    //当前 Future 是否已结束  
    boolean isDone();  
    //或取 Future 的结果值。如果当前 Future 还没有结束，当前线程阻塞等待，  
    V get() throws InterruptedException, ExecutionException;  
    //获取 Future 的结果值。与 get() 一样，不过多了超时时间设置  
    V get(long timeout, TimeUnit unit)  
        throws InterruptedException, ExecutionException, TimeoutException;  
}
```

FutureTask 就是 Runnable 和 Future 的结合体，我们可以把 Runnable 看作生产者，Future 看作消费者。而 FutureTask 是被这两者共享的，生产者运行 run 方法计算结果，消费者通过 get 方法获取结果。

生产者消费者模式，如果生产者数据还没准备的时候，消费者会被阻塞。当生产者数据准备好了以后会唤醒消费者继续执行。我们来看下 FutureTask 内部是如何实现的。

FutureTask 内部维护了任务状态 state

```
//NEW 新建状态,表示 FutureTask 新建还没开始执行  
private static final int NEW = 0;  
//完成状态,表示 FutureTask  
private static final int COMPLETING = 1;  
//任务正常完成，没有发生异常  
private static final int NORMAL = 2;  
//发生异常  
private static final int EXCEPTIONAL = 3;  
//取消任务  
private static final int CANCELLED = 4;  
//发起中断请求  
private static final int INTERRUPTING = 5;
```



```
//中断请求完成
```

```
private static final int INTERRUPTED = 6;
```

生产者 run 方法：

```
public void run() {  
    // 如果状态 state 不是 NEW，或者设置 runner 值失败，直接返回  
    if (state != NEW ||  
        !UNSAFE.compareAndSwapObject(this, runnerOffset,  
                                       null, Thread.currentThread()))  
        return;  
    try {  
        Callable<V> c = callable;  
        if (c != null && state == NEW) {  
            V result;  
            boolean ran;  
            try {  
                //调用 callable 的 call 方法，获取结果  
                result = c.call();  
                //运行成功  
                ran = true;  
            } catch (Throwable ex) {  
                result = null;  
                //运行不成功  
                ran = false;  
                //设置异常  
                setException(ex);  
            }  
            //运行成功设置返回结果  
            if (ran)  
                set(result);  
        }  
    } finally {  
        runner = null;  
        int s = state;  
        if (s >= INTERRUPTING)  
            handlePossibleCancellationInterrupt(s);  
    }  
}
```

消费者的 get 方法

```
public V get() throws InterruptedException, ExecutionException {  
    int s = state;  
    //如果状态小于等于 COMPLETING，表示 FutureTask 任务还没有完成，则调用 awaitDone 让当
```

前线程等待。

```
    if (s <= COMPLETING)
        s = awaitDone(false, 0L);
    return report(s);
}
```

awaitDone 做了什么事情呢？

```
private int awaitDone(boolean timed, long nanos)
    throws InterruptedException {
    final long deadline = timed ? System.nanoTime() + nanos : 0L;
    WaitNode q = null;
    boolean queued = false;
    for (;;) {
        // 如果当前线程是中断标记，则
        if (Thread.interrupted()) {
            //那么从列表中移除节点 q，并抛出 InterruptedException 异常
            removeWaiter(q);
            throw new InterruptedException();
        }

        int s = state;
        //如果状态已经完成，表示 FutureTask 任务已结束
        if (s > COMPLETING) {
            if (q != null)
                q.thread = null;
            //返回
            return s;
        }
        // 表示还有一些后序操作没有完成，那么当前线程让出执行权
        else if (s == COMPLETING) // cannot time out yet
            Thread.yield();
        //将当前线程阻塞等待
        else if (q == null)
            q = new WaitNode();
        else if (!queued)
            queued = UNSAFE.compareAndSwapObject(this, waitersOffset,
                                                  q.next = q);
        //timed 为 true 表示需要设置超时
        else if (timed) {
            nanos = deadline - System.nanoTime();
            if (nanos <= 0L) {
                removeWaiter(q);
                return state;
            }
        }
    }
}
```

```

    }
    //让当前线程等待 nanos 时间
    LockSupport.parkNanos(this, nanos);
    }
    else
        LockSupport.park(this);
    }
}

```

当然，面试的时候，不一定要讲到源码这么细，只需要将个大概思路就好啦。

10.5 ReentrantLock 和 Synchronized 的区别？

Synchronized 的原理？

ReentrantLock 和 Synchronized 的区别？

- Synchronized 是依赖于 JVM 实现的，而 ReentrantLock 是 API 实现的。
- 在 Synchronized 优化以前，synchronized 的性能是比 ReentrantLock 差很多的，但是自从 Synchronized 引入了偏向锁，轻量级锁（自旋锁）后，两者性能就差不多了。
- Synchronized 的使用比较方便简洁，它由编译器去保证锁的加锁和释放。而 ReentrantLock 需要手工声明来加锁和释放锁，最好在 finally 中声明释放锁。
- ReentrantLock 可以指定是公平锁还是非公平锁。而 synchronized 只能是非公平锁。
- ReentrantLock 可响应中断、可轮回，而 Synchronized 是不可以响应中断的，

至于 **Synchronized** 的原理，大家可以看我这篇文章哈

[Synchronized 解析——如果你愿意一层一层剥开我的心](#)

10.6 聊聊 AQS ? ReentrantLock 的实现原理？

AQS（抽象同步队列）的核心回答要点就是：

- state 状态的维护。
- CLH 队列
- ConditionObject 通知

- 模板方法设计模式
- 独占与共享模式。
- 自定义同步器。

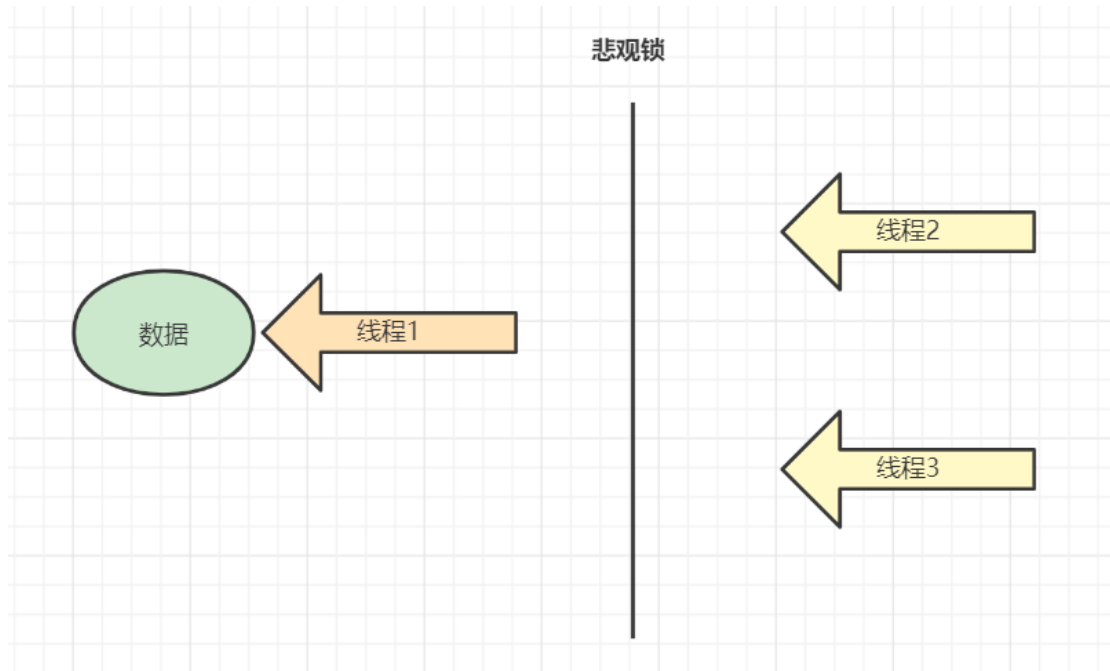
大家可以看下我之前这篇文章哈：[AQS 解析与实战](#)

大家综合 ReentrantLock 的功能，比如可重入，公平锁，非公平锁等，与 AQS 结合在一起讲就好啦。

10.7 乐观锁和悲观锁，让你来写你怎么实现？

悲观锁：

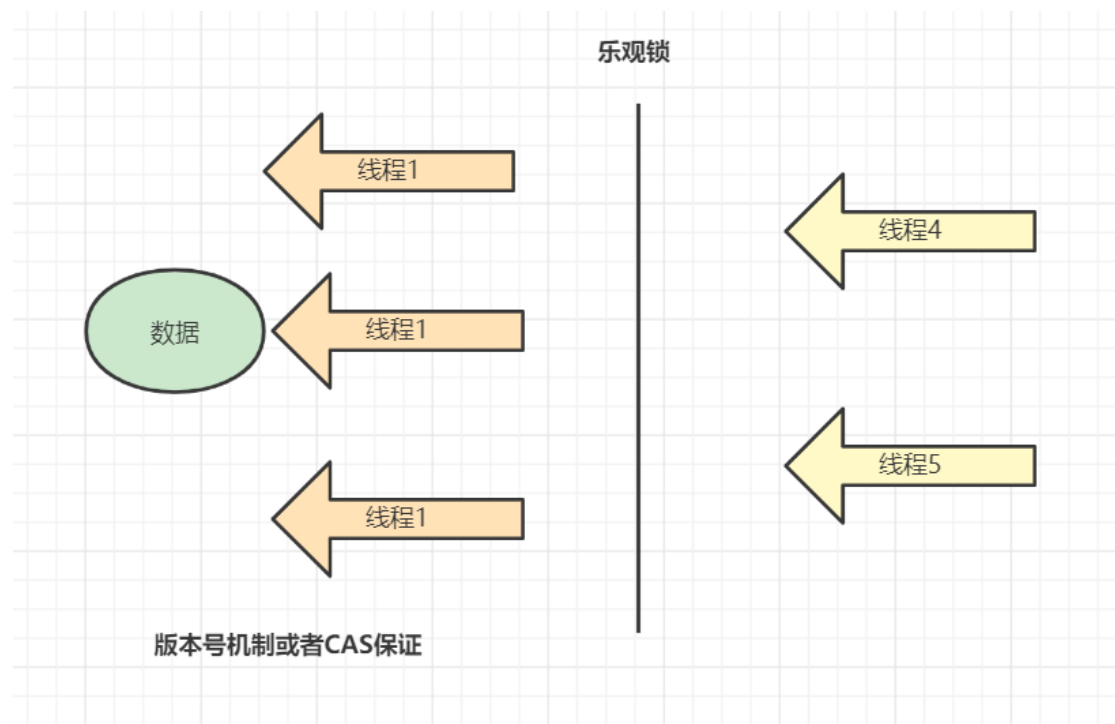
悲观锁她专一且缺乏安全感了，她的心只属于当前线程，每时每刻都担心着它心爱的数据可能被别的线程修改。因此一个线程拥有（获得）悲观锁后，其他任何线程都不能对数据进行修改啦，只能等待锁被释放才可以执行。



- SQL 语句 `select ...for update` 就是悲观锁的一种实现
- 还有 Java 的 `synchronized` 关键字也是悲观锁的一种体现

乐观锁：

乐观锁的很乐观，它认为数据的变动不会太频繁,操作时一般都不会产生并发问题。因此，它不会上锁，只是在更新数据时，再去判断其他线程在这之前有没有对数据进行过修改。**实现方式：**乐观锁一般会使用**版本号机制**或 CAS 算法实现。



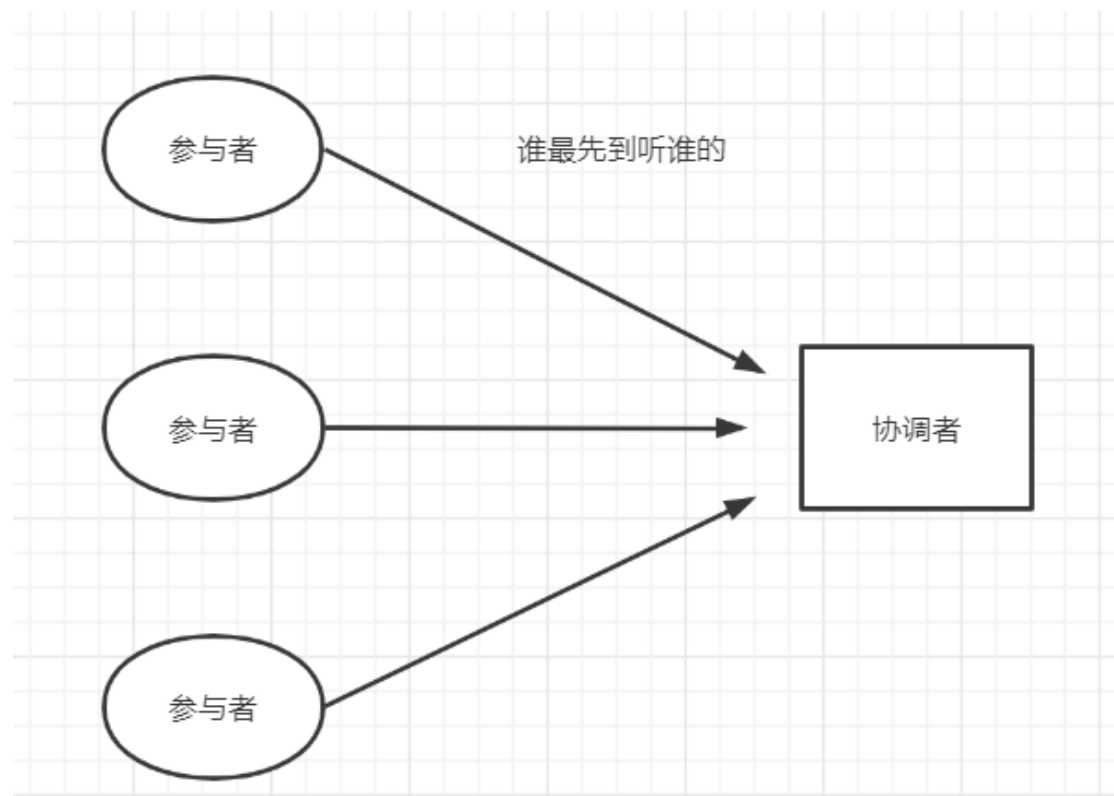
之前业务上使用过 CAS 解决并发问题，大家有兴趣可以看一下哈：

- [CAS 乐观锁解决并发问题的一次实践](#)

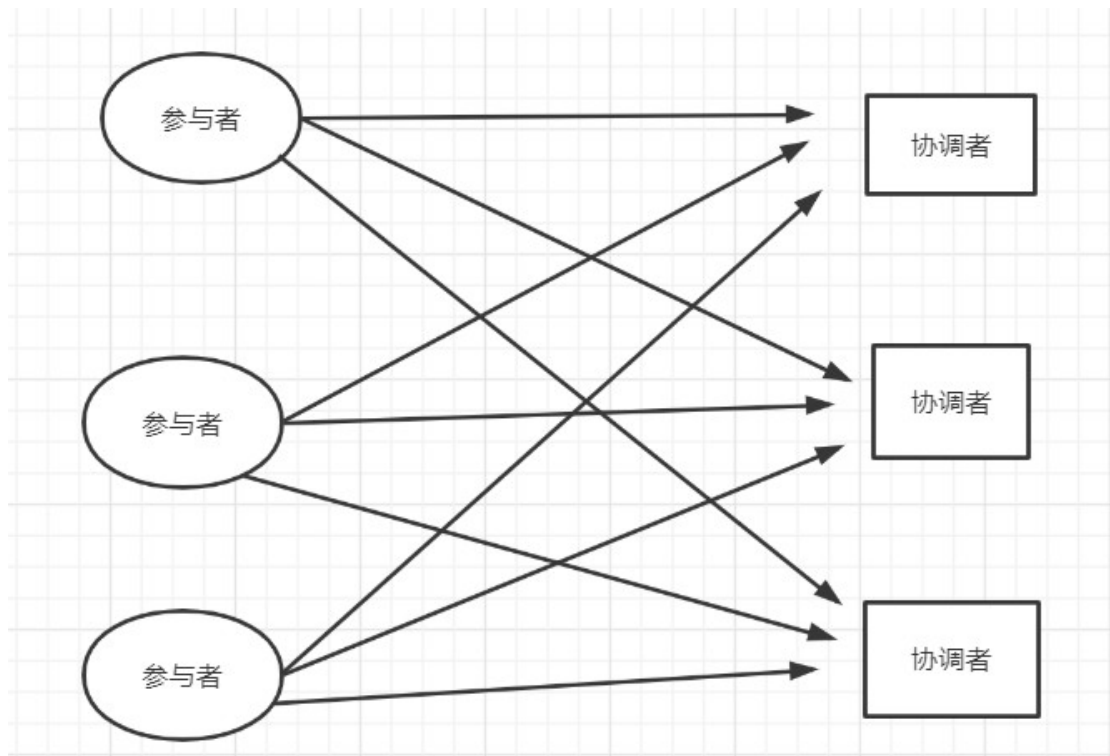
10.8 Paxos 协议了解？工作流程是怎么样的？

10.8.1 为什么需要 Paxos 算法？

当前我们应用都是集群部署的，要求所有机器状态一致。假设当前有两台机器 A 和 B，A 要把状态修改为 a，B 要把状态修改为 b，那么应该听谁的呢？这时候可以像 2PC 一样，引入一个协调者，谁最先到就听谁的。



这里有个问题，就是协调者是单节点，如果它挂了呢。因为可以引入多个协调者



但是这么多协调者，应该听谁的呢？

引入 Paxos 算法解决这个问题，Paxos 算法是一种基于消息传递的分布式一致性算法。

10.8.2 Paxos 的角色

Paxos 涉及三种角色，分别是 Proposer、Acceptor、Learners。

- Proposer：它可以提出提案 (Proposal)，提案信息包括**提案编号**和**提案值**。
- Acceptor：接受接受 (accept) 提案。一旦接受提案，提案里面的提案值 (可以用 V 表示) 就被选定了。
- Learner: 哪个提案被选定了, Learner 就学习这个被选择的提案

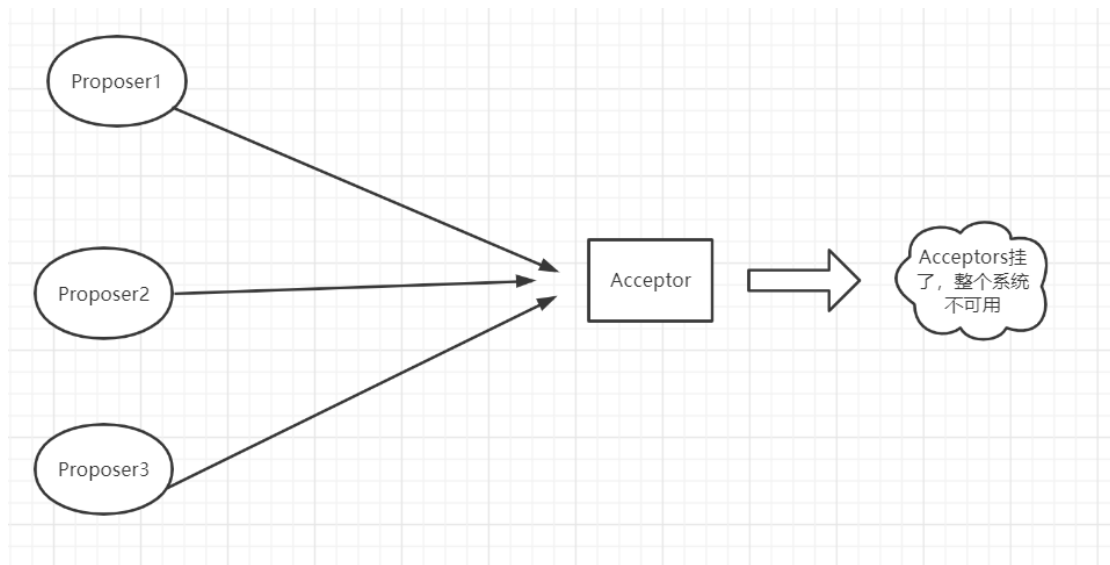
一个进程可能是 Proposer,也可能是 Acceptor，也可能是 Learner。

10.8.3 Paxos 算法推导过程

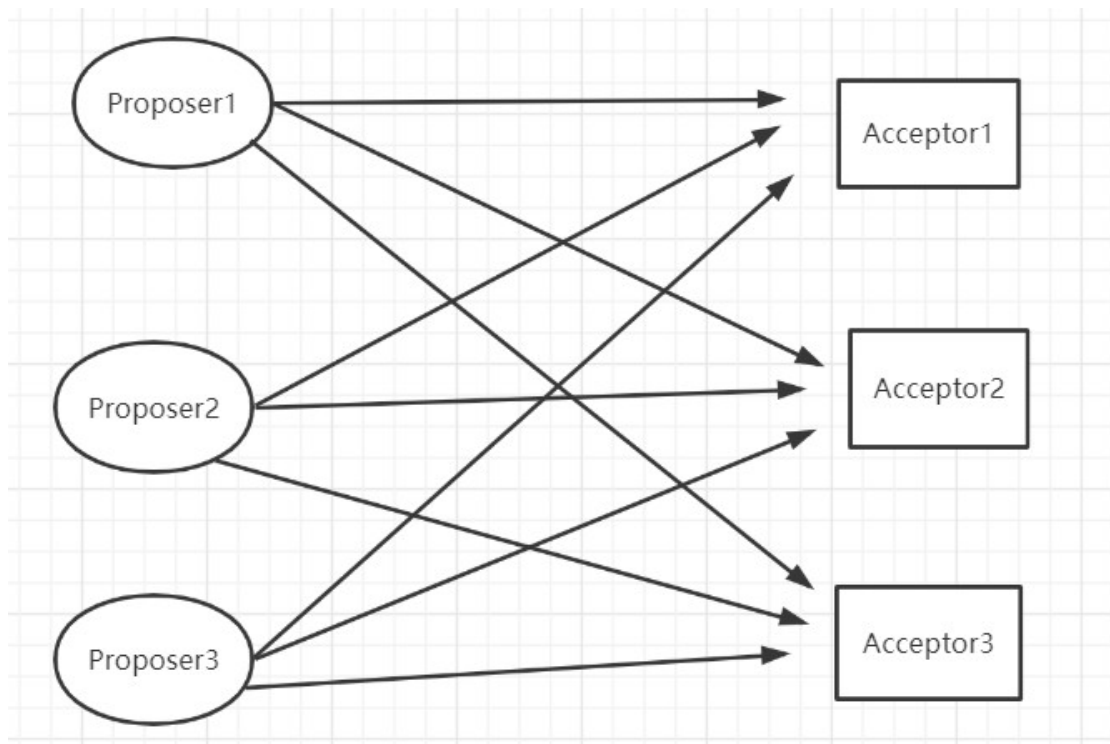
一致性算法需要前置条件

- 在这些被提出的提案中，只有一个会被选定
- 如果没有提案被提出，就不应该有被选定的提案 - 当提案被选定后，learner 可以学习被选中的提案

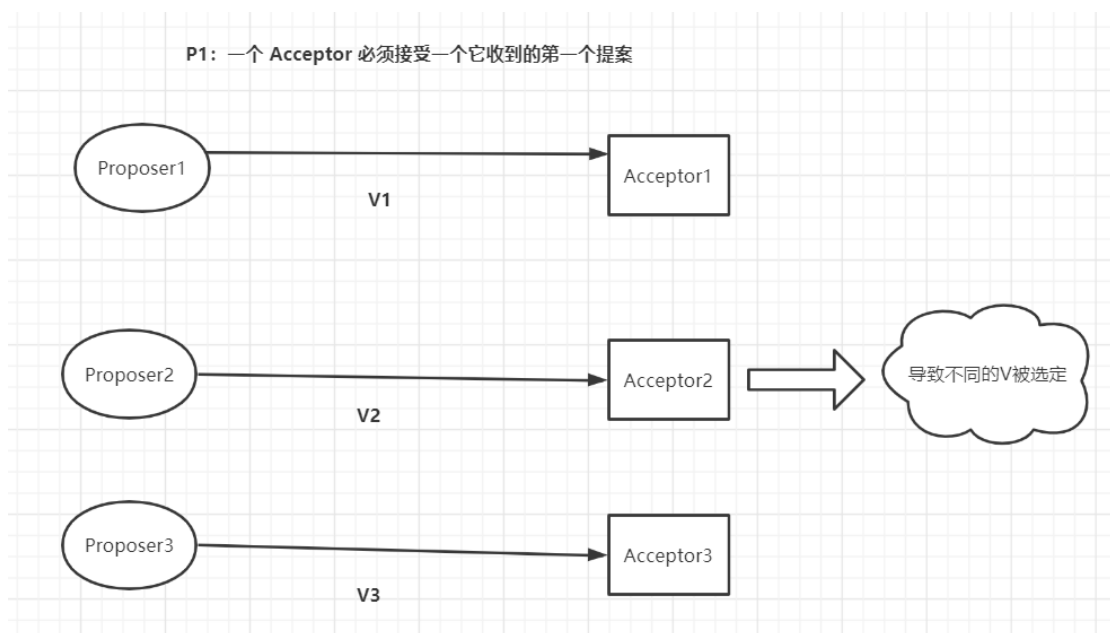
假设只有一个 Acceptor，只要 Acceptor 接受它收到的第一个提案，就可以保证只有一个 value 会被选定。但是这个 Acceptor 宕机，会导致整个系统不可用。



如果是多个 Proposer 和多个 Acceptor，如何选定一个提案呢？



我们可以加个约定条件，假设就叫约束 **P1**：一个 **Acceptor** 必须接受它收到的**第一个提案**。但是这样还是可能会有问题，如果每个 Proposer 分别提出不同的 value（如下图 V1，V2，V3），发给了不同的 Acceptor，最后会导致不同的 value 被选择。



我们可以给多一个额外的约定 **P1a: 一个提案被选定，需要被半数以上**

Acceptor 接受。这跟 **P1** 有点矛盾啦，我们可以使用一个全局的编号来标识

每一个 Acceptor 批准的提案，当一个具有某 value 值的提案被半数以上的

Acceptor 批准后，我们就认为该 value 被选定了。即提案 $P = \text{提案参数} + \text{提案值}$ ，可以记为 **【M,V】**。

现在可以允许多个提案被选定，但必须保证所有被选定的提案都具有相同的 value 值。要不然又会出现不一致啦。因此可以再加个约束 P2：

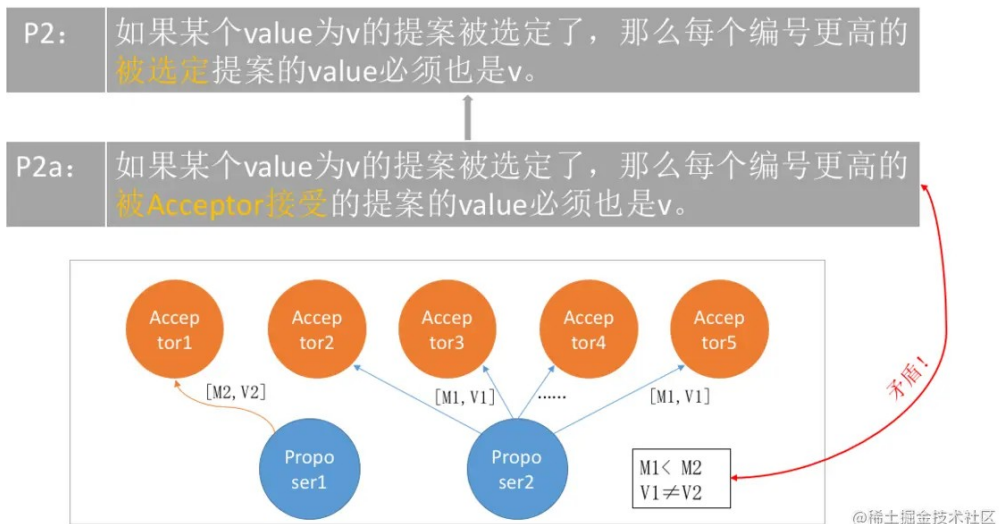
如果提案 $P[M1, V1]$ 被选定了，那么所有比 $M1$ 编号更高的被选定提案 P ，其 value 的值也必须是 $V1$ 。

一个提案要被选定，至少要被一个 Acceptor 接受，因此我们可以把 P2 约束改成对 Acceptor 接受的约束 P2a：

如果提案 $P[M1, V1]$ 被接受了，那么所有比 $M1$ 编号更高的，且被 Acceptor 接受的 P ，其值也是 $V1$ 。

多提案被选择的问题解决了，但是如果是网络不稳定或者宕机的原因，还是会有问题。

推导过程



假设有 5 个 Acceptor。Proposer2 提出 $[M1, V1]$ 的提案，Acceptor2~5 (半数以上) 均接受了该提案，于是对于 Acceptor2~5 和 Proposer2 来讲，

它们都认为 V1 被选定。Acceptor1 刚刚从宕机状态恢复过来（之前 Acceptor1 没有收到过任何提案），此时 Proposer1 向 Acceptor1 发送了 [M2,V2] 的提案（ $V2 \neq V1$ 且 $M2 > M1$ ）。对于 Acceptor1 来讲，这是它收到的第一个提案。根据 P1（一个 Acceptor 必须接受它收到的第一个提案），Acceptor1 必须接受该提案。同时 Acceptor1 认为 V2 被选定。

这就出现了两个问题：

- Acceptor1 认为 V2 被选定，Acceptor2~5 和 Proposer2 认为 V1 被选定。出现了不一致。
- V1 被选定了，但是编号更高的被 Acceptor1 接受的提案 [M2,V2] 的 value 为 V2，且 $V2 \neq V1$ 。这就跟 P2a（如果提案 P[M1,V1] 被接受了，那么所有比 M1 编号更高的，且被 Acceptor 接受的 P，其值也是 V1。）矛盾了。

我们要对 P2a 约束强化一下得到约束 P2b，

如果 P[M1,V1] 被选定后，任何 Proposer 产生的 P，其值也是 V1。

对于 P2b 中的描述，如何保证任何 Proposer 产生的 P，其值也是 V1？只要满足 P2c 即可：

对于任意的 M 和 V，如果提案 [M,V] 被提出，那么肯定存在一个由半数以上的 Acceptor 组成的集合 S，满足以下两个条件中的任意一个：

- 要么 S 中每个 Acceptor 都没有接受过编号小于 M 的提案。
- 要么 S 中所有 Acceptor 批准的所有编号小于 M 的提案中，编号最大的那个提案的 value 值为 V

10.8.4 算法流程

Proposer 生成提案

- Prepare 请求
- Accept 请求

在 **P2c** 约束基础上，如何生成提案呢？

Proposer 选择一个新的提案编号 N ，向 Acceptor 集合 S (数目在半数以上)

发送请求，要求 S 中的每一个 Acceptor 做出如下响应：

- 如果 Acceptor 没有接受过提案，则向 Proposer 保证 不再接受编号小于 N 的提案。
- 如果 Acceptor 接受过请求，则向 Proposer 返回 已经接受过的编号小于 N 的编号最大的提案。

我们将这个请求称为编号为 N 的 **Prepare 请求**。

- 如果 Proposer 收到半数以上的 Acceptor 响应，则生成编号为 N ，value 为 V 的提案 $[N, V]$ ， V 为所有响应中编号最大的提案的 value。
- 如果 Proposer 收到的响应中没有提案，那么 value 由 Proposer 自己生成，生成后将此提案发给 S ，并期望 Acceptor 能接受此提案。

我们称这个请求为 **Accept 请求**

Acceptor 接受提案

一个 Acceptor 可能会受到来自 Proposer 的两种请求:Prepare 请求和

Accept 请求。Acceptor 什么时候可以响应一个请求呢，它也有个约束：

P1b :

一个 Acceptor 只要尚未响应过任何编号大于 N 的 Prepare 请求，那么他就可以接受这个编号为 N 的提案。

Acceptor 收到编号为 N 的 Prepare 请求，如果在此之前它已经响应过编号大于 N 的 Prepare 请求。由约束 P1b，该 Acceptor 不会接受这个编号为 N 的提案。因此，Acceptor 会忽略这个请求。

一个 Acceptor 只需记住两点：已接受的编号最大的提案和已响应的请求的最大编号。

Paxos 算法描述

阶段一：

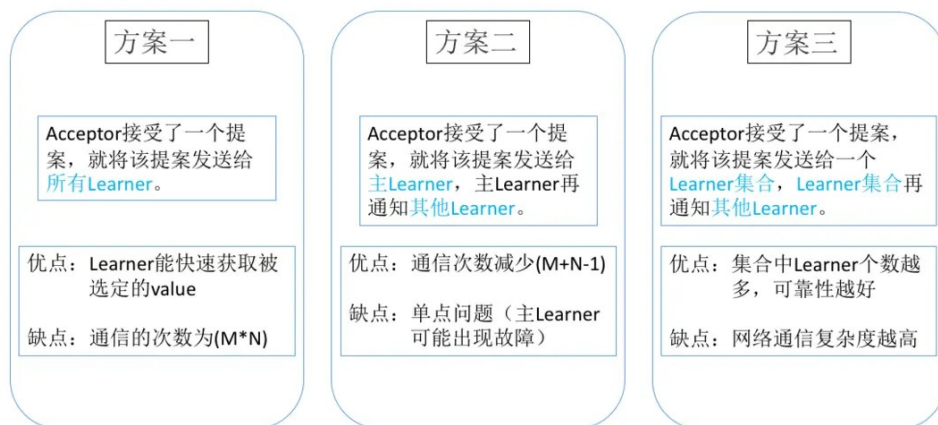
- Proposer 选择一个提案编号 N ，然后向半数以上的 Acceptor 发送编号为 N 的 Prepare 请求。
- 如果一个 Acceptor 收到一个编号为 N 的 Prepare 请求，且 N 大于该 Acceptor 已经响应过的所有 Prepare 请求的编号，那么它就会将它已经接受过的编号最大的提案(如果有的话)作为响应反馈给 Proposer，同时该 Acceptor 承诺不再接受任何编号小于 N 的提案。

阶段二：

- 如果 Proposer 收到半数以上 Acceptor 对其发出的编号为 N 的 Prepare 请求的响应，那么它就会发送一个针对 $[N, V]$ 提案的 Accept 请求给半数以上的 Acceptor。注意: V 就是收到的响应中编号最大的提案的 value，如果响应中不包含任何提案，那么 V 就由 Proposer 自己决定。
- 如果 Acceptor 收到一个针对编号为 N 的提案的 Accept 请求，只要该 Acceptor 没有对编号大于 N 的 Prepare 请求 做出过响应，它就接受该提案。

Learner 学习被选定的 value

Learner 学习（获取）被选定的 value



@稀土掘金技术社区

10.9 B+树聊一下？B+树是不是有序？B+树和 B-树的主要区别？B+树索引，一次查找过程？

B+树是有序的。

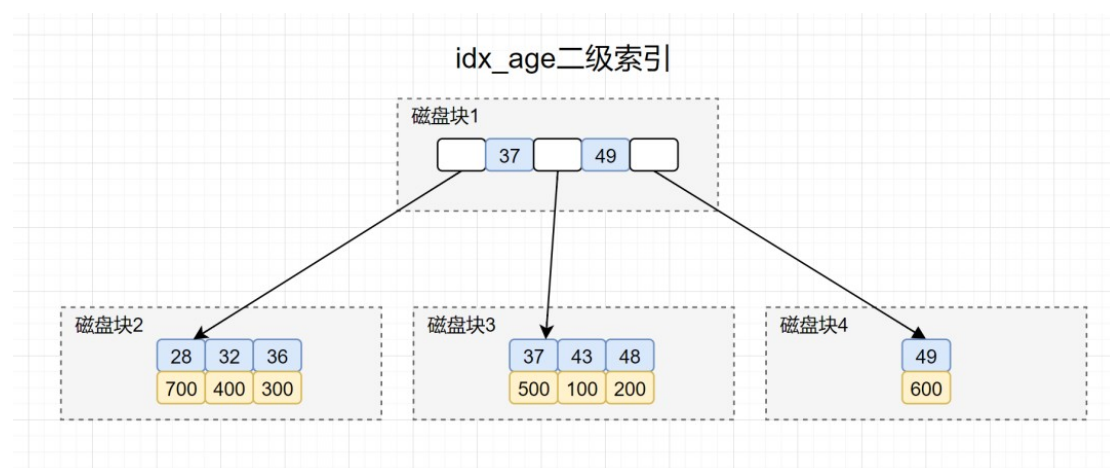
B+树和 B-树的主要区别？

- B-树内部节点是保存数据的;而 B+树内部节点是不保存数据的，只作索引作用，它的叶子节点才保存数据。
- B+树相邻的叶子节点之间是通过链表指针连起来的，B-树却不是。
- 查找过程中，B-树在找到具体的数值以后就结束，而 B+树则需要通过索引找到叶子结点中的数据才结束
- B-树中任何一个关键字出现且只出现在一个结点中，而 B+树可以出现多次。

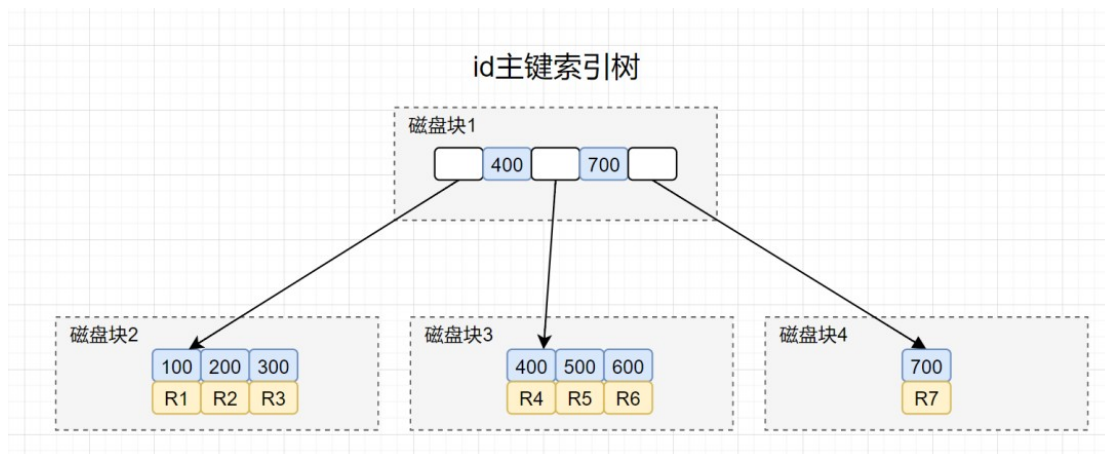
假设有这么一个 SQL：

```
select * from Templooyee where age=32;
```

age 加个一个索引，这条 SQL 是如何在索引上执行的？大家可以举例子画个示意图哈，比如二级索引树，



再画出 id 主键索引，我们先画出聚族索引结构图，如下：



因此，这条 SQL 查询语句执行大概流程就是酱紫：

- 搜索 idx_age 索引树，将磁盘块 1 加载到内存，由于 $32 < 37$ ，搜索左路分支，到磁盘寻址磁盘块 2。
- 将磁盘块 2 加载到内存中，在内存继续遍历，找到 age=32 的记录，取得 id = 400。
- 拿到 id=400 后，回到 id 主键索引树。
- 搜索 id 主键索引树，将磁盘块 1 加载内存，在内存遍历，找到了 400，但是 B+ 树索引非叶子节点是不保存数据的。索引会继续搜索 400 的右分支，到磁盘寻址磁盘块 3。
- 将磁盘块 3 加载内存，在内存遍历，找到 id=400 的记录，拿到 R4 这一行的数据，好的，大功告成。

10.10 TCP 怎么实现拥塞控制？

拥塞控制是作用于网络的，防止过多的数据包注入到网络中，避免出现网络负载过大的情况。它的目标主要是最大化利用网络上瓶颈链路的带宽。

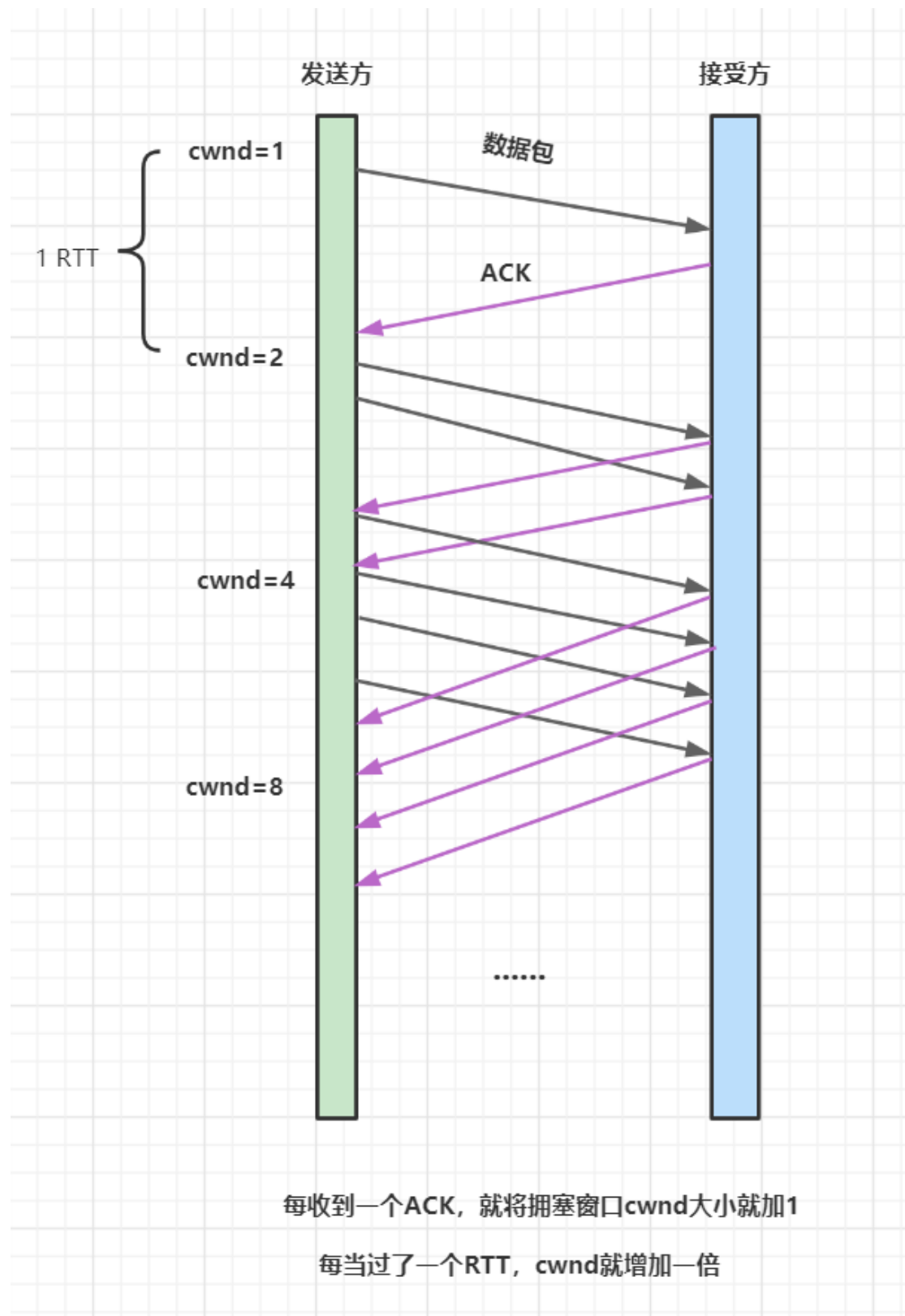
实际上，拥塞控制主要有这几种常用算法

- 慢启动
- 拥塞避免
- 拥塞发生
- 快速恢复

10.10.1 慢启动算法

慢启动算法，表面意思就是，别急慢慢来。它表示 TCP 建立连接完成后，一开始不要发送大量的数据，而是先探测一下网络的拥塞程度。由小到大逐渐增加拥塞窗口的大小，如果没有出现丢包，**每收到一个 ACK，就将拥塞窗口 cwnd 大小就加 1 (单位是 MSS)**。每轮次发送窗口增加一倍，呈指数增长，如果出现丢包，拥塞窗口就减半，进入拥塞避免阶段。

- TCP 连接完成，初始化 $cwnd = 1$ ，表明可以传一个 MSS 单位大小的数据。
- 每当收到一个 ACK，cwnd 就加一；
- 每当过了一个 RTT，cwnd 就增加一倍；呈指数让升



为了防止 cwnd 增长过大引起网络拥塞，还需设置一个**慢启动阈值**

ssthresh (slow start threshold) 状态变量。当 cwnd 到达该阈值后，就好

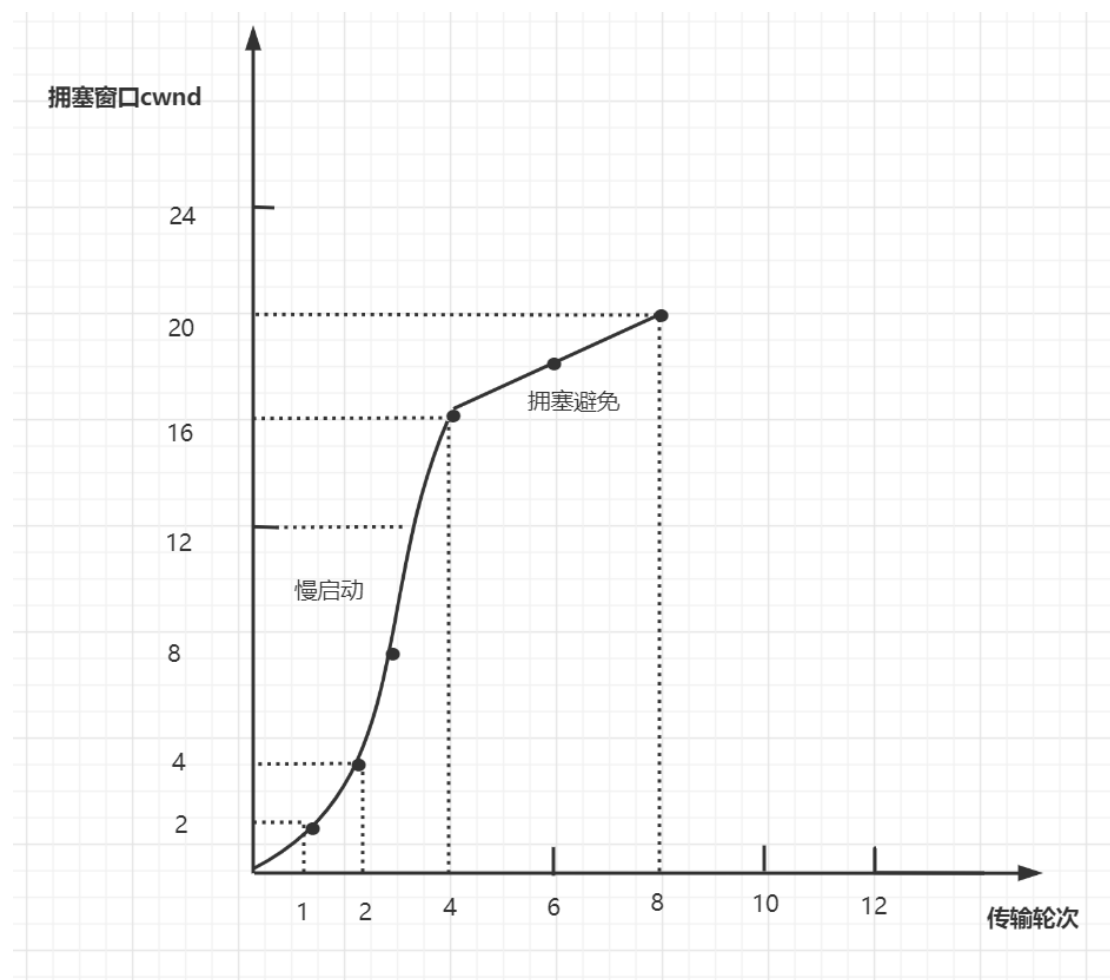
像水管被关小了水龙头一样，减少拥塞状态。即当 $cwnd > ssthresh$ 时，进入了拥塞避免算法。

10.10.2 拥塞避免算法

一般来说，慢启动阈值 $ssthresh$ 是 65535 字节， $cwnd$ 到达慢启动阈值后

- 每收到一个 ACK 时， $cwnd = cwnd + 1/cwnd$
- 当每过一个 RTT 时， $cwnd = cwnd + 1$

显然这是一个线性上升的算法，避免过快导致网络拥塞问题。



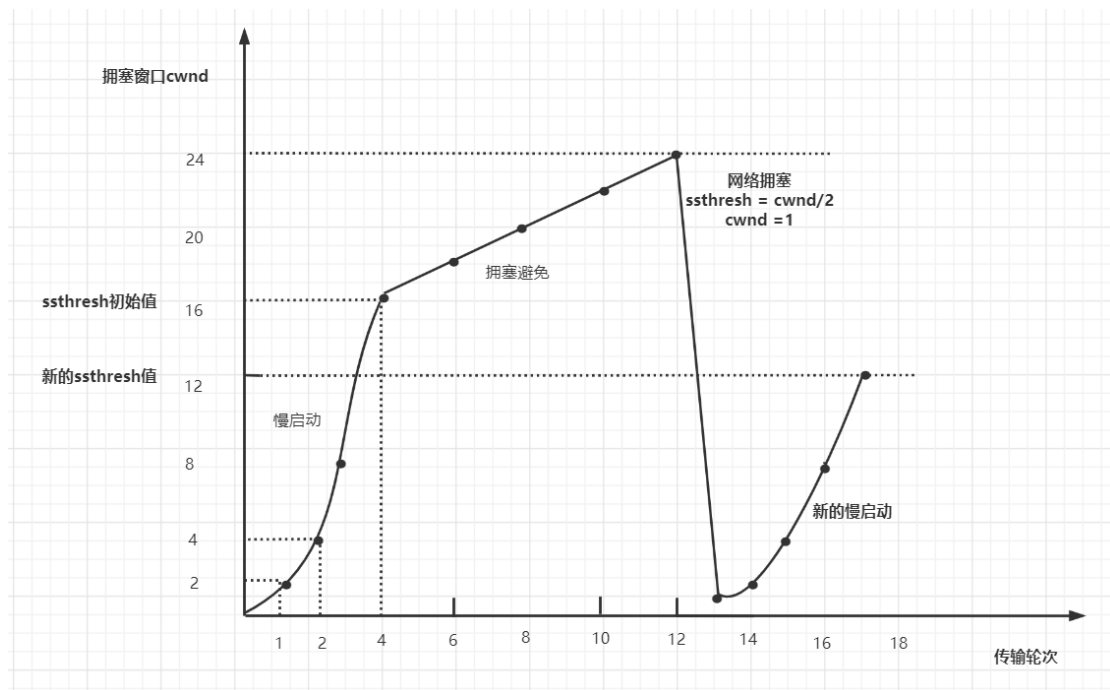
10.10.3 拥塞发生

当网络拥塞发生丢包时，会有两种情况：

- RTO 超时重传
- 快速重传

如果是发生了 **RTO 超时重传**，就会使用拥塞发生算法

- 慢启动阈值 $ssthresh = cwnd / 2$
- $cwnd$ 重置为 1
- 进入新的慢启动过程



这真的是辛辛苦苦几十年，一朝回到解放前。其实还有更好的处理方式，就是快速重传。发送方收到 3 个连续重复的 ACK 时，就会快速地重传，不必等待 **RTO 超时**再重传。

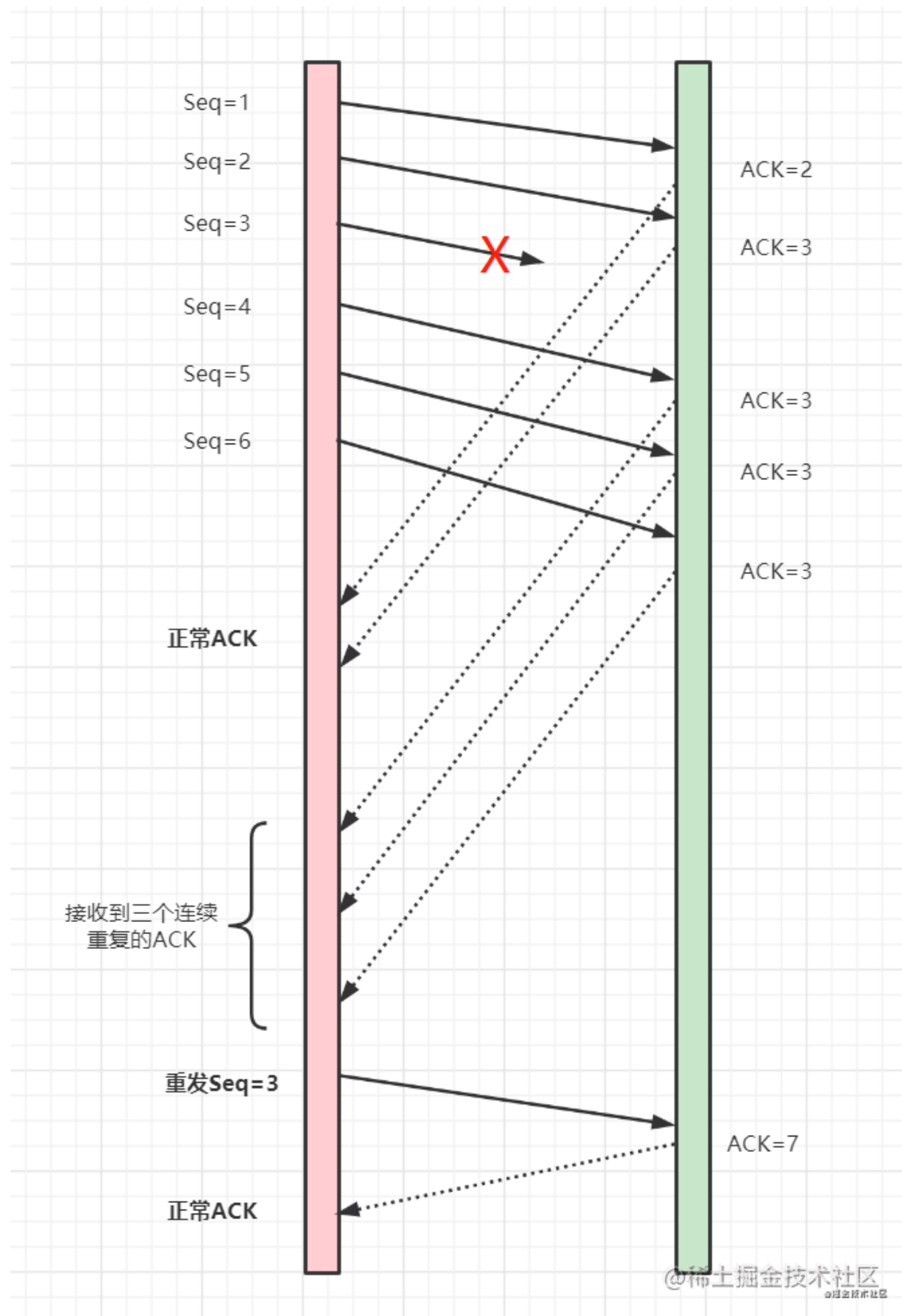


image.png

慢启动阈值 ssthresh 和 cwnd 变化如下：

- 拥塞窗口大小 $cwnd = cwnd/2$

- 慢启动阈值 $ssthresh = cwnd$
- 进入快速恢复算法

10.10.4 快速恢复

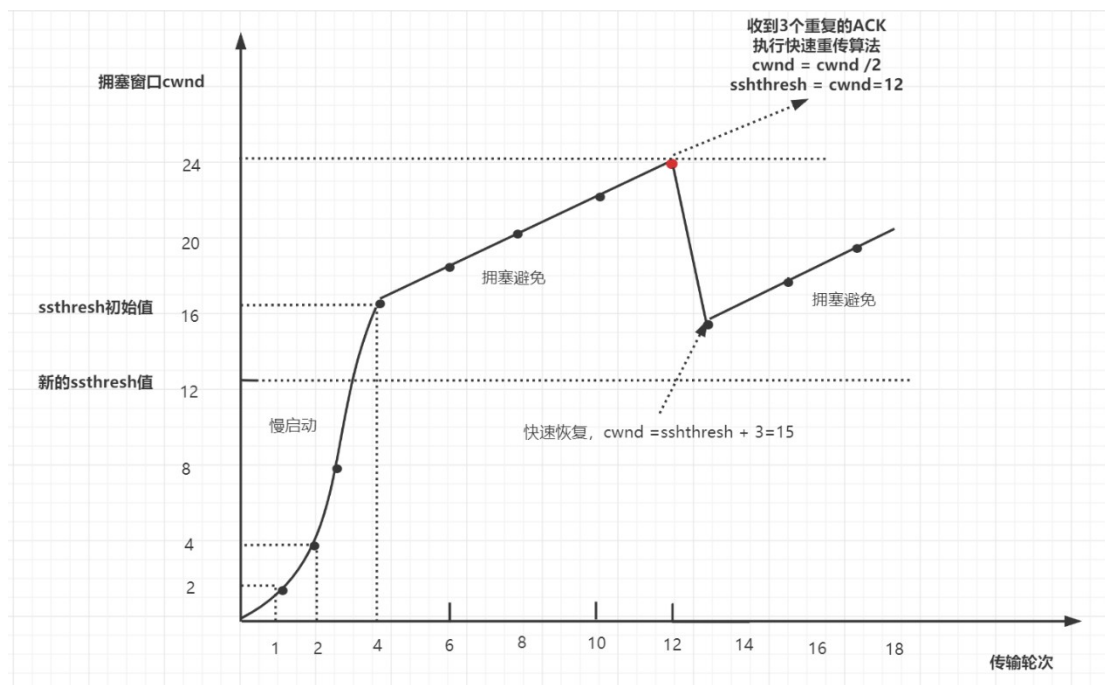
快速重传和快速恢复算法一般同时使用。快速恢复算法认为，还有 3 个重复 ACK 收到，说明网络也没那么糟糕，所以没有必要像 RTO 超时那么强烈。

正如前面所说，进入快速恢复之前， $cwnd$ 和 $ssthresh$ 已被更新：

```
- cwnd = cwnd / 2
- ssthresh = cwnd
```

然后，真正的快速算法如下：

- $cwnd = ssthresh + 3$
- 重传重复的那几个 ACK (即丢失的那几个数据包)
- 如果再收到重复的 ACK，那么 $cwnd = cwnd + 1$
- 如果收到新数据的 ACK 后， $cwnd = ssthresh$ 。因为收到新数据的 ACK，表明恢复过程已经结束，可以再次进入了拥塞避免的算法了。



10. 11. JVM 调优

10.11.1 一般什么时候考虑 JVM 调优呢？

- Heap 内存（老年代）持续上涨达到设置的最大内存值；
- Full GC 次数频繁；
- GC 停顿时间过长（超过 1 秒）；
- 应用出现 OutOfMemory 等内存异常；
- 应用中有使用本地缓存且占用大量内存空间；
- 系统吞吐量与响应性能不高或下降。

10.11.2 JVM 调优的目标

- 延迟：GC 低停顿和 GC 低频率；
- 低内存占用；
- 高吞吐量；

10.11.3 JVM 调优量化目标

- Heap 内存使用率 $\leq 70\%$ ；
- Old generation 内存使用率 $\leq 70\%$ ；
- avg pause ≤ 1 秒；
- Full gc 次数 0 或 avg pause interval ≥ 24 小时；

10.11.4 JVM 调优的步骤

- 分析 GC 日志及 dump 文件，判断是否需要优化，确定瓶颈问题点；
- 确定 JVM 调优量化目标；
- 确定 JVM 调优参数（根据历史 JVM 参数来调整）；
- 依次调优内存、延迟、吞吐量等指标；
- 对比观察调优前后的差异；
- 不断的分析和调整，直到找到合适的 JVM 参数配置；
- 找到最合适的参数，将这些参数应用到所有服务器，并进行后续跟踪。

10.11.5 常见的 JVM 参数

堆栈配置相关

```
-Xmx3550m -Xms3550m -Xmn2g -Xss128k
```

```
-XX:MaxPermSize=16m -XX:NewRatio=4 -XX:SurvivorRatio=4 -XX:MaxTenuringThreshold=0
```

- -Xmx3550m : 最大堆大小为 3550m。
- -Xms3550m : 设置初始堆大小为 3550m。
- -Xmn2g : 设置年轻代大小为 2g。
- -Xss128k : 每个线程的堆栈大小为 128k。
- -XX:MaxPermSize : 设置持久代大小为 16m
- -XX:NewRatio=4: 设置年轻代 (包括 Eden 和两个 Survivor 区) 与年老代的比值 (除去持久代) 。
- -XX:SurvivorRatio=4 : 设置年轻代中 Eden 区与 Survivor 区的大小比值。设置为 4 , 则两个 Survivor 区与一个 Eden 区的比值为 2:4 , 一个 Survivor 区占整个年轻代的 1/6
- -XX:MaxTenuringThreshold=0 : 设置垃圾最大年龄。如果设置为 0 的话 , 则年轻代对象不经过 Survivor 区 , 直接进入年老代。

垃圾收集器相关

```
-XX:+UseParallelGC
```

```
-XX:ParallelGCThreads=20
```

```
-XX:+UseConcMarkSweepGC
```

```
-XX:CMSFullGCsBeforeCompaction=5
```

```
-XX:+UseCMSCompactAtFullCollection :
```

```
-XX:+UseConcMarkSweepGC
```

- -XX:+UseParallelGC : 选择垃圾收集器为并行收集器。
- -XX:ParallelGCThreads=20 : 配置并行收集器的线程数
- -XX:+UseConcMarkSweepGC : 设置年老代为并发收集。
- -XX:CMSFullGCsBeforeCompaction : 由于并发收集器不对内存空间进行压缩、整理 , 所以运行一段时间以后会产生“碎片” , 使得运行效率降低。此值设置运行多少次 GC 以后对内存空间进行压缩、整理。
- -XX:+UseCMSCompactAtFullCollection : 打开对年老代的压缩。可能会影响性能 , 但是可以消除碎片
- -XX:+UseConcMarkSweepGC 使用 CMS 垃圾收集器

辅助信息

```
-XX:+PrintGC
```

```
-XX:+PrintGCDetails
```

10.11.6 常用调优策略

- 选择合适的垃圾回收器
- 调整内存大小(垃圾收集频率非常频繁,如果是内存太小,可适当调整内存大小)
- 调整内存区域大小比率(某一个区域的 GC 频繁,其他都正常。)
- 调整对象升老年代的年龄(老年代频繁 GC,每次回收的对象很多。)
- 调整大对象的标准(老年代频繁 GC,每次回收的对象很多,而且单个对象的体积都比较大。)
- 调整 GC 的触发时机(CMS, G1 经常 Full GC,程序卡顿严重。)
- 调整 JVM 本地内存大小(GC 的次数、时间和回收的对象都正常,堆内存空间充足,但是报 OOM)

10.12 数据库分库分表的缺点是啥？

1. 事务问题,已经不可以用本地事务了,需要用分布式事务。
2. 跨节点Join的问题:解决这一问题可以分两次查询实现
3. 跨节点的 count,order by,group by 以及聚合函数问题:分别在各个节点上得到结果后在应用程序端进行合并。
4. ID 问题:数据库被切分后,不能再依赖数据库自身的主键生成机制啦,最简单可以考虑 UUID
5. 跨分片的排序分页问题(后台加大 pagesize 处理?)

10.13 分布式事务如何解决?TCC 了解?

分布式事务:

就是指事务的参与者、支持事务的服务器、资源服务器以及事务管理器分别位于不同的分布式系统的不同节点之上。简单来说,分布式事务指的就是分布式系统中的事务,它的存在就是为了保证不同数据库节点的数据一致性。

聊到分布式事务,需要知道这两个基本理论哈。

- CAP 理论
- BASE 理论

CAP 理论

- 一致性(C : Consistency) : 一致性是指数据在多个副本之间能否保持一致的特性。例如一个数据在某个分区节点更新之后, 在其他分区节点读出来的数据也是更新之后的数据。
- 可用性(A : Availability) : 可用性是指系统提供的服务必须一直处于可用的状态, 对于用户的每一个操作请求总是能够在有限的时间内返回结果。这里的重点是"有限时间内"和"返回结果"。
- 分区容错性 (P:Partition tolerance) : 分布式系统在遇到任何网络分区故障的时候, 仍然需要能够保证对外提供满足一致性和可用性的服务。

BASE 理论

它是对 CAP 中 AP 的一个扩展, 对于我们的业务系统, 我们考虑牺牲一致性来换取系统的可用性和分区容错性。BASE 是 **Basically Available , Soft state, 和 Eventually consistent** 三个短语的缩写。

- Basically Available(基本可用) : 通过支持局部故障而不是系统全局故障来实现的。如将用户分区在 5 个数据库服务器上, 一个用户数据库的故障只影响这台特定主机那 20% 的用户, 其他用户不受影响。
- Soft State(软状态): 状态可以有一段时间不同步
- Eventually Consistent(最终一致): 最终数据是一致的就可以了, 而不是时时保持强一致。

分布式事务的几种解决方案 :

- 2PC(二阶段提交)方案、3PC
- TCC (Try、Confirm、Cancel)
- 本地消息表
- 最大努力通知
- seata 事务

TCC (补偿机制)

TCC 采用了补偿机制, 其核心思想是 : 针对每个操作, 都要注册一个与其对应的确认和补偿 (撤销) 操作。TCC (Try-Confirm-Cancel) 包括三段流程 :

- try 阶段 : 尝试去执行, 完成所有业务的一致性检查, 预留必须的业务资源。
- Confirm 阶段 : 该阶段对业务进行确认提交, 不做任何检查, 因为 try 阶段已经检查过了, 默认 Confirm 阶段是不会出错的。

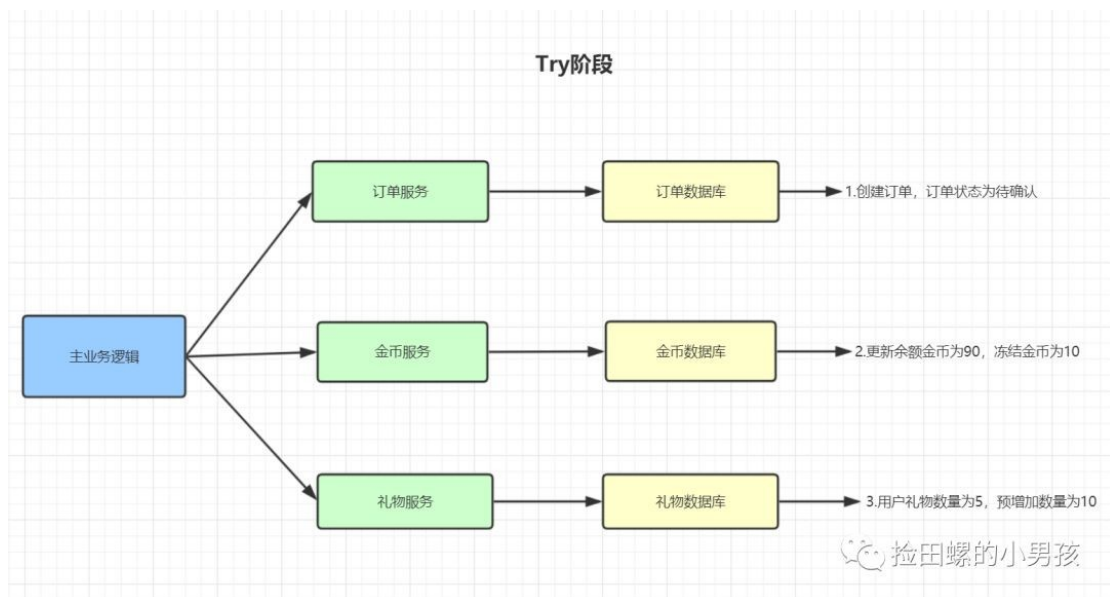
- Cancel 阶段：若业务执行失败，则进入该阶段，它会释放 try 阶段占用的所有业务资源，并回滚 Confirm 阶段执行的所有操作。

下面再拿用户下单购买礼物作为例子来模拟 TCC 实现分布式事务的过程：

假设用户 A 余额为 100 金币，拥有的礼物为 5 朵。A 花了 10 个金币，下订单，购买 10 朵玫瑰。余额、订单、礼物都在不同数据库。

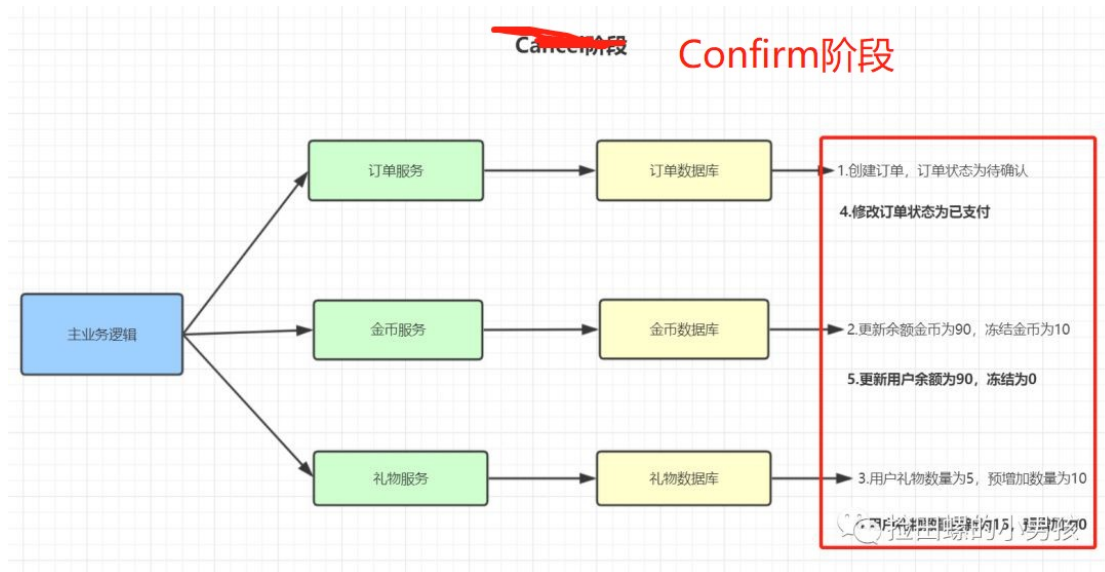
TCC 的 Try 阶段：

- 生成一条订单记录，订单状态为待确认。
- 将用户 A 的账户金币中余额更新为 90，冻结金币为 10（预留业务资源）
- 将用户的礼物数量为 5，预增加数量为 10。
- Try 成功之后，便进入 Confirm 阶段
- Try 过程发生任何异常，均进入 Cancel 阶段



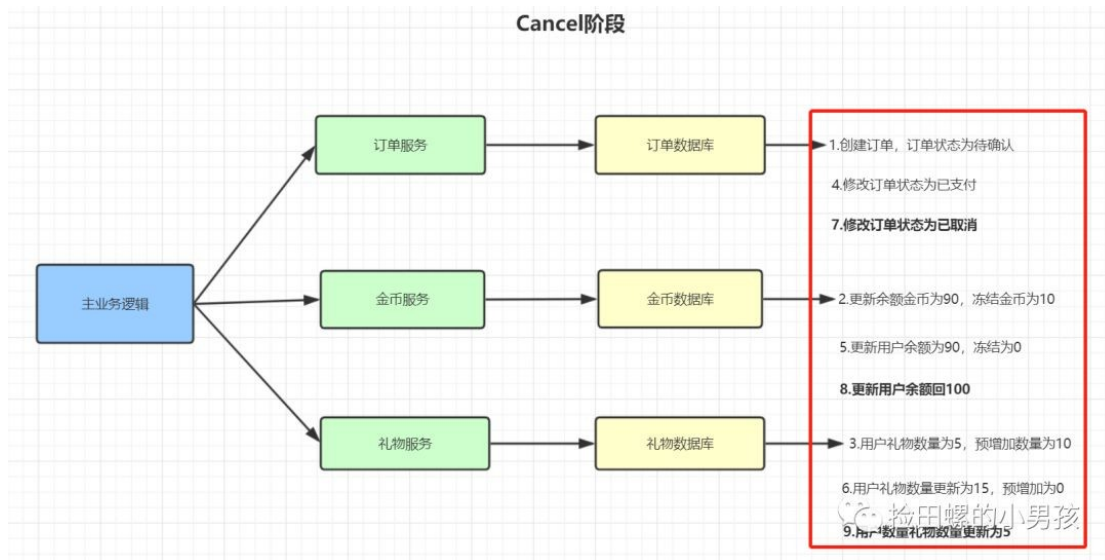
TCC 的 Confirm 阶段：

- 订单状态更新为已支付
- 更新用户余额为 90，可冻结为 0
- 用户礼物数量更新为 15，预增加为 0
- Confirm 过程发生任何异常，均进入 Cancel 阶段
- Confirm 过程执行成功，则该事务结束



TCC 的 Cancel 阶段：

- 修改订单状态为已取消
- 更新用户余额回 100
- 更新用户礼物数量为 5



- TCC 的优点是可以自定义数据库操作的粒度，降低了锁冲突，可以提升性能
- TCC 的缺点是应用侵入性强，需要根据网络、系统故障等不同失败原因实现不同的回滚策略，实现难度大，一般借助 TCC 开源框架，ByteTCC，TCC-transaction，Himly。

大家有兴趣可以看下我之前这篇文章哈：

10.14 RocketMQ 如何保证消息的准确性和安全性？

我个人理解的话，这道题换汤不换药，就是为如何保证 RocketMQ 不丢消息，保证不重复消费，消息有序性，消息堆积的处理。

消息不丢失的话，即从生产者、存储端、消费端去考虑

大家可以看下我之前这篇文章哈：

[消息队列经典十连问](#)

10.15. 三个数求和

给你一个包含 n 个整数的数组 `nums`，判断 `nums` 中是否存在三个元素

a, b, c ，使得 $a + b + c = 0$ ？请你找出所有和为 0 且不重复的三元组。

注意：答案中不可以包含重复的三元组

实例 1：

输入：`nums = [-1,0,1,2,-1,-4]`

输出：`[[-1,-1,2],[-1,0,1]]`

实例 2：

输入：`nums = [0]`

输出：`[]`

思路：

这道题可以先给数组排序，接着用左右双指针。

完整代码如下：

```
class Solution {  
    public List<List<Integer>> threeSum(int[] nums) {
```

```

        List<List<Integer>> result = new LinkedList<>();

        if(nums==null||nums.length<3){ //为空或者元素个数小于3，直接返回
            return result;
        }

        Arrays.sort(nums); //排序

        for(int i=0;i<nums.length-2;i++){ //遍历到倒数第三个，因为是三个数总和
            if(nums[i]>0){ //大于0可以直接跳出循环了
                break;
            }

            if(i>0&&nums[i]==nums[i-1]){ //过滤重复
                continue;
            }

            int left = i+1; //左指针
            int right = nums.length-1; //右指针
            int target = - nums[i]; //目标总和，是第i个的取反，也就是a+b+c=0,则b+c=-a即可

            while(left<right){
                if(nums[left]+ nums[right]==target){ //b+c=-a,满足a+b+c=0
                    result.add(Arrays.asList(nums[i],nums[left],nums[right]));
                    left++; //左指针右移
                    right--; //右指针左移
                    while(left<right&&nums[left]==nums[left-1]) left++; //继续左边过滤重复
                    while(left<right&&nums[right]==nums[right+1]) right--; //继续右边过滤重复
                }else if(nums[left]+ nums[right]<target){
                    left++; //小于目标值，需要右移，因为排好序是从小到大的
                }else{
                    right--;
                }
            }

        }

        return result;
    }
}

```

11.社招 21 连问 (关注公众号：捡田螺的小男孩。)

大家好，我是捡田螺的小男孩。有位朋友工作三年，去面试，给大家整理一下面试题，并附上答案。。如果有不正确的，欢迎指出哈，一起进步。更多干货，关注公众号：捡田螺的小男孩。



11.1 Mysql 索引在什么情况下会失效

- 查询条件包含 or，可能导致索引失效
- 如何字段类型是字符串，where 时一定用引号括起来，否则索引失效
- like 通配符可能导致索引失效。
- 联合索引，查询时的条件列不是联合索引中的第一个列，索引失效。
- 在索引列上使用 mysql 的内置函数，索引失效。
- 对索引列运算（如，+、-、*、/），索引失效。
- 索引字段上使用（!= 或者 < >，not in）时，可能会导致索引失效。
- 索引字段上使用 is null，is not null，可能导致索引失效。
- 左连接查询或者右连接查询查询关联的字段编码格式不一样，可能导致索引失效。
- mysql 估计使用全表扫描要比使用索引快,则不使用索引。

11.2 MySql 的存储引擎 InnoDB 与 MyISAM 的区别

- InnoDB 支持事务，MyISAM 不支持事务

- InnoDB 支持外键，MyISAM 不支持外键
- InnoDB 支持 MVCC(多版本并发控制)，MyISAM 不支持
- select count(*) from table 时，MyISAM 更快，因为它有一个变量保存了整个表的总行数，可以直接读取，InnoDB 就需要全表扫描。
- InnoDB 不支持全文索引，而 MyISAM 支持全文索引（5.7 以后的 InnoDB 也支持全文索引）
- InnoDB 支持表、行级锁，而 MyISAM 支持表级锁。
- InnoDB 表必须有主键，而 MyISAM 可以没有主键
- InnoDB 表需要更多的内存和存储，而 MyISAM 可被压缩，存储空间较小。
- InnoDB 按主键大小有序插入，MyISAM 记录插入顺序是，按记录插入顺序保存。
- InnoDB 存储引擎提供了具有提交、回滚、崩溃恢复能力的事务安全，与 MyISAM 比 InnoDB 写的效率差一些，并且会占用更多的磁盘空间以保留数据和索引

11.3 mysql 在项目中的优化场景，慢查询解决等

我们面对慢查询，首先想到的就是**加索引**。你可以给面试官描述一下，一个加了索引的 SQL，是怎么执行查找的，可以看下我之前这篇文章哈：

[阿里一面，给了几条 SQL，问需要执行几次树搜索操作？](#)

还有就是 order by，group by 原理，深分页等等，都跟慢查询息息相关，大家可以看下我以前的文章哈，都比较经典：

- [看一遍就理解：order by 详解](#)
- [看一遍就理解：group by 详解](#)
- [实战！聊聊如何解决 MySQL 深分页问题](#)
- [后端程序员必备：书写高质量 SQL 的 30 条建议](#)

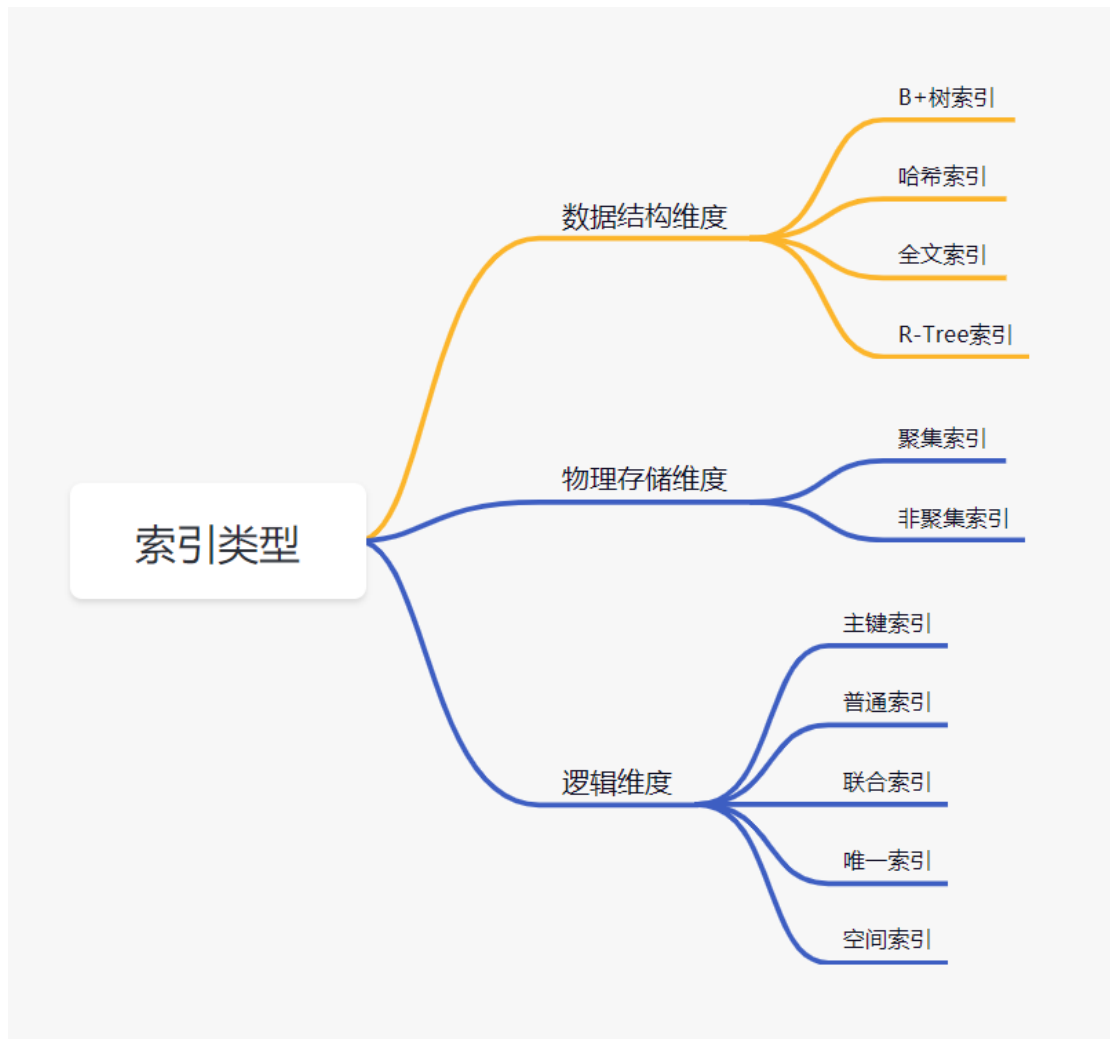
最后就是慢查询的排查解决手段：

打开慢查询日志 `slow_query_log`，确认 SQL 语句是否占用过多资源，用

`explain` 查询执行计划、对 `group by`、`order by`、`join` 等语句优化，如果数据量

实在太太，是否考虑分库分表等等。

11.4 Mysql 有什么索引，索引模型是什么



数据结构维度来讲的话，一般使用都是 B+ 树索引，大家想详细理解的话，可

以看我之前这篇文章哈：[MySQL 索引底层：B+树详解](#)

11.5 B-树与 B+树的区别？为什么不用红黑树

B-树与 B+树的区别：

- B-树内部节点是保存数据的;而 B+树内部节点是不保存数据的，只作索引作用，它的叶子节点才保存数据。
- B+树相邻的叶子节点之间是通过链表指针连起来的，B-树却不是。
- 查找过程中，B-树在找到具体的数值以后就结束，而 B+树则需要通过索引找到叶子结点中的数据才结束

- B-树中任何一个关键字出现且只出现在一个结点中，而 B+树可以出现多次。

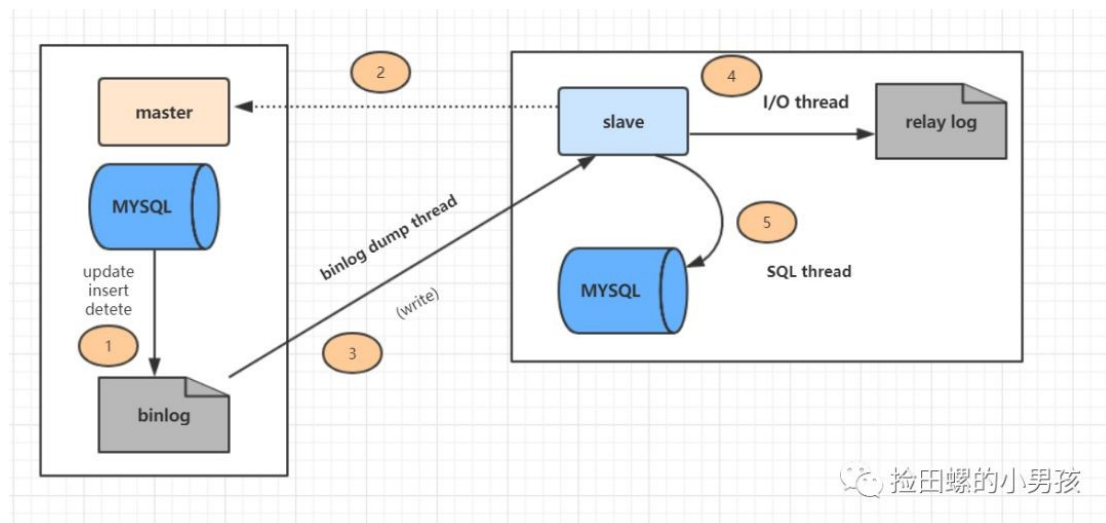
为什么索引结构默认使用 B+树，而不是 B-Tree，Hash 哈希，二叉树，红黑树？

- Hash 哈希，只适合等值查询，不适合范围查询。
- 一般二叉树，可能会特殊化为一个链表，相当于全表扫描。
- 红黑树，是一种特化的平衡二叉树，MySQL 数据量很大的时候，索引的体积也会很大，内存放不下的而从磁盘读取，树的层次太高的话，读取磁盘的次数就多了。
- B-Tree，叶子节点和非叶子节点都保存数据，相同的数据量，B+树更矮壮，也就是说，相同的数据量，B+树数据结构，查询磁盘的次数会更少。

11.6 Mysql 主从同步怎么做

大家要熟悉 MySQL 主从复制原理哈：

详细的主从复制过程如图：



上图主从复制过程分了五个步骤进行：

1. 主库的更新 SQL(update、insert、delete)被写到 binlog
2. 从库发起连接，连接到主库。
3. 此时主库创建一个 **binlog dump thread**，把 **binlog** 的内容发送到从库。
4. 从库启动之后，创建一个 **I/O 线程**，读取主库传过来的 **bin log** 内容并写入到 **relay log**

5. 从库还会创建一个 SQL 线程，从 `relay log` 里面读取内容，从 `ExecMasterLog_Pos` 位置开始执行读取到的更新事件，将更新内容写入到 `slave` 的 db

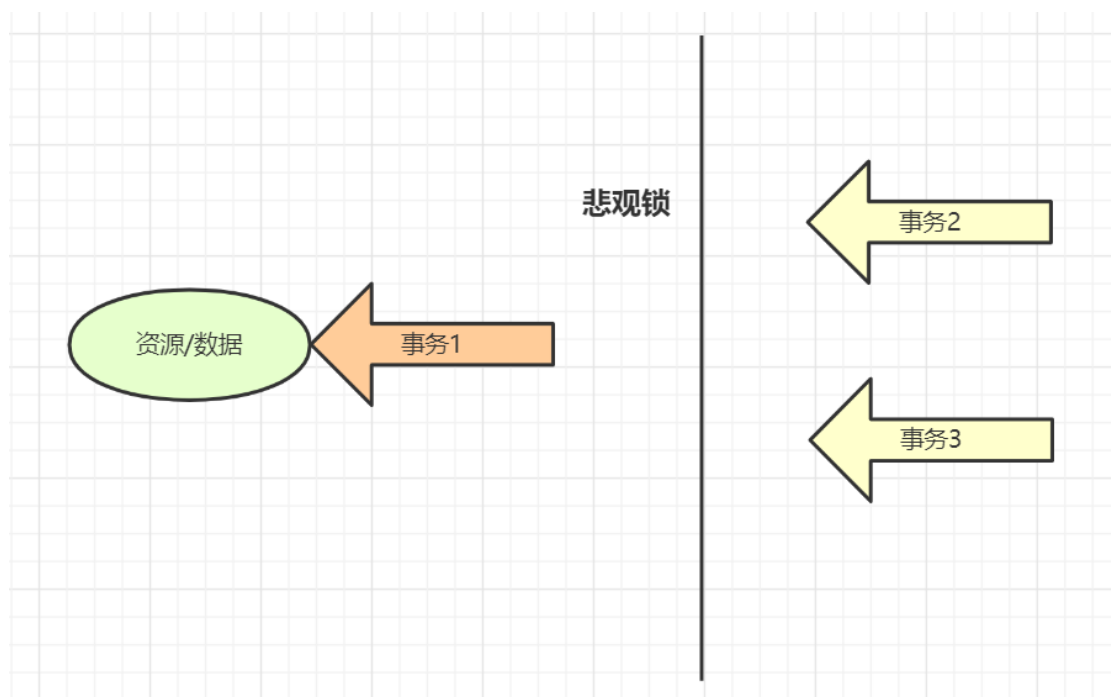
主从同步这块呢，还涉及到如何保证主从一致的、数据库主从延迟的原因与解决方案、数据库的高可用方案。

大家可以看下我最近的一篇总结哈：[面试必备：聊聊 MySQL 的主从](#)

11.7 乐观锁与悲观锁的区别？

悲观锁：

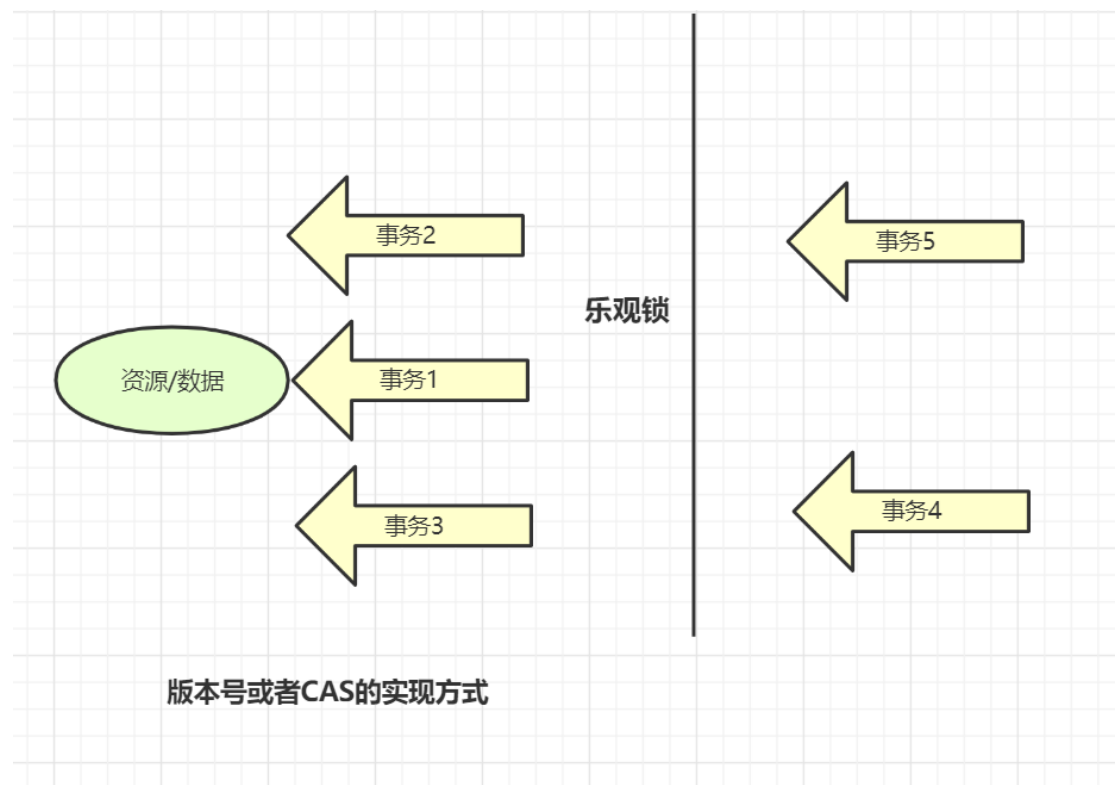
悲观锁她专一且缺乏安全感了，她的心只属于当前事务，每时每刻都担心着它心爱的数据可能被别的事务修改，所以一个事务拥有（获得）悲观锁后，其他任何事务都不能对数据进行修改啦，只能等待锁被释放才可以执行。



`select ...for update` 就是悲观锁一种实现。

乐观锁：

乐观锁的“乐观情绪”体现在，它认为数据的变动不会太频繁。因此，它允许多个事务同时对数据进行变动。实现方式：乐观锁一般会使用版本号机制或 CAS 算法实现。



之前用乐观锁解决过实战的并发问题，大家有兴趣可以加我微信，一起聊聊哈。

11.8 聊聊 binlog 日志

binlog 是归档日志，属于 **MySQL Server** 层的日志。可以实现**主从复制**和**数据恢复**两个作用。当需要恢复数据时，可以取出某个时间范围内的 binlog 进行重放恢复即可。

binlog 日志有三种格式，分别是 **statement**，**row** 和 **mixed**。

如果是 `statement` 格式，binlog 记录的是 **SQL 的原文**，他可能会导致主库不一致(主库和从库选的索引不一样时)。我们来分析一下。假设主库执行删除这个 SQL (其中 `a` 和 `create_time` 都有索引) 如下：

```
delete from t where a > '666' and create_time<'2022-03-01' limit 1;
```

我们知道，数据选择了 `a` 索引和选择 `create_time` 索引，最后 `limit 1` 出来的数据一般是不一样的。所以就会存在这种情况：在 binlog = `statement` 格式时，主库在执行这条 SQL 时，使用的是索引 `a`，而从库在执行这条 SQL 时，使用了索引 `create_time`。最后主从数据不一致了。

如何解决这个问题呢？

可以把 binlog 格式修改为 `row`。`row` 格式的 binlog 日志，记录的不是 **SQL 原文**，而是两个 `event:Table_map` 和 `Delete_rows`。`Table_map event` 说明要操作的表，`Delete_rows event` 用于定义要删除的行为，记录删除的具体行数。`row` 格式的 binlog 记录的就是要删除的主键 ID 信息，因此不会出现主从不一致的问题。

但是如果 SQL 删除 10 万行数据，使用 `row` 格式就会很占空间的，10 万条数据都在 binlog 里面，写 binlog 的时候也很耗 IO。但是 `statement` 格式的 binlog 可能会导致数据不一致，因此设计 MySQL 的大叔想了一个折中的方案，`mixed` 格式的 binlog。所谓的 `mixed` 格式其实就是 `row` 和 `statement` 格式混合使用，当 MySQL 判断可能数据不一致时，就用 `row` 格式，否则使用就用 `statement` 格式。

11.9 Redis 持久化有哪几种方式，怎么选？

既然它是基于内存的，如果 Redis 服务器挂了，数据就会丢失。为了避免数据丢失了，Redis 提供了两种持久化方式，**RDB** 和 **AOF**。

11.9.1 AOF 持久化

AOF (append only file) 持久化，采用日志的形式来记录每个写操作，追加到 AOF 文件的末尾。**Redis 默认情况是不开启 AOF 的**。重启时再重新执行 AOF 文件中的命令来恢复数据。它主要解决数据持久化的实时性问题。

AOF 是**执行完命令后才记录日志的**。为什么不先记录日志再执行命令呢？这是因为 Redis 在向 AOF 记录日志时，不会先对这些命令进行语法检查，如果先记录日志再执行命令，日志中可能记录了错误的命令，Redis 使用日志回复数据时，可能会出错。

正是因为执行完命令后才记录日志，所以不会阻塞当前的写操作。但是会存在**两个风险**：

1. 更执行完命令还没记录日志时，宕机了会导致数据丢失
2. AOF 不会阻塞当前命令，但是可能会阻塞下一个操作。

这两个风险最好的解决方案是折中妙用 **AOF 机制的三种写回策略**

appendfsync：

1. **always**，**同步写回**，每个子命令执行完，都立即将日志写回磁盘。
2. **everysec**，每个命令执行完，只是先把日志写到 AOF 内存缓冲区，每隔一秒同步到磁盘。
3. **no**：只是先把日志写到 AOF 内存缓冲区，有操作系统去决定何时写入磁盘。

always 同步写回，可以基本保证数据不丢失，**no** 策略则性能高但是数据可能会丢失，一般可以考虑折中选择 **everysec**。

如果接受的命令越来越多，AOF 文件也会越来越大，文件过大还是会带来性能问题。日志文件过大怎么办呢？**AOF 重写机制**！就是随着时间推移，AOF 文件会有一些冗余的命令如：无效命令、过期数据的命令等等，AOF 重写机制就是把它们合并为一个命令（类似批处理命令），从而达到**精简压缩空间**的目的。

AOF 重写会阻塞嘛？AOF 日志是由主线程会写的，而重写则不一样，重写过程是由后台子进程 **bgrewriteaof** 完成。

- **AOF 的优点**：数据的一致性和完整性更高，秒级数据丢失。
- **缺点**：相同的数据集，AOF 文件体积大于 RDB 文件。数据恢复也比较慢。

11.9.2 RDB

因为 AOF 持久化方式，如果操作日志非常多的话，**Redis 恢复就很慢**。有没有在宕机快速恢复的方法呢，有的，RDB！

RDB，就是把内存数据以**快照**的形式保存到磁盘上。和 AOF 相比，它记录的是某一时刻的数据，，并不是操作。

什么是快照？可以这样理解，给当前时刻的数据，拍一张照片，然后保存下来。

RDB 持久化，是指在指定的时间间隔内，执行指定次数的写操作，将内存中的数据集快照写入磁盘中，它是 Redis 默认的持久化方式。执行完操作后，在指定目录下会生成一个 dump.rdb 文件，Redis 重启的时候，通过加载 dump.rdb 文件来恢复数据。RDB 触发机制主要有以下几种：



RDB 通过 **bgsave** 命令的执行全量快照，可以避免阻塞主线程。**basave** 命令会 fork 一个子进程，然后该子进程会负责创建 RDB 文件，而服务器进程会继续处理命令请求

快照时，数据能修改嘛？Redis 接住操作系统的写时复制技术 (**copy-on-write** , **COW**) ,在执行快照的同时，正常处理写操作。

虽然 **bgsave** 执行不会阻塞主线程，但是频繁执行全量快照也会带来性能开销。比如 **bgsave** 子进程需要通过 fork 操作从主线程创建出来，创建后不会阻塞主线程，但是创建过程是会阻塞主线程的。可以做**增量快照**。

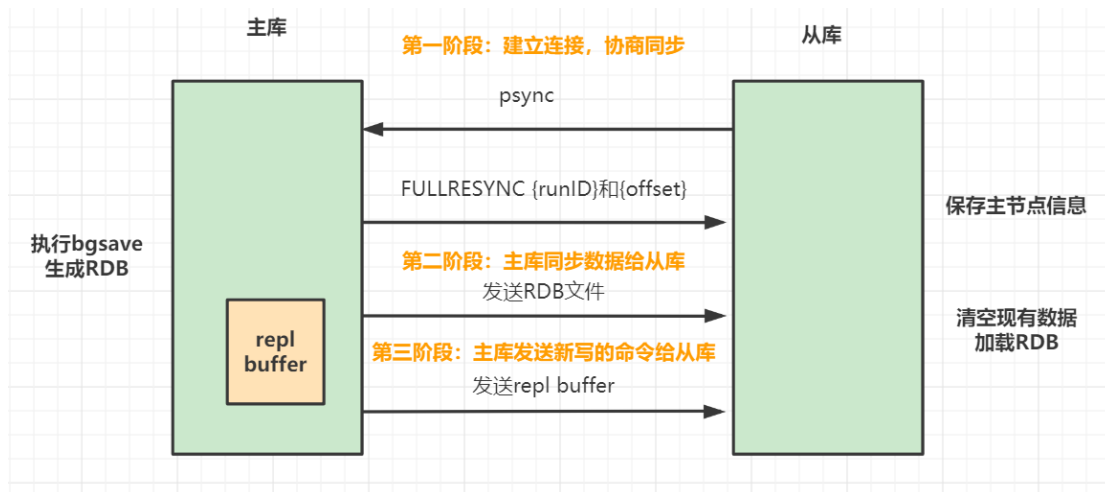
- **RDB 的优点**：与 AOF 相比，恢复大数据集的时候会更快，它适合大规模的数据恢复场景，如备份，全量复制等
- **缺点**：没办法做到实时持久化/秒级持久化。

Redis4.0 开始支持 **RDB 和 AOF 的混合持久化**，就是内存快照以一定频率执行，两次快照之间，再使用 AOF 记录这期间的所有命令操作。

11.9.3 如何选择 RDB 和 AOF

- 如果数据不能丢失，RDB 和 AOF 混用
- 如果只作为缓存使用，可以承受几分钟的数据丢失的话，可以只使用 RDB。
- 如果只使用 AOF，优先使用 **everysec** 的写回策略。

11.10 Redis 主从同步是怎样的过程？



Redis 主从同步包括三个阶段。

第一阶段：主从库间建立连接、协商同步。

- 从库向主库发送 `psync` 命令，告诉它要进行数据同步。
- 主库收到 `psync` 命令后,响应 `FULLRESYNC` 命令（它表示第一次复制采用的是全量复制），并带上主库 `runID` 和主库目前的复制进度 `offset`。

第二阶段：主库把数据同步到从库，从库收到数据后，完成本地加载。

- 主库执行 `bgsave` 命令，生成 `RDB` 文件，接着将文件发给从库。从库接收到 `RDB` 文件后，会先清空当前数据库，然后加载 `RDB` 文件。
- 主库把数据同步到从库的过程中，新来的写操作，会记录到 `replication buffer`。

第三阶段，主库把新写的命令，发送到从库。

- 主库完成 `RDB` 发送后，会把 `replication buffer` 中的修改操作发给从库，从库再重新执行这些操作。这样主从库就实现同步啦。

11.11 聊聊 Redis 的 `zset`，它是怎么实现的？

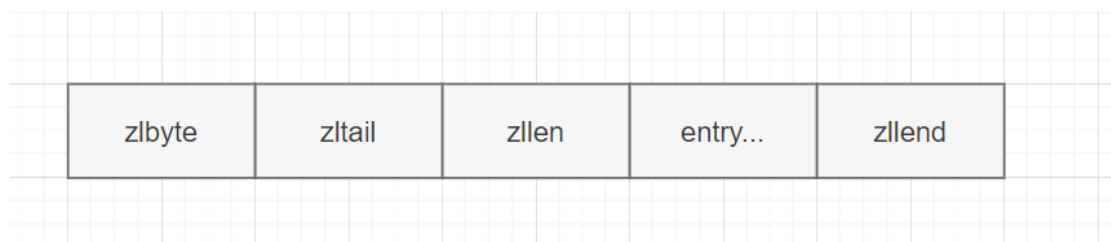
`zset` 是 Redis 常用数据类型之一，它的成员是有序排列的，一般用于排行榜类型的业务场景，比如 QQ 音乐排行榜、礼物排行榜等等。

- 它的简单格式举例：`zadd key score member [score member ...]`，`zrank key member`
- 它的底层内部编码：ziplist (压缩列表)、skiplist (跳跃表)

当 zset 满足以下条件时使用**压缩列表**：

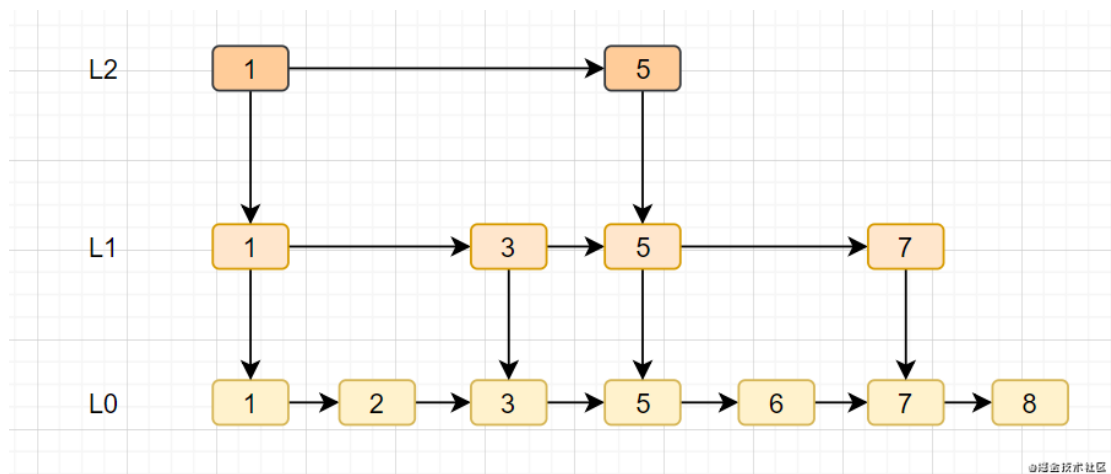
- 当成员的数量小于 128 个；
- 每个 member (成员) 的字符串长度都小于 64 个字节。

压缩列表做简单介绍，它由以下五部分组成



- zlbytes 是一个无符号整数，表示当前 ziplist 占用的总字节数；
- zltail 指的是压缩列表尾部元素相对于压缩列表起始元素的偏移量。
- zllen 指 ziplist 中 entry 的数量。当 zllen 比 $2^{16} - 2$ 大时，需要完全遍历 entry 列表来获取 entry 的总数目。
- entry 用来存放具体的数据项 (score 和 member)，长度不定，可以是字节数组或整数，entry 会根据成员的数量自动扩容。-zllend 是一个单字节的特殊值，等于 255，起到标识 ziplist 内存结束点的作用。

skiplist (跳跃表) 在链表的基础上，增加了多级索引，通过索引位置的几个跳转，实现数据的快速定位,其插入、删除、查找的时间复杂度均为 $O(\log N)$ 。



11.12 Redis 过期策略和内存淘汰策略



11.12.1 Redis 的过期策略

我们在 set key 的时候，可以给它设置一个过期时间，比如 `expire key 60`。指定这 key 60s 后过期，60s 后，redis 是如何处理的嘛？我们先来介绍几种过期策略哈：

一般有**定时过期**、**惰性过期**、**定期过期**三种。

- **定时过期**

每个设置过期时间的 key 都需要创建一个定时器，到过期时间就会立即对 key 进行清除。该策略可以立即清除过期的数据，对内存很友好；但是会占用大量的 CPU 资源去处理过期的数据，从而影响缓存的响应时间和吞吐量。

- **惰性过期**

只有当访问一个 key 时，才会判断该 key 是否已过期，过期则清除。该策略可以最大化地节省 CPU 资源，但对内存非常不友好。极端情况可能出现大量的过期 key 没有再次被访问，从而不会被清除，占用大量内存。

- **定期过期**

每隔一定的时间，会扫描一定数量的数据库的 expires 字典中一定数量的 key，并清除其中已过期的 key。该策略是前两者的一个折中方案。通过调整定时扫描的时间间隔和每次扫描的限定耗时，可以在不同情况下使得 CPU 和内存资源达到最优的平衡效果。

expires 字典会保存所有设置了过期时间的 key 的过期时间数据，其中，key 是指向键空间中的某个键的指针，value 是该键的毫秒精度的 UNIX 时间戳表示的过期时间。键空间是指该 Redis 集群中保存的所有键。

Redis 中同时使用了**惰性过期**和**定期过期**两种过期策略。

- 假设 Redis 当前存放 30 万个 key，并且都设置了过期时间，如果你每隔 100ms 就去检查这全部的 key，CPU 负载会特别高，最后可能会挂掉。
- 因此，redis 采取的是定期过期，每隔 100ms 就随机抽取一定数量的 key 来检查和删除的。
- 但是呢，最后可能会有很多已经过期的 key 没被删除。这时候，redis 采用惰性删除。在你获取某个 key 的时候，redis 会检查一下，这个 key 如果设置了过期时间并且已经过期了，此时就会删除。

但是呀，如果定期删除漏掉了很多过期的 key，然后也没走惰性删除。就会有
很多过期 key 积在内存内存，直接会导致内存爆的。或者有些时候，业务量大
起来了，redis 的 key 被大量使用，内存直接不够了，运维小哥哥也忘记加大
内存了。难道 redis 直接这样挂掉？不会的！Redis 用 8 种内存淘汰策略保护
自己~

11.12.2 Redis 内存淘汰策略

- volatile-lru：当内存不足以容纳新写入数据时，从设置了过期时间的 key 中使用 LRU（最近最少使用）算法进行淘汰；
- allkeys-lru：当内存不足以容纳新写入数据时，从所有 key 中使用 LRU（最近最少使用）算法进行淘汰。
- volatile-lfu：4.0 版本新增，当内存不足以容纳新写入数据时，在过期的 key 中，使用 LFU 算法进行删除 key。
- allkeys-lfu：4.0 版本新增，当内存不足以容纳新写入数据时，从所有 key 中使用 LFU 算法进行淘汰；
- volatile-random：当内存不足以容纳新写入数据时，从设置了过期时间的 key 中，随机淘汰数据；。
- allkeys-random：当内存不足以容纳新写入数据时，从所有 key 中随机淘汰数据。
- volatile-ttl：当内存不足以容纳新写入数据时，在设置了过期时间的 key 中，根据过期时间进行淘汰，越早过期的优先被淘汰；
- noeviction：默认策略，当内存不足以容纳新写入数据时，新写入操作会报错。

11.13. Hashmap 是怎样实现的？为什么要用红黑树，而不用平衡二叉树？为什么在 1.8 中链表大于 8 时会转红黑树？

HashMap 是线性安全的嘛？如何保证安全？

11.13.1 Hashmap 是怎样实现的？

- JDK1.7 Hashmap 的底层数据结构是数组+链表
- JDK1.8 Hashmap 的底层数据结构是数组+链表+红黑树

数据元素通过映射关系，即散列函数，映射到桶数组对应索引的位置，插入该位置时，如果发生冲突，从冲突的位置拉一个链表，把冲突元素放到链表。如果链表长度>8 且数组大小>=64，链表转为红黑树 如果红黑树节点个数<6，转为链表。

11.13.2 为什么要用红黑树，为什么不用二叉树？为什么不用平衡二叉树？

为什么不用二叉树？

红黑树是一种平衡的二叉树，其插入、删除、查找的最坏时间复杂度都为 $O(\log n)$ ，避免了二叉树最坏情况下的 $O(n)$ 时间复杂度。

为什么不用平衡二叉树？

平衡二叉树是比红黑树更严格的平衡树，为了保持平衡，需要旋转的次数更多，也就是说平衡二叉树保持平衡的效率更低，所以平衡二叉树插入和删除的效率比红黑树要低。

11.13.3 为什么在 1.8 中链表大于 8 时会转红黑树？

红黑树的平均查找长度是 $\log(n)$ ，如果长度为 8，平均查找长度为 $\log(8)=3$ ，链表的平均查找长度为 $n/2$ ，当长度为 8 时，平均查找长度为 $8/2=4$ ，这才有转换成树的必要；链表长度如果是小于等于 6， $6/2=3$ ，而 $\log(6)=2.6$ ，虽然速度也很快的，但是转化为树结构和生成树的时间并不会太短。

11.13.4 HashMap 是线性安全的嘛？如何保证安全？

HashMap 不是线程安全的，多线程下扩容死循环。可以使用 Hashtable、Collections.synchronizedMap、以及 ConcurrentHashMap 可以实现线程安全。

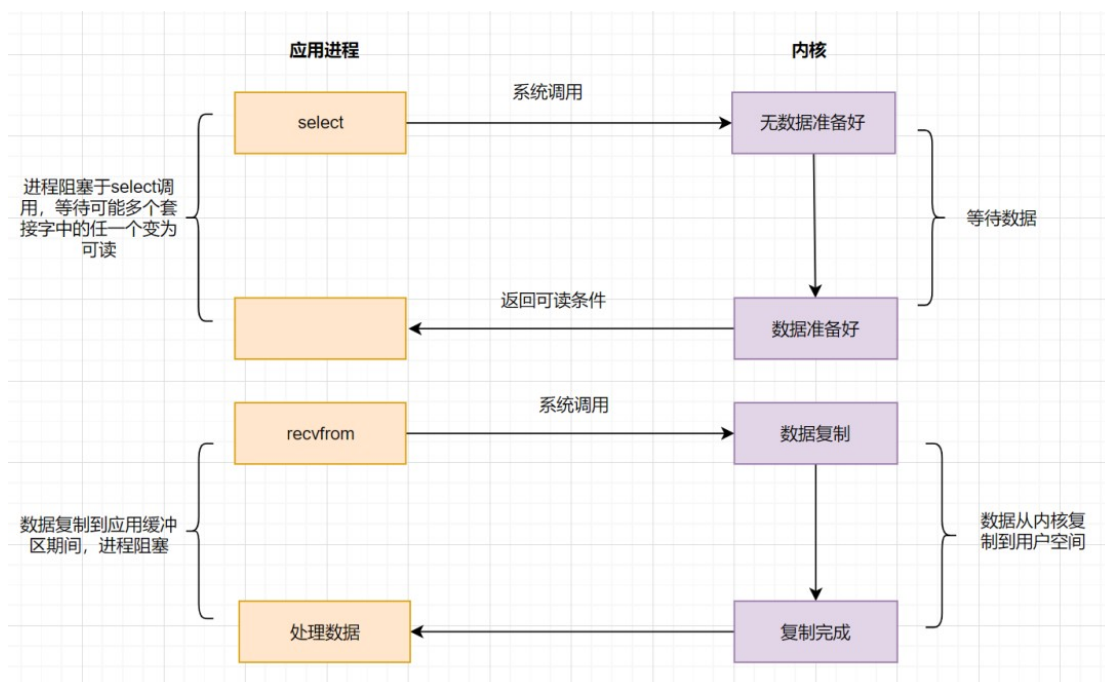
- Hashtable 是在每个方法加上 synchronized 关键字，粒度比较大；

- Collections.synchronizedMap 是使用 Collections 集合工具的内部类，通过传入 Map 封装出一个 SynchronizedMap 对象，内部定义了一个对象锁，方法内通过对象锁实现；
- ConcurrentHashMap 在 jdk1.7 中使用分段锁，在 jdk1.8 中使用 CAS+synchronized。

11.14 select 和 epoll 的区别

11.14.1 IO 多路复用之 select

应用进程通过调用 select 函数，可以同时监控多个 fd，在 select 函数监控的 fd 中，只要有任何一个数据状态准备就绪了，select 函数就会返回可读状态，这时应用进程再发起 recvfrom 请求去读取数据。



非阻塞 IO 模型 (NIO) 中，需要 $N (N \geq 1)$ 次轮询系统调用，然而借助 select 的 IO 多路复用模型，只需要发起一次询问就够了,大大优化了性能。

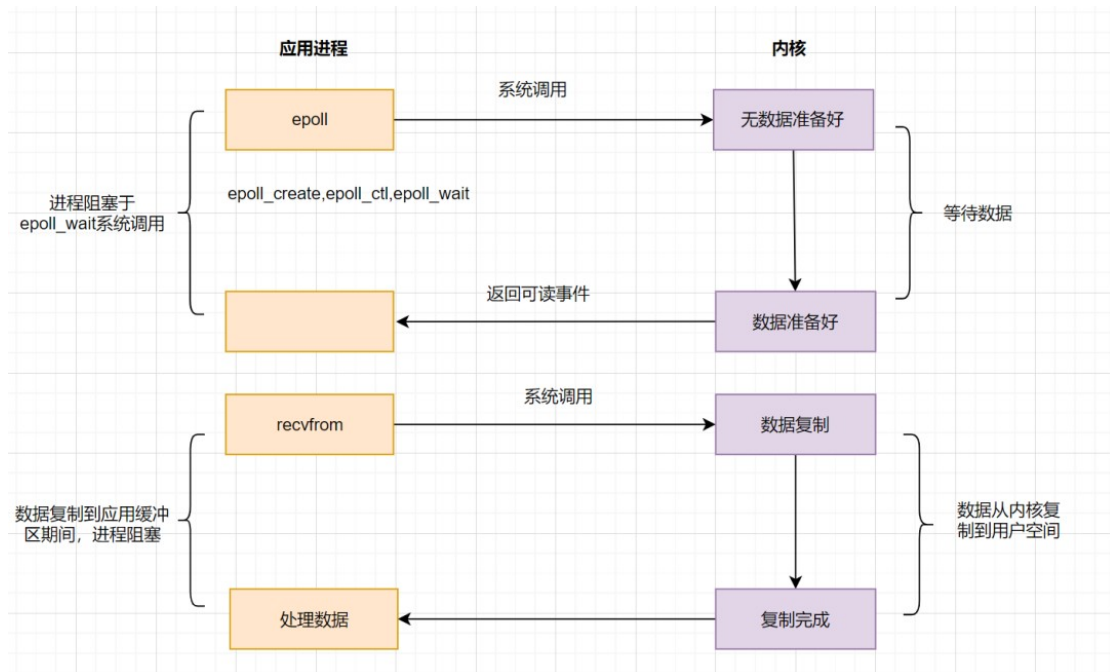
但是呢，select 有几个缺点：

- 监听的 IO 最大连接数有限，在 Linux 系统上一般为 1024。

- select 函数返回后，是通过遍历 fdset，找到就绪的描述符 fd。（仅知道有 I/O 事件发生，却不知是哪几个流，所以遍历所有流）
- 因为存在连接数限制，所以后来又提出了 poll。与 select 相比，poll 解决了连接数限制问题。但是呢，select 和 poll 一样，还是需要通过遍历文件描述符来获取已经就绪的 socket。如果同时连接的大量客户端，在一时刻可能只有极少处于就绪状态，伴随着监视的描述符数量的增长，效率也会线性下降。

11.14.2 IO 多路复用之 epoll

为了解决 select/poll 存在的问题，多路复用模型 epoll 诞生，它采用事件驱动来实现，流程图如下：



epoll 先通过 `epoll_ctl()` 来注册一个 fd（文件描述符），一旦基于某个 fd 就绪时，内核会采用回调机制，迅速激活这个 fd，当进程调用 `epoll_wait()` 时便得到通知。这里去掉了遍历文件描述符的坑爹操作，而是采用监听事件回调的机制。这就是 epoll 的亮点。

一下 select、poll、epoll 的区别

	select	poll	epoll
底 层 数 据 结 构	数组	链表	红黑树和双链表
获 取 就 绪 的 fd	遍历	遍历	事件回调
事 件 复 杂 度	O(n)	O(n)	O(1)
最 大 连 接 数	1024	无限制	无限制
fd 数 据 拷 贝	每次调用 select，需 要将 fd 数据从用户空 间拷贝到内核空间	每次调用 poll，需要 将 fd 数据从用户空 间拷贝到内核空间	使用内存映射 (mmap)， 不需要从用户空间频繁拷 贝 fd 数据到内核空间

大家可以看我这篇文章哈：[看一遍就理解：IO 模型详解](#)

11.15 http 与 https 的区别，https 的原理，如何加密的？

http 与 https 的区别

思路: 这道题实际上考察的知识点是 HTTP 与 HTTPS 的区别，这个知识点非常重要，可以从安全性、数据是否加密、默认端口等这几个方面去回答哈。其实，当你理解 HTTPS 的整个流程，就可以很好回答这个问题啦。

HTTP，即超文本传输协议，是一个基于 TCP/IP 通信协议来传递明文数据的协议。HTTP 会存在这几个问题：

- 请求信息是明文传输，容易被窃听截取。
- 没有验证对方身份，存在被冒充的风险
- 数据的完整性未校验，容易被中间人篡改

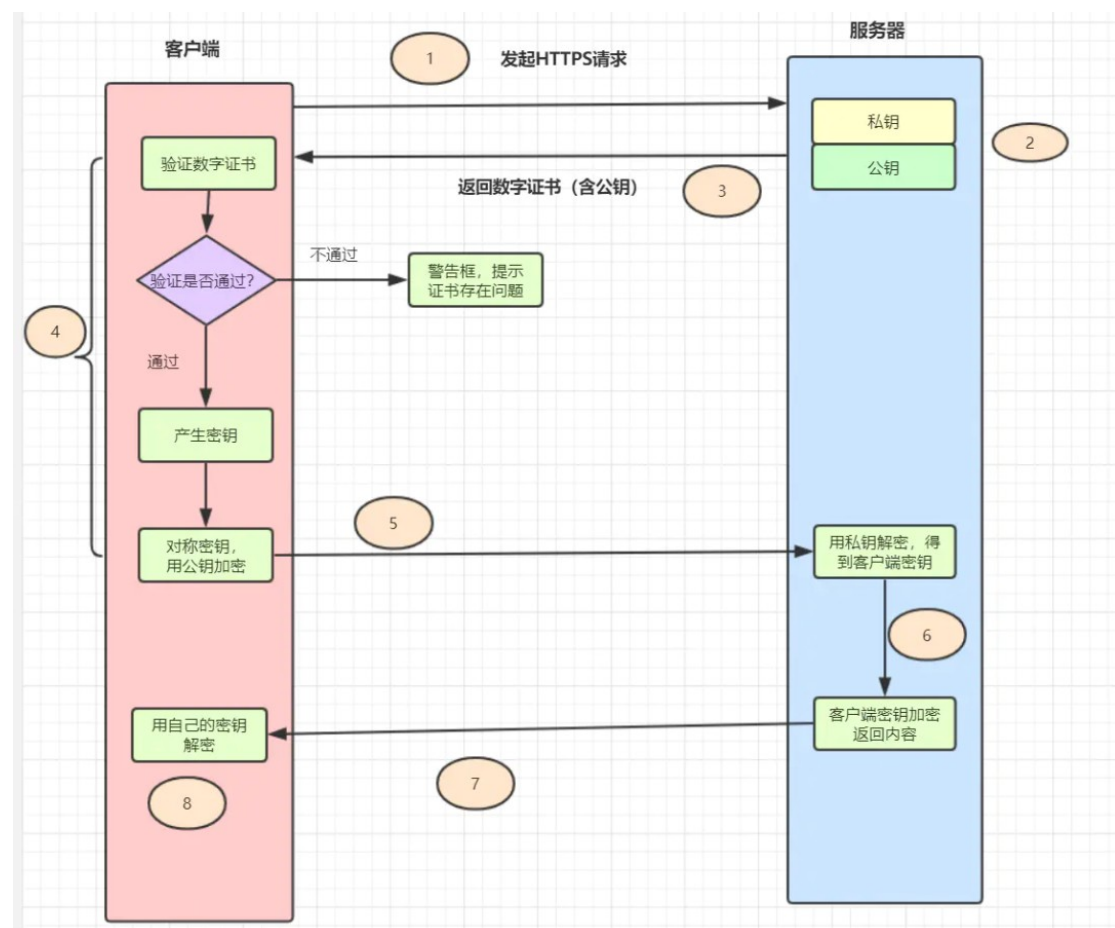
为了解决 Http 存在的问题，Https 出现啦。

HTTPS= HTTP+SSL/TLS，可以理解 Https 是身披 SSL(Secure Socket Layer，安全套接层)的 HTTP。

HTTP + HTTPS 的区别

	Http	Https
安全性	不安全	安全
默认端口	80	443
资源消耗	较少	消耗资源较多
是否需要证书	不需要	需要
报文是否加密	明文	密文

https 的原理，如何加密的



- 客户端发起 Https 请求，连接到服务器的 443 端口。
- 服务器必须要有一套数字证书（证书内容有公钥、证书颁发机构、失效日期等）。
- 服务器将自己的数字证书发送给客户端（公钥在证书里面，私钥由服务器持有）。

- 客户端收到数字证书之后，会验证证书的合法性。如果证书验证通过，就会生成一个随机的对称密钥，用证书的公钥加密。
- 客户端将公钥加密后的密钥发送到服务器。
- 服务器接收到客户端发来的密文密钥之后，用自己之前保留的私钥对其进行非对称解密，解密之后就得到客户端的密钥，然后用客户端密钥对返回数据进行对称加密，酱紫传输的数据都是密文啦。
- 服务器将加密后的密文返回到客户端。
- 客户端收到后，用自己的密钥对其进行对称解密，得到服务器返回的数据。

11.16 Raft 算法原理

Raft 算法是分布式系统开发首选的共识算法，它通过“一切以领导者为准”的方式，实现一系列值的共识和各节点日志的一致。Raft 算法一共涉及三种角色（Follower、Candidate、Leader）和两个过程（Leader 选举和日志复制）。

16.1 Raft 角色

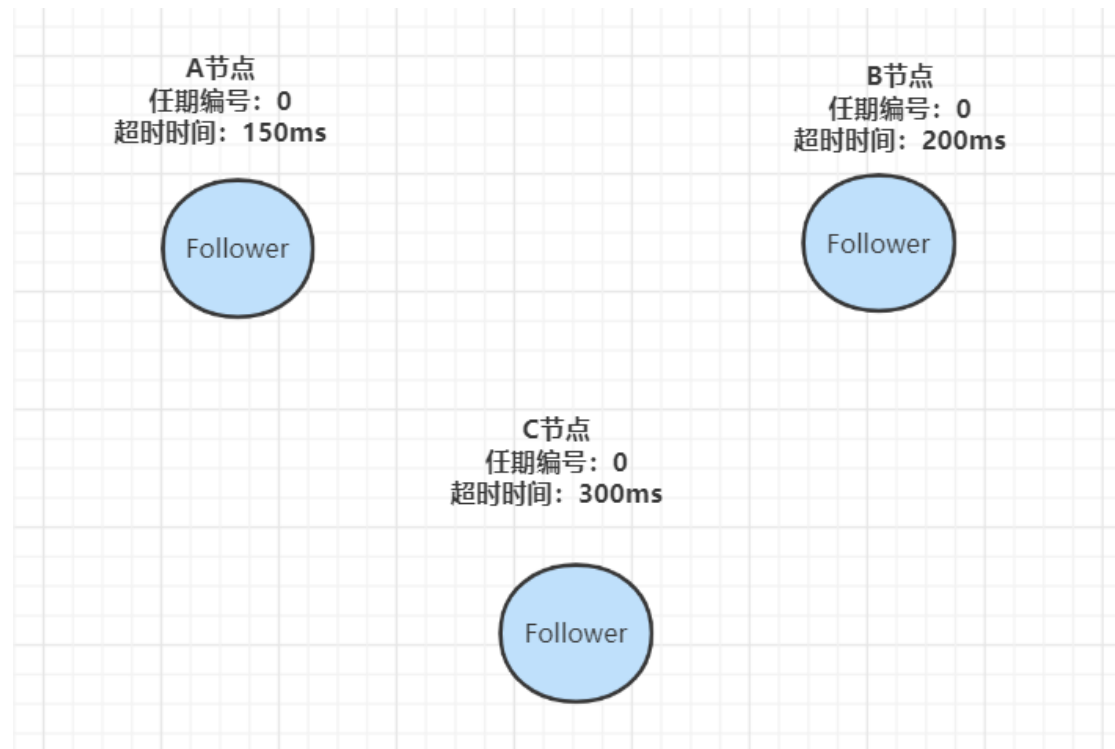
跟随者 (Follower)：，默默地接收和处理来自 Leader 的消息，当等待 Leader 心跳信息超时的时候，就主动站出来，推荐自己当候选人 (Candidate)。

候选人 (Candidate)：向其他节点发送投票请求，通知其他节点来投票，如果赢得了大多数 ($N/2+1$) 选票，就晋升领导 (Leader)。

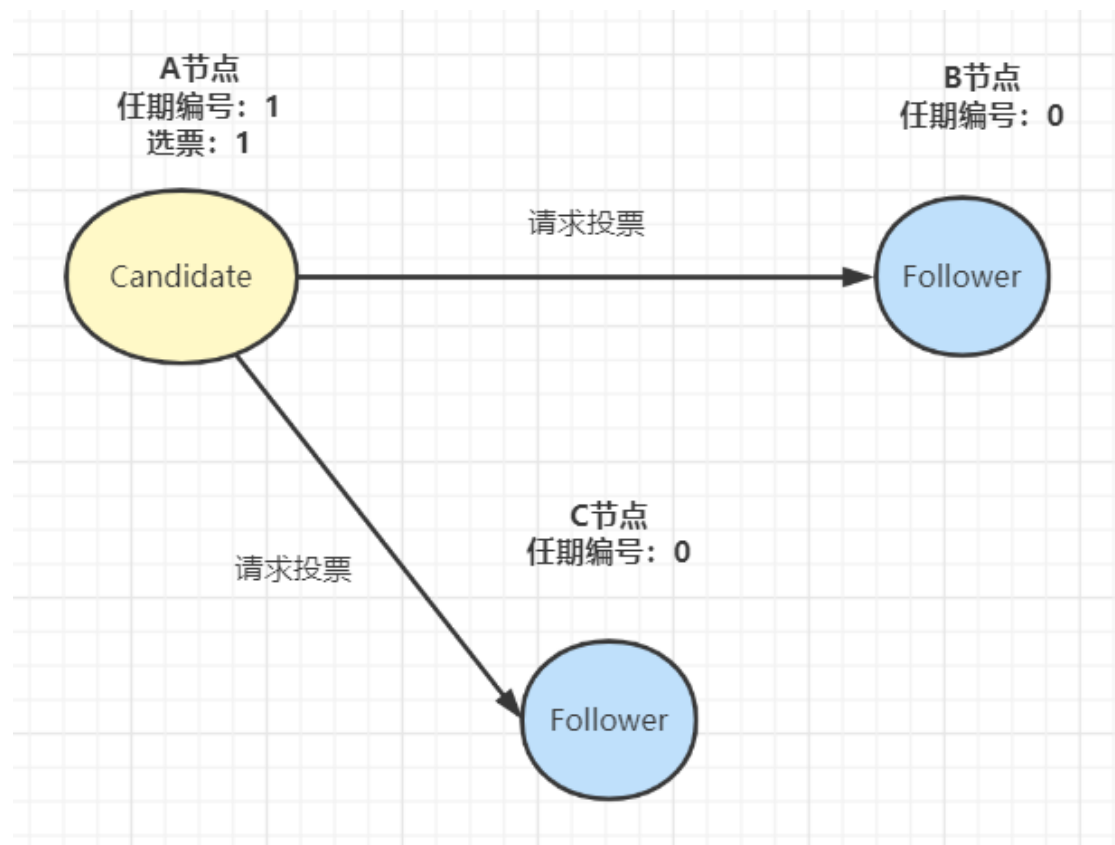
领导者 (Leader)：负责处理客户端请求，进行日志复制等操作，每一轮选举的目标就是选出一个领导者；领导者会不断地发送心跳信息，通知其他节点“我是领导者，我还活着，你们不要发起新的选举，不用找个新领导者来替代我。”

11.16.2 领导选举过程

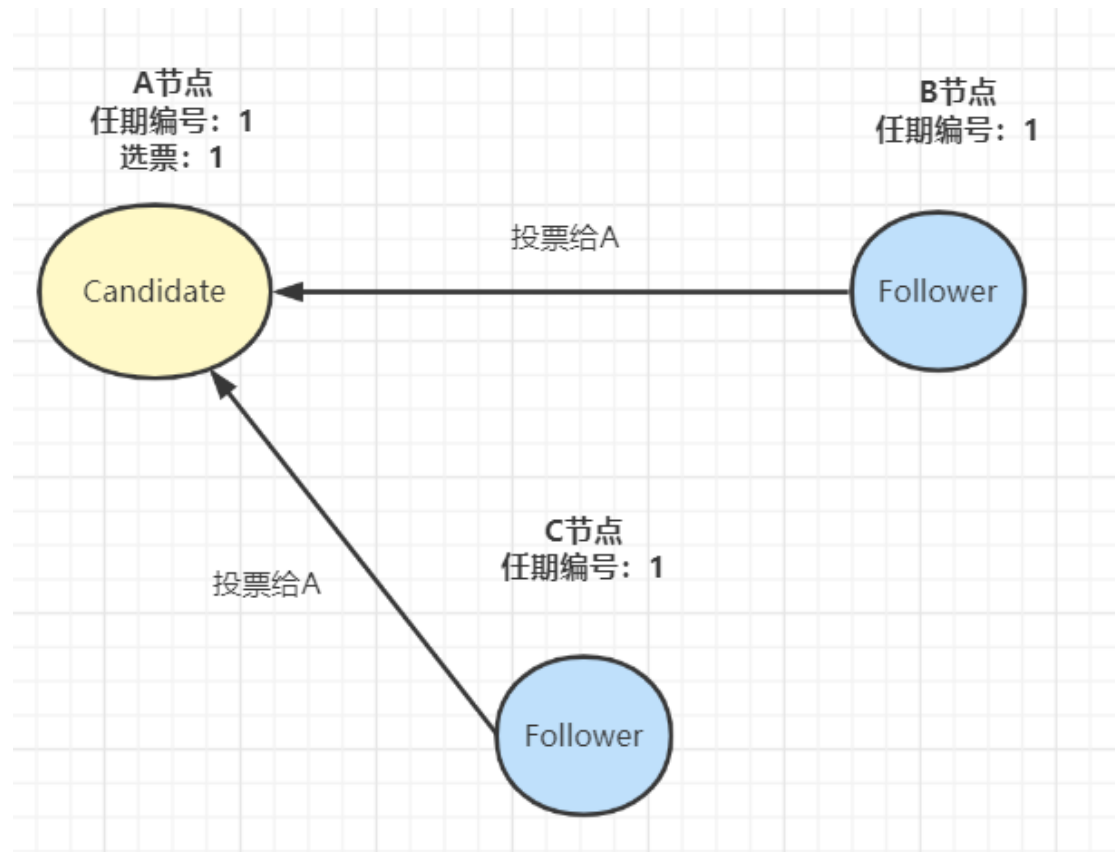
1. 在初始状态下，集群中所有的节点都是 Follower 状态，并被设定一个随机选举超时时间（一般 150ms-300ms）：



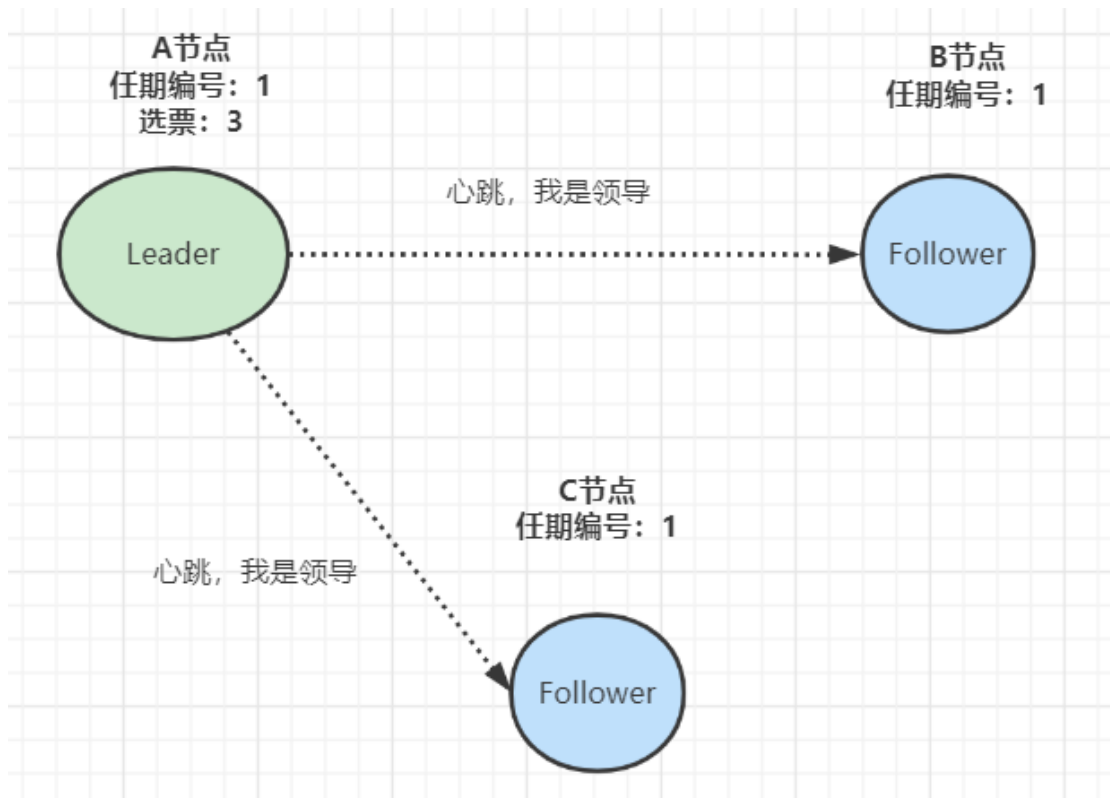
2. 如果 Follower 在规定的超时时间，都没有收到来自 Leader 的心跳，它就发起选举：将自己的状态切为 **Candidate**，增加自己的任期编号，然后向集群中的其它 **Follower** 节点发送请求，询问其是否选举自己成为 **Leader**：



1. 其他节点收到**候选人 A** 的请求投票消息后，如果在编号为 1 的这届任期内还没有进行过投票，那么它将把选票投给节点 A，并增加自己的任期编号：



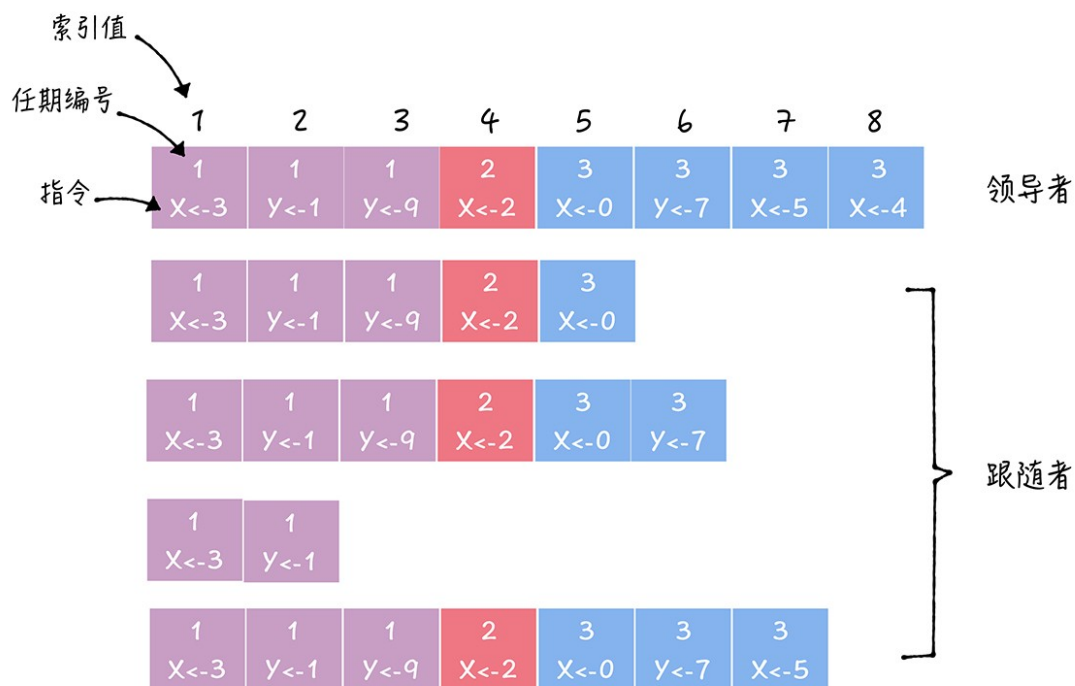
1. 当收到来自集群中过半数节点的接受投票后，A节点即成为本届任期内 Leader，他将周期性地发送心跳消息，通知其他节点我是 Leader，阻止 Follower 发起新的选举：



11.16.2 日志复制

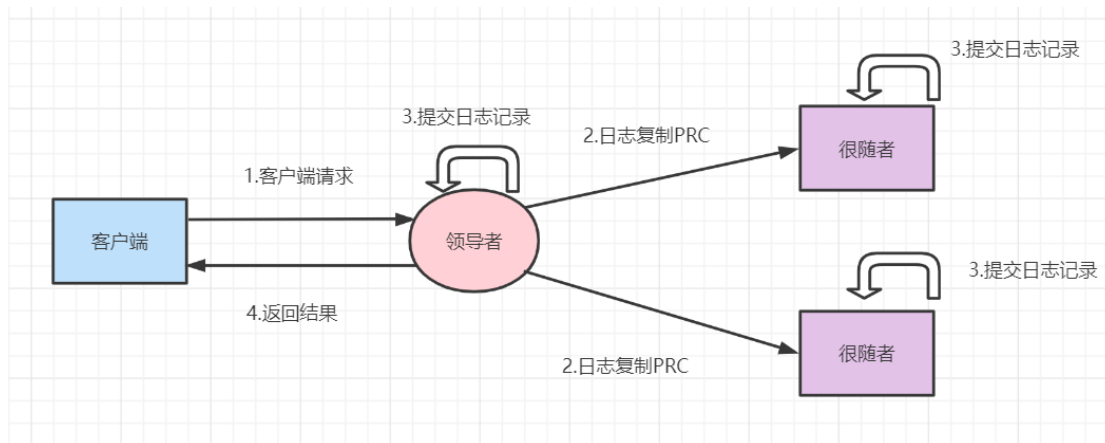
当有了 leader，系统应该进入对外工作期了。客户端的一切请求来发送到 leader，leader 来调度这些并发请求的顺序，并且保证 leader 与 followers 状态的一致性。Leader 接收到来自客户端写请求后，处理写请求的过程其实就是一个**日志复制**的过程。

日志项长什么样呢？如下图：



请求完整过程：

1. 当系统 (leader) 收到一个来自客户端的写请求，会添加一个 log entry (日志项) 到本地日志。
2. Leader 通过日志复制 (AppendEntries) RPC 消息，将日志项并行复制到集群其它 Follower 节点。
3. 如果 Leader 接收到大多数的“复制成功”响应后，它将日志项应用到自己的状态机，并返回成功给客户端。如果 Leader 没有接收到大多数的“复制成功”响应，那么就返回错误给客户端；
4. 当 Follower 接收到心跳信息，或者新的 AppendEntries 消息后，如果发现 Leader 已经提交了某条日志项，而自己还没应用，那么 Follower 就会将这条日志项应用到本地的状态机中。



Raft 算法，Leader 是通过强制 Follower 直接复制自己的日志项，来处理不一致日志，从而最终实现了集群各节点日志的一致。

大家有兴趣可以看这篇文章哈：[分布式一致性：Raft 算法原理^{\[1\]}](https://www.tpvlog.com/article/66) (<https://www.tpvlog.com/article/66>)

11.17 消息中间件如何做到高可用

消息中间件如何保证高可用呢？单机是没有高可用可言的，高可用都是对集群来说的，一起看下 kafka 的高可用吧。

Kafka 的基础集群架构，由多个 **broker** 组成，每个 **broker** 都是一个节点。当你创建一个 **topic** 时，它可以划分为多个 **partition**，而每个 **partition** 放一部分数据，分别存在于不同的 broker 上。也就是说，一个 topic 的数据，是分散放在多个机器上的，每个机器就放一部分数据。

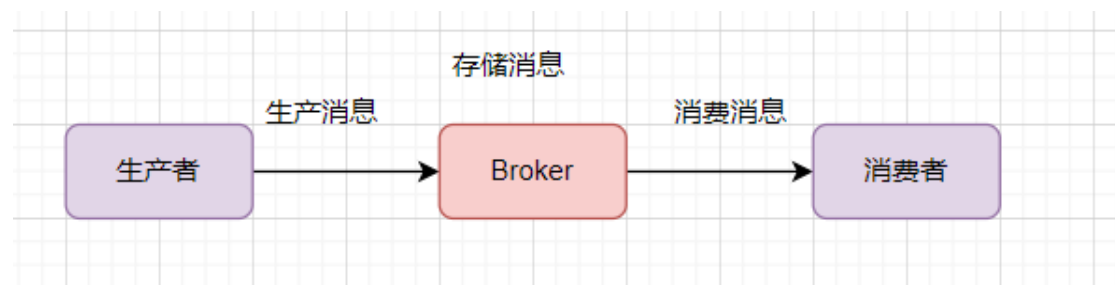
有些伙伴可能有疑问，每个 **partition** 放一部分数据，如果对应的 broker 挂了，那这部分数据是不是就丢失了？那还谈什么高可用呢？

Kafka 0.8 之后，提供了复制品副本机制来保证高可用，即每个 partition 的数据都会同步到其它机器上，形成多个副本。然后所有的副本会选举一个

leader 出来，让 leader 去跟生产和消费者打交道，其他副本都是 follower。写数据时，leader 负责把数据同步给所有的 follower，读消息时，直接读 leader 上的数据即可。如何保证高可用的？就是假设某个 broker 宕机，这个 broker 上的 partition 在其他机器上都有副本的。如果挂的是 leader 的 broker 呢？其他 follower 会重新选一个 leader 出来。

11.18 消息队列怎么保证不丢消息的

一个消息从生产者产生，到被消费者消费，主要经过这 3 个过程：



因此如何保证 MQ 不丢失消息，可以从这三个阶段阐述：

- 生产者保证不丢消息
- 存储端不丢消息
- 消费者不丢消息

11.18.1 生产者保证不丢消息

生产端如何保证不丢消息呢？确保生产的消息能到达存储端。

如果是 **RocketMQ** 消息中间件，Producer 生产者提供了三种发送消息的方式，分别是：

- 同步发送
- 异步发送
- 单向发送

生产者要想发消息时保证消息不丢失，可以：

- 采用**同步方式**发送，send 消息方法返回**成功**状态，就表示消息正常到达了存储端 Broker。
- 如果 send 消息**异常**或者返回**非成功**状态，可以**重试**。
- 可以使用事务消息，RocketMQ 的事务消息机制就是为了保证零丢失来设计的

11.18.2 存储端不丢消息

如何保证存储端的消息不丢失呢？确保消息**持久化到磁盘**。大家很容易想到就是**刷盘机制**。

刷盘机制分**同步刷盘**和**异步刷盘**：

- 生产者消息发过来时，只有持久化到磁盘，RocketMQ 的存储端 Broker 才返回一个成功的 ACK 响应，这就是**同步刷盘**。它保证消息不丢失，但是影响了性能。
- 异步刷盘的话，只要消息写入 PageCache 缓存，就返回一个成功的 ACK 响应。这样提高了 MQ 的性能，但是如果这时候机器断电了，就会丢失消息。

Broker 一般是**集群部署**的，有 master 主节点和 slave 从节点。消息到 Broker 存储端，只有主节点和从节点都写入成功，才反馈成功的 ack 给生产者。这就是**同步复制**，它保证了消息不丢失，但是降低了系统的吞吐量。与之对应的就是**异步复制**，只要消息写入主节点成功，就返回成功的 ack，它速度快，但是会有性能问题。

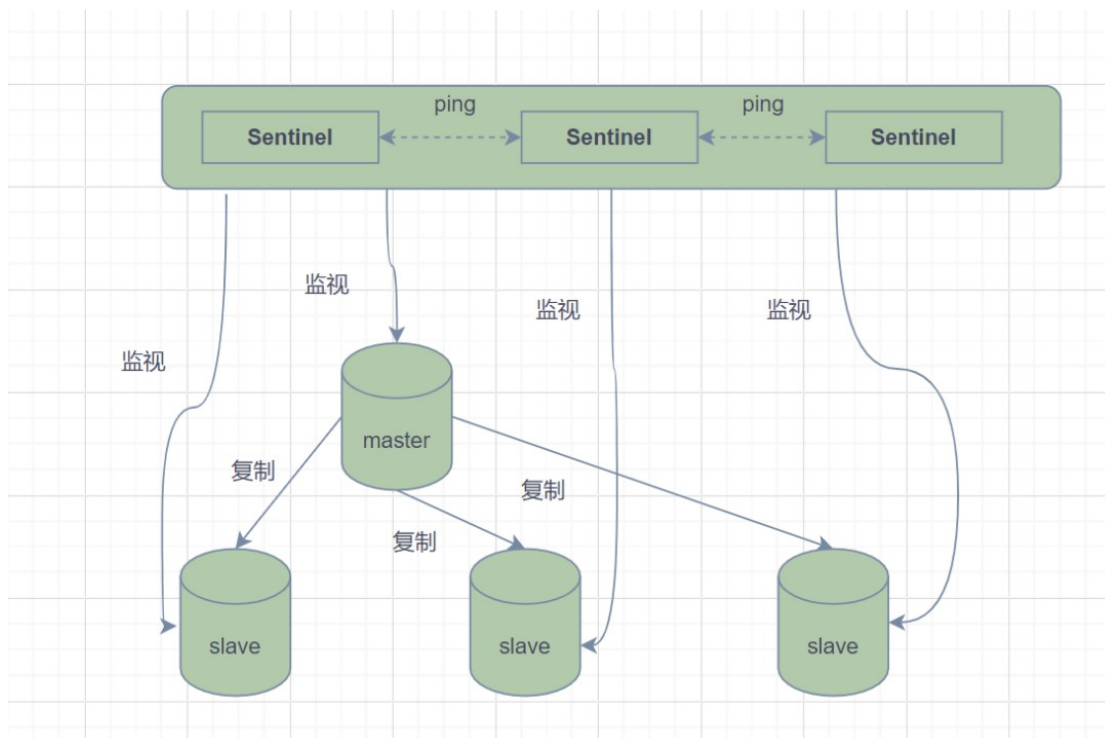
11.18.3 消费阶段不丢消息

消费者执行完业务逻辑，再反馈给 Broker 说消费成功，这样才可以保证消费阶段不丢消息。

11.19 Redis 如何保证高可用？聊聊 Redis 的哨兵机制

主从模式中，一旦主节点由于故障不能提供服务，需要人工将从节点晋升为主节点，同时还要通知应用方更新主节点地址。显然，多数业务场景都不能接受这种故障处理方式。Redis 从 2.8 开始正式提供了 Redis Sentinel (哨兵) 架构来解决这个问题。

哨兵模式，由一个或多个 Sentinel 实例组成的 Sentinel 系统，它可以监视所有的 Redis 主节点和从节点，并在被监视的主节点进入下线状态时，自动将下线主服务器属下的某个从节点升级为新的主节点。但是呢，一个哨兵进程对 Redis 节点进行监控，就可能会出现单点问题（单点问题），因此，可以使用多个哨兵来进行监控 Redis 节点，并且各个哨兵之间还会进行监控。



简单来说，哨兵模式就三个作用：

- 发送命令，等待 Redis 服务器（包括主服务器和从服务器）返回监控其运行状态；

- 哨兵监测到主节点宕机，会自动将从节点切换成主节点，然后通过发布订阅模式通知其他的从节点，修改配置文件，让它们切换主机；
- 哨兵之间还会相互监控，从而达到高可用。

故障切换的过程是怎样的呢

假设主服务器宕机，哨兵 1 先检测到这个结果，系统并不会马上进行 failover 过程，仅仅是哨兵 1 主观的认为主服务器不可用，这个现象成为主观下线。当后面的哨兵也检测到主服务器不可用，并且数量达到一定值时，那么哨兵之间就会进行一次投票，投票的结果由一个哨兵发起，进行 failover 操作。切换成功后，就会通过发布订阅模式，让各个哨兵把自己监控的从服务器实现切换主机，这个过程称为客观下线。这样对于客户端而言，一切都是透明的。

哨兵的工作模式如下：

1. 每个 Sentinel 以每秒钟一次的频率向它所知的 Master，Slave 以及其他 Sentinel 实例发送一个 PING 命令。
2. 如果一个实例（instance）距离最后一次有效回复 PING 命令的时间超过 down-after-milliseconds 选项所指定的值，则这个实例会被 Sentinel 标记为主观下线。
3. 如果一个 Master 被标记为主观下线，则正在监视这个 Master 的所有 Sentinel 要以每秒一次的频率确认 Master 的确进入了主观下线状态。
4. 当有足够数量的 Sentinel（大于等于配置文件指定的值）在指定的时间范围内确认 Master 的确进入了主观下线状态，则 Master 会被标记为客观下线。
5. 在一般情况下，每个 Sentinel 会以每 10 秒一次的频率向它已知的所有 Master，Slave 发送 INFO 命令。
6. 当 Master 被 Sentinel 标记为客观下线时，Sentinel 向下线的 Master 的所有 Slave 发送 INFO 命令的频率会从 10 秒一次改为每秒一次
7. 若没有足够数量的 Sentinel 同意 Master 已经下线，Master 的客观下线状态就会被移除；若 Master 重新向 Sentinel 的 PING 命令返回有效回复，Master 的主观下线状态就会被移除。

11.20 无重复字符的最长子串

给定一个字符串 s，请你找出其中不含有重复字符的 最长子串 的长度。

示例 1:

输入: s = "abcabcbb"

输出: 3

解释: 因为无重复字符的最长子串是 "abc", 所以其长度为 3。

示例 2:

输入: s = "bbbbbb"

输出: 1

解释: 因为无重复字符的最长子串是 "b", 所以其长度为 1。

这道题可以使用滑动窗口来实现。滑动窗口就是维护一个窗口，不断滑动，然后更新答案。

滑动窗口的大致逻辑框架，伪代码如下：

```
int left = 0, right = 0;
while (right < s.size()){
    //增大窗口
    window.add(s[right]);
    right++;
    while (window needs shrink){
        //缩小窗口
        window.remove (s[left]);
        left ++;
    }
}
```

解法流程如下：

- 首先呢，就是获取原字符串的长度。
- 接着维护一个窗口（数组、哈希、队列）
- 窗口一步一步向右扩展
- 窗口在向右扩展滑动过程，需要判断左边是否需要缩减
- 最后比较更新答案

完整代码如下：

```
int lengthOfLongestSubstring(String s){
    //获取原字符串的长度
    int len = s.length();
    //维护一个哈希集合的窗口
    Set<Character> windows = new HashSet<>();
```

```

int left=0,right =0;
int res =0;

while(right<len){
    char c = s.charAt(right);
    //窗口右移
    right++;

    //判断是否左边窗口需要缩减，如果已经包含，那就需要缩减
    while(window.contains(c)){
        window.remove(s.charAt(left));
        left++;
    }
    window.add(c);
    //比较更新答案
    res = Math.max(res,window.size());
}
return res;
}

```

之前写过一篇[滑动窗口解析](#)，大家有兴趣可以看下哈：

[leetcode 必备算法：聊聊滑动窗口](#)

12. 蚂蚁金服一面（关注公众号：捡田螺的小男孩。）

大家好，我是捡田螺的小男孩。最近编程讨论群有位小伙伴去蚂蚁金服面试了，

以下是面试的真题，跟大家一起来讨论怎么回答。。更多干货，关注公众号：

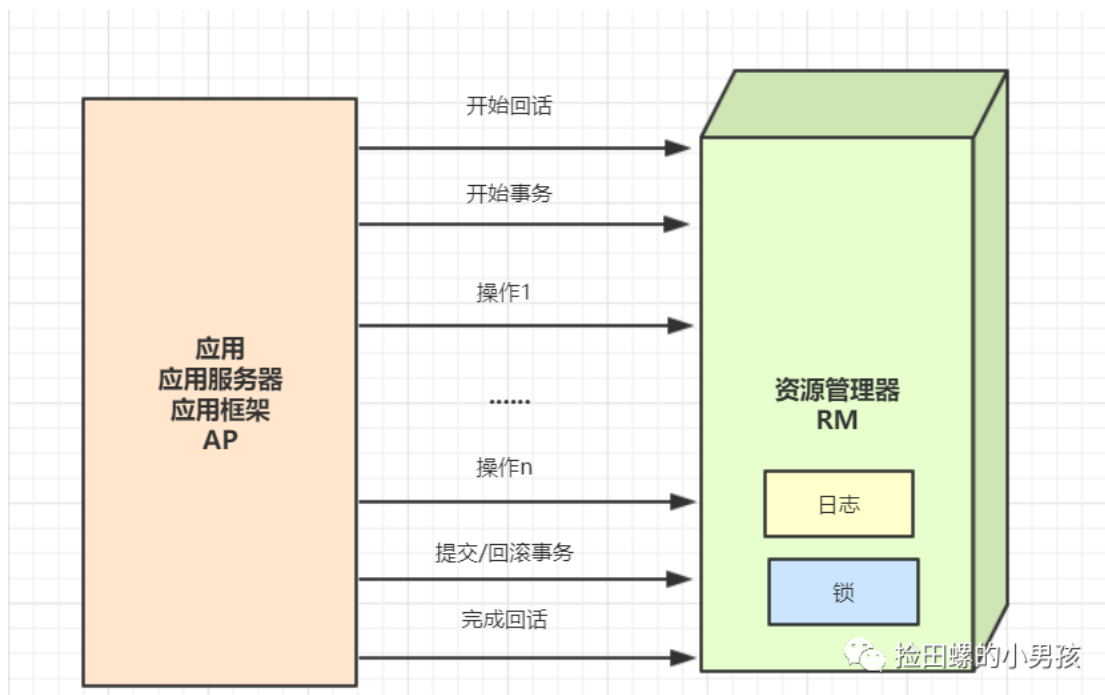
捡田螺的小男孩。



12.1 用到分布式事务嘛?为什么用这种方案,有其他方案嘛?

12.1.2 什么是分布式事务

谈到事务，我们就会想到**数据库事务**，很容易就想到**原子性、一致性、持久性、隔离性**。



分布式事务跟数据库事务有点不一样，它是指事务的参与者、支持事务的服务器、资源服务器以及事务管理器分别位于不同的分布式系统的不同节点之上。简单来说，分布式事务指的就是分布式系统中的事务，它的存在就是为了保证不同数据库节点的数据一致性。

12.1.2 分布式事务基础

分布式事务需要知道 **CAP 理论**和 **BASE 理论**。

CAP 理论

- 一致性(C : Consistency)：一致性是指数据在多个副本之间能否保持一致的特性。例如一个数据在某个分区节点更新之后，在其他分区节点读出来的数据也是更新之后的数据。
- 可用性(A : Availability)：可用性是指系统提供的服务必须一直处于可用的状态，对于用户的每一个操作请求总是能够在有限的时间内返回结果。这里的重点是"有限时间内"和"返回结果"。
- 分区容错性 (P:Partition tolerance) :分布式系统在遇到任何网络分区故障的时候，仍然需要能够保证对外提供满足一致性和可用性的服务。

一个分布式系统中，CAP 理论它只能同时满足 (一致性、可用性、分区容错性) 中的两点。

BASE 理论

BASE 理论，是对 CAP 中 AP 的一个扩展，对于我们的业务系统，我们考虑牺牲一致性来换取系统的可用性和分区容错性。BASE 是 Basically

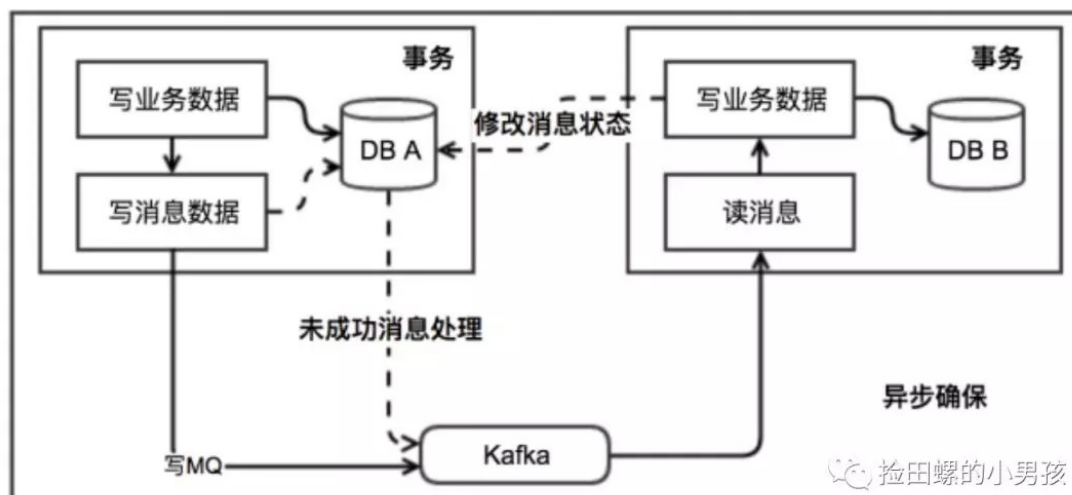
Available(基本可用) , Soft state (软状态) ,和 Eventually consistent (最终一致性) 三个短语的缩写。

- 基本可用是指，通过支持局部故障而不是系统全局故障来实现的；
- Soft State 表示状态可以有一段时间不同步；
- 最终一致，最终数据是一致的就可以了，而不是实时保持强一致。

12.1.3 分布式事务的几种解决方案

- **2PC(二阶段提交)方案**，事务的提交分为两个阶段：准备阶段和提交执行方案。
- **TCC (即 Try、Confirm、Cancel)**，它采用了补偿机制，核心思想是：针对每个操作，都要注册一个与其对应的确认和补偿（撤销）操作。
- **本地消息表**，它的核心思想就是将分布式事务拆成本地事务进行处理。
- **最大努力通知**，实现最大努力通知，可以采用 MQ 的 ack 机制。
- **Saga 事务**，它的核心思想是将长事务拆分为多个本地短事务，由 Saga 事务协调器协调，如果正常结束那就正常完成，如果某个步骤失败，则根据相反顺序一次调用补偿操作。

业界目前使用**本地消息表**这种方案是比较多的，它的核心思想就是将分布式事务拆成本地事务进行处理。可以看一下基本的实现流程图吧：



对于消息发送方：

- 首先需要有一个消息表，记录着消息状态相关信息。
- 业务数据和消息表在同一个数据库，即要保证它俩在同一个本地事务。
- 在本地事务中处理完业务数据和写消息表操作后，通过写消息到 MQ 消息队列。
- 消息会发到消息消费方，如果发送失败，即进行重试。

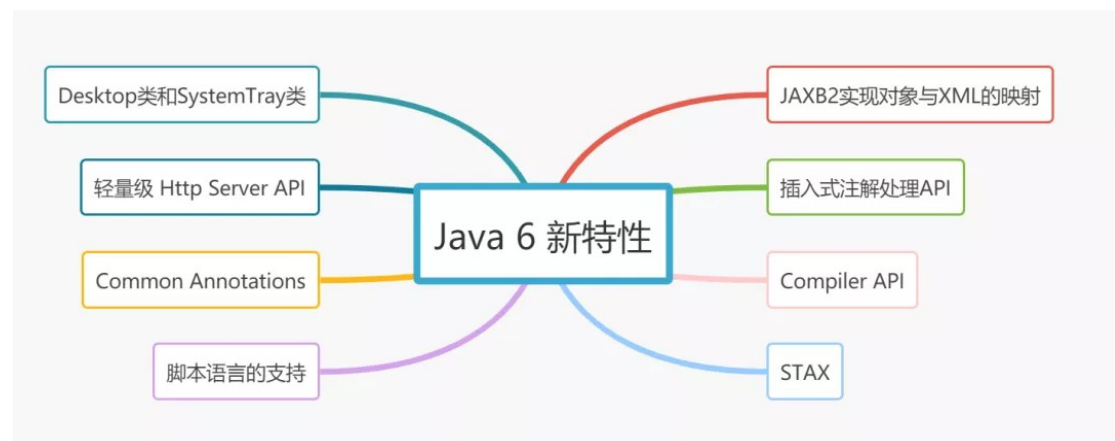
消息消费方：

- 处理消息队列中的消息，完成自己的业务逻辑。
- 此时如果本地事务处理成功，则表明已经处理成功了。
- 如果本地事务处理失败，那么就会重试执行。
- 如果是业务上面的失败，给消息生产方发送一个业务补偿消息，通知进行回滚等操作。

生产方和消费方定时扫描本地消息表，把还没处理完成的消息或者失败的消息再发送一遍。如果有靠谱的自动对账补账逻辑，这种方案还是非常实用的。

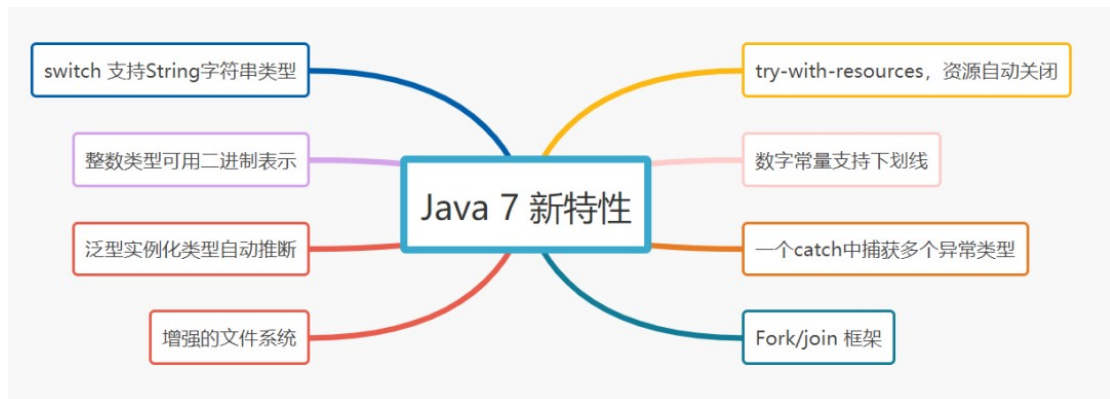
12.2 JDK6、7、8 分别提供了哪些新特性

JDK 6 新特性



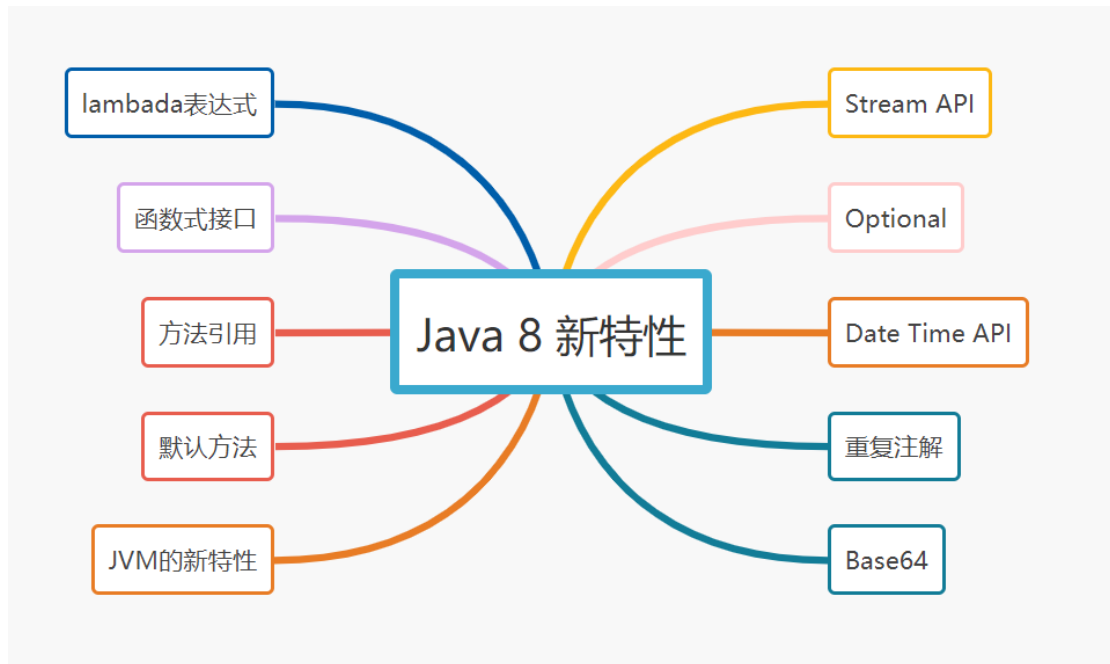
- Desktop 类 (它允许一个 Java 应用程序启动本地的另一个应用程序去处理 URI 或文件请求)
- 使用 JAXB2 来实现对象与 XML 之间的映射
- 轻量级 Http Server API
- 插入式注解处理 API (lombok 框架基于这个特性实现)
- STAX (是 JDK6 中一种处理 XML 文档的 API)

JDK 7 的新特性



- switch 支持 String 字符串类型
- try-with-resources , 资源自动关闭
- 整数类型如 (byte , short , int , long) 能够用二进制来表示
- 数字常量支持下划线
- 泛型实例化类型自动推断,即"<>"
- 一个 catch 中捕获多个异常类型 , 用 (|) 分隔开
- 增强的文件系统
- Fork/join 框架

JDK8 的新特性

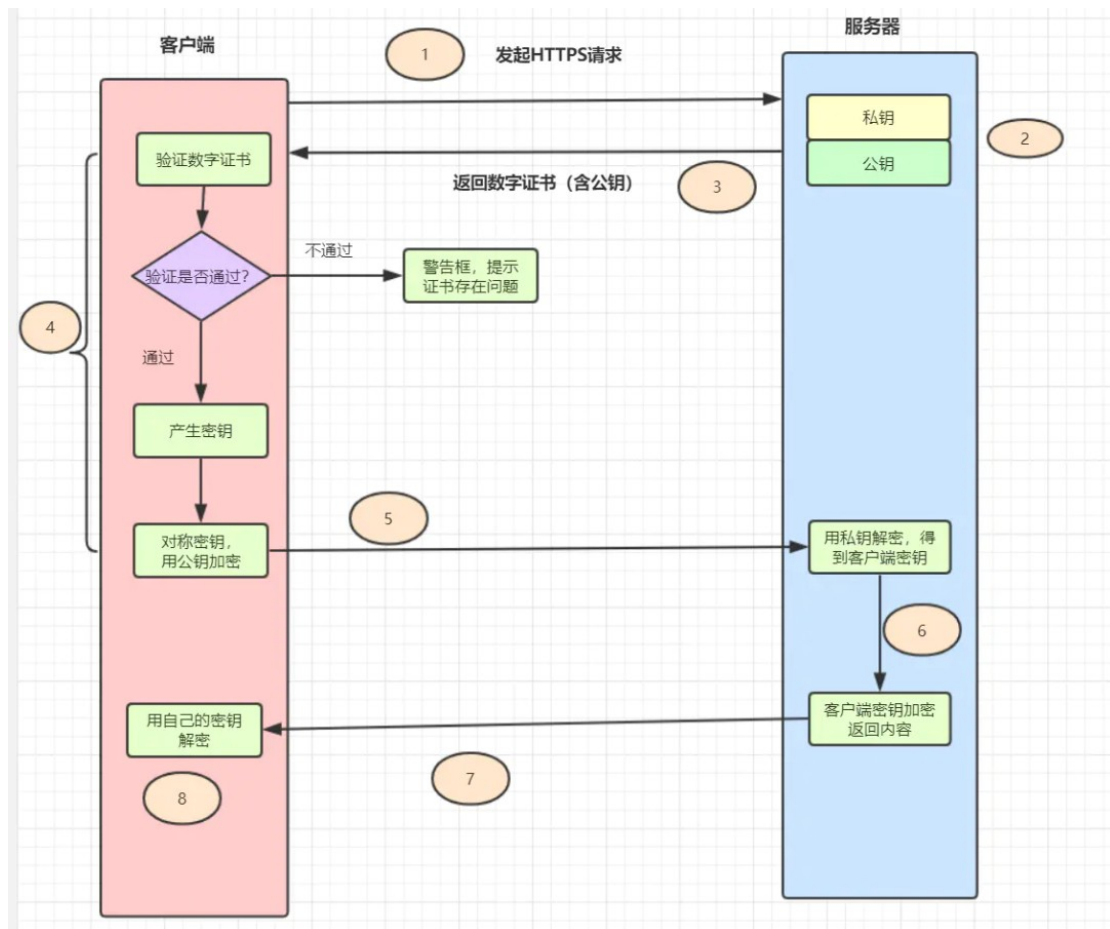


- lambada 表达式
- 函数式接口
- 方法引用
- 默认方法
- Stream API
- Optional
- Date Time API (如 LocalDate)
- 重复注解
- Base64
- JVM 的新特性 (如元空间 Metaspace 代替持久代)

12.3 https 原理，工作流程

- HTTPS = HTTP + SSL/TLS，即用 SSL/TLS 对数据进行加密和解密，Http 进行传输。

- SSL，即 Secure Sockets Layer (安全套接层协议)，是网络通信提供安全及数据完整性的一种安全协议。
- TLS，即 Transport Layer Security(安全传输层协议)，它是 SSL3.0 的后续版本。



1. 客户端发起 Https 请求，连接到服务器的 443 端口。
2. 服务器必须要有一套数字证书 (证书内容有公钥、证书颁发机构、失效日期等)。
3. 服务器将自己的数字证书发送给客户端 (公钥在证书里面，私钥由服务器持有)。

4. 客户端收到数字证书之后，会验证证书的合法性。如果证书验证通过，就会生成一个随机的对称密钥，用证书的公钥加密。
5. 客户端将公钥加密后的密钥发送到服务器。
6. 服务器接收到客户端发来的密文密钥之后，用自己之前保留的私钥对其进行非对称解密，解密之后就得到客户端的密钥，然后用客户端密钥对返回数据进行对称加密，毕竟传输的数据都是密文啦。
7. 服务器将加密后的密文返回到客户端。
8. 客户端收到后，用自己的密钥对其进行对称解密，得到服务器返回的数据。

12.4 讲讲 java jmm volatile 的实现原理

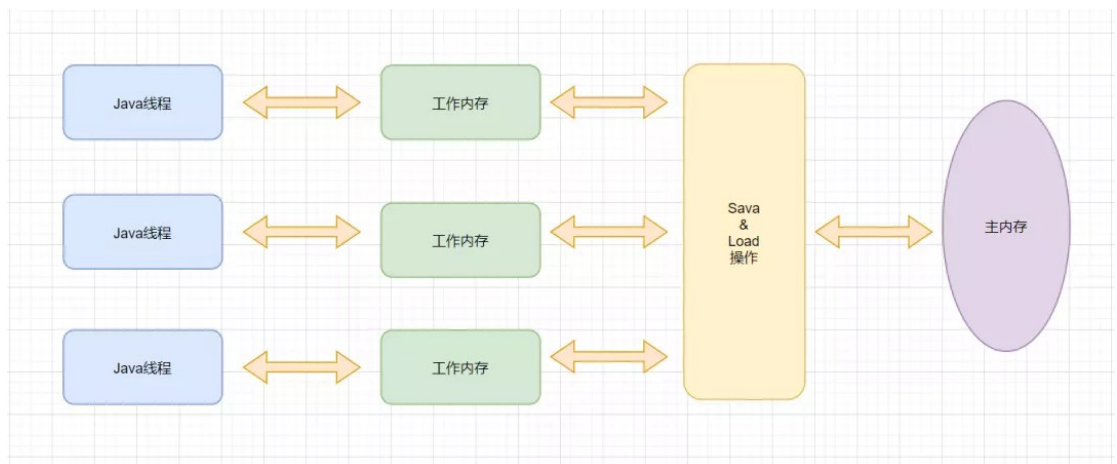
volatile 关键字是 Java 虚拟机提供的**最轻量级的同步机制**，它作为一个修饰符，用来修饰变量。它保证变量对所有线程可见性，禁止指令重排，但是不保证原子性。

volatile 是如何保证可见性的呢？我们先来看下 java 内存模型 (jmm)

- Java 虚拟机规范试图定义一种 Java 内存模型,来屏蔽掉各种硬件和操作系统的内存访问差异，以实现让 Java 程序在各种平台上都能达到一致的内存访问效果。
- 为了更好的执行性能，java 内存模型并没有限制执行引擎使用处理器的特定寄存器或缓存来和主内存打交道，也没有限制编译器进行调整代码

顺序优化。所以Java内存模型会存在缓存一致性问题 and 指令重排序问题的。

- Java内存模型规定所有的变量都是存在主内存当中，每个线程都有自己的工作内存。这里的变量包括实例变量和静态变量，但是不包括局部变量，因为局部变量是线程私有的。
- 线程的工作内存保存了被该线程使用的变量的主内存副本，线程对变量的所有操作都必须在工作内存中进行，而不能直接操作主内存。并且每个线程不能访问其他线程的工作内存。



volatile 变量，保证新值能立即同步回主内存，以及每次使用前立即从主内存刷新，所以我们说 **volatile 保证了多线程操作变量的可见性**。

指令重排是指在程序执行过程中,为了提高性能,编译器和 CPU 可能会对指令进行重新排序。volatile 是如何禁止指令重排的？在Java语言中，有一个先行发生原则 (happens-before)

- **程序次序规则**：在一个线程内，按照控制流顺序，书写在前面的操作先行发生于书写在后面的操作。

- **管程锁定规则**：一个 unlock 操作先行发生于后面对同一个锁额 lock 操作
- **volatile 变量规则**：对一个变量的写操作先行发生于后面对这个变量的读操作
- **线程启动规则**：Thread 对象的 start()方法先行发生于此线程的每个一个动作
- **线程终止规则**：线程中所有的操作都先行发生于线程的终止检测，我们可以通过 Thread.join()方法结束、Thread.isAlive()的返回值手段检测到线程已经终止执行
- **线程中断规则**：对线程 interrupt()方法的调用先行发生于被中断线程的代码检测到中断事件的发生
- **对象终结规则**：一个对象的初始化完成先行发生于他的 finalize()方法的开始
- **传递性**：如果操作 A 先行发生于操作 B，而操作 B 又先行发生于操作 C，则可以得出操作 A 先行发生于操作 C

实际上 volatile 保证可见性和禁止指令重排都跟**内存屏障**有关。我们来看一段 volatile 使用的 demo 代码

```
public class Singleton {  
    private volatile static Singleton instance;  
    private Singleton (){}  
    public static Singleton getInstance() {  
        if (instance == null) {  
            synchronized (Singleton.class) {  
                if (instance == null) {  
                    instance = new Singleton();  
                }  
            }  
        }  
    }  
}
```

```

    }
}
}
return instance;
}
}

```

编译后，对比有 volatile 关键字和没有 volatile 关键字时所生成的汇编代码，发现有 volatile 关键字修饰时，会多出一个**lock addl \$0x0,(%esp)**，即多出一个 lock 前缀指令，lock 指令相当于一个「内存屏障」

lock 指令相当于一个**内存屏障**，它保证以下这几点：

- 1.重排序时不能把后面的指令重排序到内存屏障之前的位置
- 2.将本处理器的缓存写入内存
- 3.如果是写入动作，会导致其他处理器中对应的缓存无效。

第 2 点和第 3 点就是保证 volatile 保证可见性的体现嘛，**第 1 点就是禁止指令重排列的体现**。内存屏障又是什么呢？

内存屏障四大分类：（ Load 代表读取指令，Store 代表写入指令 ）

内存屏障类型	抽象场景	描述
LoadLoad 屏障	Load1; LoadLoad; Load2	在 Load2 要读取的数据被访问前，保证 Load1 要读取的数据被读取完毕。
StoreStore 屏障	Store1; StoreStore; Store2	在 Store2 写入执行前，保证 Store1 的写入操作对其它处理器可见

内存屏障类型	抽象场景	描述
LoadStore 屏障	Load1; LoadStore; Store2	在 Store2 被写入前，保证 Load1 要读取的数据被读取完毕。
StoreLoad 屏障	Store1; StoreLoad; Load2	在 Load2 读取操作执行前，保证 Store1 的写入对所有处理器可见。

为了实现 volatile 的内存语义，Java 内存模型采取以下的保守策略

- 在每个 volatile 写操作的前面插入一个 StoreStore 屏障。
- 在每个 volatile 写操作的后面插入一个 StoreLoad 屏障。
- 在每个 volatile 读操作的后面插入一个 LoadLoad 屏障。
- 在每个 volatile 读操作的后面插入一个 LoadStore 屏障。

有些小伙伴，可能对这个还是有点疑惑，内存屏障这玩意太抽象了。我们照着代码看下吧：

```
volatile bool flag = false;

public void read() {
    b = 1;           //1
    flag = true;     //2
}

public void add() {
    if (flag) {      //3
        int sum = b + b; //4
        System.out.println("bb sum is" + sum);
    }
}
```

StoreStore内存屏障

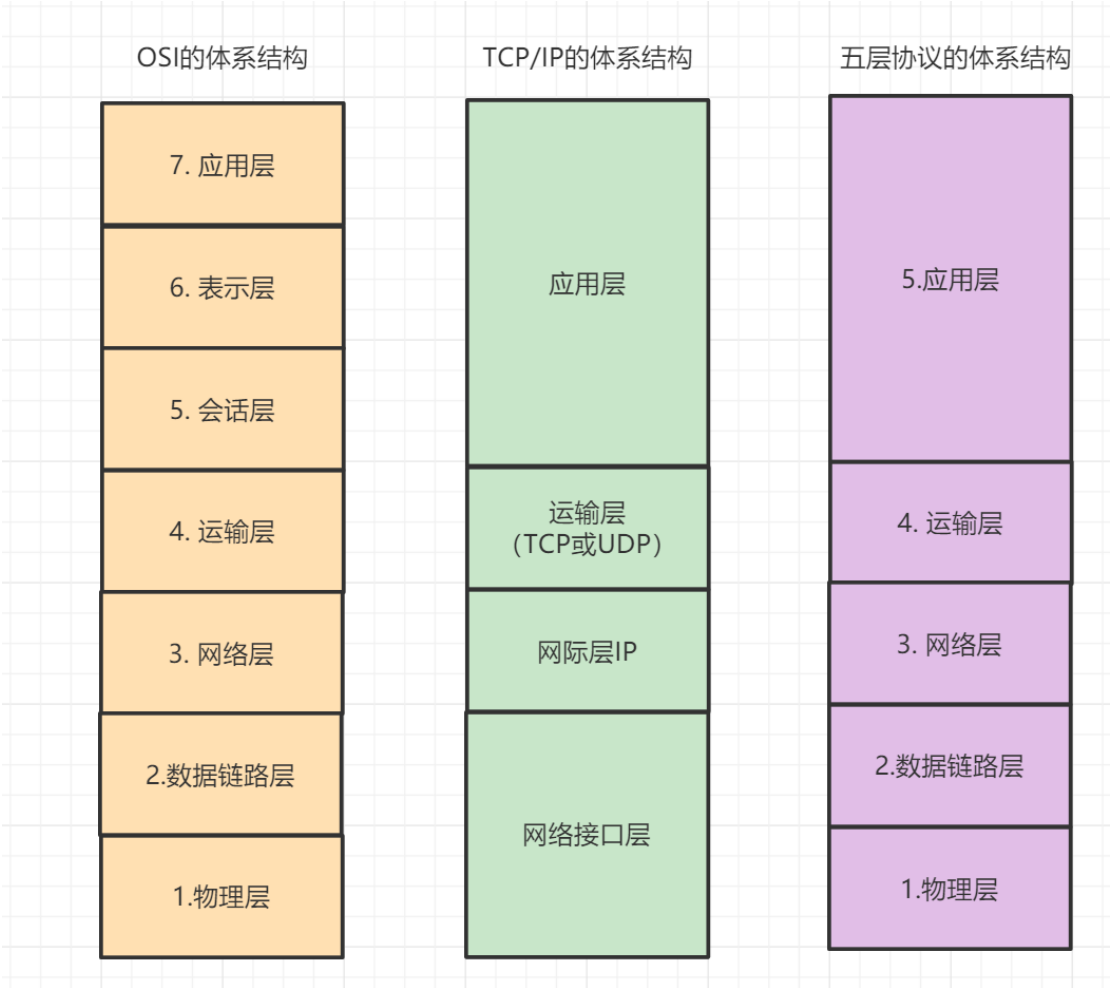
StoreLoad内存屏障

LoadLoad内存屏障 LoadStore内存屏障

内存屏障保证前面的指令先执行，所以这就保证了禁止了指令重排啦，同时内存屏障保证缓存写入内存和其他处理器缓存失效，这也就保证了可见性，哈哈～

12.5 讲一讲 7 层网络模型，tcp 的为什么要三次握手

计算机网路体系结构有三层：OSI 七层模型、TCP/IP 四层模型、五层体系结构，如图：

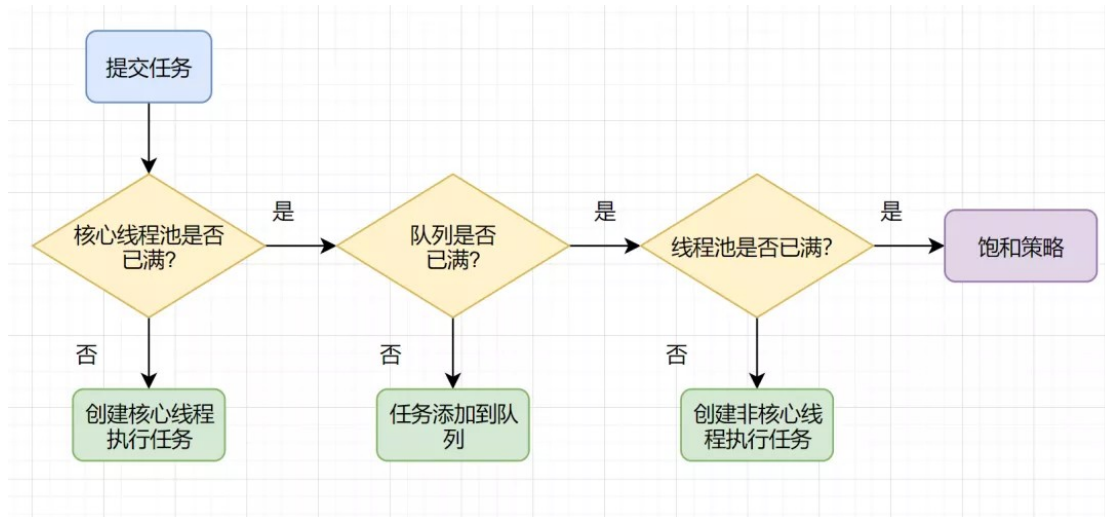


七层模型，亦称 OSI (Open System Interconnection)，国际标准化组织 (International Organization for Standardization) 制定的一个用于计算机或通信系统间互联的标准体系。

- 应用层：网络服务与最终用户的一个接口，常见的协议有：HTTP FTP SMTP SNMP DNS.
- 表示层：数据的表示、安全、压缩。，确保一个系统的应用层所发送的信息可以被另一个系统的应用层读取。
- 会话层：建立、管理、终止会话,对应主机进程，指本地主机与远程主机正在进行的会话.
- 传输层：定义传输数据的协议端口号，以及流控和差错校验,协议有 TCP UDP.
- 网络层：进行逻辑地址寻址，实现不同网络之间的路径选择,协议有 ICMP IGMP IP 等.
- 数据链路层：在物理层提供比特流服务的基础上，建立相邻结点之间的数据链路。
- 物理层：建立、维护、断开物理连接。

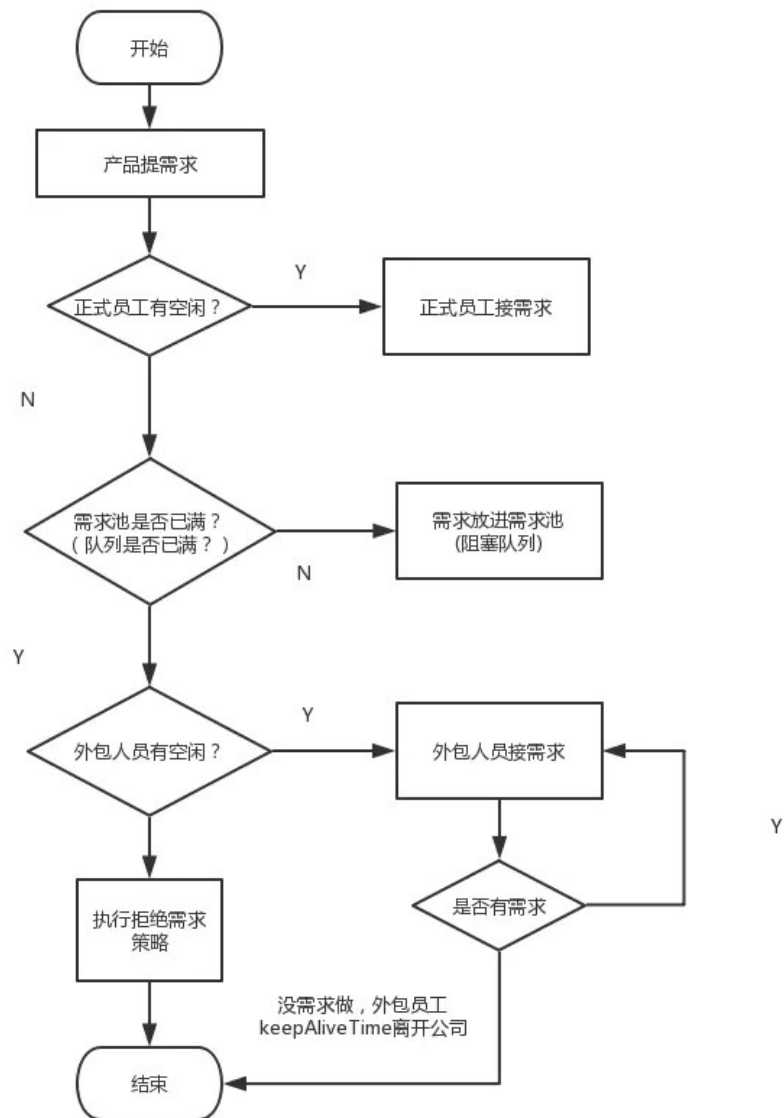
12.6 说说线程池的工作原理

面试官如果要我们讲下线程池工作原理的话，大家讲下以下这个流程图就可以啦：



为了形象描述线程池执行，加深大家的理解，我打个比喻：

- 核心线程比作公司正式员工
- 非核心线程比作外包员工
- 阻塞队列比作需求池
- 提交任务比作提需求



- 当产品提个需求，正式员工（核心线程）先接需求（执行任务）
- 如果正式员工都有需求在做，即核心线程数已满），产品就把需求先放需求池（阻塞队列）。
- 如果需求池(阻塞队列)也满了，但是这时候产品继续提需求,怎么办呢？那就请外包（非核心线程）来做。

- 如果所有员工（最大线程数也满了）都有需求在做了，那就执行拒绝策略。
- 如果外包员工把需求做完了，它经过一段（keepAliveTime）空闲时间，就离开公司了。

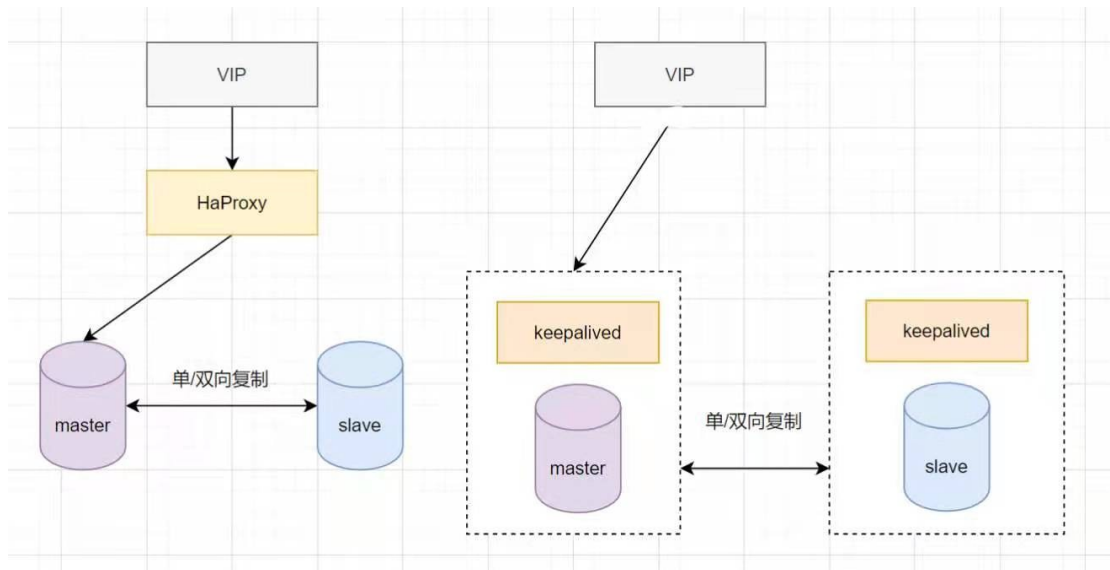
12.7 你们数据库的高可用是怎么实现的？

高可用，即 **High Availability**，是分布式系统架构设计中必须考虑的因素之一，它通常是指，通过设计减少系统不能提供服务的时间。单机部署谈不上高可用，因为单点故障问题。高可用都是多个节点的，我们在考虑 MySQL 数据库的高可用的架构时，需要考虑这几个方面：

- 如果数据库节点宕机，需要尽快回复，保证业务不受宕机影响。
- 从数据库节点的数据，尽可能跟主节点数据实时保持一致，至少保证最终一致性。
- 数据库节点切换时，数据不能缺失。

12.7.1 主从或主主半同步复制

用双节点数据库，搭建单向或者双向的半同步复制。架构如下：



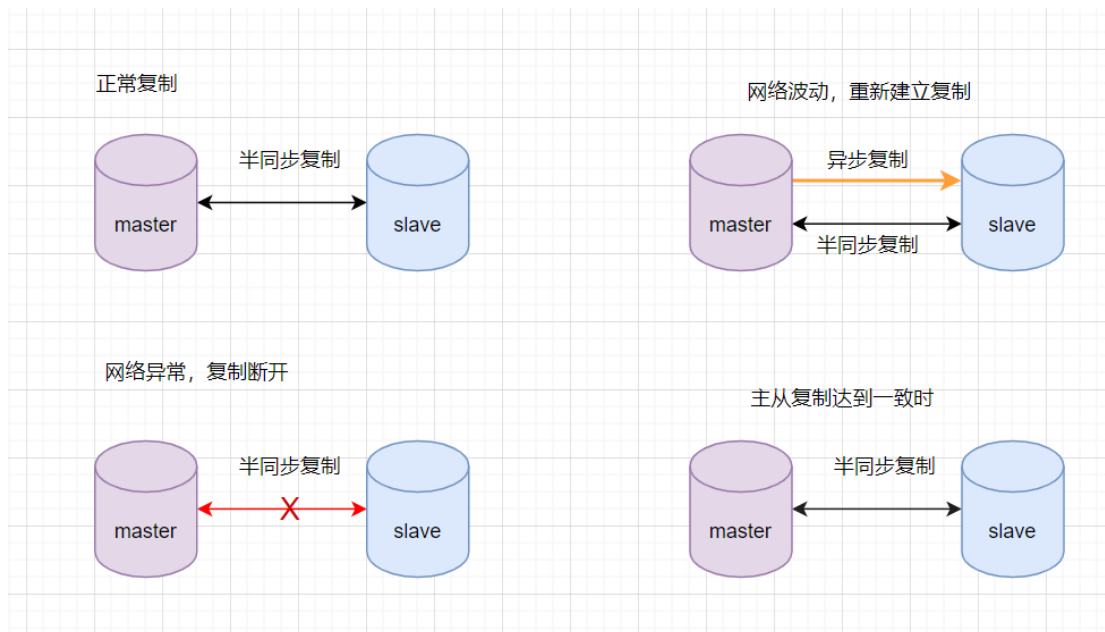
通常会和 proxy、keepalived 等第三方软件同时使用，即可以用来监控数据库的健康，又可以执行一系列管理命令。如果主库发生故障，切换到备库后仍然可以继续使用数据库。

这种方案优点是架构、部署比较简单，主机宕机直接切换即可。缺点是完全依赖于半同步复制，半同步复制退化为异步复制，无法保证数据一致性；另外，还需要额外考虑 **haproxy**、**keepalived** 的高可用机制。

12.7.2 半同步复制优化

半同步复制机制是可靠的，可以保证数据一致性的。但是如果网络发生波动，半同步复制发生超时会切换为异步复制，异步复制是无法保证数据的一致性的。

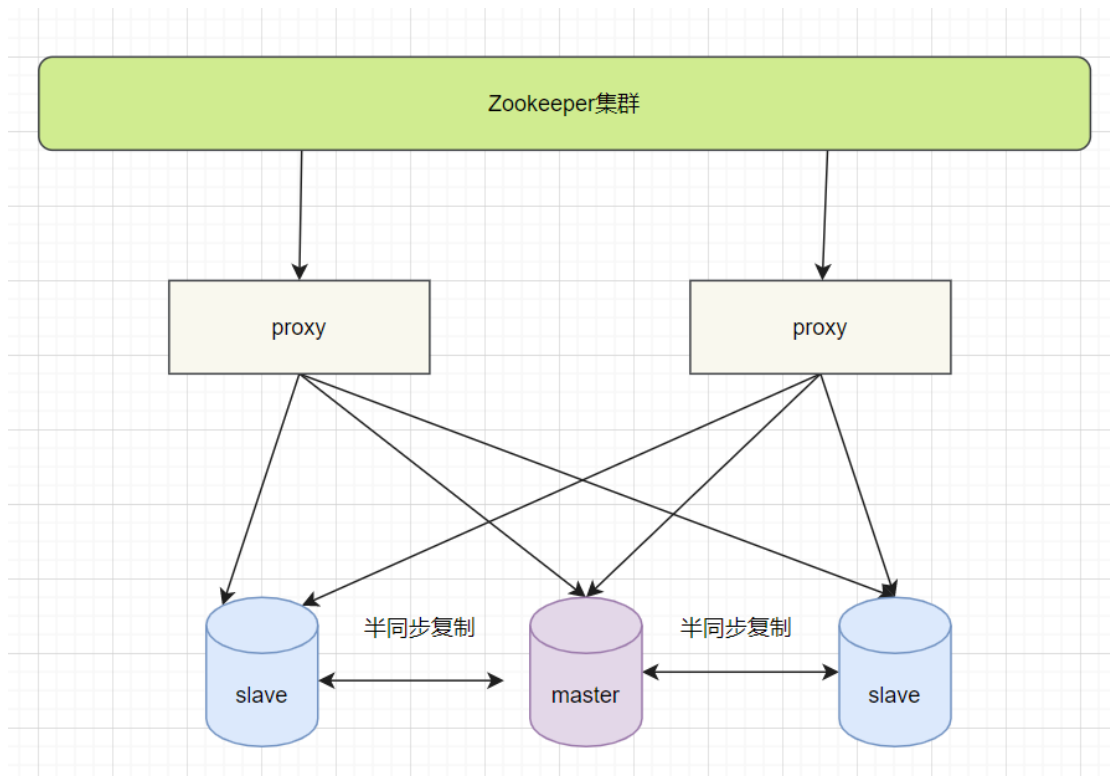
因此，可以在半同复制的基础上优化一下，尽可能保证半同复制。如**双通道复制方案**



- 优点：这种方案架构、部署也比较简单，主机宕机也是直接切换即可。
比方案 1 的半同步复制，更能保证数据的一致性。
- 缺点：需要修改内核源码或者使用 mysql 通信协议，没有从根本上解决数据一致性问题。

12.7.3 高可用架构优化

保证高可用，可以把主从双节点数据库扩展为数据库集群。Zookeeper 可以作为集群管理，它使用分布式算法保证集群数据的一致性，可以较好的避免网络分区现象的产生。



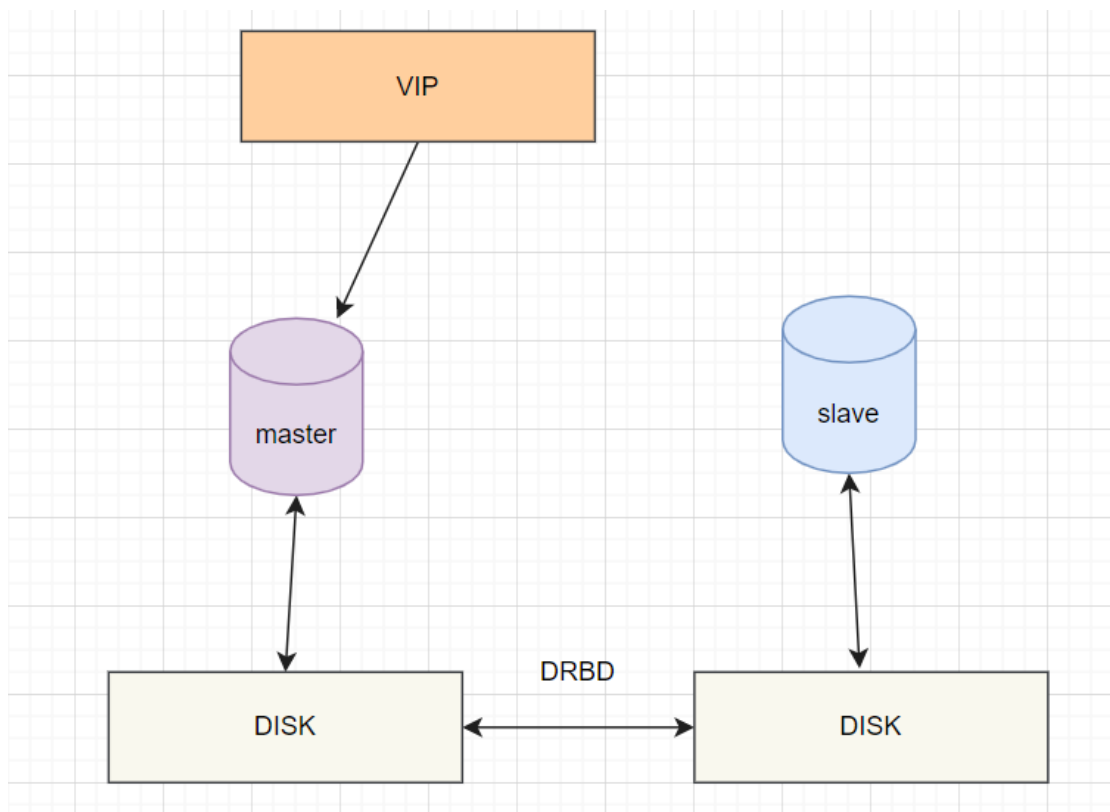
- 优点：保证了整个系统的高可用性，扩展性也较好，可以扩展为大规模集群。
- 缺点：数据一致性仍然依赖于原生的 **mysql** 半同步复制；引入 Zookeeper 使系统逻辑更复杂。

12.7.4 共享存储

共享存储实现了数据库服务器和存储设备的解耦，不同数据库之间的数据同步不再依赖于 MySQL 的原生复制功能，而是通过磁盘数据同步的手段，来保证数据的一致性。

DRBD 磁盘复制

DRBD 是一个用软件实现的、无共享的、服务器之间镜像块设备内容的存储复制解决方案。主要用于对服务器之间的磁盘、分区、逻辑卷等进行数据镜像，当用户将数据写入本地磁盘时，还会将数据发送到网络中另一台主机的磁盘上，这样的本地主机(主节点)与远程主机(备节点)的数据就可以保证实时同步。常用架构如下：

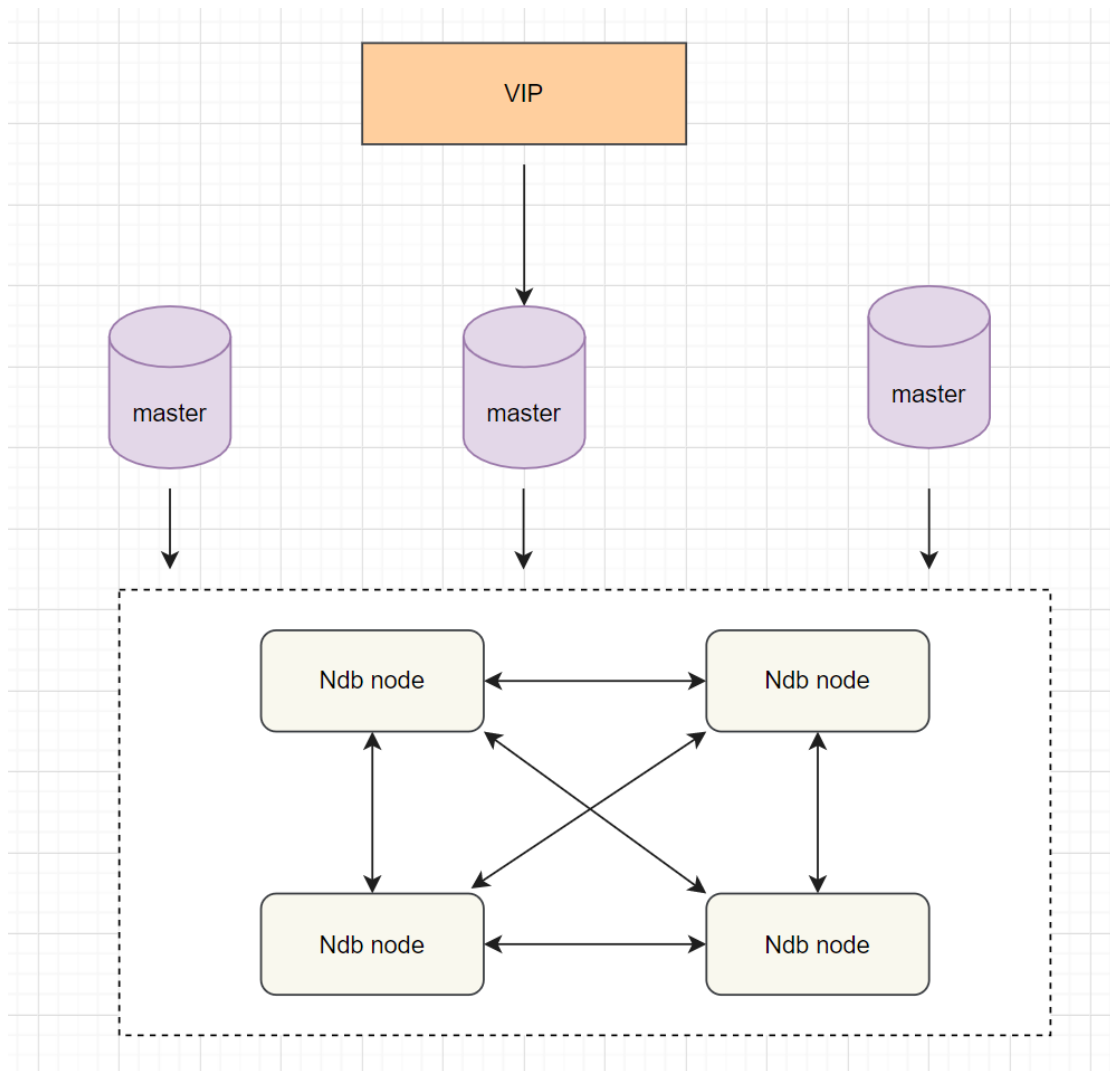


当本地主机出现问题，远程主机上还保留着一份相同的数据，即可以继续使用，保证了数据的安全。

- 优点：部署简单，价格合适，保证数据的强一致性
- 缺点：对 IO 性能影响较大，从库不提供读操作

12.7.5 分布式协议

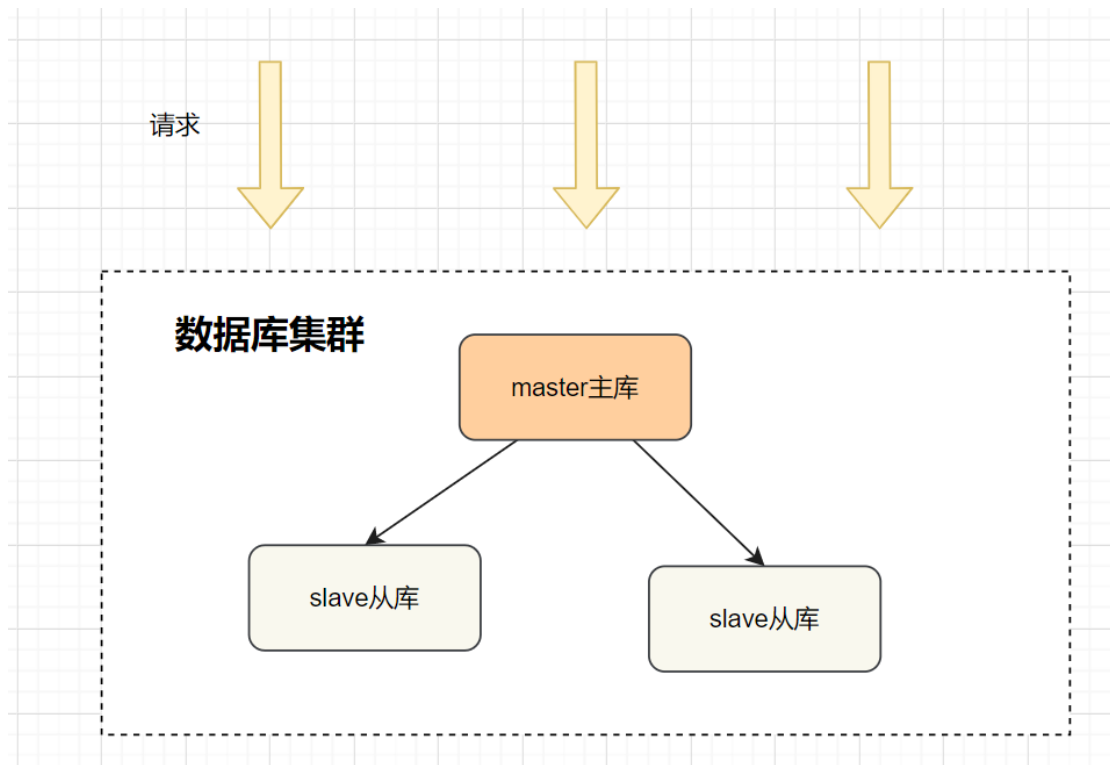
分布式协议可以很好解决数据一致性问题。常见的部署方案就是 **MySQL cluster**，它是官方集群的部署方案，通过使用 NDB 存储引擎实时备份冗余数据，实现数据库的高可用性和数据一致性。如下：



- 优点：不依赖于第三方软件，可以实现数据的强一致性；
- 缺点：配置较复杂；需要使用 NDB 储存引擎；至少三节点；

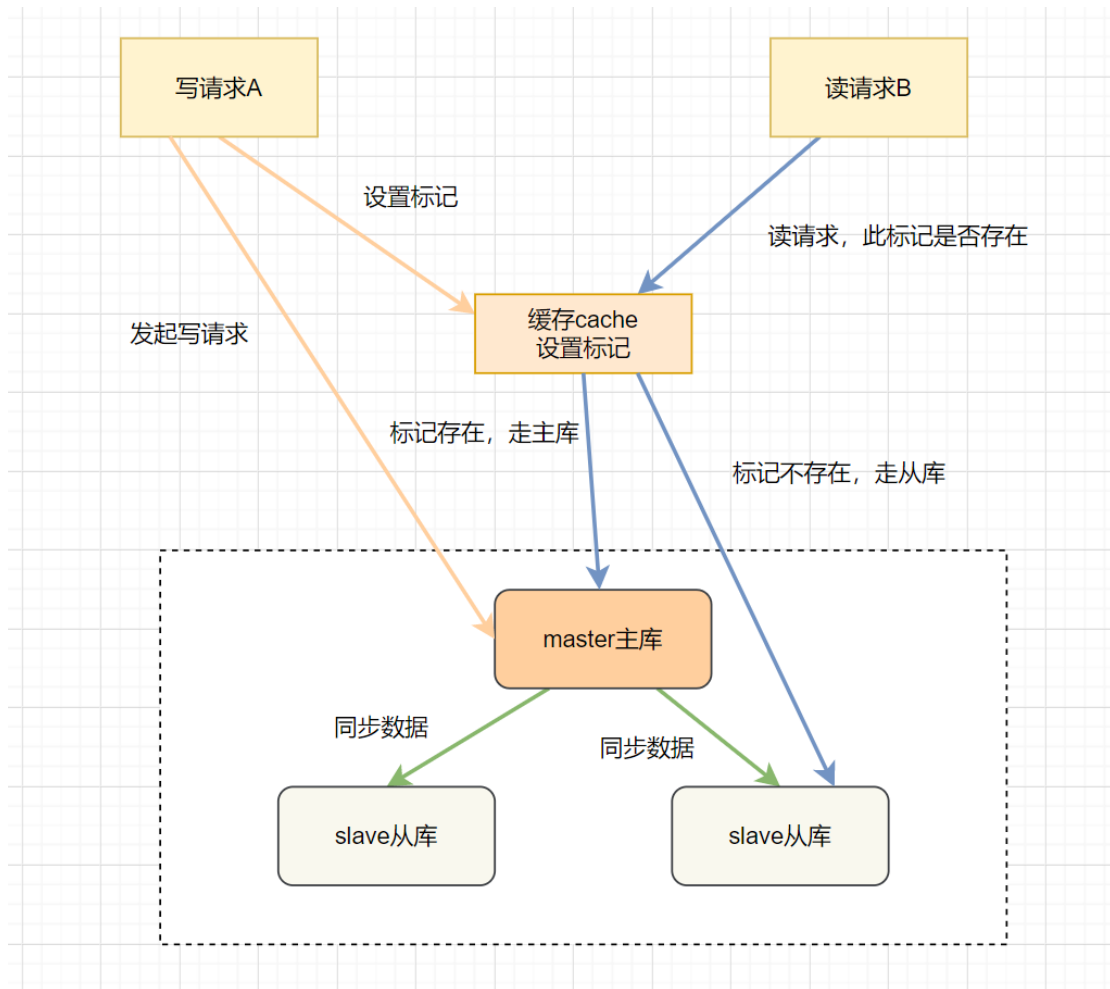
12.8 读写分离的场景下,怎么保证从数据库读到最新的数据?

数据库读写分离，主要解决高并发时，提高系统的吞吐量。来看下读写分离数据库模型：



- 写请求是直接写主库，然后同步数据到从库
- 读请求一般直接读从库，除非强制读主库

在高并发场景或者网络不佳的场景，如果存在较大的主从同步数据延迟，这时候读请求去读从库，就会读到旧数据。这时候最简单暴力的方法，就是**强制读主库**。实际上可以使用**缓存标记法**。



- A 发起写请求，更新主库数据，并在缓存中设置一个标记，表示数据已更新，标记格式为：userId+业务 Id。
- 设置此标记，设置过期时间（估值为主库和从库同步延迟的时间）
- B 发起读请求，先判断此请求，在缓存中有没有更新标记。
- 如果存在标记，走主库；如果没有，请求走从库。

这个方案，解决了数据不一致问题，但是每次请求都要先跟缓存打交道，会影响系统吞吐。

12.9 如何保证 MySQL 数据不丢？

MySQL 这种关系型数据库，是日志先行策略（Write-Ahead Logging），只要 **binlog** 和 **redo log** 日志能保证持久化到磁盘，我们就能确保 MySQL 异常重启后，数据不丢失。

12.9.1 binlog 日志

binlog，又称为二进制日志，它会记录数据库执行更改的所有操作，但是不包括查询 select 等操作。一般用于恢复、复制等功能。它的格式有三种：

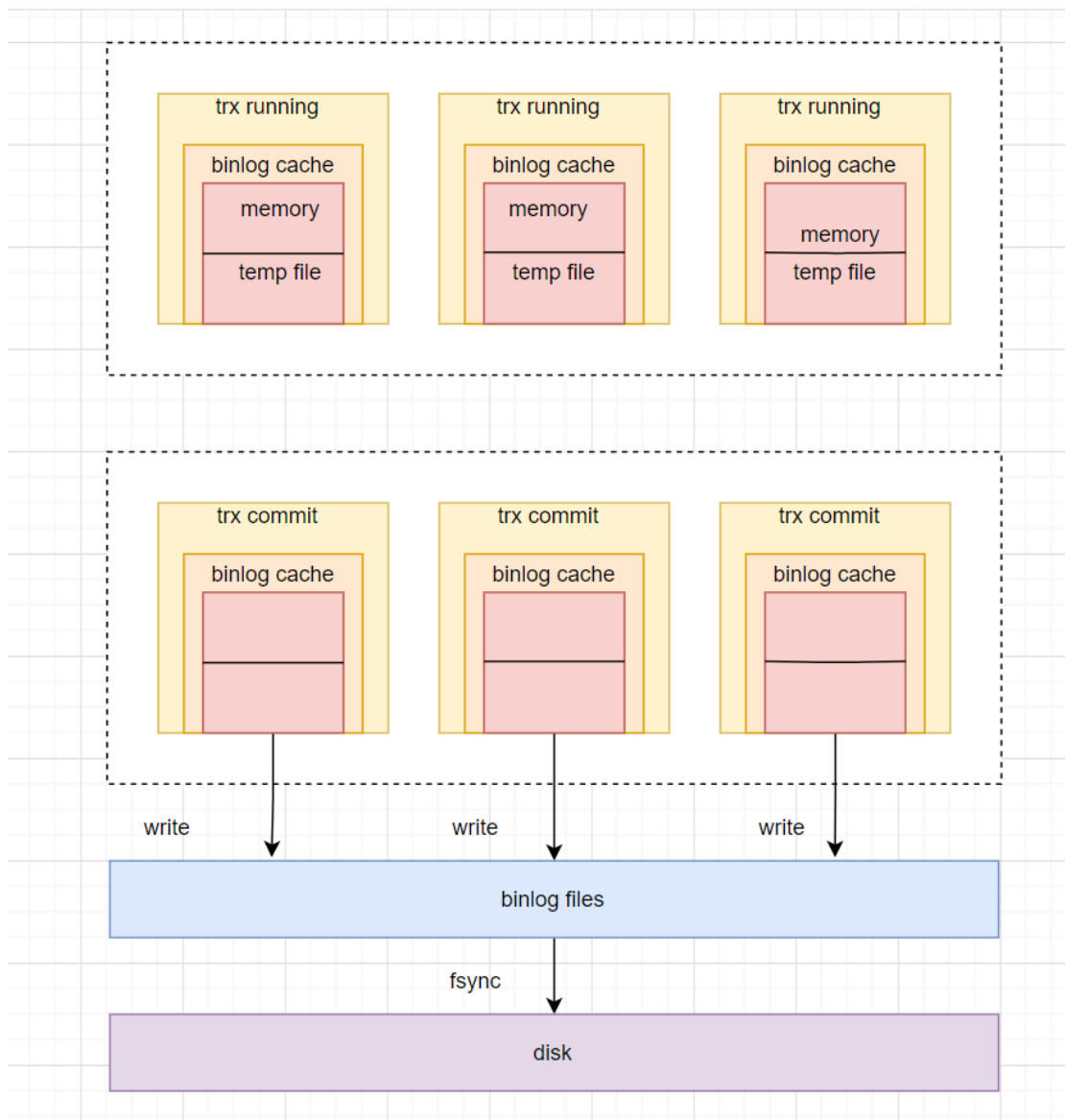
statement、mixed 和 row。

- statement：每一条会修改数据的 sql 都会记录到 binlog 中，不建议使用。
- row：基于行的变更情况记录，会记录行更改前后的内容，**推荐使用**。
- mixed：混合 statement 和 row 两个模式，不建议使用。

binlog 的写入机制是怎样的呢？

事务执行过程中，先把日志写到 binlog cache，事务提交的时候，再把 binlog cache 写到 binlog 文件中。

系统为每个客户端线程分配一个 **binlog cache**，其大小值控制参数是 **binlog_cache_size**。如果 **binlog cache** 的值超过阈值，就会临时持久化到磁盘。当事务提交的时候，再将 binlog cache 中完整的事务持久化到磁盘中，并且清空 binlog cache。



binlog 写文件

binlog 写文件分 **write** 和 **fsync** 两个过程：

- **write**：指把日志写到文件系统的 page cache，并没有把数据持久化到磁盘，因此速度较快。
- **fsync**，实际的写盘操作，即把数据持久化到磁盘。

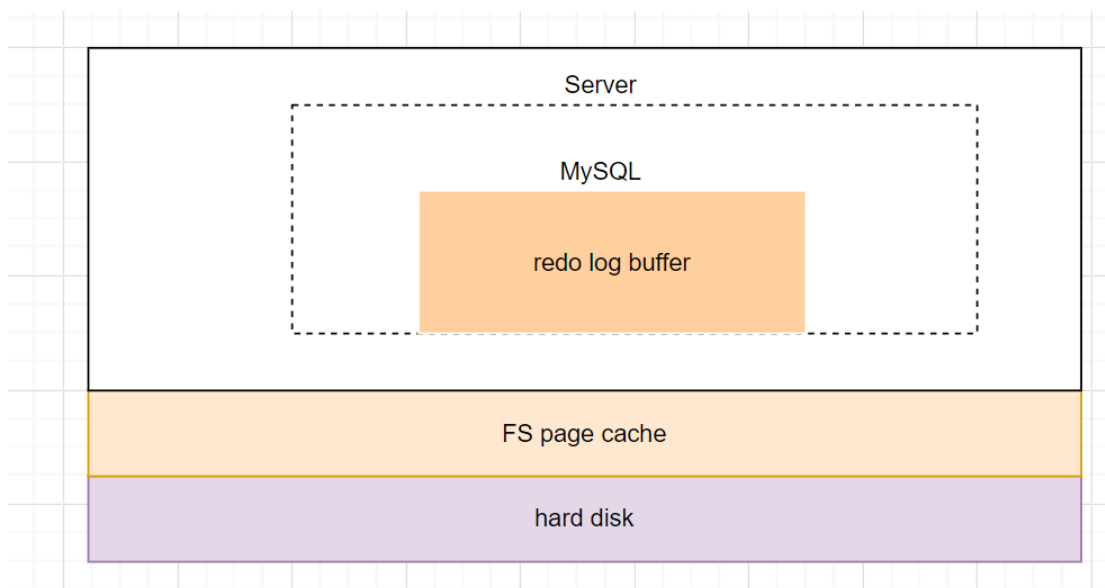
write 和 **fsync** 的写入时机，是由变量 `sync_binlog` 控制的：



如果 IO 出现性能瓶颈，可以将 **sync_binlog** 设置成一个较大的值。比如设置为 (100~1000)。但是，会存在数据丢失的风险，当主机异常重启时，会丢失 **N** 个最近提交的事务 binlog。

12.9.2 redo log 日志

redo log，又称为**重做日志文件**，只记录事务对数据页做了哪些修改，它记录的是数据修改之后的值。redo 有三种状态



- 物理上是在 MySQL 进程内存中，存在 redo log buffer 中，
- 物理上在文件系统的 page cache 里，写到磁盘 (write)，但是还没有持久化 (fsync)。

- 存在 hard disk，已经持久化到磁盘。

日志写到 **redo log buffer** 是很快的；write 到 **page cache** 也很快，但是持久化到磁盘的速度就慢多了。

为了控制 redo log 的写入策略，InnoDB 根据

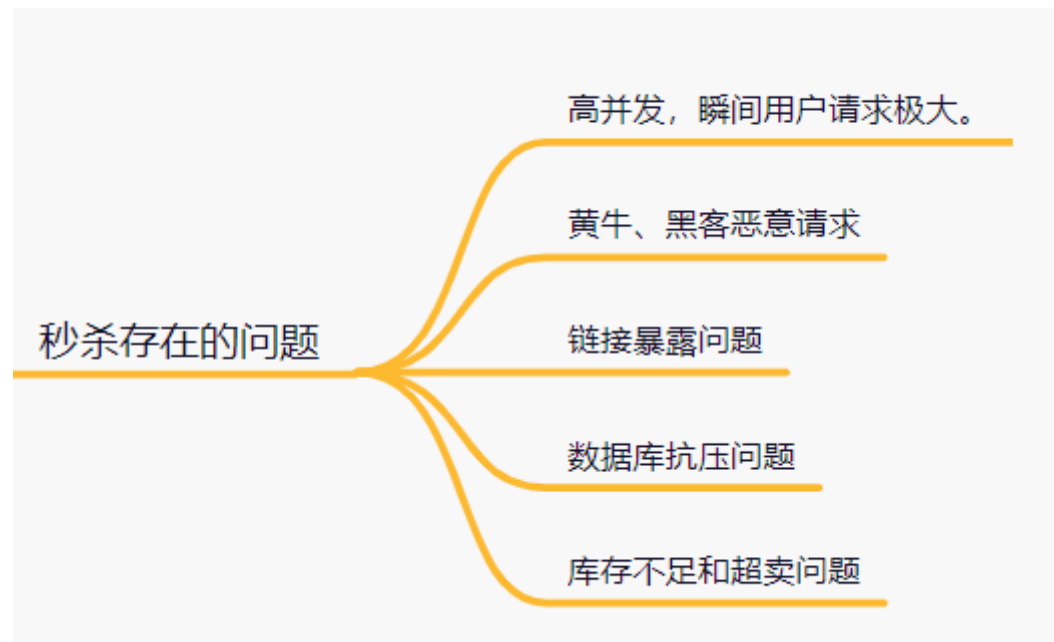
innodb_flush_log_at_trx_commit 参数不同的取值采用不同的策略，它有三种不同的取值：

- 0. 设置为 0 时，表示每次事务提交时都只是把 redo log 留在 redo log buffer 中；
- 0. 设置为 1 时，表示每次事务提交时都将 redo log 直接持久化到磁盘；
- 0. 设置为 2 时，表示每次事务提交时都只是把 redo log 写到 page cache。

三种模式下，0 的性能最好，但是不安全，MySQL 进程一旦崩溃会导致丢失一秒的数据。1 的安全性最高，但是对性能影响最大，2 的话主要由操作系统自行控制刷磁盘的时间，如果仅仅是 MySQL 宕机，对数据不会产生影响，如果是主机异常宕机了，同样会丢失数据。

12.10 高并发下如何设计秒杀系统？

设计一个秒杀系统，需要考虑这些问题：



如何解决这些问题呢？

- 页面静态化
- 按钮至灰控制
- 服务单一职责
- 秒杀链接加盐
- 限流
- 分布式锁
- MQ 异步处理
- 限流&降级&熔断

页面静态化

秒杀活动的页面，大多数内容都是固定不变的，如商品名称，商品图片等等，

可以对活动页面做静态化处理，减少访问服务端的请求。秒杀用户会分布在全

国各地，有的在上海，有的在深圳，地域相差很远，网速也各不相同。为了让用户最快访问到活动页面，可以使用 CDN (Content Delivery Network , 内容分发网络) 。CDN 可以让用户就近获取所需内容。

按钮至灰控制

秒杀活动开始前，按钮一般需要置灰的。只有时间到了，才能变得可以点击。这是防止，秒杀用户在时间快到的前几秒，疯狂请求服务器，然后秒杀时间点还没到，服务器就自己挂了。

服务单一职责

我们都知道微服务设计思想，也就是把各个功能模块拆分，功能那个类似的放在一起，再用分布式的部署方式。

如用户登录相关的，就设计个用户服务，订单相关的就搞个订单服务，再到礼物相关的就搞个礼物服务等等。那么，秒杀相关的业务逻辑也可以放到一起，搞个**秒杀服务**，单独给它搞个秒杀数据库。

服务单一职责有个好处：如果秒杀没抗住高并发的压力，秒杀库崩了，服务挂了，也不会影响到系统的其他服务。

秒杀链接加盐

链接如果明文暴露的话，会有人获取到请求 Url，提前秒杀了。因此，需要给秒杀链接加盐。可以把 URL 动态化，如通过 MD5 加密算法加密随机的字符串去做 url。

限流

一般有两种方式限流：nginx 限流和 redis 限流。

- 为了防止某个用户请求过于频繁，我们可以对**同一用户限流**；
- 为了防止黄牛模拟几个用户请求，我们可以对**某个 IP 进行限流**；
- 为了防止有人使用代理，每次请求都更换 IP 请求，我们可以对**接口进行限流**。
- 为了防止瞬时过大的流量压垮系统，还可以使用阿里的 Sentinel、Hystrix 组件进行限流。

分布式锁

可以使用 redis 分布式锁解决超卖问题。

使用 Redis 的 **SET EX PX NX + 校验唯一随机值**,再删除释放锁。

```
if (jedis.set(key_resource_id, uni_request_id, "NX", "EX", 100s) == 1) { //加锁
    try {
        do something //业务处理
    } catch ({
    }
    finally {
        //判断是不是当前线程加的锁,是才释放
        if (uni_request_id.equals(jedis.get(key_resource_id))) {
            jedis.del(lockKey); //释放锁
        }
    }
}
```

在这里，判断是不是当前线程加的锁和释放锁不是一个原子操作。如果调用 jedis.del() 释放锁的时候，可能这把锁已经不属于当前客户端，会解除他人加的锁。

```

if (jedis.set(key_resource_id, uni_request_id, "NX", "EX", 100s) == 1) { //加锁
    try {
        do something //业务处理
    } catch ({
    }
    finally {
        //判断是不是当前线程加的锁,是才释放
        if (uni_request_id.equals(jedis.get(key_resource_id))) {
            jedis.del(lockKey);
        }
    }
}

```

非原子

为了更严谨，一般也是用 lua 脚本代替。lua 脚本如下：

```

if redis.call('get',KEYS[1]) == ARGV[1] then
    return redis.call('del',KEYS[1])
else
    return 0
end;

```

MQ 异步处理

如果瞬间流量特别大，可以使用消息队列削峰，异步处理。用户请求过来的时候，先放到消息队列，再拿出来消费。

限流&降级&熔断

- 限流，就是限制请求，防止过大的请求压垮服务器；
- 降级，就是秒杀服务有问题了，就降级处理，不要影响别的服务；
- 熔断，服务有问题就熔断，一般熔断降级是一起出现。

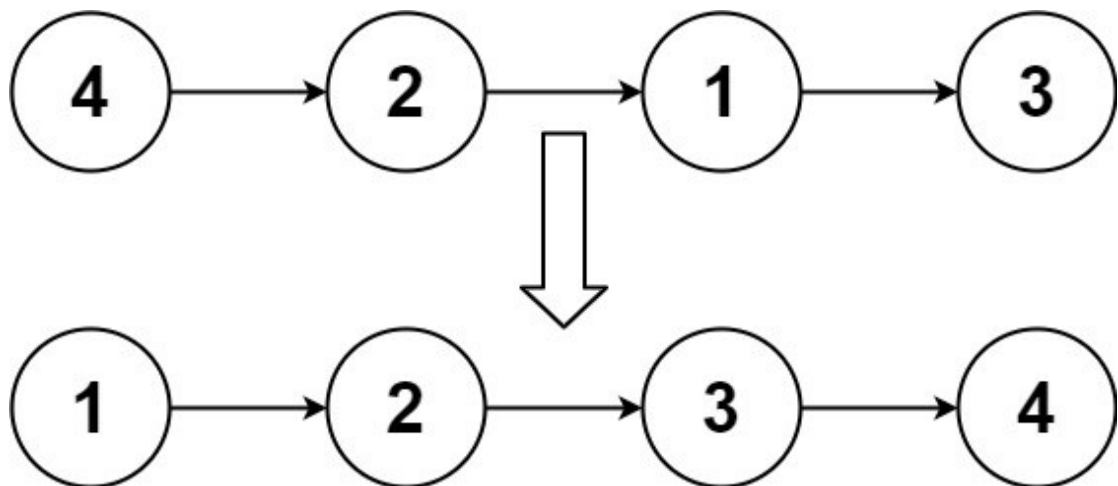
13. 虾皮服务端 15 连问（关注公众号：捡田螺的小男孩。）

大家好，我是捡田螺的小男孩。最近有位读者去虾皮面试啦，分享一下面试的真题~如果有不正确的，欢迎指出哈，一起进步。更多干货，关注公众号：[捡田螺的小男孩](#)。



13.1 排序链表

给你链表的头结点 `head`，请将其按升序排列并返回排序后的链表。

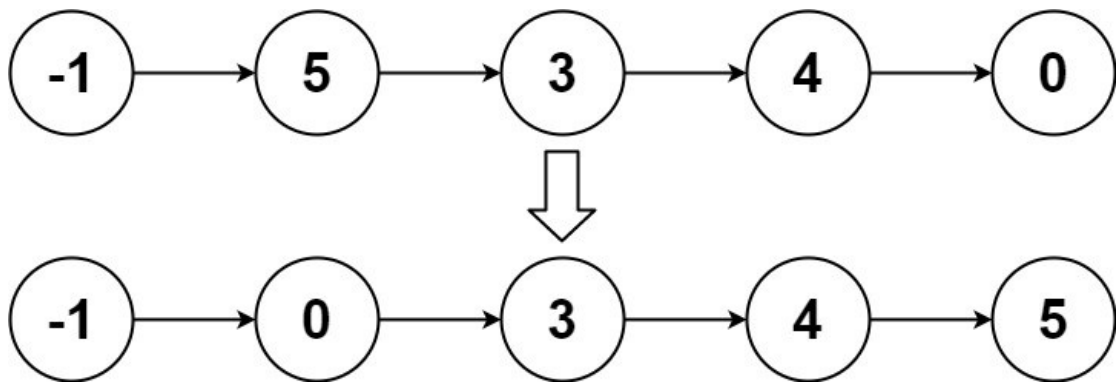


实例 1：

输入：head = [4,2,1,3]

输出：[1,2,3,4]

实例 2：



输入：head = [-1,5,3,4,0]

输出：[-1,0,3,4,5]

这道题可以用双指针+归并排序算法解决，主要以下四个步骤

- 0. 快慢指针法，遍历链表找到中间节点
- 0. 中间节点切断链表
- 0. 分别用归并排序排左右子链表
- 0. 合并子链表

完整代码如下：

```
class Solution {
    public ListNode sortList(ListNode head) {
        //如果链表为空，或者只有一个节点，直接返回即可，不用排序
        if (head == null || head.next == null)
            return head;

        //快慢指针移动，以寻找到中间节点
        ListNode slow = head;
        ListNode fast = head;
        while(fast.next!=null && fast.next.next !=null){
            fast = fast.next.next;
        }
    }
}
```

```

        slow = slow.next;
    }

    //找到中间节点，slow节点的next指针，指向mid
    ListNode mid = slow.next;
    //切断链表
    slow.next = null;

    //排序左子链表
    ListNode left = sortList(head);
    //排序右子链表
    ListNode right = sortList(mid);

    //合并链表
    return merge(left, right);
}

public ListNode merge(ListNode left, ListNode right) {
    ListNode head = new ListNode(0);
    ListNode temp = head;
    while (left != null && right != null) {
        if (left.val <= right.val) {
            temp.next = left;
            left = left.next;
        } else {
            temp.next = right;
            right = right.next;
        }
        temp = temp.next;
    }
    if (left != null) {
        temp.next = left;
    } else if (right != null) {
        temp.next = right;
    }
    return head.next;
}
}

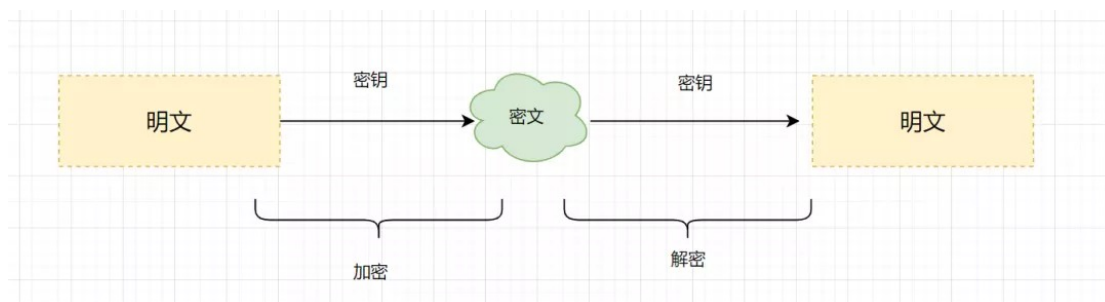
```

13.2 对称与非对称加密算法的区别

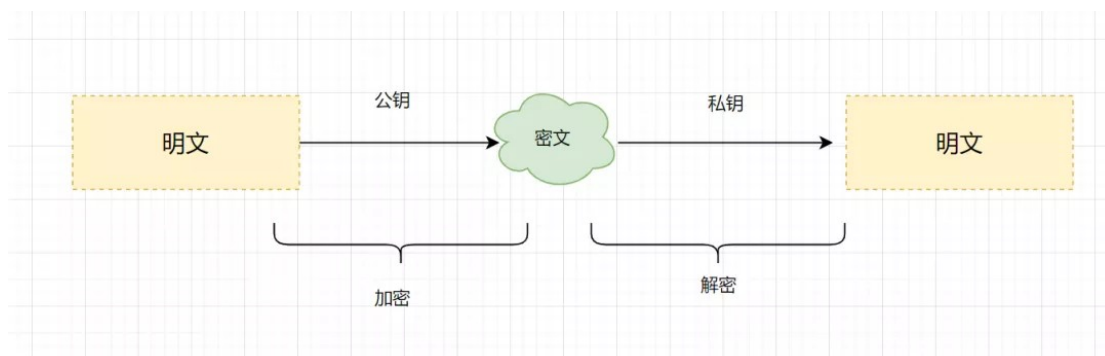
先复习一下相关概念：

- 明文：指没有经过加密的信息/数据。
- 密文：明文被加密算法加密之后，会变成密文，以确保数据安全。
- 密钥：是一种参数，它是在明文转换为密文或将密文转换为明文的算法中输入的参数。密钥分为对称密钥与非对称密钥。
- 加密：将明文变成密文的过程。
- 解密：将密文还原为明文的过程。

对称加密算法:加密和解密使用**相同密钥**的加密算法。常见的对称加密算法有 **AES、3DES、DES、RC5、RC6** 等。



非对称加密算法：非对称加密算法需要两个密钥（公开密钥和私有密钥）。公钥与私钥是成对存在的，如果用公钥对数据进行加密，只有对应的私钥才能解密。主要的非对称加密算法有：**RSA、Elgamal、DSA、D-H、ECC**。



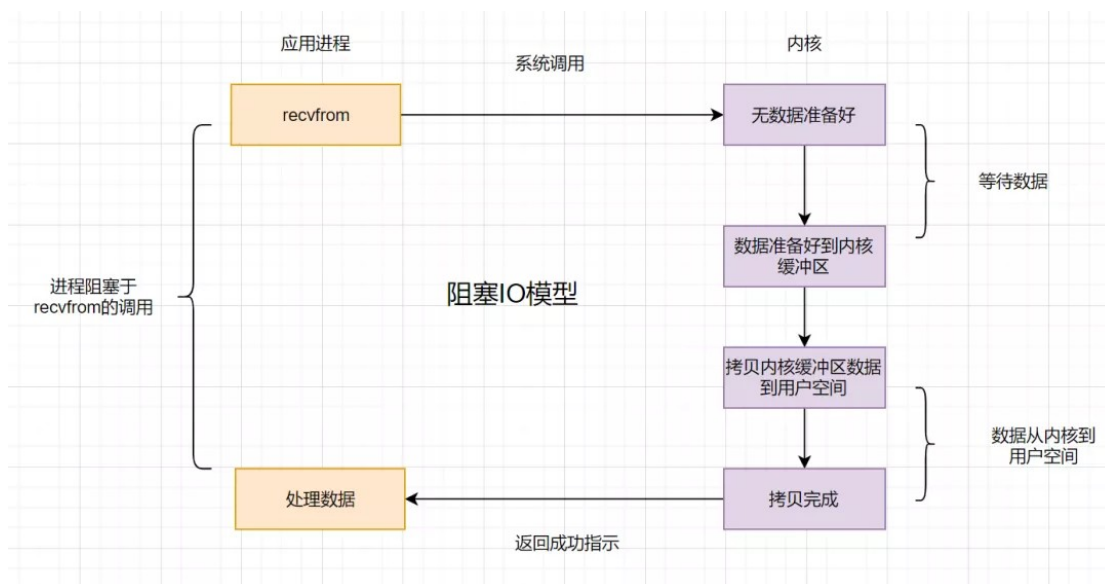
13.3 TCP 如何保证可靠性

- 首先，TCP 的连接是基于三次握手，而断开则是四次挥手。确保连接和断开的可靠性。
- 其次，TCP 的可靠性，还体现在有状态;TCP 会记录哪些数据发送了，哪些数据被接受了，哪些没有被接受，并且保证数据包按序到达，保证数据传输不出差错。
- 再次，TCP 的可靠性，还体现在可控制。它有报文校验、ACK 应答、超时重传(发送方)、失序数据重传 (接收方)、丢弃重复数据、流量控制 (滑动窗口) 和拥塞控制等机制。

13.4 聊聊五种 IO 模型

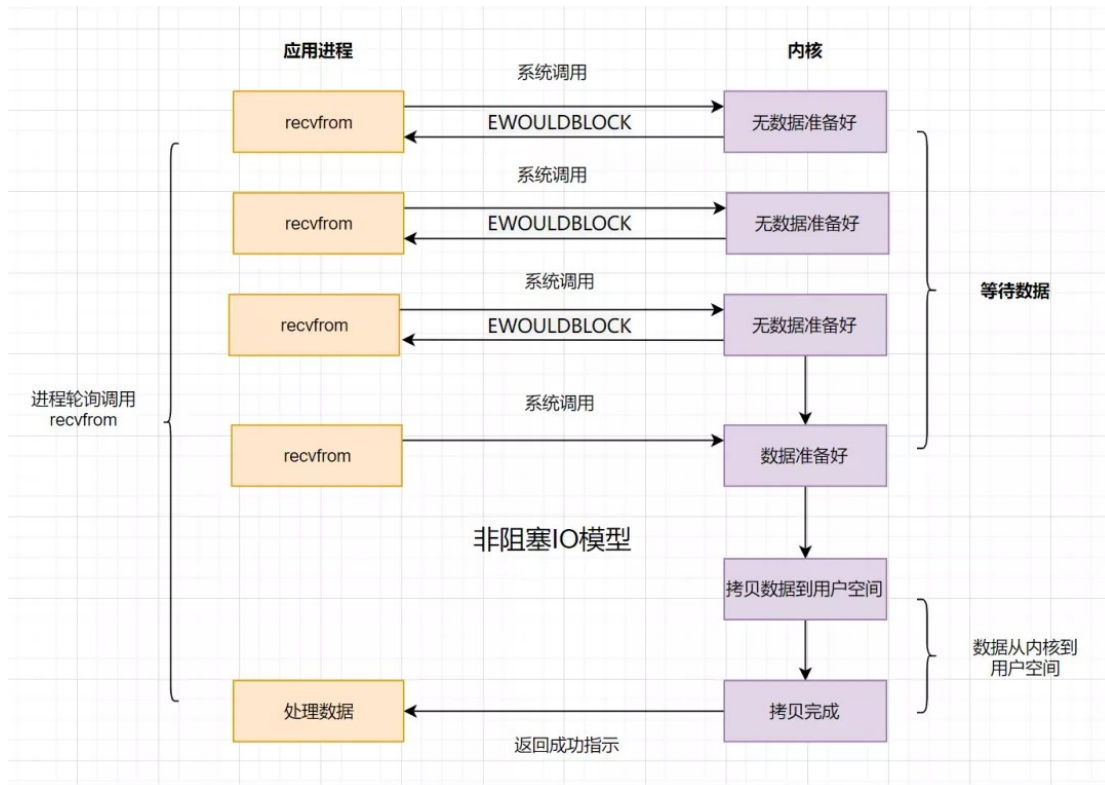
13.4.1 阻塞 IO 模型

假设应用程序的进程发起 IO 调用，但是如果内核的数据还没准备好的话，那应用程序进程就一直在阻塞等待，一直等到内核数据准备好了，从内核拷贝到用户空间，才返回成功提示，此次 IO 操作，称之为阻塞 IO。



13.4.2 非阻塞 IO 模型

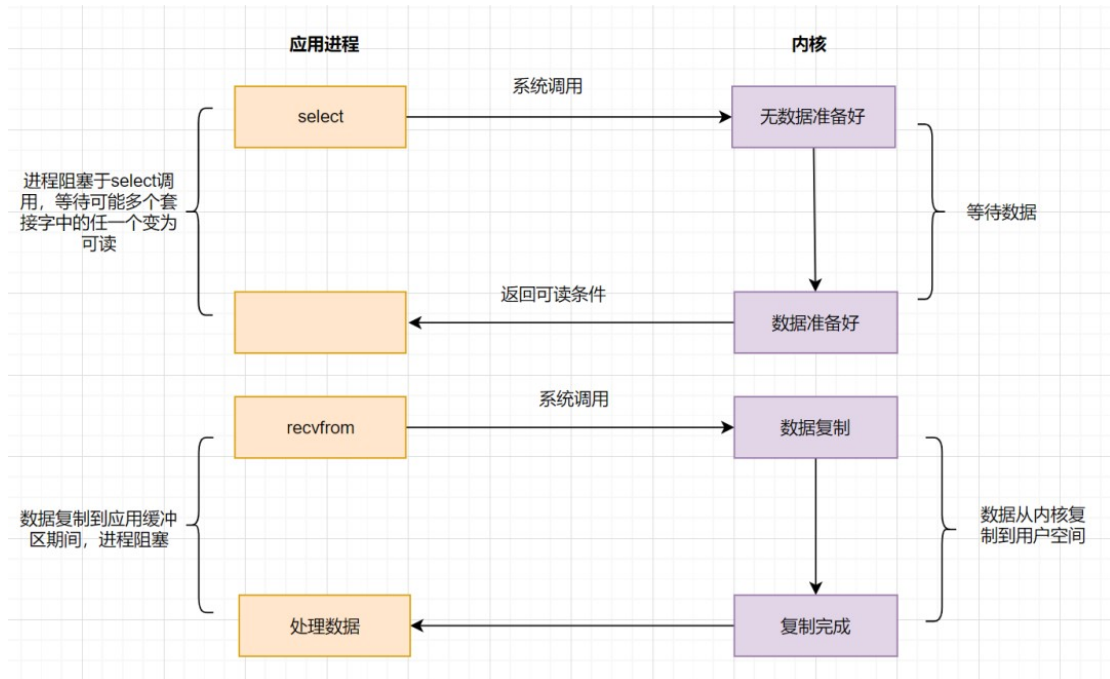
如果内核数据还没准备好，可以先返回错误信息给用户进程，让它不需要等待，而是通过轮询的方式再来请求。这就是非阻塞 IO，流程图如下：



13.4.3 IO 多路复用模型

IO 多路复用之 `select`

应用进程通过调用 `select` 函数，可以同时监控多个 `fd`，在 `select` 函数监控的 `fd` 中，只要有任何一个数据状态准备就绪了，`select` 函数就会返回可读状态，这时应用进程再发起 `recvfrom` 请求去读取数据。

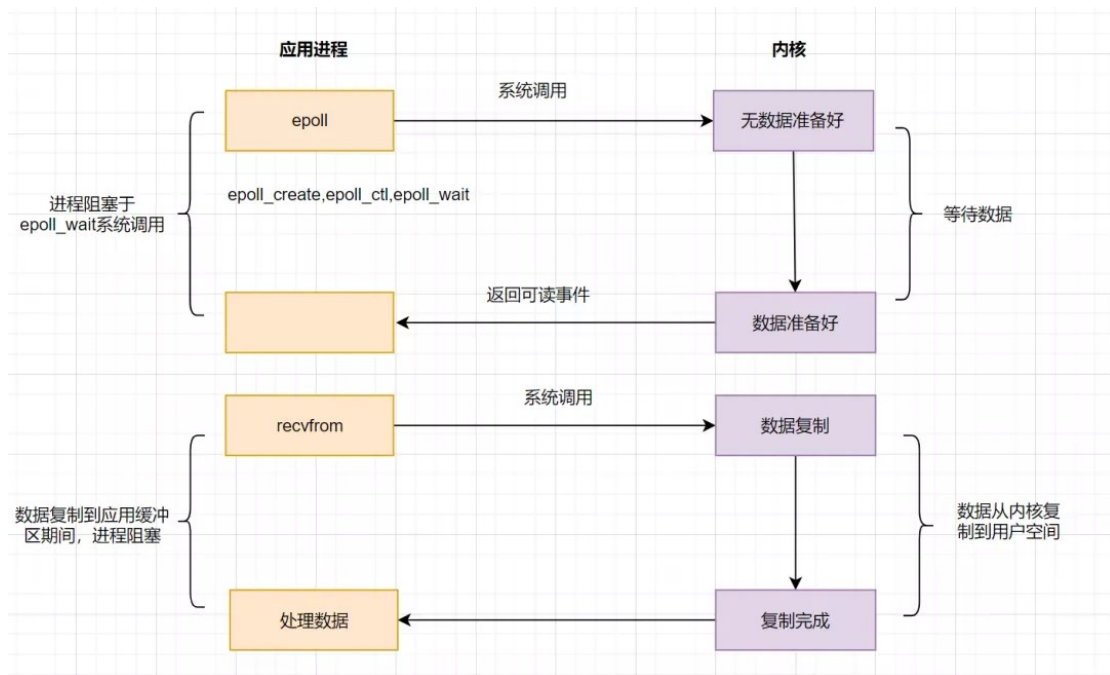


`select` 有几个缺点：

- 最大连接数有限，在 Linux 系统上一般为 1024。
- `select` 函数返回后，是通过遍历 `fdset`，找到就绪的描述符 `fd`。

IO 多路复用之 `epoll`

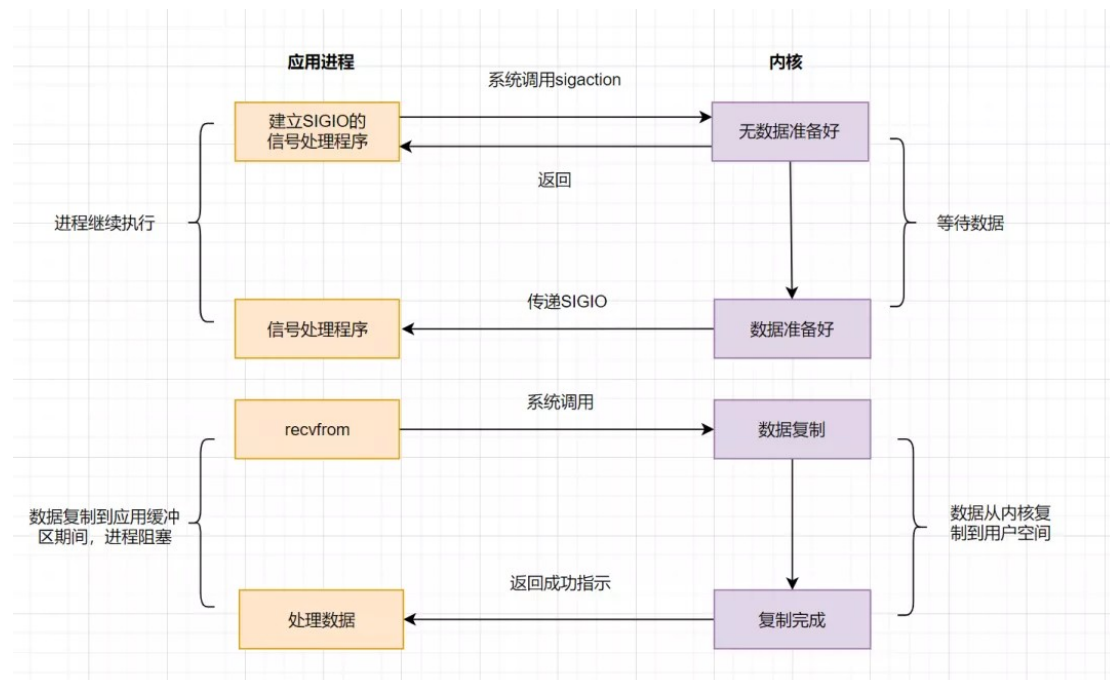
为了解决 `select` 存在的问题，多路复用模型 `epoll` 诞生，它采用事件驱动来实现，流程图如下：



epoll 先通过 `epoll_ctl()` 来注册一个 fd (文件描述符) , 一旦基于某个 fd 就绪时, 内核会采用回调机制, 迅速激活这个 fd , 当进程调用 `epoll_wait()` 时便得到通知。这里去掉了遍历文件描述符的坑爹操作, 而是采用监听事件回调的机制。这就是 epoll 的亮点。

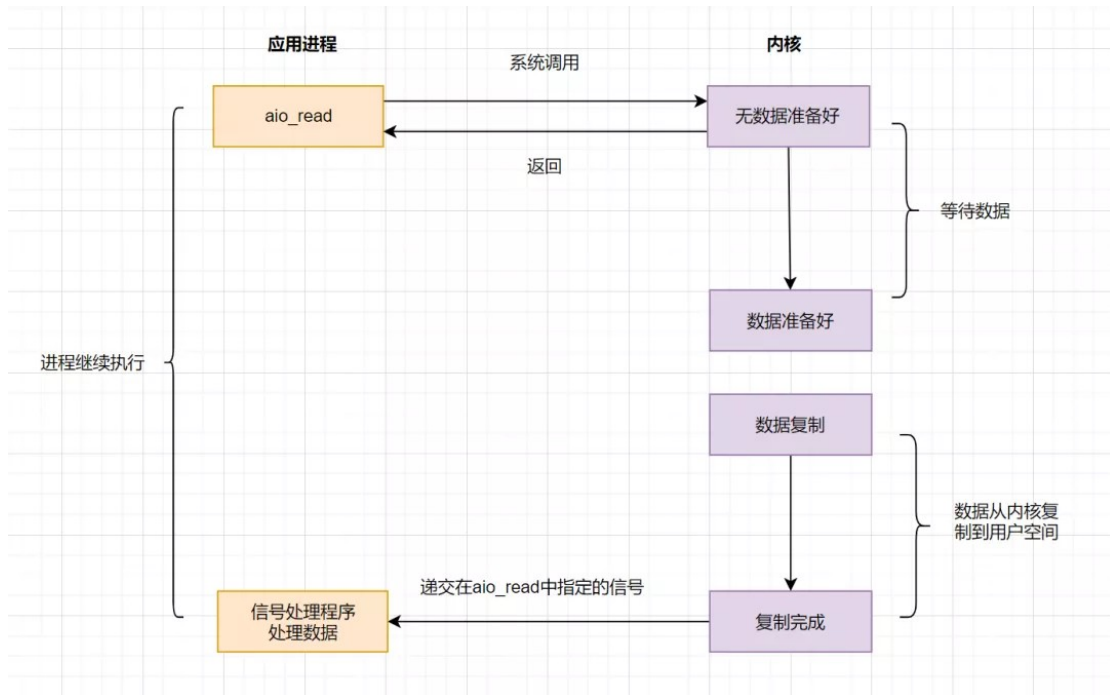
13.4.4 IO 模型之信号驱动模型

信号驱动 IO 不再用主动询问的方式去确认数据是否就绪, 而是向内核发送一个信号 (调用 `sigaction` 的时候建立一个 SIGIO 的信号) , 然后应用用户进程可以去做别的事, 不用阻塞。当内核数据准备好后, 再通过 SIGIO 信号通知应用进程, 数据准备好后的可读状态。应用用户进程收到信号之后, 立即调用 `recvfrom` , 去读取数据。



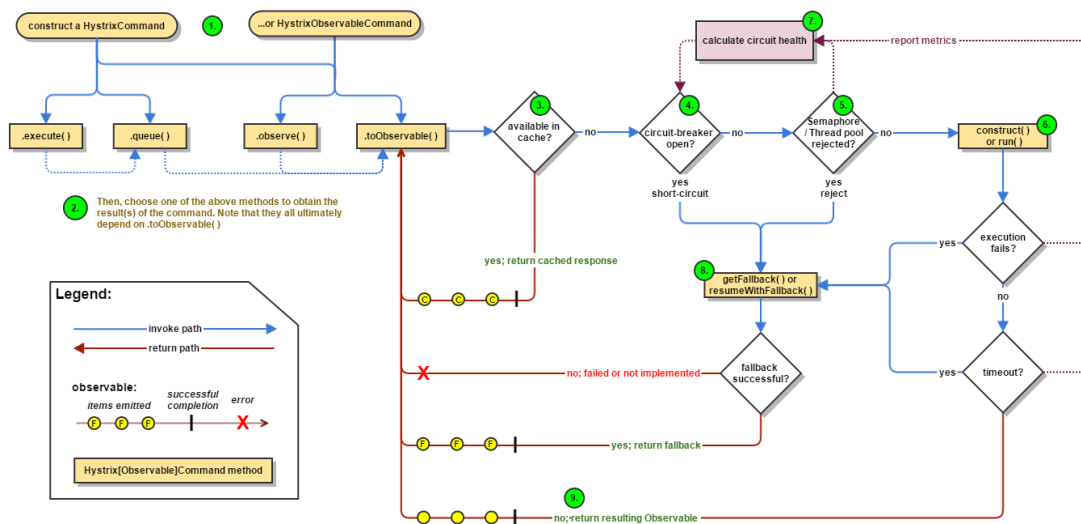
13.4.5 IO 模型之异步 IO(AIO)

AIO 实现了 IO 全流程的非阻塞，就是应用进程发出系统调用后，是立即返回的，但是立即返回的不是处理结果，而是表示提交成功类似的意思。等内核数据准备好，将数据拷贝到用户进程缓冲区，发送信号通知用户进程 IO 操作执行完毕。流程如下：



13.5 hystrix 工作原理

Hystrix 工作流程图如下:



1. 构建命令

Hystrix 提供了两个命令对象：HystrixCommand 和 HystrixObservableCommand，它将代表你的一个依赖请求任务，向构造函数中传入请求依赖所需要的参数。

1. 执行命令

有四种方式执行 Hystrix 命令。分别是：

- `R execute()`：同步阻塞执行的，从依赖请求中接收到单个响应。
- `Future queue()`：异步执行，返回一个包含单个响应的 Future 对象。
- `Observable observe()`：创建 Observable 后会订阅 Observable，从依赖请求中返回代表响应的 Observable 对象
- `Observable toObservable()`：cold observable，返回一个 Observable，只有订阅时才会执行 Hystrix 命令，可以返回多个结果

1. 检查响应是否被缓存

如果启用了 Hystrix 缓存，任务执行前将先判断是否有相同命令执行的缓存。如果有则直接返回包含缓存响应的 Observable；如果没有缓存的结果，但启动了缓存，将缓存本次执行结果以供后续使用。

1. 检查回路器是否打开 回路器(circuit-breaker)和保险丝类似，保险丝在发生危险时将会烧断以保护电路，而回路器可以在达到我们设定的阈值时触发短路(比如请求失败率达到 50%)，拒绝执行任何请求。

如果回路器被打开，Hystrix 将不会执行命令，直接进入 Fallback 处理逻辑。

1. 检查线程池/信号量/队列情况 Hystrix 隔离方式有线程池隔离和信号量隔离。当使用 Hystrix 线程池时，Hystrix 默认为每个依赖服务分配 10 个线程，当 10 个线程都繁忙时，将拒绝执行命令，而是立即跳到执行 fallback 逻辑。
2. 执行具体的任务 通过 `HystrixObservableCommand.construct()` 或者 `HystrixCommand.run()` 来运行用户真正的任务。
3. 计算回路健康情况 每次开始执行 command、结束执行 command 以及发生异常等情况时，都会记录执行情况，例如：成功、失败、拒绝和超时等指标情况，会定期处理这些数据，再根据设定的条件来判断是否开启回路器。
4. 命令失败时执行 Fallback 逻辑 在命令失败时执行用户指定的 Fallback 逻辑。上图中的断路、线程池拒绝、信号量拒绝、执行执行、执行超时都会进入 Fallback 处理。
5. 返回执行结果 原始对象结果将以 Observable 形式返回，在返回给用户之前，会根据调用方式的不同做一些处理。

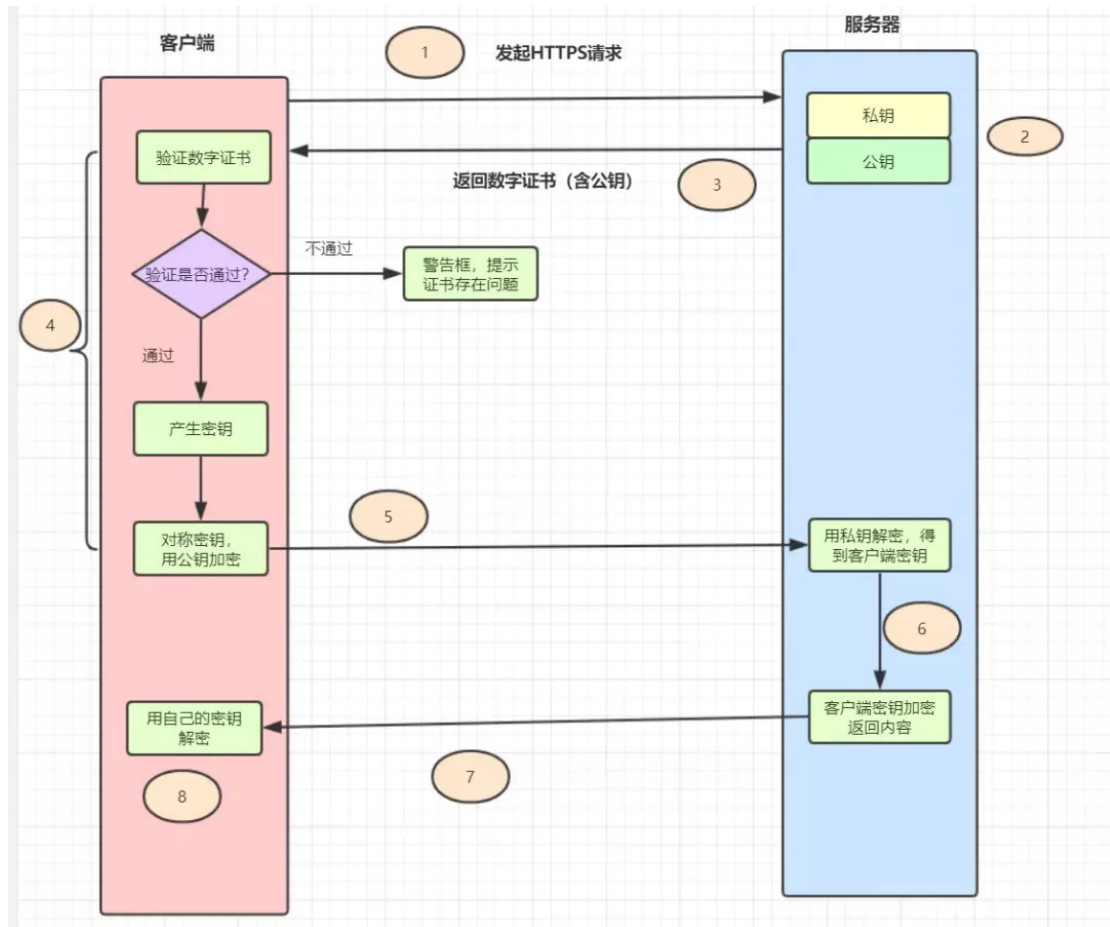
13.6 延时场景处理

日常开发中，我们经常遇到这种业务场景，如：外卖订单超 30 分钟未支付，则自动取消订单；用户注册成功 15 分钟后，发短信消息通知用户等等。这就是延时任务处理场景。针对此类场景我们主要有以下几种处理方案：

- JDK 的 DelayQueue 延迟队列
- 时间轮算法
- 数据库定时任务 (如 Quartz)
- Redis ZSet 实现
- MQ 延时队列实现

13.7 https 请求过程

- HTTPS = HTTP + SSL/TLS , 即用 SSL/TLS 对数据进行加密和解密 ,
Http 进行传输。
- SSL , 即 Secure Sockets Layer (安全套接层协议) , 是网络通信提供安全及数据完整性的一种安全协议。
- TLS , 即 Transport Layer Security(安全传输层协议) , 它是 SSL 3.0 的后续版本。



http 请求流程

1. 用户在浏览器里输入一个 https 网址，然后连接到 server 的 443 端口。
2. 服务器必须要有一套数字证书，可以自己制作，也可以向组织申请，区别就是自己颁发的证书需要客户端验证通过。这套证书其实就是一对公钥和私钥。
3. 服务器将自己的数字证书（含有公钥）发送给客户端。
4. 客户端收到服务器端的数字证书之后，会对其进行检查，如果不通过，则弹出警告框。如果证书没问题，则生成一个密钥（对称加密），用证书的公钥对它加密。

5. 客户端会发起 HTTPS 中的第二个 HTTP 请求，将加密之后的客户端密钥发送给服务器。
6. 服务器接收到客户端发来的密文之后，会用自己的私钥对其进行非对称解密，解密之后得到客户端密钥，然后用客户端密钥对返回数据进行对称加密，这样数据就变成了密文。
7. 服务器将加密后的密文返回给客户端。
8. 客户端收到服务器发返回的密文，用自己的密钥（客户端密钥）对其进行对称解密，得到服务器返回的数据。

13.8 聊聊事务隔离级别，以及可重复读实现原理

13.8.1 数据库四大隔离级别

为了解决并发事务存在的脏读、不可重复读、幻读等问题，数据库大叔设计了四种隔离级别。分别是读未提交，读已提交，可重复读，串行化 (Serializable)。

- 读未提交隔离级别：只限制了两个数据不能同时修改，但是修改数据的时候，即使事务未提交，都是可以被别的事务读取到的，这级别的事务隔离有脏读、重复读、幻读的问题；
- 读已提交隔离级别：当前事务只能读取到其他事务提交的数据，所以这种事务的隔离级别解决了脏读问题，但还是会存在重复读、幻读问题；

- 可重复读：限制了读取数据的时候，不可以进行修改，所以解决了重复读的问题，但是读取范围数据的时候，是可以插入数据，所以还会存在幻读问题；
- 串行化：事务最高的隔离级别，在该级别下，所有事务都是进行串行化顺序执行的。可以避免脏读、不可重复读与幻读所有并发问题。但是这种事务隔离级别下，事务执行很耗性能。

四大隔离级别，都会存在哪些并发问题呢

隔离级别	脏读	不可重复读	幻读
读未提交	√	√	√
读已提交	×	√	√
可重复读	×	×	√
串行化	×	×	×

13.8.2 Read View 可见性规则

变量	描述
m_ids	当前系统中那些活跃(未提交)的读写事务 ID, 它数据结构为一个 List。
max_limit_id	表示生成 Read View 时，系统中应该分配给下一个事务的 id 值。

变量	描述
min_limit_id	表示在生成 Read View 时，当前系统中活跃的读写事务中最小的事务 id，即 m_ids 中的最小值。
creator_trx_id	创建当前 Read View 的事务 ID

Read View 的可见性规则如下：

1. 如果数据事务 ID `trx_id < min_limit_id`，表明生成该版本的事务在生成 Read View 前，已经提交(因为事务 ID 是递增的)，所以该版本可以被当前事务访问。
2. 如果 `trx_id >= max_limit_id`，表明生成该版本的事务在生成 Read View 后才生成，所以该版本不可以被当前事务访问。
3. 如果 `min_limit_id <= trx_id < max_limit_id`，需要分 3 种情况讨论
 - 1) 如果 m_ids 包含 `trx_id`，则代表 Read View 生成时刻，这个事务还未提交，但是如果数据的 `trx_id` 等于 `creator_trx_id` 的话，表明数据是自己生成的，因此是可见的。
 - 2) 如果 m_ids 包含 `trx_id`，并且 `trx_id` 不等于 `creator_trx_id`，则 Read View 生成时，事务未提交，并且不是自己生产的，所以当前事务也是看不见的；
 - 3) 如果 m_ids 不包含 `trx_id`，则说明你这个事务在 Read View 生成之前就已经提交了，修改的结果，当前事务是能看见的。

13.8.3 可重复读实现原理

数据库是通过加锁实现隔离级别的，比如，你想一个人静静，不被别人打扰，你可以把自己关在房子，并在房门上加上一把锁！串行化隔离级别就是加锁实

现的。但是如果频繁加锁，性能会下降。因此设计数据库的大叔想到了

MVCC。

可重复读的实现原理就是 **MVCC 多版本并发控制**。在一个事务范围内，两个相同的查询，读取同一条记录，却返回了不同的数据，这就是**不可重复读**。可重复读隔离级别，就是为了解决**不可重复读**问题。

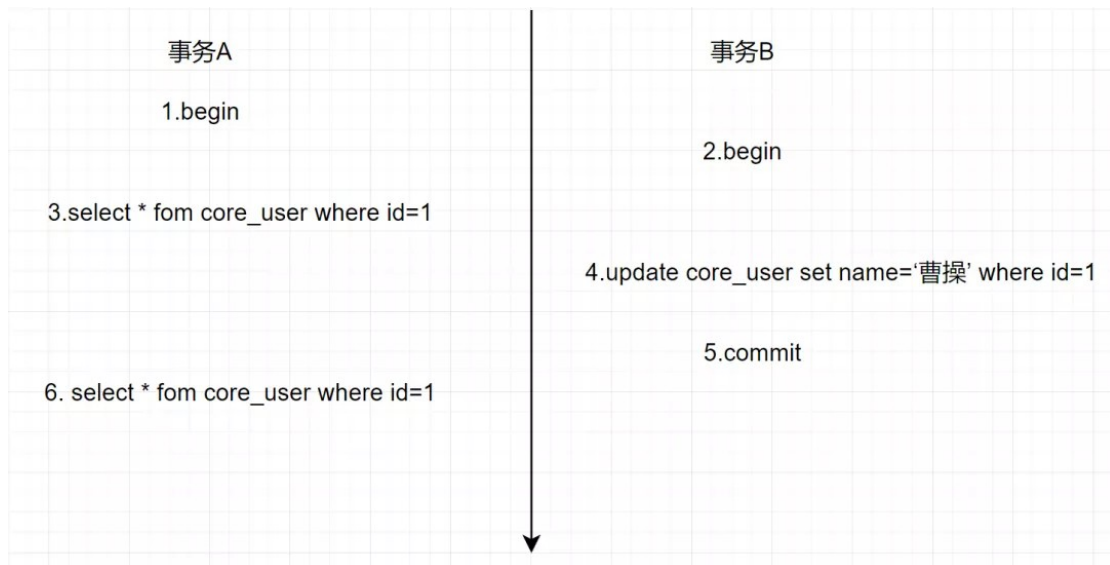
查询一条记录，基于 **MVCC**，是怎样的流程呢？

1. 获取事务自己的版本号，即事务 ID
2. 获取 Read View
3. 查询得到的数据，然后 Read View 中的事务版本号进行比较。
4. 如果不符合 Read View 的可见性规则，即就需要 Undo log 中历史快照；
5. 最后返回符合规则的数据

InnoDB 实现 **MVCC**，是通过 **Read View+ Undo Log** 实现的，**Undo Log** 保存了历史快照，**Read View** 可见性规则帮助判断当前版本的数据是否可见。

可重复读 (RR) 隔离级别，是如何解决不可重复读问题的？

假设存在事务 A 和 B，SQL 执行流程如下



在可重复读 (RR) 隔离级别下，一个事务里只会获取一次 **read view**，都是副本共用的，从而保证每次查询的数据都是一样的。

假设当前有一张 core_user 表，插入一条初始化数据,如下：

id	name	age	trx_id	roll_pointer
1	孙权	18	100	

基于 MVCC，我们来看看执行流程

1. A 开启事务，首先得到一个事务 ID 为 100
2. B 开启事务，得到事务 ID 为 101
3. 事务 A 生成一个 Read View，read view 对应的值如下

变量	值
m_ids	100 , 101
max_limit_id	102

变量	值
min_limit_id	100
creator_trx_id	100

然后回到版本链：开始从版本链中挑选可见的记录：

id	name	age	trx_id	roll_pointer
1	孙权	18	100	

由图可以看出，最新版本的列 name 的内容是孙权，该版本的 trx_id 值为

100。开始执行 **read view** 可见性规则校验：

```
min_limit_id(100) <= trx_id (100) < 102;
creator_trx_id = trx_id = 100;
```

由此可得，trx_id=100 的这个记录，当前事务是可见的。所以查到是 **name** 为孙权的记录。

1. 事务 B 进行修改操作，把名字改为曹操。把原数据拷贝到 undo log, 然后对数据进行修改，标记事务 ID 和上一个数据版本在 undo log 的地址。

id	name	age	trx_id	roll_pointer
1	曹操	18	101	

id	name	age	trx_id	roll_pointer
1	孙权	18	100	

undo 日志

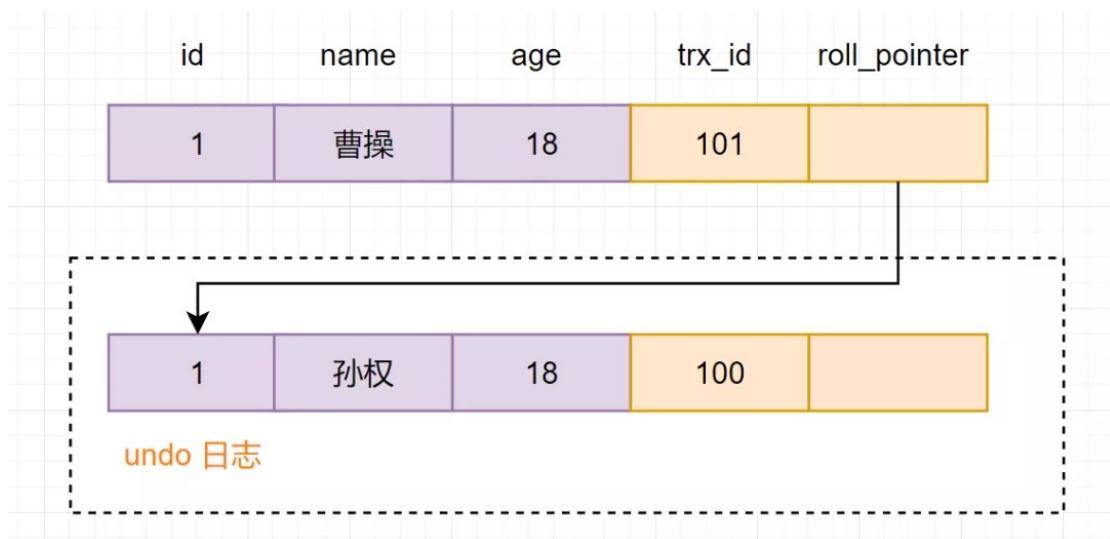
1. 事务 B 提交事务

2. 事务 A 再次执行查询操作，因为是 RR (可重复读) 隔离级别，因此会

复用老的 Read View 副本，Read View 对应的值如下

变量	值
m_ids	100 , 101
max_limit_id	102
min_limit_id	100
creator_trx_id	100

然后再次回到版本链：从版本链中挑选可见的记录：



从图可得，最新版本的列 name 的内容是曹操，该版本的 trx_id 值为 101。开

始执行 read view 可见性规则校验：

```
min_limit_id(100) <= trx_id ( 101 ) < max_limit_id ( 102 );
```

```
因为 m_ids { 100, 101 } 包含 trx_id ( 101 ) ,
```

```
并且 creator_trx_id ( 100 ) 不等于 trx_id ( 101 )
```

所以，**trx_id=101** 这个记录，对于当前事务是不可见的。这时候呢，版本链

roll_pointer 跳到下一个版本，**trx_id=100** 这个记录，再次校验是否可见：

```
min_limit_id(100)=<trx_id(100)<max_limit_id(102);  
因为 m_ids{100,101}包含 trx_id(100) ,  
并且 creator_trx_id(100) 等于 trx_id(100)
```

所以，trx_id=100 这个记录，对于当前事务是可见的，所以两次查询结果，都是 name=孙权的那个记录。即在可重复读（RR）隔离级别下，复用老的 Read View 副本，解决了不可重复读的问题。

13.9 聊聊索引在哪些场景下会失效？

- - 0. 查询条件包含 or，可能导致索引失效
- - 0. 如何字段类型是字符串，where 时一定要用引号括起来，否则索引失效
- - 0. like 通配符可能导致索引失效。
- - 0. 联合索引，查询时的条件列不是联合索引中的第一个列，索引失效。
- - 0. 在索引列上使用 mysql 的内置函数，索引失效。
- - 0. 对索引列运算（如，+、-、*、/），索引失效。
-

0. 索引字段上使用 (!= 或者 < > , not in) 时 , 可能会导致索引失效。

•

0. 索引字段上使用 is null , is not null , 可能导致索引失效。

•

0. 左连接查询或者右连接查询查询关联的字段编码格式不一样 , 可能导致索引失效。

•

0. mysql 估计使用全表扫描要比使用索引快,则不使用索引。

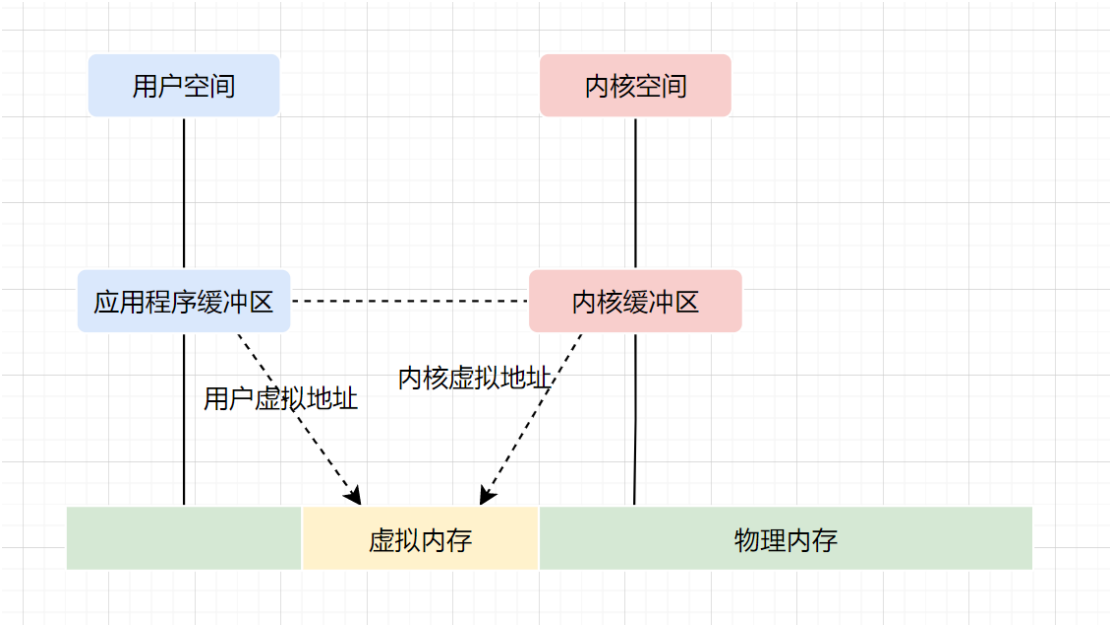
13.10 什么是虚拟内存

虚拟内存，是虚拟出来的内存，它的核心思想就是确保每个程序拥有自己的地址空间，地址空间被分成多个块，每一块都有连续的地址空间。同时物理空间也分成多个块，块大小和虚拟地址空间的块大小一致，操作系统会自动将虚拟地址空间映射到物理地址空间，程序只需关注虚拟内存，请求的也是虚拟内存，真正使用却是物理内存。

现代操作系统使用**虚拟内存**，即虚拟地址取代物理地址，使用虚拟内存可以有 2 个好处：

- 虚拟内存空间可以远远大于物理内存空间
- 多个虚拟内存可以指向同一个物理地址

零拷贝实现思想，就利用了**虚拟内存**这个点：多个虚拟内存可以指向同一个物理地址，可以把内核空间和用户空间的虚拟地址映射到同一个物理地址，这样的话，就可以减少 IO 的数据拷贝次数啦，示意图如下：



13.11 排行榜的实现，比如高考成绩排序

排行版的实现，一般使用 redis 的 **zset** 数据类型。

- 使用格式如下：

```
zadd key score member [score member ...], zrank key member
```

- 层内部编码：ziplist (压缩列表)、skiplist (跳跃表)
- 使用场景如排行榜，社交需求 (如用户点赞)

实现 demo 如下：

```
> zadd gaokao 680 "小明"

(integer) 1

> zadd gaokao 580 "小华"

(integer) 1

> zadd gaokao 600 "小红"

(integer) 1

> zrange gaokao 0 -1 withscores

1) "小华"
2) 580.0
3) "小红"
4) 600.0
5) "小明"
6) 680.0
```

13.12 分布式锁实现

分布式锁，是控制分布式系统不同进程共同访问共享资源的一种锁的实现。秒杀下单、抢红包等等业务场景，都需要用到分布式锁，我们项目中经常使用 Redis 作为分布式锁。

选了 Redis 分布式锁的几种实现方法，大家来讨论下，看有没有啥问题哈。

- 命令 setnx + expire 分开写
- setnx + value 值是过期时间

- set 的扩展命令 (set ex px nx)
- set ex px nx + 校验唯一随机值,再删除
- Redisson

13.12.1 命令 setnx + expire 分开写

```
if ( jedis.setnx(key,lock_value) == 1 ) { //加锁
    expire ( key , 100 ); //设置过期时间
    try {
        do something //业务请求
    }catch(){
    }
    finally {
        jedis.del(key); //释放锁
    }
}
```

如果执行完 **setnx** 加锁，正要执行 **expire** 设置过期时间时，进程 crash 掉或者要重启维护了，那这个锁就“长生不老”了，别的线程永远获取不到锁啦，所以分布式锁不能这么实现。

13.12.2 setnx + value 值是过期时间

```
long expires = System.currentTimeMillis() + expireTime; //系统时间+设置的过期时间
String expiresStr = String.valueOf(expires);

// 如果当前锁不存在，返回加锁成功
if (jedis.setnx(key, expiresStr) == 1) {
    return true;
}

// 如果锁已经存在，获取锁的过期时间
String currentValueStr = jedis.get(key);

// 如果获取到的过期时间，小于系统当前时间，表示已经过期
if (currentValueStr != null && Long.parseLong(currentValueStr) < System.currentTimeMillis()) {
```

```

    // 锁已过期，获取上一个锁的过期时间，并设置现在锁的过期时间（不了解 redis 的 getSet 命令的小伙伴，可以去官网看下哈）
    String oldValueStr = jedis.getSet(key_resource_id, expiresStr);

    if (oldValueStr != null && oldValueStr.equals(currentValueStr)) {
        // 考虑多线程并发的情况，只有一个线程的设置值和当前值相同，它才可以加锁
        return true;
    }
}
//其他情况，均返回加锁失败
return false;
}

```

笔者看过有开发小伙伴就是这么实现分布式锁的，但是这种方案也有这些缺点：

- 过期时间是客户端自己生成的，分布式环境下，每个客户端的时间必须同步。
- 没有保存持有者的唯一标识，可能被别的客户端释放/解锁。
- 锁过期的时候，并发多个客户端同时请求过来，都执行了

`jedis.getSet()`，最终只能有一个客户端加锁成功，但是该客户端锁的过期时间，可能被别的客户端覆盖。

13.12.3：set 的扩展命令（set ex px nx）（注意可能存在的问题）

```

if (jedis.set(key, lock_value, "NX", "EX", 100s) == 1) { //加锁
    try {
        do something //业务处理
    } catch ({
    }
}
finally {
    jedis.del(key); //释放锁
}

```

```
}  
}
```

这个方案可能存在这样的问题：

- 锁过期释放了，业务还没执行完。
- 锁被别的线程误删。

13.12.4 set ex px nx + 校验唯一随机值,再删除

```
if (jedis.set(key, uni_request_id, "NX", "EX", 100s) == 1) { //加锁  
    try {  
        do something //业务处理  
    } catch ({  
    }  
    finally {  
        //判断是不是当前线程加的锁,是才释放  
        if (uni_request_id.equals(jedis.get(key))) {  
            jedis.del(key); //释放锁  
        }  
    }  
}
```

在这里，判断当前线程加的锁和释放锁是不是一个原子操作。如果调用
`jedis.del()`释放锁的时候，可能这把锁已经不属于当前客户端，会解除他人加的锁。

一般也是用 lua 脚本代替。lua 脚本如下：

```
if redis.call('get',KEYS[1]) == ARGV[1] then  
    return redis.call('del',KEYS[1])  
else  
    return 0  
end;
```

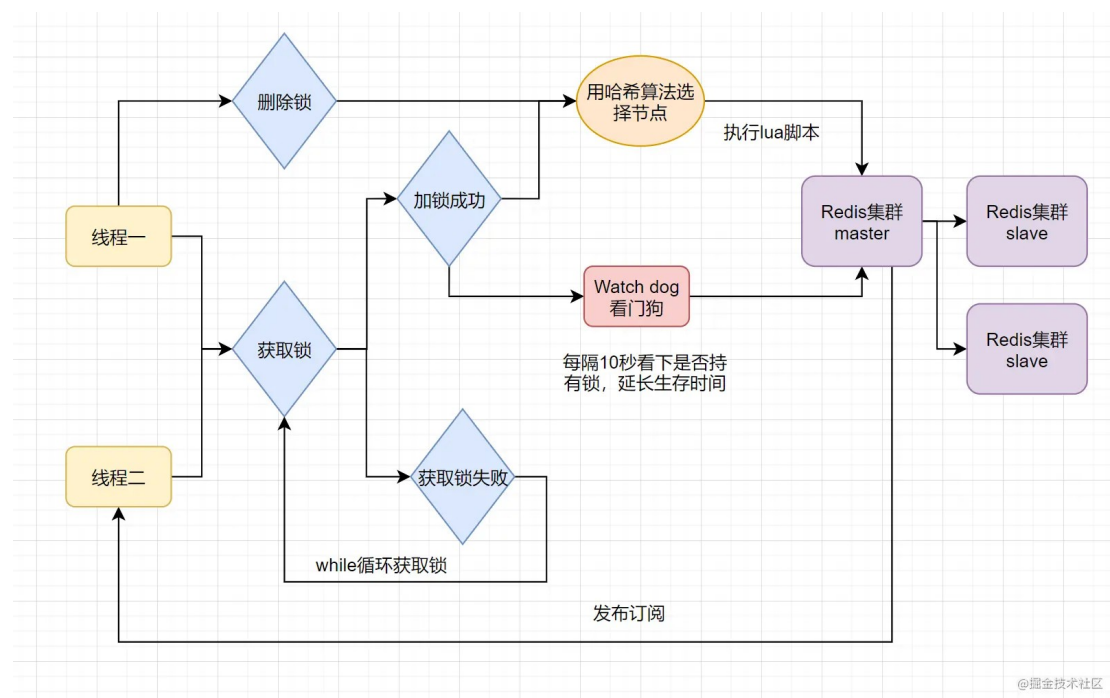
这种方式比较不错了，一般情况下，已经可以使用这种实现方式。但是存在锁过期释放了，业务还没执行完的问题（实际上，估算个业务处理的时间，一般没啥问题了）。

13.12.5 Redisson

分布式锁可能存在锁过期释放，业务没执行完的问题。有些小伙伴认为，稍微把锁过期时间设置长一些就可以啦。其实我们设想一下，是否可以给获得锁的线程，开启一个定时守护线程，每隔一段时间检查锁是否还存在，存在则对锁的过期时间延长，防止锁过期提前释放。

当前开源框架 Redisson 就解决了这个分布式锁问题。我们一起来看下

Redisson 底层原理是怎样的吧：



只要线程一 加锁成功，就会启动一个 **watch dog** 看门狗，它是一个后台线程，会每隔 10 秒检查一下，如果线程 1 还持有锁，那么就会不断的延长锁 key 的

生存时间。因此，Redisson 就是使用 Redisson 解决了锁过期释放，业务没执行完问题。

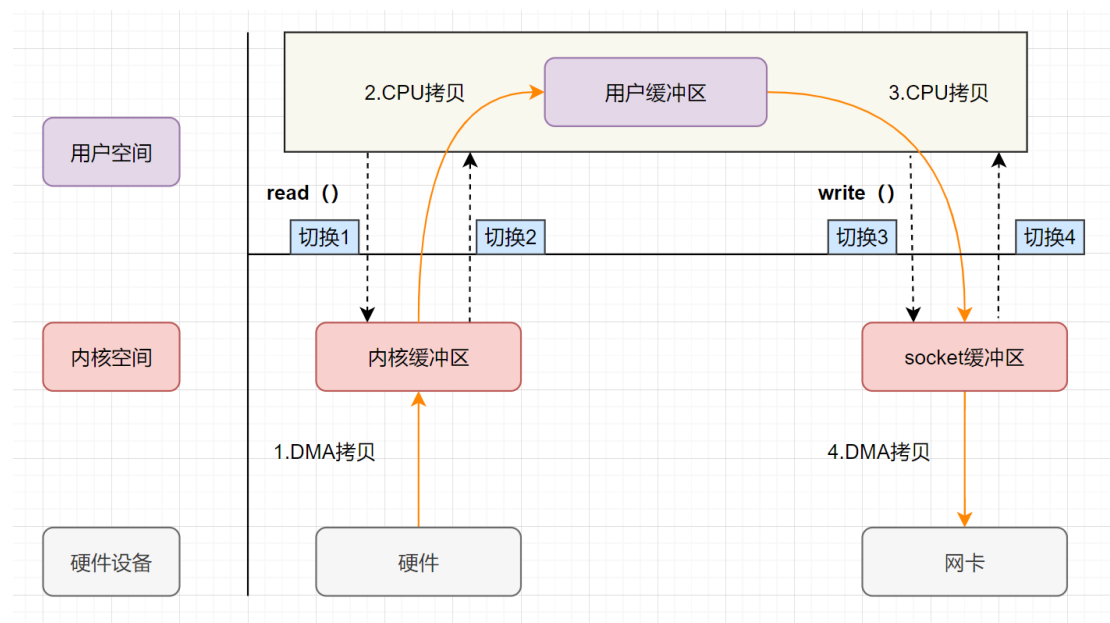
13.13. 零拷贝

零拷贝就是不需要将数据从一个存储区域复制到另一个存储区域。它是指在传统 IO 模型中，指 CPU 拷贝的次数为 0。它是 IO 的优化方案

- 传统 IO 流程
- 零拷贝实现之 mmap+write
- 零拷贝实现之 sendfile
- 零拷贝实现之带有 DMA 收集拷贝功能的 sendfile

13.13.1 传统 IO 流程

流程图如下：



- 用户应用进程调用 read 函数，向操作系统发起 IO 调用，上下文从用户态转为内核态（切换 1）
- DMA 控制器把数据从磁盘中，读取到内核缓冲区。
- CPU 把内核缓冲区数据，拷贝到用户应用缓冲区，上下文从内核态转为用户态（切换 2），read 函数返回
- 用户应用进程通过 write 函数，发起 IO 调用，上下文从用户态转为内核态（切换 3）
- CPU 将应用缓冲区中的数据，拷贝到 socket 缓冲区
- DMA 控制器把数据从 socket 缓冲区，拷贝到网卡设备，上下文从内核态切换回用户态（切换 4），write 函数返回

从流程图可以看出，传统 IO 的读写流程，包括了 4 次上下文切换（4 次用户态和内核态的切换），4 次数据拷贝（两次 CPU 拷贝以及两次的 DMA 拷贝）。

13.13.2 mmap+write 实现的零拷贝

mmap 的函数原型如下：

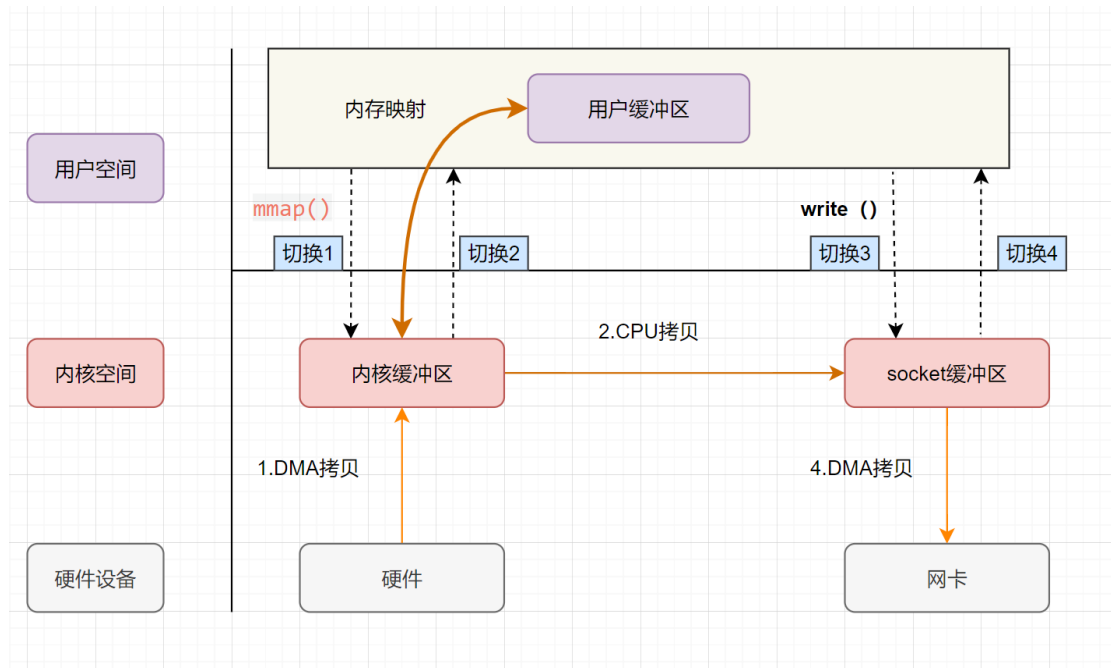
```
void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
```

- addr：指定映射的虚拟内存地址
- length：映射的长度
- prot：映射内存的保护模式
- flags：指定映射的类型
- fd:进行映射的文件句柄

- offset:文件偏移量

mmap 使用了**虚拟内存**，可以把内核空间和用户空间的虚拟地址映射到同一个物理地址，从而减少数据拷贝次数！

mmap+write 实现的零拷贝流程如下：



- 用户进程通过 **mmap** 方法向操作系统内核发起 IO 调用，上下文从用户态切换为内核态。
- CPU 利用 DMA 控制器，把数据从硬盘中拷贝到内核缓冲区。
- 上下文从内核态切换回用户态，mmap 方法返回。
- 用户进程通过 **write** 方法向操作系统内核发起 IO 调用，上下文从用户态切换为内核态。
- CPU 将内核缓冲区的数据拷贝到的 socket 缓冲区。
- CPU 利用 DMA 控制器，把数据从 socket 缓冲区拷贝到网卡，上下文从内核态切换回用户态，write 调用返回。

可以发现，`mmap+write` 实现的零拷贝，I/O 发生了 **4** 次用户空间与内核空间的上下文切换，以及 3 次数据拷贝。其中 3 次数据拷贝中，包括了 **2 次 DMA 拷贝** 和 **1 次 CPU 拷贝**。

`mmap` 是将读缓冲区的地址和用户缓冲区的地址进行映射，内核缓冲区和应用缓冲区共享，所以节省了一次 CPU 拷贝“并且用户进程内存是**虚拟的**，只是映射到内核的读缓冲区，可以节省一半的内存空间。

13.13.3 sendfile 实现的零拷贝

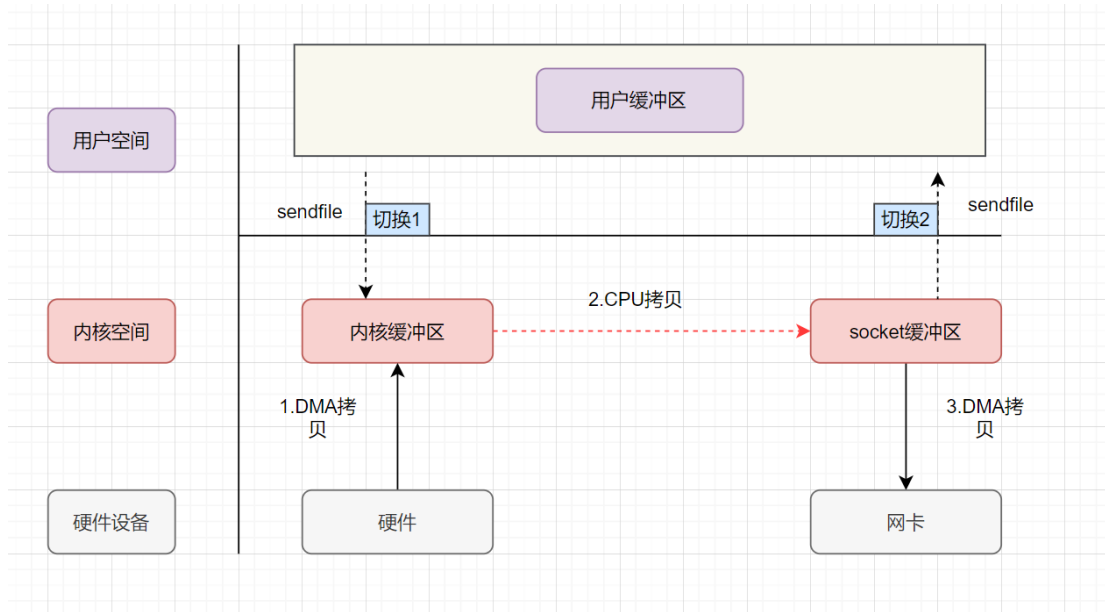
`sendfile` 是 Linux2.1 内核版本后引入的一个系统调用函数，API 如下：

```
ssize_t sendfile(int out_fd, int in_fd, off_t *offset, size_t count);
```

- `out_fd`: 为待写入内容的文件描述符，一个 socket 描述符。
- `in_fd`: 为待读出内容的文件描述符，必须是真实的文件，不能是 socket 和管道。
- `offset`：指定从读入文件的哪个位置开始读，如果为 NULL，表示文件的默认起始位置。
- `count`：指定在 `fdout` 和 `fdin` 之间传输的字节数。

`sendfile` 表示在两个文件描述符之间传输数据，它是在**操作系统内核**中操作的，**避免了数据从内核缓冲区和用户缓冲区之间的拷贝操作**，因此可以使用它来实现零拷贝。

`sendfile` 实现的零拷贝流程如下：



sendfile 实现的零拷贝

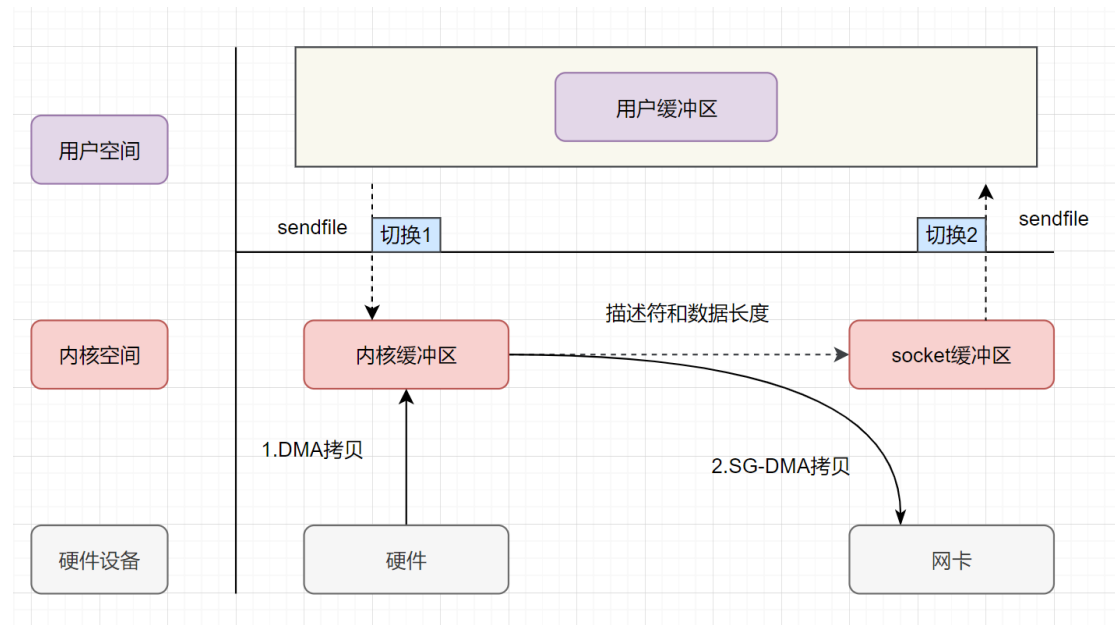
1. 用户进程发起 `sendfile` 系统调用，上下文（切换 1）从用户态转向内核态
2. DMA 控制器，把数据从硬盘中拷贝到内核缓冲区。
3. CPU 将读缓冲区中数据拷贝到 socket 缓冲区
4. DMA 控制器，异步把数据从 socket 缓冲区拷贝到网卡，
5. 上下文（切换 2）从内核态切换回用户态，`sendfile` 调用返回。

可以发现，`sendfile` 实现的零拷贝，I/O 发生了 **2** 次用户空间与内核空间的上下文切换，以及 3 次数据拷贝。其中 3 次数据拷贝中，包括了 **2 次 DMA 拷贝** 和 **1 次 CPU 拷贝**。那能不能把 CPU 拷贝的次数减少到 0 次呢？有的，即带有 DMA 收集拷贝功能的 `sendfile`！

13.13.4 sendfile+DMA scatter/gather 实现的零拷贝

linux 2.4 版本之后，对 `sendfile` 做了优化升级，引入 SG-DMA 技术，其实就是对 DMA 拷贝加入了 `scatter/gather` 操作，它可以直接从内核空间缓冲区中将数据读取到网卡。使用这个特点搞零拷贝，即还可以多省去一次 **CPU 拷贝**。

`sendfile`+DMA scatter/gather 实现的零拷贝流程如下：



1. 用户进程发起 `sendfile` 系统调用，上下文（切换 1）从用户态转向内核态
2. DMA 控制器，把数据从硬盘中拷贝到内核缓冲区。
3. CPU 把内核缓冲区中的文件描述符信息（包括内核缓冲区的内存地址和偏移量）发送到 socket 缓冲区
4. DMA 控制器根据文件描述符信息，直接把数据从内核缓冲区拷贝到网卡
5. 上下文（切换 2）从内核态切换回用户态，`sendfile` 调用返回。

可以发现，`sendfile`+DMA scatter/gather 实现的零拷贝，I/O 发生了 **2** 次用户空间与内核空间的上下文切换，以及 2 次数据拷贝。其中 2 次数据拷贝都是包

DMA 拷贝。这就是真正的 **零拷贝 (Zero-copy)** 技术，全程都没有通过 CPU 来搬运数据，所有的数据都是通过 DMA 来进行传输的。

13.14 synchronized

synchronized 是 Java 中的关键字，是一种同步锁。synchronized 关键字可以作用于方法或者代码块。

一般面试时。可以这么回答：

- 反编译后，monitorenter、monitorexit、ACC_SYNCHRONIZED
- monitor 监视器
- Java Monitor 的工作机理
- 对象与 monitor 关联

13.14.1

monitorenter、monitorexit、ACC_SYNCHRONIZED

如果 **synchronized** 作用于**代码块**，反编译可以看到两个指令：

monitorenter、monitorexit，JVM 使用 monitorenter 和 monitorexit 两个指令

实现同步；如果作用 synchronized 作用于**方法**,反编译可以看到

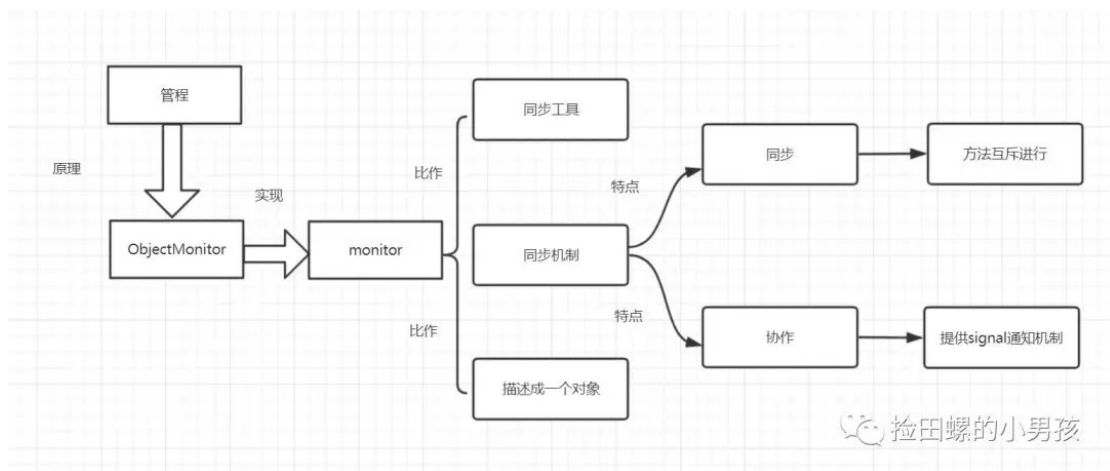
ACCSYNCHRONIZED 标记，JVM 通过在方法访问标识符(flags)中加入

ACCSYNCHRONIZED 来实现同步功能。

- 同步代码块是通过 `monitorenter` 和 `monitorexit` 来实现，当线程执行到 `monitorenter` 的时候要先获得 monitor 锁，才能执行后面的方法。当线程执行到 `monitorexit` 的时候则要释放锁。
- 同步方法是通过中设置 `ACCSYNCHRONIZED` 标志来实现，当线程执行有 `ACCSYNCHRONI` 标志的方法，需要获得 monitor 锁。每个对象都与一个 monitor 相关联，线程可以占有或者释放 monitor。

13.14.2 monitor 监视器

monitor 是什么呢？操作系统的管程 (monitors) 是概念原理，ObjectMonitor 是它的原理实现。



在 Java 虚拟机 (HotSpot) 中，Monitor (管程) 是由 ObjectMonitor 实现的，其主要数据结构如下：

```

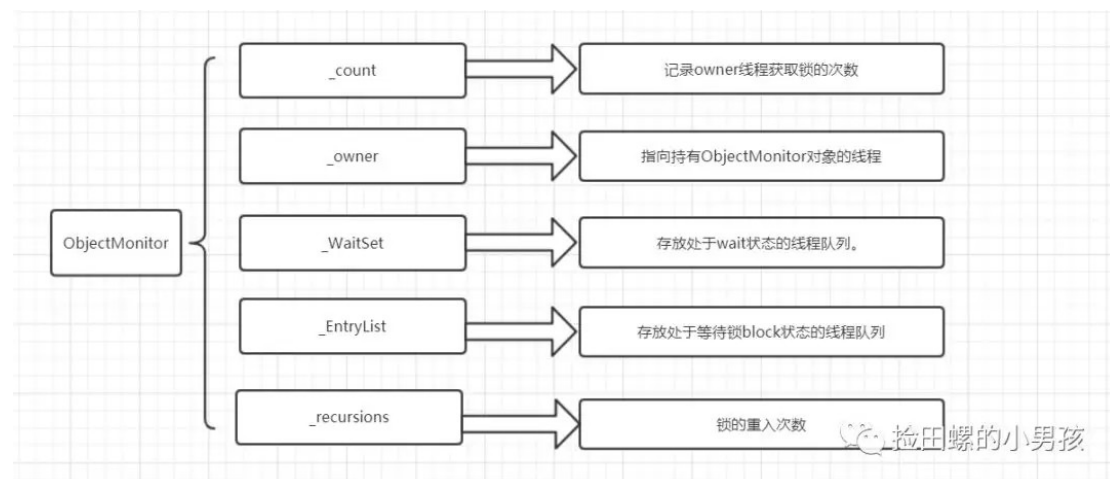
ObjectMonitor() {
    _header = NULL;
    _count = 0; // 记录个数
    _waiters = 0;
    _recursions = 0;
    _object = NULL;
    _owner = NULL;
}
  
```

```

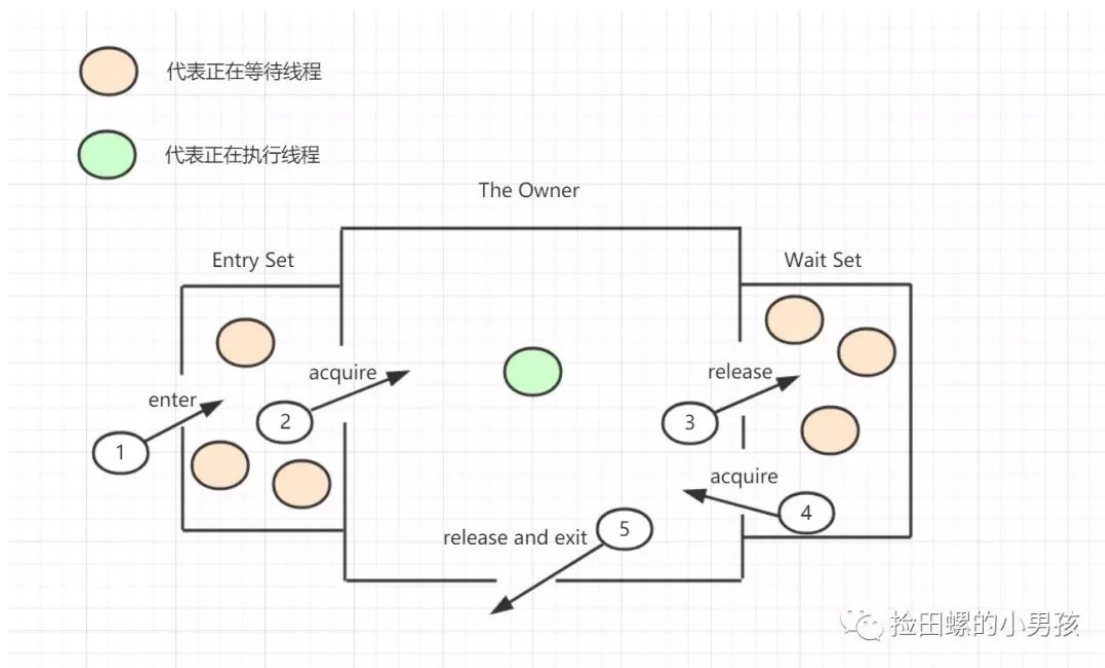
_WaitSet      = NULL; // 处于wait状态的线程，会被加入到_WaitSet
_WaitSetLock  = 0;
_Responsible  = NULL;
_succ         = NULL;
_cxq          = NULL;
FreeNext      = NULL;
_EntryList    = NULL; // 处于等待锁block状态的线程，会被加入到该列表
_SpinFreq     = 0;
_SpinClock    = 0;
OwnerIsThread = 0;
}

```

ObjectMonitor 中几个关键字段的含义如图所示：

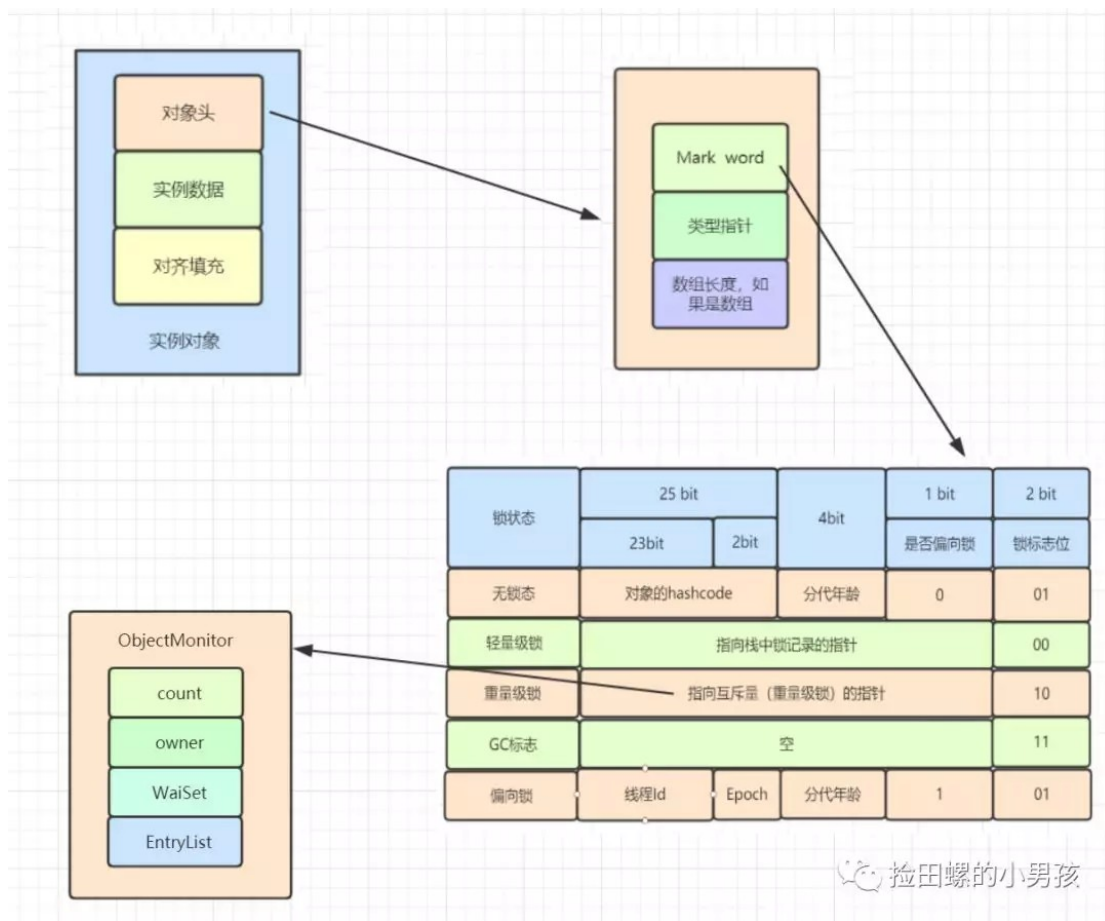


13.14.3 Java Monitor 的工作机理



- 想要获取 monitor 的线程,首先会进入 `_EntryList` 队列。
- 当某个线程获取到对象的 monitor 后,进入 Owner 区域,设置为当前线程,同时计数器 `count` 加 1。
- 如果线程调用了 `wait()` 方法,则会进入 WaitSet 队列。它会释放 monitor 锁,即将 `owner` 赋值为 `null`,`count` 自减 1,进入 WaitSet 队列阻塞等待。
- 如果其他线程调用 `notify()` / `notifyAll()`,会唤醒 WaitSet 中的某个线程,该线程再次尝试获取 monitor 锁,成功即进入 Owner 区域。
- 同步方法执行完毕了,线程退出临界区,会将 monitor 的 `owner` 设为 `null`,并释放监视锁。

13.14.4 对象与 monitor 关联



- 在 HotSpot 虚拟机中,对象在内存中存储的布局可以分为 3 块区域：**对象头 (Header)**，**实例数据 (Instance Data)** 和**对象填充 (Padding)**。
- 对象头主要包括两部分数据：**Mark Word (标记字段)**、**Class Pointer (类型指针)**。

Mark Word 是用于存储对象自身的运行时数据，如哈希码

(HashCode)、GC 分代年龄、锁状态标志、线程持有的锁、偏向线程 ID、偏向时间戳等。

锁状态	25 bit		4bit	1 bit	2 bit
	23bit	2bit		是否偏向锁	锁标志位
无锁态	对象的hashcode		分代年龄	0	01
轻量级锁	指向栈中锁记录的指针				00
重量级锁	指向互斥量（重量级锁）的指针				10
GC标志	空				11
偏向锁	线程Id	Epoch	分代年龄	1	01

重量级锁，指向互斥量的指针。其实 **synchronized** 是重量级锁，也就是说 **Synchronized** 的对象锁，**Mark Word** 锁标识位为 **10**，其中指针指向的是 **Monitor** 对象的起始地址。

13.15 分布式 id 生成方案有哪些？什么是雪花算法？

分布式 id 生成方案主要有：

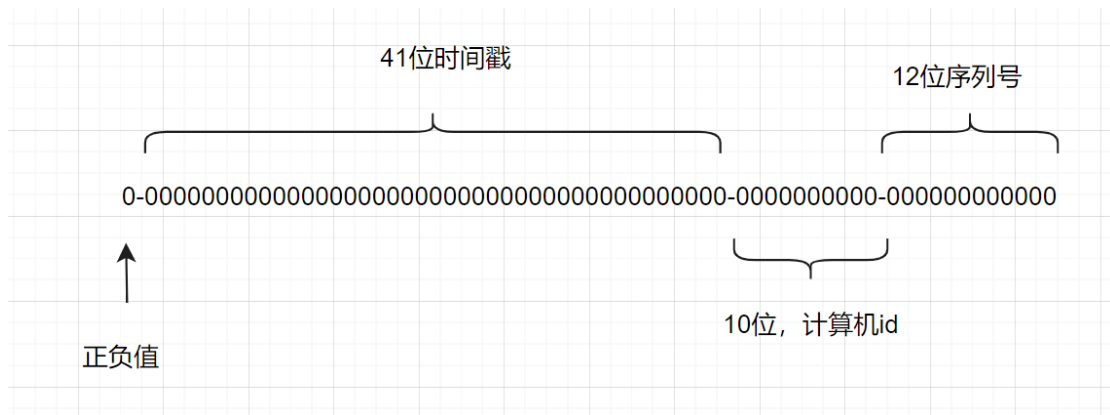
- UUID
- 数据库自增 ID
- 基于雪花算法 (Snowflake) 实现
- 百度 (Uidgenerator)
- 美团 (Leaf)

什么是雪花算法？

雪花算法是一种生成分布式全局唯一 ID 的算法，生成的 ID 称为 Snowflake IDs。这种算法由 Twitter 创建，并用于推文的 ID。

一个 Snowflake ID 有 64 位元。

- 第 1 位：Java 中 long 的最高位是符号位代表正负，正数是 0，负数是 1，一般生成 ID 都为正数，所以默认为 0。
- 接下来前 41 位是时间戳，表示了自选定的时期以来的毫秒数。
- 接下来的 10 位代表计算机 ID，防止冲突。
- 其余 12 位代表每台机器上生成 ID 的序列号，这允许在同一毫秒内创建多个 Snowflake ID。



雪花算法

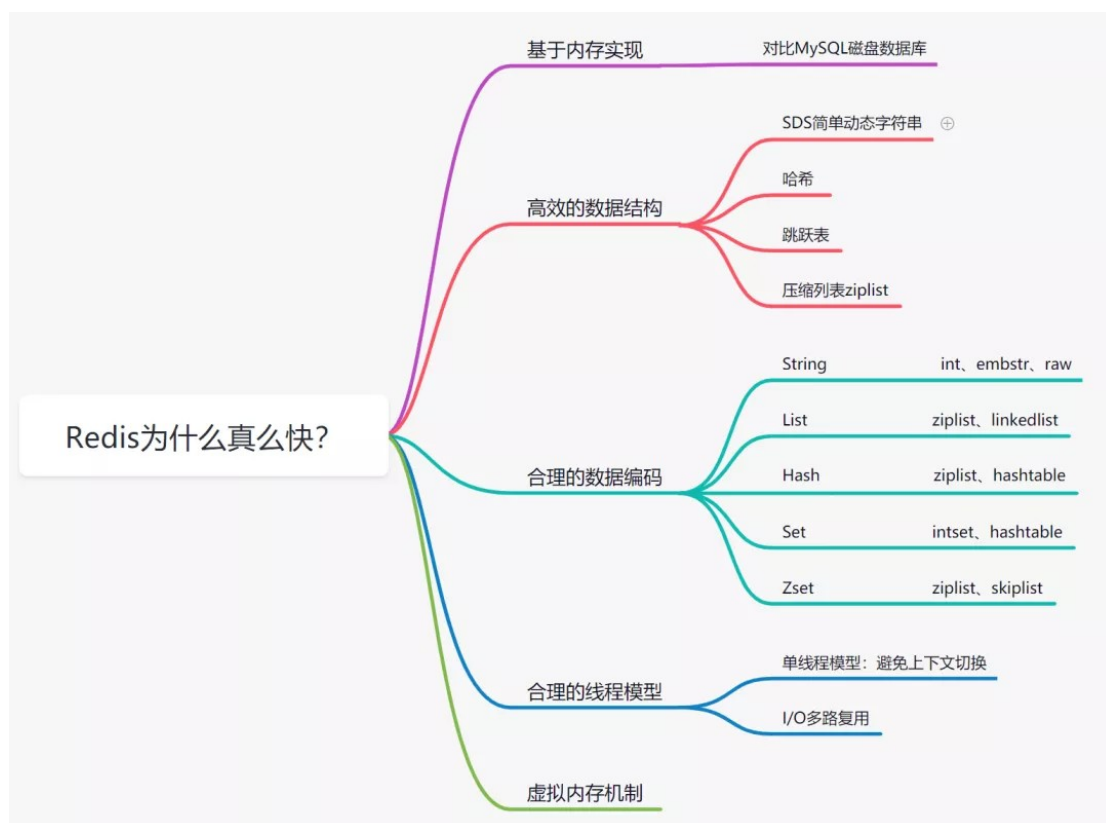
14.字节跳动面试（关注公众号：捡田螺的小男孩。）

大家好，我是捡田螺的小男孩，有位伙伴面试了字节（四年半工作经验），分享下面面试真题，大家一起加油哈。如果有不正确的，欢迎指出哈，一起进步。

更多干货，关注公众号：捡田螺的小男孩。



14.1 说说 Redis 为什么快

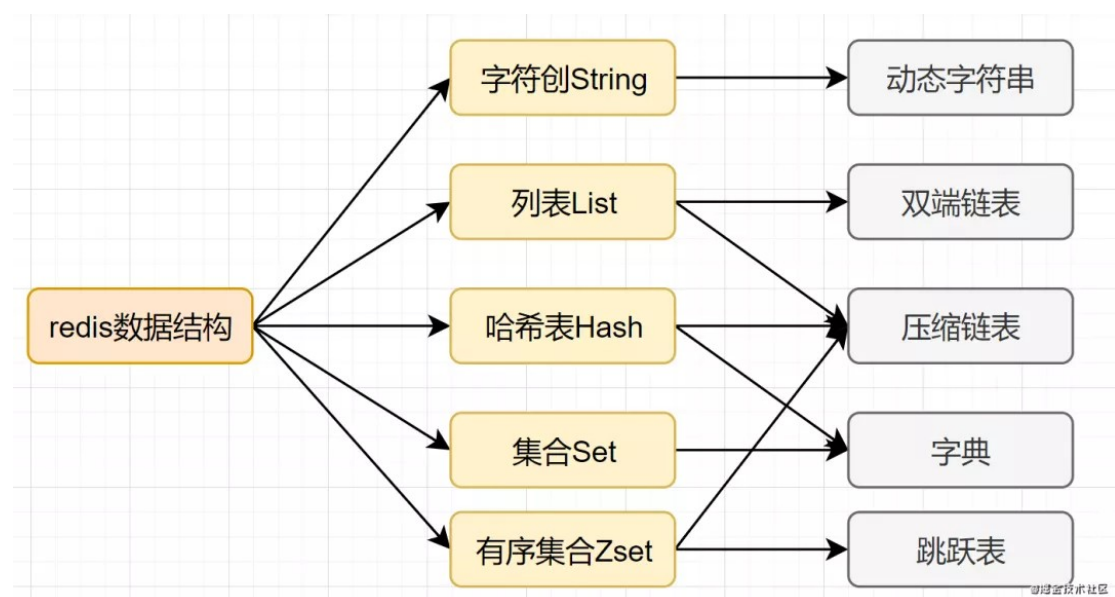


14.1.1 基于内存存储实现

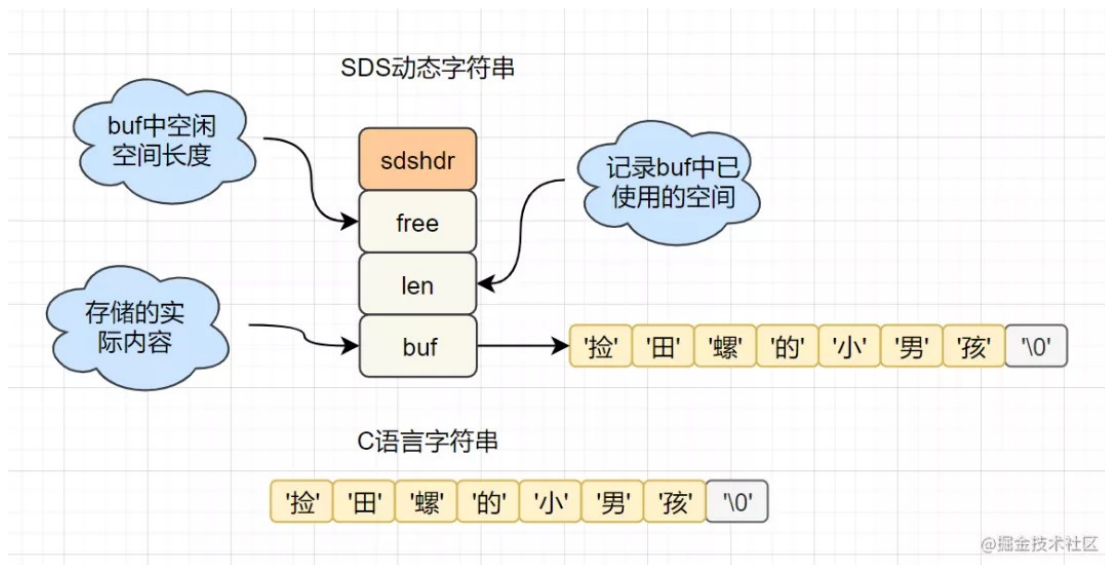
内存读写是比在磁盘快很多的，Redis 基于内存存储实现的数据库，相对于数据存在磁盘的 MySQL 数据库，省去磁盘 I/O 的消耗。

14.1.2 高效的数据结构

Mysql 索引为了提高效率，选择了 B+树的数据结构。其实合理的数据结构，就是可以让你的应用/程序更快。先看下 Redis 的数据结构&内部编码图：



14.1.2.1 SDS 简单动态字符串

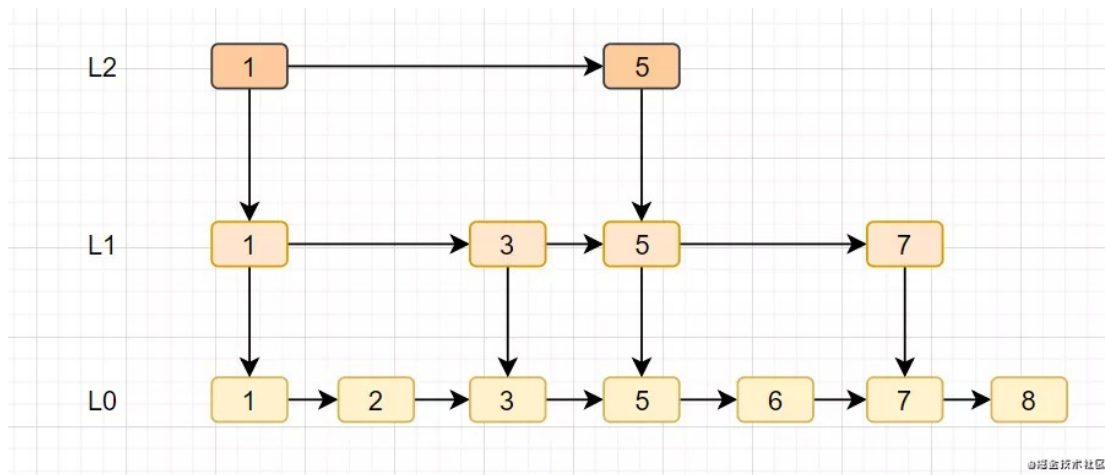


- 字符串长度处理：Redis 获取字符串长度，时间复杂度为 $O(1)$ ，而 C 语言中，需从头开始遍历，复杂度为 $O(n)$ ；
- 空间预分配：字符串修改越频繁的话，内存分配越频繁，就会消耗性能，而 SDS 修改和空间扩充，会额外分配未使用的空间，减少性能损耗。
- 惰性空间释放：SDS 缩短时，不是回收多余的内存空间，而是 free 记录下多余的空间，后续有变更，直接使用 free 中记录的空间，减少分配。

14.1.2.2 字典

Redis 作为 K-V 型内存数据库，所有的键值就是用字典来存储。字典就是哈希表，比如 HashMap，通过 key 就可以直接获取到对应的 value。而哈希表的特性，在 $O(1)$ 时间复杂度就可以获得对应的值。

14.1.2.3 跳跃表



- 跳跃表是 Redis 特有的数据结构，就是在链表的基础上，增加多级索引提升查找效率。
- 跳跃表支持平均 $O(\log N)$ ，最坏 $O(N)$ 复杂度的节点查找，还可以通过顺序性操作批量处理节点。

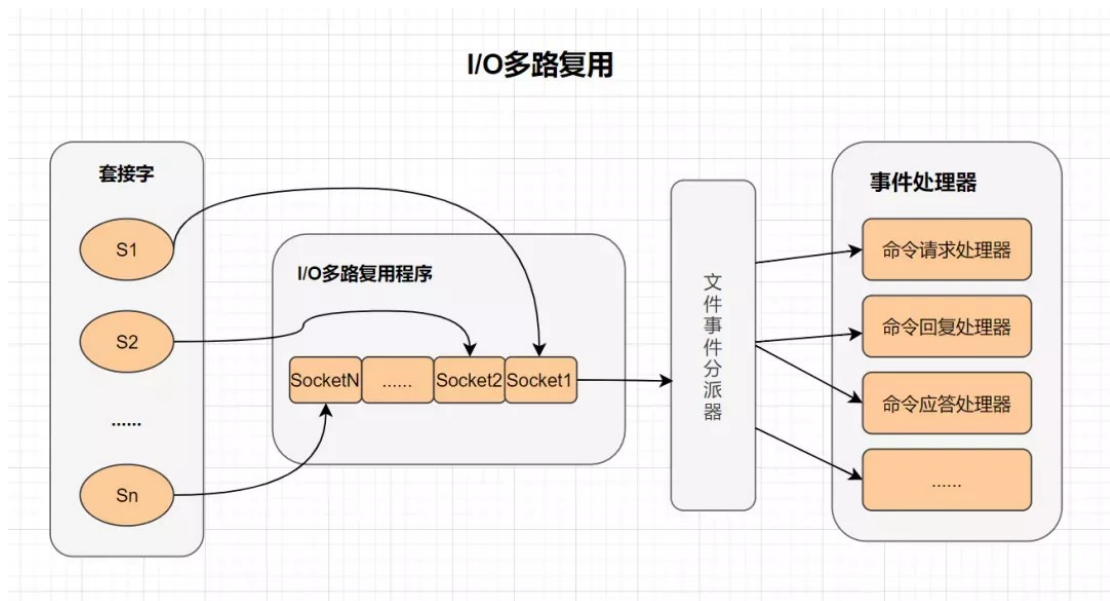
14.1.3 合理的数据编码

Redis 支持多种数据类型，每种基本类型，可能对多种数据结构。什么时候，使用什么样数据结构，使用什么样编码，是 redis 设计者总结优化的结果。

- String：如果存储数字的话，是用 int 类型的编码；如果存储非数字，小于等于 39 字节的字符串，是 embstr；大于 39 个字节，则是 raw 编码。
- List：如果列表的元素个数小于 512 个，列表每个元素的值都小于 64 字节（默认），使用 ziplist 编码，否则使用 linkedlist 编码
- Hash：哈希类型元素个数小于 512 个，所有值小于 64 字节的话，使用 ziplist 编码，否则使用 hashtable 编码。
- Set：如果集合中的元素都是整数且元素个数小于 512 个，使用 intset 编码，否则使用 hashtable 编码。
- Zset：当有序集合的元素个数小于 128 个，每个元素的值小于 64 字节时，使用 ziplist 编码，否则使用 skiplist（跳跃表）编码

14.1.4 合理的线程模型

I/O 多路复用



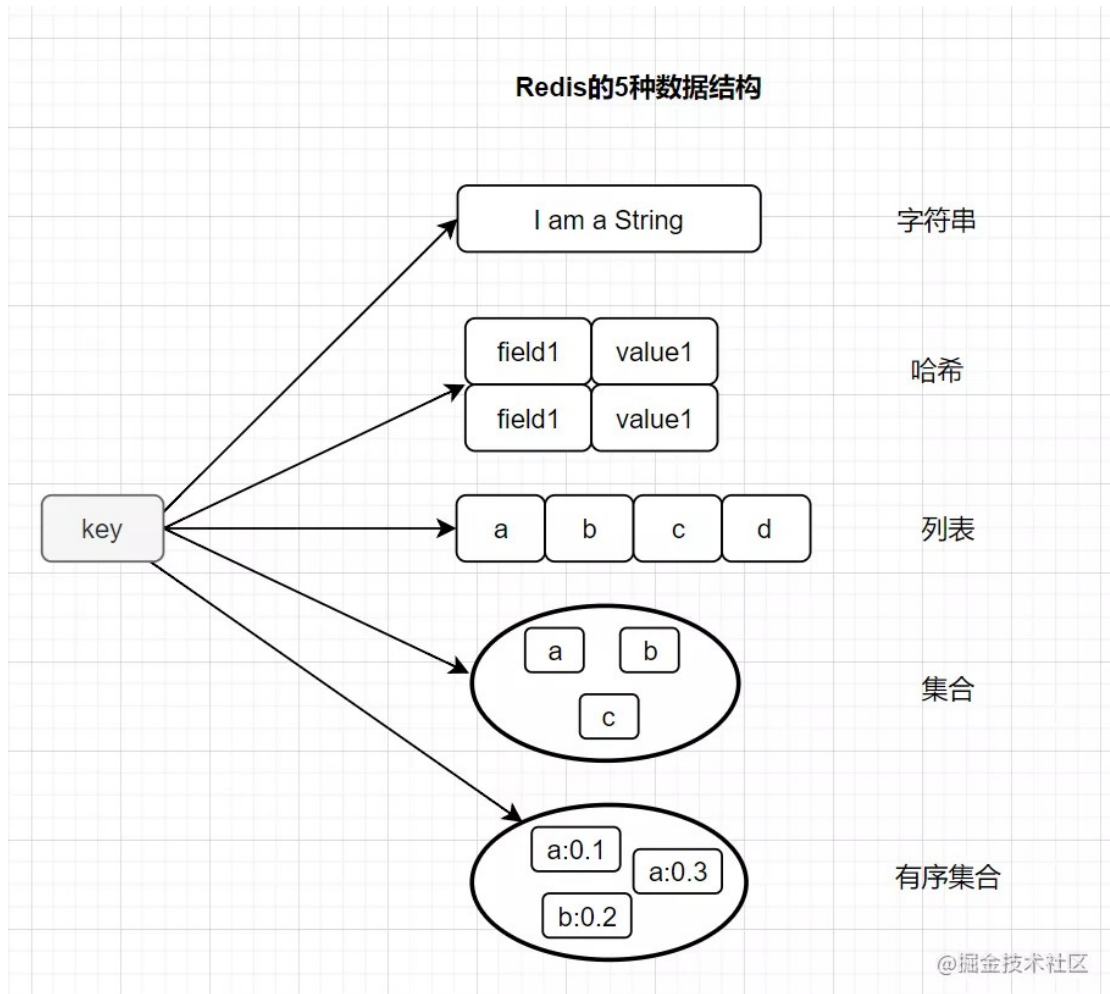
多路 I/O 复用技术可以让单个线程高效的处理多个连接请求，而 Redis 使用用 `epoll` 作为 I/O 多路复用技术的实现。并且，Redis 自身的事件处理模型将 `epoll` 中的连接、读写、关闭都转换为事件，不在网络 I/O 上浪费过多的时间。

14.2 Redis 有几种数据结构，底层分别是怎么存储的

常用的，Redis 有以下这五种基本类型：

- String (字符串)
- Hash (哈希)
- List (列表)
- Set (集合)
- zset (有序集合) 它还有三种特殊的数据结构类型
- Geospatial
- Hyperloglog
- Bitmap

14.2.1 Redis 的五种基本数据类型



String (字符串)

- 简介:String 是 Redis 最基础的数据结构类型，它是二进制安全的，可以存储图片或者序列化的对象，值最大存储为 512M
- 简单使用举例: set key value、get key 等
- 应用场景：共享 session、分布式锁，计数器、限流。
- 内部编码有 3 种，int (8 字节长整型) / embstr (小于等于 39 字节字符串) / raw (大于 39 个字节字符串)

Hash (哈希)

- 简介：在 Redis 中，哈希类型是指 v (值) 本身又是一个键值对 (k-v) 结构
- 简单使用举例：hset key field value 、 hget key field
- 内部编码：ziplist (压缩列表) 、 hashtable (哈希表)
- 应用场景：缓存用户信息等。

List (列表)

- 简介：列表 (list) 类型是用来存储多个有序的字符串，一个列表最多可以存储 $2^{32}-1$ 个元素。
- 简单实用举例：`lpush key value [value ...]`、`lrange key start end`
- 内部编码：ziplist (压缩列表)、linkedlist (链表)
- 应用场景：消息队列，文章列表

Set (集合)

- 简介：集合 (set) 类型也是用来保存多个的字符串元素，但是不允许重复元素
- 简单使用举例：`sadd key element [element ...]`、`smembers key`
- 内部编码：intset (整数集合)、hashtable (哈希表)
- 应用场景：用户标签,生成随机数抽奖、社交需求。

有序集合 (zset)

- 简介：已排序的字符串集合，同时元素不能重复
- 简单格式举例：`zadd key score member [score member ...]`，`zrank key member`
- 底层内部编码：ziplist (压缩列表)、skiplist (跳跃表)
- 应用场景：排行榜，社交需求 (如用户点赞)。

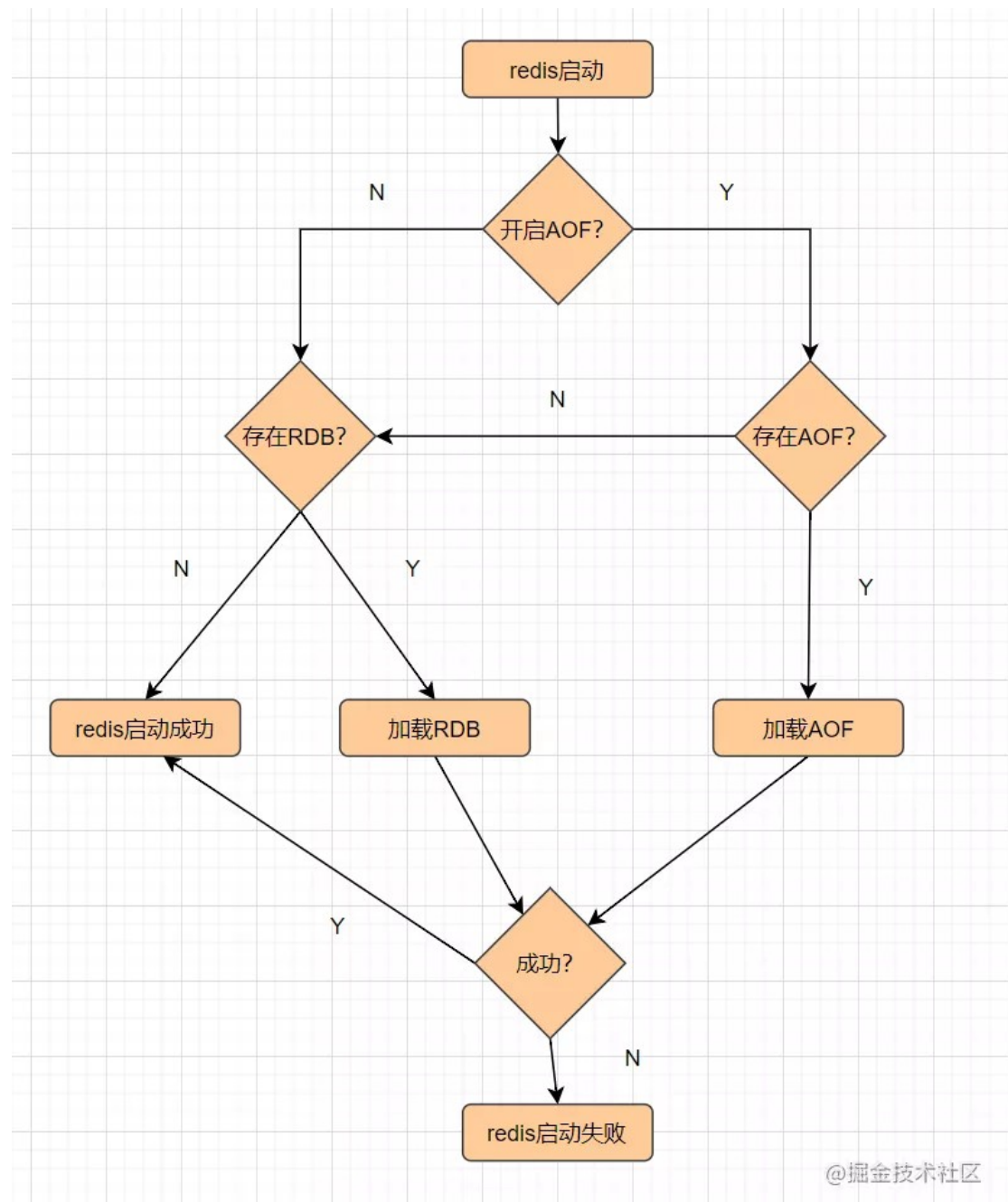
14.2.2 Redis 的三种特殊数据类型

- Geo：Redis3.2 推出的，地理位置定位，用于存储地理位置信息，并对存储的信息进行操作。
- HyperLogLog：用来做基数统计算法的数据结构，如统计网站的 UV。
- Bitmaps：用一个比特位来映射某个元素的状态，在 Redis 中，它的底层是基于字符串类型实现的，可以把 bitmaps 当作一个以比特位为单位的数组

14.3 Redis 有几种持久化方式

Redis 是基于内存的非关系型 K-V 数据库，既然它是基于内存的，如果 Redis 服务器挂了，数据就会丢失。为了避免数据丢失了，Redis 提供了持久化，即把数据保存到磁盘。

Redis 提供了 RDB 和 AOF 两种持久化机制，它持久化文件加载流程如下：



14.3.1 RDB

RDB，就是把内存数据以快照的形式保存到磁盘上。

什么是快照?可以这样理解，给当前时刻的数据，拍一张照片，然后保存下来。

RDB 持久化，是指在指定的时间间隔内，执行指定次数的写操作，将内存中的数据快照写入磁盘中，它是 Redis 默认的持久化方式。执行完操作后，在指定目录下会生成一个 dump.rdb 文件，Redis 重启的时候，通过加载 dump.rdb 文件来恢复数据。RDB 触发机制主要有以下几种：



RDB 的优点

适合大规模的数据恢复场景，如备份，全量复制等

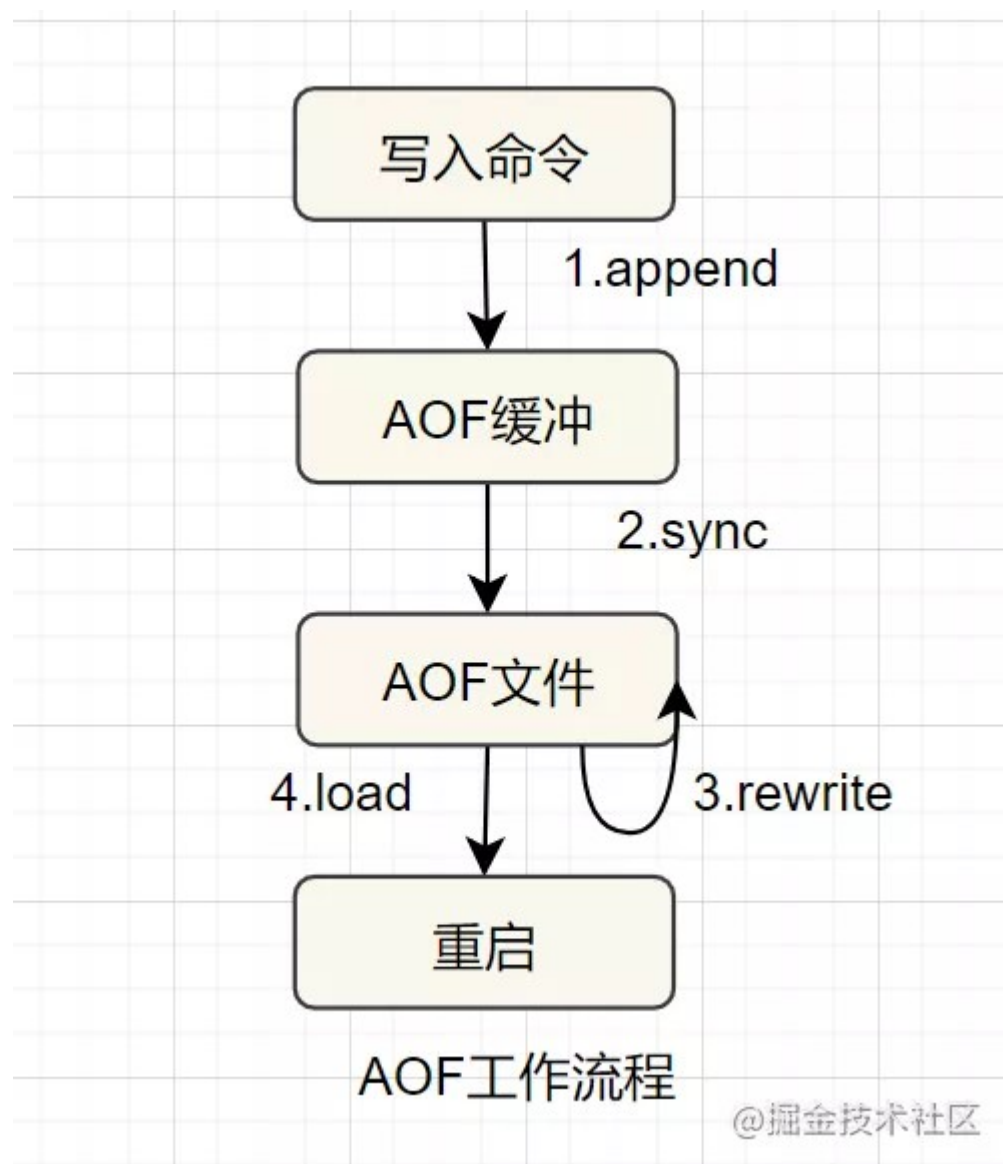
RDB 缺点

- 没办法做到实时持久化/秒级持久化。
- 新老版本存在 RDB 格式兼容问题

14.3.2 AOF

AOF (append only file) 持久化，采用日志的形式来记录每个写操作，追加到文件中，重启时再重新执行 AOF 文件中的命令来恢复数据。它主要解决数据持久化的实时性问题。默认是不开启的。

AOF 的工作流程如下：



AOF 的优点

数据的一致性和完整性更高

AOF 的缺点

AOF 记录的内容越多，文件越大，数据恢复变慢。

14.4 多线程情况下,如何保证线程安全？

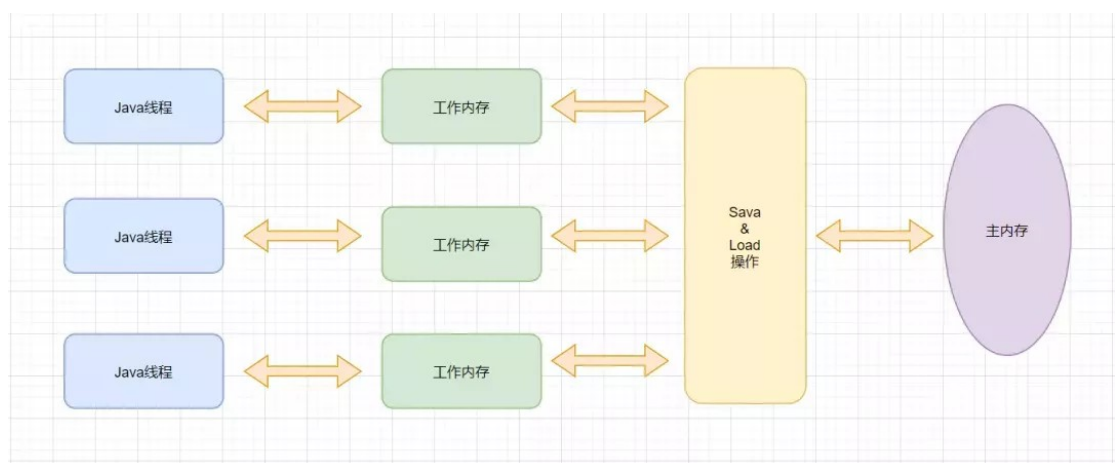
加锁，比如悲观锁 `select for update` , `synchronized` 等，如，乐观锁，乐观锁如 `CAS` 等，还有 `redis` 分布式锁等等。

14.5 用过 `volatile` 吗？它是如何保证可见性的，原理是什么

`volatile` 关键字是 Java 虚拟机提供的的最轻量级的同步机制，它作为一个修饰符，用来修饰变量。它保证变量对所有线程可见性，禁止指令重排，但是不保证原子性。

我们先来看下 java 内存模型 (jmm)：

- Java 虚拟机规范试图定义一种 Java 内存模型,来屏蔽掉各种硬件和操作系统的内存访问差异，以实现让 Java 程序在各种平台上都能达到一致的内存访问效果。
- Java 内存模型规定所有的变量都是存在主内存当中，每个线程都有自己的工作内存。这里的变量包括实例变量和静态变量，但是不包括局部变量，因为局部变量是线程私有的。
- 线程的工作内存保存了被该线程使用的变量的主内存副本，线程对变量的所有操作都必须在工作内存中进行，而不能直接操作主内存。并且每个线程不能访问其他线程的工作内存。



`volatile` 变量，保证新值能立即同步回主内存，以及每次使用前立即从主内存刷新，所以我们说 `volatile` 保证了多线程操作变量的可见性。

volatile 保证可见性跟内存屏障有关。我们来看一段 volatile 使用的 demo 代码：

```
public class Singleton {  
    private volatile static Singleton instance;  
    private Singleton (){}  
    public static Singleton getInstance() {  
        if (instance == null) {  
            synchronized (Singleton.class) {  
                if (instance == null) {  
                    instance = new Singleton();  
                }  
            }  
        }  
        return instance;  
    }  
}
```

编译后，对比有 volatile 关键字和没有 volatile 关键字时所生成的汇编代码，发现有 volatile 关键字修饰时，会多出一个 lock addl \$0x0, (%esp)，即多出一个 lock 前缀指令，lock 指令相当于一个内存屏障

lock 指令相当于一个内存屏障，它保证以下这几点：

1. 重排序时不能把后面的指令重排序到内存屏障之前的位置
2. 将本处理器的缓存写入内存
3. 如果是写入动作，会导致其他处理器中对应的缓存无效。

第 2 点和第 3 点就是保证 volatile 保证可见性的体现嘛

14.6 MySQL 的索引结构，聚簇索引和非聚簇索引的区别

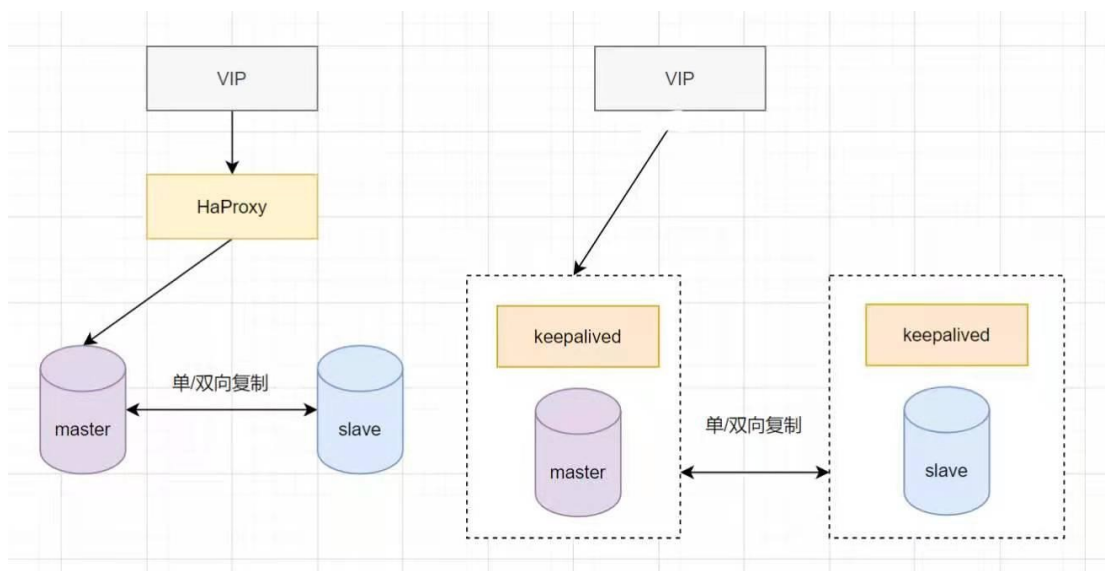
- 一个表中只能拥有一个聚集索引，而非聚集索引一个表可以存在多个。
- 索引是通过二叉树的数据结构来描述的，我们可以这么理解聚簇索引：索引的叶节点就是数据节点。而非聚簇索引的叶节点仍然是索引节点，只不过有一个指针指向对应的数据块。
- 聚集索引：物理存储按照索引排序；非聚集索引：物理存储不按照索引排序

14.7 MySQL 有几种高可用方案，你们用的是哪一种

- 主从或主主半同步复制
- 半同步复制优化
- 高可用架构优化
- 共享存储
- 分布式协议

14.7.1 主从或主主半同步复制

用双节点数据库，搭建单向或者双向的半同步复制。架构如下：

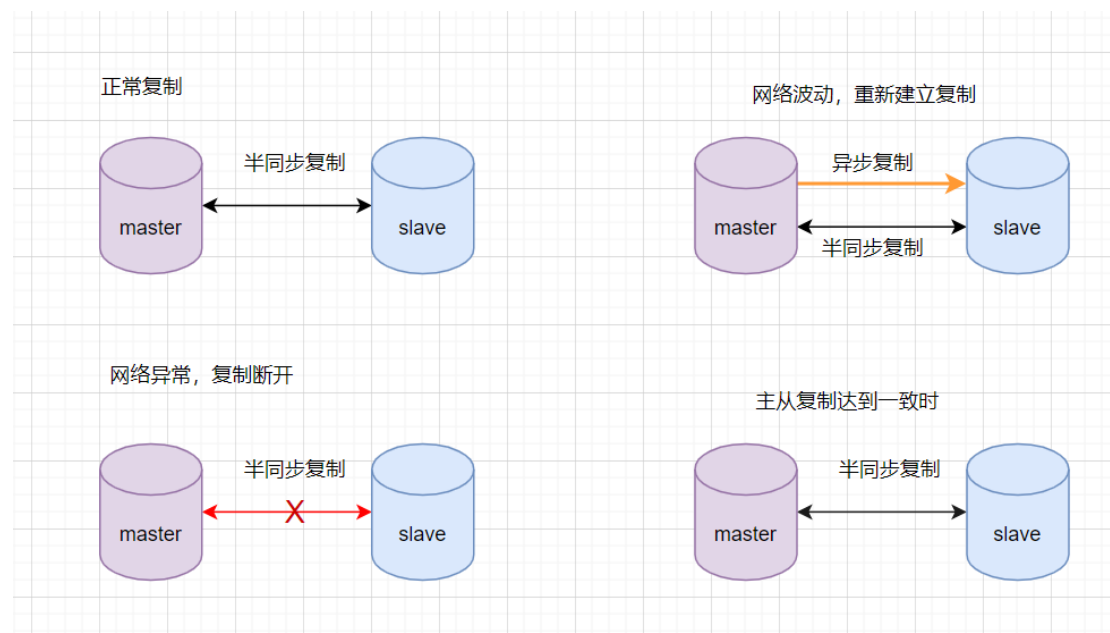


通常会和 proxy、keepalived 等第三方软件同时使用，即可以用来监控数据库的健康，又可以执行一系列管理命令。如果主库发生故障，切换到备库后仍然可以继续使用数据库。

这种方案优点是架构、部署比较简单，主机宕机直接切换即可。缺点是完全依赖于半同步复制，半同步复制退化为异步复制，无法保证数据一致性；另外，还需要额外考虑 **haproxy**、**keepalived** 的高可用机制。

14.7.2 半同步复制优化

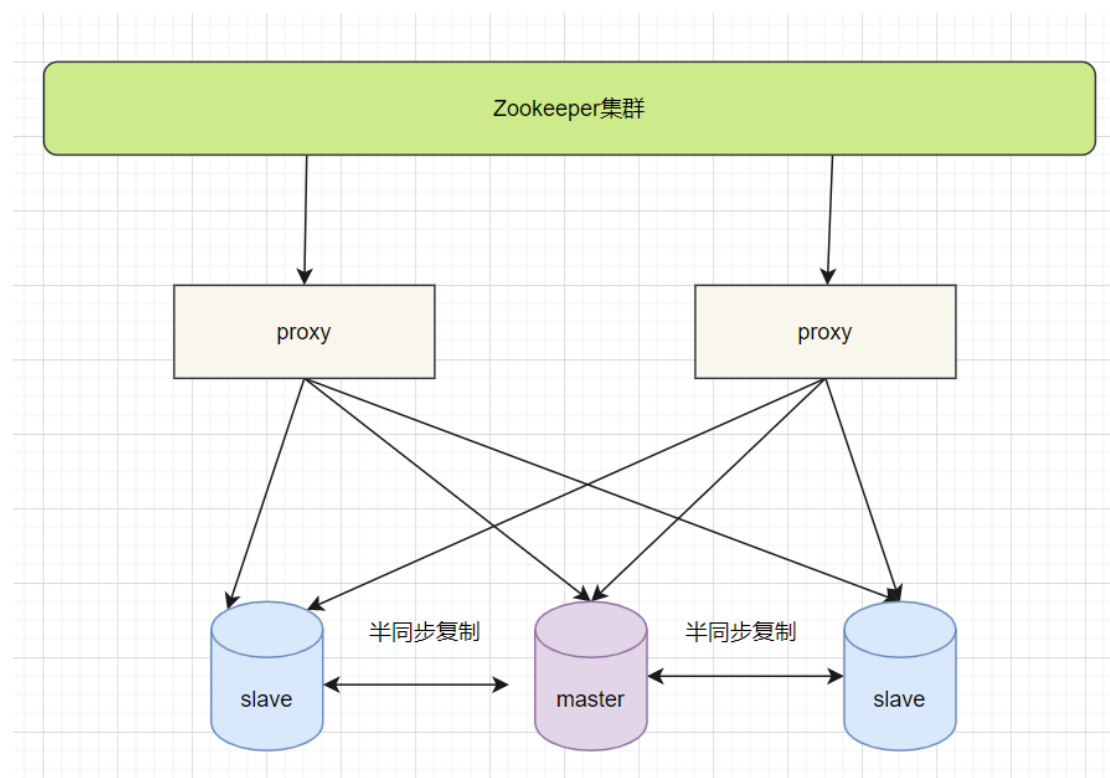
半同步复制机制是可靠的，可以保证数据一致性的。但是如果网络发生波动，半同步复制发生超时会切换为异步复制，异步复制是无法保证数据的一致性的。因此，可以在半同复制的基础上优化一下，尽可能保证半同复制。如**双通道复制方案**



- 优点：这种方案架构、部署也比较简单，主机宕机也是直接切换即可。比方案 1 的半同步复制，更能保证数据的一致性。
- 缺点：需要修改内核源码或者使用 mysql 通信协议，没有从根本上解决数据一致性问题。

14.7.3 高可用架构优化

保证高可用，可以把主从双节点数据库扩展为数据库集群。Zookeeper 可以作为集群管理，它使用分布式算法保证集群数据的一致性，可以较好的避免网络分区现象的产生。



- 优点：保证了整个系统的高可用性，扩展性也较好，可以扩展为大规模集群。
- 缺点：数据一致性仍然依赖于原生的 **mysql 半同步复制**；引入 Zookeeper 使系统逻辑更复杂。

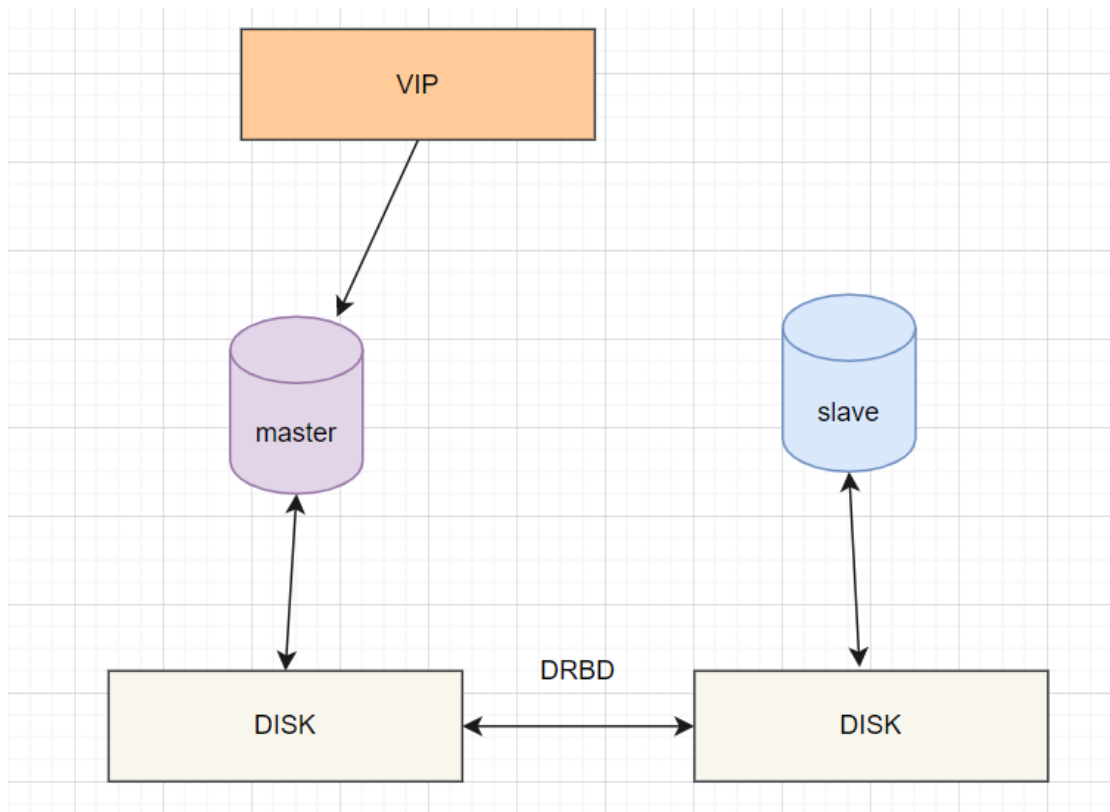
14.7.4 共享存储

共享存储实现了数据库服务器和存储设备的解耦，不同数据库之间的数据同步不再依赖于 MySQL 的原生复制功能，而是通过磁盘数据同步的手段，来保证数据的一致性。

DRBD 磁盘复制

DRBD 是一个用软件实现的、无共享的、服务器之间镜像块设备内容的存储复制解决方案。主要用于对服务器之间的磁盘、分区、逻辑卷等进行数据镜像，当用户将数据写入本地磁盘时，还会将数据发送到网络中另一台主机的磁盘上，

这样的本地主机(主节点)与远程主机(备节点)的数据就可以保证实时同步。常用架构如下：

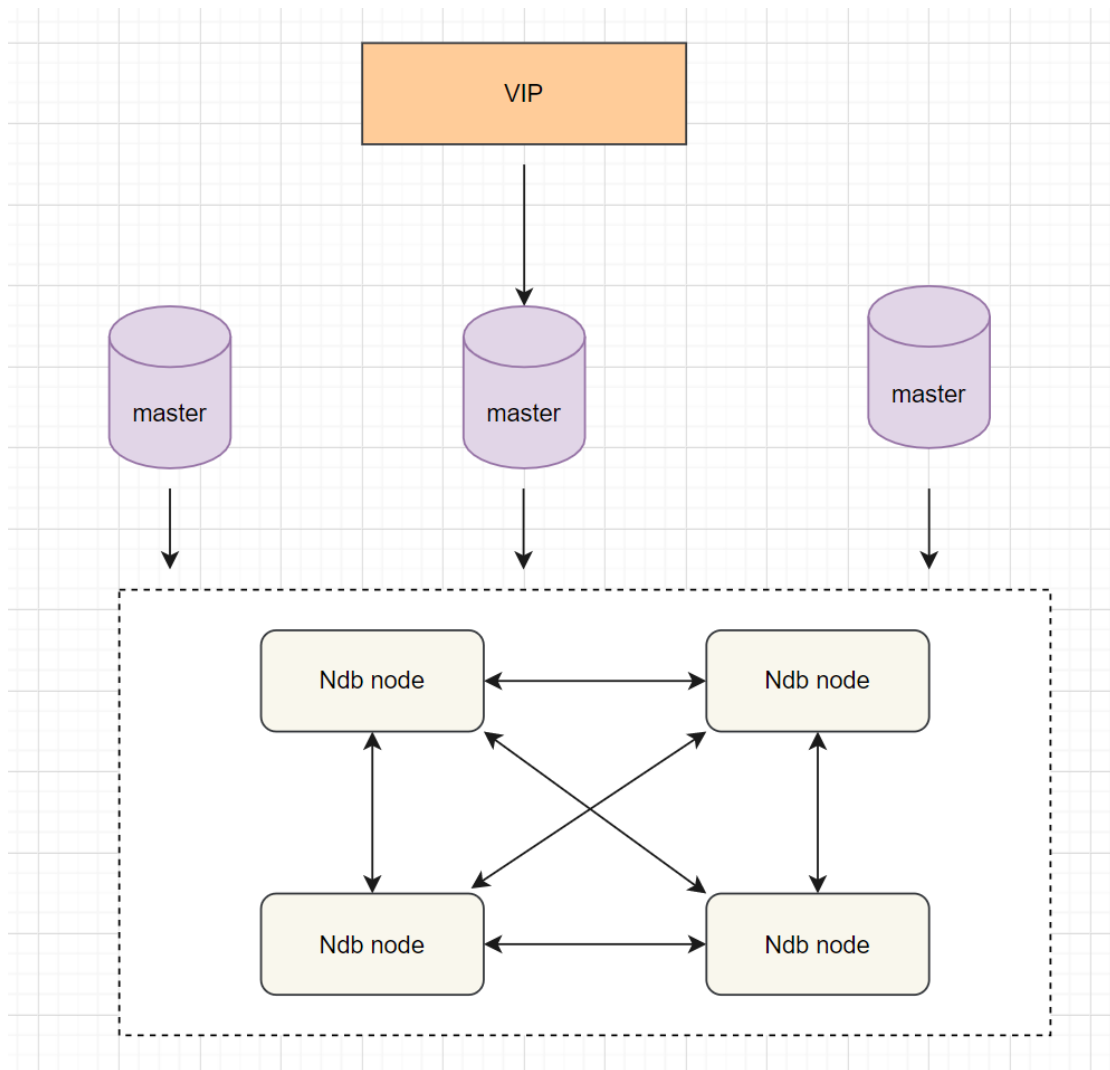


当本地主机出现问题，远程主机上还保留着一份相同的数据，即可以继续使用，保证了数据的安全。

- 优点：部署简单，价格合适，保证数据的强一致性
- 缺点：对 IO 性能影响较大，从库不提供读操作

14.7.5 分布式协议

分布式协议可以很好解决数据一致性问题。常见的部署方案就是 **MySQL cluster**，它是官方集群的部署方案，通过使用 NDB 存储引擎实时备份冗余数据，实现数据库的高可用性和数据一致性。如下：



- 优点：不依赖于第三方软件，可以实现数据的强一致性；
- 缺点：配置较复杂；需要使用 NDB 储存引擎；至少三节点；

14.8 说说你做过最有挑战性的项目，你负责那个模块,哪些最有挑战性，说说你做了哪些优化

项目这块的话，大家可以结合自己实际做的项目说哈。也可以加我微信，跟我一起交流哈，加油加油。

14.9 秒杀采用什么方案。

设计一个秒杀系统，需要考虑这些问题：



如何解决这些问题呢？

- 页面静态化
- 按钮至灰控制
- 服务单一职责
- 秒杀链接加盐
- 限流
- 分布式锁
- MQ 异步处理
- 限流&降级&熔断

14.9.1 页面静态化

秒杀活动的页面，大多数内容都是固定不变的，如商品名称，商品图片等等，可以对活动页面做静态化处理，减少访问服务端的请求。秒杀用户会分布在全国各地，有的在上海，有的在深圳，地域相差很远，网速也各不相同。为了让用户最快访问到活动页面，可以使用 CDN (Content Delivery Network，内容分发网络)。CDN 可以让用户就近获取所需内容。

14.9.2 按钮至灰控制

秒杀活动开始前，按钮一般需要置灰的。只有时间到了，才能变得可以点击。

这是防止，秒杀用户在时间快到的前几秒，疯狂请求服务器，然后秒杀时间点还没到，服务器就自己挂了。

14.9.3 服务单一职责

我们都知道微服务设计思想，也就是把各个功能模块拆分，功能那个类似的放在一起，再用分布式的部署方式。

如用户登录相关的，就设计个用户服务，订单相关的就搞个订单服务，再到礼物相关的就搞个礼物服务等等。那么，秒杀相关的业务逻辑也可以放到一起，搞个秒杀服务，单独给它搞个秒杀数据库。”服务单一职责有个好处：如果秒杀没抗住高并发的压力，秒杀库崩了，服务挂了，也不会影响到系统的其他服务。

14.9.4 秒杀链接加盐

链接如果明文暴露的话，会有人获取到请求 Url，提前秒杀了。因此，需要给秒杀链接加盐。可以把 URL 动态化，如通过 MD5 加密算法加密随机的字符串去做 url。

14.9.5 限流

一般有两种方式限流：nginx 限流和 redis 限流。

- 为了防止某个用户请求过于频繁，我们可以对同一用户限流；
- 为了防止黄牛模拟几个用户请求，我们可以对某个 IP 进行限流；
- 为了防止有人使用代理，每次请求都更换 IP 请求，我们可以对接口进行限流。
- 为了防止瞬时过大的流量压垮系统，还可以使用阿里的 Sentinel、Hystrix 组件进行限流。

14.9.6 分布式锁

可以使用 redis 分布式锁解决超卖问题。

使用 Redis 的 SET EX PX NX + 校验唯一随机值,再删除释放锁。

```
if (jedis.set(key_resource_id, uni_request_id, "NX", "EX", 100s) == 1) { //加锁
    try {
        do something //业务处理
    } catch ({
    }
    finally {
        //判断是不是当前线程加的锁,是才释放
        if (uni_request_id.equals(jedis.get(key_resource_id))) {
            jedis.del(lockKey); //释放锁
        }
    }
}
```

在这里，判断是不是当前线程加的锁和释放锁不是一个原子操作。如果调用 jedis.del() 释放锁的时候，可能这把锁已经不属于当前客户端，会解除他人加的锁。

```
if (jedis.set(key_resource_id, uni_request_id, "NX", "EX", 100s) == 1) { //加锁
    try {
        do something //业务处理
    } catch ({
    }
    finally {
        //判断是不是当前线程加的锁,是才释放
        if (uni_request_id.equals(jedis.get(key_resource_id))) {
            jedis.del(lockKey);
        }
    }
}
```

非原子

为了更严谨，一般也是用 lua 脚本代替。lua 脚本如下：

```
if redis.call('get',KEYS[1]) == ARGV[1] then
    return redis.call('del',KEYS[1])
else
    return 0
end;
```

14.9.7 MQ 异步处理

如果瞬间流量特别大，可以使用消息队列削峰，异步处理。用户请求过来的时候，先放到消息队列，再拿出来消费。

14.9.8 限流&降级&熔断

- 限流，就是限制请求，防止过大的请求压垮服务器；
- 降级，就是秒杀服务有问题了，就降级处理，不要影响别的服务；
- 熔断，服务有问题就熔断，一般熔断降级是一起出现。

14.10 聊聊分库分表,分表为什么要停服这种操作，如果不停服可以怎么做

14.10.1 分库分表方案

- 水平分库：以字段为依据，按照一定策略（hash、range 等），将一个库中的数据拆分到多个库中。
- 水平分表：以字段为依据，按照一定策略（hash、range 等），将一个表中的数据拆分到多个表中。
- 垂直分库：以表为依据，按照业务归属不同，将不同的表拆分到不同的库中。
- 垂直分表：以字段为依据，按照字段的活跃性，将表中字段拆到不同的表（主表和扩展表）中。

14.10.2 常用的分库分表中间件：

- sharding-jdbc (当当)
- Mycat
- TDDL (淘宝)
- Oceanus(58 同城数据库中间件)
- vitess (谷歌开发的数据库中间件)
- Atlas(Qihoo 360)

14.10.3 分库分表可能遇到的问题

- 事务问题：需要用分布式事务啦
- 跨节点 Join 的问题：解决这一问题可以分两次查询实现
- 跨节点的 count, order by, group by 以及聚合函数问题：分别在各个节点上得到结果后在应用程序端进行合并。
- 数据迁移，容量规划，扩容等问题
- ID 问题：数据库被切分后，不能再依赖数据库自身的主键生成机制啦，最简单可以考虑 UUID
- 跨分片的排序分页问题（后台加大 pagesize 处理？）

14.10.4 分表要停服嘛？不停服怎么做？

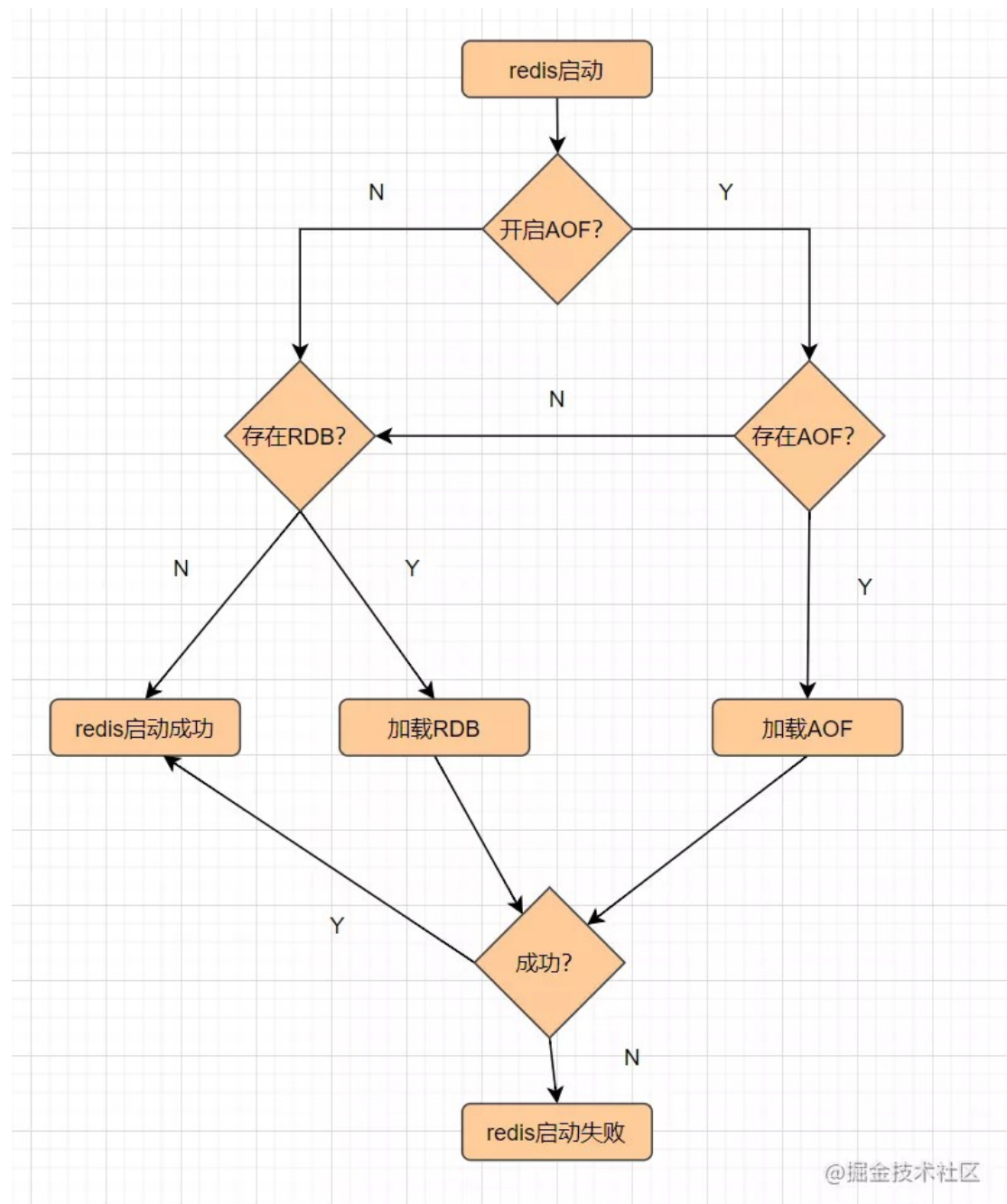
不用。不停服的时候，应该怎么做呢，分五个步骤：

1. 编写代理层，加个开关（控制访问新的 DAO 还是老的 DAO，或者是都访问），灰度期间，还是访问老的 DAO。
2. 发版全量后，开启双写，既在旧表新增和修改，也在新表新增和修改。日志或者临时表记下新表 ID 起始值，旧表中小于这个值的数据就是存量数据，这批数据就是要迁移的。
3. 通过脚本把旧表的存量数据写入新表。
4. 停读旧表改读新表，此时新表已经承载了所有读写业务，但是这时候不要立刻停写旧表，需要保持双写一段时间。
5. 当读写新表一段时间之后，如果没有业务问题，就可以停写旧表啦

14.11 redis 挂了怎么办？

Redis 是基于内存的非关系型 K-V 数据库，既然它是基于内存的，如果 Redis 服务器挂了，数据就会丢失。为了避免数据丢失了，Redis 提供了持久化，即把数据保存到磁盘。

Redis 提供了 **RDB** 和 **AOF** 两种持久化机制，它持久化文件加载流程如下：



14.11.1 RDB

RDB，就是把内存数据以快照的形式保存到磁盘上。

什么是快照？可以这样理解，给当前时刻的数据，拍一张照片，然后保存下来。

RDB 持久化，是指在指定的时间间隔内，执行指定次数的写操作，将内存中的数据集快照写入磁盘中，它是 Redis 默认的持久化方式。执行完操作后，在指定目录下会生成一个 dump.rdb 文件，Redis 重启的时候，通过加载 dump.rdb 文件来恢复数据。RDB 触发机制主要有以下几种：



RDB 的优点

- 适合大规模的数据恢复场景，如备份，全量复制等

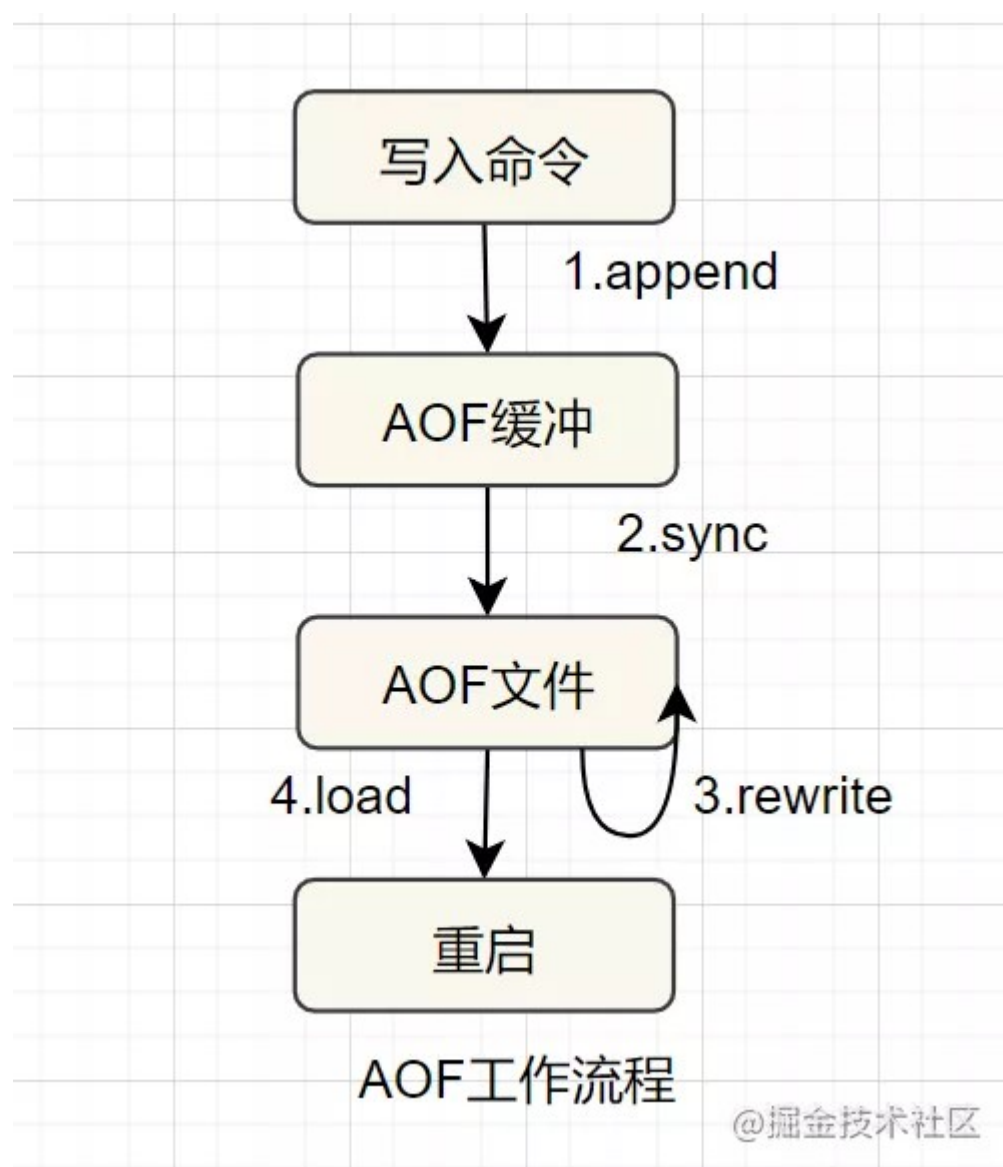
RDB 缺点

- 没办法做到实时持久化/秒级持久化。
- 新老版本存在 RDB 格式兼容问题

14.11.2 AOF

AOF (append only file) 持久化，采用日志的形式来记录每个写操作，追加到文件中，重启时再重新执行 AOF 文件中的命令来恢复数据。它主要解决数据持久化的实时性问题。默认是不开启的。

AOF 的工作流程如下：



AOF 的优点

- 数据的一致性和完整性更高

AOF 的缺点

- AOF 记录的内容越多，文件越大，数据恢复变慢。

14.12 你怎么防止优惠券有人重复刷？

对于重复请求，要考虑接口幂等和接口防重。

大家可以看下之前我写的这篇文章哈：[聊聊幂等设计](#)

防刷的话，可以限流以及加入黑名单处理。

- 为了防止某个用户请求优惠券过于频繁，我们可以对同一用户限流。
- 为了防止黄牛等模拟几个用户请求，我们可以对某个 IP 进行限流。
- 为了防止有人使用代理，每次请求都更换 IP 请求，我们可以对接口进行限流。

14.13 抖音评论系统怎么设计，如果加入好友关系呢？

需要考虑性能，以及可扩展性。大家平时有没有做过评论、好友关注等项目需求呀，发挥你聪明的小脑袋瓜，怎么去回答好这道题吧。

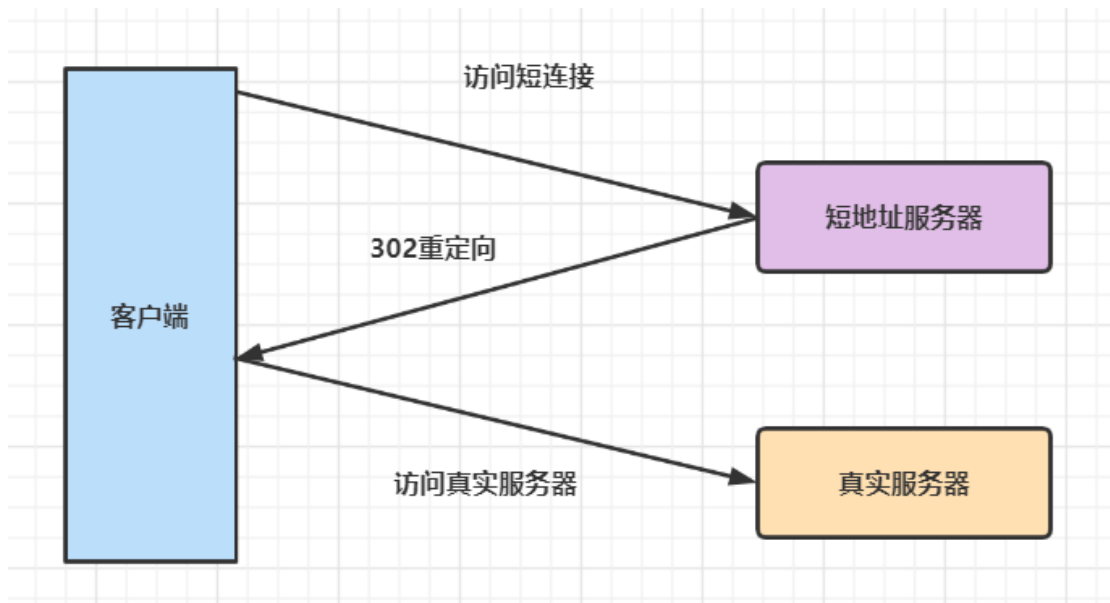
14.14 怎么设计一个短链地址，要考虑跨机房部署问题

14.14.1 为什么需要短连接？

为什么需要短连接呢？长链接不香吗？因为有些平台有长度限制，并且链接太长容易被识别为超链接等等。

14.14.2 短链接的原理

其实就是一个 **302 重定向** 而已。



302 状态码表示临时重定向。

14.14.3 短链接生成的方法

可以用哈希算法生成短链，但是会存在哈希冲突。怎么解决呢？可以用布隆过滤器。

有没有别的方案？自增序列算法，每次收到一个长链时，就分配一个 ID，并转成 62 进制拼接到短域后面。因为高并发下，ID 自增生成器可能成为瓶颈。

一般有四种分布式 ID 生成方法：

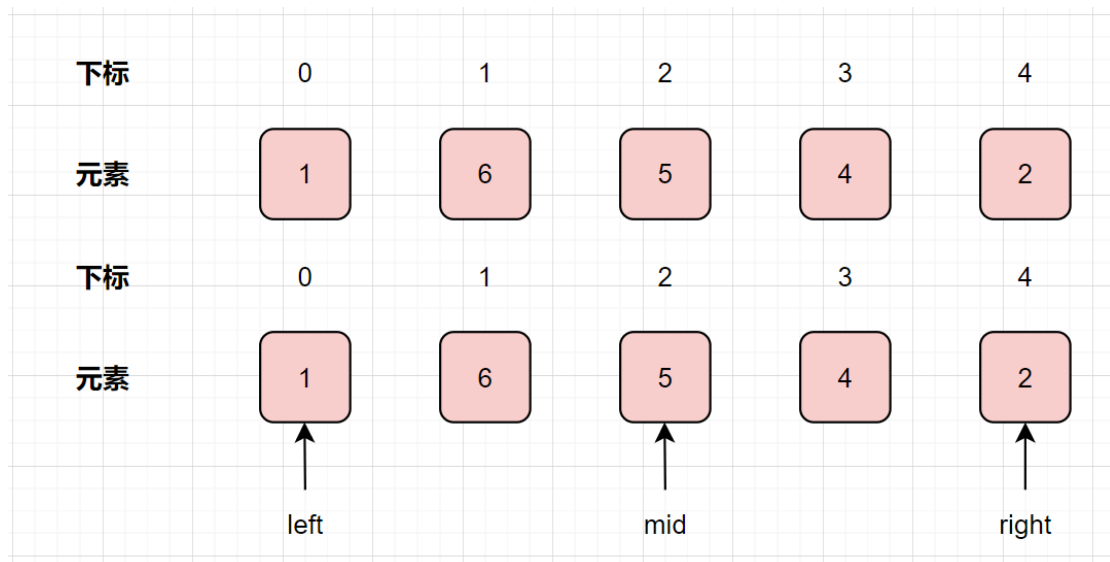
1. uuid，它保证对在同一时空中的所有机器都是唯一的，但是这种方式生成的 id 比较长，并且是无序的，插入浪费空间。
2. Snowflake 雪花算法，这种方案不错，但是如果某台机器的系统时钟回拨，有可能造成 ID 冲突重复，或者 ID 乱序（考虑跨机房部署问题）
3. Mysql 自增主键，在高并发下，db 的写压力会很大
4. 用 Redis 做自增 id 生成器，性能高，但要考虑持久性的问题；或者改造雪花算法，通过改造 workId 解决时钟回拨的问题）

14.15 有一个整型数组，数组元素不重复，数组元素先升序后降序，找出最大值。

例如：1,3,5,7,9,8,6,4,2，请写一个函数找出数组最大的元素

这道题大家都会觉得很简单，因为用快速排序排一下，时间复杂是 $O(n\lg n)$ ，面试官可能不是很满意。其实可以用二分查找法，只要找到升序最后一个元素即可。

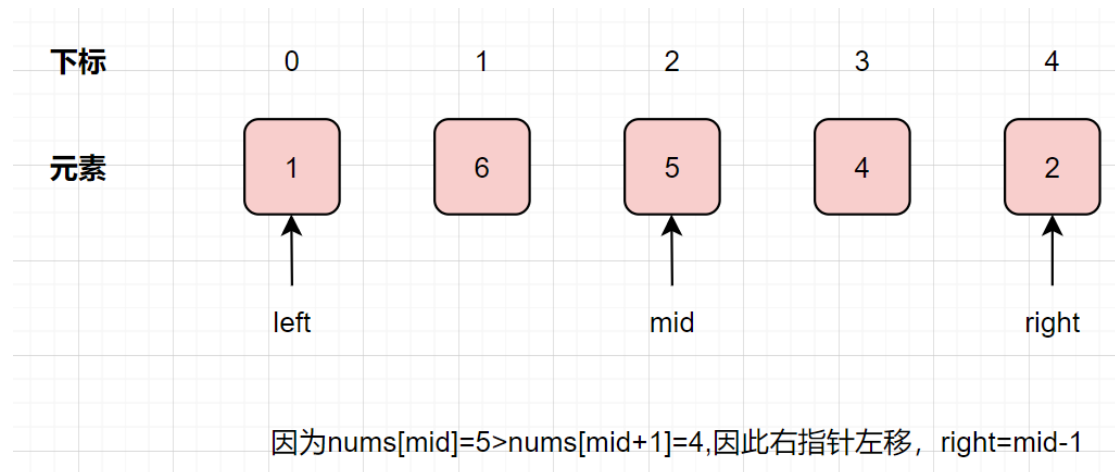
我们以 1,6,5,4,2 为例子，用二分法图解一下哈：



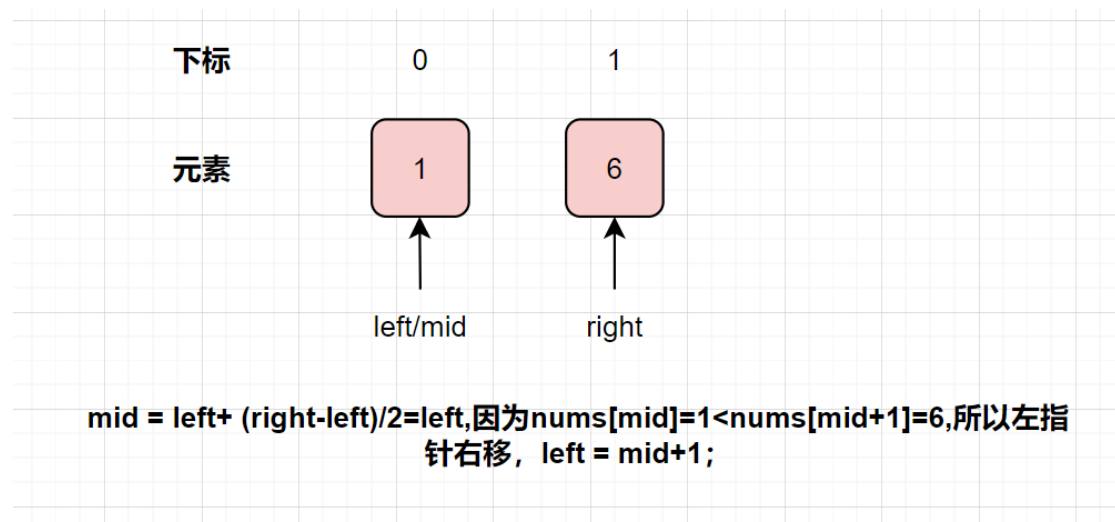
如何用二分法缩小空间呢？只要比较中间元素与其下一个元素大小即可

1. 如果中间元素大于其下一个元素大小，证明最大值在左侧，因此右指针左移
2. 如果中间元素小于其下一个元素大小，证明最大值在右侧，因此左指针右移

因为 $nums[mid]=5 > nums[mid+1]=4$, 因此右指针左移， $right=mid-1=2-1=1$



$\text{mid} = \text{left} + (\text{right} - \text{left}) / 2 = \text{left} = 0$, 因为 $\text{nums}[\text{mid}] = 1 < \text{nums}[\text{mid}+1] = 6$, 所以左指针右移, $\text{left} = \text{mid} + 1 = 1$;



最后得出最大值是 $\text{nums}[\text{left}] = \text{nums}[1] = 6$

实现代码如下：

```

class Solution {

    public static void main(String[] args) {

        int[] nums = {1,3,5,7,9,8,6,4,2};
        System.out.println(getLargestNumInArray(nums));
    }

    private static int getLargestNumInArray(int[] nums) {
        if (nums == null || nums.length == 0) {

```

```
        return -1;
    }

    int left = 0, right = nums.length - 1;
    while (left <= right) {
        int mid = left + ((right - left) / 2);
        if (nums[mid] < nums[mid + 1]) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return nums[left];
}
```

15. 字节二面 (关注公众号：捡田螺的小男孩。)

大家好，我是捡田螺的小男孩。有位朋友面试了字节后端方向。整理了这几道面试真题以及答案。如果有不正确的，欢迎指出哈，一起进步。更多干货，关注公众号：捡田螺的小男孩。



15.1 http 请求头里，expire 和 cache-control 字段含义，说说 HTTP 状态码

15.1.1 expire 和 cache-control 字段含义

- Cache-Control 是 HTTP/1.1 的头字段，用来区分对缓存机制的支持情况，请求头和响应头都支持这个属性。通过它提供的不同的值来定义缓存策略。主要有 `public`、`private`、`no-cache` 等值。
- expires 是 http1.0 的头字段，过期时间，如果设置了时间，则浏览器会在设置的时间内直接读取缓存，不再请求。

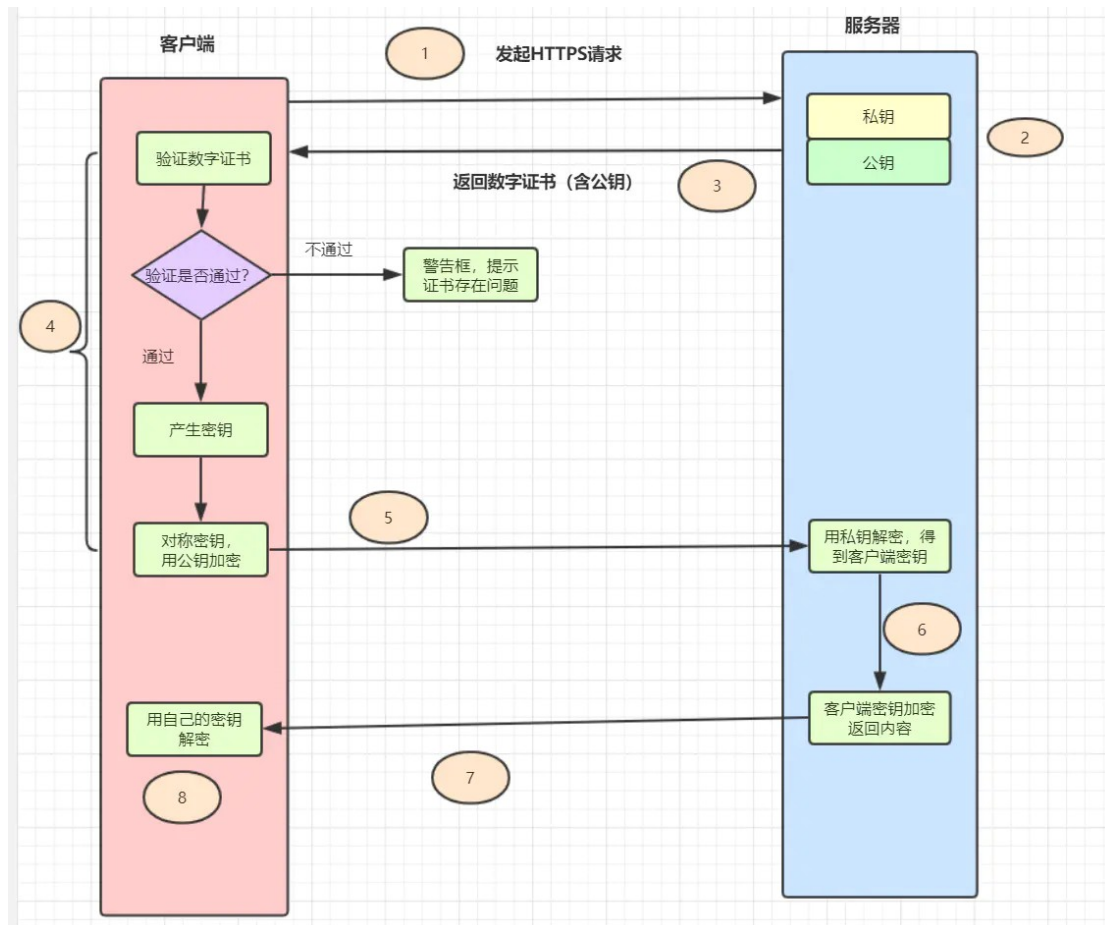
15.1.2 常见 HTTP 状态码

状态码	含义
101	切换请求协议
200	请求成功
301	永久性重定向，会缓存
302	临时重定向，不会缓存
400	客户端请求的语法错误
403	服务器禁止访问，权限有关
404	服务器无法根据客户端的请求找到资源
500	服务端错误

15.2 https 原理，数字签名，数字证书。

15.2.1 https 原理

- HTTPS = HTTP + SSL/TLS，即用 SSL/TLS 对数据进行加密和解密，Http 进行传输。
- SSL，即 Secure Sockets Layer (安全套接层协议)，是网络通信提供安全及数据完整性的一种安全协议。
- TLS，即 Transport Layer Security(安全传输层协议)，它是 SSL 3.0 的后续版本。



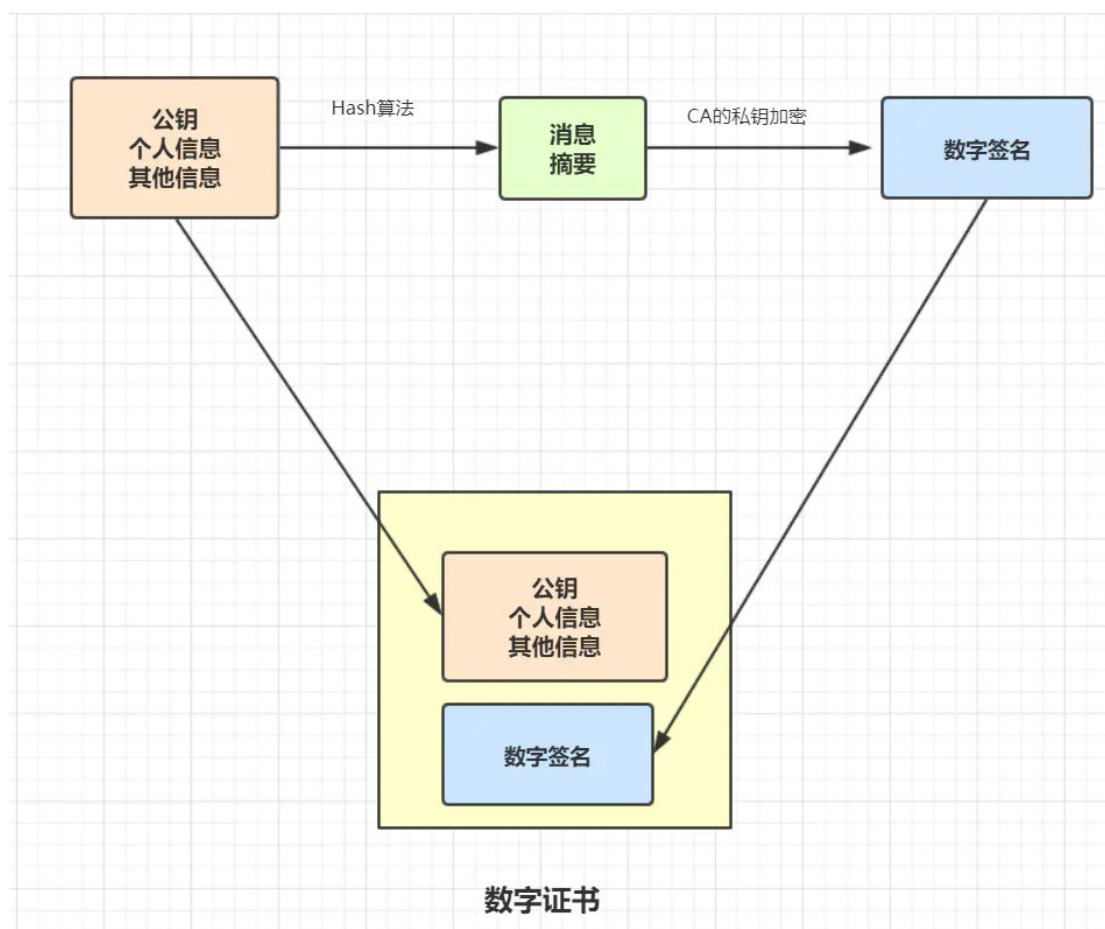
Https 工作流程

1. 用户在浏览器里输入一个 https 网址，然后连接到 server 的 443 端口。
2. 服务器必须要有一套数字证书，可以自己制作，也可以向组织申请，区别就是自己颁发的证书需要客户端验证通过。这套证书其实就是一对公钥和私钥。
3. 服务器将自己的数字证书（含有公钥）发送给客户端。
4. 客户端收到服务器端的数字证书之后，会对其进行检查，如果不通过，则弹出警告框。如果证书没问题，则生成一个密钥（对称加密），用证书的公钥对它加密。
5. 客户端会发起 HTTPS 中的第二个 HTTP 请求，将加密之后的客户端密钥发送给服务器。
6. 服务器接收到客户端发来的密文之后，会用自己的私钥对其进行非对称解密，解密之后得到客户端密钥，然后用客户端密钥对返回数据进行对称加密，这样数据就变成了密文。
7. 服务器将加密后的密文返回给客户端。

8. 客户端收到服务器发返回的密文，用自己的密钥（客户端密钥）对其进行对称解密，得到服务器返回的数据。

15.2.2 数字签名，数字证书

了解过 **Https 原理** 的小伙伴，都知道数字证书这玩意。为了避免公钥被篡改，引入了数字证书，如下：



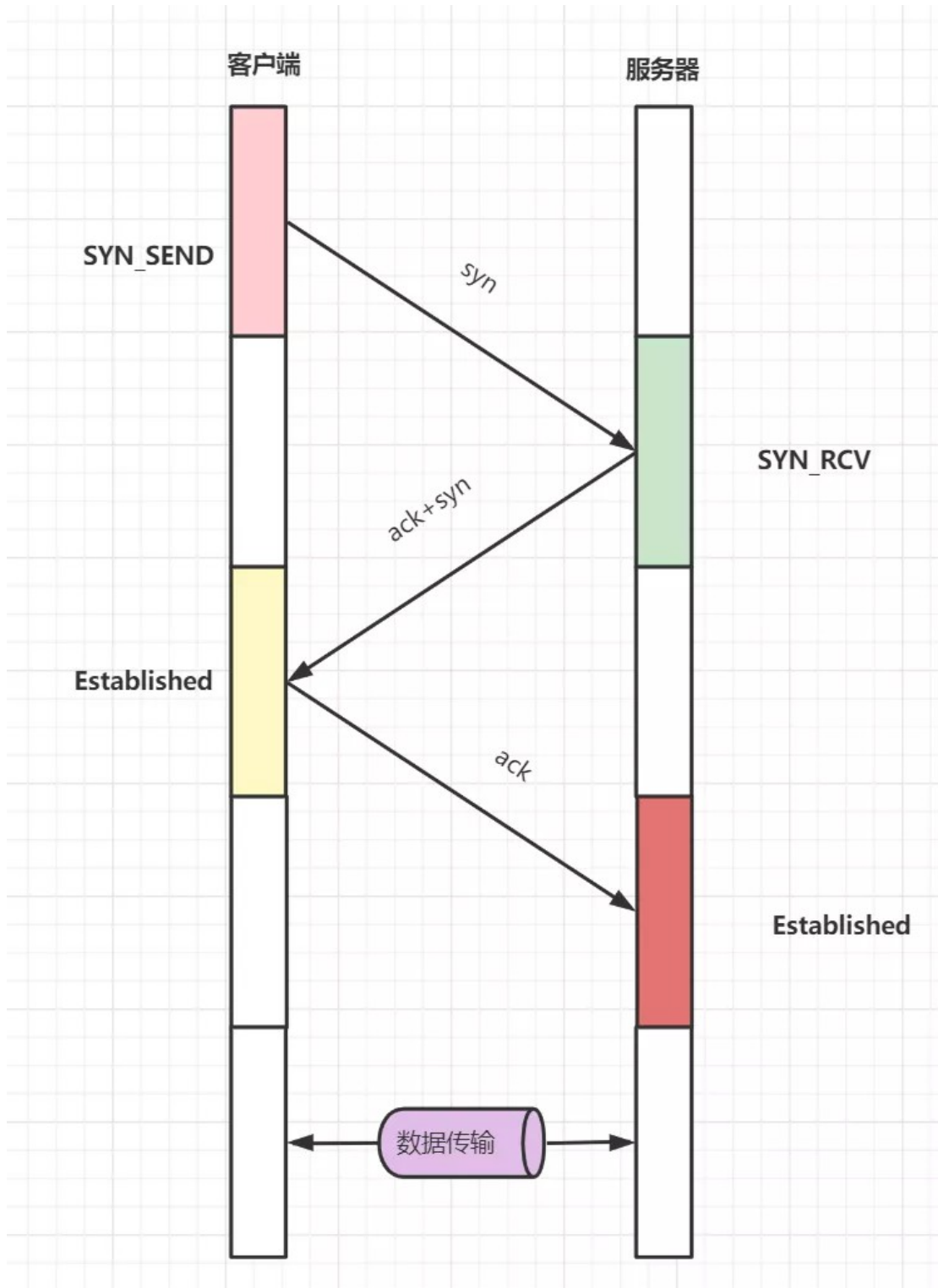
数字证书构成

- 公钥和个人信息，经过 Hash 算法加密，形成消息摘要；将消息摘要拿到拥有公信力的认证中心（CA），用它的私钥对消息摘要加密，形成数字签名。
- 公钥和个人信息、数字签名共同构成数字证书。

15.3 tcp 连接 client 和 server 有哪些状态，time_wait 状态

15.3.1 tcp 连接

tcp 连接时，客户端 client 有 `SYN_SEND`、`ESTABLISHED` 状态，服务端 server 有 `SYN_RCVD`、`ESTABLISHED` 状态。



tcp 三次握手

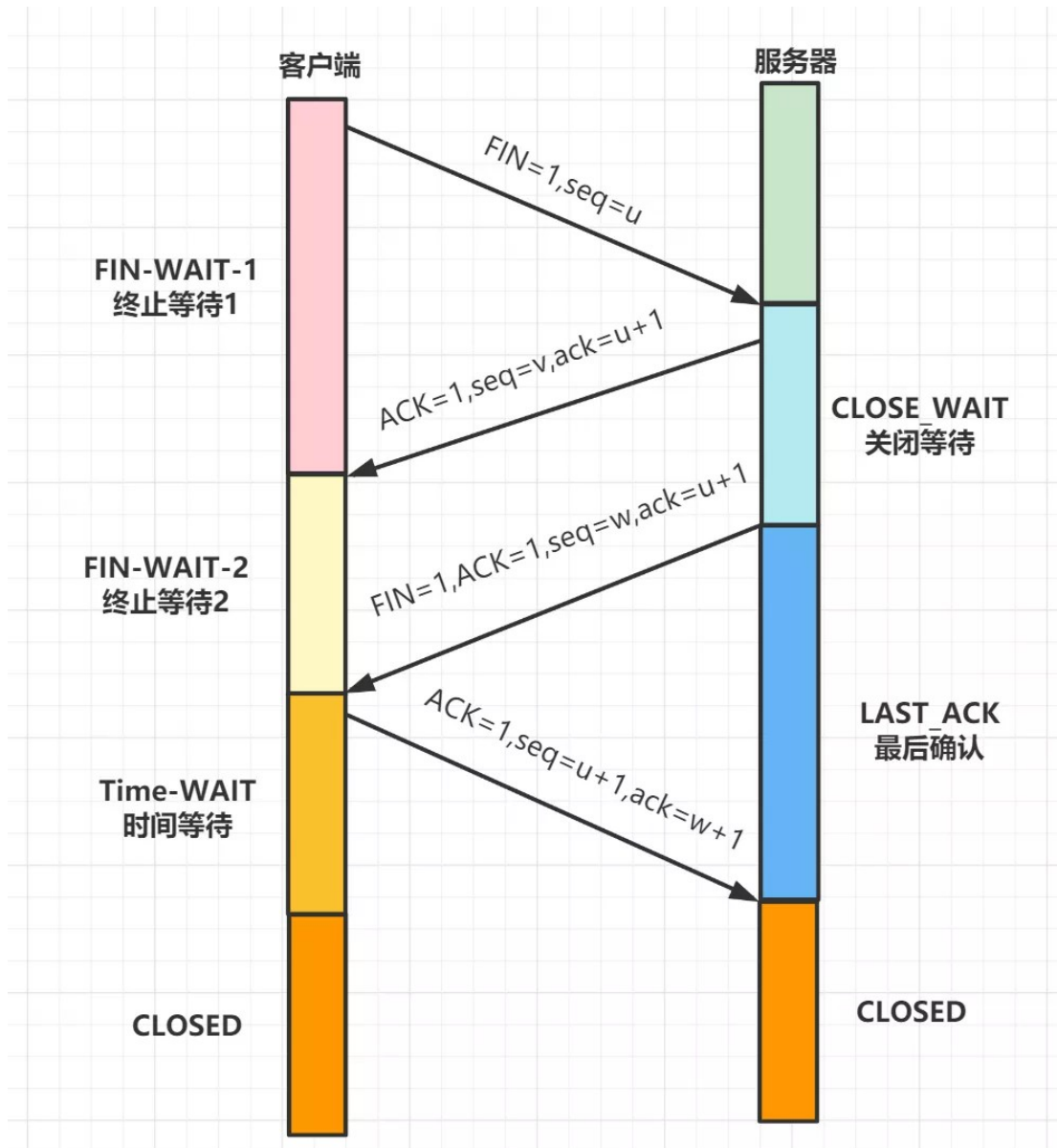
开始客户端和服务端都处于 CLOSED 状态，然后服务端开始监听某个端口，进入 LISTEN 状态

- 第一次握手(SYN=1, seq=x)，发送完毕后，客户端进入 SYN_SEND 状态

- 第二次握手($\text{SYN}=1, \text{ACK}=1, \text{seq}=y, \text{ACKnum}=x+1$)，发送完毕后，服务器端进入 SYN_RCVD 状态。
- 第三次握手($\text{ACK}=1, \text{ACKnum}=y+1$)，发送完毕后，客户端进入 ESTABLISHED 状态，当服务器端接收到这个包时,也进入 ESTABLISHED 状态，TCP 握手，即可以开始数据传输。

15.3.2 time_wait 状态

可以先回忆下 TCP 的四次挥手哈，



TCP 四次挥手

- 第一次挥手($\text{FIN}=1, \text{seq}=u$)，发送完毕后，客户端进入 **FIN_WAIT_1** 状态

- 第二次挥手($ACK=1$, $ack=u+1, seq=v$) , 发送完毕后 , 服务器端进入 **CLOSE_WAIT** 状态 , 客户端接收到这个确认包之后 , 进入 **FIN_WAIT_2** 状态
- 第三次挥手($FIN=1$, $ACK=1, seq=w, ack=u+1$) , 发送完毕后 , 服务器端进入 **LAST_ACK** 状态 , 等待来自客户端的最后一个 ACK。
- 第四次挥手($ACK=1$, $seq=u+1, ack=w+1$) , 客户端接收到来自服务器端的关闭请求 , 发送一个确认包 , 并进入 **TIME_WAIT** 状态 , 等待了某个固定时间 (两个最大段生命周期 , $2MSL$, $2\text{ Maximum Segment Lifetime}$) 之后 , 没有收到服务器端的 ACK , 认为服务器端已经正常关闭连接 , 于是自己也关闭连接 , 进入 **CLOSED** 状态。服务器端接收到这个确认包之后 , 关闭连接 , 进入 **CLOSED** 状态。

TIME-WAIT 状态为什么需要等待 2MSL

$2MSL$, $2\text{ Maximum Segment Lifetime}$, 即两个最大段生命周期

- 1 个 MSL 保证四次挥手中主动关闭方最后的 ACK 报文能最终到达对端
- 1 个 MSL 保证对端没有收到 ACK 那么进行重传的 FIN 报文能够到达

15.4 什么是虚拟内存？什么是物理内存？

15.4.1 什么是虚拟内存？

虚拟内存 , 是虚拟出来的内存 , 它的核心思想就是确保每个程序拥有自己的地址空间 , 地址空间被分成多个块 , 每一块都有连续的地址空间。同时物理空间也分成多个块 , 块大小和虚拟地址空间的块大小一致 , 操作系统会自动将虚拟地址空间映射到物理地址空间 , 程序只需关注虚拟内存 , 请求的也是虚拟内存 , 真正使用却是物理内存。

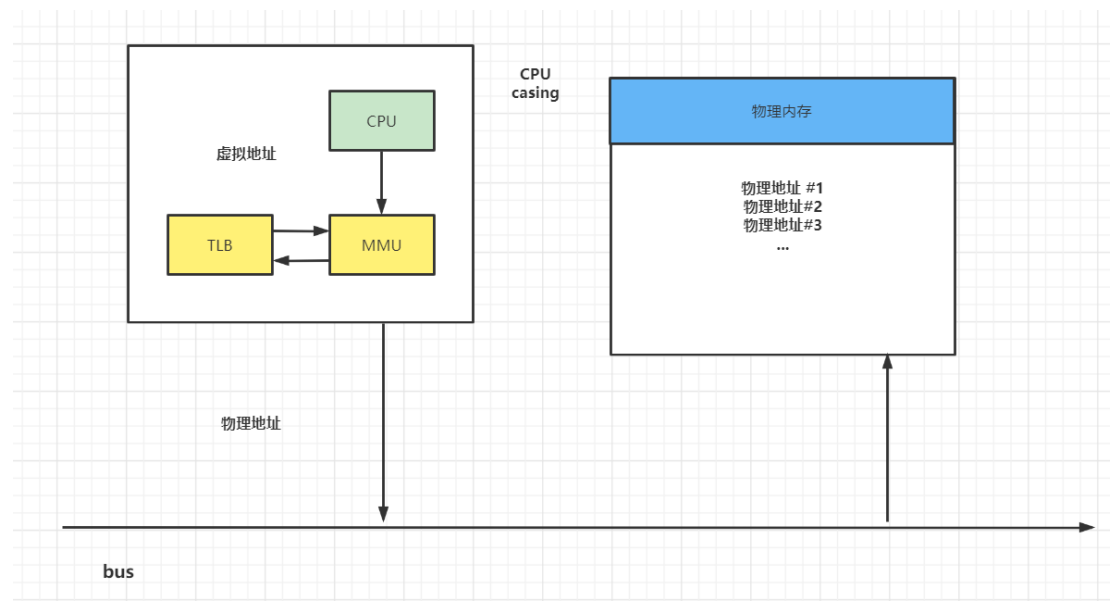
15.4.2 什么是物理内存

物理内存 , 指通过物理内存条而获得的内存空间 , 而虚拟内存则是指将硬盘的一块区域划分来作为内存。

我们常说的物理内存大小，其实是指内存条的大小。一般买电脑时，我们都会看下内存条是多大容量的，话说如果内存条大小是 100G，那这 100G 就都能够被使用吗？不一定的，更多的还是要看 CPU 地址总线的位数，如果地址总线只有 20 位，那么它的寻址空间就是 1MB，即使可以安装 100G 的内存条也没有意义，也只能视物理内存大小为 1MB。

15.4.3 虚拟内存如何映射到物理内存？

如下图，CPU 里有一个内存管理单元 (Memory Management Unit)，简称为 MMU，虚拟内存不是直接送到内存总线，而是先给到 MMU，由 MMU 来把虚拟地址映射到物理地址，程序只需要管理虚拟内存就好，映射的逻辑自然有其它模块自动处理。



15.5 一台机器最多可以建立多少个 tcp 连接，client 端，server 端，超过了怎么办

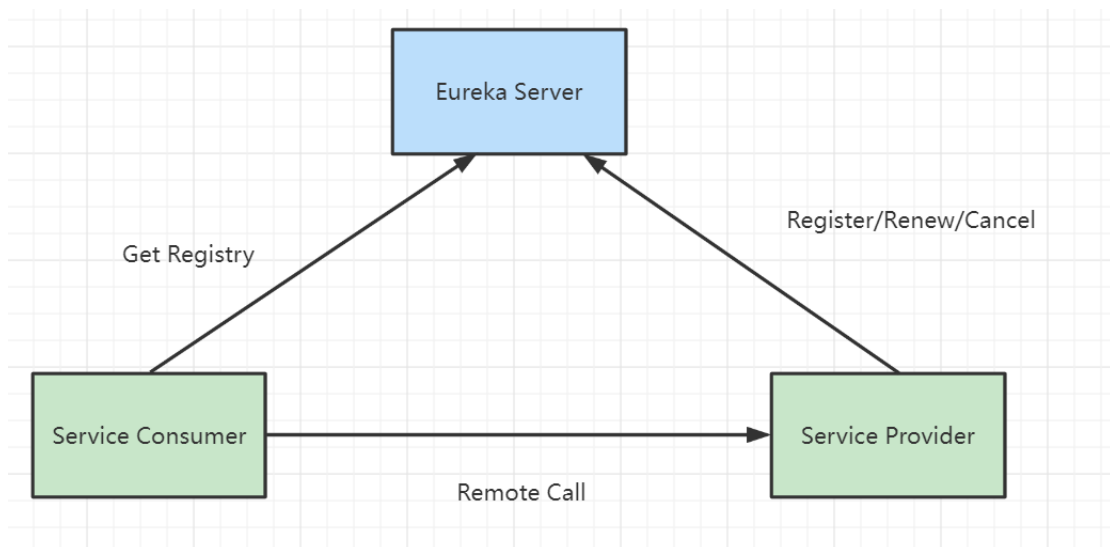
- TCP 连接的客户端机：每一个 ip 可建立的 TCP 连接理论受限于 ip_local_port_range 参数，也受限于 65535。但可以通过配置多 ip 的方式来加大自己的建立连接的能力。
- TCP 连接的服务器机：每一个监听的端口虽然理论值很大，但这个数字没有实际意义。最大并发数取决你的内存大小，每一条静止状态的 TCP 连接大约需要吃 3 .3K 的内存。

15.6 Eureka 原理，是否是强一致性，eureka 集群。宕机了服务还能调用么？Eureka 和 ZooKeeper 对比

15.6.1 eureka 架构

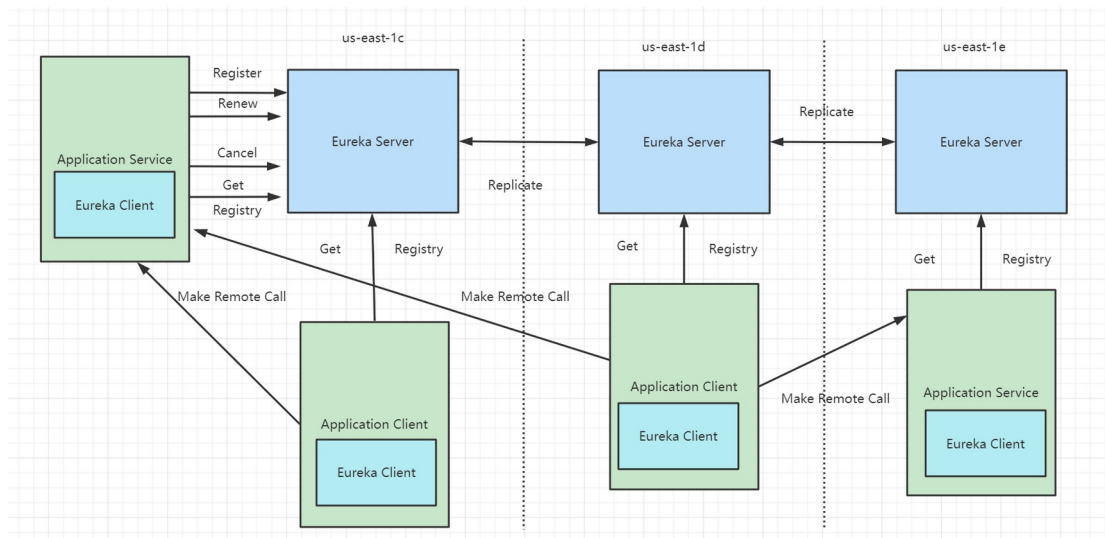
注册中心是分布式开发的核心组件之一，而 eureka 是 spring cloud 推荐的注册中心实现。

架构图如下：



- Eureka Server：提供服务注册和发现，多个 Eureka Server 之间会同步数据，做到状态一致
- Service Provider：服务提供方，将自身服务注册到 Eureka，从而使服务消费方能够找到
- Service Consumer：服务消费方，从 Eureka 获取注册服务列表，从而能够消费服务

15.6.2 基于集群的 Eureka 架构图



Eureka server 可以集群部署，多个节点之间会通过 Replicate（异步方式）进行数据同步，保证数据**最终一致性**。Eureka Server 作为一个开箱即用的服务注册中心，提供的功能包括：服务注册、接收服务心跳、服务剔除、服务下线等。

服务启动后向 Eureka 注册，Eureka Server 会将注册信息向其他 Eureka Server 进行同步，当服务消费者要调用服务提供者，则向服务注册中心获取服务提供者地址，然后将服务提供者地址缓存在本地，下次再调用时，则直接从本地缓存中取，完成一次调用。

15.6.3 宕机了服务还能调用么？

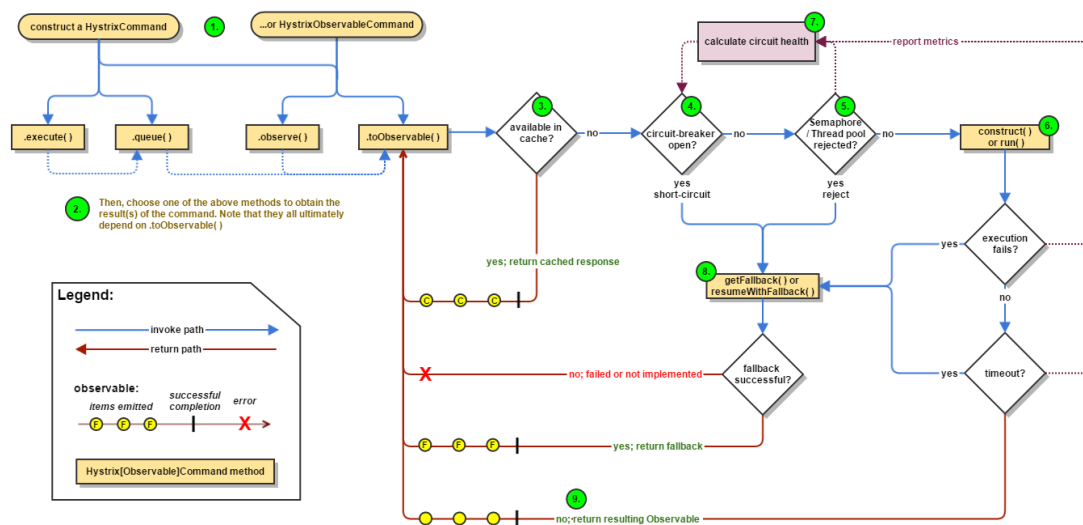
Eureka 挂了，微服务是可以调通的，不过有个前提：**provider 的地址没变**！如果 provider 换了一个 IP 地址或者端口，这个时候，consumer 就无法及时感知到这种变化，就会调不通。

15.6.4 Eureka 和 ZooKeeper 对比

- Zookeeper 保证 CP(一致性和分区容错性)，但是不保证可用性，ZK 的 leader 选举期间，是不可用的。
- Eureka 保证 AP (可用性和分区容错性)，它优先保证可用性，几个节点挂掉不会影响正常节点的工作。

15.7.Hystrix 了解嘛？说说 Hystrix 的工作原理

Hystrix 工作流程图如下：



1. 构建命令

Hystrix 提供了两个 Command，HystrixCommand 和 HystrixObservableCommand，可以使用这两个对象来包裹待执行的任务。

1. 执行命令

有四种方式执行 command。分别是：

- R execute()：同步执行，从依赖服务得到单一结果对象
- Future queue()：异步执行，返回一个 Future 以便获取执行结果，也是单一结果对象
- Observable observe()：hot observable，创建 Observable 后会订阅 Observable，可以返回多个结果

- `Observable toObservable()` : cold observable , 返回一个 Observable , 只有订阅时才会执行 , 可以返回多个结果

1. 检查缓存

如果启用了 Hystrix Cache , 任务执行前将先判断是否有相同命令执行的缓存。如果有则直接返回缓存的结果 ; 如果没有缓存的结果 , 但启动了缓存 , 将缓存本次执行结果以供后续使用。

4.检查断路器是否打开 断路器(circuit-breaker)和保险丝类似 , 保险丝在发生危险时将会烧断以保护电路 , 而断路器可以在达到我们设定的阈值时触发短路 (比如请求失败率达到 50%) , 拒绝执行任何请求。

如果断路器被打开 , Hystrix 将不会执行命令 , 直接进入 Fallback 处理逻辑。

5.检查线程池/信号量情况 Hystrix 隔离方式有线程池隔离和信号量隔离。当使用 Hystrix 线程池时 , Hystrix 默认为每个依赖服务分配 10 个线程 , 当 10 个线程都繁忙时 , 将拒绝执行命令。信号量同理。

6.执行具体的任务 通过 `HystrixObservableCommand.construct()` 或者 `HystrixCommand.run()` 来运行用户真正的任务。

7.计算链路健康情况 每次开始执行 command、结束执行 command 以及发生异常等情况时 , 都会记录执行情况 , 例如 : 成功、失败、拒绝以及超时等情况 , 会定期处理这些数据 , 再根据设定的条件来判断是否开启断路器。

8.命令失败时执行 Fallback 逻辑 在命令失败时执行用户指定的 Fallback 逻辑。上图中的断路、线程池拒绝、信号量拒绝、执行执行、执行超时都会进入 Fallback 处理。

9.返回执行结果 原始结果将以 Observable 形式返回，在返回给用户之前，会根据调用方式的不同做一些处理。

15.8 zookeeper 一致性保证，zab 协议原理，zookeeper 属于哪种一致性，强一致性么，还是最终一致性

Zab 协议，英文全称是 Zookeeper Atomic Broadcast (Zookeeper 原子广播)。Zookeeper 是通过 Zab 协议来保证分布式事务的**最终一致性**。

Zab 协议是为分布式协调服务 Zookeeper 专门设计的一种支持崩溃恢复的原子广播协议，是 Zookeeper 保证数据一致性的核心算法。Zab 借鉴了 Paxos 算法，是一种通用的分布式一致性算法。

基于 Zab 协议，Zookeeper 实现了一种主备模型 (即 Leader 和 Follower 模型) 的系统架构来保证集群中各个副本之间数据的一致性。就是指只有一台 Leader 节点负责处理外部的写事务请求，然后它 (Leader) 将数据同步到其他 Follower 节点。

Zookeeper 客户端会随机的链接到 zookeeper 集群中的一个节点，如果是读请求，就直接从当前节点中读取数据；如果是写请求，那么节点就会向 Leader 提交事务，Leader 接收到事务提交，会广播该事务，只要超过半数节点写入成功，该事务就会被提交。

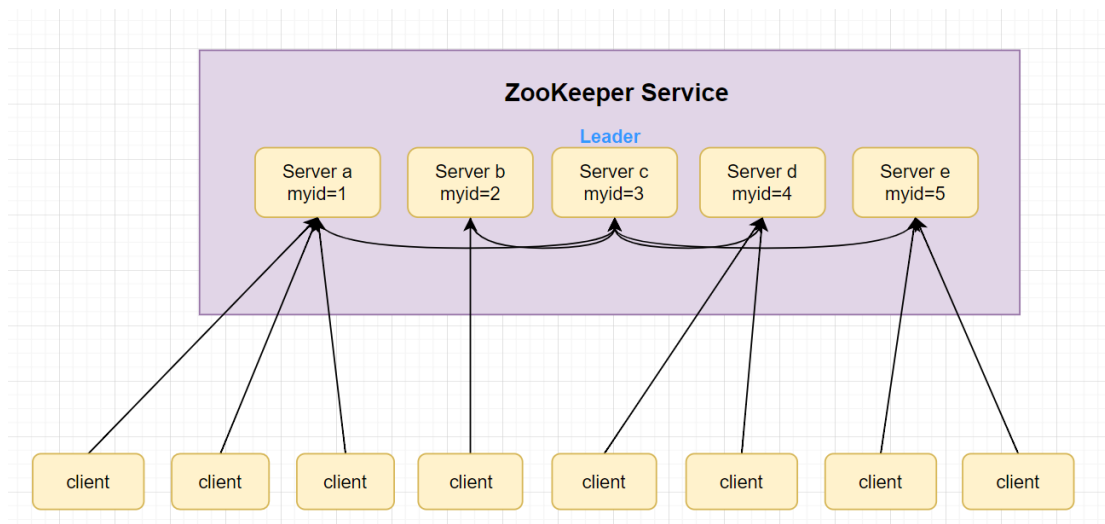
Zab 协议要求每个 Leader 都要经历三个阶段：**发现，同步，广播**。

- 发现：要求 zookeeper 集群必须选举出一个 Leader 进程，同时 Leader 会维护一个 Follower 可用客户端列表。将来客户端可以和这些 Follower 节点进行通信。

- 同步：Leader 要负责将本身的数据与 Follower 完成同步，做到多副本存储。这样也是提现了 CAP 中的高可用和分区容错。Follower 将队列中未处理完的请求消费完成后，写入本地事务日志中。
- 广播：Leader 可以接受客户端新的事务 Proposal 请求，将新的 Proposal 请求广播给所有的 Follower。

15.9 聊聊 zookeeper 选举机制

服务器启动或者服务器运行期间（Leader 挂了），都会进入 Leader 选举，我们来看一下~假设现在 ZooKeeper 集群有五台服务器，它们 myid 分别是服务器 1、2、3、4、5，如图：



15.9.1 服务器启动的 Leader 选举

zookeeper 集群初始化阶段，服务器（myid=1-5）依次启动，开始 zookeeper 选举 Leader~



1. 服务器 1 (myid=1) 启动，当前只有一台服务器，无法完成 Leader 选举
2. 服务器 2 (myid=2) 启动，此时两台服务器能够相互通讯，开始进入 Leader 选举阶段

- 2.1. 每个服务器发出一个投票

服务器 1 和服务器 2 都将自己作为 Leader 服务器进行投票，投票的基本元素包括：服务器的 myid 和 ZXID，我们以 (myid , ZXID) 形式表示。初始阶段，服务器 1 和服务器 2 都会投给自己，即服务器 1 的投票为 (1,0)，服务器 2 的投票为 (2,0)，然后各自将这个投票发给集群中的其他所有机器。

- 2.2 接受来自各个服务器的投票

每个服务器都会接受来自其他服务器的投票。同时，服务器会校验投票的有效性，是否本轮投票、是否来自 LOOKING 状态的服务器。

- 2.3. 处理投票

收到其他服务器的投票，会将别人的投票跟自己的投票 PK，PK 规则如下：

- 优先检查 ZXID。ZXID 比较大的服务器优先作为 leader。
- 如果 ZXID 相同的话，就比较 myid，myid 比较大的服务器作为 leader。

服务器 1 的投票是 (1,0)，它收到投票是 (2,0)，两者 zxid 都是 0，因为收到的 myid=2，大于自己的 myid=1，所以它更新自己的投票为 (2,0)，然后重新将投票发出去。对于服务器 2 呢，即不再需要更新自己的投票，把上一轮的投票信息发出即可。

- 2.4. 统计投票

每次投票后，服务器会统计所有投票，判断是否有过半的机器接受到相同的投票信息。服务器 2 收到两票，少于 3 ($n/2+1$, n 为总服务器)，所以继续保持 LOOKING 状态

1. 服务器 3 (myid=3) 启动，继续进入 Leader 选举阶段

- 3.1 跟前面流程一致，服务器 1 和 2 先投自己一票，因为服务器 3 的 myid 最大，所以大家把票改投给它。此时，服务器为 3 票 (大于等于 $n/2+1$)，所以服务器 3 当选为 Leader。服务器 1，2 更改状态为 FOLLOWING，服务器 3 更改状态为 LEADING；

1. 服务器 4 启动，发起一次选举。

- 4.1 此时服务器 1，2，3 已经不是 LOOKING 状态，不会更改选票信息。选票信息结果：服务器 3 为 3 票，服务器 4 为 1 票。服务器 4 并更改状态为 FOLLOWING；

1. 服务器 5 启动，发起一次选举。

- 同理，服务器也是把票投给服务器 3，服务器 5 并更改状态为 FOLLOWING；

1. 投票结束，服务器 3 当选为 Leader

15.9.2 服务器运行期间的 Leader 选举

zookeeper 集群的五台服务器 (myid=1-5) 正在运行中，突然某个瞬间，Leader 服务器 3 挂了，这时候便开始 Leader 选举～



1. 变更状态

Leader 服务器挂了之后，余下的非 Observer 服务器都会把自己的服务器状态更改为 LOOKING，然后开始进入 Leader 选举流程。

1. 每个服务器发起投票

每个服务器都把票投给自己，因为是运行期间，所以每台服务器的 ZXID 可能不相同。假设服务 1,2,4,5 的 zxid 分别为 333,666,999,888，则分别产生投票 (1,333)，(2, 666)，(4,999) 和 (5,888)，然后各自将这个投票发给集群中的其他所有机器。

1. 接受来自各个服务器的投票
2. 处理投票

投票规则是跟 Zookeeper 集群启动期间一致的，优先检查 ZXID，大的优先作为 Leader，所以显然服务器 zxid=999 具有优先权。

1. 统计投票
2. 改变服务器状态

15.10 算法：给定一个字符串 s，请你找出其中不含有重复字符的最长连续子字符串的长度。

可以使用滑动窗口实现，代码如下：

```
public int lengthOfLongestSubstring2(String s) {  
    int n = s.length();  
    if (n <= 1) return n;  
    int maxLen = 1;  
  
    //左、右指针  
    int left = 0, right = 0;  
  
    Set<Character> window = new HashSet<>();
```



```
    while (right < n) {  
        char rightChar = s.charAt(right);  
        while (window.contains(rightChar)) {  
            window.remove(s.charAt(left));  
            left++;  
        }  
        //最大长度对比  
        maxLen = Math.max(maxLen, right - left + 1);  
        window.add(rightChar);  
        right++;  
    }  
  
    return maxLen;  
}
```